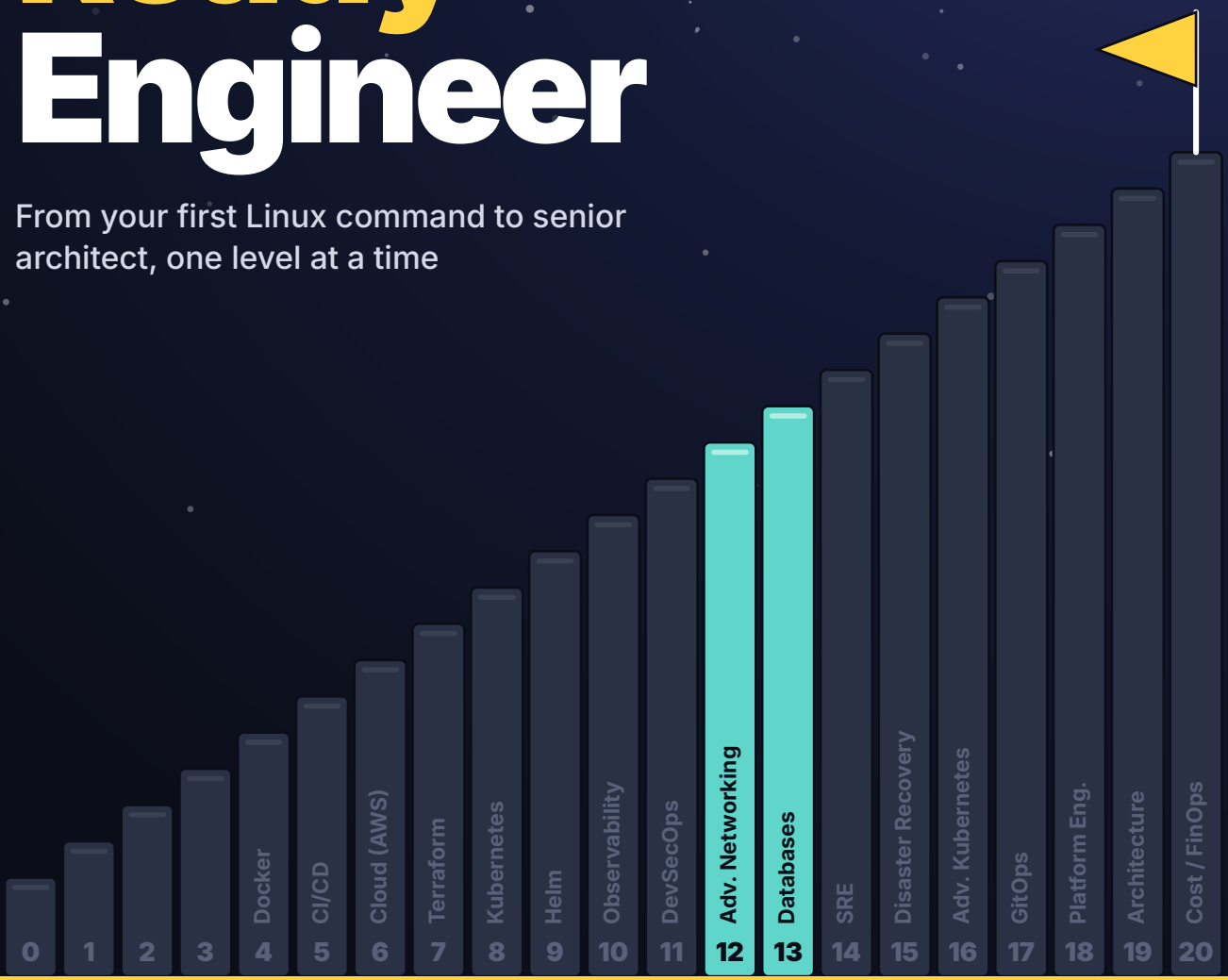
THE COMPLETE DEVOPS ROADMAP · VOLUME 4 OF 6

The Production-Ready Engineer

From your first Linux command to senior architect, one level at a time



Ubong Eduok

Under the Hood

2 levels · **29** topics · **2** capstones

The Production-Ready Engineer

Volume 4: Under the Hood

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** Base Camp and Part I: DevOps, Linux, Networking, Git	pp. 1–734
2	**Build, Ship and Cloud** Parts II and III: Docker, CI/CD, AWS, Terraform	pp. 735–1544
3	**Orchestrate, See and Secure** Parts IV and V: Kubernetes, Helm, Observability, DevSecOps	pp. 1545–2318
4	**Under the Hood** **Part VI: Advanced Networking, Databases and Distributed Systems**	**pp. 2319–2814**
5	**Reliability and Platforms at Scale** Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering	pp. 2815–3548
6	**Senior Altitude and the Summit** Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects	pp. 3549–3972

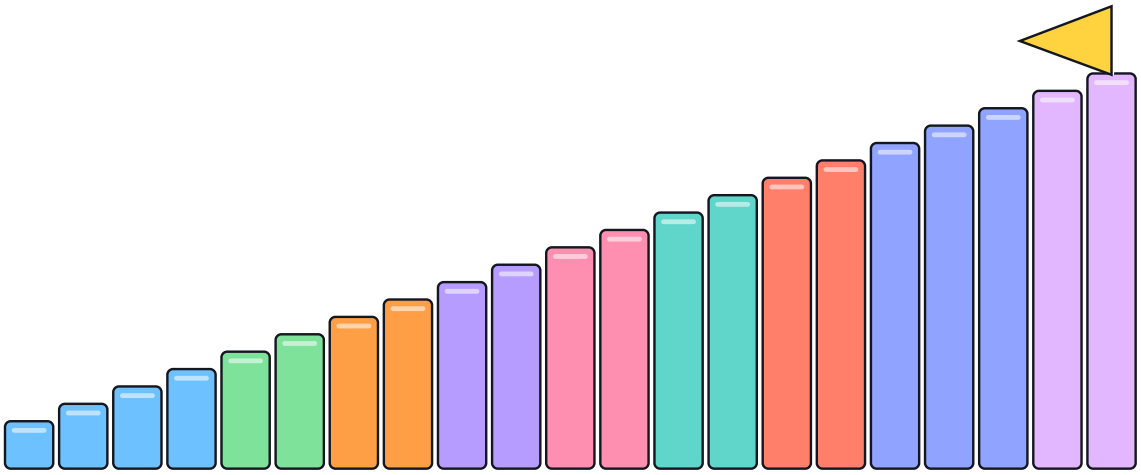
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 4 of 6: Under the Hood

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 4 of 6: Under the Hood

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents of Volume 4

PART VI **Under the Hood** 2319

12	**Advanced Networking**	2321
12.1	OSI and TCP/IP Models	2323
12.2	BGP Fundamentals	2335
12.3	DNS Architecture	2349
12.4	TLS	2365
12.5	HTTP/2, HTTP/3 and QUIC	2381
12.6	Load Balancing	2399
12.7	CDNs	2415
12.8	VPNs	2431
12.9	Service Mesh	2445
12.10	Connection Pooling	2463
12.11	TLS Termination	2479
12.12	mTLS	2497
12.13	Rate Limiting	2513
12.14	Circuit Breakers	2533
★	Capstone: Secure Edge and Resilient Services Lab	2553
13	**Databases & Distributed Systems**	2577
13.1	PostgreSQL Indexes	2579
13.2	Transactions and Locks	2593
13.3	MVCC	2607
13.4	Replication	2623
13.5	Backups and Point-in-Time Recovery	2639
13.6	Database Connection Pooling	2655
13.7	Redis	2669
13.8	Kafka	2685
13.9	RabbitMQ	2701
13.10	Horizontal and Vertical Scaling	2715
13.11	Partitioning and Sharding	2727
13.12	Consistency, Availability and the CAP Theorem	2739
13.13	Eventual Consistency	2751
13.14	Idempotency	2763
13.15	Distributed Locks and Queues	2777
★	Capstone: Postgres, Redis and Kafka under failure	2793

PART VI

Under the Hood

The deep systems knowledge that separates debugging from guessing: advanced networking, databases and distributed systems.



IN THIS PART

12	Advanced Networking	2321
13	Databases & Distributed Systems	2577

LAB NOTES What I ran, what I saw, what it means

LEVEL 12 · PART VI

Advanced Networking

12

Goal: Understand the network layers, protocols and traffic-management patterns behind large-scale production systems.

RECOMMENDED TIME
3-4 weeks

TOPICS
14

PRACTICE / CAPSTONE
Build a lab: Envoy/Nginx with TLS termination, mTLS between two services, rate limiting and a circuit breaker, and observe it with tcpdump/curl.

Topics

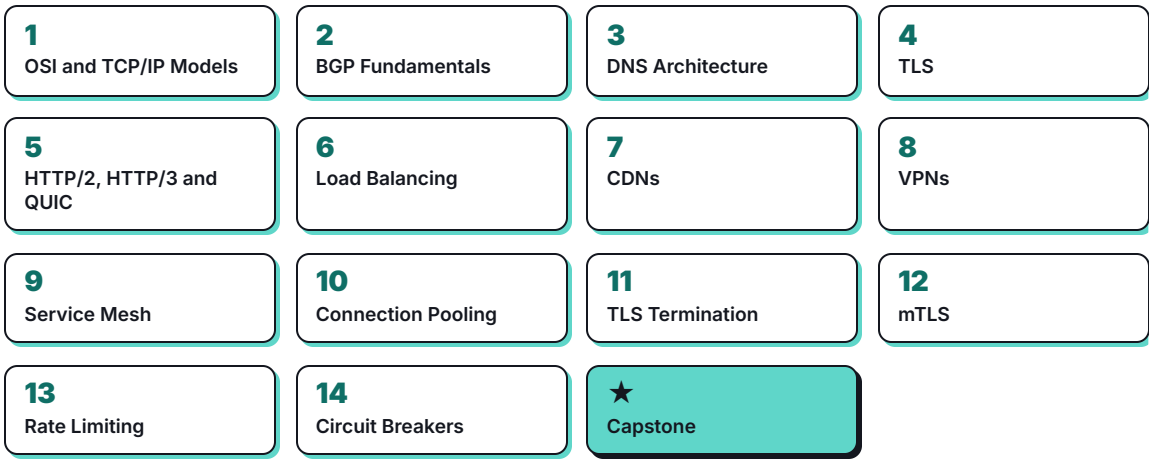
#	Topic	What you will learn
1	OSI and TCP/IP Models **p.2323**	layers, encapsulation, where each tool/product lives
2	BGP Fundamentals **p.2335**	autonomous systems, route advertisement, anycast, famous BGP outages
3	DNS Architecture **p.2349**	anycast resolvers, split-horizon, GeoDNS, DNSSEC, DNS failures at scale
4	TLS **p.2365**	PKI, cert chains, ACME/Let's Encrypt, cipher suites, rotation
5	HTTP/2, HTTP/3 and QUIC **p.2381**	multiplexing, head-of-line blocking, QUIC over UDP, 0-RTT
6	Load Balancing **p.2399**	round-robin, least-conn, consistent hashing, health checks, global LB
7	CDNs **p.2415**	edge caching, cache keys, purge, origin shield, Cache-Control
8	VPNs **p.2431**	IPsec vs WireGuard, site-to-site, client VPN, bastion vs VPN vs zero-trust
9	Service Mesh **p.2445**	sidecar vs ambient, control/data plane, when it is worth it
10	Connection Pooling **p.2463**	keep-alive, pool sizing, PgBouncer, pool exhaustion
11	TLS Termination **p.2479**	edge vs passthrough vs re-encrypt, offload trade-offs

#	Topic	What you will learn
12	mTLS **p.2497**	mutual auth, client certs, SPIFFE/SPIRE, cert rotation
13	Rate Limiting **p.2513**	token bucket, leaky bucket, sliding window, distributed limiting with Redis
14	Circuit Breakers **p.2533**	closed/open/half-open, retries with backoff+jitter, bulkheads, timeouts
★	Level capstone project **p.2553**	Build a lab: Envoy/Nginx with TLS termination, mTLS between two services, rate limiting and a circuit breaker, and observe it with tcpdump/curl.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



12.1 OSI and TCP/IP Models

Topic 1 of 14 | ~3 hours | Difficulty: Intermediate

1 What it is

A **network model** is a way to split the enormous problem "get these bytes from program A on one computer to program B on another" into **layers**. Each layer solves one narrow problem, talks only to the layer directly above and below it, and hides its details from everyone else. The two models you will meet are the **OSI model** (Open Systems Interconnection, seven layers, a teaching and vocabulary tool published by ISO) and the **TCP/IP model** (four or five layers, the architecture the real Internet actually runs on).

When a program sends data, every layer wraps it in its own envelope — a **header** (and sometimes a trailer) carrying that layer's addressing and control information. This wrapping is called **encapsulation**; the receiver unwraps in reverse order (**decapsulation**). If you remember one sentence from this topic: *a packet on the wire is an onion of headers, and every network tool you own looks at one particular layer of the onion.*

2 Why it matters in DevOps

- **Debugging is layer-hunting.** "The site is down" can mean a dead cable (L1), a wrong VLAN or ARP entry (L2), a missing route or security group (L3), a closed port or exhausted ephemeral ports (L4), an expired certificate (L5/L6 in OSI terms) or a bad `Host` header (L7). Walking up from the bottom turns a vague outage into a yes/no question per layer.
- **Choosing the right product.** A "load balancer" at L4 (AWS NLB, HAProxy in `mode tcp`) and one at L7 (AWS ALB, Nginx, Envoy) have different capabilities, costs and failure modes. You cannot choose without knowing what each layer can see.
- **Security rules.** Security groups and NACLs reason about L3/L4 (IPs, ports); a WAF reasons about L7 (paths, headers, bodies).
- **Vocabulary for teammates.** "Is that an L4 or L7 problem?" is a question you will hear on every incident bridge.

3 Real-world system example

An AWS request path. A browser request to `https://shop.example.com` crosses several layers' worth of products, each operating at a different layer of the stack (all publicly documented AWS behaviours):

Layer of the onion	AWS/Linux component that works there	What it can "see"
L2 (Ethernet/ENI)	the instance's virtual NIC (ENI)	MAC addresses, frames
L3 (IP)	VPC route tables, Internet Gateway, NAT Gateway, security group source/CIDR rules, NACLs	IP addresses and protocol (TCP/UDP/ICMP)
L4 (TCP/UDP)	Network Load Balancer, security group port rules	IP + port, connection state — not HTTP
L5-L6 (TLS)	an ALB or CloudFront terminating HTTPS with an ACM certificate	the TLS handshake, SNI (server name), certificates
L7 (HTTP)	Application Load Balancer, AWS WAF, Nginx/Envoy inside the cluster	method, path, headers, cookies, body

This is exactly why an **NLB can forward TLS traffic it cannot read** (it only needs IP and port), while an **ALB can route `/api/*` to one target group and `/static/*` to another** (it terminates TLS and parses HTTP). The same split exists in Kubernetes: a `Service` of type `LoadBalancer` is L4, an `Ingress` controller such as ingress-nginx or Envoy-based gateways is L7.

4 Diagram

How data from the `shop-api` is wrapped on its way down the stack — the TCP/IP model on the left, the OSI names on the right:

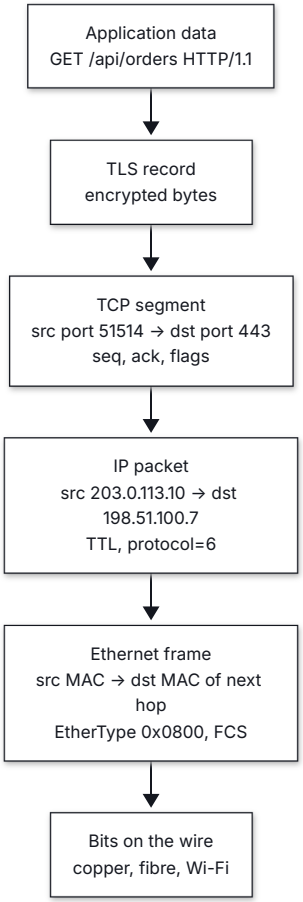


Figure 12.1-1

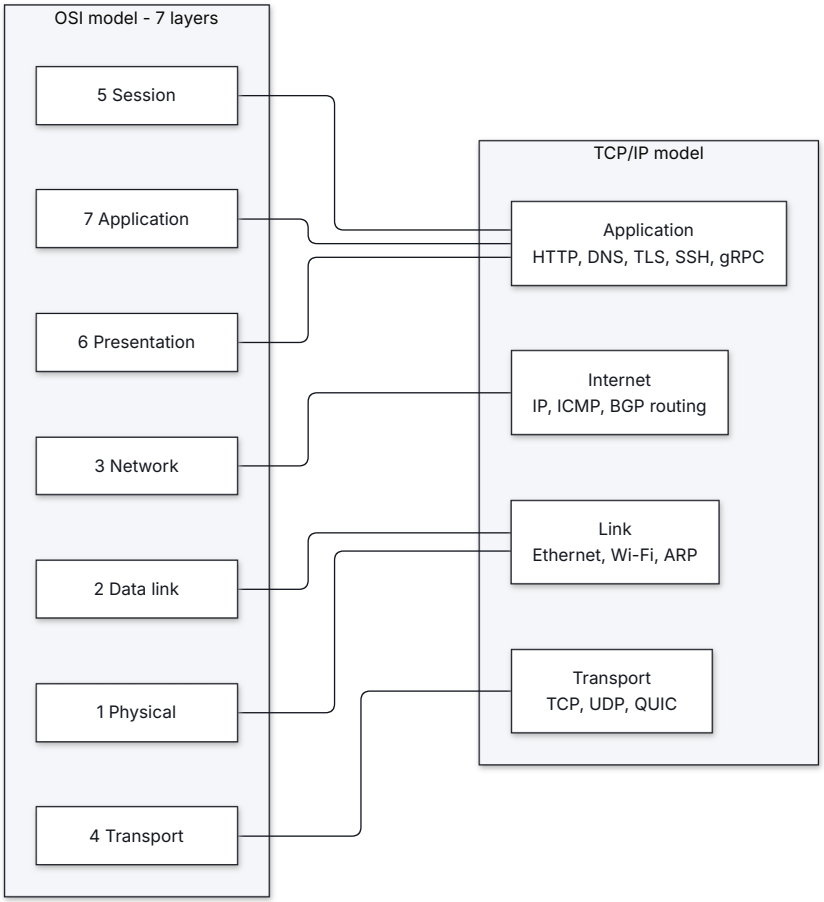


Figure 12.1-2

5 How it works

The layers and the tools that live there

OSI #	Name	PDU (unit of data)	Addressing	Typical protocols	Tools and products you use here
7	Application	message / request	URL, hostname	HTTP, gRPC, DNS, SMTP, SSH	`curl`, browser dev tools, Nginx, Envoy, ALB, WAF, API gateway
6	Presentation	—	—	TLS (encryption), JSON/Protobuf encoding	`openssl s_client`, cert-manager, mTLS
5	Session	—	—	TLS session resumption, RPC sessions	HTTP keep-alive and connection pools (informally)

OSI #	Name	PDU (unit of data)	Addressing	Typical protocols	Tools and products you use here
4	Transport	segment (TCP) / datagram (UDP)	port numbers	TCP, UDP, QUIC (over UDP)	`ss`, `nc`, `nmap`, NLB, HAProxy `mode tcp`, security-group port rules
3	Network	packet	IP address	IPv4, IPv6, ICMP, BGP/OSPF for route exchange	`ip route`, `ping`, `traceroute`, `mtr`, route tables, NAT, iptables/nftables
2	Data link	frame	MAC address	Ethernet, ARP, VLAN (802.1Q), Wi-Fi	`ip neigh`, `ip link`, switches, bridges, `tcpdump -e`
1	Physical	bits	—	copper, fibre, radio	cables, SFP optics, NIC link lights, `ethtool`

Notes that save arguments later:

- **OSI layers 5-7 collapse into "Application" in TCP/IP.** TLS is hard to place: it sits above TCP and below HTTP, so people call it "L5", "L6" or "L4.5". Say "TLS" and nobody is confused.
- **QUIC** (the transport under HTTP/3, topic 5) is a transport protocol implemented in user space *on top of UDP*, with TLS 1.3 built in — it blurs the L4/L5 line on purpose.
- **"L4 load balancer" vs "L7 load balancer"** simply means *the highest layer whose headers the device reads before deciding where to send the packet*. See also L4 versus L7 Load Balancing §2.9.
- **MTU (Maximum Transmission Unit)** is an L2/L3 limit — typically 1500 bytes for Ethernet payload. TCP's **MSS** (Maximum Segment Size) is derived from it (1460 for IPv4). Tunnels (VPNs, VXLAN, overlay networks) add headers and shrink the usable MTU — a classic source of "small requests work, large ones hang".

Encapsulation overhead for one HTTP request

For a small request over a normal Ethernet link: Ethernet header 14 bytes + IPv4 header 20 + TCP header 20 (more with options) = **54+ bytes of overhead before a single byte of your payload**. This is why chatty protocols and tiny messages are expensive, and why topic 10 (connection pooling) matters.

6 Step-by-step guide

Prerequisites: a Linux shell with `tcpdump`, `curl`, `ip`, `ss` (`sudo apt-get install -y tcpdump curl iproute2 python3`) and `sudo`. Everything stays on `127.0.0.1`.

1. **Start a tiny web server (L7) on a port (L4).**

```bash
python3 -m http.server 8099 --bind 127.0.0.1 &
ss -ltn 'sport = :8099'
```

   You should see a `LISTEN` socket on `127.0.0.1:8099` — the L4 endpoint (IP + port + protocol) that the application owns.

2. **Look at L2 and L3 of your own machine.**

```bash
ip -br link        # interfaces and MAC addresses (L2)
ip -br addr        # IP addresses (L3)
ip route           # where packets go next (L3)
ip neigh           # the ARP/NDP table: IP → MAC (L2/L3 glue)
```

   Output is host-specific. Find your default gateway in `ip route` (`default via … dev eth0`) and its MAC in `ip neigh`.

3. **Capture one HTTP request and read the onion from the outside in.**

```bash
sudo tcpdump -i lo -nn -e -v -c 8 'tcp port 8099' &
sleep 1; curl -s -o /dev/null http://127.0.0.1:8099/
```

   Output (abridged, similar to):

```
00:00:00:00:00:00 > 00:00:00:00:00:00, ethertype IPv4 (0x0800), length 74: (tos 0x0,
  ttl 64, ..., proto TCP (6), length 60)
    127.0.0.1.58188 > 127.0.0.1.8099: Flags [S], seq 1393828775, win 65495, options
  [mss 65495,sackOK,TS ...], length 0
```

   Read it layer by layer: `ethertype IPv4` and the MACs are **L2** (all zero on loopback), `ttl 64 ... proto TCP (6)` is **L3**, `Flags [S]` with ports `58188 > 8099` is **L4**. The first three packets (`[S]`, `[S.]`, `[.]`) are the TCP three-way handshake. Checksum warnings such as `cksum ... (incorrect → ...)` on `lo` are normal: the kernel skips checksums on loopback ("checksum offload").

4. **See the application layer in clear text.**

```bash
sudo tcpdump -i lo -nn -A -c 12 'tcp port 8099' &
sleep 1; curl -s -o /dev/null http://127.0.0.1:8099/
```

`-A` prints payload as ASCII: you will see `GET / HTTP/1.1`, `Host: 127.0.0.1:8099`, `User-Agent: curl/…` and the `HTTP/1.0 200 OK` response. That is **L7**, readable because there is no TLS.

5. **Prove which layer fails: walk the stack with deliberate breakage.**

```bash
curl -sS -m 3 http://127.0.0.1:8100/ ; echo "exit=$?"     # nothing listens on 8100
curl -sS -m 3 http://127.0.0.1:8099/nope -o /dev/null -w '%{http_code}\n'
```

The first command fails with `Failed to connect ... Connection refused` (exit 7): L3 worked (the host answered), **L4 refused** (RST — no listener). The second connects fine and returns `404`: L4 is healthy and the **L7** application said "no such path". "Refused" vs "timeout" vs "404" are three different layers.

6. **Map a real request path with `traceroute` / `mtr` (L3) and `curl -w` timings (L4-L7).**

```bash
curl -s -o /dev/null -w 'dns=%{time_namelookup}s tcp=%{time_connect}s tls=%
  {time_appconnect}s ttfb=%{time_starttransfer}s total=%{time_total}s\n'
  https://example.com
```

Output (values vary): the jumps between consecutive numbers are DNS, TCP handshake, TLS handshake and server think time — each belongs to a different layer.

7. **Clean up.**

```bash
kill %1 2>/dev/null; jobs
```

7 Real-world scenario

(Illustrative.) After a deployment, `shop-api` pods can reach PostgreSQL for small queries, but the nightly report that returns 8 MB result sets hangs forever. The on-call engineer works up the stack: DNS resolves (L7), TCP connects (L4), small queries return (so L3 routing is fine). Large responses stall — a pattern that points at **MTU**. `ping -M do -s 1472 <db-ip>` fails but `-s 1372` succeeds: a new overlay network added a tunnel header, so full-size 1500-byte packets are silently dropped because ICMP "fragmentation needed" messages were blocked by a firewall. Fix: lower the interface MTU on the overlay (or clamp TCP MSS) and allow ICMP type 3 code 4. **Lesson:** the symptom appeared at L7 ("report hangs"), but the cause was an L2/L3 size limit; layered debugging found it in minutes.

8 Common mistakes and best practices

Don't

- Treat the OSI model as a literal description of real software — HTTP over TLS over TCP does not map neatly onto layers 5-7. Use it for vocabulary.
- Block all ICMP "for security" — Path MTU Discovery and `traceroute` break, producing hanging large transfers.
- Debug an L7 symptom only at L7 (reading app logs for an hour) before confirming the lower layers with `ping`, `ss`, `curl -v`.
- Say "the load balancer is down" without saying which layer — an L4 and an L7 balancer fail differently.

Do

- Debug **bottom-up** (is the link up, is there a route, is the port open, does TLS complete, does the app answer) or **split the stack in half** with `curl -v` first.
- Learn what each tool sees: `tcpdump` sees everything on the wire but can only decode TLS if you supply keys; `curl -v` shows L4-L7; `ss` shows L4 sockets.
- Keep an MTU budget in mind for any tunnel (VPN, VXLAN, GRE, WireGuard: typically 1420 inside a 1500 link).
- Use the layer in your incident notes: "L4 connection resets from the NLB", not "network issue".

9 Tasks

- [ ] **T1 — Label the onion** *(≈30 min, beginner)*
  - **Goal:** attach a layer to every field of a real packet capture.
  - **Steps:**
    1. Follow steps 1 and 3 of the guide and save the output: `sudo tcpdump -i lo -nn -e -v -c 8 'tcp port 8099' 2>&1 | tee capture.txt`.
    2. In `capture.txt`, mark each field with its OSI layer (MAC = L2, TTL = L3, ports and flags = L4).
    3. Identify which three packets are the TCP handshake.
  - **Done when:** your annotated file labels at least 8 fields correctly and names SYN, SYN-ACK and ACK.
- [ ] **T2 — Same request, three tools, three layers** *(≈30 min, beginner)*
  - **Goal:** show that each tool reports a different layer.

- **Steps:**
  1. Run `ip route get 1.1.1.1` (L3), `ss -tn` while a `curl` is open (L4), and `curl -v http://127.0.0.1:8099/` (L7).
  2. Write one sentence per tool: what it observed and which layer it belongs to.
- **Done when:** each sentence names a layer and quotes a specific field from the output.

☐ **T3 — Read a timing waterfall** *(≈30 min, intermediate)*

- **Goal:** attribute latency to layers.
- **Steps:**
  1. Run the `curl -w` command from step 6 against three different HTTPS sites (e.g. `example.com`, one from your own cloud, one in another region).
  2. Compute DNS = `time_namelookup`, TCP = `time_connect - time_namelookup`, TLS = `time_appconnect - time_connect`, server = `time_starttransfer - time_appconnect`.
- **Done when:** you have a table of the four numbers for each site and one sentence on which layer dominates and why (distance, handshake round-trips, slow backend).

☐ **T4 — Break each layer on purpose** *(≈45 min, intermediate)*

- **Goal:** recognise the signature of failure at L3, L4 and L7.
- **Steps:**
  1. L7: request a missing path and record the status code.
  2. L4: connect to an unused port and record the error; then drop traffic silently with `sudo iptables -I INPUT -p tcp --dport 8099 -j DROP` and observe a **timeout** instead of "refused". Remove the rule with `sudo iptables -D INPUT -p tcp --dport 8099 -j DROP`.
  3. L3: `curl -m 3 http://192.0.2.1/` (a documentation address with no route to a real host) and record the result.
- **Done when:** you can explain, from the error text alone, why "refused" (an active RST), "timeout" (silently dropped) and "404" (application answered) point to different layers.

☐ **T5 — Discover your path MTU** *(≈30 min, advanced)*

- **Goal:** use L3 knowledge to find a size limit.
- **Steps:**
  1. Run `ping -c 2 -M do -s 1472 example.com` (1472 bytes payload + 28 bytes of headers = 1500).
  2. Reduce `-s` until pings succeed; add 28 to get your path MTU.
  3. Explain what would change if you were behind a VPN that adds 60 bytes of overhead.

- **Done when:** you report a measured MTU (typically 1500, or lower behind a tunnel) and the MSS TCP would derive from it.

☐ **T6 — Prove it: place the product** *(≈30 min, advanced)*

- **Goal:** classify products by layer without notes.
- **Steps:**
  1. On a blank page write each of: security group, NACL, AWS NLB, AWS ALB, WAF, Nginx, HAProxy `mode tcp`, `iptables`, Envoy, VPN gateway, CDN.
  2. Put each at the highest layer it inspects and list one thing it *cannot* do because of that.
  3. Check against section 3 and 5 and fix your errors, then explain the page to someone aloud.
- **Done when:** at least 10 of 11 placements are correct and each has a correct "cannot do" example (for instance, an NLB cannot route on URL path).

10 Self-check questions

1. What does encapsulation mean, and in which order are headers added when sending?
2. Which layer does `Connection refused` come from, and how does it differ from a connect timeout?
3. Why can a Network Load Balancer pass TLS traffic through but not route on URL path?
4. Why does adding a VPN or overlay network sometimes make large transfers hang while small ones work?
5. Where does TLS sit in the OSI and TCP/IP models, and why is that awkward?
6. QUIC runs over UDP. Which responsibilities that TCP normally handles must QUIC implement itself?

ANSWERS

1. Each layer wraps the data from the layer above in its own header: application data, then TLS record, then TCP, then IP, then Ethernet. The receiver removes them in reverse order.
2. Layer 4. "Refused" means the host answered with a TCP RST because nothing listens on that port; a timeout means packets were silently dropped (firewall, routing, host down).
3. It reads only IP and port (L4). The path is inside the encrypted HTTP payload, which only a device that terminates TLS and parses HTTP (an ALB or proxy) can see.
4. Tunnels add headers, reducing the usable MTU. Full-size packets exceed it and get dropped; if ICMP "fragmentation needed" is also blocked, Path MTU Discovery fails and the connection stalls.

5. Above TCP and below HTTP; it does not map cleanly to OSI 5 or 6, so it is usually just called "TLS" or "L4.5".
6. Reliability (retransmission), ordering per stream, congestion control, flow control and connection setup — plus its own encryption via integrated TLS 1.3.

11 Further reading

- *Computer Networking: A Top-Down Approach* by Kurose and Ross — the clearest layered treatment of the TCP/IP stack.
- *TCP/IP Illustrated, Volume 1* by W. Richard Stevens (and Kevin Fall, 2nd ed.) — packet-level detail with captures.
- RFC 1122 (Requirements for Internet Hosts) and RFC 9293 (TCP) — the primary sources; find them on rfc-editor.org.
- `man tcpdump`, `man ss`, `man ip` — option reference for the tools used above.
- Earlier in this roadmap: IP, MAC, ARP and DHCP §2.1 and Network Debugging Tools §2.10.

LAB NOTES What I ran, what I saw, what it means

12.2 BGP Fundamentals

Topic 2 of 14 | ~4 hours | Difficulty: Advanced

1 What it is

The Internet is not one network; it is roughly a hundred thousand independently run networks glued together. Each of these is an **Autonomous System (AS)**: a set of IP prefixes under one administrative policy, identified by an **AS Number (ASN)** such as `AS13335` (Cloudflare) or `AS16509` (Amazon). **BGP — the Border Gateway Protocol (version 4, RFC 4271)** — is the protocol ASes use to tell each other *"I can reach these IP prefixes, and here is the chain of ASes your traffic would cross."* Routers that speak BGP keep a table of prefixes, each with a list of candidate paths, pick the best, and re-advertise it to neighbours.

BGP is a **path-vector** protocol: every advertisement carries an **AS_PATH** (the ordered list of ASes it has crossed). It carries no notion of link speed or latency; it expresses **policy** (who do I trust, who do I pay, who do I prefer). Inside your own AS you use other protocols (OSPF, IS-IS, static routes); BGP is the *inter-domain* protocol. Two flavours: **eBGP** between different ASes, **iBGP** between routers inside the same AS.

2 Why it matters in DevOps

- **Cloud connectivity runs on it.** AWS Direct Connect, Site-to-Site VPN with dynamic routing, Transit Gateway, Google Cloud Router and Azure ExpressRoute all use BGP. A mis-announced prefix is a connectivity outage.
- **Anycast and global availability.** CDNs, public DNS resolvers and DDoS scrubbers announce the *same* IP from many places using BGP (section 5). You cannot understand "why did users in Europe hit Frankfurt" without it.
- **Kubernetes on bare metal.** MetalLB (BGP mode), Calico and Cilium peer with top-of-rack routers to advertise Service and Pod IPs.
- **Blast radius is the whole Internet.** BGP trusts what neighbours say. Typos, leaks and hijacks cause outages for companies that did nothing wrong, and "we can't even reach our own servers to fix it" is a recurring theme in post-mortems.

3 Real-world system example

Facebook, 4 October 2021. Facebook has described the incident in its own engineering write-up. During routine maintenance on the global backbone, a command intended to assess backbone capacity unintentionally disconnected Facebook's data centres from each other, and an audit tool that should have blocked the command failed to do so. Facebook's authoritative DNS servers are designed to withdraw their BGP advertisements when they cannot reach the data centres (a health-check reaction to avoid serving bad answers). The result: the routes to the DNS servers disappeared from the Internet, `facebook.com` stopped resolving, and the outage lasted roughly six hours. Recovery was slowed because internal tools and physical access systems depended on the same network. Lessons: **DNS, BGP and your own management plane are one dependency chain**, and an out-of-band path to your routers is not optional.

Pakistan Telecom and YouTube, 24 February 2008. Pakistan Telecom (AS17557) was ordered to block YouTube domestically and originated a more specific prefix, `208.65.153.0/24`, inside YouTube's `208.65.152.0/22`. The route leaked to its upstream provider (PCCW, AS3491) and spread across the Internet. Because routers always prefer the **longest matching prefix**, much of the world's YouTube traffic flowed to Pakistan. YouTube responded by announcing more specific prefixes of its own; the disruption lasted on the order of two hours. This is the textbook **prefix hijack by more-specific announcement**, and the reason RPKI and prefix filtering exist.

Cloudflare, Google and AWS use BGP for anycast. Cloudflare's `1.1.1.1` resolver and Google's `8.8.8.8` are single IP addresses announced from many data centres; each user is routed to a "nearby" one by BGP's shortest-AS-path preference. AWS Global Accelerator provides static anycast IP addresses that enter the AWS network at the edge location nearest the user.

4 Diagram

How an announcement travels and how a hijack by a more specific prefix wins (this is what the lab below reproduces):

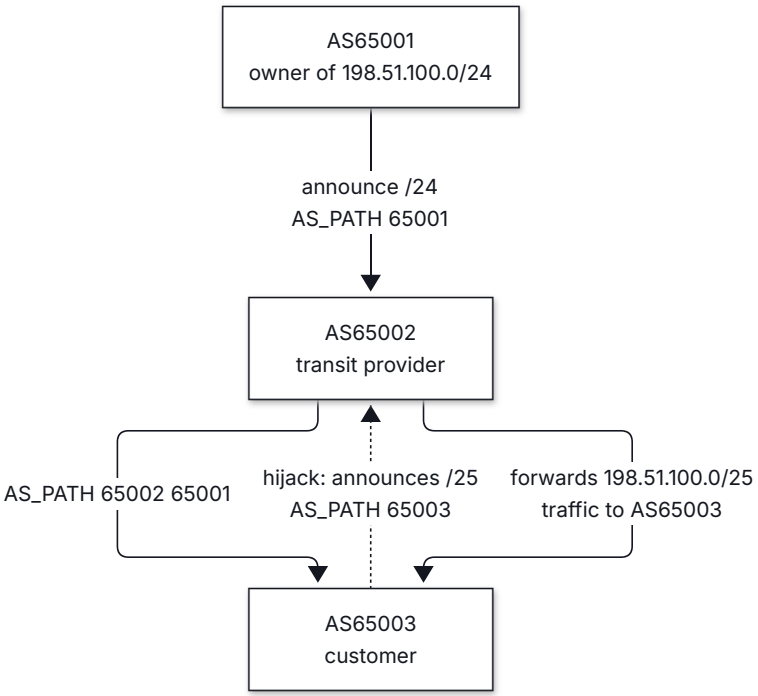


Figure 12.2-1

The BGP session itself is a small state machine over TCP port 179:

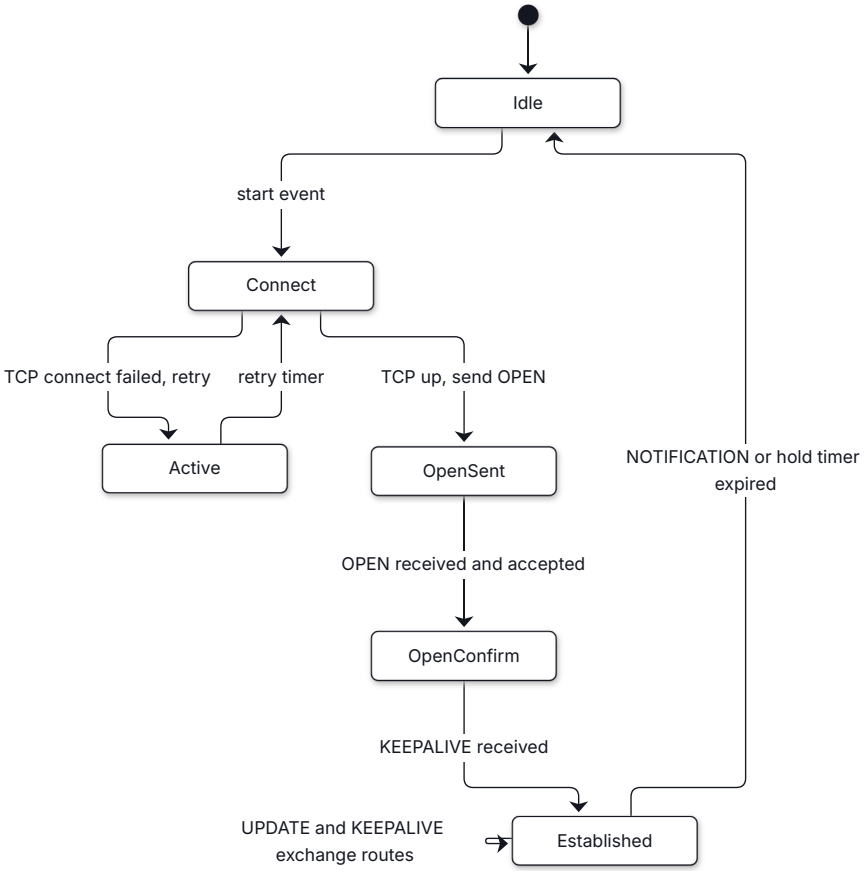


Figure 12.2-2

5 How it works

Messages and sessions

Message	Purpose
`OPEN`	Start a session: sends my ASN, hold time, router ID and capabilities (IPv6, 4-byte ASNs, route refresh, graceful restart)
`UPDATE`	Announce new/changed routes (prefix + attributes) **or withdraw** routes — the only message that changes the routing table
`KEEPALIVE`	Sent about every hold-time/3 to prove the neighbour is alive (default hold time often 90 s, 180 s in some implementations)

Message	Purpose
`NOTIFICATION`	Report an error and close the session

If the hold timer expires with no KEEPALIVE/UPDATE, the neighbour is declared dead and **all its routes are withdrawn** — this is how a failed link or router "disappears" from the Internet. Tuning faster detection is done with **BFD** (Bidirectional Forwarding Detection), which fails over in under a second.

Key path attributes

Attribute	Meaning
`AS_PATH`	ASes crossed so far. Used for loop prevention (a router rejects a path containing its own ASN) and as a tie-breaker (shorter is preferred)
`NEXT_HOP`	IP of the router to send packets to
`LOCAL_PREF`	iBGP-only preference inside your AS: higher wins. The main knob for "prefer provider A over B"
`MED`	Hint to a neighbour AS about which of your entry points you prefer (lower wins)
`ORIGIN`, `COMMUNITY`	Origin type and free-form tags that providers use to trigger actions (e.g. blackhole, do not export to X, set local-pref)

Best-path selection (simplified)

1. Longest prefix wins before BGP even compares — the forwarding table uses **longest-prefix match** (`/25` beats `/24`).
2. For the same prefix: highest `LOCAL_PREF` → shortest `AS_PATH` → lowest origin type → lowest MED → eBGP over iBGP → lowest IGP metric to next hop → lowest router ID.

Policy: who gets what

Commercial relationships drive filters. **Customers** pay you (you announce their routes to everyone), **providers** you pay (you announce only your own and your customers' routes to them), **peers** exchange traffic for free (you announce only your own and your customers' routes). Violating this — re-announcing a provider's routes to another provider — is a **route leak**.

Anycast

Anycast means announcing the same prefix from multiple locations. Every location is a valid destination; BGP delivers each client to the topologically closest one (by AS-path length and policy). Withdraw the prefix at a failing site and traffic automatically shifts to the next-closest. It underpins DNS roots, public resolvers and CDN edges, and works especially well

for UDP and short, stateless requests. Because the path to a site can change mid-flow, long-lived TCP connections need care (e.g. consistent hashing or connection-state design, topic 6).

Securing BGP

- **Prefix filters / IRR objects:** accept from customers only the prefixes they are registered to originate.
- **Max-prefix limits:** tear down a session if a neighbour suddenly sends far more prefixes than expected.
- **RPKI (Resource Public Key Infrastructure) and ROV (Route Origin Validation):** the address owner publishes a signed **ROA** (Route Origin Authorization) saying "AS X may originate prefix P up to length N"; routers mark announcements `valid`, `invalid` or `not-found` and drop invalids. It stops many accidental hijacks but not forged AS paths.
- **MANRS** (Mutually Agreed Norms for Routing Security) is an industry programme describing baseline operator practices.

6 Step-by-step guide

You will build three routers as Linux **network namespaces** on one machine with the **BIRD 2** routing daemon, bring up two eBGP sessions, watch a route propagate, hijack it with a more specific prefix, then stop the hijack with a filter. All addresses are private/documentation ranges; nothing touches the real Internet.

Prerequisites: a Linux VM or laptop with root (`sudo`); Docker is not needed. Install BIRD:

```bash
sudo apt-get update && sudo apt-get install -y bird2
sudo systemctl disable --now bird 2>/dev/null || true   # we will run our own instances
mkdir -p ~/bgplab && cd ~/bgplab
```

Topology: `AS65001 (r1) ──10.12.0.0/30── AS65002 (r2) ──10.23.0.0/30── AS65003 (r3)`. r1 owns `198.51.100.0/24` (a documentation range).

1. **Create the topology.**

```bash
cat > mk.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
for n in r1 r2 r3; do ip netns add $n; ip netns exec $n ip link set lo up; done
ip link add v12a type veth peer name v12b
ip link set v12a netns r1; ip link set v12b netns r2
ip link add v23a type veth peer name v23b
ip link set v23a netns r2; ip link set v23b netns r3
ip netns exec r1 ip addr add 10.12.0.1/30 dev v12a; ip netns exec r1 ip link set v12a
  up
ip netns exec r2 ip addr add 10.12.0.2/30 dev v12b; ip netns exec r2 ip link set v12b
  up
ip netns exec r2 ip addr add 10.23.0.1/30 dev v23a; ip netns exec r2 ip link set v23a
  up
ip netns exec r3 ip addr add 10.23.0.2/30 dev v23b; ip netns exec r3 ip link set v23b
  up
EOF
sudo bash mk.sh && ip netns list
```

   Expect `r3`, `r2`, `r1` to be listed.

2. **Write the three BIRD configs.** r1 originates the prefix; r2 is a transit that passes everything; r3 only listens.

```bash
cat > r1.conf <<'EOF'
router id 10.0.0.1;
protocol device {}
protocol static origin {
  ipv4;
  route 198.51.100.0/24 blackhole;     # the prefix AS65001 owns
}
protocol bgp to_r2 {
  local 10.12.0.1 as 65001;
  neighbor 10.12.0.2 as 65002;
  ipv4 { import all; export where source = RTS_STATIC; };
}
EOF
cat > r2.conf <<'EOF'
router id 10.0.0.2;
protocol device {}
protocol bgp to_r1 {
  local 10.12.0.2 as 65002;
  neighbor 10.12.0.1 as 65001;
  ipv4 { import all; export all; };
}
protocol bgp to_r3 {
  local 10.23.0.1 as 65002;
```

```
  neighbor 10.23.0.2 as 65003;
  ipv4 { import all; export all; };
}
EOF
cat > r3.conf <<'EOF'
router id 10.0.0.3;
protocol device {}
protocol bgp to_r2 {
  local 10.23.0.2 as 65003;
  neighbor 10.23.0.1 as 65002;
  ipv4 { import all; export none; };
}
EOF
```

(BIRD 2 requires explicit `import` / `export` for eBGP; the `blackhole` static route just gives r1 something to announce.)

3. **Start the daemons, one per namespace.**

```bash
for n in r1 r2 r3; do sudo ip netns exec $n bird -c $PWD/$n.conf -s $PWD/$n.ctl; done
sleep 6
for n in r1 r2 r3; do echo "== $n"; sudo birdc -s $PWD/$n.ctl show protocols | tail -n
  +3; done
```

Expected (similar to): every `BGP` line shows `up ... Established`.

4. **Watch the route propagate.**

```bash
sudo birdc -s $PWD/r3.ctl show route all
```

```
198.51.100.0/24      unicast [to_r2 07:45:32.028] * (100) [AS65001i]
        via 10.23.0.1 on v23b
        BGP.origin: IGP
        BGP.as_path: 65002 65001
        BGP.next_hop: 10.23.0.1
        BGP.local_pref: 100
```

r3 learned r1's prefix with `as_path: 65002 65001` — it crossed AS65002 then originated in AS65001. That is the path vector.

5. **Withdraw and re-announce.**

```bash
sudo birdc -s $PWD/r1.ctl disable to_r2; sleep 2; sudo birdc -s $PWD/r3.ctl show route
  # table is empty
sudo birdc -s $PWD/r1.ctl enable to_r2;  sleep 6; sudo birdc -s $PWD/r3.ctl show route
  # route is back
```

   This is what a failed uplink looks like to the rest of the world: an `UPDATE` withdrawing the prefix, then convergence on the alternative (here, none).

6. **Hijack it with a more specific prefix.** r3 now announces `198.51.100.0/25`, which it does not own:

```bash
cat > r3.conf <<'EOF'
router id 10.0.0.3;
protocol device {}
protocol static mine {
  ipv4;
  route 198.51.100.0/25 blackhole;     # not ours: the hijack
}
protocol bgp to_r2 {
  local 10.23.0.2 as 65003;
  neighbor 10.23.0.1 as 65002;
  ipv4 { import all; export where source = RTS_STATIC; };
}
EOF
sudo birdc -s $PWD/r3.ctl configure; sleep 4
sudo birdc -s $PWD/r2.ctl show route
sudo birdc -s $PWD/r2.ctl show route for 198.51.100.10
```

```
198.51.100.0/25      unicast [to_r3 ...] * (100) [AS65003i]   via 10.23.0.2 on v23a
198.51.100.0/24      unicast [to_r1 ...] * (100) [AS65001i]   via 10.12.0.1 on v12b
...
198.51.100.0/25      unicast [to_r3 ...] * (100) [AS65003i]   via 10.23.0.2 on v23a
  ← lookup for .10 goes to the hijacker
```

   Both routes are in the table, but a packet for `198.51.100.10` matches the longer `/25` and goes to AS65003, even though AS65001's path is shorter and legitimate.

7. **Stop it with an inbound prefix filter on the transit.** Add to the top of `r2.conf` and

change the `to_r3` session:

```bash
python3 - <<'EOF'
s = open('r2.conf').read()
s = s.replace("neighbor 10.23.0.2 as 65003;\n  ipv4 { import all; export all; };",
              "neighbor 10.23.0.2 as 65003;\n  ipv4 { import filter customer_r3_in;
  export all; };")
s = '''filter customer_r3_in {
  # AS65003 may only originate 203.0.113.0/24 -- anything else is rejected
  if net ~ [ 203.0.113.0/24 ] then accept;
  reject;
}
''' + s
open('r2.conf', 'w').write(s)
EOF
sudo birdc -s $PWD/r2.ctl configure; sleep 2
sudo birdc -s $PWD/r2.ctl show route
```

Only the legitimate `198.51.100.0/24 ... [AS65001i]` remains. This is a miniature version of what real providers do with IRR/RPKI-derived customer prefix lists.

8. **Tear down.**

```bash
for n in r1 r2 r3; do sudo birdc -s $PWD/$n.ctl down; done
for n in r1 r2 r3; do sudo ip netns del $n; done
ip netns list    # empty
```

7 Real-world scenario

(Illustrative.) `shop-api` runs in AWS `us-east-1` with a Direct Connect link to a partner's data centre, plus a backup Site-to-Site VPN, both with BGP. One Monday the partner's network team adds a new `/24` and accidentally announces the VPN's prefix with a shorter AS path than the Direct Connect. Orders flow over the slower, encrypted VPN, latency doubles and the VPN gateway saturates; the on-call engineer sees p99 latency climb but no packet loss. Investigation: `show route` on the router shows the preferred next hop changed; the AWS console's BGP status shows more prefixes received than expected. Fix: raise `LOCAL_PREF` on Direct Connect routes so provider choice is pinned by policy, not by AS-path accident, and set a max-prefix limit and an inbound prefix-list on both sessions. **Lesson:** never leave path choice to implicit tie-breakers; set `LOCAL_PREF` explicitly and filter what you accept.

8 Common mistakes and best practices

Don't

- Accept all routes from a customer or peer ("`import all`") on a real network.
- Announce prefixes that are not yours — including "just a more specific for traffic engineering" without checking who else owns the space.
- Re-advertise routes learned from one provider to another (a route leak).
- Build a management plane that depends on the same network you are fixing (out-of-band access, console servers and a second DNS provider matter).
- Assume anycast keeps long-lived TCP flows pinned; path changes can reset them.

Do

- Filter inbound by prefix and AS-path, set max-prefix limits, and enable RPKI ROV where your platform supports it.
- Publish ROAs for your own prefixes in your RIR's portal.
- Use BFD for fast failure detection and graceful restart for planned maintenance.
- Set `LOCAL_PREF` explicitly for "primary" and "backup" links, and test failover during business hours before you need it.
- Monitor your prefixes from the outside (public looking glasses, RIPE RIS or RouteViews data, or a commercial BGP monitoring service) and alert on origin changes.

9 Tasks

- [ ] **T1 — Build the three-AS lab** *(≈45 min, intermediate)*
  - **Goal:** bring up two eBGP sessions and see a route with its AS path.
  - **Steps:**
    1. Follow guide steps 1-4 on a Linux VM with `sudo`.
    2. Run `sudo birdc -s $PWD/r2.ctl show protocols all to_r1` and find the neighbour AS, session type and capabilities.
  - **Done when:** `show route all` on r3 shows `BGP.as_path: 65002 65001` and all sessions read `Established`.
- [ ] **T2 — Watch a withdrawal** *(≈20 min, beginner)*
  - **Goal:** see how the rest of the network reacts when an origin disappears.
  - **Steps:**
    1. Run step 5 (`disable` then `enable`).

2. Time how long it takes for the route to return on r3 after `enable`.

- **Done when:** you have observed an empty table, then the route back, and can say which BGP message type carried the withdrawal.

☐ **T3 — Hijack and defend** *(≈45 min, advanced)*

- **Goal:** reproduce a more-specific hijack and stop it with a filter.
- **Steps:**
  1. Run steps 6 and 7.
  2. Change the filter so r3 may originate `203.0.113.0/24` only, add a static route for that prefix on r3 and confirm r2 accepts it while rejecting `198.51.100.0/25`.
- **Done when:** `show route for 198.51.100.10` on r2 returns the `/24` via AS65001, and `203.0.113.0/24` is accepted from r3.

☐ **T4 — Break it: prepend and prefer** *(≈45 min, advanced)*

- **Goal:** control path choice with policy.
- **Steps:**
  1. Add a fourth router r4 (AS65004) connected to both r2 and r3 and originating `192.0.2.0/24`, or simply a second link r1-r3 so r3 has two paths to `198.51.100.0/24`.
  2. Make r3 prefer the longer AS-path by setting `bgp_local_pref = 200` on an import filter, and observe the best-path change with `show route all`.
  3. Undo it using AS-path prepending (`bgp_path.prepend(65001);` in an export filter) and observe the other direction.
- **Done when:** `show route all` shows the chosen path flipping between the two sessions as you change local-pref and prepending, and you can state which attribute outranked which.

☐ **T5 — Read real outage data** *(≈45 min, intermediate)*

- **Goal:** connect theory to a public incident.
- **Steps:**
  1. Read Facebook's engineering post "More details about the October 4 outage" and Cloudflare's blog analysis of the same event.
  2. Draw the dependency chain (backbone → data centres → DNS servers' health check → BGP advertisement → DNS resolution → apps).
  3. Write down three changes that would have shortened recovery.
- **Done when:** your diagram contains at least five links in the chain and your three proposals include an out-of-band access path.

- [ ] **T6 — Prove it: explain a hijack in five minutes** *(≈30 min, advanced)*
  - **Goal:** teach BGP trust to someone else.
  - **Steps:**
    1. Without notes, explain: AS, prefix, AS_PATH, longest-prefix match, why a `/25` beats a `/24`, and one control (filters, max-prefix, RPKI).
    2. Draw the topology from your lab on paper and mark where the filter sits.
  - **Done when:** a listener can state what RPKI ROV would have done to the `/25` announcement.

10 Self-check questions

1. What is an Autonomous System and what does an AS_PATH contain?
2. Why does a more specific prefix (`/25`) hijack traffic even when the legitimate `/24` has a shorter AS path?
3. How does a BGP neighbour failure become visible to the rest of the Internet?
4. How does anycast give you both "nearest site" and automatic failover?
5. What does RPKI ROV protect against, and what does it not?
6. In the Facebook 2021 outage, why did withdrawing BGP routes to DNS servers take the whole site down?

ANSWERS

1. An AS is a set of IP prefixes under a single routing policy, identified by an ASN. AS_PATH is the ordered list of ASNs the announcement has crossed.
2. Forwarding uses longest-prefix match before any BGP preference comparison, so the `/25` is selected for addresses inside it.
3. The hold timer expires (or BFD declares the link down), the session closes, and the router sends UPDATE messages withdrawing all routes learned via that neighbour.
4. The same prefix is announced from many sites. Routers pick the best path (usually the shortest AS path), so users reach a nearby site; if a site withdraws, the next-best path takes over after convergence.
5. It validates that the origin AS is authorised for the prefix and length, stopping many accidental hijacks and origin mistakes; it does not validate the full AS path, so path forgery and some leaks remain possible.
6. The authoritative DNS servers withdrew their own routes after a health check failed; with no route to the DNS servers, nobody could resolve the domain, and internal tooling that relied on the same network could not be used to repair it quickly.

11 Further reading

- RFC 4271 (BGP-4), RFC 6811 (RPKI-based prefix origin validation) and RFC 7454 (BGP operations and security) at rfc-editor.org.
- *Internet Routing Architectures* by Sam Halabi, and *BGP* by Iljitsch van Beijnum (O'Reilly) — operator-focused introductions.
- Facebook Engineering, "More details about the October 4 outage"; Cloudflare's blog posts on the same event and on route leaks.
- The BIRD user's guide (bird.network.cz) and the MANRS website (manrs.org).
- Related topics in this level: Load Balancing §12.6 for global traffic steering and DNS Architecture §12.3 for anycast resolvers.

12.3 DNS Architecture

Topic 3 of 14 | ~4 hours | Difficulty: Advanced

1 What it is

The **Domain Name System (DNS)** turns names such as `api.example.com` into records (usually IP addresses). In Level 2 §2.5 you learned the lookup path: a stub resolver asks a **recursive resolver**, which walks the hierarchy (root → TLD → authoritative servers) and caches the answer for its **TTL** (time to live). This topic is about the *architecture* around that path at production scale:

- **Authoritative servers** hold the truth for a zone and answer only for it. **Recursive resolvers** (also called caching resolvers) answer for everyone by asking authoritative servers, and cache. Running both on one box is a classic mistake.
- **Anycast resolvers and authoritative servers:** one IP address announced from many locations over BGP (topic 2 §12.2), so each client reaches a nearby instance and a failed site simply stops being announced.
- **Split-horizon DNS** (split-brain): the same name returns *different answers depending on who asks* — typically a private IP inside your network and a public IP outside.
- **GeoDNS / latency-based / weighted routing:** the authoritative server picks the answer based on where the resolver (or, with EDNS Client Subnet, the client) appears to be, or on health and weights.
- **DNSSEC (DNS Security Extensions):** zones are cryptographically signed so a validating resolver can detect forged or tampered answers.

2 Why it matters in DevOps

- **DNS is the dependency of every dependency.** Service discovery, TLS certificate issuance (ACME DNS-01), failover, canary releases and multi-region routing all work by changing DNS answers. When DNS is wrong, everything looks broken at once.
- **TTLs are your rollback speed.** A 24-hour TTL on `api.example.com` means a bad record cannot be fixed for up to a day for clients that already cached it; a 30-second TTL means more query load but fast cutover.

- **Failures are correlated and hard to see.** Caches hide a dead authoritative server for minutes or hours, then everything fails together. Retries from stubs and resolvers can multiply a small problem into a flood.
- **Hybrid networks need split-horizon.** Private hosted zones, VPN users and Kubernetes clusters each want to resolve `db.example.com` to a private address that must never be exposed publicly.
- **DNSSEC misconfiguration is an outage.** A validating resolver treats a broken signature as `SERVFAIL`, not as "unsigned", so an expired signature or a stale DS record takes your domain offline for validating users.

3 Real-world system example

Public anycast resolvers. Cloudflare (`1.1.1.1`), Google Public DNS (`8.8.8.8`) and Quad9 (`9.9.9.9`) each publish one address that is announced from many sites via BGP. A query from Lagos and a query from Frankfurt to `1.1.1.1` reach different data centres. These services validate DNSSEC and document support for EDNS Client Subnet, QNAME minimisation and encrypted transports (DNS over TLS and DNS over HTTPS).

The root and TLD servers. The DNS root is served by 13 named server identities (`a.root-servers.net` … `m.root-servers.net`) operated by multiple organisations; each identity is anycast from hundreds of instances worldwide. Most large TLD operators and DNS hosting providers also use anycast.

Amazon Route 53 offers public and **private hosted zones** (split-horizon by VPC association), plus weighted, latency, geolocation, failover and multi-value routing policies, with health checks that remove unhealthy endpoints from answers. **Kubernetes** runs CoreDNS as the in-cluster authoritative/forwarding server for `cluster.local` and forwards everything else upstream (Level 8, Service Discovery §8.17).

Dyn, 21 October 2016. A large DDoS attack by the Mirai botnet (compromised IoT devices) against the managed DNS provider Dyn made many customers' names unresolvable for parts of the day. Well-known sites including Twitter, Netflix, Reddit and GitHub were widely reported as affected for many users. The lesson that stuck: **a single DNS provider is a single point of failure**, and many organisations afterwards added a second authoritative provider.

Facebook, 4 October 2021 (see BGP Fundamentals §12.2): authoritative DNS servers withdrew their own BGP routes after a health check failed, so a backbone problem became a "the name does not exist anywhere" problem.

AWS, 19-20 October 2025. AWS's published post-event summary of the large `us-east-1` disruption attributes its start to a latent race condition in the DNS management automation for the DynamoDB regional endpoint, which left an empty DNS record for that endpoint; many dependent services were affected. The generic lesson: DNS automation is production code and needs the same guard rails as any deploy pipeline.

4 Diagram

Where each architectural piece sits on the path of one query:

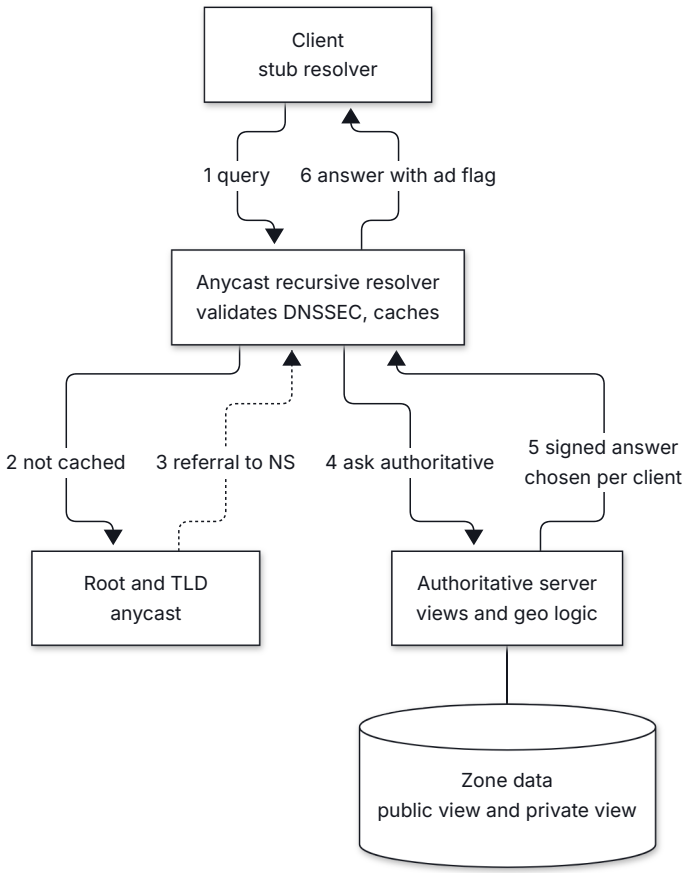


Figure 12.3-1

How a validating resolver proves an answer, from the root's trust anchor down to your A record (the *chain of trust*):

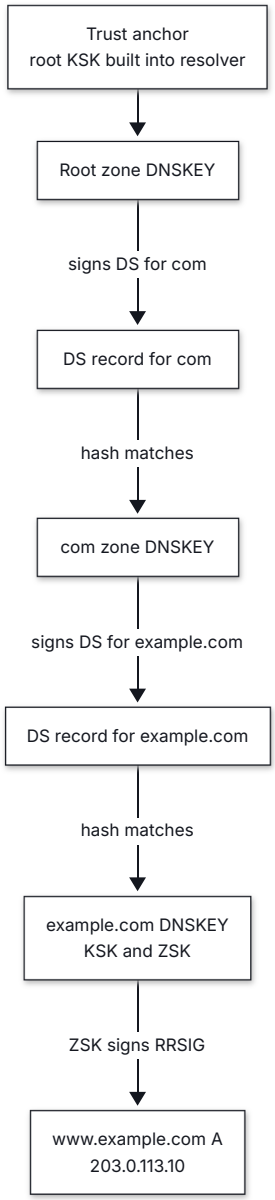


Figure 12.3-2

5 How it works

Authoritative versus recursive

	Authoritative	Recursive (caching) resolver
Answers for	Only the zones it hosts	Any name, by querying others
Caches?	No (it is the source)	Yes, honouring TTL
Open to the Internet?	Yes, must be	Should be restricted or well rate-limited (open resolvers are abused for DDoS amplification)
Examples	BIND, NSD, Knot DNS, PowerDNS, Route 53, Cloudflare	Unbound, BIND, Knot Resolver, CoreDNS (forwarding), 1.1.1.1, 8.8.8.8

Anycast for DNS

DNS over UDP is ideal for anycast: a request and response are one packet each, so a mid-flight route change rarely matters. Large queries and zone transfers use TCP, which tolerates anycast because sites are chosen by BGP and change rarely. Operators add health-based **withdrawal**: a local agent checks the DNS daemon, and if it fails the site stops announcing the prefix, so traffic shifts to the next-closest site in seconds to minutes. Each site is also given a unique unicast address so you can talk to it directly (for debugging and zone transfers). You can often ask *which* instance answered with `dig CH TXT id.server @<resolver>` or `dig +nsid`; the exact identifiers are operator-specific.

Split-horizon

Two ways to implement it:

1. **Views** in the DNS server (BIND `view` with `match-clients`): one server, different zone files by source IP. Simple but easy to leak private data if the match rule is too broad.
2. **Separate servers or private zones** (Route 53 private hosted zones, CoreDNS in a cluster, a corporate resolver forwarding `corp.example.com`): clearer blast radius and the preferred cloud pattern.

Rules of thumb: private zones must only be reachable from private networks; keep public and private zone content as similar as possible (otherwise engineers debug the wrong horizon); make sure every record that exists in both has the same *meaning*.

GeoDNS and traffic-steering policies

Policy	Chooses answer by	Typical use
Geolocation	Country or continent of the resolver (or ECS client subnet)	Data residency, regional content

Policy	Chooses answer by	Typical use
Latency-based	Measured latency from the resolver's region to your regions	Multi-region APIs
Weighted	Fixed percentages	Canary and migration (Level 5, Canary §5.12)
Failover	Health check status (primary/secondary)	Active-passive DR
Multi-value / round-robin	Several healthy records returned	Cheap client-side load spreading

Limits: the authoritative server sees the **recursive resolver's** address, not the user's. If a user in Berlin uses a resolver in the US, GeoDNS thinks they are in the US. **EDNS Client Subnet (ECS, RFC 7871)** passes a truncated client prefix upstream to fix this, at a privacy and cache-efficiency cost; some public resolvers do not send it. GeoDNS also cannot steer faster than TTLs expire, so pair it with short TTLs and health checks, or use anycast or a global load balancer (topic 6 §12.6) for fast failover.

DNSSEC essentials

- Each zone has a **ZSK** (zone-signing key) that signs every record set, producing an `RRSIG`, and a **KSK** (key-signing key) that signs the `DNSKEY` set. Public keys are published as `DNSKEY` records.
- The parent zone holds a **DS** (delegation signer) record, a hash of the child's KSK. This is the link in the chain of trust, and it is created *at your registrar*, not in your zone.
- **NSEC / NSEC3** records provide authenticated denial of existence ("this name really does not exist").
- Signatures have **inception and expiration times** (a few weeks is typical) and must be re-signed before they expire. Modern servers (BIND `dnssec-policy`, Knot, PowerDNS, managed providers) automate signing and key rollover.
- A validating resolver sets the **AD** (Authenticated Data) flag on good answers and returns `SERVFAIL` for bogus ones. `dig +cd` (checking disabled) shows what the resolver would have returned without validation.
- DNSSEC protects *integrity*, not confidentiality. It does not encrypt (that is DoT/DoH), and it adds larger responses and more operational risk, which is why adoption varies by organisation.

DNS failures at scale

Failure	Mechanism	Mitigation
Single DNS provider down or DDoSed	All NS records point to one operator	Two independent providers with synchronised zones
Long TTL on a bad record	Resolvers legitimately keep serving the old data	Lower TTL *before* planned changes; keep default TTLs moderate (60-300 s for dynamic records)
Negative caching	`NXDOMAIN` is cached for the SOA minimum	Avoid creating a name right after clients probed it; keep the SOA negative TTL short
Retry storms	Clients and resolvers retry on timeout, multiplying load on struggling servers	Capacity headroom, response rate limiting, jittered backoff (topic 14 §12.14)
Resolver overload in-cluster	Many pods, `ndots:5`, search domains: each lookup becomes several queries	NodeLocal DNSCache, tune `ndots`, use trailing dots on external names
UDP truncation, MTU black holes	Large (DNSSEC) responses fragment or get dropped	EDNS buffer size around 1232 bytes, TCP fallback works
Expired DNSSEC signatures or stale DS	Validators return SERVFAIL	Monitor signature expiry; automate rollovers; coordinate DS changes with the registrar
Circular dependency	The DNS servers' own address or the config system depends on DNS	Hard-code critical IPs, out-of-band access (BGP topic §12.2)
Bad automation	A script deletes or empties a record set	Review and staged rollout for DNS changes, backups of zone data, delete protection

6 Step-by-step guide

You will build a tiny DNS system on one Linux machine using loopback addresses: an authoritative **BIND** server with split-horizon and geo-style views, then DNSSEC signing, then a validating **Unbound** resolver that catches a forged answer. No Internet traffic is used and all names are fake (`example.com` is reserved for documentation, and you will serve it only to yourself).

Prerequisites: Ubuntu/Debian with root (`sudo`), about 15 minutes, free. Verified with BIND 9.18 and dig from `dnsutils`.

1. **Install and create loopback "client" addresses.** We pretend 127.0.0.3 is an internal host, 127.0.0.4 is a European user and 127.0.0.1 is everyone else. BIND only listens on addresses

that exist on an interface.

```bash
sudo apt-get update && sudo apt-get install -y bind9 dnsutils unbound
sudo systemctl disable --now named unbound 2>/dev/null || true
for ip in 2 3 4; do sudo ip addr add 127.0.0.$ip/32 dev lo; done
mkdir -p ~/dns-lab/zones ~/dns-lab/keys && cd ~/dns-lab
```

2. **Write three versions of the zone.** Same names, different answers per audience.

```bash
mk() { # $1=file  $2=www  $3=api  $4=db
cat > zones/$1 <<EOF
\$TTL 300
@   IN SOA ns1.example.com. admin.example.com. ( 1 3600 600 86400 300 )
    IN NS  ns1.example.com.
ns1 IN A   127.0.0.2
www IN A   $2
api IN A   $3
db  IN A   $4
EOF
}
mk example.com.zone          203.0.113.10  203.0.113.20 203.0.113.30   # public
mk example.com.internal.zone 10.0.1.10     10.0.1.20    10.0.2.30      # private
mk example.com.eu.zone       198.51.100.10 198.51.100.20 203.0.113.30  # Europe region
```

Paste each block exactly as shown (the closing `EOF` must start at column 0 in your terminal).

3. **Configure BIND with views.** Order matters: the first view whose `match-clients` matches wins.

```bash
cat > named.conf <<EOF
options {
  directory "$PWD";
  listen-on port 5300 { 127.0.0.2; };
  listen-on-v6 { none; };
  pid-file "$PWD/named.pid";
  recursion no;               # authoritative only
  dnssec-validation no;
};
acl eu_clients { 127.0.0.4; };
view "internal" { match-clients { 127.0.0.3; };
  zone "example.com" { type primary; file "zones/example.com.internal.zone"; }; };
view "eu" { match-clients { eu_clients; };
  zone "example.com" { type primary; file "zones/example.com.eu.zone"; }; };
view "default" { match-clients { any; };
  zone "example.com" { type primary; file "zones/example.com.zone"; }; };
```

```
EOF
named-checkconf named.conf && echo config OK
sudo named -c $PWD/named.conf -u root     # daemonises; use "-g" in another terminal
  to watch the log
```

4. **Ask the same question from three "places".** `-b` chooses the source address.

```bash
for src in 127.0.0.3 127.0.0.4 127.0.0.1; do
  printf '%s → ' $src; dig @127.0.0.2 -p 5300 -b $src www.example.com +short
done
```

```
127.0.0.3 → 10.0.1.10        # internal horizon
127.0.0.4 → 198.51.100.10    # "Europe" GeoDNS answer
127.0.0.1 → 203.0.113.10     # everyone else
```

This is exactly what Route 53 private hosted zones and geolocation records do, with the match based on VPC association or resolver location instead of a source IP.

5. **Switch to a signed zone.** Stop BIND and use one zone with `dnssec-policy default` (ECDSA P-256 keys, automatic signing and rollover). `inline-signing` keeps your plain zone file untouched.

```bash
sudo pkill named; sleep 1
cat > signed.conf <<EOF
options { directory "$PWD"; listen-on port 5300 { 127.0.0.2; }; listen-on-v6 { none;
  };
          pid-file "$PWD/signed.pid"; recursion no; dnssec-validation no; };
zone "example.com" { type primary; file "zones/example.com.zone"; key-directory
  "keys";
                     inline-signing yes; dnssec-policy default; };
EOF
sudo named -c $PWD/signed.conf -u root; sleep 3
ls keys
dig @127.0.0.2 -p 5300 www.example.com +dnssec +noall +answer
```

```
Kexample.com.+013+60187.key  Kexample.com.+013+60187.private
  Kexample.com.+013+60187.state
www.example.com.  300 IN A      203.0.113.10
www.example.com.  300 IN RRSIG  A 13 3 300 20261023072821 20261009112931 60187
  example.com. QR9DRJp...
```

Your key tag (`60187`) and dates will differ. The `RRSIG` fields are: type covered, algorithm 13 (ECDSA P-256), labels, original TTL, **expiration**, **inception**, key tag, signer, signature. With the default policy the key is a combined signing key; in a real deployment you then publish its DS at your registrar:

```bash
dnssec-dsfromkey -2 keys/Kexample.com.+013+*.key
# example.com. IN DS 60187 13 2 486E8BCD...   (give this to your registrar)
```

6. **Validate with Unbound.** Real resolvers start from the root's trust anchor; here we install your zone's public key as a local trust anchor and point a stub zone at your server.

```bash
KEY=$(dig @127.0.0.2 -p 5300 example.com DNSKEY +short | head -1)
cat > unbound.conf <<EOF
server:
  interface: 127.0.0.1
  port: 5301
  do-ip6: no
  access-control: 127.0.0.0/8 allow
  do-not-query-localhost: no
  chroot: ""
  username: ""
  pidfile: "$PWD/unbound.pid"
  trust-anchor: "example.com. DNSKEY $KEY"
stub-zone:
  name: "example.com"
  stub-addr: 127.0.0.2@5300
EOF
unbound-checkconf unbound.conf
unbound -c $PWD/unbound.conf; sleep 2
dig @127.0.0.1 -p 5301 www.example.com | grep -E 'status|flags'
```

```
;; ⟶»HEADER«⟵ opcode: QUERY, status: NOERROR, id: 51796
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
```

The `ad` flag means the resolver cryptographically verified the answer.

7. **Forge an answer and watch validation reject it.** Replace the signed server with an unsigned one that returns an attacker's address for the same zone, simulating a spoofed or

hijacked authoritative server.

```bash
sudo pkill named; sleep 1
sed 's/203.0.113.20/192.0.2.66/' zones/example.com.zone > zones/forged.zone
cat > forged.conf <<EOF
options { directory "$PWD"; listen-on port 5300 { 127.0.0.2; }; listen-on-v6 { none;
  };
         pid-file "$PWD/forged.pid"; recursion no; dnssec-validation no; };
zone "example.com" { type primary; file "zones/forged.zone"; };
EOF
sudo named -c $PWD/forged.conf -u root; sleep 3
dig @127.0.0.1 -p 5301 api.example.com | grep -E 'status|flags'          # validating:
  refuses
dig @127.0.0.1 -p 5301 api.example.com +cd +short                         # checking
  disabled: forged IP appears
```

```
;; ⟶»HEADER«⟵ opcode: QUERY, status: SERVFAIL, id: 15286
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1
192.0.2.66
```

`SERVFAIL` instead of a wrong IP is DNSSEC doing its job. It is also what *your* users see if your signatures expire or your DS is stale, which is why monitoring matters.

8. **Clean up.**

```bash
sudo pkill named; pkill unbound
for ip in 2 3 4; do sudo ip addr del 127.0.0.$ip/32 dev lo; done
cd ~ && rm -r ~/dns-lab
```

7 Real-world scenario

(Illustrative.) `shop-api` is moving from a single `us-east-1` deployment to two regions. The team creates a latency-based record set for `api.example.com` with a 60-second TTL and health checks on each region's load balancer, and a private hosted zone so that internal callers resolve `db.example.com` to private addresses. On cutover day, p99 for European users improves, but a partner behind a corporate resolver 300 ms from them still lands in `us-east-1`, because their resolver does not send EDNS Client Subnet. Later that month a Terraform change accidentally deletes the health check on the Frankfurt endpoint; when Frankfurt degrades, DNS keeps sending traffic there until the TTL-bound failover is fixed by hand. Investigation: `dig +short api.example.com @<resolver>` from several vantage points, comparing answers; `dig +trace` to confirm the authoritative answer; checking health check status in the console. Fix: restore health checks as code with delete protection, document

that latency routing is resolver-based, and add synthetic probes from several regions.
Lesson: DNS-based steering is coarse and TTL-bound. Treat its records and health checks like production code, and use an anycast or global load balancer in front when you need sub-minute failover.

8 Common mistakes and best practices

Don't

- Point all NS records at a single provider or a single anycast network.
- Combine authoritative and recursive service on a public-facing server, or leave a recursive resolver open to the world.
- Leave 24-hour TTLs on records you may need to change in an emergency.
- Enable DNSSEC by hand without automation, monitoring of signature expiry, or a tested process for DS updates at the registrar.
- Expose private zone contents through a too-broad split-horizon match rule.
- Assume GeoDNS knows where the *user* is.

Do

- Use two independent authoritative providers (or at least separate failure domains) and test failover.
- Lower the TTL a day before a planned migration, and raise it again afterward.
- Manage DNS as code (Terraform, OctoDNS or similar) with review, CI checks and delete protection (Level 7 §7.1).
- Monitor from outside: resolve your key names from several networks and alert on `SERVFAIL`, latency and DNSSEC signature lifetime.
- Run a local caching resolver (NodeLocal DNSCache in Kubernetes) and tune `ndots` for external lookups.
- Document which horizon a given engineer is querying when debugging.

9 Tasks

- [ ] **T1 — Build the three-horizon authoritative server** *(≈25 min, intermediate)*
  - **Goal:** serve different answers for the same name to internal, "EU" and default clients.
  - **Steps:**
    1. Follow steps 1-4 of the guide.
    2. Run the three `dig -b` queries for `www`, `api` and `db`.

- **Done when:** the table of nine answers matches the zone files, and `db.example.com` returns a private address only to 127.0.0.3.

- [ ] **T2 — Add a fourth horizon** *(≈20 min, intermediate)*
  - **Goal:** extend views safely.
  - **Steps:**
    1. Add `127.0.0.5` as a loopback address and a new view `apac` returning `192.0.2.10` for `www`, placed *before* the `default` view.
    2. Run `named-checkconf`, restart, query from `127.0.0.5` and from `127.0.0.1`.
    3. Move the `apac` view *after* `default` and re-test.
  - **Done when:** you can show that view order changes the answer and explain why (first match wins).

- [ ] **T3 — Sign the zone and inspect the chain** *(≈25 min, advanced)*
  - **Goal:** produce and read DNSSEC data.
  - **Steps:**
    1. Follow step 5; fetch `DNSKEY`, `RRSIG` and compute the `DS` with `dnssec-dsfromkey`.
    2. Convert the RRSIG expiration timestamp (`YYYYMMDDHHMMSS`) to a date and state how many days of validity remain.
  - **Done when:** you can state which record the registrar needs and why signature expiry would cause `SERVFAIL`.

- [ ] **T4 — Break and fix validation** *(≈25 min, advanced)*
  - **Goal:** reproduce a bogus-answer outage and recover.
  - **Steps:**
    1. Follow steps 6 and 7 to get `SERVFAIL` through Unbound.
    2. Restart the signed server and flush the resolver (`unbound-control` is disabled by default; simply restart Unbound).
    3. Re-query and confirm `ad` returns.
  - **Done when:** you have output showing `SERVFAIL` then `NOERROR` with `ad`, and can explain what `+cd` showed.

- [ ] **T5 — Measure TTL and cache behaviour** *(≈20 min, intermediate)*
  - **Goal:** see caching hide a failure.
  - **Steps:**
    1. With the signed server up, query Unbound for `www.example.com` and note the TTL.

2. Stop BIND and query again every 30 seconds; watch the TTL count down and the answer persist.
3. After the TTL expires, observe `SERVFAIL`.

- **Done when:** you can state, with timestamps, how long the cache masked the outage.

☐ **T6 — Prove it: design DNS for `shop-api`** *(≈45 min, advanced)*

- **Goal:** explain a resilient DNS architecture without notes.
- **Steps:**
  1. Without the guide, redo T1 and T4 from scratch.
  2. Write one page: two authoritative providers, TTL policy per record type, private vs public horizon, DNSSEC rollout plan with monitoring, and how you would detect and roll back a bad change within 5 minutes.
- **Done when:** the lab works from memory and a colleague can find no single point of failure in your page.

10 Self-check questions

1. Why is anycast a good fit for DNS over UDP, and how does a site "fail over"?
2. A user in Berlin gets the US answer from your GeoDNS. Name two plausible causes.
3. What is the difference between `RRSIG`, `DNSKEY` and `DS`, and which one lives in the parent zone?
4. Your DNSSEC signatures expired overnight. What do validating and non-validating users see?
5. You must move `api.example.com` to a new load balancer with minimal downtime. What do you do with the TTL and when?
6. Why do many teams use two DNS providers after the Dyn incident?

ANSWERS

1. A query and answer are single small packets, so a routing change mid-flow almost never breaks anything; each site announces the same prefix and a health agent withdraws it when the DNS daemon fails, so BGP moves traffic to the next-nearest site.
2. They use a resolver located in the US (public or corporate) that does not send EDNS Client Subnet, or the GeoIP database mis-locates the resolver address; also cached answers from an earlier location.
3. `DNSKEY` is a zone's public key, `RRSIG` is the signature over a record set, and `DS` is a hash of the child's KSK published in the **parent** zone to link the chain of trust.

4. Validating resolvers return `SERVFAIL` (outage for those users); non-validating resolvers keep resolving normally, so the problem is partial and hard to notice without validating probes.
5. Lower the TTL (for example to 60 s) at least one old-TTL period before the change, make the switch, verify, then raise the TTL again.
6. A single provider is a single point of failure for the whole name; with two independent providers serving the same zone, resolvers can still reach the other provider if one is under attack or down.

11 Further reading

- RFC 1034 and RFC 1035 (DNS concepts and implementation); RFC 4033-4035 (DNSSEC); RFC 7871 (EDNS Client Subnet); RFC 9199 (considerations for large authoritative DNS operators).
- BIND 9 Administrator Reference Manual (views, `dnssec-policy`) and the Unbound documentation, both on isc.org and nlnetlabs.nl.
- Amazon Route 53 Developer Guide (routing policies, private hosted zones).
- Cloudflare Learning Center articles on DNS, anycast and DNSSEC; the AWS post-event summaries archive; the root-servers.org site.
- Cricket Liu and Paul Albitz, *DNS and BIND*.

LAB NOTES What I ran, what I saw, what it means

12.4 TLS

Topic 4 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

Transport Layer Security (TLS) is the protocol that gives a TCP (or, with QUIC, UDP) connection three properties: **confidentiality** (nobody on the path can read it), **integrity** (nobody can alter it undetected) and **authentication** (you are really talking to `shop.example.com`, not an impostor). Level 2 §2.6 walked through one handshake. This topic is about running TLS in production: where trust comes from, how certificate chains are built and validated, how certificates are issued and renewed automatically, which cipher suites to allow, and how to rotate everything without an outage.

The trust system behind it is a **PKI (Public Key Infrastructure)**: a **Certificate Authority (CA)** signs a certificate that binds a name (or other identity) to a public key. Your client already trusts a small set of **root CAs** (shipped with the OS, browser or container image). A server presents its **leaf certificate** plus the **intermediate certificates** linking it to a trusted root; the client verifies each signature up the **chain**, checks names, dates and usage, and only then trusts the key.

2 Why it matters in DevOps

- **Expired certificates are one of the most common self-inflicted outages.** Nothing is wrong with your code; every client simply refuses to connect. They happen at 3 a.m. because someone's calendar reminder was the only automation.
- **Chain mistakes are invisible from your laptop.** Browsers cache intermediates and can fetch missing ones; `curl`, Java, Python, mobile apps and other servers cannot. "Works in Chrome, fails in the webhook client" is almost always an incomplete chain.
- **Certificate lifetimes are shrinking.** The CA/Browser Forum has approved a schedule that cuts maximum public certificate validity to 200 days (from March 2026), 100 days (March 2027) and 47 days (March 2029). Manual renewal stops being viable; automation (ACME) is mandatory.

- **Security posture and compliance.** Weak protocols (TLS 1.0/1.1, SSLv3), weak ciphers, missing forward secrecy or an exposed private key lead to audit failures and real compromise.
- **Everything else in this level depends on it:** TLS termination §12.11, mTLS §12.12, service meshes and HTTP/3 all reuse these concepts.

3 Real-world system example

Let's Encrypt (run by the nonprofit ISRG, Internet Security Research Group) issues free certificates automatically over the **ACME** protocol (Automatic Certificate Management Environment, RFC 8555). Its certificates have been 90 days by default, with clients such as `certbot`, `acme.sh`, Caddy, Traefik and cert-manager (Kubernetes) renewing roughly every 60 days without human involvement. It publishes a **staging** environment with generous rate limits for testing. Let's Encrypt has also been moving towards shorter-lived certificates and away from OCSP towards CRLs. On 30 September 2021 the old "DST Root CA X3" cross-sign expired. Modern clients were fine because Let's Encrypt's own root, **ISRG Root X1**, was already trusted, but older devices and some OpenSSL 1.0.x-based software that lacked ISRG Root X1 or mishandled the alternate chain failed with certificate errors. It is a textbook case of "your trust store is part of your system".

Certificate Transparency (CT). Public CAs must log the certificates they issue in public, append-only CT logs, and browsers check for proof of logging. Anyone (including you) can monitor logs such as crt.sh for certificates issued for your domains, which detects misissuance.

Cloud providers. AWS Certificate Manager (ACM) issues and automatically renews public certificates for use on ALB, CloudFront and API Gateway; the private key never leaves the service. Google Cloud and Azure offer similar managed certificates. Cloudflare terminates TLS at its edge for millions of sites and offers modern TLS defaults (TLS 1.3, HTTP/3). **Kubernetes** uses TLS everywhere internally (API server, etcd, kubelet) and the **cert-manager** project automates issuance and renewal for Ingress resources.

4 Diagram

The chain a client builds and validates (the server sends the leaf and intermediate; the root comes from the client's trust store):

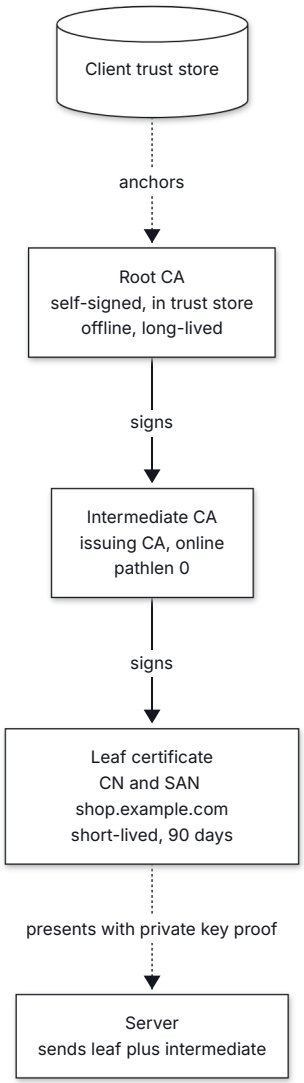


Figure 12.4-1

TLS 1.3 handshake and the point where automation (ACME) fits in:

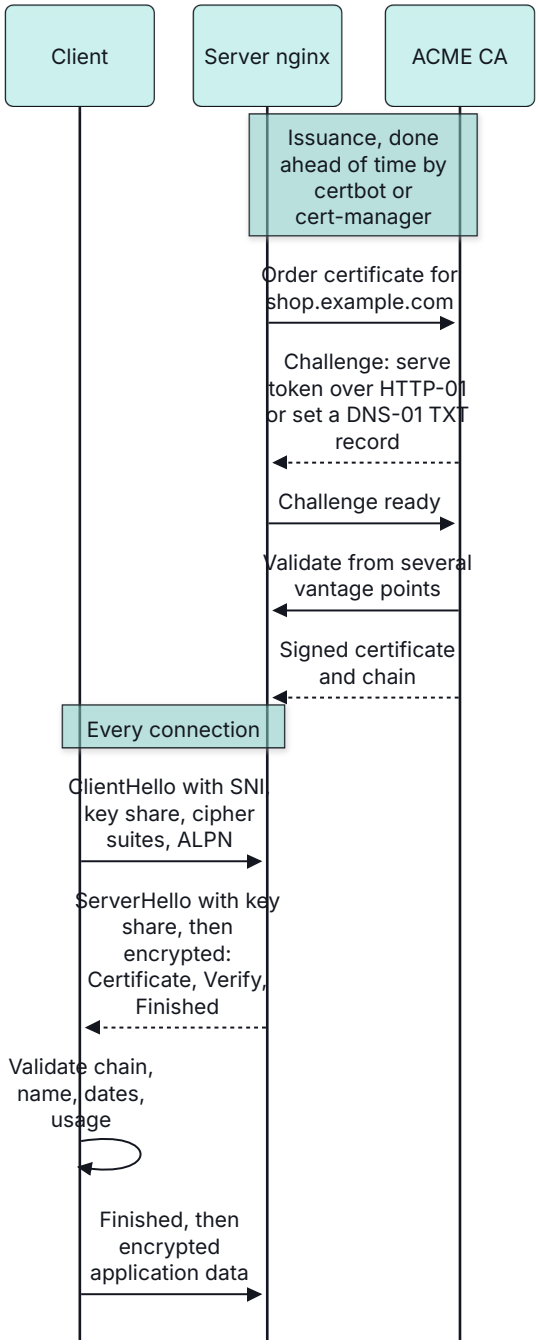


Figure 12.4-2

5 How it works

What a certificate contains (X.509)

Field	Why it matters
Subject / SAN (Subject Alternative Name)	The DNS names or IPs the certificate is valid for. Clients check the **SAN**; the old Common Name (CN) is ignored by modern clients
Issuer and **signature**	Who vouched for it and the math that proves it
Validity (`notBefore` , `notAfter`)	Hard time window; clock skew breaks it
Public key	RSA (2048+) or ECDSA (P-256 is common); Ed25519 is not yet widely used for public web certs
Basic Constraints	`CA:TRUE` for CAs, `CA:FALSE` for leaves; `pathlen` limits how many CAs may follow
Key Usage / EKU	What the key may do: `serverAuth` , `clientAuth` (needed for mTLS §12.12), `digitalSignature` , `keyCertSign`
AIA / CRL / OCSP / SCT	Where to fetch the issuer, revocation info, and Certificate Transparency proofs

Chain validation, in order

1. Build a path from the leaf up to a root in the client's trust store using the intermediates the server sent (and AKI/SKI identifiers).
2. Verify each signature, validity period, `CA:TRUE` and `pathlen` for every CA, and that the leaf's EKU allows the intended use.
3. Match the requested hostname against the SAN list (wildcards `*.example.com` match exactly one label).
4. Optionally check revocation (CRL/OCSP) or a stapled OCSP response; many clients "soft-fail", which is why short lifetimes are the real mitigation.

The server must send **leaf + intermediates** (never the root, never the leaf alone). nginx wants them concatenated in one file (`fullchain.pem`).

ACME: automated issuance

ACME proves you control a name before the CA signs anything. Challenges:

Challenge	You prove control by	Good for	Limits
HTTP-01	Serving a token at `http://name/.well-known/acme-challenge/<token>` on port 80	Single public web servers	Needs port 80 reachable; no wildcards

Challenge	You prove control by	Good for	Limits
DNS-01	Creating a `_acme-challenge.name` TXT record	Wildcards, internal services, load-balanced fleets	Needs DNS API credentials (guard them, scope them)
TLS-ALPN-01	Answering a special TLS handshake on port 443	Environments where port 80 is blocked	Fewer clients support it

Typical clients: `certbot` (from the EFF), `acme.sh`, `lego`, Caddy (built in), cert-manager (Kubernetes). Always test with the CA's **staging** endpoint first to avoid rate limits.

Cipher suites and protocol versions

- **TLS 1.3** (RFC 8446) has only five AEAD suites, three of which are in common use: `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`. Key exchange is always ephemeral (ECDHE) so you get **forward secrecy**: stealing the server key later does not decrypt recorded traffic. There is nothing to tune beyond the list.
- **TLS 1.2** needs care. A modern suite name reads `ECDHE-ECDSA-AES256-GCM-SHA384`: key exchange (ECDHE gives forward secrecy), certificate key type (ECDSA or RSA), bulk cipher (AES-GCM or ChaCha20-Poly1305) and hash. Allow only ECDHE + AEAD suites. Remove RSA key exchange, CBC modes, 3DES, RC4 and export ciphers.
- **TLS 1.0 and 1.1 are deprecated (RFC 8996)**; disable them. SSLv3 is long dead.
- Use a vetted starting point such as the Mozilla SSL Configuration Generator ("intermediate" for general web, "modern" if you control all clients) rather than inventing a list.
- Other knobs: **ALPN** (selects `h2` / `http/1.1`, essential for HTTP/2 and HTTP/3 §12.5), **OCSP stapling**, **HSTS** (tells browsers to only use HTTPS), **session resumption/tickets** (fewer full handshakes).
- **SNI (Server Name Indication)** is sent in plaintext in the ClientHello, which is why one IP can host many certificates, and why on-path observers can see the hostname (Encrypted Client Hello is the emerging fix).

Rotation

A rotation plan answers: *what renews it, how does the new cert reach the process, and how do we know it worked?*

Layer	Practice
Leaf certs	Automate with ACME, renew at 2/3 of lifetime, reload (not restart) the server; monitor expiry in days
Private keys	Generate on the server, never in tickets or Git. Consider a fresh key per renewal; keep file mode `0600`

Layer	Practice
Intermediates	CAs rotate them; always serve the chain your CA gives you at issuance time
Roots	Hard to rotate: depends on client trust stores. Pin to nothing unless you must, and if you pin pin the intermediate or use a backup pin
Internal CAs	Short-lived leaves (hours to days) plus an offline root; see mTLS and SPIFFE §12.12
Alerting	Probe from outside and alert at 21, 14 and 7 days before expiry; the *monitor* must not depend on the renewal job it is checking

6 Step-by-step guide

You will build a two-level private PKI with OpenSSL, serve it with nginx, inspect what the client sees, break the chain on purpose, restrict protocols and ciphers, and rotate the certificate with a zero-downtime reload. Everything runs on loopback (`127.0.0.1:8443`), free, and was verified with OpenSSL 3.0, nginx 1.24 and curl 8.5.

Prerequisites: Linux (or WSL2/macOS with OpenSSL 3), `openssl`, `curl`, `nginx`, `tcpdump` (root for the capture). `sudo apt-get install -y nginx openssl curl tcpdump` on Ubuntu; stop the packaged service so it does not hold port 80: `sudo systemctl disable --now nginx`. If your shell sets `https_proxy`, run `unset https_proxy HTTPS_PROXY` first or `curl` will try to send loopback traffic through the proxy.

1. **Create a root CA and an intermediate.** The root signs only the intermediate and in real life would then be taken offline.

```bash
mkdir -p ~/tls-lab/pki && cd ~/tls-lab/pki
# Root CA: self-signed, 10 years
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
  -keyout root.key -out root.crt -days 3650 -subj "/O=Example Lab/CN=Example Lab Root
  CA" \
  -addext "basicConstraints=critical,CA:TRUE" -addext
  "keyUsage=critical,keyCertSign,cRLSign"
# Intermediate: CSR then sign with the root; pathlen:0 means it may only sign leaves
openssl req -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout inter.key -out
  inter.csr \
  -subj "/O=Example Lab/CN=Example Lab Issuing CA 1"
printf
  'basicConstraints=critical,CA:TRUE,pathlen:0\nkeyUsage=critical,keyCertSign,cRLSign\
  nsubjectKeyIdentifier=hash\nauthorityKeyIdentifier=keyid\n' > inter.ext
openssl x509 -req -in inter.csr -CA root.crt -CAkey root.key -CAcreateserial -out
  inter.crt -days 1825 -extfile inter.ext
```

Expected: `Certificate request self-signature ok` and `subject=O = Example Lab, CN = Example Lab Issuing CA 1`.

2. **Issue a leaf certificate** for `shop.example.com` (and `api.example.com`) valid for 90 days, good for server authentication only.

```bash
openssl req -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout leaf.key -out leaf.csr -subj "/CN=shop.example.com"
printf 'basicConstraints=critical,CA:FALSE\nkeyUsage=critical,digitalSignature\nextendedKeyUsage=serverAuth\nsubjectAltName=DNS:shop.example.com,DNS:api.example.com\nauthorityKeyIdentifier=keyid\n' > leaf.ext
openssl x509 -req -in leaf.csr -CA inter.crt -CAkey inter.key -CAcreateserial -out leaf.crt -days 90 -extfile leaf.ext
cat leaf.crt inter.crt > fullchain.pem       # leaf first, then intermediate; never the root
openssl x509 -in leaf.crt -noout -subject -issuer -dates -ext subjectAltName
```

```
subject=CN = shop.example.com
issuer=O = Example Lab, CN = Example Lab Issuing CA 1
notBefore=Oct  9 12:32:24 2026 GMT
notAfter=Jan  7 12:32:24 2027 GMT
X509v3 Subject Alternative Name:
    DNS:shop.example.com, DNS:api.example.com
```

3. **Verify the chain the way a client would.** Verification of the leaf alone fails because the issuer is unknown; adding the intermediate fixes it.

```bash
openssl verify -CAfile root.crt leaf.crt                      # error 20: unable to get local issuer certificate
openssl verify -CAfile root.crt -untrusted inter.crt leaf.crt  # leaf.crt: OK
```

4. **Serve it with nginx on port 8443.** Use a private config so you do not touch the system's nginx.

```bash
cd ~/tls-lab && mkdir -p nginx/logs
cat > nginx/nginx.conf <<EOF
worker_processes 1;
daemon off;
error_log $PWD/nginx/logs/error.log;
pid $PWD/nginx/nginx.pid;
events {}
http {
  access_log off;
```

```
  server {
    listen 127.0.0.1:8443 ssl;
    server_name shop.example.com;
    ssl_certificate     $PWD/pki/fullchain.pem;
    ssl_certificate_key $PWD/pki/leaf.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    location / { return 200 "hello from shop.example.com\n"; }
  }
}
EOF
nginx -t -e $PWD/nginx/logs/error.log -c $PWD/nginx/nginx.conf   # syntax is ok / test
  is successful
nginx -e $PWD/nginx/logs/error.log -c $PWD/nginx/nginx.conf &     # runs in the
  background
sleep 1
R="--resolve shop.example.com:8443:127.0.0.1"   # map the name to loopback without
  editing /etc/hosts
curl -sS $R --cacert pki/root.crt https://shop.example.com:8443/
curl -sS $R https://shop.example.com:8443/ ; echo "exit=$?"
```

Expected: `hello from shop.example.com` with your private root as trust anchor, and `curl: (60) SSL certificate problem: unable to get local issuer certificate` without it (the system store does not know your CA, which is exactly what should happen).

5. **Inspect what the client negotiated.**

```bash
curl -sv $R --cacert pki/root.crt https://shop.example.com:8443/ 2>&1 | grep -E 'SSL
  connection|subject:|issuer:|expire date|ALPN'
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -CAfile
  pki/root.crt 2>/dev/null | sed -n '/Certificate chain/,/Server certificate/p'
```

Look for `TLSv1.3 / TLS_AES_256_GCM_SHA384`, `0 s:CN = shop.example.com i:...Issuing CA 1` and `1 s:...Issuing CA 1 i:...Root CA`. The server sends two certificates; the root is not sent.

6. **Break the chain on purpose.** Serve the leaf alone, reload and compare the client error with step 4's.

```bash
sed -i 's#pki/fullchain.pem#pki/leaf.crt#' nginx/nginx.conf
kill -HUP $(cat nginx/nginx.pid); sleep 1
curl -sS $R --cacert pki/root.crt https://shop.example.com:8443/    # curl: (60) ...
  unable to get local issuer certificate
sed -i 's#pki/leaf.crt#pki/fullchain.pem#' nginx/nginx.conf
kill -HUP $(cat nginx/nginx.pid); sleep 1
curl -sS $R --cacert pki/root.crt https://shop.example.com:8443/    # works again
```

Same error text as an unknown CA, different root cause: this is why you test with a **clean** client (`curl`, `openssl s_client`, SSL Labs) and not only a browser.

7. **Restrict protocols and cipher suites.** Edit the `ssl_protocols` line, reload, and attempt old protocol versions.

```bash
sed -i 's/ssl_protocols TLSv1.2 TLSv1.3;/ssl_protocols TLSv1.3;/' nginx/nginx.conf
nginx -t -c $PWD/nginx/nginx.conf && kill -HUP $(cat nginx/nginx.pid); sleep 1
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -tls1_2
  2>&1 | grep -E 'alert'
```

```
...ssl3_read_bytes:tlsv1 alert protocol version...SSL alert number 70
```

Now allow TLS 1.2 again but offer only one TLS 1.2 suite (add `ssl_ciphers ECDHE-ECDSA-CHACHA20-POLY1305;` below `ssl_protocols`). A client that insists on `-cipher ECDHE-ECDSA-AES128-GCM-SHA256` gets `sslv3 alert handshake failure` (alert 40), while default clients negotiate `ECDHE-ECDSA-CHACHA20-POLY1305`. TLS 1.3 suites are not controlled by `ssl_ciphers` in nginx/OpenSSL.

8. **Watch what is visible on the wire.** The hostname (SNI) is sent in clear even with TLS 1.3; the certificate is encrypted in 1.3 but visible in 1.2.

```bash
sudo timeout 5 tcpdump -i lo -n -A 'tcp port 8443' > /tmp/cap.txt &
sleep 1; curl -s $R --cacert pki/root.crt https://shop.example.com:8443/ >/dev/null;
  sleep 5
grep -a -o 'shop.example.com' /tmp/cap.txt | head -1       # SNI in the ClientHello
```

Repeat after allowing TLS 1.2 and run `curl --tlsv1.2 --tls-max 1.2 ...`; then `grep -a -c 'Example Lab' /tmp/cap.txt` is non-zero because the certificate subject travels unencrypted in TLS 1.2.

9. **Rotate the certificate with zero downtime.** Issue a second leaf with a new key, switch the files, and reload. nginx starts new workers with the new certificate while old workers fin-

ish in-flight requests.

```bash
cd pki
openssl req -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout leaf2.key -out
  leaf2.csr -subj "/CN=shop.example.com"
openssl x509 -req -in leaf2.csr -CA inter.crt -CAkey inter.key -CAcreateserial -out
  leaf2.crt -days 30 -extfile leaf.ext
cat leaf2.crt inter.crt > fullchain2.pem; cd ..
get() { echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com
  2>/dev/null | openssl x509 -noout -serial -enddate; }
get
sed -i 's#fullchain.pem#fullchain2.pem#; s#leaf.key#leaf2.key#' nginx/nginx.conf
nginx -t -c $PWD/nginx/nginx.conf && kill -HUP $(cat nginx/nginx.pid); sleep 1
get
```

The serial number and `notAfter` change while the port never closed. If `nginx -t` fails, the reload is skipped and the old certificate keeps serving, which is the behaviour you want from an automated renewer (`certbot --deploy-hook "nginx -t && systemctl reload nginx"`).

10. **ACME on a real server (optional, free).** On a VPS with a DNS name pointing at it and port 80 open, use the staging CA first:

```bash
sudo apt-get install -y certbot python3-certbot-nginx
sudo certbot --nginx -d shop.example.com --staging --register-unsafely-without-email -
  -agree-tos
sudo certbot renew --dry-run                # proves the renewal path works
systemctl list-timers | grep certbot        # certbot's twice-daily renewal timer
```

Output is similar to `Congratulations! You have successfully enabled HTTPS`. Remove `--staging` to get a browser-trusted certificate. Use your own domain: `example.com` is reserved and cannot be validated.

11. **Clean up.**

```bash
kill $(cat ~/tls-lab/nginx/nginx.pid); rm -r ~/tls-lab /tmp/cap.txt
```

7 Real-world scenario

(Illustrative.) A partner reports that webhooks from `shop-api` have started failing with `certificate verify failed: unable to get local issuer certificate`. The site looks fine in a browser and the monitoring check (an HTTPS GET from the same host) is green.

Investigation: `openssl s_client -connect api.example.com:443 -servername api.example.com` shows a chain depth of one: only the leaf. Last week a deploy replaced `fullchain.pem` with `cert.pem` while moving certbot output paths in the Nginx role. Browsers repair the chain using the intermediate they cached or fetch via AIA; the partner's Java client cannot. Fix: point `ssl_certificate` back to `fullchain.pem`, reload, and add a CI check that fetches the served chain with `openssl s_client` and fails unless `openssl verify` succeeds with a clean trust store. Add an external probe that verifies the chain and alerts under 21 days to expiry. **Lesson:** test TLS from a client that does not "help" you, and treat certificate files and renewals as code.

8 Common mistakes and best practices

Don't

- Serve only the leaf certificate, or include the root in the chain.
- Rely on a calendar reminder to renew; or on a browser check to validate the chain.
- Commit private keys, or reuse one key across environments and years.
- Disable verification (`curl -k`, `verify=False`, `InsecureSkipVerify`) to "fix" an error; fix the trust.
- Keep TLS 1.0/1.1, RC4, 3DES or static-RSA key exchange for a legacy client you no longer have.
- Pin leaf certificates in mobile apps without a rotation plan.
- Forget that wildcard certificates share one key across many hosts: a leak is bigger.

Do

- Use ACME everywhere possible, with short lifetimes and a tested `renew --dry-run`.
- Monitor expiry and chain validity from outside the system, and alert days ahead.
- Start from the Mozilla intermediate or modern profile, enable HSTS only after HTTPS works everywhere, and review settings annually.
- Reload, do not restart; validate with `nginx -t` (or the equivalent) before reloading.
- Watch Certificate Transparency for unexpected certificates for your domains.
- Keep roots offline and intermediates scoped (`pathlen:0`) in internal PKIs; issue short-lived leaves.

9 Tasks

- [ ] **T1 — Build and inspect a two-level PKI** *(≈30 min, intermediate)*
  - **Goal:** create root, intermediate and leaf, and read their fields.

- **Steps:**
  1. Do steps 1-3 of the guide.
  2. Print each certificate with `openssl x509 -noout -text` and find `CA:TRUE`, `pathlen`, the SAN and EKU.
- **Done when:** `openssl verify -CAfile root.crt -untrusted inter.crt leaf.crt` prints `OK` and you can point to the field that makes the intermediate unable to sign another CA.

☐ **T2 — Serve and verify with a clean client** *(≈25 min, intermediate)*

- **Goal:** run nginx with the chain and see what a client receives.
- **Steps:**
  1. Do steps 4-5.
  2. Capture `openssl s_client -showcerts` output and count certificates in the chain.
- **Done when:** curl returns the page with `--cacert` and fails with exit code 60 without it, and you can explain both.

☐ **T3 — Break and fix the chain** *(≈20 min, advanced)*

- **Goal:** reproduce the "works in browser, fails in the API client" bug.
- **Steps:**
  1. Do step 6, then try `openssl s_client -connect ... -CAfile pki/root.crt` and note `Verify return code: 21`.
  2. Fix it and re-verify, then write the CI one-liner that would have caught it.
- **Done when:** the check script exits non-zero with the broken chain and zero with the good chain.

☐ **T4 — Harden protocols and ciphers** *(≈25 min, advanced)*

- **Goal:** enforce a policy and prove it.
- **Steps:**
  1. Set TLS 1.3 only, then TLS 1.2 + 1.3 with one AEAD suite (step 7).
  2. Test each with `openssl s_client -tls1_2`, `-tls1_3` and `-cipher ...`; record accepted vs rejected cases in a table.
- **Done when:** your table matches the alerts you observed (70 protocol version, 40 handshake failure) and the config passes `nginx -t`.

☐ **T5 — Zero-downtime rotation plus an expiry monitor** *(≈40 min, advanced)*

- **Goal:** automate rotation and detect near-expiry.
- **Steps:**

1. Run a loop `while true; do curl -s $R --cacert pki/root.crt https://shop.example.com:8443/ || echo FAIL; sleep 0.2; done` in a second terminal.
2. Rotate the certificate (step 9) while the loop runs.
3. Write a script that exits 1 if the served leaf expires in under 21 days (`openssl x509 -checkend $((21*86400))`), and test it with a 10-day certificate.

- **Done when:** the loop prints no `FAIL` across the rotation and your script flags the 10-day certificate.

☐ **T6 — Prove it: explain a certificate outage** *(≈45 min, advanced)*

- **Goal:** diagnose from scratch.
- **Steps:**
  1. Without the guide, rebuild the PKI and nginx, then have a friend (or a script) break one thing: expiry, wrong SAN, missing intermediate or wrong key.
  2. Using only `openssl s_client`, `openssl x509` and `curl -v`, identify which failure it is and fix it.
  3. Write a five-line runbook: symptoms, command, expected vs actual output, fix, prevention.
- **Done when:** you correctly diagnose all four failure types and your runbook fits in one screen.

10 Self-check questions

1. Why must a server send the intermediate certificate but not the root?
2. A site works in Chrome but not in `curl` or a Java client. What is the most likely cause and how do you confirm it?
3. What gives TLS 1.3 forward secrecy, and why does it matter if the server key is stolen next year?
4. Compare ACME HTTP-01 and DNS-01: when must you use DNS-01?
5. Which cipher-suite name components tell you whether a TLS 1.2 suite is acceptable today?
6. Your certificate expires in three days and renewal is failing. What do you do, in order?

ANSWERS

1. The client needs the intermediate to connect the leaf to a trusted root and cannot be expected to find it; the root is already in the client's trust store, and sending it adds no trust (a client that does not have it should not trust it).

2. An incomplete chain (leaf only). Browsers cache or fetch intermediates; other clients do not. Confirm with `openssl s_client -connect host:443 -servername host` and look at the number of certificates and `Verify return code`.
3. Ephemeral (EC)DH key exchange, whose per-connection secrets are discarded; a later key theft cannot decrypt recorded traffic. The server key is only used to sign the handshake.
4. HTTP-01 proves control via a file on port 80; DNS-01 proves it via a TXT record. Use DNS-01 for wildcard certificates, servers not reachable from the Internet, or fleets behind load balancers.
5. ECDHE (forward secrecy), AEAD cipher (GCM or ChaCha20-Poly1305) and not CBC/RC4/3DES/static RSA key exchange.
6. Check the logs and run the renewer manually or with a dry-run; fix the cause (DNS credentials, port 80 blocked, rate limit); if it cannot be fixed fast, issue a certificate manually through another route or use a cloud-managed certificate; deploy and reload; verify from outside; then add monitoring and a postmortem action.

11 Further reading

- RFC 8446 (TLS 1.3), RFC 5280 (X.509 and chain validation), RFC 8555 (ACME), RFC 8996 (deprecating TLS 1.0/1.1), RFC 6962 and RFC 9162 (Certificate Transparency).
- Let's Encrypt documentation (letsencrypt.org/docs), including staging environment and rate limits; certbot and cert-manager documentation.
- Mozilla SSL Configuration Generator and the Mozilla "Server Side TLS" guidelines.
- OpenSSL `openssl-x509`, `openssl-req`, `openssl-verify` and `openssl-s_client` manual pages.
- Ivan Ristić, *Bulletproof TLS and PKI*.

LAB NOTES What I ran, what I saw, what it means

12.5 HTTP/2, HTTP/3 and QUIC

Topic 5 of 14 | ~4 hours | Difficulty: Advanced

1 What it is

HTTP has three wire formats in use today, all carrying the same semantics (methods, status codes, headers, bodies from Level 2 §2.7):

- **HTTP/1.1** sends text requests one at a time per TCP connection. To get parallelism a browser opens about six connections per host.
- **HTTP/2 (RFC 9113)** keeps *one* TCP+TLS connection and splits it into binary **frames** belonging to numbered **streams**. Many requests and responses are **multiplexed** (interleaved) on the same connection, and headers are compressed with **HPACK**.
- **HTTP/3 (RFC 9114)** maps the same stream model onto **QUIC (RFC 9000)**, a transport that runs over **UDP**, has TLS 1.3 built into its handshake, and handles loss and ordering **per stream** instead of per connection.

The reason HTTP/3 exists is **head-of-line (HOL) blocking**. TCP promises one ordered byte stream, so if one packet is lost, every byte after it waits for the retransmission, even bytes belonging to unrelated HTTP/2 streams. HTTP/2 fixed HOL blocking at the HTTP layer but moved it into TCP; QUIC removes it by making streams independent at the transport.

2 Why it matters in DevOps

- **It changes your infrastructure assumptions.** HTTP/3 needs **UDP 443** open on firewalls, security groups and load balancers, and clients discover it only after a first HTTP/1.1 or HTTP/2 response (`Alt-Svc`). A forgotten UDP rule means "HTTP/3 enabled" that nobody ever uses.
- **Latency on mobile and lossy networks.** Fewer round trips to connect (1-RTT, or 0-RTT on resumption) and no cross-stream HOL blocking improve tail latency where packet loss and Wi-Fi/cellular handoff are common.
- **Capacity and load balancing.** One long-lived multiplexed HTTP/2 connection can pin lots of traffic onto one backend; gRPC (HTTP/2) needs L7 load balancing because L4 balancing sees a single connection (Load Balancing §12.6).

- **Observability and security tooling.** QUIC encrypts almost everything, including transport headers, so `tcpdump` shows opaque UDP, and middleboxes that depend on inspecting TCP fail or fall back. HTTP/2 introduced new attack surface (for example the 2023 Rapid Reset DDoS, section 3).
- **0-RTT is a security trade-off.** Resumption data can be **replayed**, so only idempotent requests belong in it.

3 Real-world system example

Google developed QUIC internally (called gQUIC) and deployed it in Chrome and its services before the IETF standardised an improved version as RFC 9000 (2021); HTTP/3 followed as RFC 9114 (2022). **Cloudflare**, **Fastly** and the large cloud CDNs offer HTTP/3 at their edge, and Meta publicly describes using QUIC (mvfst) for mobile app traffic. **AWS CloudFront** and **Google Cloud Load Balancing** support HTTP/3 to clients. On the server side, **nginx** (1.25+ built with QUIC), **HAProxy** (2.6+), **Envoy**, **Caddy** and **LiteSpeed** can terminate HTTP/3.

gRPC uses HTTP/2 as its transport: each RPC is a stream, metadata travels as headers and trailers, and the multiplexing is why one gRPC channel can carry thousands of concurrent calls. That is also why Kubernetes needs special handling (headless Services, mesh or L7 balancing) to spread gRPC across pods (Level 8, Services §8.4).

HTTP/2 Rapid Reset (CVE-2023-44487, October 2023). Google, Cloudflare and AWS jointly disclosed a DDoS technique that abuses HTTP/2 stream cancellation: the client opens a stream and immediately resets it (`RST_STREAM`), so it never hits the server's concurrent-stream limit yet still forces request setup work. It produced record-setting request rates against large providers and led to patches and new rate limits on stream resets in HTTP/2 servers and proxies. The takeaway: protocol features have limits, and the limits need monitoring.

4 Diagram

Where head-of-line blocking comes from, and how QUIC avoids it:

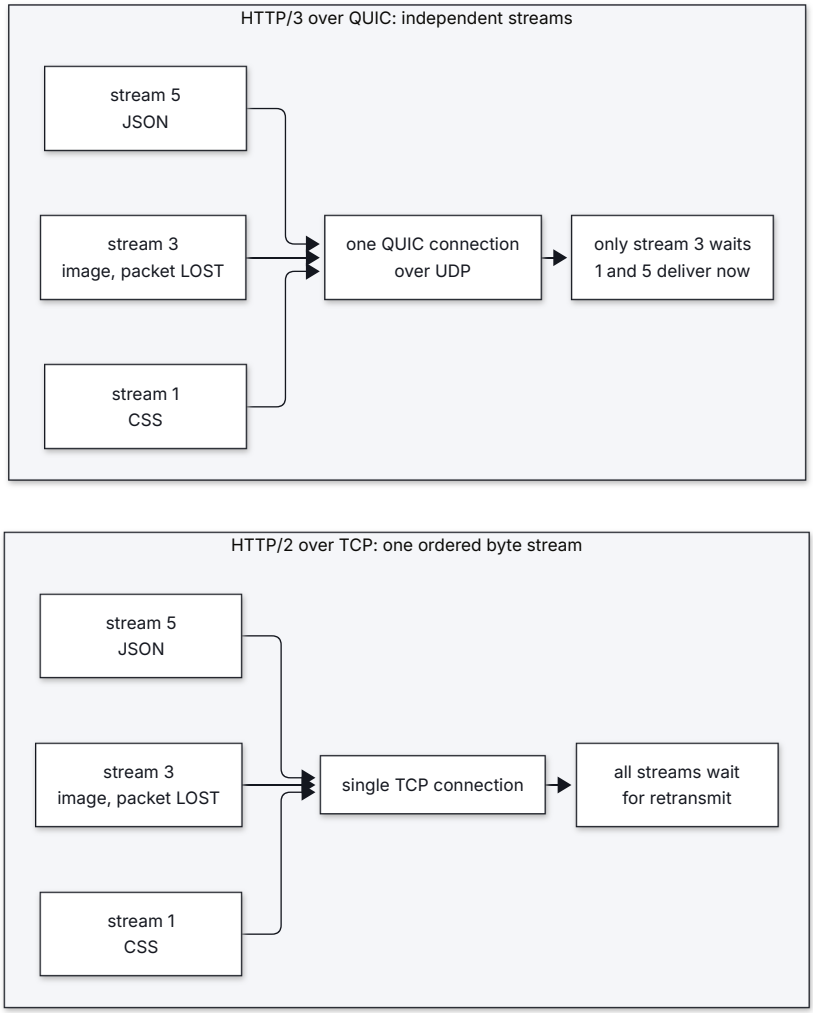


Figure 12.5-1

Connection setup compared. HTTP/2 needs a TCP handshake then a TLS handshake; QUIC

does transport and TLS 1.3 together; resumption can send data in the first flight (0-RTT):

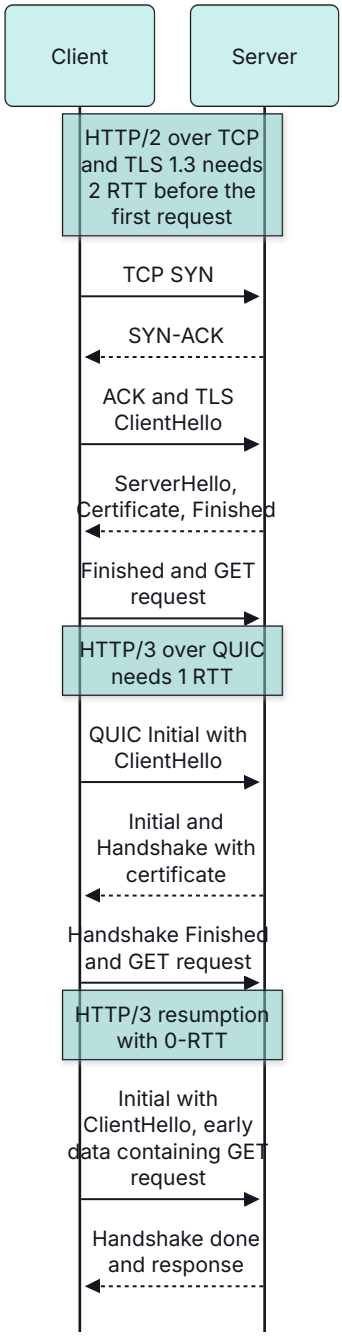


Figure 12.5-2

5 How it works

HTTP/1.1 vs HTTP/2 vs HTTP/3

	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC over UDP
Encryption	optional (TLS)	TLS in practice (h2 negotiated by ALPN)	always TLS 1.3, inside QUIC
Concurrency	~6 connections per host, one request each	many streams on one connection	many streams on one connection
Header compression	none	HPACK	QPACK (designed to avoid HOL blocking)
HOL blocking	per connection (pipelining unusable)	gone at HTTP layer, remains at TCP	removed (per-stream loss recovery)
Connection setup	TCP + TLS	TCP + TLS (2 RTT with TLS 1.3, 3 with 1.2)	1 RTT, 0-RTT on resumption
Connection migration	no	no	yes (connection IDs survive an IP change)
Discovery	URL scheme	ALPN `h2` during TLS	`Alt-Svc: h3=":443"` header or HTTPS DNS record, then ALPN `h3`

HTTP/2 in detail

- **Frames and streams.** The connection carries `HEADERS`, `DATA`, `SETTINGS`, `WINDOW_UPDATE`, `RST_STREAM`, `GOAWAY` and other frames. Each request/response is a stream (odd IDs for client-initiated). Frames from different streams interleave.
- **Flow control** is per stream and per connection (`WINDOW_UPDATE`) so a slow consumer does not buffer unbounded data. Servers cap `SETTINGS_MAX_CONCURRENT_STREAMS` (often 100-128).
- **HPACK** compresses repeated headers (cookies, user agent) against a dynamic table, saving many bytes per request.
- **What it does not fix:** TCP HOL blocking, slow start, and the single congestion window shared by all streams. On a clean network HTTP/2 is excellent; on a lossy one a single connection can be slower than six HTTP/1.1 connections.
- **Server push** was removed from browsers in practice; do not build on it. Use `103 Early Hints` or `<link rel=preload>`.
- **Operations:** `GOAWAY` is how a server asks a client to move to a new connection gracefully during deploys; keep-alive and idle timeouts matter for pool behaviour (Connection Pooling §12.10).

QUIC in detail

- **UDP-based but reliable.** QUIC implements packet numbers, acknowledgements, retransmission and congestion control (RFC 9002) in user space, so it can evolve without OS or middlebox upgrades. Packet payloads are encrypted, including most of the header, which stops middleboxes ossifying the protocol.
- **Streams are independent.** Loss of a packet carrying stream 3 stalls only stream 3.
- **Integrated TLS 1.3 (RFC 9001).** Transport parameters and TLS handshake complete together; there is no TCP handshake plus TLS handshake sequence.
- **Connection IDs** identify a connection instead of the 4-tuple, so a phone moving from Wi-Fi to cellular can keep the connection (**connection migration**), and load balancers can route by connection ID (QUIC-LB) rather than by address.
- **Anti-amplification:** a client's first packet is padded to at least 1200 bytes and a server may send only about three times what it has received until the client's address is validated.
- **Version negotiation and fallback.** UDP can be blocked or throttled by some networks. Browsers race or fall back to HTTP/2 over TCP, so HTTP/3 should be *additive*; never remove TCP 443.
- **Costs.** User-space crypto and per-packet UDP syscalls use more CPU than kernel TCP (UDP GSO/GRO and kernel offloads reduce this); some corporate networks drop UDP 443.

0-RTT and replay

On resumption, a TLS 1.3 client can send application data in its first flight, encrypted with a key derived from the previous session. It saves a round trip but has two weaknesses: no forward secrecy for that data, and **replay** (an attacker can capture the first flight and send it again, and a stateless server may process it twice). Rules: accept 0-RTT only for idempotent requests (`GET` for cacheable resources), pass an `Early-Data: 1` header to the application so it can reject risky requests with `425 Too Early` (RFC 8470), and use server-side anti-replay where available. `POST /orders` must never be processed from early data without an idempotency key (Level 13, Idempotency §13.14).

Deploying HTTP/3

1. Keep HTTP/1.1 and HTTP/2 on TCP 443 as the baseline.
2. Open **UDP 443** in security groups, firewalls and cloud load balancer listeners.
3. Advertise it: `Alt-Svc: h3=":443"; ma=86400` on HTTP/2 responses (and optionally an `HTTPS` DNS record).
4. Make sure each QUIC packet for a connection reaches the same backend: use consistent hashing on connection ID or a QUIC-aware load balancer.

5. Measure: client-side protocol mix, fallback rate, handshake time, errors by protocol.

6 Step-by-step guide

You will serve HTTP/2 with nginx, prove multiplexing from the server logs, serve HTTP/3 with HAProxy, call it with a small Python HTTP/3 client, look at QUIC packets with tcpdump, and finally demonstrate the 0-RTT replay problem. All of it runs on loopback and is free.

Prerequisites: Linux, `nginx` 1.18+, `haproxy` 2.8+ (Ubuntu 24.04's package has QUIC support), `curl` with HTTP/2 (nearly all builds), Python 3.10+, `openssl` 3, `tcpdump`, and the certificate from L12.4 TLS steps 1-2 in `~/tls-lab/pki` (`fullchain.pem`, `leaf.key`, `root.crt`; if you already cleaned it up, redo those two steps). Run `unset https_proxy HTTPS_PROXY` if your shell uses a proxy. Verified with nginx 1.24, HAProxy 2.8.16, curl 8.5, OpenSSL 3.0 and aioquic 1.3.

1. **Set up files and an HTTP/2 server.** The log format records the TCP connection number so you can see multiplexing.

```bash
mkdir -p ~/h2-lab/www ~/h2-lab/logs && cd ~/h2-lab
for i in $(seq 1 10); do echo "asset $i" > www/a$i.txt; done
cat > nginx.conf <<EOF
user $USER;
worker_processes 1;
daemon off;
error_log $PWD/logs/error.log;
pid $PWD/nginx.pid;
events {}
http {
  log_format h '\$remote_port conn=\$connection req_on_conn=\$connection_requests
  proto=\$server_protocol \$uri early=\$ssl_early_data';
  access_log $PWD/logs/access.log h;
  server {
    listen 127.0.0.1:8443 ssl http2;       # nginx 1.25.1+ prefers a separate "http2
  on;" line
    server_name shop.example.com;
    ssl_certificate     $HOME/tls-lab/pki/fullchain.pem;
    ssl_certificate_key $HOME/tls-lab/pki/leaf.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    root $PWD/www;
    add_header Alt-Svc 'h3=":9443"; ma=60' always;
  }
}
EOF
nginx -t -e $PWD/logs/error.log -c $PWD/nginx.conf
nginx -e $PWD/logs/error.log -c $PWD/nginx.conf &
sleep 1
R="--resolve shop.example.com:8443:127.0.0.1 --cacert $HOME/tls-lab/pki/root.crt"
```

```
curl -sv $R https://shop.example.com:8443/a1.txt 2>&1 | grep -E 'ALPN|using HTTP|<
  HTTP|alt-svc'
```

```
* ALPN: curl offers h2,http/1.1
* ALPN: server accepted h2
* using HTTP/2
< HTTP/2 200
< alt-svc: h3=":9443"; ma=60
```

`ALPN` (Application-Layer Protocol Negotiation) is how the TLS handshake selects `h2`. If nginx cannot read your certificate files as `$USER`, fix the permissions or run it with `sudo`.

2. **Prove multiplexing from the server's point of view.** Request 10 URLs in parallel with HTTP/2, then with HTTP/1.1, and look at the connection numbers.

```bash
: > logs/access.log
curl -s $R --parallel "https://shop.example.com:8443/a[1-10].txt" -o /dev/null
awk '{print $2}' logs/access.log | sort | uniq -c        # one connection carrying 10
  requests
: > logs/access.log
curl -s $R --http1.1 --parallel "https://shop.example.com:8443/a[1-10].txt" -o
  /dev/null
awk '{print $2}' logs/access.log | sort | uniq -c        # ten connections with one
  request each
```

```
     10 conn=2          # HTTP/2: same client port, req_on_conn counts 1..10
      1 conn=3          # HTTP/1.1: a new connection per request (10 lines like this)
```

Connection numbers vary by run. The signature of multiplexing is a single `conn=` value whose `req_on_conn` climbs to 10, with the same client port on every line.

3. **See the handshake cost on the wire.** Capture a new HTTP/2 connection and count the packets before the request.

```bash
sudo timeout 5 tcpdump -i lo -n 'tcp port 8443' > /tmp/h2.txt &
sleep 1; curl -s $R https://shop.example.com:8443/a1.txt -o /dev/null; sleep 5
head -12 /tmp/h2.txt
```

You will see `[S]`, `[S.]`, `[.]` (TCP's three-way handshake) and only then the TLS ClientHello (`[P.]`). Compare it with the QUIC capture in step 6.

4. **Optional: head-of-line blocking under packet loss.** This needs the `netem` kernel module (present on most distributions, not in some containers/sandboxes), so output is described, not shown. Add 3% loss and 25 ms delay to loopback, download 12 files of about 300 KB each over one HTTP/2 connection, then over HTTP/1.1 with 6 connections, and time both.

```bash
for i in $(seq 1 12); do head -c 300000 /dev/urandom | base64 > www/f$i.bin; done
sudo tc qdisc add dev lo root netem delay 25ms loss 3%
for p in --http1.1 --http2; do
  echo $p; ( time curl -s $R $p --parallel --parallel-max 6
  "https://shop.example.com:8443/f[1-12].bin" -o /dev/null ) 2>&1 | grep real
done
sudo tc qdisc del dev lo root          # ALWAYS remove it afterwards
```

   Expect HTTP/2's single connection to lose its advantage, or even run slower, as loss rises, because every lost packet stalls all streams; results vary run to run. Always run `tc qdisc del` or all loopback traffic stays degraded.

5. **Serve HTTP/3 with HAProxy and call it.** HAProxy listens on TCP 9443 for HTTP/1.1 and HTTP/2, and on UDP 9443 for QUIC. It answers with a fixed response so no backend is needed. Ubuntu's HAProxy 2.8 uses OpenSSL's compatibility layer for QUIC, enabled with `limited-quic` (this layer does not offer 0-RTT; newer HAProxy builds with a native-QUIC TLS library do).

```bash
```

```
cat $HOME/tls-lab/pki/fullchain.pem $HOME/tls-lab/pki/leaf.key > hap.pem && chmod 600
  hap.pem
cat > haproxy.cfg <<EOF
global
  limited-quic
defaults
  mode http
  timeout connect 5s
  timeout client 30s
  timeout server 30s
frontend fe
  bind 127.0.0.1:9443 ssl crt $PWD/hap.pem alpn h2,http/1.1
  bind quic4@127.0.0.1:9443 ssl crt $PWD/hap.pem alpn h3
  http-after-response set-header alt-svc 'h3=":9443"; ma=900'
  http-request return status 200 content-type text/plain lf-string "hello over HTTP/%
  [fc_http_major]\n"
EOF
haproxy -c -f haproxy.cfg          # Configuration file is valid
haproxy -f haproxy.cfg &
sleep 1
curl -s $R --resolve shop.example.com:9443:127.0.0.1 https://shop.example.com:9443/ -D
  - | head -6
```

```
HTTP/2 200
content-length: 18
content-type: text/plain
alt-svc: h3=":9443"; ma=900
hello over HTTP/2
```

That response is exactly how a browser learns HTTP/3 exists: over TCP first, then it switches to UDP. Stock `curl` packages rarely include HTTP/3; if yours does (`curl -V` lists `HTTP3`), use `curl --http3 ...`. Otherwise use this small client:

BASH

```
python3 -m venv ~/h3venv && ~/h3venv/bin/pip install -q aioquic
cat > h3get.py <<'EOF'
import asyncio, sys, time
from aioquic.asyncio.protocol import QuicConnectionProtocol
from aioquic.h3.connection import H3_ALPN, H3Connection
from aioquic.h3.events import DataReceived, HeadersReceived
from aioquic.quic.configuration import QuicConfiguration
from aioquic.quic.connection import QuicConnection

class H3Client(QuicConnectionProtocol):
    def __init__(self, *a, **kw):
        super().__init__(*a, **kw)
        self.h3, self.done, self.body = H3Connection(self._quic), {}, {}
    def quic_event_received(self, event):
        for e in self.h3.handle_event(event):
```

```
            if isinstance(e, HeadersReceived):
                print("headers:", {k.decode(): v.decode() for k, v in e.headers})
            elif isinstance(e, DataReceived):
                self.body[e.stream_id] = self.body.get(e.stream_id, b"") + e.data
                if e.stream_ended:
                    self.done[e.stream_id].set_result(self.body[e.stream_id])
    async def get(self, host, path):
        sid = self._quic.get_next_available_stream_id()
        self.done[sid] = asyncio.get_running_loop().create_future()
        self.h3.send_headers(sid, [(b":method", b"GET"), (b":scheme", b"https"),
            (b":authority", host.encode()), (b":path", path.encode())],
  end_stream=True)
        self.transmit()
        return await self.done[sid]

async def main(host, port, path, ca, sni):
    cfg = QuicConfiguration(is_client=True, alpn_protocols=H3_ALPN, server_name=sni)
    cfg.load_verify_locations(ca)
    t0 = time.time()
    _, c = await asyncio.get_running_loop().create_datagram_endpoint(
        lambda: H3Client(QuicConnection(configuration=cfg)), local_addr=("0.0.0.0",
  0))
    c.connect((host, port))
    await c.wait_connected()
    print(f"QUIC handshake done in {1000*(time.time()-t0):.1f} ms, alpn=
  {c._quic.tls.alpn_negotiated}")
    print("body:", (await c.get(sni, path)).decode().strip())
    c.close(); await c.wait_closed()

asyncio.run(main(sys.argv[1], int(sys.argv[2]), sys.argv[3], sys.argv[4],
  sys.argv[5]))
EOF
~/h3venv/bin/python h3get.py 127.0.0.1 9443 / $HOME/tls-lab/pki/root.crt
  shop.example.com
```

```
QUIC handshake done in 13.7 ms, alpn=h3
headers: {':status': '200', 'content-length': '18', 'content-type': 'text/plain',
  'alt-svc': 'h3=":9443"; ma=900'}
body: hello over HTTP/3
```

Timing will differ. `alpn=h3` and the body proves the request went over HTTP/3.

6. **Look at QUIC on the wire.** It is UDP with opaque payloads.

```bash
sudo timeout 4 tcpdump -i lo -n -c 8 'udp port 9443' &
sleep 1; ~/h3venv/bin/python h3get.py 127.0.0.1 9443 / $HOME/tls-lab/pki/root.crt
  shop.example.com >/dev/null; sleep 4
```

```
IP 127.0.0.1.56395 > 127.0.0.1.9443: UDP, length 1200
IP 127.0.0.1.9443 > 127.0.0.1.56395: UDP, length 1252
IP 127.0.0.1.9443 > 127.0.0.1.56395: UDP, length 293
...
```

The first client datagram is exactly 1200 bytes: QUIC's anti-amplification padding. There is no SYN, no readable SNI, and no certificate in clear text, unlike in the TCP capture of step 3. The ports will differ.

7. **Demonstrate the 0-RTT replay risk (TLS 1.3 early data).** Enable early data in nginx, create a session with a ticket, then replay the same early-data request three times.

```bash
sed -i 's|    ssl_protocols TLSv1.2 TLSv1.3;|    ssl_protocols TLSv1.2 TLSv1.3;\n
  ssl_early_data on;|' nginx.conf
nginx -t -e $PWD/logs/error.log -c $PWD/nginx.conf && kill -HUP $(cat nginx.pid);
  sleep 1
printf 'GET /a1.txt HTTP/1.1\r\nHost: shop.example.com\r\nConnection: close\r\n\r\n' >
  req.txt
# 1) normal connection, save the session ticket (give the server a moment to send it)
(cat req.txt; sleep 1) | openssl s_client -connect 127.0.0.1:8443 -servername
  shop.example.com \
   -tls1_3 -alpn http/1.1 -sess_out sess.pem -quiet >/dev/null 2>&1
# 2) replay the identical early-data request three times with the same ticket
: > logs/access.log
for i in 1 2 3; do
  openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -tls1_3 -alpn
  http/1.1 \
     -sess_in sess.pem -early_data req.txt -ign_eof 2>&1 | grep 'Early data'
done
sleep 1; cat logs/access.log
```

```
Early data was accepted
Early data was accepted
Early data was accepted
37546 conn=17 req_on_conn=1 proto=HTTP/1.1 /a1.txt early=1
37560 conn=18 req_on_conn=1 proto=HTTP/1.1 /a1.txt early=1
37570 conn=19 req_on_conn=1 proto=HTTP/1.1 /a1.txt early=1
```

The same request, sent in the first flight, was executed three times. For `GET /a1.txt` that is harmless; for `POST /orders` it would create three orders. That is why you forward `$ssl_early_data` as `Early-Data` to applications (`proxy_set_header Early-Data $ssl_early_data;`) and answer `425 Too Early` for non-idempotent early requests. Turn `ssl_early_data off;` again if you do not need it.

8. **Clean up.**

```bash
kill $(cat ~/h2-lab/nginx.pid); pkill -f 'haproxy -f haproxy.cfg'
sudo tc qdisc del dev lo root 2>/dev/null; rm -r ~/h2-lab ~/h3venv /tmp/h2.txt
```

7 Real-world scenario

(Illustrative.) After a CDN rollout, the `shop-api` mobile team reports that image-heavy screens are slow for users on poor cellular links in one region, though server-side latency is unchanged. Real User Monitoring shows the p95 page load on HTTP/2 is much worse than the median, correlating with packet-loss spikes. Investigation: the CDN dashboard shows HTTP/3 is switched on, yet only a tiny share of requests use it, and `curl -sI https://www.example.com | grep -i alt-svc` returns nothing. The setting had been enabled on the staging hostname, not on production, so production responses never advertised `Alt-Svc`. A second problem surfaces when the team tests the origin directly: the cloud security group allows only TCP 443, so QUIC to the origin's HTTP/3 listener silently times out and clients fall back. Fix: enable HTTP/3 on the production hostname, open UDP 443 where QUIC is actually terminated, verify the `alt-svc` header in responses, and add a dashboard of request share by protocol and fallback rate. After the change, tail latency improves for lossy cellular clients while desktop clients show little difference. **Lesson:** HTTP/3 is an opt-in optimisation negotiated per client; verify end to end that it is advertised, reachable and actually used, and measure by percentile, not by average.

8 Common mistakes and best practices

Don't

- Enable HTTP/3 without opening UDP 443 on every layer (host firewall, security group, NLB/ALB listener, on-prem firewall).
- Remove TCP 443 or treat HTTP/3 as mandatory; some networks block UDP and clients must fall back.
- Accept 0-RTT for non-idempotent requests or ignore `Early-Data`.
- Load-balance gRPC/HTTP/2 at L4 and expect even distribution.
- Assume HTTP/2 is always faster; benchmark with realistic loss and latency.
- Leave HTTP/2 stream and header limits at defaults after a vulnerability advisory; patch proxies and servers promptly.

Do

- Keep HTTP/1.1, HTTP/2 and HTTP/3 enabled together, advertised with `Alt-Svc`, and monitor protocol share and fallbacks.
- Pass the negotiated protocol (`$server_protocol`) and `Early-Data` through to logs and applications.
- Use L7 load balancing for HTTP/2 and gRPC, with `GOAWAY`/connection max-age so long-lived connections rebalance after deploys.
- Test with `curl --http1.1/--http2/--http3`, `openssl s_client -alpn`, and browser dev tools' protocol column.
- Use consistent hashing on QUIC connection IDs (or a QUIC-aware balancer) so packets of one connection reach one backend.
- Apply resource limits: max concurrent streams, request rate per connection, header size, idle timeout.

9 Tasks

- [ ] **T1 — Show HTTP/2 multiplexing in logs** *(≈25 min, intermediate)*
  - **Goal:** prove many requests share one connection.
  - **Steps:**
    1. Do steps 1-2 of the guide.
    2. Count requests per `conn=` for HTTP/2 and HTTP/1.1 runs.
  - **Done when:** HTTP/2 shows one `conn=` carrying all 10 requests and HTTP/1.1 shows 10 connections.
- [ ] **T2 — Compare handshakes on the wire** *(≈30 min, advanced)*
  - **Goal:** count round trips for TCP+TLS versus QUIC.
  - **Steps:**
    1. Capture an HTTP/2 request (step 3) and an HTTP/3 request (step 6).
    2. Annotate each capture: which packets are the transport handshake, which carry TLS, when the request is sent.
  - **Done when:** you can state the RTT count before the first request for both and see 1200-byte padding in the QUIC capture.
- [ ] **T3 — Discover HTTP/3 like a browser** *(≈20 min, intermediate)*
  - **Goal:** reproduce `Alt-Svc` discovery.
  - **Steps:**
    1. Do step 5, request over h2 with curl and read `alt-svc`.
    2. Change `ma=900` to `ma=0` in `haproxy.cfg`, reload, and explain what that tells clients.

3. Remove the `bind quic4@` line and confirm the Python client now times out while curl still works.

- **Done when:** you can explain from observed behaviour why HTTP/3 must be additive to TCP.

- [ ] **T4 — Break and fix: the missing UDP rule** *(≈30 min, advanced)*
  - **Goal:** diagnose "HTTP/3 enabled but unused".
  - **Steps:**
    1. Block UDP 9443 with `sudo iptables -A INPUT -i lo -p udp --dport 9443 -j DROP` (note the rule), run the Python client, observe the timeout.
    2. Confirm TCP/h2 still works, then delete the rule with `-D`.
    3. Write the three checks you would run in production to catch this (listener, firewall, `Alt-Svc`).
  - **Done when:** the client times out with the rule and succeeds without it, and your three checks are written down.

- [ ] **T5 — Make 0-RTT safe** *(≈35 min, advanced)*
  - **Goal:** stop replayed non-idempotent requests.
  - **Steps:**
    1. Reproduce the replay (step 7).
    2. Add a small backend (for example `python3 -m http.server` or a tiny app) behind nginx with `proxy_set_header Early-Data $ssl_early_data;`, and make it return `425` when the header is `1` and the method is `POST`.
    3. Replay a `POST` as early data and confirm it is refused, then retried normally.
  - **Done when:** the replayed `POST` gets `425 Too Early` while a normal `POST` succeeds.

- [ ] **T6 — Prove it: choose protocols for `shop-api`** *(≈40 min, advanced)*
  - **Goal:** justify a protocol plan.
  - **Steps:**
    1. Without the guide, rebuild an HTTP/2 + HTTP/3 server and show both with commands.
    2. Write half a page: which protocol each hop uses (browser to CDN, CDN to ALB, ALB to Nginx, Nginx to Node, Node to PostgreSQL), what must be opened on firewalls, and what you monitor.
  - **Done when:** you can run both protocols from memory and defend each hop's choice, including why internal hops may stay on HTTP/1.1.

10 Self-check questions

1. What is head-of-line blocking, and why does HTTP/2 still suffer from it but HTTP/3 does not?
2. How does a browser learn that a site supports HTTP/3?
3. Why is the first QUIC packet padded to 1200 bytes?
4. What is the risk of 0-RTT, and which requests are safe in it?
5. Why does gRPC need L7 load balancing, and what role does `GOAWAY` play?
6. A customer on a corporate network cannot use HTTP/3. Should that break the site? Why not?

ANSWERS

1. A lost or delayed item blocks everything behind it in an ordered sequence. HTTP/2 multiplexes streams over one TCP byte stream, so one lost TCP segment stalls all streams until retransmitted; QUIC recovers loss per stream, so only the affected stream waits.
2. Via an `Alt-Svc: h3=":443"` response header (or an HTTPS DNS record) seen on an HTTP/1.1 or HTTP/2 connection; the next connection attempt tries QUIC and falls back if it fails.
3. As a defence against amplification attacks: the server will only send a limited multiple of what it receives before validating the client address, so a small spoofed packet cannot trigger a large reflected response.
4. Early data can be replayed by an attacker or a retry, so the server might process it twice; only idempotent, safe requests (for example cacheable `GET`) are acceptable. Servers signal the situation with `Early-Data: 1` and may answer `425 Too Early`.
5. A gRPC channel is one long-lived HTTP/2 connection with many streams, so an L4 balancer places all calls on one backend. An L7 balancer or proxy balances per request/stream, and `GOAWAY` lets servers drain connections gracefully for rebalancing and deploys.
6. No. HTTP/3 is negotiated opportunistically; the client falls back to HTTP/2 or HTTP/1.1 over TCP. That is why TCP 443 must remain enabled.

11 Further reading

- RFC 9113 (HTTP/2), RFC 9114 (HTTP/3), RFC 9000, 9001, 9002 (QUIC transport, TLS, loss recovery), RFC 9204 (QPACK), RFC 8470 (Using early data in HTTP), RFC 7541 (HPACK).
- The HTTP/3 explained book (curl author Daniel Stenberg), and the nginx, HAProxy and Envoy documentation sections on HTTP/3/QUIC.
- Cloudflare and Google engineering blog posts on QUIC deployment, and the joint 2023 Rapid Reset disclosures (CVE-2023-44487).

- Ilya Grigorik, *High Performance Browser Networking* (free online).

12.6 Load Balancing

Topic 6 of 14 | ~4 hours | Difficulty: Advanced

1 What it is

A **load balancer (LB)** spreads requests or connections across a pool of backends so that no single server is overloaded and a failed server is skipped. Level 2 §2.8 and L4 vs L7 §2.9 covered what an LB is and the layers it works at. This topic is about the *decisions* it makes:

- **Algorithms:** how it picks a backend (round-robin, weighted, least connections, least request, random-two-choices, hashing, consistent hashing).
- **Health checking:** how it knows which backends are fit to receive traffic (active probes, passive outlier detection) and how it brings them back safely.
- **Topology:** where balancing happens (client-side, in a proxy, in the network, at a global DNS or anycast layer) and how layers stack: a **global load balancer** picks a region, a regional LB picks a zone or cluster, a local LB picks an instance.

2 Why it matters in DevOps

- **The wrong algorithm creates hotspots.** Round-robin is fair in *requests* but not in *work*; a few slow or heavy requests can pile up on one server while others idle.
- **Health checks decide your availability.** A check that is too lax sends users to dead servers; one that is too strict, or that depends on a shared database, can eject the entire fleet at once and turn a partial failure into a full outage.
- **Stateful and cache-heavy workloads need affinity.** Caches, sharded in-memory state, WebSocket sessions and per-user rate limits perform best when the same key reaches the same backend, and badly when a pool change reshuffles everything.
- **Deploys and autoscaling are balancing events.** New instances must receive traffic gradually (slow start) and leaving instances must finish in-flight work (connection draining), or every rollout causes errors (Level 5, Zero-downtime §5.8).
- **It is a capacity and failure-domain design tool.** Regional failover, active-active and DR all rely on global traffic steering (Level 0, HA and scalability §0.8).

3 Real-world system example

Google Maglev is Google's published software network load balancer: a cluster of commodity servers announces service virtual IPs, the network spreads packets over Maglev machines with ECMP (equal-cost multi-path), and each machine picks a backend with a **consistent hashing** scheme built for fast lookup and minimal disruption, so connections keep going to the same backend even when machines fail or join. The Maglev paper (2016) is public. **Meta's Katran** (open source, XDP/eBPF based) is a layer 4 forwarding plane that uses a consistent-hashing scheme in the same spirit. **GitHub's GLB Director** (documented on the GitHub engineering blog) is a layer 4 balancer that uses a consistent hashing technique so that adding or draining proxies does not break existing connections.

Envoy (the data plane behind Istio and many API gateways) supports round robin, **least request** (by default picking two random hosts and sending to the one with fewer active requests, the "power of two choices"), ring hash, Maglev and random, plus active health checks and **outlier detection** (passive ejection of hosts returning errors). **AWS** offers Application Load Balancers (layer 7; round robin or "least outstanding requests" algorithms), Network Load Balancers (layer 4; flow-hash based), and **AWS Global Accelerator** and **Route 53** for global steering with health checks (Level 6, ALB §6.14, Route 53 §6.12). **Google Cloud's** global external Application Load Balancer exposes a single anycast IP address and routes users to the nearest healthy backend region with spillover when a region is full. **Kubernetes** Services balance across pods in `kube-proxy` (iptables or IPVS) and Ingress controllers (nginx, Envoy) add L7 logic (Level 8, Services §8.4).

4 Diagram

A typical stacked design: DNS or anycast picks a region, a regional L7 LB picks an instance, and health checks feed back into every decision:

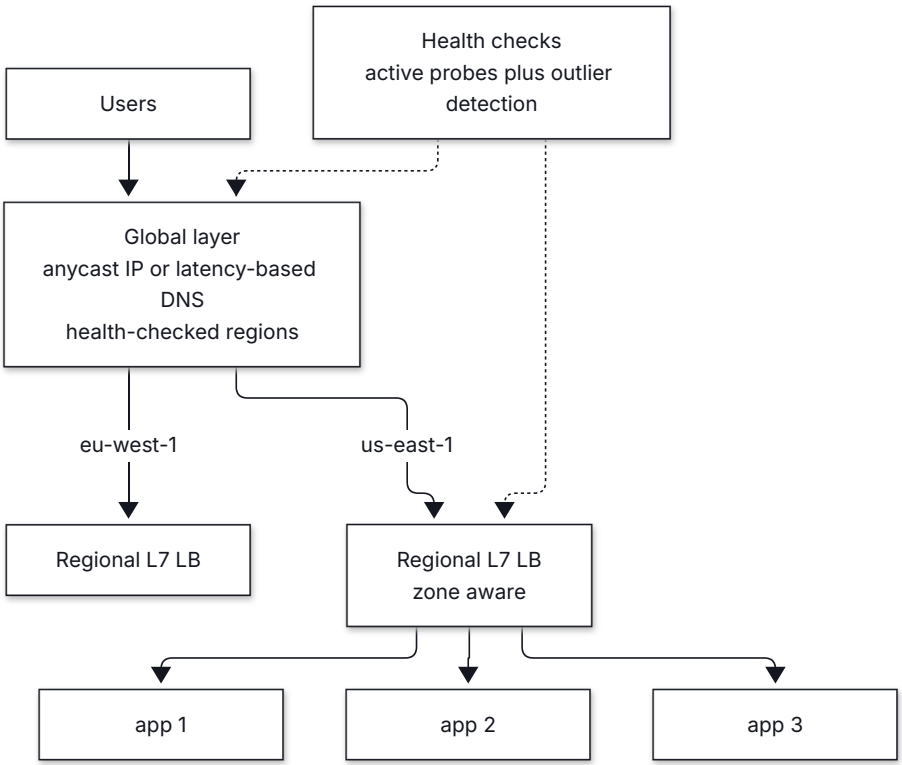


Figure 12.6-1

What a health-checked LB does when a backend breaks and recovers (this is what the lab reproduces):

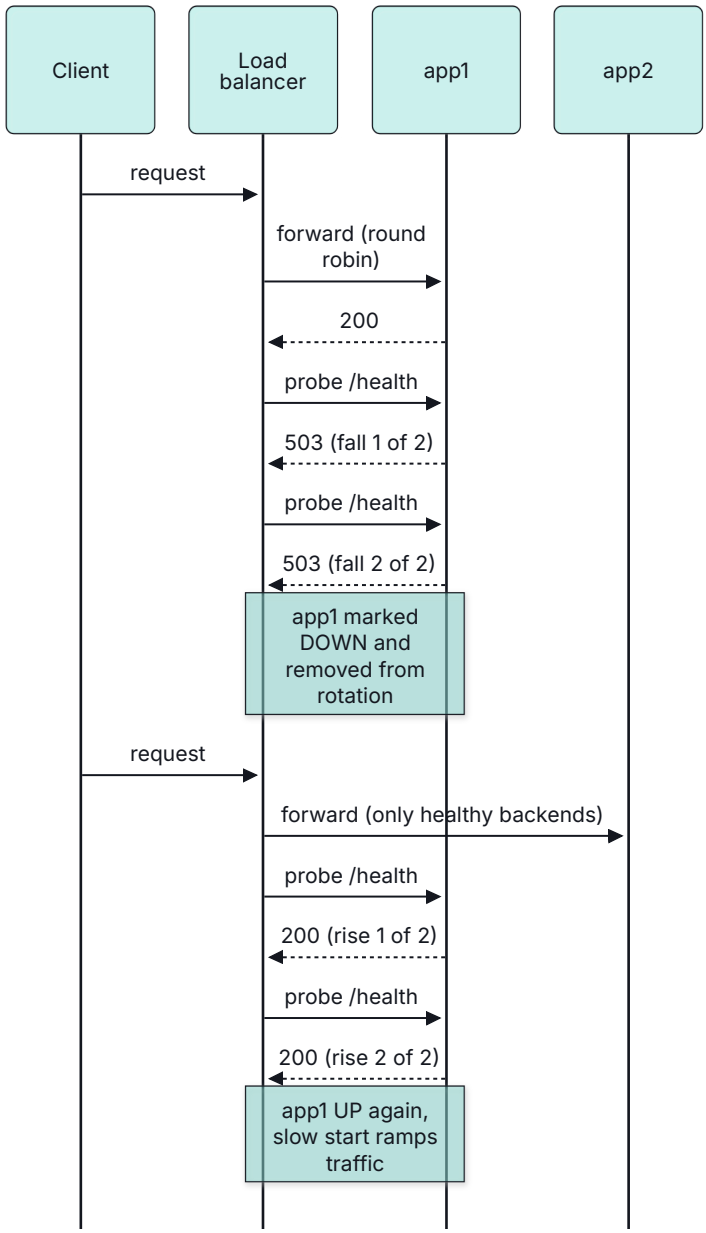


Figure 12.6-2

5 How it works

Algorithms

Algorithm	How it picks	Good for	Watch out for
Round robin	Next backend in a loop	Homogeneous backends, short uniform requests	Ignores load; long requests pile up on one server
Weighted round robin	Loop, but backends appear in proportion to weight	Mixed instance sizes, canary traffic (5 percent to the new version)	Weights are static
Random	Uniform random	Simple, stateless clients	Variance at low request rates
Least connections	Backend with fewest open connections (weighted variants exist)	Long-lived or uneven requests, WebSockets, gRPC streams	Needs shared connection counts across LB nodes; new backends get a burst
Least request / least outstanding requests	Fewest in-flight requests (L7)	Variable latency, heterogeneous backends	Same as above; per-LB-node view only
Power of two choices (P2C)	Pick 2 at random, take the less loaded	Large fleets or many LB nodes: near-optimal balance without global state	Slightly more complex to reason about
Source IP hash	`hash(client IP) mod N`	Crude stickiness	Reshuffles almost everything when N changes; NAT and proxies skew it
Consistent hashing	Key (URL, user id, header) mapped onto a ring or lookup table of backends	Caches, sharded state, per-key rate limits, QUIC connection ID routing	Skew without virtual nodes; loses balance if the key is hot
Maglev hashing	Lookup table built from per-backend permutations	L4 balancers needing speed and minimal disruption	Table rebuild on membership change
Power-aware or latency-aware (EWMA, peak EWMA)	Prefers lowest recent latency	Mixed-latency backends	Needs warm-up and careful smoothing

Why consistent hashing? With `hash(key) mod N`, adding a fifth server to four remaps roughly 80 percent of the keys, which destroys every cache at once. With a consistent-hash ring (each server placed at many pseudo-random points, called virtual nodes), a key maps to the next server clockwise, so adding one server moves only about 1/N of the keys and removing a server moves only the keys it owned. You will measure both in the lab.

Health checks

Type	How	Notes
Active, L4	TCP connect	Cheap; proves the port is open, not that the app works
Active, L7	`GET /health` expecting `200` (and optionally a body)	The common default; check *this process can serve*, not "everything it depends on"
Passive / outlier detection	Watch real traffic: eject a host after N consecutive 5xx or timeouts, re-admit after a back-off	Catches failures a probe misses; Envoy, HAProxy (`observe`), nginx (`max_fails`)
Liveness vs readiness	Kubernetes distinguishes "restart me" from "do not send me traffic"	Level 8, probes §8.15

Parameters to tune: interval, timeout, **fall** (failures before DOWN), **rise** (successes before UP), and a **panic threshold** or "fail open" rule (if too many hosts are unhealthy, route to all of them rather than none; Envoy defaults to a panic threshold of 50 percent). Design rules: a shallow health endpoint (process alive, thread pool not stuck) for the LB; a deeper "dependency" check only for dashboards or readiness gating with care, because a shared database blip should degrade you, not eject every server simultaneously. Add **slow start** to ramp a newly healthy server and **connection draining** (also called deregistration delay) to let a leaving server finish in-flight requests.

Where balancing happens

- **Client-side:** the client holds the list (service discovery plus a library or gRPC `round_robin`). No extra hop; every client needs the logic.
- **Proxy / L7:** nginx, HAProxy, Envoy, ALB; can route by path, header and cookie, terminate TLS, retry.
- **L4:** NLB, IPVS, Maglev, Katran; passes TCP/UDP flows without parsing HTTP, higher throughput. With **DSR (direct server return)** replies skip the LB.
- **Service mesh / sidecar:** per-pod proxies do client-side balancing with central policy (Service Mesh §12.9).

Global load balancing

Directing users to the right *region* uses mechanisms that live above the individual LB:

Mechanism	How	Failover speed	Caveat
DNS-based (GSLB)	Authoritative DNS returns different answers by latency, geography, weight or health (DNS Architecture §12.3)	TTL-bound (tens of seconds to minutes; some resolvers ignore TTLs)	Sees the resolver, not the user

Mechanism	How	Failover speed	Caveat
Anycast IP	Same IP announced from many sites via BGP (BGP §12.2); network routes to the nearest	Seconds, by withdrawing routes	Long-lived TCP connections may move mid-flow; need connection-ID aware design
Global anycast L7 LB / CDN	One anycast address, edge proxies pick an origin using health and latency	Seconds	Vendor feature; origin must be reachable from edges (CDNs §12.7)
Client-side / app-level	App or SDK picks a region and fails over	Fastest	Logic in every client

Design for **capacity during failover**: if `us-east-1` fails, `eu-west-1` must absorb its traffic. Use spillover limits and test them; otherwise failover just moves the overload.

6 Step-by-step guide

You will run three tiny Python backends, put HAProxy in front with round robin, least-connections and consistent-hash backends, watch each algorithm behave, kill a backend and watch health checks react, and quantify how many keys move with modulo hashing versus a consistent-hash ring. Free, local, no Docker needed (a Compose version is at the end).

Prerequisites: Linux with `python3`, `haproxy` 2.4+ and `curl` (`sudo apt-get install -y haproxy curl`; if the package starts a system service, `sudo systemctl disable --now haproxy`). Verified with HAProxy 2.8. Run `unset https_proxy HTTPS_PROXY` if your shell sets a proxy.

1. **Create the backends.** Each answers with its name; `app3` is deliberately slow (0.5 s); `/health` reports status; `/break` and `/fix` toggle health so you can fail a server on demand.

```bash
mkdir -p ~/lb-lab && cd ~/lb-lab
cat > backend.py <<'EOF'
import sys, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

name, port = sys.argv[1], int(sys.argv[2])
delay = float(sys.argv[3]) if len(sys.argv) > 3 else 0.0
healthy = True

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def do_GET(self):
        global healthy
        if self.path == "/health":
            code, body = (200, b"ok\n") if healthy else (503, b"sick\n")
```

```
        elif self.path == "/break":
            healthy = False; code, body = 200, b"now unhealthy\n"
        elif self.path == "/fix":
            healthy = True; code, body = 200, b"now healthy\n"
        else:
            time.sleep(delay)
            code, body = 200, f"{name}\n".encode()
        self.send_response(code)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass

ThreadingHTTPServer(("127.0.0.1", port), H).serve_forever()
EOF
python3 backend.py app1 9001 &
python3 backend.py app2 9002 &
python3 backend.py app3 9003 0.5 &
sleep 1; for p in 9001 9002 9003; do curl -s localhost:$p/; done
```

Expected: `app1`, `app2`, `app3` (the last after a half-second pause).

2. **Configure HAProxy with three backends using three algorithms.** Ports 8081 (round robin), 8082 (least connections) and 8083 (consistent hash on the URI). Health checks run every second: two failures mark a server DOWN, two successes mark it UP.

```bash
cat > haproxy.cfg <<'EOF'
global
  maxconn 1000
defaults
  mode http
  timeout connect 2s
  timeout client 10s
  timeout server 10s
frontend fe_rr
  bind 127.0.0.1:8081
  default_backend be_rr
frontend fe_lc
  bind 127.0.0.1:8082
  default_backend be_lc
frontend fe_hash
  bind 127.0.0.1:8083
  default_backend be_hash
frontend stats
  bind 127.0.0.1:8404
  stats enable
  stats uri /stats
backend be_rr
  balance roundrobin
  option httpchk GET /health
```

```
  http-check expect status 200
  default-server inter 1s fall 2 rise 2
  server app1 127.0.0.1:9001 check
  server app2 127.0.0.1:9002 check
  server app3 127.0.0.1:9003 check
backend be_lc
  balance leastconn
  option httpchk GET /health
  default-server inter 1s fall 2 rise 2
  server app1 127.0.0.1:9001 check
  server app2 127.0.0.1:9002 check
  server app3 127.0.0.1:9003 check
backend be_hash
  balance uri
  hash-type consistent
  option httpchk GET /health
  default-server inter 1s fall 2 rise 2
  server app1 127.0.0.1:9001 check
  server app2 127.0.0.1:9002 check
  server app3 127.0.0.1:9003 check
EOF
haproxy -c -f haproxy.cfg           # Configuration file is valid
haproxy -f haproxy.cfg &
sleep 2
```

3. **Round robin vs least connections under uneven load.** Fire 30 concurrent requests at each; `app3` is slow, so it holds connections open longer.

```bash
echo "round robin:";  for i in $(seq 1 30); do curl -s localhost:8081/ & done | sort |
  uniq -c; wait
echo "least conn:";   for i in $(seq 1 30); do curl -s localhost:8082/ & done | sort |
  uniq -c; wait
```

```
round robin:
     10 app1
     10 app2
     10 app3
least conn:
     12 app1
     15 app2
      3 app3
```

Round robin sends the slow server its full third; least-connections notices its open connections and shifts work to the fast ones. The exact least-conn split varies from run to run, but `app3` always gets far less.

4. **Consistent hashing gives affinity.** The same URI always reaches the same backend.

```bash
for k in 1 2 3 4 5 6; do echo -n "/p/$k → "; curl -s localhost:8083/p/$k; done
for k in 1 2 3 4 5 6; do echo -n "/p/$k → "; curl -s localhost:8083/p/$k; done    # identical answers
```

5. **Fail a backend and watch what moves.** Record where 12 keys land, break `app1`, wait for the health check to eject it, and compare.

```bash
for k in $(seq 1 12); do echo -n "/p/$k → "; curl -s localhost:8083/p/$k; done > before.txt
curl -s localhost:9001/break; sleep 3.5          # two failed probes at 1 s interval
for k in $(seq 1 12); do echo -n "/p/$k → "; curl -s localhost:8083/p/$k; done > after.txt
diff before.txt after.txt                          # only keys that were on app1 changed
curl -s 'localhost:8404/stats;csv' | cut -d, -f1,2,18 | grep -E 'be_hash,app'
```

```
be_hash,app1,DOWN
be_hash,app2,UP
be_hash,app3,UP
```

Every line in the `diff` is a key that used to go to `app1`; keys that lived on `app2` and `app3` did not move. That is the property that keeps caches warm during incidents. Now check round robin with a failure, then restore:

```bash
curl -s localhost:9001/fix; curl -s localhost:9002/break; sleep 3.5
for i in $(seq 1 6); do curl -s localhost:8081/; done | sort | uniq -c       # app1 and app3 only
curl -s localhost:9002/fix; sleep 3.5
```

You can also browse the live table at `http://127.0.0.1:8404/stats`.

6. **Measure how many keys move: modulo hashing vs a consistent-hash ring.** This is pure arithmetic, no servers needed.

```bash
cat > ring.py <<'EOF'
import bisect, hashlib

def h(s):
    return int(hashlib.md5(s.encode()).hexdigest()[:16], 16)
```

```
class Ring:
    def __init__(self, nodes, vnodes=100):
        self.ring = sorted((h(f"{n}#{i}"), n) for n in nodes for i in range(vnodes))
        self.keys = [k for k, _ in self.ring]
    def get(self, key):
        i = bisect.bisect(self.keys, h(key)) % len(self.ring)
        return self.ring[i][1]

keys = [f"user-{i}" for i in range(100_000)]
old, new = ["s1", "s2", "s3", "s4"], ["s1", "s2", "s3", "s4", "s5"]
mod_moved = sum(old[h(k) % 4] != new[h(k) % 5] for k in keys)
r_old, r_new = Ring(old), Ring(new)
ring_moved = sum(r_old.get(k) != r_new.get(k) for k in keys)
print(f"hash(key) % N : {mod_moved/len(keys):.1%} of keys move when going from 4 to 5
  servers")
print(f"consistent ring: {ring_moved/len(keys):.1%} of keys move (ideal is 1/5 =
  20.0%)")
EOF
python3 ring.py
```

```
hash(key) % N : 80.0% of keys move when going from 4 to 5 servers
consistent ring: 16.9% of keys move (ideal is 1/5 = 20.0%)
```

Try `vnodes=1` and see how uneven the ring becomes; virtual nodes are what smooth the distribution.

7. **The same ideas in nginx** (syntax validated with `nginx -t`; paste into an `http {}` block):

```nginx
upstream shop_rr   { server 10.0.1.11:3000; server 10.0.1.12:3000; server
  10.0.1.13:3000 weight=2; }
upstream shop_lc   { least_conn; server 10.0.1.11:3000; server 10.0.1.12:3000
  max_fails=2 fail_timeout=10s; }
upstream shop_hash { hash $request_uri consistent; server 10.0.1.11:3000; server
  10.0.1.12:3000; }
server {
  listen 80;
  location / {
    proxy_pass http://shop_hash;
    proxy_next_upstream error timeout http_503;   # retry another backend, only safe
  for idempotent requests
  }
}
```

Open-source nginx health checking is passive (`max_fails` / `fail_timeout`); active probes need NGINX Plus or an external check. HAProxy and Envoy do active checks out of the box.

8. **Clean up.**

```bash
pkill haproxy; pkill -f backend.py; cd ~ && rm -r ~/lb-lab
```

Docker Compose variant. The same lab runs as three `python:3.12-slim` services (`command: python /app/backend.py app1 9001`, with the file bind-mounted) plus an `haproxy:2.8` service mounting `haproxy.cfg` with server addresses changed from `127.0.0.1:900x` to the service names (`app1:9001`). If you use this, validate first with `docker run --rm -v "$PWD/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro" haproxy:2.8 haproxy -c -f /usr/local/etc/haproxy/haproxy.cfg`.

7 Real-world scenario

(Illustrative.) `shop-api` runs 12 instances behind an ALB with a `/health` endpoint that runs `SELECT 1` against PostgreSQL and a Redis `PING`. During a routine PostgreSQL failover the primary is unreachable for 40 seconds. Every instance's health check fails, the ALB marks all 12 unhealthy and, with no healthy targets, begins returning errors to everyone, including requests for the product catalog that could have been served from Redis. Investigation: target group metrics show `HealthyHostCount` fall to zero while each application's own error rate was moderate; the instances were fine, the check was too deep. Recovery took longer than the database event because instances re-entered service one by one after `rise` thresholds, and all new connections hit freshly healthy instances at once. Fix: the LB check now hits a shallow `/healthz` (process alive and event loop responsive); a separate `/readyz` includes dependencies and is used only for deployment gating. Slow start (2 minutes) is enabled on the target group, and the app degrades gracefully when the database is down. **Lesson:** a health check answers "should the LB send traffic here?", not "is the whole system healthy?". Make it shallow, add slow start, and make sure "all unhealthy" fails open or degrades rather than going dark.

8 Common mistakes and best practices

Don't

- Use `hash(key) mod N` for anything where changing N matters (caches, shards).
- Put dependency checks (database, third-party API) in the LB health endpoint.
- Use round robin for long-lived or highly variable requests without checking the distribution.
- Retry non-idempotent requests (`POST /orders`) automatically on another backend.

- Forget connection draining and slow start in deployment and autoscaling configs.
- Assume DNS failover is instant; TTL and resolver behaviour make it slower than health checks suggest.
- Let one LB become a single point of failure; LBs need redundancy too (VRRP, anycast, multi-AZ managed LBs).

Do

- Choose the algorithm from the workload: least-request or P2C for variable latency, consistent hashing for caches and sharded state.
- Combine active health checks with passive outlier detection, and tune `interval`, `fall`, `rise` and timeouts deliberately.
- Define a fail-open or panic behaviour for "most backends unhealthy".
- Export per-backend request rate, error rate, latency, active connections and ejections as metrics (Level 10 §10.11).
- Test failover: kill a backend, an AZ and a region and watch the traffic redistribute; confirm surviving capacity is enough.
- Keep weights and capacity in code (Level 7 §7.3) so canary shifts and scale changes are reviewed.

9 Tasks

- [ ] **T1 — Observe round robin vs least connections** *(≈25 min, intermediate)*
  - **Goal:** see how algorithm choice changes distribution under uneven load.
  - **Steps:**
    1. Follow steps 1-3.
    2. Repeat the 30-request test three times for each algorithm and record counts per backend.
    3. Change `app3` delay to 2 seconds and repeat.
  - **Done when:** you have a table of nine runs and can explain why least-conn shifts away from `app3` while round robin does not.

- [ ] **T2 — Weighted canary** *(≈20 min, intermediate)*
  - **Goal:** send roughly 10 percent of traffic to one backend.
  - **Steps:**
    1. In `be_rr`, set `weight 90` on `app1` and `app2` and `weight 20` on `app3` (about 10 percent), reload with `haproxy -f haproxy.cfg -sf $(pidof haproxy) &`.
    2. Send 200 requests and count.

- **Done when:** `app3` receives roughly 10 percent of the 200 requests (counts will vary by a few percent).

- [ ] **T3 — Break it: health check tuning** *(≈30 min, advanced)*
  - **Goal:** feel the trade-off between detection speed and flapping.
  - **Steps:**
    1. Set `inter 5s fall 3 rise 3`; break `app1` and time (with `date`) how long until `curl` stops hitting it.
    2. Set `inter 500ms fall 1 rise 1`; script a flaky backend that fails every other `/health` call and watch the stats page flap.
    3. Choose a defensible setting and justify it in two sentences.
  - **Done when:** you have measured detection time for both settings and observed flapping in the aggressive one.

- [ ] **T4 — Consistent hashing and the ring** *(≈30 min, advanced)*
  - **Goal:** quantify disruption.
  - **Steps:**
    1. Run `ring.py`; then change `vnodes` to 1, 10, 100, 1000 and record the fraction moved and the per-server key counts (add a counter).
    2. Add a sixth server and a removal case, and compare with modulo hashing.
  - **Done when:** you can show that more virtual nodes give a more even distribution and that removing a node only moves that node's keys.

- [ ] **T5 — Zero-error restart with draining** *(≈35 min, advanced)*
  - **Goal:** deploy without errors.
  - **Steps:**
    1. Run a load loop (`while true; do curl -s -o /dev/null -w '%{http_code}\n' localhost:8081/; done | sort | uniq -c` and stop with Ctrl-C afterwards) in one terminal.
    2. Restart `app2` abruptly (`kill`) and count non-200 responses.
    3. Instead call `/break`, wait for DOWN, restart, `/fix`, and count again.
  - **Done when:** the abrupt kill produces some errors and the drain-first procedure produces none (or fewer), and you write the 4-step safe procedure.

- [ ] **T6 — Prove it: design global traffic steering for `shop-api`** *(≈45 min, advanced)*
  - **Goal:** explain a multi-region balancing design.
  - **Steps:**

1. Without the guide, rebuild the three-backend HAProxy lab and demonstrate one algorithm per frontend.
2. Write one page covering: global mechanism (DNS vs anycast) and failover time, regional LB algorithm and health check design, what happens at 100 percent capacity in a failover, and what you monitor.

- **Done when:** the lab runs from memory and the page states the failover time and the spare capacity needed in the surviving region.

10 Self-check questions

1. Why does `hash(key) % N` hurt a cache fleet when you scale from 4 to 5 servers, and how does consistent hashing help?
2. When is least-connections better than round robin, and what is the power-of-two-choices trick?
3. What do `fall`, `rise` and `inter` control in a health check, and what is the risk of setting them too aggressively?
4. Why should a health check not query the database?
5. Compare DNS-based and anycast-based global load balancing for failover speed.
6. What are slow start and connection draining for?

ANSWERS

1. Changing N changes `key % N` for most keys (about 80 percent from 4 to 5), so almost every cached item is looked up on the wrong server and misses. A consistent-hash ring moves only about 1/N of keys (those that fall between the new server and its neighbour).
2. When requests vary in duration or backends vary in speed, because open-connection count reflects real load. P2C picks two random backends and uses the less loaded, giving near-least-loaded balance without a global view, which scales to many LB nodes.
3. `inter` is the probe interval, `fall` how many consecutive failures mark a server DOWN, `rise` how many successes mark it UP. Too aggressive means flapping and ejecting healthy servers on a hiccup (and amplified load on the rest); too lax means sending users to dead servers.
4. A shared dependency failure would fail every instance's check at once, ejecting the whole fleet and converting a degraded state into a total outage. Keep LB checks shallow and handle dependency failure in the application.
5. DNS failover is bounded by TTLs and resolver caching (tens of seconds to minutes or more, with some clients ignoring TTLs); anycast reacts to routing changes (seconds), but long-lived connections may be re-routed mid-flow.

6. Slow start gradually ramps traffic to a newly healthy or newly launched server so cold caches and JIT warm-up do not cause errors; draining lets a leaving server finish in-flight requests and stop receiving new ones before it is stopped.

11 Further reading

- The Maglev paper: "Maglev: A Fast and Reliable Software Network Load Balancer" (NSDI 2016), and the Envoy documentation on load balancing and outlier detection.
- HAProxy configuration manual (`balance`, `hash-type`, `option httpchk`, `slowstart`) and the nginx `ngx_http_upstream_module` documentation.
- AWS Elastic Load Balancing documentation (target groups, deregistration delay, slow start) and Google Cloud Load Balancing overview.
- GitHub Engineering blog on GLB Director; Meta's open-source Katran repository documentation.
- Michael Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing" (paper).
- Brendan Burns et al., *Designing Distributed Systems*; Google, *Site Reliability Engineering* (load balancing chapters).

12.7 CDNs

Topic 7 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

A **Content Delivery Network (CDN)** is a geographically distributed fleet of caching reverse proxies (called **edge servers** or **points of presence, PoPs**) that sit between users and your **origin** (the servers you run). A user is steered to a nearby edge using anycast or DNS (DNS Architecture §12.3, BGP §12.2). The edge keeps a copy of cacheable responses and answers from memory or disk; only a **cache miss** travels all the way to the origin.

A CDN delivers four things at once: **lower latency** (the TLS handshake and the content are close to the user), **less origin load** (a cache hit never reaches you), **absorbed spikes and attacks** (huge aggregate capacity, DDoS filtering, a web application firewall) and **resilience** (stale content can be served when the origin is down). What it caches and for how long is controlled almost entirely by HTTP headers, above all `Cache-Control`, and by the **cache key** (the string that identifies "the same object").

2 Why it matters in DevOps

- **A huge share of production traffic should never hit your origin.** Images, JavaScript, CSS, downloads, video segments and even cacheable API responses can be served from the edge. A good cache-hit ratio is the cheapest capacity you will ever buy, and a collapsed one (after a bad deploy or a cache-key mistake) is a classic origin-melting incident.
- **Wrong headers are production bugs.** A missing `private` leaks one user's page to another; an over-long `max-age` on an unversioned file ships a bad deploy to users for a day, and you cannot "roll back" a cache you do not control.
- **Purging, versioning and invalidation are part of your release process.** CDN configuration belongs in code and in CI/CD (Level 5 §5.1).
- **Cost.** Egress from your cloud is expensive; CDN egress is typically cheaper and a high hit ratio cuts both (Level 20, egress costs §20.7).

- **Security.** The CDN is your first line against volumetric DDoS and bots, but it is also a place where **cache poisoning** and **web cache deception** (caching something private under a public key) happen.

3 Real-world system example

Netflix Open Connect is Netflix's own CDN: purpose-built appliances (Open Connect Appliances) that Netflix offers to place inside ISP networks and at internet exchange points, pre-filled with popular titles during off-peak hours. Netflix documents that its control plane steers clients to the best appliance, and the vast majority of video bytes are served from these caches rather than from AWS.

Cloudflare, Fastly, Akamai, AWS CloudFront and Google Cloud CDN cache the web's static and semi-static content for millions of sites. Their documented features map to this topic: **tiered caching / origin shield** (Cloudflare Tiered Cache, CloudFront Origin Shield, Fastly shielding), **purge by URL, prefix or tag** (Fastly surrogate keys, Cloudflare cache tags, CloudFront invalidations), and **stale-while-revalidate / stale-if-error** support. See Level 6, CloudFront §6.13 for the AWS flavour.

Fastly, 8 June 2021. Fastly publicly described a global outage lasting under an hour in which a large fraction of its network returned errors. A software bug that had been introduced in an earlier deployment lay dormant until a customer pushed a valid configuration change that triggered it. Many well-known sites that depend on Fastly were affected, and recovery was fast because the trigger was identified and reverted within minutes. Lessons: the CDN is a dependency with its own blast radius; consider multi-CDN for critical properties and make sure you can route around the CDN (for example by DNS) when needed.

4 Diagram

The tiers a request crosses and where each cache decision is made:

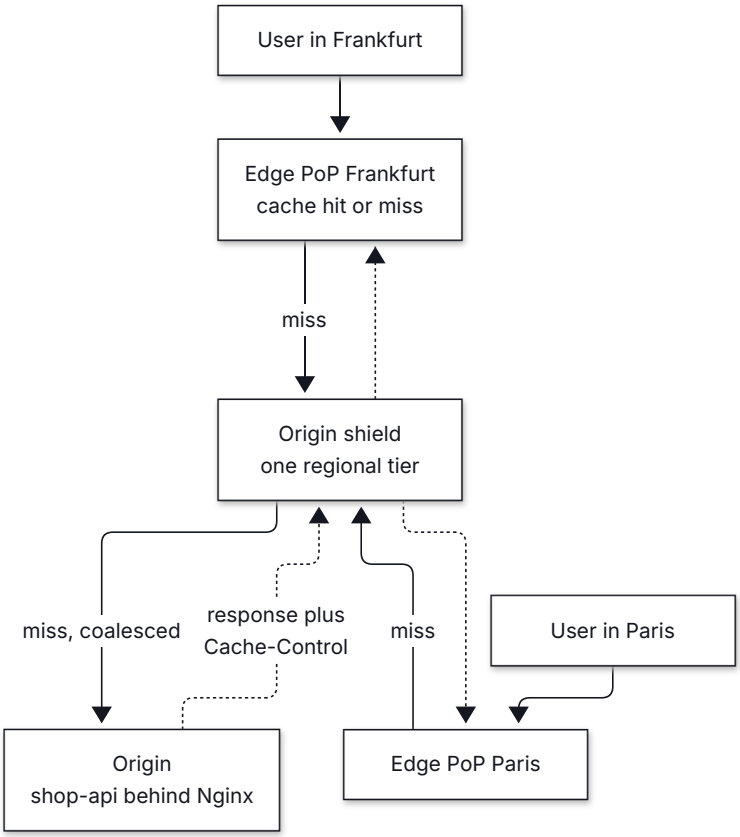


Figure 12.7-1

The lifecycle of one cached object, driven by `Cache-Control` and revalidation:

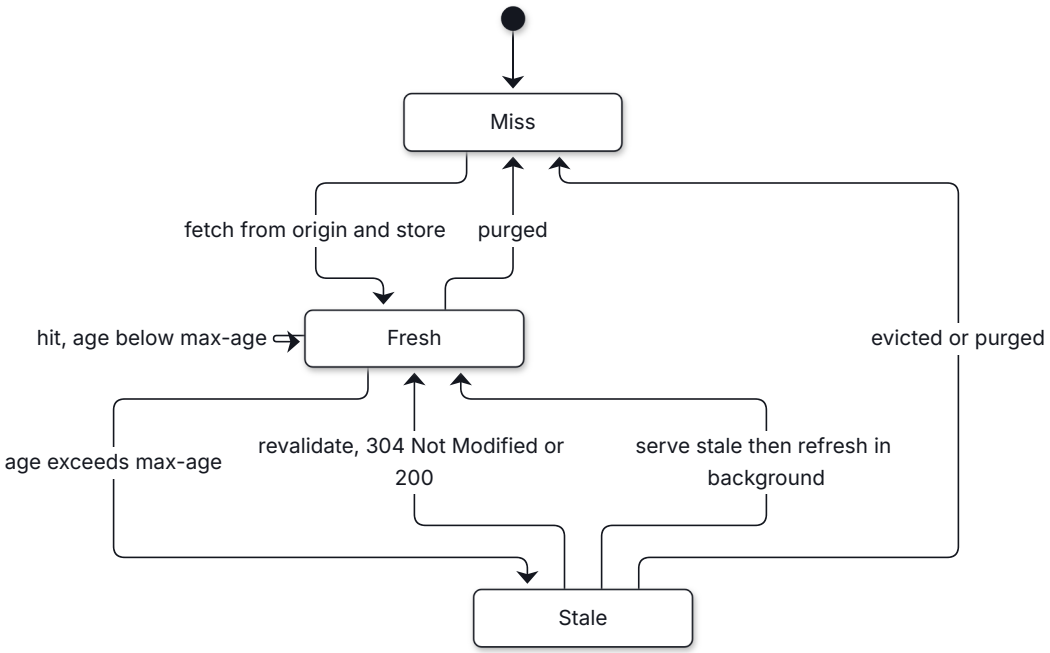


Figure 12.7-2

5 How it works

Cache-Control, the contract with caches

The origin states how a response may be stored. Directives you must know:

Directive	Meaning
`max-age=N`	Fresh for N seconds in any cache (browser and CDN)
`s-maxage=N`	Like `max-age` but for **shared** caches (CDNs, proxies) only; overrides `max-age` there
`public` / `private`	`private` means only the user's own browser may store it, never a CDN; use for per-user data
`no-store`	Never write it to any cache (sensitive data)
`no-cache`	May be stored, but must be **revalidated** with the origin before every reuse (confusing name)
`must-revalidate`	After it goes stale it must not be served without successful revalidation
`immutable`	The resource will never change at this URL; browsers skip revalidation

Directive	Meaning
`stale-while-revalidate=N`	After expiry, serve the stale copy for up to N seconds while refreshing in the background (RFC 5861)
`stale-if-error=N`	Serve stale for up to N seconds if the origin errors (RFC 5861)

Related: `ETag` and `Last-Modified` let a cache ask "has this changed?" with `If-None-Match` / `If-Modified-Since` and get a tiny `304 Not Modified`. `Vary: Accept-Encoding` tells caches the response depends on a request header, so they store a copy per variant. `Vary: Cookie` or `Vary: User-Agent` can destroy your hit ratio. Targeted headers such as `CDN-Cache-Control` / `Surrogate-Control` (RFC 9213) give the CDN different rules from the browser.

A typical policy for `shop-api`:

Content	Header	Why
Fingerprinted assets (`/static/app.3f9c1a.js`)	`public, max-age=31536000, immutable`	The filename changes when content changes, so cache forever
HTML pages	`public, max-age=0, s-maxage=60, stale-while-revalidate=300`	Short edge TTL, always fresh enough, fast under load
Product catalog API	`public, s-maxage=30, stale-if-error=600`	Hot read endpoint, tolerate origin trouble
User, cart, checkout	`private, no-store`	Per-user and sensitive; never at the edge

The cache key

The **cache key** decides whether two requests are "the same". By default it is the scheme, host, path and query string. Everything you add fragments the cache; everything you leave out risks serving the wrong content.

- **Normalise the query string.** `?utm_source=mail` and `?utm_source=ads` produce the same bytes but separate cache entries unless the CDN ignores or whitelists parameters. Sort parameters, drop tracking ones.
- **Vary by what really matters:** `Accept-Encoding` (gzip/brotli), device class or language *if the content differs*; not the whole `User-Agent` or `Cookie`.
- **Never include a cookie in the key for public content**; strip cookies on cacheable routes so they do not bypass the cache.
- **Unkeyed inputs are poisoning risks.** If the response depends on a header that is not in the key (for example `X-Forwarded-Host` reflected into the HTML), an attacker can poison the cached copy for everyone. Reflect only what is keyed.

Invalidation, purge and versioning

1. **Versioned URLs (cache busting):** the best "purge" is no purge: a new filename for new content (`app.3f9c1a.js`, built by your bundler in CI). The old objects age out harmlessly.
2. **Purge by URL, by prefix or by tag/surrogate key:** the origin tags responses (for example `Surrogate-Key: product-123 catalog`) and a deploy or admin action purges the tag. Fastly and Cloudflare do this globally in seconds; CloudFront invalidations are path-based, take longer and are limited in the free allowance (check current pricing).
3. **Soft purge / revalidate:** mark content stale rather than deleting, so users still get a fast response while it refreshes.
4. **Short TTL + stale-while-revalidate** for content that changes unpredictably.

With tiered caching, **a purge must reach every tier**: purging only the edge while the shield still holds the old object just refills the edge with old content.

Origin shield and request collapsing

Without a shield, 200 edge PoPs missing on a new object means 200 simultaneous origin requests. An **origin shield** is a designated cache tier in front of the origin: edges miss to the shield, and the shield alone talks to the origin, giving one origin fetch per object per shield instead of one per PoP. **Request collapsing** (also *cache lock* or *request coalescing*) means that when many requests for the same uncached object arrive at once, only the first goes upstream and the rest wait for its result. Together they prevent the **thundering herd** after a purge, deploy or cache expiry.

What not to cache

Responses with `Set-Cookie` per user, `Authorization`-protected content, `POST` / `PUT` / `DELETE`, and anything labelled `private` or `no-store` should not be cached by shared caches. Cache `404` and error responses very briefly (seconds) so a broken origin is not hammered, but never for long.

Beyond caching

CDNs also terminate TLS and HTTP/2/3 near the user (HTTP/2, HTTP/3 and QUIC §12.5, TLS termination §12.11), compress and transform images, run edge compute, apply WAF and rate limiting (Rate Limiting §12.13), and balance across multiple origins with health checks (Load Balancing §12.6).

6 Step-by-step guide

You will build a miniature CDN on your laptop: a slow origin, an **origin shield**, and **two edge PoPs**, all nginx caching proxies. You will watch cache status headers, see the cache key normalise query strings, collapse a thundering herd, use stale-while-revalidate, refresh and purge an object, and count how many requests reach the origin. Free and local. Verified with nginx 1.24 and Python 3.

Prerequisites: Linux with `nginx` (OSS), `python3`, `curl`. Run as an ordinary user (not root) so the cache files are yours. If your shell sets a proxy, `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY` first. Stop any packaged nginx service (`sudo systemctl disable --now nginx`). Ports used: 18081 and 18083 (edges), 18090 (shield), 19000 (origin).

1. **Create the origin.** It sleeps 200 ms per request (so you can feel cache hits), counts hits per path, and sends a different `Cache-Control` per route.

```bash
mkdir -p ~/cdn-lab/logs ~/cdn-lab/cache_edge ~/cdn-lab/cache_edge_b ~/cdn-
  lab/cache_shield && cd ~/cdn-lab
cat > origin.py <<'EOF'
import json, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

hits = {}

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def do_GET(self):
        path = self.path
        hits[path] = hits.get(path, 0) + 1
        if path == "/_hits":
            body, ctype, cc = json.dumps(hits, indent=1).encode(), "application/json",
  "no-store"
        elif path.startswith("/static/"):
            body = f"/* {path} generated at {time.strftime('%H:%M:%S')} */\n".encode()
            ctype, cc = "application/javascript", "public, max-age=60"
        elif path.startswith("/api/"):
            body = f'{{"path": "{path}", "t": "
  {time.strftime("%H:%M:%S")}"}}\n'.encode()
            ctype, cc = "application/json", "private, no-store"
        elif path.startswith("/news"):
            body = f"news page generated at {time.strftime('%H:%M:%S')}\n".encode()
            ctype, cc = "text/html", "public, max-age=5, stale-while-revalidate=30"
        else:
            body, ctype, cc = b"ok\n", "text/plain", "public, max-age=30"
        time.sleep(0.2)   # pretend the origin is slow
        self.send_response(200)
        self.send_header("Content-Type", ctype)
        self.send_header("Content-Length", str(len(body)))
```

```
        self.send_header("Cache-Control", cc)
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass

ThreadingHTTPServer(("127.0.0.1", 19000), H).serve_forever()
EOF
python3 origin.py &
sleep 1; curl -si localhost:19000/static/app.js | head -6
```

Expected: `HTTP/1.1 200 OK` and `Cache-Control: public, max-age=60`.

2. **Configure the shield and two edges.** The edges cache on `host + path` (no query string) and collapse concurrent misses; the shield is the only tier that talks to the origin. Edge B is a second PoP with a separate cache.

```bash
cat > nginx.conf <<EOF
user $USER;
worker_processes 1;
daemon off;
error_log $PWD/logs/error.log;
pid $PWD/nginx.pid;
events {}
http {
  access_log off;
  proxy_cache_path $PWD/cache_edge   levels=1:2 keys_zone=edge:10m   max_size=100m
  inactive=10m;
  proxy_cache_path $PWD/cache_edge_b levels=1:2 keys_zone=edgeb:10m  max_size=100m
  inactive=10m;
  proxy_cache_path $PWD/cache_shield levels=1:2 keys_zone=shield:10m max_size=100m
  inactive=10m;
  map \$http_x_refresh \$bypass { default 0; "1" 1; }

  server {                                    # origin shield
    listen 127.0.0.1:18090;
    location / {
      proxy_pass http://127.0.0.1:19000;
      proxy_cache shield;
      proxy_cache_key \$host\$uri;
      proxy_cache_lock on;
      proxy_cache_use_stale error timeout updating http_500 http_502 http_503;
    }
  }
  server {                                    # edge PoP A
    listen 127.0.0.1:18081;
    location / {
      proxy_pass http://127.0.0.1:18090;
      proxy_cache edge;
      proxy_cache_key \$host\$uri;              # the query string is NOT part of the
  key
```

```
      proxy_cache_lock on;                    # collapse concurrent misses
      proxy_cache_bypass \$bypass;            # X-Refresh: 1 forces a re-fetch that
  updates the cache
      proxy_cache_use_stale error timeout updating;
      proxy_cache_background_update on;
      add_header X-Cache-Status \$upstream_cache_status always;
    }
  }
  server {                                  # edge PoP B, same logic, own cache
    listen 127.0.0.1:18083;
    location / {
      proxy_pass http://127.0.0.1:18090;
      proxy_cache edgeb;
      proxy_cache_key \$host\$uri;
      proxy_cache_lock on;
      proxy_cache_bypass \$bypass;
      proxy_cache_use_stale error timeout updating;
      proxy_cache_background_update on;
      add_header X-Cache-Status \$upstream_cache_status always;
    }
  }
}
EOF
nginx -t -e $PWD/logs/error.log -c $PWD/nginx.conf       # syntax is ok / test is
  successful
nginx -e $PWD/logs/error.log -c $PWD/nginx.conf &
sleep 1
```

`nginx` honours the origin's `Cache-Control` and `Expires` by default, so no TTLs are configured here: **the origin is in charge**.

3. **See MISS then HIT.** Define a helper that prints status lines, then request a static file three times.

```bash
H="Host: shop.example.com"
h() { curl -si -H "$H" "$@" | grep -iE '^(HTTP|x-cache)' | tr -d '\r' | tr '\n' ' ';
  echo; }
for i in 1 2 3; do h localhost:18081/static/app.js; done
```

```
HTTP/1.1 200 OK X-Cache-Status: MISS
HTTP/1.1 200 OK X-Cache-Status: HIT
HTTP/1.1 200 OK X-Cache-Status: HIT
```

Time it: `curl -s -o /dev/null -w '%{time_total}\n' -H "$H" localhost:18081/static/app.js` takes a few milliseconds (HIT) versus over 0.2 s for a MISS.

4. **Cache key: query strings and private content.**

```bash
h "localhost:18081/static/app.js?utm_source=mail"      # HIT: the query string is
  ignored by our key
h localhost:18081/api/me; h localhost:18081/api/me     # MISS, MISS: "private, no-
  store" is never cached
curl -s localhost:19000/_hits | tr -d '\n '; echo
```

```
{"/static/app.js":1,"/api/me":2,"/_hits":1}
```

One origin hit for `app.js` across all requests; two for `/api/me`. If you change the key to `$host$request_uri` and reload, each distinct `?utm_...` value becomes a separate MISS: this is how tracking parameters silently wreck hit ratios.

5. **Origin shield: two PoPs, one origin fetch.**

```bash
h localhost:18081/static/shield.js        # MISS at edge A, fills the shield
h localhost:18083/static/shield.js        # MISS at edge B, but the shield answers
curl -s localhost:19000/_hits | tr -d '\n '; echo
```

Both edges report `MISS` (neither had the object) yet `/static/shield.js` appears in the origin counter only once. Without the shield tier, it would be two.

6. **Request collapsing: 20 simultaneous requests, one origin fetch.**

```bash
for i in $(seq 1 20); do curl -s -H "$H" -o /dev/null -w '%{http_code}\n'
  localhost:18081/static/herd.js & done | sort | uniq -c; wait
curl -s localhost:19000/_hits | tr -d '\n '; echo
```

```
     20 200
{...,"/static/herd.js":1,...}
```

Remove `proxy_cache_lock on;` from both edge and shield, reload (`kill -HUP $(cat nginx.pid)`) and repeat with a new filename: the origin counter will be much higher than 1. That is a thundering herd.

7. **stale-while-revalidate.** `/news` has `max-age=5, stale-while-revalidate=30`.

```bash
h localhost:18081/news; sleep 6; h localhost:18081/news; sleep 1; h
  localhost:18081/news
```

```
HTTP/1.1 200 OK X-Cache-Status: MISS
HTTP/1.1 200 OK X-Cache-Status: STALE
HTTP/1.1 200 OK X-Cache-Status: HIT
```

After expiry the user instantly gets the stale copy (`STALE`, no 200 ms wait) while nginx refreshes in the background; the next request is a fresh `HIT`.

8. **Refresh and purge.** Two ways to evict an object from an edge that has no purge API:

```bash
# a) Refresh: bypass the cache read but store the new response (restrict this header
  to trusted callers in real life)
curl -si -H "$H" -H 'X-Refresh: 1' localhost:18081/static/app.js | grep -i x-cache
  # BYPASS
h localhost:18081/static/app.js
  # HIT with the new copy

# b) Purge: delete the cache file. The file name is md5 of the cache key (host + path)
KEY="shop.example.com/static/app.js"; MD5=$(printf '%s' "$KEY" | md5sum | cut -d' ' -
  f1)
find ~/cdn-lab/cache_edge -type f -name "$MD5" -print -delete
h localhost:18081/static/app.js; h localhost:18081/static/app.js
  # MISS then HIT
curl -s localhost:19000/_hits | tr -d '\n '; echo
```

Note the origin counter for `/static/app.js` did not grow after the purge: the **shield** still had the object and refilled the edge. To really change content you must purge (or refresh) the shield too, which is the tiered-cache pitfall from section 5. (Production CDNs, and NGINX Plus's purge API, do this for you; for open-source nginx, third-party modules exist.)

9. **Clean up.**

```bash
kill $(cat ~/cdn-lab/nginx.pid); pkill -f 'python3 origin.py'; cd ~ && rm -r ~/cdn-lab
```

7 Real-world scenario

(Illustrative.) Black Friday for `shop-api`. Marketing launches an email campaign with links like `/products?utm_campaign=bf&uid=8f3a...`, where `uid` is a per-recipient identifier. The CDN cache key includes the entire query string, so every recipient generates a unique key: hit ratio drops from 95 to 12 percent, and the origin, sized for the usual 5 percent miss rate, hits its connection limit and starts returning 502s. Investigation: the CDN dashboard shows the hit-ratio collapse starting at the email send time; origin logs show thousands of requests

for the same `/products` with different `uid`s; response headers show `X-Cache: Miss from cloudfront` / `MISS` on every request. Fix: change the cache policy to include only an allow-list of query parameters (`page`, `sort`, `category`), drop `uid` and `utm_*` from the key (they still reach analytics client-side), enable origin shield and `stale-while-revalidate`, and add an alert on hit ratio below 85 percent. **Lesson:** the cache key is a production interface. Review it like an API, keep it small and monitor the hit ratio and origin request rate as first-class SLIs (Level 14 §14.1).

8 Common mistakes and best practices

Don't

- Put long TTLs on unversioned URLs (`/app.js`) and then wonder why users run last week's code.
- Cache anything containing user data without `private` / `no-store` and a checked cache key (web cache deception).
- Include cookies, `User-Agent` or all query parameters in the key for public content.
- Use `Vary: Cookie` or `Vary: *` on cacheable routes.
- Purge only one tier, or purge everything ("purge all") as a routine deploy step: it creates a thundering herd.
- Rely on a single CDN for a revenue-critical path with no way to bypass it.
- Trust forwarded headers (`X-Forwarded-Host`) in cached responses without keying them.

Do

- Fingerprint static assets and cache them for a year with `immutable`; keep HTML on short TTLs with `stale-while-revalidate` and `stale-if-error`.
- Use `s-maxage` for CDN-specific TTLs and `private` for per-user responses.
- Normalise query strings, use an allow-list of parameters, and compress at the edge (`Vary: Accept-Encoding`).
- Enable origin shield and request collapsing; size the origin for the miss path, not the hit path.
- Tag responses (surrogate keys) so you can purge precisely, and automate purges from CI/CD or admin actions.
- Monitor hit ratio, origin request rate, edge error rate, p95 latency by region and cache status; test your CDN-bypass runbook.

9 Tasks

- [ ] **T1 — Build the three-tier mini CDN** *(≈35 min, intermediate)*
  - **Goal:** observe MISS, HIT and BYPASS across edge, shield and origin.
  - **Steps:**
    1. Follow steps 1-4.
    2. Record `X-Cache-Status` and the origin counter after each request.
  - **Done when:** a table shows `app.js` fetched once from the origin for four edge requests and `/api/me` fetched every time.
- [ ] **T2 — Measure the cost of a bad cache key** *(≈25 min, intermediate)*
  - **Goal:** quantify fragmentation.
  - **Steps:**
    1. Change `proxy_cache_key` on edge A to `$host$request_uri`, reload.
    2. Send 50 requests to `/static/app.js?uid=$i` for `i` in 1..50 and read `/_hits`.
    3. Revert to `$host$uri` and repeat.
  - **Done when:** you can show 50 origin fetches versus 1 and compute the hit ratio for each.
- [ ] **T3 — Thundering herd and collapsing** *(≈30 min, advanced)*
  - **Goal:** prove `proxy_cache_lock` protects the origin.
  - **Steps:**
    1. Send 50 concurrent requests for a new URL with the lock on, then with it off (remove from edge and shield).
    2. Compare the origin count and the p99 latency of the 50 requests.
  - **Done when:** the origin sees 1 request with the lock and many without, with numbers recorded.
- [ ] **T4 — Break and fix: leaked private data** *(≈30 min, advanced)*
  - **Goal:** reproduce and prevent web cache deception.
  - **Steps:**
    1. Change `/api/` in `origin.py` to return `Cache-Control: public, max-age=60` and keep the cache key host+path, with a `Cookie`-dependent body (for example `user=` from the `Cookie` header).
    2. Request as user A, then as user B, and see B receive A's body from the cache.

3. Fix it three ways: `private`, `Vary: Cookie` (and note the hit-ratio cost), and bypassing the cache when a cookie is present (`proxy_cache_bypass $http_cookie; proxy_no_cache $http_cookie;`).

- **Done when:** you have shown the leak, and shown B no longer sees A's data after each fix.

☐ **T5 — Purge strategy for a release** *(≈40 min, advanced)*

- **Goal:** design and automate invalidation.
- **Steps:**
  1. Serve `/static/app.js` with a 1-hour TTL; change the content at the origin and show users still see the old file.
  2. Fix it twice: purge the file on both edge and shield; then switch to a versioned filename (`app.<hash>.js`) generated by a small script.
  3. Write the deploy step list (build, upload, purge or version, verify).
- **Done when:** both fixes work and you can say why versioning needed no purge.

☐ **T6 — Prove it: cache policy for `shop-api`** *(≈45 min, advanced)*

- **Goal:** produce and defend a cache design.
- **Steps:**
  1. Without the guide, rebuild edge + shield + origin and demonstrate MISS, HIT, collapse and stale-while-revalidate.
  2. Write the policy table for static assets, HTML, catalog API and cart/checkout: headers, key, purge method and the metrics you alert on.
- **Done when:** the demo works from memory and the table has no route that could cache private data.

10 Self-check questions

1. What is the difference between `no-cache` and `no-store`?
2. A deploy shipped broken JavaScript and users still see it an hour after you fixed it. What are the two likely causes and the long-term fix?
3. Why can adding `Vary: Cookie` or including all query parameters in the key lower your hit ratio drastically?
4. What problem does an origin shield solve, and what is request collapsing?
5. How could a cache be poisoned through an unkeyed header, and how do you prevent it?
6. Why is purging only the edge insufficient with tiered caching?

ANSWERS

1. `no-store`: nothing may be stored anywhere. `no-cache`: it may be stored but must be revalidated with the origin before each reuse (often resulting in a cheap 304).
2. The old file was cached under a long TTL at the edge/browser (and perhaps the shield); purge the URL on all tiers and tell browsers to revalidate. Long term: fingerprint asset filenames (new content, new URL) and keep HTML short-lived so it points at the new hashes.
3. Every distinct cookie or query value creates its own cache entry, so almost every request is unique and misses.
4. Many edges missing at once would each hit the origin; the shield makes one regional tier fetch from the origin on their behalf. Request collapsing makes concurrent requests for the same uncached object wait for a single upstream fetch instead of each going upstream.
5. If the response varies on a header that is not part of the key (for example reflecting `X-Forwarded-Host` into HTML), an attacker sends a crafted header once and the poisoned response is cached for everyone. Prevent by not reflecting unkeyed inputs, adding the needed headers to the key, stripping unexpected ones at the edge, and validating input.
6. The shield still holds the old object and will refill the edge with it; you must purge every tier (or use a CDN purge that is tier-aware) or use versioned URLs.

11 Further reading

- RFC 9111 (HTTP Caching), RFC 5861 (`stale-while-revalidate`, `stale-if-error`), RFC 9213 (Targeted HTTP Cache Control).
- MDN Web Docs: HTTP caching and the `Cache-Control` header.
- nginx documentation for `ngx_http_proxy_module` (`proxy_cache*` directives).
- Cloudflare, Fastly and Amazon CloudFront documentation on cache keys, tiered caching/origin shield, purging and cache tags.
- Netflix Open Connect documentation and Netflix Tech Blog posts on content delivery.
- James Kettle (PortSwigger), research on web cache poisoning; Omer Gil, original web cache deception write-up.

LAB NOTES What I ran, what I saw, what it means

12.8 VPNs

Topic 8 of 14 | ~4 hours | Difficulty: Advanced

1 What it is

A **Virtual Private Network (VPN)** builds an encrypted, authenticated tunnel across an untrusted network (usually the Internet) so that two endpoints behave as if they were on the same private network. Packets from the private network are wrapped (**encapsulated**) inside outer packets addressed between the tunnel endpoints, encrypted, sent across the Internet, then unwrapped at the other end. The inner traffic keeps its private addresses (RFC 1918, see IPv4, IPv6, CIDR and Subnets §2.3).

Two shapes matter in DevOps:

- **Site-to-site VPN:** a gateway at each site (your data centre and an AWS VPC, or two clouds) is permanently connected, and everything behind each gateway can reach the other side. No user action is involved.
- **Client (remote-access) VPN:** one person's laptop or one CI runner connects to a VPN server and gets access to a private network, usually after authenticating with a key, certificate or single sign-on.

Three protocol families dominate:

Protocol	Layer and transport	Where you meet it
IPsec (IKEv2 + ESP)	Network layer; IKE on UDP 500/4500, ESP as IP protocol 50 (or in UDP 4500 for NAT traversal)	AWS Site-to-Site VPN, Azure/GCP VPN gateways, hardware firewalls, strongSwan, Libreswan
WireGuard	Network layer virtual interface; UDP	Linux kernel since 5.6, Tailscale, many cloud and self-hosted VPNs
OpenVPN (TLS-based)	Userspace; UDP or TCP 1194 or 443	AWS Client VPN, older remote-access setups

2 Why it matters in DevOps

- **Hybrid and multi-cloud connectivity.** A VPN is the quickest way to link an office, a data centre or another cloud to your VPC (Level 6, VPC §6.4) without waiting for a dedicated circuit, and it is the standard backup path for a Direct Connect link (BGP §12.2).
- **Private access to private systems.** Databases, Kubernetes API servers and admin consoles should not have public IP addresses. How engineers reach them (bastion, VPN, zero-trust proxy) is an architecture decision with security and productivity consequences.
- **Subtle failure modes.** Overlapping CIDR ranges, MTU/MSS problems ("ping works but big requests hang"), asymmetric routing, expired pre-shared keys and one-sided rekeys cause outages that look like application bugs.
- **Security posture.** A flat VPN that places every user on the whole network is lateral movement waiting to happen. The trend is least-privilege, identity-aware access (Level 11 §11.2).

3 Real-world system example

AWS Site-to-Site VPN gives each VPN connection two IPsec tunnels terminating in different Availability Zones on the AWS side, so a single tunnel endpoint maintenance event does not cut connectivity. It supports static routes or dynamic routing with BGP, attaches to a virtual private gateway or a Transit Gateway, and each tunnel has a documented bandwidth limit (about 1.25 Gbps); equal-cost multipath over several tunnels via Transit Gateway scales it. **AWS Client VPN** is a managed OpenVPN-based service with certificate, Active Directory or SAML authentication.

Tailscale builds a mesh VPN on top of **WireGuard**: devices authenticate through your identity provider, a coordination server distributes public keys and policy, and devices connect directly to each other, using NAT traversal techniques and relays (DERP servers) as a fallback. It is a practical example of the shift from "one big VPN concentrator" to identity-based, peer-to-peer access.

Google's BeyondCorp (published in a series of papers from 2014 on) is the canonical **zero-trust** design: access to internal applications depends on user identity and device state through an access proxy, not on being "inside the network", which removed the need for a traditional corporate VPN for most applications. Commercial equivalents include Cloudflare Access, AWS Verified Access and Google Identity-Aware Proxy.

WireGuard itself was merged into the mainline Linux kernel in version 5.6 (2020) and is used by many providers and tools for its small codebase and simple configuration.

4 Diagram

A site-to-site VPN between a data centre and the `shop-api` VPC (what the lab below builds, in miniature):

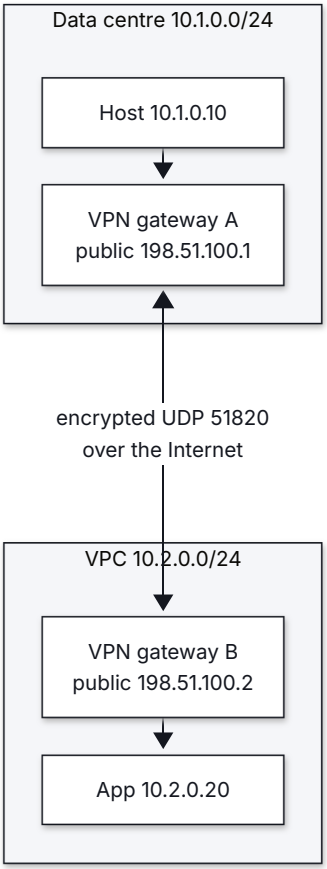


Figure 12.8-1

Three ways to give engineers access to a private database, ordered by increasing trust placed in the identity layer:

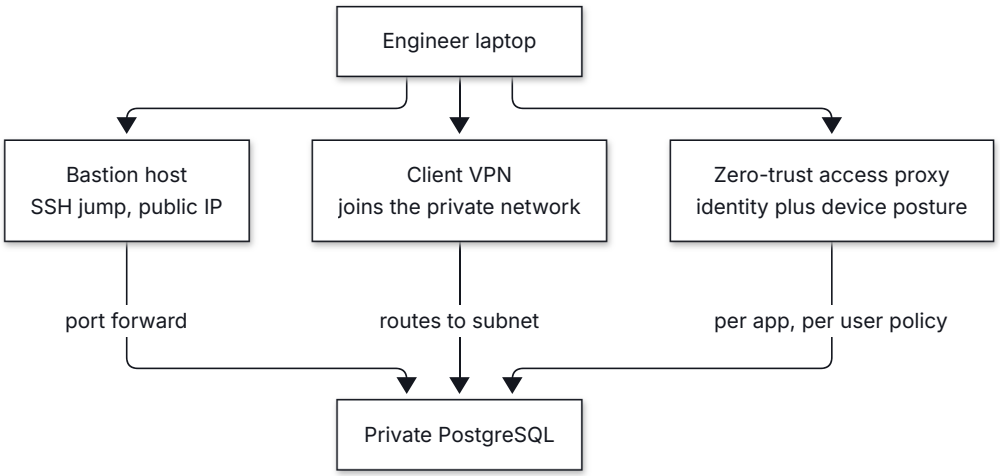


Figure 12.8-2

5 How it works

Tunnels, encapsulation and routing

A VPN is a virtual network interface plus routes. Traffic for a remote prefix is routed to the interface, the VPN software encrypts it and sends it as UDP (or ESP) to the peer's public address, and the peer decrypts it and injects it into its own network. Because the outer header and encryption add overhead, the effective **MTU** shrinks (WireGuard's default is 1420 on a 1500-byte path). If your TCP path MTU discovery is broken (ICMP blocked), large packets vanish and sessions hang after the handshake. The fix is a lower tunnel MTU or **MSS clamping** (`iptables -t mangle ... --clamp-mss-to-pmtu`).

Two routing styles exist:

- **Policy-based** (older IPsec): the tunnel carries only traffic matching selectors such as `10.1.0.0/24 ⟷ 10.2.0.0/24`.
- **Route-based:** the tunnel is an interface and the routing table (static routes or BGP) decides what goes in. This is more flexible and is what cloud VPN gateways, WireGuard and modern IPsec use.

IPsec in a nutshell

1. **IKE (Internet Key Exchange, v2)** authenticates the peers (pre-shared key or certificates) and negotiates keys and algorithms. This creates the IKE security association (SA), then one or more **child SAs**.
2. **ESP (Encapsulating Security Payload)** carries the encrypted, authenticated packets. **Tunnel mode** wraps the whole original packet (the normal VPN case); transport mode protects only the payload.
3. **NAT traversal (NAT-T)** wraps ESP in UDP 4500 when a NAT device sits in the path.
4. Algorithms are negotiated from long lists (AES-GCM, SHA-2, DH groups), which gives compatibility with almost every vendor and a large configuration surface (and mismatch errors like `NO_PROPOSAL_CHOSEN`). Rekeying happens periodically, and **dead peer detection** (DPD) tears down broken tunnels.

WireGuard in a nutshell

- Each peer has a **key pair** (Curve25519). You exchange *public* keys, like SSH `authorized_keys`. There is no negotiation: one fixed modern suite (Noise protocol, ChaCha20-Poly1305, BLAKE2s) means very little to misconfigure.
- **Cryptokey routing.** Each peer has an `AllowedIPs` list. Outbound, it is a routing table: packets for those prefixes go to that peer. Inbound, it is an access list: a decrypted packet from the peer is accepted only if its source IP is in that list. You will test this.
- It is **silent**: a peer ignores packets not authenticated with a known key, so an unauthenticated scanner gets no reply. It supports **roaming** (the endpoint address updates when the peer's IP changes) and rekeys frequently (about every two minutes of activity).
- It has no built-in user management, address assignment or dynamic routing; tools like Tailscale, Netmaker or your own automation add them. For BGP over WireGuard use a routing daemon on the tunnel interface.
- `PersistentKeepalive = 25` keeps NAT mappings alive for peers behind NAT.

Comparison

	IPsec (IKEv2)	WireGuard	OpenVPN
Typical use	Site-to-site with clouds and appliances	Fast site-to-site and client VPN, mesh	Client VPN with legacy compatibility
Interop	Universal, every cloud and vendor	Growing, not offered by all cloud-native VPN gateways	Wide client support
Config complexity	High (proposals, selectors, lifetimes)	Very low	Medium

	IPsec (IKEv2)	WireGuard	OpenVPN
Performance	Good, kernel, hardware offload	Very good, kernel	Lower (userspace, TLS)
Crypto agility	Many suites, negotiated	One fixed modern suite (change = new protocol version)	Many via OpenSSL
Identity	PSK or X.509 certificates	Static public keys (add your own layer)	Certificates, username/password, SSO plugins
Passes restrictive firewalls	UDP 500/4500	UDP only	Can use TCP 443

Bastion vs VPN vs zero trust

	Bastion (jump host)	VPN	Zero-trust access proxy
Unit of access	One host or port at a time (SSH)	Whole network or subnets	Specific application per user
Exposure	One public host to harden and patch	VPN endpoint(s) exposed	Identity provider and proxy (managed or self-hosted)
Identity and audit	SSH keys or certs; session logs need extra work	Often only "connected"; weak per-resource audit	Per-request identity, device posture, central audit
Network position	Trusted once you can reach the bastion	Trusted once connected (lateral movement risk)	No implicit trust from network location
Good for	Small teams, break-glass access, SSH to private servers (Level 1, SSH §1.13)	Site-to-site, legacy apps that need real network access	Web apps, SSH/DB access with SSO, remote workforce at scale

Modern practice combines them: site-to-site VPN or private links for networks, a hardened bastion or cloud-native session service (for example AWS Systems Manager Session Manager, no inbound ports) for admin shells, and an identity-aware proxy for human access to internal apps. Whatever you choose, keep the VPN's reach narrow with routes, security groups and firewall rules (Level 6, Security Groups §6.6).

6 Step-by-step guide

You will build a site-to-site WireGuard VPN between two simulated sites on one machine using Linux network namespaces, so there is nothing to buy and nothing touches your real network. A pair of namespaces plays the two gateways, a virtual cable plays the Internet, and each gateway has a private LAN address.

Prerequisites: Linux with root (`sudo`), `iproute2`, `wireguard-tools`, `tcpdump`, `iputils-ping`, `curl`, `python3`: `sudo apt-get install -y wireguard-tools tcpdump iputils-ping curl`. The Linux kernel has had WireGuard since 5.6. If `ip link add x type wireguard` says `Unknown device type` (some containers and sandboxes), also install the userspace implementation, `sudo apt-get install -y wireguard-go`; the script below falls back to it automatically (it needs `/dev/net/tun`). Verified with `wireguard-go` on Ubuntu 24.04; the kernel path uses the same `wg` commands.

1. **Build the topology.** Namespace `siteA` is the data centre gateway (LAN `10.1.0.0/24`), `siteB` is the VPC gateway (LAN `10.2.0.0/24`). Their "public" addresses are 198.51.100.1 and 198.51.100.2 (documentation range).

   ```bash
   mkdir -p ~/vpn-lab && cd ~/vpn-lab
   sudo ip netns add siteA; sudo ip netns add siteB
   sudo ip link add vethA type veth peer name vethB          # the "Internet"
   sudo ip link set vethA netns siteA; sudo ip link set vethB netns siteB
   sudo ip -n siteA addr add 198.51.100.1/24 dev vethA; sudo ip -n siteB addr add
     198.51.100.2/24 dev vethB
   for n in siteA siteB; do sudo ip -n $n link set lo up; done
   sudo ip -n siteA link set vethA up; sudo ip -n siteB link set vethB up
   sudo ip -n siteA addr add 10.1.0.1/24 dev lo            # a host address standing in
     for LAN A
   sudo ip -n siteB addr add 10.2.0.1/24 dev lo            # a host address standing in
     for LAN B
   sudo ip netns exec siteA ping -c1 -W1 198.51.100.2      # underlay reachability
   ```

   Expected: `1 packets transmitted, 1 received`.

2. **Generate key pairs and a pre-shared key.** Each side keeps its private key secret and shares only the public key. The optional pre-shared key (PSK) adds a layer of symmetric protection against future cryptanalysis.

   ```bash
   umask 077
   for n in a b; do wg genkey > $n.key; wg pubkey < $n.key > $n.pub; done
   wg genpsk > psk
   cat a.pub b.pub
   ```

3. **Create the WireGuard interfaces and peers.** `AllowedIPs` lists what each side will send to,

and accept from, its peer: the peer's tunnel address and the LAN behind it.

```bash
sudo ip -n siteA link add wgA type wireguard 2>/dev/null || sudo ip netns exec siteA
  wireguard-go wgA
sudo ip -n siteB link add wgB type wireguard 2>/dev/null || sudo ip netns exec siteB
  wireguard-go wgB
sudo ip -n siteA addr add 10.99.0.1/30 dev wgA; sudo ip -n siteB addr add 10.99.0.2/30
  dev wgB
sudo ip netns exec siteA wg set wgA private-key a.key listen-port 51820 \
    peer $(cat b.pub) preshared-key psk endpoint 198.51.100.2:51820 \
    allowed-ips 10.99.0.2/32,10.2.0.0/24 persistent-keepalive 25
sudo ip netns exec siteB wg set wgB private-key b.key listen-port 51820 \
    peer $(cat a.pub) preshared-key psk endpoint 198.51.100.1:51820 \
    allowed-ips 10.99.0.1/32,10.1.0.0/24 persistent-keepalive 25
sudo ip -n siteA link set wgA up; sudo ip -n siteB link set wgB up
sudo ip -n siteA route add 10.2.0.0/24 dev wgA            # send LAN B traffic into
  the tunnel
sudo ip -n siteB route add 10.1.0.0/24 dev wgB
```

4. **Send traffic through the tunnel and read the status.** Source the ping from the LAN A address so it behaves like a host on that LAN.

```bash
sudo ip netns exec siteA ping -c2 -I 10.1.0.1 10.2.0.1
sudo ip netns exec siteA wg show wgA
```

```
64 bytes from 10.2.0.1: icmp_seq=1 ttl=64 time=0.8 ms
...
peer: <public key of B>
  endpoint: 198.51.100.2:51820
  allowed ips: 10.99.0.2/32, 10.2.0.0/24
  latest handshake: 1 second ago
  transfer: 436 B received, 528 B sent
```

`latest handshake` is the single best health indicator of a WireGuard link: if it is missing or older than a few minutes while traffic should be flowing, the tunnel is down.

5. **See what an eavesdropper sees, and what the tunnel carries.** Capture on the "Internet" link and on the tunnel interface at the same time.

```bash
sudo ip netns exec siteA timeout 4 tcpdump -i vethA -n -c 4 &
sudo ip netns exec siteA timeout 4 tcpdump -i wgA -n -c 2 &
sleep 1; sudo ip netns exec siteA ping -c2 -I 10.1.0.1 10.2.0.1 >/dev/null; wait
```

```
# on vethA (what the Internet sees)
IP 198.51.100.1.51820 > 198.51.100.2.51820: UDP, length 128
IP 198.51.100.2.51820 > 198.51.100.1.51820: UDP, length 128
# on wgA (what the hosts see)
IP 10.1.0.1 > 10.2.0.1: ICMP echo request, id 30049, seq 1, length 64
IP 10.2.0.1 > 10.1.0.1: ICMP echo reply, id 30049, seq 1, length 64
```

The Internet sees only UDP between the two gateways; the private addresses and ICMP exist only inside the tunnel.

6. **Run a real application across the VPN.** Start a web server on LAN B and fetch it from LAN A.

```bash
sudo ip netns exec siteB python3 -m http.server 8080 --bind 10.2.0.1 >/dev/null 2>&1 &
sleep 1
sudo ip netns exec siteA curl -s --interface 10.1.0.1 -o /dev/null -w '%{http_code}\n'
  http://10.2.0.1:8080/
```

Expected: `200`.

7. **Prove cryptokey routing: AllowedIPs is both routing and firewall.** Add a route for a prefix that is not in A's AllowedIPs for B and try to reach it.

```bash
sudo ip -n siteA route add 10.3.0.0/24 dev wgA
sudo ip netns exec siteA ping -c1 -W1 -I 10.1.0.1 10.3.0.1       # fails: no peer owns
  10.3.0.0/24
sudo ip netns exec siteA wg set wgA peer $(cat b.pub) allowed-ips
  10.99.0.2/32,10.2.0.0/24,10.3.0.0/24
sudo ip netns exec siteA wg show wgA allowed-ips                  # now includes
  10.3.0.0/24
```

Before the change, WireGuard has no peer to send `10.3.0.0/24` to and drops it (the kernel implementation reports `Required key not available`). After you add it to the peer's AllowedIPs it is encrypted and sent to B; B will still drop it unless its own AllowedIPs list for A includes the packet's source, which is the inbound access check.

8. **A client VPN uses the same mechanics.** A road-warrior laptop is just a peer with one address and a gateway peer that lists it. Write `wg0.conf` for wg-quick (placeholders only; do

not activate here):

```ini
# laptop: /etc/wireguard/wg0.conf  (generate your own keys; never commit private keys)
[Interface]
Address = 10.99.0.10/32
PrivateKey = <LAPTOP_PRIVATE_KEY>
DNS = 10.2.0.2

[Peer]
PublicKey = <GATEWAY_PUBLIC_KEY>
Endpoint = vpn.example.com:51820
AllowedIPs = 10.2.0.0/24        # split tunnel: only the VPC range goes through the
  VPN
PersistentKeepalive = 25
```

`AllowedIPs = 0.0.0.0/0` would instead send all traffic through the VPN (full tunnel). On the gateway, add a `[Peer]` for the laptop with `AllowedIPs = 10.99.0.10/32`, enable IP forwarding (`sysctl -w net.ipv4.ip_forward=1`), and allow the traffic in the gateway's firewall.

9. **Clean up.**

```bash
sudo pkill wireguard-go 2>/dev/null; sudo pkill -f 'http.server 8080' 2>/dev/null
sudo ip netns del siteA; sudo ip netns del siteB
cd ~ && rm -r ~/vpn-lab
```

Deleting the namespaces also removes the veth pair and kernel WireGuard interfaces.

For IPsec, the practical equivalent is a cloud-managed gateway: create a virtual private gateway and a customer gateway in a free-tier-aware sandbox, download the vendor configuration and apply it to strongSwan. Note that AWS Site-to-Site VPN is billed per connection-hour plus data transfer, so delete it promptly (Level 6 §6.5).

7 Real-world scenario

(Illustrative.) `shop-api` runs in a VPC (`10.20.0.0/16`) and an acquired company's data centre (`10.20.0.0/16`, the same range by coincidence) must now be linked over a Site-to-Site VPN for a data migration. The tunnels come up, BGP is established, yet nothing can talk across: both sides think `10.20.x.x` is local. Investigation: `ip route get 10.20.4.15` on a VPC instance shows the local subnet route; the VPN's routes are shadowed by the identical connected route. Renumbering either network would take months. Fix: a NAT layer in the tunnel: the data centre's hosts are presented to the VPC as `10.120.0.0/16` through one-to-one NAT

on the VPN gateway, and DNS entries for the migration point to the translated range. After that, a second problem appears: bulk transfers stall at about 1.4 KB because an on-prem firewall drops the ICMP "fragmentation needed" messages; the team lowers the tunnel MTU and clamps TCP MSS. **Lesson:** plan non-overlapping address space before connecting networks (Level 6, VPC §6.4), and always test MTU with large packets (`ping -M do -s 1372`) rather than only a single small ping.

8 Common mistakes and best practices

Don't

- Interconnect networks with overlapping CIDR ranges and hope routing sorts it out.
- Rely on a single VPN tunnel or a single VPN gateway as your only path to production.
- Give every VPN user a route to the entire network or an "allow all" firewall rule.
- Commit private keys or pre-shared keys to Git, or share one key among many people.
- Ignore MTU: broken PMTUD yields hangs that only affect large responses.
- Leave a bastion on a public IP with password SSH, or forget to patch it.
- Treat "connected to VPN" as "authenticated and authorised".

Do

- Plan IP space centrally, and prefer route-based tunnels with BGP for site-to-site links.
- Run at least two tunnels in different failure domains and test failover.
- Use separate keys per device or peer, rotate them, and revoke departed users promptly.
- Restrict reach with narrow `AllowedIPs`, security groups and firewall rules; log connections.
- Prefer identity-aware access (SSO, MFA, device posture) for humans, and short-lived credentials for automation.
- Monitor tunnel state (`latest handshake`, BGP session, bytes, packet loss, latency) and alert on a down tunnel.
- Document DNS: how private names resolve for VPN users.

9 Tasks

- [ ] **T1 — Build the site-to-site tunnel** *(≈35 min, intermediate)*
  - **Goal:** connect two private LANs over an untrusted link.
  - **Steps:**
    1. Follow steps 1-4 of the guide.

2. Ping both directions and capture `wg show` output.

- **Done when:** pings succeed in both directions and `latest handshake` shows seconds ago.

☐ **T2 — Prove the traffic is encrypted** *(≈20 min, intermediate)*

- **Goal:** compare underlay and overlay captures.
- **Steps:**
  1. Run step 5.
  2. Then fetch the web page (step 6) with `tcpdump -A` on `vethA` and search the capture for the string "Directory listing".
- **Done when:** the string appears on `wgA` but never on `vethA`, and you can explain why packet length differs from the ping payload.

☐ **T3 — Break it: revoke a peer, then fix an AllowedIPs bug** *(≈30 min, advanced)*

- **Goal:** practise failure diagnosis.
- **Steps:**
  1. On B, remove A (`wg set wgB peer $(cat a.pub) remove`) and observe the failure from A's side (`wg show` handshake ages).
  2. Re-add the peer but with a typo in `AllowedIPs` (`10.1.0.0/25`), find which hosts fail and why.
  3. Write the three commands you would run first in an incident.
- **Done when:** you restore connectivity and can explain each symptom from `wg show` and `tcpdump` output.

☐ **T4 — MTU and MSS** *(≈30 min, advanced)*

- **Goal:** see the black hole and fix it.
- **Steps:**
  1. Find the tunnel MTU (`ip -n siteA link show wgA`).
  2. Ping with the Don't Fragment bit and increasing sizes: `ping -M do -s 1300 -I 10.1.0.1 10.2.0.1` up to the limit; note the largest that works.
  3. Lower the tunnel MTU on both ends (`ip link set dev wgA mtu 1280`) and repeat.
- **Done when:** you can state the relationship between the MTU and the largest payload that passes (MTU minus 28 bytes of IP and ICMP headers).

☐ **T5 — Add a third site and a road-warrior client** *(≈45 min, advanced)*

- **Goal:** extend to a hub-and-spoke topology.
- **Steps:**

1. Add `siteC` with LAN `10.3.0.0/24` peering with `siteA` as hub; configure routes so C can reach B through A (A forwards: `sysctl -w net.ipv4.ip_forward=1` in the namespace; AllowedIPs on each side must list the remote LANs).
2. Ping C to B and read the AllowedIPs tables.

- **Done when:** C reaches B via A and you can explain why WireGuard needs every transit prefix in AllowedIPs.

☐ **T6 — Prove it: choose an access model for `shop-api`** *(≈45 min, advanced)*

- **Goal:** justify a design.
- **Steps:**
  1. Without the guide, rebuild the two-site tunnel from memory.
  2. Write one page choosing between bastion, client VPN and zero-trust access for: (a) a developer querying the production replica, (b) a CI runner deploying to a private cluster, (c) the data centre to VPC link. State the protocol, authentication, audit trail and blast radius for each.
- **Done when:** the tunnel works from memory and the page names a different, justified mechanism for at least two of the three cases.

10 Self-check questions

1. What is the difference between a site-to-site VPN and a client VPN?
2. What does `AllowedIPs` do in WireGuard, in both directions?
3. Name two reasons why IPsec is still common in cloud and enterprise environments despite being harder to configure.
4. A VPN "works" for ping but large file downloads hang. What is the likely cause and fix?
5. What are the weaknesses of a flat VPN compared with zero-trust access?
6. Why do overlapping CIDRs break VPN connectivity, and what are the options?

ANSWERS

1. Site-to-site connects two networks permanently via gateways, transparently to hosts; a client VPN connects an individual device to a network on demand, usually with user authentication.
2. Outbound it is a routing table (which peer to send a destination to); inbound it is an access list (a decrypted packet is accepted only if its source address is within the peer's AllowedIPs).

3. Universal interoperability (every cloud gateway, firewall and router vendor supports it) and mature features such as certificate auth, dynamic routing with BGP, hardware offload and compliance acceptance.
4. MTU/MSS: the tunnel overhead makes full-size packets too large and fragmentation-needed ICMP is blocked, so PMTUD fails. Lower the tunnel MTU and/or clamp the TCP MSS, and allow the ICMP messages.
5. Connecting grants broad network access (lateral movement), authorisation is coarse, auditing is poor per application, and device health is not considered; zero trust authorises per request based on identity and posture.
6. The local connected route wins, so the remote network is unreachable. Renumber, or use NAT (one-to-one translation) at the VPN edge, or reorganise with a different non-overlapping range for the shared segment.

11 Further reading

- WireGuard documentation and whitepaper (wireguard.com), and `man wg`, `man wg-quick`.
- RFC 7296 (IKEv2), RFC 4303 (ESP), RFC 3948 (UDP encapsulation of ESP); strongSwan documentation.
- AWS documentation: Site-to-Site VPN, Client VPN, Transit Gateway; Systems Manager Session Manager.
- Google, "BeyondCorp: A New Approach to Enterprise Security" and follow-up papers; the Tailscale blog "How Tailscale works".
- Jason A. Donenfeld, WireGuard papers; Michael W. Lucas, *Networking for System Administrators*.

12.9 Service Mesh

Topic 9 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

A **service mesh** is a dedicated infrastructure layer that handles service-to-service ("east-west") communication so that application code does not have to. It moves cross-cutting network concerns, namely **mutual TLS (mTLS) identity and encryption, retries, timeouts, circuit breaking, load balancing, traffic splitting and telemetry**, out of every application and library and into proxies that sit on the request path and are configured centrally.

A mesh has two parts:

- The **data plane**: the proxies that carry the traffic. Most meshes use **Envoy** (Istio, Consul, Cilium's L7 mode, many gateways) or a purpose-built proxy (Linkerd's Rust `linkerd2-proxy`). They terminate and originate mTLS, apply routing and resilience policy, and emit metrics, logs and traces.
- The **control plane**: the software that decides what the proxies should do. It watches your platform (for example the Kubernetes API), issues short-lived workload certificates, and pushes configuration to the proxies over a streaming API (Envoy's **xDS** protocols: LDS listeners, RDS routes, CDS clusters, EDS endpoints, SDS secrets).

There are two deployment models:

Model	How the proxy is placed	Used by
Sidecar	One proxy container per pod/workload, sharing its network namespace; traffic is redirected through it (iptables or eBPF)	Classic Istio, Linkerd, Consul
Ambient / sidecarless	A shared per-node L4 proxy handles mTLS and identity for all pods on the node, with optional shared L7 "waypoint" proxies per namespace or service	Istio ambient mode (ztunnel + waypoint), Cilium (eBPF in the kernel plus per-node Envoy)

2 Why it matters in DevOps

- **Security by default.** A mesh can give every workload a cryptographic identity and encrypt and authenticate all in-cluster traffic without changing application code (mTLS §12.12, Level 11 §11.8).
- **Uniform resilience and observability.** Timeouts, retries with budgets, circuit breaking and the "golden signals" (Level 10 §10.11) appear consistently for services written in any language (Circuit Breakers §12.14).
- **Safer releases.** Weighted routing, header-based routing and mirroring make canary and blue/green strategies a configuration change (Level 5, Canary §5.12).
- **But it is a significant operational commitment.** A mesh adds latency, CPU and memory per workload, new failure modes (a broken control plane or expired root certificate can take down all service-to-service traffic), a new upgrade lifecycle and a new debugging surface. The honest question is "is it worth it for us?", answered in section 5.

3 Real-world system example

Istio (started in 2017 by Google, IBM and Lyft, and a graduated CNCF project) uses Envoy as its data plane, with the `istiod` control plane distributing xDS configuration and issuing workload certificates (SPIFFE-style identities, see mTLS §12.12). Its **ambient mode** replaces per-pod sidecars with a per-node `ztunnel` for mTLS and L4 policy plus optional **waypoint proxies** for L7 features, aimed at cutting the resource overhead and operational friction of sidecars; it has been declared production-ready in recent Istio releases (check the current release notes for status in your version). **Envoy** itself came from Lyft, which built it to make a polyglot microservice fleet observable and resilient with one consistent proxy; it is now a CNCF graduated project used far beyond meshes (ingress gateways, API gateways, cloud load balancers). **Linkerd** (CNCF graduated) emphasises simplicity: a small Rust proxy per pod, automatic mTLS and golden-signal metrics out of the box. **Cilium** implements mesh features in the kernel with eBPF and per-node Envoy. **Consul** (HashiCorp) provides service discovery plus a mesh with sidecar proxies that works across VMs and Kubernetes. Large platforms such as Lyft and others have publicly described meshes as the way to apply uniform policy across hundreds of services; smaller teams often report that the cost exceeded the benefit, which is why the "when is it worth it" question matters. For the Kubernetes-specific view see Level 16: Service Mesh with Istio or Linkerd §16.9 and Mesh traffic management §16.10.

4 Diagram

Sidecar data plane and control plane. The application only talks to `localhost`; the sidecars do mTLS, routing and telemetry; the control plane pushes configuration and certificates:

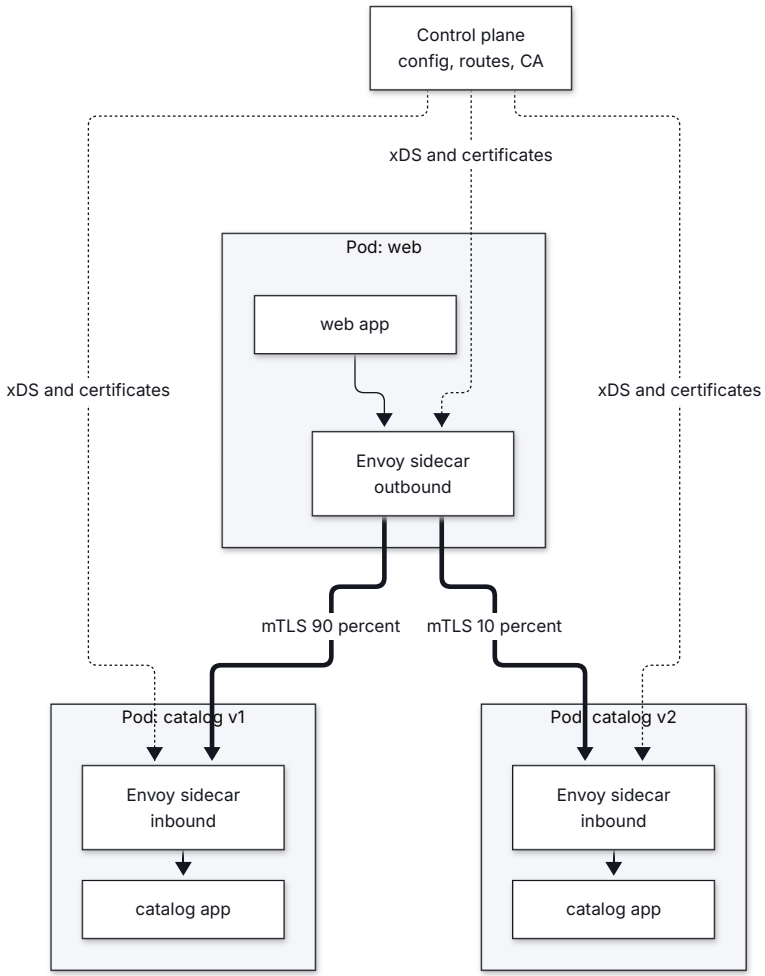


Figure 12.9-1

Sidecar versus ambient, in terms of where proxies run:

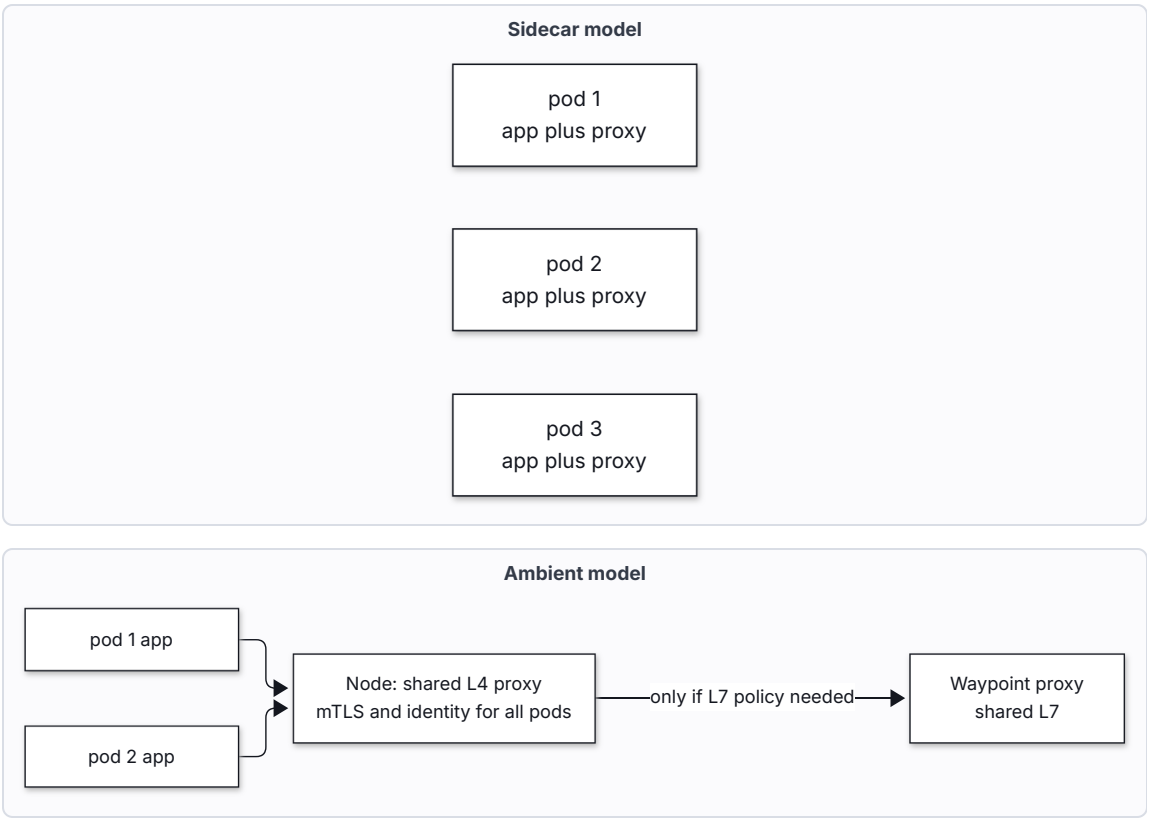


Figure 12.9-2

5 How it works

Request path with sidecars

1. Traffic redirection (iptables rules inserted by an init container, or eBPF/CNI) sends the app's outbound connection to the local sidecar's outbound listener instead of directly to the destination.
2. The sidecar resolves the logical destination (`catalog.shop.svc`) to endpoints it received from the control plane (EDS), applies routing rules (path or header match, weighted clusters), picks an endpoint with a load-balancing policy (Load Balancing §12.6), opens an **mTLS** connection presenting its own workload certificate, and verifies the peer's identity.
3. The destination's sidecar terminates mTLS, authorises the caller (is this identity allowed to call this service and path?), and forwards plain HTTP to the app on `localhost` .

4. Both sidecars record metrics (rate, errors, latency per route and per source/destination identity), access logs and trace spans, and apply timeouts, retries and circuit-breaking limits (Connection Pooling §12.10).

What the control plane does

Function	Detail
Service discovery	Watches the platform's endpoints and keeps each proxy's cluster membership current
Configuration translation	Turns high-level resources (Istio `VirtualService` / `DestinationRule`, Gateway API `HTTPRoute`, Linkerd `ServiceProfile`/policies) into xDS
Certificate authority	Issues short-lived workload certificates (hours) with an identity such as `spiffe://example.com/ns/shop/sa/web` and rotates them automatically
Policy	Distributes authorisation rules (who can call what) enforced in the proxies
Observability wiring	Configures metric, log and trace emission

The data plane keeps working if the control plane is briefly down (proxies keep their last configuration and certificates), but certificates expire, so a long control-plane outage becomes a data-plane outage. Treat the control plane as tier-0 infrastructure.

Sidecar vs ambient

	Sidecar	Ambient / sidecarless
Resource cost	Proxy per pod: memory and CPU scale with pod count	Per-node L4 proxy; L7 proxies only where needed
Isolation	Strong per-workload (own proxy, own failure domain)	Shared proxies: noisy neighbour and larger blast radius per node
Onboarding	Pod restart to inject the sidecar	No pod restart; enable by namespace label
Features	Full L7 on every hop	L4 (mTLS, identity, TCP policy) cheap; L7 requires a waypoint hop
Maturity and tooling	Longest track record	Newer; evolving quickly; check your mesh's current status
Debugging	Per-pod proxy logs and admin endpoint	Node-level and waypoint logs

When is a service mesh worth it?

Worth considering when **most** of these are true:

- Dozens of services owned by many teams, in several languages (so a per-language library cannot enforce policy).

- A requirement for **mTLS everywhere** with identity-based authorisation (regulation, zero-trust initiative, multi-tenant clusters).
- You need uniform traffic management (canaries, fault injection, mirroring) and consistent telemetry, and the platform team can own the mesh.

Probably not worth it when:

- Fewer than roughly ten to twenty services, or a single language where a shared library or gRPC's built-in features suffice.
- A small team without dedicated platform capacity; the mesh becomes the new thing nobody understands at 3 a.m.
- Latency or resource budgets are tight, or the workload is mostly batch or north-south.

Alternatives that give part of the value with less cost: an ingress gateway or API gateway for north-south traffic (Level 8, Ingress §8.5), Kubernetes NetworkPolicies (Level 8 §8.19), CNI-level transparent encryption (WireGuard or IPsec in Cilium/Calico), cloud-native service networking (for example VPC Lattice), gRPC with xDS ("proxyless mesh"), and cert-manager-issued certificates with a small client library.

Adopt incrementally: start with observability and mTLS in permissive mode on one namespace, then enforce strict mTLS, then add traffic policy. Have a documented bypass for a broken mesh.

6 Step-by-step guide

You will build a miniature service mesh **without Kubernetes**, using Docker Compose and Envoy, so you can see the moving parts: three Envoy sidecars (one outbound for a `web` client, one inbound for each of two `catalog` versions), mTLS with identities from a tiny CA, retries and timeouts, a weighted canary, and a "control plane" that updates the proxy by rewriting a file (Envoy's file-based xDS). Everything is free and local.

Prerequisites: Docker with Compose v2, `openssl`, `curl`. About 1 GB of image downloads (Envoy and Node Alpine images). Verified with `envoyproxy/envoy:v1.36-latest` and `node:24-alpine`. If your shell sets `https_proxy`, `unset` it for `curl localhost`.

1. **Create the project and a CA with two workload identities.** Identities are SPIFFE-style URIs in the certificate's Subject Alternative Name.

```bash
mkdir -p ~/mesh-lab/{certs,web,inbound,app} && cd ~/mesh-lab
cat > gen-certs.sh <<'EOF'
#!/usr/bin/env bash
```

```
# Tiny CA plus two workload certificates carrying SPIFFE-style identities in a URI
  SAN.
set -euo pipefail
cd "$(dirname "$0")/certs"
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout ca.key -
  out ca.crt \
  -days 30 -subj "/O=Example Lab/CN=Mesh Lab CA" -addext
  "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" 2>/dev/null
issue() {  # name  spiffe-id  dns-name
  openssl req -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes -keyout "$1.key" -out
  "$1.csr" -subj "/CN=$1" 2>/dev/null
  printf
  'basicConstraints=CA:FALSE\nkeyUsage=digitalSignature\nextendedKeyUsage=serverAuth,c
  lientAuth\nsubjectAltName=URI:%s,DNS:%s\n' "$2" "$3" > "$1.ext"
  openssl x509 -req -in "$1.csr" -CA ca.crt -CAkey ca.key -CAcreateserial -out
  "$1.crt" -days 7 -extfile "$1.ext" 2>/dev/null
  rm -f "$1.csr" "$1.ext"
}
issue web     spiffe://example.com/ns/shop/sa/web     web.shop.svc
issue catalog spiffe://example.com/ns/shop/sa/catalog catalog.shop.svc
chmod 644 *.crt *.key   # lab only: production uses secret mounts or SDS, never world-
  readable keys
ls
EOF
chmod +x gen-certs.sh && ./gen-certs.sh
openssl x509 -in certs/web.crt -noout -ext subjectAltName
```

Expected: `URI:spiffe://example.com/ns/shop/sa/web, DNS:web.shop.svc`. Note both services share the `catalog` identity because identity is per *service*, not per version.

2. **The application.** A 20-line Node service that can be made to fail or stall on demand.

```bash
cat > app/server.js <<'EOF'
// Tiny demo service: node server.js  (VERSION env var names the build)
const http = require('http');
const version = process.env.VERSION || 'v1';
http.createServer((req, res) => {
  const url = new URL(req.url, 'http://x');
  const send = (code, body) => { res.writeHead(code, { 'content-type': 'text/plain',
  'x-app-version': version }); res.end(body + '\n'); };
  if (url.pathname === '/flaky') {                        // fails with probability p
  (default 0.5)
    const p = parseFloat(url.searchParams.get('p') || '0.5');
    return Math.random() < p ? send(503, `catalog ${version}: transient failure`) :
  send(200, `catalog ${version}: ok`);
  }
  if (url.pathname === '/slow') {                         // answers after ms
  milliseconds
```

```
    return setTimeout(() => send(200, `catalog ${version}: slow answer`),
  parseInt(url.searchParams.get('ms') || '3000'));
  }
  send(200, `catalog ${version}: hello from ${url.pathname}`);
}).listen(8080, () => console.log(`catalog ${version} listening on 8080`));
EOF
```

3. **The inbound sidecar (data plane, server side).** It accepts only mTLS connections whose client certificate chains to the CA **and** whose URI SAN is exactly the `web` identity, then forwards to the local app. The same file is used for both versions.

```bash
cat > inbound/inbound.yaml <<'EOF'
# Sidecar for the catalog service: accepts ONLY mTLS from the web identity and
  forwards to the local app.
admin:
  address: { socket_address: { address: 0.0.0.0, port_value: 9901 } }
static_resources:
  listeners:
  - name: inbound
    address: { socket_address: { address: 0.0.0.0, port_value: 15006 } }
    filter_chains:
    - transport_socket:
        name: envoy.transport_sockets.tls
        typed_config:
          "@type":
  type.googleapis.com/envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
          require_client_certificate: true
          common_tls_context:
            tls_certificates:
            - certificate_chain: { filename: /certs/catalog.crt }
              private_key: { filename: /certs/catalog.key }
            validation_context:
              trusted_ca: { filename: /certs/ca.crt }
              match_typed_subject_alt_names:
              - san_type: URI
                matcher: { exact: "spiffe://example.com/ns/shop/sa/web" }
      filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type":
  type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.Http
  ConnectionManager
          stat_prefix: inbound
          access_log:
          - name: envoy.access_loggers.stdout
            typed_config:
              "@type":
  type.googleapis.com/envoy.extensions.access_loggers.stream.v3.StdoutAccessLog
              log_format:
```

```
                text_format_source: { inline_string: "[inbound] %REQ(:METHOD)%
  %REQ(:PATH)% → %RESPONSE_CODE% peer=%DOWNSTREAM_PEER_URI_SAN%\n" }
        route_config:
          virtual_hosts:
          - name: local
            domains: ["*"]
            routes:
            - match: { prefix: "/" }
              route: { cluster: local_app }
        http_filters:
        - name: envoy.filters.http.router
          typed_config: { "@type":
  type.googleapis.com/envoy.extensions.filters.http.router.v3.Router }
  clusters:
  - name: local_app
    type: STATIC
    connect_timeout: 1s
    load_assignment:
      cluster_name: local_app
      endpoints: [ { lb_endpoints: [ { endpoint: { address: { socket_address: {
  address: 127.0.0.1, port_value: 8080 } } } } ] } ]
EOF
```

4. **The outbound sidecar (data plane, client side) and its "control plane" files.** `bootstrap.yaml` tells Envoy to load listeners (LDS) and clusters (CDS) from files and to reload them when the directory changes. That is the same mechanism a real control plane uses, just over gRPC instead of the file system.

```bash
cat > web/bootstrap.yaml <<'EOF'
node: { id: web-proxy, cluster: web }
admin:
  address: { socket_address: { address: 0.0.0.0, port_value: 9901 } }
dynamic_resources:
  lds_config:
    resource_api_version: V3
    path_config_source: { path: /etc/envoy/lds.yaml, watched_directory: { path:
  /etc/envoy } }
  cds_config:
    resource_api_version: V3
    path_config_source: { path: /etc/envoy/cds.yaml, watched_directory: { path:
  /etc/envoy } }
EOF
cat > web/cds.yaml <<'EOF'
resources:
- "@type": type.googleapis.com/envoy.config.cluster.v3.Cluster
  name: catalog_v1
  type: STRICT_DNS
  connect_timeout: 1s
  load_assignment:
```

```
    cluster_name: catalog_v1
    endpoints: [ { lb_endpoints: [ { endpoint: { address: { socket_address: { address:
  catalog-v1-proxy, port_value: 15006 } } } } ] } ]
  transport_socket: &mtls
    name: envoy.transport_sockets.tls
    typed_config:
      "@type":
  type.googleapis.com/envoy.extensions.transport_sockets.tls.v3.UpstreamTlsContext
      sni: catalog.shop.svc
      common_tls_context:
        tls_certificates:
        - certificate_chain: { filename: /certs/web.crt }
          private_key: { filename: /certs/web.key }
        validation_context:
          trusted_ca: { filename: /certs/ca.crt }
          match_typed_subject_alt_names:
          - san_type: URI
            matcher: { exact: "spiffe://example.com/ns/shop/sa/catalog" }
- "@type": type.googleapis.com/envoy.config.cluster.v3.Cluster
  name: catalog_v2
  type: STRICT_DNS
  connect_timeout: 1s
  load_assignment:
    cluster_name: catalog_v2
    endpoints: [ { lb_endpoints: [ { endpoint: { address: { socket_address: { address:
  catalog-v2-proxy, port_value: 15006 } } } } ] } ]
  transport_socket: *mtls
EOF
cat > web/lds.yaml <<'EOF'
resources:
- "@type": type.googleapis.com/envoy.config.listener.v3.Listener
  name: outbound_catalog
  address: { socket_address: { address: 0.0.0.0, port_value: 15001 } }
  filter_chains:
  - filters:
    - name: envoy.filters.network.http_connection_manager
      typed_config:
        "@type":
  type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.Http
  ConnectionManager
        stat_prefix: outbound_catalog
        access_log:
        - name: envoy.access_loggers.stdout
          typed_config:
            "@type":
  type.googleapis.com/envoy.extensions.access_loggers.stream.v3.StdoutAccessLog
            log_format:
              text_format_source: { inline_string: "[outbound] %REQ(:PATH)% →
  %RESPONSE_CODE% cluster=%UPSTREAM_CLUSTER% attempts=%UPSTREAM_REQUEST_ATTEMPT_COUNT%
  flags=%RESPONSE_FLAGS% %DURATION%ms\n" }
        route_config:
          virtual_hosts:
          - name: catalog
```

```
            domains: ["*"]
            routes:
            - match: { prefix: "/" }
              route:
                timeout: 2s
                retry_policy: { retry_on: "5xx,connect-failure", num_retries: 2,
  per_try_timeout: 1s }
                weighted_clusters:
                  clusters:
                  - { name: catalog_v1, weight: 100 }
                  - { name: catalog_v2, weight: 0 }
        http_filters:
        - name: envoy.filters.http.router
          typed_config: { "@type":
  type.googleapis.com/envoy.extensions.filters.http.router.v3.Router }
EOF
```

5. **Wire it together with Compose.** The key line is `network_mode: "service:..."`: the app container joins its sidecar's network namespace, which is exactly what a Kubernetes pod is. In a real mesh, iptables redirects the app's traffic to the sidecar; here you call the outbound listener on port 15001 yourself.

```bash
cat > compose.yaml <<'EOF'
name: mesh-lab
x-envoy: &envoy
  image: envoyproxy/envoy:v1.36-latest
  networks: [mesh]
services:
  catalog-v1-proxy:
    <<: *envoy
    command: ["-c", "/etc/envoy/inbound.yaml", "--log-level", "warn"]
    volumes: ["./inbound:/etc/envoy:ro", "./certs:/certs:ro"]
  catalog-v1:
    image: node:24-alpine
    network_mode: "service:catalog-v1-proxy"      # same network namespace, like
  containers in one pod
    environment: { VERSION: v1 }
    command: ["node", "/app/server.js"]
    volumes: ["./app:/app:ro"]
  catalog-v2-proxy:
    <<: *envoy
    command: ["-c", "/etc/envoy/inbound.yaml", "--log-level", "warn"]
    volumes: ["./inbound:/etc/envoy:ro", "./certs:/certs:ro"]
  catalog-v2:
    image: node:24-alpine
    network_mode: "service:catalog-v2-proxy"
    environment: { VERSION: v2 }
    command: ["node", "/app/server.js"]
    volumes: ["./app:/app:ro"]
```

```
  web-proxy:
    <<: *envoy
    command: ["-c", "/etc/envoy/bootstrap.yaml", "--log-level", "warn"]
    volumes: ["./web:/etc/envoy", "./certs:/certs:ro"]
    ports: ["127.0.0.1:15001:15001", "127.0.0.1:19901:9901"]
    depends_on: [catalog-v1-proxy, catalog-v2-proxy]
networks:
  mesh: {}
EOF
docker compose config -q && docker compose up -d
sleep 6; docker compose ps --format '{{.Name}} {{.Status}}'
```

Expected: five containers `Up`.

6. **Send traffic through the mesh.** Your `curl` plays the web application, speaking plain HTTP to its local sidecar; everything after that hop is mTLS.

```bash
for i in 1 2 3; do curl -s localhost:15001/hello; done
sleep 3     # Envoy flushes access logs every few seconds
docker compose logs web-proxy catalog-v1-proxy 2>&1 | grep -E '\[(outbound|inbound)\]'
  | tail -4
```

```
catalog v1: hello from /hello
catalog v1: hello from /hello
catalog v1: hello from /hello
[outbound] /hello → 200 cluster=catalog_v1 attempts=1 flags=- 239ms
[inbound] GET /hello → 200 peer=spiffe://example.com/ns/shop/sa/web
```

The inbound log shows the authenticated caller identity (`peer=spiffe://...`): that is what authorisation policy is based on.

7. **Prove mTLS is enforced.** Use `openssl s_client` inside the `web-proxy` container (the Envoy image includes OpenSSL) to connect to the `catalog` sidecar directly with three different identities.

```bash
t() { docker compose exec -T web-proxy sh -c "printf 'GET /who HTTP/1.1\r\nHost:
  catalog\r\nConnection: close\r\n\r\n' | timeout 5 openssl s_client -connect catalog-
  v1-proxy:15006 -CAfile /certs/ca.crt $1 -quiet 2>&1 | grep -E 'alert|HTTP/1.1'"; }
echo "no client cert:";        t ""
echo "correct web identity:";  t "-cert /certs/web.crt -key /certs/web.key"
echo "wrong identity:";        t "-cert /certs/catalog.crt -key /certs/catalog.key"
```

```
no client cert:         ...alert certificate required...
correct web identity:   HTTP/1.1 200 OK
wrong identity:         ...alert certificate unknown...
```

Even a certificate signed by the same CA is rejected if its identity is not on the allow-list. That is identity-based authorisation, not just encryption.

8. **Resilience policy: retries and timeouts.** The route retries 5xx up to 2 times and times out after 2 seconds.

```bash
for i in $(seq 1 10); do curl -s -o /dev/null -w '%{http_code} '
  'localhost:15001/flaky?p=0.5'; done; echo
curl -s -w ' [%{http_code} in %{time_total}s]\n' 'localhost:15001/slow?ms=3000'
sleep 3   # Envoy flushes access logs every few seconds
docker compose logs web-proxy 2>&1 | grep '\[outbound\] /flaky' | awk '{print $6,$8}'
  | sort | uniq -c
```

With a 50 percent failure rate and two retries, roughly 1 in 8 requests still fails (0.5 cubed), versus 1 in 2 without retries. The slow call returns `upstream request timeout` with code `504` after about 2 seconds, not 3. The log summary (status code and attempt count, for example `200 attempts=2`) shows how many calls needed retries. Retries are only safe for idempotent requests, and they can multiply load during an incident, which is why circuit breakers and retry budgets §12.14 exist.

9. **Play the control plane: shift 10 percent of traffic to v2 without restarting anything.** Rewrite the listener file atomically (write a temp file, then rename). Envoy notices the rename and applies the new routes.

```bash
sed -e 's/name: catalog_v1, weight: 100/name: catalog_v1, weight: 90/' \
    -e 's/name: catalog_v2, weight: 0/name: catalog_v2, weight: 10/' web/lds.yaml >
  web/lds.yaml.new
mv web/lds.yaml.new web/lds.yaml
sleep 2
for i in $(seq 1 100); do curl -s localhost:15001/x; done | sort | uniq -c
curl -s localhost:19901/stats | grep -E
  'listener_manager.lds.update_success|cluster.catalog_v[12].upstream_rq_2xx'
```

```
     90 catalog v1: hello from /x
     10 catalog v2: hello from /x
cluster.catalog_v1.upstream_rq_2xx: 103
cluster.catalog_v2.upstream_rq_2xx: 10
listener_manager.lds.update_success: 2
```

Counts will be close to 90/10 but not necessarily exact. A real control plane (Istio's `istiod`, for example) does the same over a streaming gRPC connection when you apply a `VirtualService` or Gateway API `HTTPRoute`. `lds.update_success: 2` says the proxy accepted two configuration versions; if a bad config were pushed it would increment `update_rejected` and keep the old one.

10. **Look inside a proxy.** The admin endpoint is the mesh debugger.

```bash
curl -s localhost:19901/clusters | grep -E 'catalog_v1::.*
  (cx_active|rq_total|health_flags)'
curl -s localhost:19901/config_dump | head -c 600; echo
```

The `/clusters` view shows endpoints, active connections (pool size, see Connection Pooling §12.10) and health state; `/config_dump` is the effective xDS configuration. `istioctl proxy-config` and `linkerd diagnostics` read these same interfaces.

11. **Clean up.**

```bash
docker compose down -t 1 && cd ~ && rm -r ~/mesh-lab
```

7 Real-world scenario

(Illustrative.) `shop-api` has grown to about 60 services across Node, Go and Java. A security audit requires encryption in transit between all services, and a series of cascading failures (one slow dependency exhausting threads everywhere) shows that every team configured timeouts and retries differently. The platform team installs a mesh in one namespace in permissive mode, enabling mTLS and golden-signal dashboards without changing any application. During the rollout of strict mTLS, a legacy batch job that calls the pricing service over plain HTTP starts failing with connection resets. Investigation: the sidecar's inbound log shows rejected plaintext connections; the job's pod has no sidecar injected. Fix: inject the sidecar into the job (or add an explicit policy exception for that workload), then enable strict mode namespace by namespace. A month later, a node-level control-plane outage caused no immediate traffic loss, but the team found the workload certificate lifetime was only 6 hours and the CA's own intermediate certificate had no alert on expiry. **Lesson:** a mesh gives you consistency and security, but it makes certificate and control-plane health critical; roll out gradually (permissive, then strict), monitor certificate expiry and control-plane availability, and keep a tested break-glass path.

8 Common mistakes and best practices

Don't

- Adopt a mesh because it is fashionable: count services, teams and the concrete problem it solves first.
- Enable strict mTLS everywhere on day one; start permissive and watch which clients are still plaintext.
- Put retries on non-idempotent calls, or let every layer retry (app, sidecar and gateway multiply attempts).
- Treat the mesh as a substitute for application-level timeouts, authentication of end users or input validation.
- Ignore the proxy's resource usage; size requests and limits for sidecars or node proxies.
- Run unpinned or very old mesh versions, or skip the control-plane upgrade runbook.

Do

- Pilot on a small namespace, measure added latency and resource use, and write down an exit plan.
- Use short-lived workload certificates with automatic rotation; alert on CA and intermediate expiry.
- Define default timeouts, bounded retries with budgets, and circuit-breaking limits as platform policy.
- Use authorisation policy (identity to service to method) as the next step after encryption.
- Standardise dashboards and alerts on mesh metrics (request rate, errors, latency percentiles by source and destination).
- Keep ingress (north-south) and mesh (east-west) concerns clearly separated, and document debugging steps (`istioctl`, `linkerd`, Envoy admin).

9 Tasks

- [ ] **T1 — Build the mini mesh** *(≈45 min, intermediate)*
  - **Goal:** run mTLS sidecars and route a request.
  - **Steps:**
    1. Follow steps 1-6 of the guide.
    2. Capture the outbound and inbound log lines for one request.
  - **Done when:** `curl localhost:15001/hello` returns `catalog v1: hello ...` and the inbound log shows `peer=spiffe://example.com/ns/shop/sa/web`.

- [ ] **T2 — Prove identity-based authorisation** *(≈30 min, advanced)*
  - **Goal:** show that encryption alone is not authorisation.
  - **Steps:**
    1. Run step 7's three connection attempts.
    2. Issue a third identity (`checkout`) from the same CA with `gen-certs.sh`-style commands and test it; then add it to the inbound allow-list by adding a second `match_typed_subject_alt_names` entry and restarting the sidecar.
  - **Done when:** `checkout` is rejected before the change and accepted after, and you can explain each alert code.
- [ ] **T3 — Canary and rollback via the control plane** *(≈30 min, intermediate)*
  - **Goal:** shift traffic gradually and roll back instantly.
  - **Steps:**
    1. Move v2 to 10, then 50, then 100 percent by rewriting `lds.yaml` atomically each time, counting responses over 100 requests.
    2. Roll back to 0 percent and confirm.
  - **Done when:** you have four count tables and the `lds.update_success` counter increments each time.
- [ ] **T4 — Break it: a bad config push** *(≈30 min, advanced)*
  - **Goal:** see how the data plane protects itself.
  - **Steps:**
    1. Rewrite `lds.yaml` with an invalid field (for example a typo in `weighted_clusters`) and `mv` it into place.
    2. Check `curl localhost:19901/stats | grep update_rejected`, the proxy logs, and that traffic still flows on the old config.
    3. Fix and push again.
  - **Done when:** `update_rejected` is non-zero, traffic never failed, and the fixed config is applied.
- [ ] **T5 — Measure the overhead** *(≈40 min, advanced)*
  - **Goal:** put numbers on the cost of the mesh.
  - **Steps:**
    1. Time 200 sequential requests through the mesh (`time` over a curl loop) and directly to a local Node server on the host (`node app/server.js` on another port).
    2. Run `docker stats --no-stream` and note the memory of each Envoy container.

- **Done when:** you have median added latency per request and MiB of RAM per sidecar, and write one paragraph on what this means for 500 pods.

☐ **T6 — Prove it: decide whether `shop-api` needs a mesh** *(≈45 min, advanced)*

- **Goal:** make and defend a decision.
- **Steps:**
  1. Without the guide, redraw the sidecar request path and name every component (control plane, xDS, CA, inbound, outbound).
  2. Write a half-page recommendation for a company with 12 services, 3 teams and a PCI requirement for encryption in transit: mesh, ambient, or alternatives? Include exit criteria and costs.
- **Done when:** the diagram is accurate and the recommendation names concrete trade-offs and at least one alternative.

10 Self-check questions

1. What is the difference between the data plane and the control plane of a service mesh?
2. How does the sidecar model differ from ambient mode, and what does each trade away?
3. What does xDS stand for in practice and why does it matter?
4. Why is mTLS alone not enough, and what does identity-based authorisation add?
5. List three situations where you would not adopt a service mesh.
6. What happens to traffic if the control plane goes down for an hour?

ANSWERS

1. The data plane is the set of proxies that carry and act on requests (route, encrypt, retry, measure). The control plane configures them: service discovery, routing and policy translation, certificate issuance and distribution.
2. Sidecars run a proxy per pod, giving strong isolation and full L7 features at higher resource cost and with injection/restart friction. Ambient uses shared per-node L4 proxies (and optional shared L7 waypoints), lowering overhead but concentrating blast radius and adding a hop for L7 features.
3. xDS is the family of Envoy discovery APIs (listeners, routes, clusters, endpoints, secrets) over which a control plane pushes configuration dynamically, so proxies update without restarts.
4. mTLS proves who each side is and encrypts the link, but any holder of a valid certificate could call anything. Authorisation policy decides which identities may call which services or methods.

5. Few services and a small team; a single-language stack where a shared library or gRPC features suffice; tight latency/resource budgets; no platform team to own it; mostly north-south traffic.
6. Existing proxies keep their last configuration and certificates, so traffic continues; but new pods cannot get configuration or identity, config changes cannot roll out, and when certificates expire connections start failing.

11 Further reading

- Envoy documentation: xDS protocol overview, mTLS (`UpstreamTlsContext` / `DownstreamTlsContext`), retry and outlier detection.
- Istio documentation (architecture, ambient mode, security) and Linkerd documentation (architecture, automatic mTLS); Cilium service mesh docs.
- SPIFFE and SPIRE documentation (spiffe.io).
- Matt Klein's talks and blog posts on Envoy and service mesh data planes; William Morgan's "What's a service mesh? And why do I need one?" (Linkerd).
- Level 16 of this roadmap for Kubernetes-native Istio and Linkerd labs.

12.10 Connection Pooling

Topic 10 of 14 | ~4 hours | Difficulty: Intermediate

1 What it is

Opening a network connection is expensive: a TCP handshake (1 round trip), usually a TLS handshake on top (TLS §12.4), TCP slow start before the connection reaches full speed, and, for databases, authentication and a whole server-side process or session. **Connection pooling** keeps a set of already-open connections and **reuses** them for many requests instead of opening and closing one per request.

Pooling appears at several layers, and they stack:

- **HTTP keep-alive (persistent connections):** a client or proxy keeps a TCP/TLS connection open after a response and sends the next request on it (HTTP/1.1 default; HTTP/2 and HTTP/3 go further by multiplexing many requests on one connection, see HTTP/2, HTTP/3 and QUIC §12.5).
- **Client-side pools:** an application library holds N connections to a database, cache or downstream service (HikariCP, `pg-pool`, SQLAlchemy, Go's `database/sql`, Envoy and nginx upstream pools).
- **Server-side / proxy pools:** a middle tier such as **PgBouncer** multiplexes thousands of application connections over a few dozen real database connections.

Do not confuse **HTTP keep-alive** (reusing a connection for more requests) with **TCP keepalive** (the kernel sending probes on an idle connection to detect a dead peer); they have different timers and purposes.

2 Why it matters in DevOps

- **Latency and CPU.** Without reuse every request pays handshakes. For `shop-api` calling PostgreSQL, Redis and a payment API, that overhead can exceed the work itself, and it burns CPU on TLS handshakes.

- **Ephemeral ports and `TIME_WAIT`.** A proxy opening a fresh upstream connection per request leaves thousands of sockets in `TIME_WAIT` and can exhaust the ~28,000 default ephemeral ports to one destination, producing `Cannot assign requested address` errors. The lab shows it.
- **Databases cannot take unlimited connections.** PostgreSQL runs a backend process per connection; hundreds or thousands of them waste memory and make scheduling and locking worse. A fleet of 200 pods with a pool of 20 each is 4,000 connections, far above what one database can serve efficiently (Level 13, Database Connection Pooling §13.6).
- **Pool exhaustion is a classic outage.** One slow query or a leaked connection holds a slot; the pool drains; every request now waits for a connection; latency explodes, threads pile up, and healthy parts of the system look broken. It often cascades (Circuit Breakers §12.14).
- **Mismatched timeouts cause intermittent 502/504 errors.** If a proxy or client thinks a connection is alive but the other side has closed it, the next request fails.

3 Real-world system example

PgBouncer is the de facto lightweight PostgreSQL pooler. It is documented as part of GitLab's production architecture and is bundled by hosted platforms (Heroku offers a PgBouncer buildpack; many Kubernetes operators for PostgreSQL deploy it alongside the cluster). **Amazon RDS Proxy** is AWS's managed equivalent for RDS and Aurora: it pools and shares database connections, which is especially useful for AWS Lambda functions that would otherwise open a connection per invocation (Level 6, RDS and Aurora §6.10).

Envoy keeps a per-upstream-host connection pool and exposes `circuit_breakers` thresholds (`max_connections`, `max_pending_requests`, `max_requests`) so a slow dependency cannot absorb unlimited sockets and memory; its stats expose pool overflow. **nginx** reuses upstream connections only if you configure `keepalive` in the `upstream` block with `proxy_http_version 1.1`; this is a frequently missed optimisation (the lab measures it). **HikariCP**, the default Java pool in Spring Boot, ships a well-known guide ("About Pool Sizing") arguing that small pools often outperform large ones.

AWS load balancers have an idle timeout (60 seconds by default for the Application Load Balancer). AWS documentation advises setting the backend's keep-alive timeout higher than the load balancer's idle timeout; Node.js's HTTP server default `keepAliveTimeout` of 5 seconds is the classic cause of intermittent 502 errors behind an ALB.

4 Diagram

How pooling layers stack for a request in `shop-api`, and where each pool can run dry:

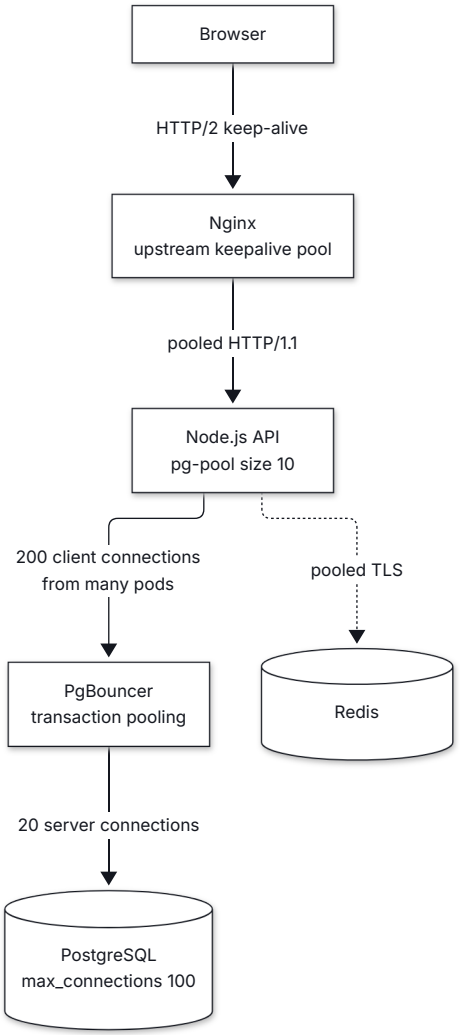


Figure 12.10-1

Pool exhaustion as a state-and-wait problem (what the lab triggers with `query_wait_timeout`):

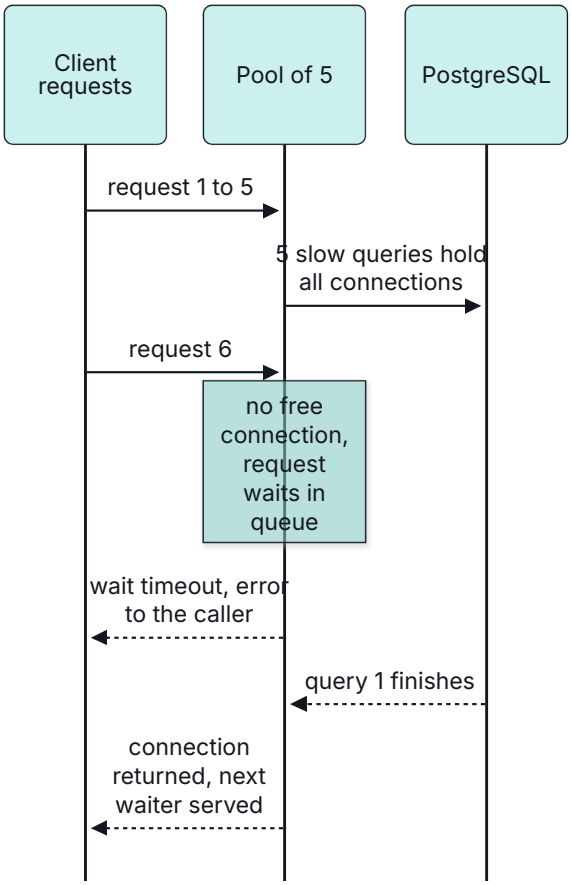


Figure 12.10-2

5 How it works

Keep-alive in HTTP

- **HTTP/1.0** closed after each response unless `Connection: keep-alive` was negotiated; **HTTP/1.1** keeps connections open by default; closing needs `Connection: close`.
- Each side has an **idle timeout** (nginx `keepalive_timeout`, Apache `KeepAliveTimeout`, Node `server.keepAliveTimeout`, ALB idle timeout, client library setting). **The party that closes first wins; the other side then sees a reset.** The safe rule: the *caller's* idle timeout must be **shorter** than the *callee's*, so the caller always retires a connection before the server drops it. Layer by layer this means: client pool idle < proxy idle < backend idle.

- **Reverse proxies need explicit upstream keep-alive.** In nginx: `upstream { keepalive 32; }`, `proxy_http_version 1.1;`, `proxy_set_header Connection "";`. Without it nginx opens a new TCP connection to the backend for every request.
- **Request limits** per connection (`keepalive_requests`, `max_requests_per_connection`) force periodic reconnects, which helps rebalance after deploys but costs handshakes.
- With HTTP/2, one connection carries many concurrent streams, so you need far fewer connections, but one connection can concentrate traffic on one backend (Load Balancing §12.6).

Client-side pool parameters

Parameter	Meaning	Typical guidance
Max size	Upper bound on open connections	Start small (for example 10) and measure; see sizing below
Min idle	Connections kept warm	Keep modest to avoid reconnect storms after idle periods
Acquire / wait timeout	How long a caller waits for a free connection	Short (hundreds of ms to a few seconds); fail fast instead of piling up
Max lifetime	Retire connections after N minutes	Slightly less than any LB/DB-imposed limit; spreads reconnects
Idle timeout	Close connections unused for N seconds	Shorter than the server/proxy idle timeout
Validation / keepalive probe	Test a connection before handing it out	Prefer cheap checks; handle "connection reset" with a safe retry for idempotent work
Leak detection	Warn if a connection is held too long	Enable in non-prod and canary; the best way to find missing `release()` calls

Sizing a pool

- **Little's Law:** concurrency = throughput x latency. If a service does 200 queries per second and each holds a connection for 20 ms, it needs about 200 x 0.020 = 4 connections on average. Add headroom for bursts, not for fear.
- A database can usefully execute only about as many queries at once as it has CPU cores and I/O capacity. The HikariCP guide cites a starting point of about `(core_count x 2) + effective_spindle_count` for the *database server*; the rest queue. A **smaller pool with a queue** usually beats a huge pool that overloads the database.
- **Total across the fleet matters:** `pods x pool size` must stay below the database limit, with room for admin, migrations and replicas. Use a pooler (PgBouncer) when it does not.

- Make **pool sizes explicit configuration**, and expose pool metrics: in use, idle, waiting, wait time, timeouts.

PgBouncer

PgBouncer speaks the PostgreSQL protocol; applications connect to it as if it were the database.

Pool mode	A server connection is held for	Notes
session	The whole client session	Safest, least multiplexing; needed for session state
transaction	One transaction	Best multiplexing and the usual choice; session features break (see below)
statement	One statement	Rare; multi-statement transactions not allowed

In **transaction mode**, consecutive transactions from one client may run on different server connections, so features that rely on session state are unsafe: `SET` (without `SET LOCAL`), session-level advisory locks, `LISTEN/NOTIFY`, temporary tables, `WITH HOLD` cursors and (historically) protocol-level prepared statements. Recent PgBouncer versions (1.21+) can track prepared statements via `max_prepared_statements`; check your version and driver. Key settings: `default_pool_size` (server connections per database/user pair), `max_client_conn` (how many app connections it accepts), `reserve_pool_size`, `query_wait_timeout`, `server_idle_timeout`, `server_lifetime`, `auth_type` / `auth_file` or `auth_query`. The admin console (`psql ... pgbouncer`) offers `SHOW POOLS`, `SHOW STATS`, `SHOW CLIENTS`, `SHOW SERVERS`. PgBouncer is single-threaded per process, so at very high connection counts run several instances. Alternatives and neighbours: PgCat, Odyssey, Supavisor, RDS Proxy and, for MySQL, ProxySQL.

Pool exhaustion: causes and cures

Cause	Signature	Fix
Connection leak (code path misses `release`)	In-use climbs steadily to max, never falls	Always use `try/finally` or the library's scoped helper; enable leak detection
Slow query or lock holding connections	Waiters grow, DB shows long-running queries/ `idle in transaction`	Query timeouts (`statement_timeout`), `idle_in_transaction_session_timeout`, fix queries and indexes
Pool too small for load	Brief queue at peaks	Increase carefully, or add replicas/caching
Pool too big	Database CPU saturated, many active backends	Reduce pool, add PgBouncer, scale the database
Retry storm	Failures trigger retries which take more connections	Backoff with jitter, retry budgets, circuit breakers (Circuit Breakers §12.14)

Cause	Signature	Fix
Long-held connections across slow external calls	Transaction open while calling another API	Never hold a DB connection/transaction across network calls

6 Step-by-step guide

You will (a) measure what upstream keep-alive saves in nginx, (b) overload a tiny PostgreSQL (`max_connections=20`) directly, (c) put PgBouncer in front and run the same load successfully, and (d) trigger pool exhaustion and watch the queue.

Prerequisites: Linux; Docker with Compose v2 (for PostgreSQL and `pgbench`), `nginx`, `apache2-utils` (provides `ab`), and `pgbouncer` (`sudo apt-get install -y nginx apache2-utils pgbouncer`; then `sudo systemctl disable --now nginx pgbouncer` so packaged services do not run). Free, local. Verified with nginx 1.24, PgBouncer 1.22, PostgreSQL 17 and Docker Compose v2. If your shell sets a proxy, `unset https_proxy HTTPS_PROXY`.

Part A: HTTP keep-alive between proxy and backend

1. **Create a proxy that does not reuse upstream connections (the nginx default) and one that does.** Port 18081 is without keep-alive, 18082 is with it.

```bash
mkdir -p ~/pool-lab/ka/logs && cd ~/pool-lab/ka
cat > nginx.conf <<EOF
user $USER;
worker_processes 1;
daemon off;
error_log $PWD/logs/error.log;
pid $PWD/nginx.pid;
events { worker_connections 4096; }
http {
  access_log off;
  upstream app_nokeepalive { server 127.0.0.1:19001; }
  upstream app_keepalive   { server 127.0.0.1:19001; keepalive 32; keepalive_time 1h;
  keepalive_timeout 60s; }
  server {                                  # the backend
    listen 127.0.0.1:19001;
    location / { return 200 "ok\n"; }
  }
  server {                                  # proxy WITHOUT upstream keep-alive
    listen 127.0.0.1:18081;
    location / { proxy_pass http://app_nokeepalive; }
  }
  server {                                  # proxy WITH upstream keep-alive
    listen 127.0.0.1:18082;
    location / {
```

```
      proxy_pass http://app_keepalive;
      proxy_http_version 1.1;               # keep-alive to upstreams needs HTTP/1.1
      proxy_set_header Connection "";       # and no "Connection: close" header
    }
  }
}
EOF
nginx -t -e $PWD/logs/error.log -c $PWD/nginx.conf
nginx -e $PWD/logs/error.log -c $PWD/nginx.conf &
sleep 1
```

2. **Load both and count sockets stuck in `TIME_WAIT` between the proxy and the backend.** `ab -k` keeps the *client* side persistent, so the only difference is nginx's upstream side.

```bash
tw() { ss -tan state time-wait '( dport = :19001 or sport = :19001 )' | tail -n +2 |
  wc -l; }
echo "TIME_WAIT before: $(tw)"
ab -k -q -n 5000 -c 20 http://127.0.0.1:18081/ 2>&1 | grep -E 'Requests per
  second|Failed'
echo "TIME_WAIT after, no upstream keep-alive: $(tw)"
sleep 62; echo "after 62 s: $(tw)"
ab -k -q -n 5000 -c 20 http://127.0.0.1:18082/ 2>&1 | grep -E 'Requests per
  second|Failed'
echo "TIME_WAIT after, with upstream keep-alive: $(tw)"
ss -tan state established '( dport = :19001 )' | tail -n +2 | wc -l      # pooled
  connections still open
```

```
TIME_WAIT before: 0
Requests per second:    13568.37 [#/sec] (mean)
TIME_WAIT after, no upstream keep-alive: 4582
after 62 s: 0
Requests per second:    31828.89 [#/sec] (mean)
TIME_WAIT after, with upstream keep-alive: 0
20
```

   Numbers will differ on your machine, but the pattern should not: thousands of `TIME_WAIT` sockets and roughly half the throughput without reuse; no `TIME_WAIT` and about 20 persistent connections (one per concurrent client) with reuse. Each `TIME_WAIT` socket occupies an ephemeral port for about 60 seconds; at higher rates this exhausts the range.

3. **Stop nginx.** `kill $(cat ~/pool-lab/ka/nginx.pid)`.

Part B: a database with too few connections, then PgBouncer

4. **Start PostgreSQL with a deliberately small** `max_connections` and create the `pgbench` tables.

```bash
cd ~/pool-lab
cat > compose.yaml <<'EOF'
name: pool-lab
services:
  postgres:
    image: postgres:17-alpine
    command: ["postgres", "-c", "max_connections=20"]      # deliberately tiny
    environment:
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: devpass-only                       # lab only
      POSTGRES_DB: shop
    ports: ["127.0.0.1:15432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 2s
      retries: 20
EOF
docker compose up -d --wait
pg() { docker run --rm --network host -e PGPASSWORD=devpass-only "$@"; }      # run
  client tools from the postgres image
pg postgres:17-alpine pgbench -i -s 1 -h 127.0.0.1 -p 15432 -U shop shop
```

   Expected: `done in 0.20 s (...)`.

5. **Overload the database directly: 40 clients against 20 connections.**

```bash
pg postgres:17-alpine pgbench -c 40 -j 4 -T 5 -h 127.0.0.1 -p 15432 -U shop shop 2>&1
  | grep -E 'FATAL|error' | sort | uniq -c | head -5
```

```
      ...  error: connection to server at "127.0.0.1", port 15432 failed: FATAL:
  sorry, too many clients already
```

   This is what a fleet of pods with large pools does to a database at deploy time: `sorry, too many clients already`. (You may also see unrelated client errors from the flood.)

6. **Configure PgBouncer in transaction mode with a pool of 5 server connections.** Run it as your own user (it refuses to run as root).

```bash
cat > pgbouncer.ini <<EOF
[databases]
```

```
shop = host=127.0.0.1 port=15432 dbname=shop

[pgbouncer]
listen_addr = 127.0.0.1
listen_port = 16432
unix_socket_dir =
auth_type = scram-sha-256
auth_file = $PWD/userlist.txt
admin_users = shop
pool_mode = transaction
default_pool_size = 5
max_client_conn = 200
query_wait_timeout = 3
server_idle_timeout = 60
logfile = $PWD/pgbouncer.log
pidfile = $PWD/pgbouncer.pid
EOF
printf '"shop" "devpass-only"\n' > userlist.txt        # plaintext only for the lab;
  use SCRAM secrets or auth_query in production
pgbouncer -d pgbouncer.ini
sleep 1; tail -2 pgbouncer.log
```

Expected: `LOG listening on 127.0.0.1:16432` and `process up: PgBouncer 1.22.0`.

7. **Run the same 40-client load through PgBouncer while watching the pool.**

```bash
pg postgres:17-alpine pgbench -c 40 -j 4 -T 6 -h 127.0.0.1 -p 16432 -U shop shop >
  load.out 2>&1 &
sleep 3
pg postgres:17-alpine psql -h 127.0.0.1 -p 16432 -U shop pgbouncer -c 'SHOW POOLS'
pg postgres:17-alpine psql -h 127.0.0.1 -p 15432 -U shop shop -Atc "select count(*)
  from pg_stat_activity where datname='shop'"
wait; tail -6 load.out
```

```
number of failed transactions: 0 (0.000%)
latency average = 41.380 ms
tps = 966.657813 (without initial connection time)
```

`SHOW POOLS` shows the `shop` row with `cl_active` near 40 clients served by `sv_active` / `sv_idle` of at most 5; PostgreSQL itself sees only about 5-6 backends. Forty clients, zero failures, versus failures against the same database directly.

8. **Cause pool exhaustion on purpose.** Ten clients each run a 5-second query against a pool

of 5. The sixth to tenth must queue and, after `query_wait_timeout = 3`, are rejected.

```bash
printf 'select pg_sleep(5);\n' > sleep.sql
pg -v $PWD/sleep.sql:/sleep.sql postgres:17-alpine pgbench -n -f /sleep.sql -c 10 -j 2
  -t 1 \
    -h 127.0.0.1 -p 16432 -U shop shop > ex.out 2>&1 &
sleep 2
pg postgres:17-alpine psql -h 127.0.0.1 -p 16432 -U shop pgbouncer -c 'SHOW POOLS' |
  cut -c1-80
wait; grep -c query_wait_timeout ex.out
```

```
 database  |   user    | cl_active | cl_waiting | ...
 shop      | shop      |         5 |          5 | ...
5
```

Five clients are active and five are **waiting** (`cl_waiting`); the waiters fail with `FATAL: query_wait_timeout`. That queue depth (`cl_waiting`) and wait time are the metrics to alert on. Raising the pool would let these through only until the database's own limit is hit; the fix for a slow query is the query, and the fix for bursts is bounded queues plus fast failure.

9. **See session state leak under transaction pooling.** One client sets a session variable and disconnects; a completely different client then connects and inherits it, because both landed on the same pooled server connection.

```bash
pg postgres:17-alpine psql -h 127.0.0.1 -p 16432 -U shop shop -Atc "SET
  statement_timeout = '7s'"    # client 1 (via PgBouncer)
pg postgres:17-alpine psql -h 127.0.0.1 -p 16432 -U shop shop -Atc "SHOW
  statement_timeout"           # client 2 (via PgBouncer)
pg postgres:17-alpine psql -h 127.0.0.1 -p 15432 -U shop shop -Atc "SHOW
  statement_timeout"           # directly to PostgreSQL
```

```
SET
7s
0
```

Client 2 sees `7s` through PgBouncer but the default `0` when talking to PostgreSQL directly. In transaction mode the server connection is shared and not reset between clients, so a plain `SET` leaks to whoever uses that connection next. Use `SET LOCAL` inside a transaction, set parameters per role/database on the server, or use session pooling for code that really needs session state.

10. **Clean up.**

```bash
kill $(cat ~/pool-lab/pgbouncer.pid)
cd ~/pool-lab && docker compose down -v && cd ~ && rm -r ~/pool-lab
```

7 Real-world scenario

(Illustrative.) After moving `shop-api` behind an Application Load Balancer, the team sees 1-2 percent of requests failing with 502 from the ALB, never reproducible in staging and unrelated to load. The ALB idle timeout is the default 60 seconds; the Node.js server uses its default `keepAliveTimeout` of 5 seconds. Investigation: the ALB reuses connections to the instances; when a connection has been idle between 5 and 60 seconds, Node closes it just as the ALB sends the next request on it, which fails with a reset and a 502. Access logs show 502s clustered after quiet periods; `tcpdump` shows the instance sending FIN, then the ALB's request meeting a RST. Fix: set the Node server `keepAliveTimeout` to 65 seconds and `headersTimeout` slightly above it, so the backend never closes first. The 502s vanish. Two weeks later a different incident: a new report query forgets to release its connection on an error path; the 10-connection pools in each of 30 pods drain over an hour, latency spikes, and health checks fail. Pool metrics show `waiting` growing while PostgreSQL shows `idle in transaction` sessions. Fix: wrap the code in `try/finally`, set `idle_in_transaction_session_timeout` and `statement_timeout`, enable leak detection, and place PgBouncer between the pods and PostgreSQL. **Lesson:** pooling needs consistent timeouts across layers and explicit exhaustion behaviour: bounded waits, limits, metrics and alerts.

8 Common mistakes and best practices

Don't

- Open a new database or HTTP connection per request.
- Size pools by guessing "bigger is safer"; a giant pool can crush the database.
- Hold a connection or transaction while calling another service or sleeping.
- Forget nginx's upstream `keepalive` + `proxy_http_version 1.1` + `Connection ""`.
- Let a downstream idle timeout be shorter than the upstream pool's, so the server closes first.
- Use session features (`SET`, advisory locks, `LISTEN`) through a transaction-mode pooler.
- Retry instantly on pool timeouts: that makes the exhaustion worse.

Do

- Set explicit pool sizes, acquire timeouts and max lifetimes for every client pool; compute `pods x pool` against the database limit.
- Use PgBouncer (or RDS Proxy) when many app instances or serverless functions connect.
- Align timeouts: client idle < proxy idle < backend idle; keep-alive on backends higher than the load balancer's idle timeout.
- Set `statement_timeout`, `idle_in_transaction_session_timeout` and use leak detection.
- Export and alert on pool metrics: in use, idle, waiting, wait duration, timeouts, `TIME_WAIT` count.
- Prefer HTTP/2 or gRPC multiplexing between services to reduce the number of connections, with L7 balancing (Load Balancing §12.6).

9 Tasks

- [ ] **T1 — Measure keep-alive savings** *(≈30 min, intermediate)*
  - **Goal:** prove reuse matters with numbers.
  - **Steps:**
    1. Do Part A, steps 1-2.
    2. Repeat with `-n 20000 -c 100` and record requests per second and the peak `TIME_WAIT` count for both proxies.
  - **Done when:** you have a two-row table showing throughput and `TIME_WAIT` for each, and can explain why the second proxy keeps about `-c` connections open.
- [ ] **T2 — Overload and rescue PostgreSQL** *(≈30 min, intermediate)*
  - **Goal:** reproduce `too many clients` and fix it with a pooler.
  - **Steps:**
    1. Do steps 4-7.
    2. Change `default_pool_size` to 2 and then 15 and re-run the benchmark.
  - **Done when:** you can show tps for pool sizes 2, 5 and 15 against the 20-connection database and say where the knee is.
- [ ] **T3 — Break it: exhaust the pool and read the signals** *(≈30 min, advanced)*
  - **Goal:** recognise exhaustion from metrics.
  - **Steps:**
    1. Do step 8 and capture `SHOW POOLS`, `SHOW CLIENTS` and `SHOW SERVERS`.
    2. Set `query_wait_timeout = 30` and repeat: observe that waiters now succeed late instead of failing.

3. Write which of the two behaviours you want in production and why.

- **Done when:** you can point to `cl_waiting` and `maxwait` in the output and justify a timeout choice.

☐ **T4 — Leak a connection** *(≈40 min, advanced)*

- **Goal:** find a leak using the database.
- **Steps:**
  1. Write a ten-line script (Python `psycopg` or Node `pg`) that opens a transaction and never closes it; run 10 copies in the background against PgBouncer.
  2. Find them with `select pid, state, now()-xact_start from pg_stat_activity where state='idle in transaction'`.
  3. Set `ALTER ROLE shop SET idle_in_transaction_session_timeout='5s'` and rerun.
- **Done when:** the leaked sessions appear in `pg_stat_activity` and are terminated automatically after 5 seconds.

☐ **T5 — Align idle timeouts** *(≈40 min, advanced)*

- **Goal:** reproduce and fix an idle-timeout race.
- **Steps:**
  1. Run a Node server with `server.keepAliveTimeout = 1000` behind the keep-alive nginx proxy from Part A (`keepalive_timeout 60s`).
  2. Send requests spaced 1.0 to 1.2 seconds apart with a loop and look for `502`s or `upstream prematurely closed connection` in nginx's error log.
  3. Fix by setting the Node timeout above nginx's upstream idle time (or lower nginx's `keepalive_timeout` below the server's).
- **Done when:** the errors appear with the bad pairing and disappear with the fix, and you can state the rule in one sentence.

☐ **T6 — Prove it: pooling plan for `shop-api`** *(≈45 min, advanced)*

- **Goal:** design and justify settings.
- **Steps:**
  1. Without the guide, rebuild the PgBouncer lab and show a successful 40-client run against `max_connections=20`.
  2. Write the plan for 40 pods at 300 requests/s total, each query 15 ms: compute needed connections with Little's Law, choose app pool size, PgBouncer `default_pool_size`, and the timeouts for each layer; list the five metrics you will alert on.

- **Done when:** the lab works from memory and the plan's arithmetic is consistent (total server connections under `max_connections` with headroom).

10 Self-check questions

1. What does a connection pool save, and what is the difference between HTTP keep-alive and TCP keepalive?
2. A fleet scales from 20 to 200 pods with `pool=20`. What happens to PostgreSQL and what are your options?
3. Why does nginx need `proxy_http_version 1.1` and `proxy_set_header Connection ""` for upstream keep-alive?
4. Which idle timeout must be larger: the load balancer's or the backend's, and why?
5. What breaks in PgBouncer transaction mode?
6. Name three causes of pool exhaustion and the first signal for each.

ANSWERS

1. It avoids repeated TCP/TLS handshakes, authentication, slow start and (for databases) backend startup. HTTP keep-alive reuses an application-level connection for more requests; TCP keepalive is a kernel probe on an idle connection to detect dead peers.
2. Connections jump to 4,000, far above what PostgreSQL handles well, causing `too many clients`, memory pressure and slowdowns. Options: smaller per-pod pools, a pooler (PgBouncer/RDS Proxy), fewer pods or larger pods, read replicas, and caching.
3. The default is HTTP/1.0 to upstreams with `Connection: close`, which closes after every request. HTTP/1.1 supports persistent connections and clearing the `Connection` header prevents nginx from sending `close`.
4. The backend's keep-alive timeout must be larger than the load balancer's idle timeout, so the load balancer is always the one to close an idle connection. Otherwise the backend closes while the LB sends a request on it, causing 502s.
5. Anything depending on session state across transactions: plain `SET`, session advisory locks, `LISTEN/NOTIFY`, temporary tables, and possibly prepared statements (version dependent).
6. A leak (in-use climbs to max and stays), slow queries or locks (long-running queries/ `idle in transaction` in the database), and retry storms (waiters and retries spike with errors). Others: pool too small for the load, holding a connection across external calls.

11 Further reading

- PgBouncer documentation (config reference, `SHOW` commands, pooling modes) and the PostgreSQL docs on `max_connections`, `idle_in_transaction_session_timeout`.

- HikariCP wiki, "About Pool Sizing"; the Amazon RDS Proxy documentation.
- nginx `ngx_http_upstream_module` (`keepalive`) and `ngx_http_proxy_module`; Envoy documentation on connection pooling and circuit breaking.
- AWS Elastic Load Balancing documentation on idle timeout and backend keep-alive; Node.js `http.Server` documentation (`keepAliveTimeout`).
- RFC 9112 (HTTP/1.1) section on persistent connections; Brendan Gregg's writing on TCP `TIME_WAIT` and ephemeral ports.

12.11 TLS Termination

Topic 11 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

TLS termination means a component in front of your application (a reverse proxy, load balancer, CDN edge, Kubernetes ingress controller or service-mesh sidecar) ends the client's TLS session: it holds the certificate and private key, performs the handshake, decrypts the traffic, and forwards the request onward. The thing that "terminates" TLS is the endpoint of the encrypted tunnel; everything behind it sees whatever the proxy chooses to send.

There are three deployment patterns, and choosing between them is a real architecture decision:

- **Edge termination (TLS offload):** TLS ends at the proxy; the proxy talks **plaintext HTTP** to the backend over the internal network. Simple and fast, but the internal hop is unencrypted.
- **Passthrough:** the proxy works at layer 4 (TCP) and forwards the **encrypted bytes untouched**. The backend holds the certificate. The proxy cannot read, route on, or modify HTTP; it can only see the destination IP/port and the **SNI** (Server Name Indication) hostname that the client sends in clear text in the ClientHello (TLS §12.4).
- **Re-encryption (TLS bridging):** the proxy terminates the client's TLS, inspects or routes the request at layer 7, then opens a **new, separate TLS connection** to the backend. Two handshakes, two certificates, full visibility in the middle, encrypted everywhere on the wire.

The TLS topic §12.4 explained certificates and handshakes. This topic is about **where in your architecture** that handshake happens, what that placement buys, and what it costs.

2 Why it matters in DevOps

- **Where TLS ends decides what you can do.** Rate limiting by URL path, routing by header, WAF rules, caching, request logging and canary splits all need the proxy to read HTTP, so they need termination (Rate Limiting §12.13, Load Balancing §12.6). A passthrough balancer can do none of it.

- **Certificate management scales with the number of termination points.** Terminating at one edge means one place to renew certificates. Terminating on every pod means thousands. Expired certificates are a top-tier self-inflicted outage.
- **Plaintext inside the network is a security and compliance decision.** "The VPC is private" is a weak argument once an attacker has one foothold, and some standards (for example PCI DSS for cardholder data in transit) push you towards encrypting internal hops too.
- **Handshakes cost CPU.** Public-key operations dominate. Offloading them to a few proxies, using session resumption, and keeping connections alive (Connection Pooling §12.10) is how a fleet stays cheap. The lab measures a roughly 14x throughput gap between a handshake per request and keep-alive.
- **Broken termination fails in confusing ways:** redirect loops because the app thinks it is on HTTP, mixed-content warnings, wrong client IPs in logs, 502s from certificate-name mismatch on the re-encrypted hop.

3 Real-world system example

AWS Elastic Load Balancing. An **Application Load Balancer (ALB)** terminates HTTPS using a certificate from **AWS Certificate Manager (ACM)**, then forwards to targets over HTTP or HTTPS (re-encryption). A **Network Load Balancer (NLB)** can either forward raw TCP (passthrough, the backend holds the certificate) or use a TLS listener to terminate. See ALB §6.14 and CloudFront §6.13 for the AWS pieces. AWS documents which of these checks it performs on the target-side certificate; read that section before relying on the ALB-to-target hop for authentication, and use mTLS §12.12 between your own services if you need it.

Cloudflare documents its SSL/TLS encryption modes in exactly these terms. "Flexible" encrypts browser-to-Cloudflare only and talks plain HTTP to your origin; "Full" re-encrypts to the origin without validating its certificate; "Full (strict)" re-encrypts and validates the origin certificate. Flexible is the classic edge-offload pattern with its classic problem (the origin cannot tell it is behind HTTPS and may redirect forever).

Kubernetes Ingress. An Ingress object's `tls:` section references a Secret holding a certificate; the controller (ingress-nginx, Traefik, HAProxy Ingress, Envoy-based gateways) terminates TLS and routes by host and path (Ingress §8.5). ingress-nginx also supports an `ssl-passthrough` mode for backends that must keep their own TLS. **cert-manager** automates the certificates. **Service meshes** like Istio and Linkerd deliberately do the opposite for internal traffic: every pod's sidecar terminates and originates mTLS so the application sees plain HTTP on localhost while the network sees only ciphertext (Service Mesh §12.9).

HAProxy and nginx both document SNI-based routing in TCP mode (`ssl_preread` in nginx's `stream` module; `req.ssl_sni` in HAProxy), which is how one IP address fronts many TLS backends without decrypting anything.

4 Diagram

The three patterns side by side. The client always starts with TLS; the difference is where it stops:

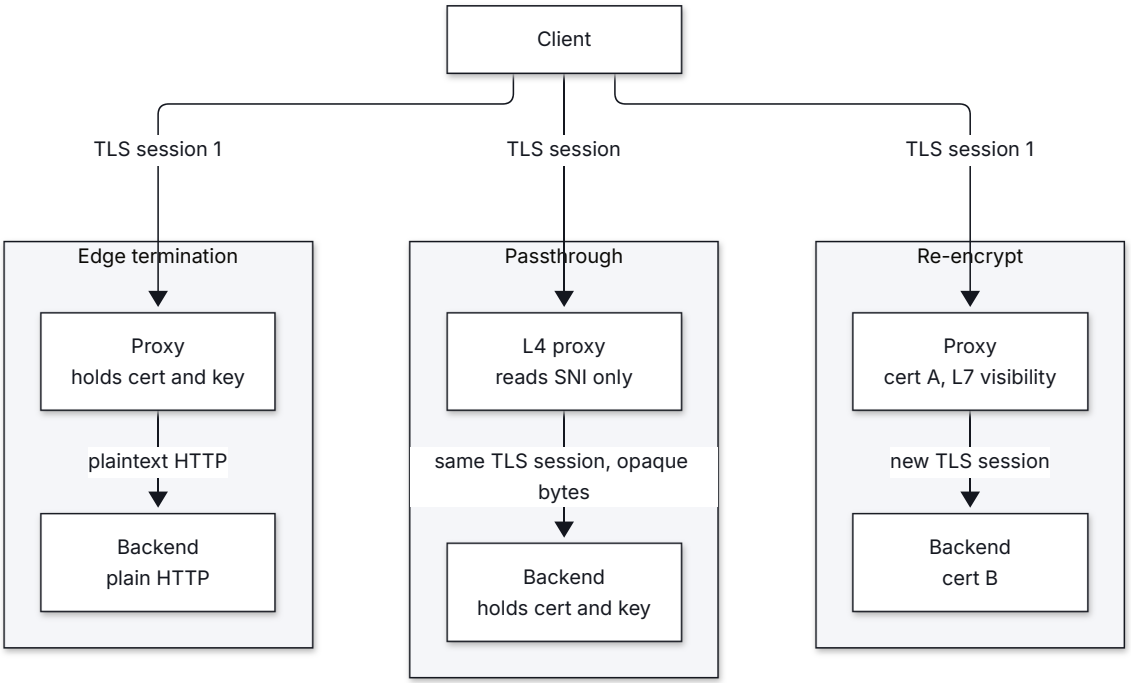


Figure 12.11-1

What happens on the wire in the re-encrypt case (two independent sessions, with HTTP visible only inside the proxy):

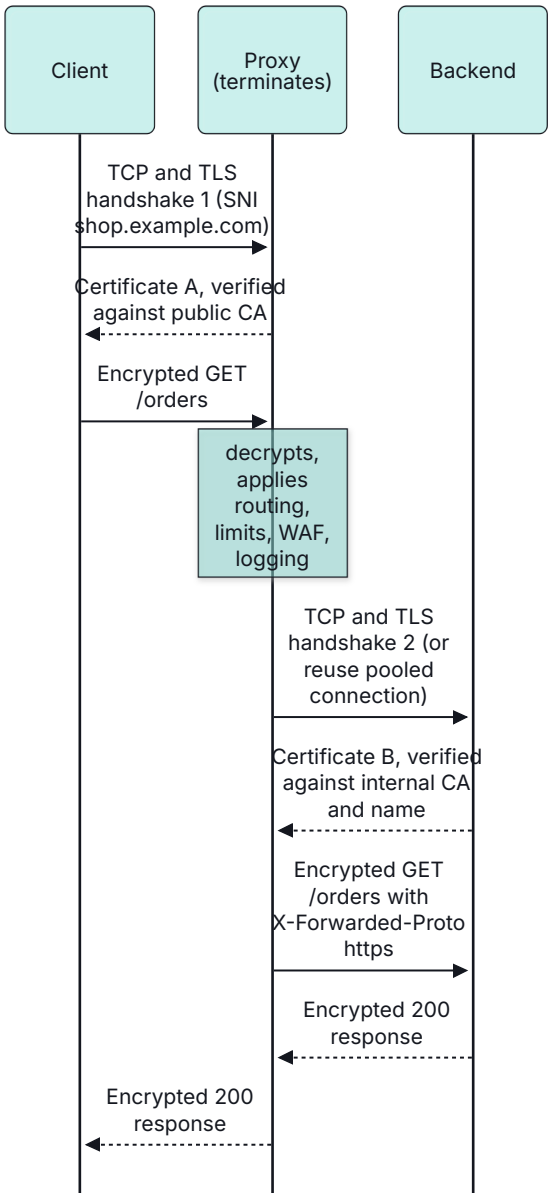


Figure 12.11-2

5 How it works

Comparing the three patterns

	Edge termination	Passthrough	Re-encrypt
Layer the proxy works at	7	4	7
Who holds the public certificate	Proxy	Backend	Proxy (and a second cert on the backend)
Proxy can read and route on HTTP	Yes	**No** (SNI and IPs only)	Yes
Internal hop encrypted	**No**	Yes	Yes
Handshakes per new client connection	1 (at proxy)	1 (at backend)	2
Proxy CPU	TLS work	Minimal	TLS work, twice
Certificate management	One place	Every backend	Edge public certs plus internal certs
Typical use	Public web tier in a trusted network	Databases, gRPC with backend mTLS, protocols you must not touch	Compliance, zero-trust internals

Offload trade-offs

- **Gains.** Central certificate and cipher policy; backends stay simple and need no TLS code; a proxy can use session resumption and large connection reuse; hardware or optimised libraries do crypto efficiently; the proxy can inspect traffic (WAF, caching, compression, rate limits, metrics).
- **Costs.** The proxy becomes a high-value target (it holds the private key and sees all plaintext); the plain-HTTP internal hop can be sniffed or tampered with by anyone on that network; the application sees the proxy's address and `http` scheme, not the client's; the proxy is a capacity bottleneck that you must scale and make highly available (High Availability §0.8).

What the backend must be told

After termination the backend has lost the original connection facts, so the proxy passes them as headers, and the backend must be configured to trust them **only from the proxy**:

Header	Meaning	Gotcha
`X-Forwarded-Proto`	Original scheme (`https`)	Without it frameworks generate `http://` links and redirect loops

Header	Meaning	Gotcha
`X-Forwarded-For`	Chain of client IPs	Spoofable if you trust it from the internet; only trust known proxy hops
`X-Forwarded-Host` / `Host`	Original host name	Needed for correct absolute URLs
`Forwarded` (RFC 7239)	Standardised version of the above	Less widely used than the `X-` forms
PROXY protocol	A small header at connection start carrying the real client IP/port	For L4 proxies (including passthrough) that cannot add HTTP headers; both ends must agree to use it

Costs of a handshake and how to reduce them

- **Full handshake:** public-key operations (ECDHE key exchange, certificate signature verification and, on the server, signing with the certificate's key). RSA signing is noticeably more expensive than ECDSA (P-256) signing, which is one reason ECDSA certificates are popular at high scale.
- **TLS 1.3** needs 1 round trip for a fresh connection (TLS 1.2 needs 2) and supports **0-RTT** resumption, which has replay risk and should be limited to idempotent requests.
- **Session resumption** (session IDs or tickets) skips most public-key work on reconnect. nginx: `ssl_session_cache shared:SSL:10m; ssl_session_timeout 1h;`. Rotate ticket keys regularly if you use tickets across a fleet.
- **Keep-alive and HTTP/2** amortise one handshake over thousands of requests (HTTP/2, HTTP/3 and QUIC §12.5, Connection Pooling §12.10).

Same idea in Envoy

Envoy terminates with a `DownstreamTlsContext` on the listener's filter chain and re-encrypts with an `UpstreamTlsContext` on the cluster (shape shown for reference; the lab below uses nginx and HAProxy):

YAML

```yaml
# listener filter_chain (terminate)
transport_socket:
  name: envoy.transport_sockets.tls
  typed_config:
    "@type":
  type.googleapis.com/envoy.extensions.transport_sockets.tls.v3.DownstreamTlsContext
    common_tls_context:
      tls_certificates:
        - certificate_chain: { filename: /certs/edge.crt }
          private_key:       { filename: /certs/edge.key }
# cluster (re-encrypt to the backend)
transport_socket:
  name: envoy.transport_sockets.tls
```

```
  typed_config:
    "@type":
  type.googleapis.com/envoy.extensions.transport_sockets.tls.v3.UpstreamTlsContext
    sni: backend.internal
    common_tls_context:
      validation_context:
        trusted_ca: { filename: /certs/ca.crt }
```

6 Step-by-step guide

You will run all three patterns side by side on your laptop, prove with `tcpdump` which hops are plaintext, and measure what a handshake costs.

Prerequisites: Linux with `nginx`, `haproxy`, `openssl`, `curl`, `tcpdump` (needs `sudo`) and `apache2-utils` for `ab` (`sudo apt-get install -y nginx haproxy openssl curl tcpdump apache2-utils`, then `sudo systemctl disable --now nginx haproxy` so the packaged services do not interfere). Free and local. Verified with nginx 1.24, HAProxy 2.x, OpenSSL 3.0 and curl 8.5. If your shell exports a proxy, run `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY` first. All ports below are on `127.0.0.1`. The Docker Compose variant is trivial (the same config files, one container per proxy), but host processes make `tcpdump` simpler.

1. **Create a lab CA and two certificates.** One certificate is for the edge (`shop.example.com`), one for the backend (`backend.internal` and `shop.example.com`, so passthrough can present it).

   ```bash
   export LAB=$HOME/tls-term-lab; mkdir -p $LAB/certs $LAB/logs $LAB/tmp && cd $LAB/certs
   openssl req -x509 -newkey rsa:2048 -nodes -keyout ca.key -out ca.crt -days 30 \
     -subj "/CN=Lab Root CA" -addext "basicConstraints=critical,CA:TRUE" -addext
     "keyUsage=critical,keyCertSign,cRLSign"
   issue() {   # issue <name> <CN> <SAN list>
     openssl req -newkey rsa:2048 -nodes -keyout $1.key -out $1.csr -subj "/CN=$2"
     printf "subjectAltName=%s\nextendedKeyUsage=serverAuth\n" "$3" > $1.ext
     openssl x509 -req -in $1.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days 30 -
     extfile $1.ext -out $1.crt
   }
   issue edge    shop.example.com "DNS:shop.example.com,DNS:localhost,IP:127.0.0.1"
   issue backend backend.internal "DNS:backend.internal,DNS:shop.example.com"
   openssl verify -CAfile ca.crt edge.crt backend.crt
   ```

   Expected: `edge.crt: OK` and `backend.crt: OK`. The CA is for this lab only; delete it afterwards. Never reuse lab keys.

2. **Write the nginx config: a plain backend, a TLS backend, an edge-termination server and a re-encrypt server.**

BASH

```
cd $LAB
cat > nginx.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/error.log notice;
pid $LAB/nginx.pid;
events { worker_connections 1024; }
http {
  access_log $LAB/logs/access.log;
  client_body_temp_path $LAB/tmp/body; proxy_temp_path $LAB/tmp/proxy;
  fastcgi_temp_path $LAB/tmp/fcgi; uwsgi_temp_path $LAB/tmp/uwsgi; scgi_temp_path
  $LAB/tmp/scgi;

  server {                                # plaintext backend (what shop-api speaks)
    listen 127.0.0.1:9001;
    return 200 "plain backend: scheme=\$scheme xfp=\$http_x_forwarded_proto
  xff=\$http_x_forwarded_for\n";
  }
  server {                                # TLS backend (re-encrypt target and
  passthrough target)
    listen 127.0.0.1:9444 ssl;
    ssl_certificate     $LAB/certs/backend.crt;
    ssl_certificate_key $LAB/certs/backend.key;
    return 200 "tls backend: scheme=\$scheme xfp=\$http_x_forwarded_proto
  proto=\$server_protocol\n";
  }
  server {                                # MODE 1: edge termination
    listen 127.0.0.1:8443 ssl http2;      # on nginx 1.25.1+ use: listen ... ssl;
  http2 on;
    server_name shop.example.com;
    ssl_certificate     $LAB/certs/edge.crt;
    ssl_certificate_key $LAB/certs/edge.key;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 1h;
    location / {
      proxy_pass http://127.0.0.1:9001;
      proxy_set_header Host              \$host;
      proxy_set_header X-Forwarded-Proto \$scheme;
      proxy_set_header X-Forwarded-For   \$proxy_add_x_forwarded_for;
    }
  }
  server {                                # MODE 3: re-encrypt
    listen 127.0.0.1:10443 ssl http2;
    server_name shop.example.com;
    ssl_certificate     $LAB/certs/edge.crt;
    ssl_certificate_key $LAB/certs/edge.key;
    location / {
      proxy_pass https://127.0.0.1:9444;
      proxy_ssl_server_name on;
```

```
      proxy_ssl_name backend.internal;                # name to verify and send as
  SNI
      proxy_ssl_verify on;
      proxy_ssl_trusted_certificate $LAB/certs/ca.crt; # trust only the internal CA
      proxy_set_header X-Forwarded-Proto \$scheme;
    }
  }
}
EOF
nginx -t -c $LAB/nginx.conf -e $LAB/logs/error.log
```

Expected: `syntax is ok` and `test is successful`. Note the `\$` escapes: the heredoc is unquoted so `$LAB` expands while nginx variables survive.

3. **Write the HAProxy config for passthrough.** HAProxy in `mode tcp` reads the SNI from the ClientHello and forwards bytes without decrypting.

```bash
cat > haproxy.cfg <<EOF
global
  log stdout format raw local0
defaults
  mode tcp
  timeout connect 3s
  timeout client  30s
  timeout server  30s
  log global
  option tcplog
frontend passthrough
  bind 127.0.0.1:9443
  tcp-request inspect-delay 3s
  tcp-request content accept if { req.ssl_hello_type 1 }
  use_backend tls_backend if { req.ssl_sni -i backend.internal }
  use_backend tls_backend if { req.ssl_sni -i shop.example.com }
  default_backend tls_backend
backend tls_backend
  server s1 127.0.0.1:9444
EOF
haproxy -c -f haproxy.cfg
```

Expected: `Configuration file is valid`.

4. **Start everything and call all three modes.**

```bash
(nginx -c $LAB/nginx.conf -e $LAB/logs/error.log >/dev/null 2>&1 &)
haproxy -f haproxy.cfg -D -p $LAB/haproxy.pid
sleep 1
C="--cacert $LAB/certs/ca.crt --resolve shop.example.com:8443:127.0.0.1 --resolve
  shop.example.com:9443:127.0.0.1 --resolve shop.example.com:10443:127.0.0.1"
curl -sS $C https://shop.example.com:8443/     # edge
curl -sS $C https://shop.example.com:9443/     # passthrough
curl -sS $C https://shop.example.com:10443/    # re-encrypt
```

```
plain backend: scheme=http xfp=https xff=127.0.0.1
tls backend: scheme=https xfp= proto=HTTP/1.1
tls backend: scheme=https xfp=https proto=HTTP/1.0
```

Read the three lines: in edge mode the backend sees `scheme=http` but learns the truth from `X-Forwarded-Proto: https`; in passthrough the backend itself speaks TLS and *no proxy could add a header* (`xfp=` is empty); in re-encrypt the backend is TLS and the proxy added the header (and nginx talks HTTP/1.0 to upstreams by default, hence `HTTP/1.0`).

5. **See which certificate the client receives in each mode.** The serial number tells you whose certificate it is.

```bash
for p in 8443 9443 10443; do
  echo -n "$p  "; openssl s_client -connect 127.0.0.1:$p -servername shop.example.com
  </dev/null 2>/dev/null \
    | openssl x509 -noout -subject -serial | tr '\n' ' '; echo
done
```

```
8443  subject=CN = shop.example.com serial=56AD...F5
9443  subject=CN = backend.internal serial=56AD...F7
10443  subject=CN = shop.example.com serial=56AD...F5
```

Edge and re-encrypt present the **edge** certificate (serial ending `F5`; yours will differ). Passthrough presents the **backend's own** certificate (`F7`): the proxy never owned one.

6. **Prove with `tcpdump` that the edge-mode internal hop is plaintext.** Capture the proxy-to-

backend port while sending one request.

```bash
sudo timeout 6 tcpdump -i lo -nn -A -l 'tcp port 9001' > $LAB/edge.txt 2>/dev/null &
sleep 1; curl -sS $C https://shop.example.com:8443/ >/dev/null; wait
grep -aE 'GET|Host:|X-Forwarded|plain backend' $LAB/edge.txt
```

```
..H.qe.GET / HTTP/1.0
Host: shop.example.com
X-Forwarded-Proto: https
X-Forwarded-For: 127.0.0.1
plain backend: scheme=http xfp=https client=127.0.0.1 xff=127.0.0.1
```

Anyone who can capture on that network segment reads the request and response. That is the price of offload.

7. **Repeat for re-encrypt (port 9444) and passthrough (port 9443).**

```bash
sudo timeout 6 tcpdump -i lo -nn -A -l 'tcp port 9444' > $LAB/reenc.txt 2>/dev/null &
sleep 1; curl -sS $C https://shop.example.com:10443/ >/dev/null; wait
grep -acE 'GET|X-Forwarded|tls backend' $LAB/reenc.txt        # expect 0
sudo timeout 6 tcpdump -i lo -nn -A -l 'tcp port 9443' > $LAB/pass.txt 2>/dev/null &
sleep 1; curl -sS $C https://shop.example.com:9443/ >/dev/null; wait
grep -ac 'GET /' $LAB/pass.txt                                # expect 0
grep -a 'shop.example.com' $LAB/pass.txt | head -2            # the SNI is still in
  clear text
```

The first count is `0`: the backend leg carries only ciphertext. The second is `0` for the request line, but the hostname **is** visible inside the ClientHello. That clear-text SNI is how a passthrough proxy routes, and is also what censors and network monitors see (Encrypted Client Hello, ECH, is the emerging fix).

8. **Measure what offload and handshakes cost.** `ab` without `-k` forces a new TCP and TLS handshake for every request.

```bash
ab -q -n 3000 -c 10 http://127.0.0.1:9001/        2>&1 | grep 'Requests per second'
  # plain backend
ab -q -n 3000 -c 10 https://127.0.0.1:8443/       2>&1 | grep 'Requests per second'
  # TLS, new handshake each time
ab -q -k -n 3000 -c 10 https://127.0.0.1:8443/    2>&1 | grep 'Requests per second'
  # TLS, keep-alive
```

```
Requests per second:    15170.21 [#/sec] (mean)
Requests per second:    1083.72 [#/sec] (mean)
Requests per second:    13690.13 [#/sec] (mean)
```

Your numbers will differ, but the pattern is stable: a handshake per request cuts throughput by an order of magnitude against a single nginx worker, and keep-alive brings TLS almost back to plaintext speed. Check session resumption separately:

```bash
openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -tls1_2 -
  reconnect </dev/null 2>&1 \
  | grep -E '^(New|Reused)' | sort | uniq -c
```

```
      1 New, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
      5 Reused, TLSv1.2, Cipher is ECDHE-RSA-AES256-GCM-SHA384
```

One full handshake, five cheap resumed ones. `curl -w` also shows the split: `time_connect` is the TCP time, `time_appconnect` is TCP plus TLS.

9. **Break the re-encrypt hop on purpose.** Make nginx verify the wrong name, reload and call it.

```bash
sed -i 's#proxy_ssl_name backend.internal;#proxy_ssl_name wrong.internal;#'
  $LAB/nginx.conf
nginx -s reload -c $LAB/nginx.conf -e $LAB/logs/error.log; sleep 1
curl -sS -o /dev/null -w '%{http_code}\n' $C https://shop.example.com:10443/
grep -a 'upstream SSL' $LAB/logs/error.log | tail -1
```

```
502
... upstream SSL certificate does not match "wrong.internal" while SSL handshaking to
  upstream ...
```

The client got a clean TLS session with the edge; the failure is entirely on the second leg, visible only in the proxy log. Revert the `sed` and reload to restore `200`.

10. **Clean up.**

```bash
kill $(cat $LAB/nginx.pid) $(cat $LAB/haproxy.pid)
rm -r "$LAB"; unset LAB
```

7 Real-world scenario

(Illustrative.) `shop-api` sits behind an ALB that terminates HTTPS and forwards plain HTTP to Node.js containers. After a marketing campaign adds a login page, users report an endless redirect loop. The browser shows "too many redirects" only on `https://shop.example.com/login`. Investigation: the app's middleware redirects any request it believes is `http` to `https`. The ALB's hop to the container is plain HTTP, so every request *looks* like `http` to the app, which redirects, the ALB terminates again, and the cycle repeats. Logs show `scheme=http` on every request even though users are on HTTPS, and `curl -v` shows the `301` pointing at the same URL. Fix: the ALB adds `X-Forwarded-Proto`; the app is told to trust that header from the load balancer's subnet only (`trust proxy` in Express, restricted to the ALB CIDR). The loop disappears. During the postmortem a security review points out that the ALB-to-container hop is plaintext inside the VPC; since the app handles session cookies and payment tokens, the team moves to re-encryption with a certificate from a private CA, rotated automatically, and later to service-mesh mTLS (mTLS §12.12). **Lesson:** after termination the backend loses the facts about the client's connection. Pass them in headers, trust them only from the proxy, and decide deliberately whether the internal hop must be encrypted.

8 Common mistakes and best practices

Don't

- Trust `X-Forwarded-*` headers from arbitrary clients; an attacker can set `X-Forwarded-For` to anything the proxy does not overwrite.
- Re-encrypt with `proxy_ssl_verify off` or Cloudflare "Full" and call it secure; an unverified hop is encrypted but unauthenticated, so a man in the middle can still impersonate the backend.
- Use passthrough and then expect L7 features (path routing, header-based rate limits, WAF); they need termination.
- Share one private key across many proxies and environments, or leave keys world-readable on the proxy host.
- Forget that the plaintext internal hop exists; network ACLs and "it's a private subnet" are not encryption.
- Leave the proxy as a single instance; it is now the availability bottleneck for the whole site.

Do

- Terminate at a small, hardened, well-monitored edge and automate renewal (ACME, ACM, cert-manager).
- Re-encrypt (or use mesh mTLS) for sensitive traffic, validating the backend certificate chain **and name**.
- Set `X-Forwarded-Proto` / `For` / `Host`, configure the app to trust them only from known proxies, or use PROXY protocol for L4.
- Enable TLS 1.3, modern ciphers, HTTP/2, session resumption and upstream keep-alive; measure handshake rate and CPU.
- Choose ECDSA or at least 2048-bit RSA certificates and monitor expiry with alerts well ahead of the deadline.
- Use passthrough for protocols you must not interpret (databases, client-cert-authenticated backends) or where end-to-end encryption is required.

9 Tasks

- [ ] **T1 — Run the three patterns** *(≈45 min, intermediate)*
  - **Goal:** have edge, passthrough and re-encrypt working locally.
  - **Steps:**
    1. Do steps 1-4 of the guide.
    2. For each mode record: which certificate the client sees (step 5), whether the backend sees `X-Forwarded-Proto`, and whether the proxy can route on HTTP paths.
  - **Done when:** you have a three-row table matching the comparison in section 5, backed by your own `curl` and `openssl` output.
- [ ] **T2 — Capture the plaintext hop** *(≈30 min, intermediate)*
  - **Goal:** prove by packet capture which hop is readable.
  - **Steps:**
    1. Do steps 6-7 with `tcpdump`.
    2. Change the edge `proxy_pass` so the path is `/secret?token=abc123` (or add a header `Authorization: Bearer lab-token` with `curl -H`) and find it in the capture.
  - **Done when:** you can show the token in the edge-mode capture and its absence in the re-encrypt and passthrough captures.
- [ ] **T3 — Break the re-encrypt hop three ways** *(≈40 min, advanced)*
  - **Goal:** recognise each failure from the proxy log.
  - **Steps:**
    1. Wrong name (step 9).

2. Remove `proxy_ssl_trusted_certificate` while `proxy_ssl_verify on`.
3. Point the backend at a certificate signed by a different, untrusted CA (create a second lab CA and re-issue `backend.crt` from it).

- **Done when:** each failure produces a 502, and you can quote the matching `error.log` line and name the fix.

- [ ] **T4 — Fix the redirect loop** *(≈40 min, advanced)*
  - **Goal:** reproduce the `X-Forwarded-Proto` bug and fix it.
  - **Steps:**
    1. Put a tiny app behind the edge server (for example `python3 -m http.server` is not enough; write 10 lines of Python or Node that returns `301` to `https://$Host$path` when `X-Forwarded-Proto` is missing or `http`).
    2. Remove `proxy_set_header X-Forwarded-Proto` from nginx and observe the loop with `curl -L --max-redirs 5 -v`.
    3. Restore the header and verify the loop is gone.
  - **Done when:** `curl -L` loops with the header removed and returns `200` with it present.

- [ ] **T5 — Measure and tune handshake cost** *(≈45 min, advanced)*
  - **Goal:** quantify offload decisions.
  - **Steps:**
    1. Repeat step 8 with an ECDSA certificate (`openssl ecparam -name prime256v1 -genkey`) instead of RSA 2048 and compare handshake-per-request throughput.
    2. Disable `ssl_session_cache` and compare `-reconnect` results.
    3. Compare `ab` with `-c 1` and `-c 50`.
  - **Done when:** you have a table of requests per second for RSA vs ECDSA, resumption on vs off, and keep-alive vs not, and a one-paragraph recommendation.

- [ ] **T6 — Prove it: choose a pattern for `shop-api`** *(≈45 min, advanced)*
  - **Goal:** make and defend an architecture decision.
  - **Steps:**
    1. Without the guide, rebuild the edge and re-encrypt servers and demonstrate the plaintext vs ciphertext difference with `tcpdump`.
    2. Write a half-page decision: public web tier, internal API, and a PostgreSQL connection. For each, choose edge, passthrough or re-encrypt, and name the certificate owner, renewal mechanism and what you give up.

- **Done when:** the demo works from memory and your decision names at least two risks you accept and one control for each.

10 Self-check questions

1. What is the difference between edge termination, passthrough and re-encryption?
2. Why can a passthrough load balancer not do path-based routing, but still route by hostname?
3. A user sees an infinite redirect loop behind an HTTPS load balancer. What is the likely cause and fix?
4. Why is re-encryption without certificate verification little better than plaintext?
5. Name three ways to reduce the CPU cost of TLS at a proxy.
6. When would you choose passthrough over termination?

ANSWERS

1. Edge: TLS ends at the proxy and the backend hop is plaintext. Passthrough: the proxy forwards encrypted bytes and the backend terminates. Re-encrypt: the proxy terminates, then opens a new TLS session to the backend.
2. The HTTP request is encrypted, so only layer-4 facts are visible. The SNI hostname, however, is sent in clear text in the ClientHello, so the proxy can pick a backend from it.
3. The app sees plain HTTP on the proxied hop and redirects to HTTPS repeatedly. Send `X-Forwarded-Proto` from the proxy and make the app honour it (only from trusted proxies).
4. Encryption without authentication protects against passive eavesdropping but not against an active attacker who impersonates the backend; verification of chain and name provides the authentication.
5. Session resumption, keep-alive/HTTP/2 or upstream connection reuse, TLS 1.3, ECDSA certificates, and spreading load over more proxy workers or instances (also hardware offload on some platforms).
6. When the proxy must not see plaintext (end-to-end encryption requirement), for protocols you do not want to interpret (databases, custom TCP), or when the backend authenticates clients itself with client certificates.

11 Further reading

- nginx documentation: `ngx_http_ssl_module`, `ngx_http_proxy_module` (`proxy_ssl_*` directives), and `ngx_stream_ssl_preread_module`.
- HAProxy documentation: SNI inspection in TCP mode (`req.ssl_sni`) and the PROXY protocol specification.

- Envoy documentation: TLS (`DownstreamTlsContext`, `UpstreamTlsContext`) and listener filters such as TLS Inspector.
- AWS Elastic Load Balancing documentation (HTTPS listeners, TLS listeners on NLB, target-side encryption) and Cloudflare's SSL/TLS encryption modes documentation.
- RFC 8446 (TLS 1.3), RFC 7239 (Forwarded header), and Ivan Ristic, *Bulletproof TLS and PKI*.

LAB NOTES What I ran, what I saw, what it means

12.12 mTLS

Topic 12 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

In ordinary TLS only the **server** proves who it is: the browser checks that `shop.example.com` presents a certificate signed by a CA it trusts, but the server has no cryptographic idea who the browser is (it relies on passwords, cookies or tokens afterwards). **Mutual TLS (mTLS)** adds the reverse: during the handshake the server sends a **CertificateRequest**, the client answers with its own **client certificate** and proves possession of the matching private key, and the server validates it against a CA it trusts. Both ends are authenticated before a single byte of HTTP is exchanged.

The result is **strong, cryptographic service identity**. Instead of "this request came from 10.0.4.17, so it must be `shop-api`" (a claim anyone on that network can forge), you get "this connection was made by whoever holds the private key for a certificate that our CA issued to `shop-api`". mTLS is the foundation of **zero-trust** networking between services, of service meshes (Service Mesh §12.9), and of many B2B and device APIs.

This topic builds on TLS §12.4 (chains, CAs, rotation) and TLS Termination §12.11 (where the handshake ends). It covers four things: how mutual authentication works, how client certificates carry identity, how **SPIFFE/SPIRE** automate identity at scale, and how to rotate certificates without outages.

2 Why it matters in DevOps

- **Network location is not identity.** Firewalls and security groups say which IPs may connect, not which *service* is calling. After one compromised pod, "inside the VPC" means nothing. mTLS binds a call to a cryptographic identity that policy can reference (Security Architecture §19.3).
- **It encrypts east-west traffic for free-ish.** The same handshake that authenticates also encrypts, satisfying "encryption in transit" requirements for internal traffic.

- **Authentication is not authorization.** mTLS answers "who are you?", not "may you delete orders?". Every real deployment needs both: the certificate identifies the caller, a policy decides what that identity may do. The lab shows a valid certificate being rejected with `403`.
- **The operational burden is the real cost.** Issuing, distributing, rotating and revoking certificates for hundreds of workloads by hand does not work; certificates that expire silently are a classic 3 a.m. outage. Short-lived, automatically rotated identities (SPIFFE/SPIRE, mesh CAs, cert-manager) are what make mTLS practical.
- **It is a common interview and incident topic** because failure modes are subtle: wrong trust bundle, missing `clientAuth` usage, clock skew, expired intermediates, a proxy that terminates TLS and drops the client identity.

3 Real-world system example

Service meshes. Istio's control plane (istiod) acts as a certificate authority: it issues each workload a short-lived certificate whose identity is a SPIFFE ID derived from the pod's Kubernetes service account, and the Envoy sidecars use those certificates to do mTLS between every pair of pods, rotating them automatically. Linkerd does the same with its own identity component and lightweight proxies, enabling mTLS by default for meshed TCP traffic. The application code is unchanged. See Level 16: Service Mesh with Istio or Linkerd §16.9 and Mesh: mTLS, Routing, Retries, Timeouts, Circuit Breaking §16.10.

SPIFFE and SPIRE. SPIFFE (Secure Production Identity Framework For Everyone) is a CNCF-hosted specification for workload identity; SPIRE is its reference implementation. A **SPIRE Server** is the CA and registry; a **SPIRE Agent** on each node attests the node, then attests workloads (for example by Kubernetes namespace and service account, or Unix process attributes) and hands them identities through the **Workload API**, a local Unix domain socket. It is used to give identities to workloads across clouds, VMs and Kubernetes.

Kubernetes itself uses mutual TLS between control-plane components: the API server authenticates clients such as kubelets, and etcd peers authenticate each other with certificates. **Cloudflare** documents "Authenticated Origin Pulls", where the origin server requires Cloudflare's client certificate so only Cloudflare's edge can talk to it. **AWS** offers mTLS on API Gateway custom domains and on Application Load Balancers (a verify mode and a passthrough mode). Banks and payment APIs in many jurisdictions require mTLS between institutions, and IoT fleets commonly use per-device certificates.

4 Diagram

The handshake with the extra client-authentication steps highlighted (TLS 1.3 shown; in TLS 1.2 the certificates travel in clear text, in 1.3 they are encrypted):

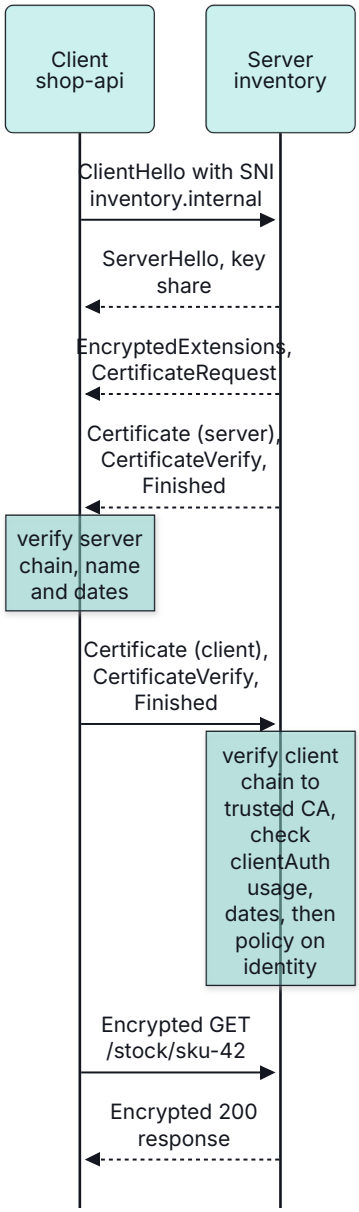


Figure 12.12-1

How SPIRE hands out short-lived, automatically rotated identities so nobody manages certificate files by hand:

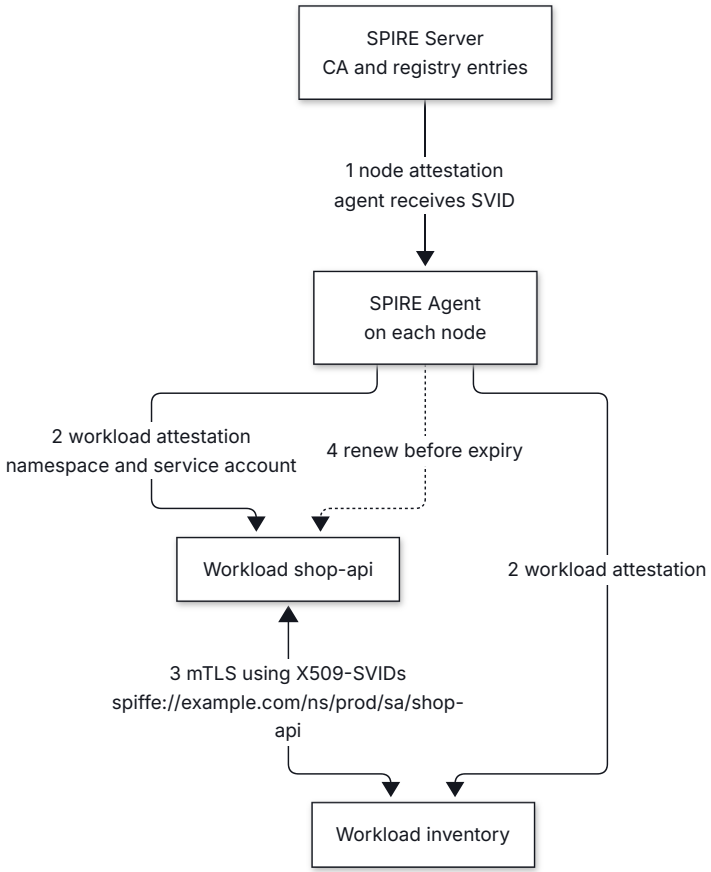


Figure 12.12-2

5 How it works

What each side checks

Check	Server verifies the client	Client verifies the server
Chain	Client cert chains to a CA in the server's `ssl_client_certificate` / trust bundle	Server cert chains to a CA in the client's trust store
Dates	Not expired, not yet valid (clock skew matters)	Same
Key usage	Extended Key Usage includes `clientAuth`	Includes `serverAuth`

Check	Server verifies the client	Client verifies the server
Name	No hostname; instead an **identity** (CN or, better, a SAN) is mapped to policy	Server name matches a SAN for the host the client dialled
Possession	`CertificateVerify` signature proves the client holds the private key	Same
Revocation	Optional: CRL or OCSP; in practice short lifetimes replace revocation	Same

Client certificates and identity

- A client certificate is an ordinary X.509 certificate with `extendedKeyUsage = clientAuth`. A certificate with only `serverAuth` is rejected as a client certificate by strict servers.
- Identity lives in the **Subject** (CN) or, better, in a **Subject Alternative Name (SAN)**. The standards-based form is a URI SAN such as `spiffe://example.com/ns/prod/sa/shop-api`; DNS SANs are also used. Policies should match the full identity, not a substring.
- The server **trusts a set of CAs for client authentication**. Use a *separate* internal CA for workload identities; do not point `ssl_client_certificate` at the public web CA bundle, or anyone with a public certificate can authenticate.
- Proxies must forward the verified identity to the app in a trustworthy way (nginx variables `$ssl_client_s_dn`, `$ssl_client_verify`; Envoy `x-forwarded-client-cert` header). If a proxy terminates mTLS (TLS Termination §12.11), the application sees only what the proxy chooses to pass on, and must trust it only from that proxy.

SPIFFE and SPIRE vocabulary

Term	Meaning
Trust domain	The administrative boundary of an identity system, for example `example.com`; one root of trust
SPIFFE ID	A URI naming a workload: `spiffe://<trust-domain>/<path>`
SVID (SPIFFE Verifiable Identity Document)	The credential proving a SPIFFE ID: an **X509-SVID** (a certificate with the ID in a URI SAN) or a **JWT-SVID** (a signed token)
Trust bundle	The CA certificates that validate SVIDs of a trust domain; can be federated between domains
Workload API	Local API (Unix socket) from which a workload fetches its SVIDs and bundles, and receives updates when they rotate
Attestation	Evidence used to decide who a node or workload is, instead of a pre-shared secret

SVIDs are deliberately **short-lived** (SPIRE's defaults are on the order of an hour, configurable) and renewed well before expiry, so there is no long-lived key to steal and revocation is rarely needed.

Certificate rotation

Rotation is the hard part. Rotate three kinds of things, each differently:

- **Leaf (workload) certificates:** automate. The agent, sidecar or cert-manager renews at around half of the lifetime; the proxy or app must **reload without dropping connections** (Envoy's SDS, the Secret Discovery Service, pushes new certificates live; nginx needs `nginx -s reload`; most Go and Java servers can hot-load via a custom callback or library).
- **Intermediate CAs:** issue new intermediates from the root on a schedule shorter than their expiry; clients must receive the new chain.
- **Root / trust anchor:** the dangerous one. Use **overlap**: publish a bundle containing old and new roots, move signers to the new root, wait for all old leaves to expire, then drop the old root. Doing it in any other order breaks connections. The lab practises exactly this.

Revocation (CRL, OCSP) exists but is fragile at runtime; prefer lifetimes of hours to days over trying to revoke.

6 Step-by-step guide

You will build a private CA, a server that **requires** client certificates, three different clients (valid and allowed, valid but not allowed, forged), watch the handshake with `tcpdump`, then practise rotation, expiry and a CA-bundle rollover.

Prerequisites: Linux with `nginx`, `openssl`, `curl` and `tcpdump` (`sudo apt-get install -y nginx openssl curl tcpdump`; then `sudo systemctl disable --now nginx`). Free and local. Verified with nginx 1.24, OpenSSL 3.0 and curl 8.5. If your shell exports a proxy, run `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY`. The Docker Compose version simply mounts the same files into an `nginx` container and a `curlimages/curl` container; host processes make `tcpdump` easier.

1. **Create two CAs (the real internal one and a rogue one) and four certificates.**

```bash
export LAB=$HOME/mtls-lab; mkdir -p $LAB/certs $LAB/logs $LAB/tmp && cd $LAB/certs
mkca() { openssl req -x509 -newkey rsa:2048 -nodes -keyout $1.key -out $1.crt -days 30
  \
           -subj "/CN=$2" -addext "basicConstraints=critical,CA:TRUE" -addext
  "keyUsage=critical,keyCertSign,cRLSign"; }
mkca ca "Lab Internal CA"; mkca rogue-ca "Rogue CA"
issue() {   # issue <name> <CN> <SAN> <EKU> <signing-ca> <days>
  openssl req -newkey rsa:2048 -nodes -keyout $1.key -out $1.csr -subj "/CN=$2"
  printf "subjectAltName=%s\nextendedKeyUsage=%s\n" "$3" "$4" > $1.ext
  openssl x509 -req -in $1.csr -CA $5.crt -CAkey $5.key -CAcreateserial -days $6 -
  extfile $1.ext -out $1.crt
}
issue inventory inventory.internal "DNS:inventory.internal"
  serverAuth ca       30
issue shop-api  shop-api  "URI:spiffe://example.com/ns/prod/sa/shop-api"
  clientAuth ca       7
issue billing   billing   "URI:spiffe://example.com/ns/prod/sa/billing"
  clientAuth ca       7
issue rogue     shop-api  "URI:spiffe://example.com/ns/prod/sa/shop-api"
  clientAuth rogue-ca 7
openssl x509 -in shop-api.crt -noout -subject -ext subjectAltName,extendedKeyUsage -
  dates
```

```
subject=CN = shop-api
X509v3 Subject Alternative Name:
    URI:spiffe://example.com/ns/prod/sa/shop-api
X509v3 Extended Key Usage:
    TLS Web Client Authentication
notBefore=...
notAfter=...
```

The `rogue` certificate has the *same* name and SPIFFE ID as `shop-api` but is signed by a CA the server will not trust: the point is that a matching name proves nothing without the chain.

2. **Configure nginx to require and verify client certificates, and to authorize by identity.**

```bash
cd $LAB
cat > nginx.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/error.log info;
pid $LAB/nginx.pid;
events { worker_connections 256; }
http {
```

```
  access_log $LAB/logs/access.log;
  client_body_temp_path $LAB/tmp/body; proxy_temp_path $LAB/tmp/proxy;
  fastcgi_temp_path $LAB/tmp/fcgi; uwsgi_temp_path $LAB/tmp/uwsgi; scgi_temp_path
  $LAB/tmp/scgi;
  map \$ssl_client_s_dn \$caller_allowed { "CN=shop-api" 1; default 0; }   #
  authorization policy
  server {
    listen 127.0.0.1:9443 ssl;
    server_name inventory.internal;
    ssl_certificate        $LAB/certs/inventory.crt;
    ssl_certificate_key    $LAB/certs/inventory.key;
    ssl_client_certificate $LAB/certs/ca.crt;      # CAs trusted to sign CLIENT
  certificates
    ssl_verify_client      on;                      # on = required; optional = ask
  but allow none
    ssl_verify_depth       2;
    location / {
      if (\$caller_allowed = 0) { return 403 "authenticated as [\$ssl_client_s_dn] but
  not allowed\n"; }
      return 200 "hello \$ssl_client_s_dn verify=\$ssl_client_verify
  serial=\$ssl_client_serial days_left=\$ssl_client_v_remain\n";
    }
  }
}
EOF
mkdir -p tmp/body tmp/proxy tmp/fcgi tmp/uwsgi tmp/scgi
nginx -t -c $LAB/nginx.conf -e $LAB/logs/error.log && (nginx -c $LAB/nginx.conf -e
  $LAB/logs/error.log >/dev/null 2>&1 &)
sleep 1
```

Expected: `syntax is ok` / `test is successful`. Authentication happens in the TLS layer (`ssl_verify_client`); the `map` and `if` are the separate authorization step. (`if` in a `location` that only uses `return` is safe in nginx.)

3. **Try four clients.**

```bash
R="--cacert $LAB/certs/ca.crt --resolve inventory.internal:9443:127.0.0.1"
echo "1 no cert:";        curl -sS $R https://inventory.internal:9443/ | grep -o
  '<title>.*</title>'
echo "2 shop-api:";       curl -sS $R --cert $LAB/certs/shop-api.crt --key
  $LAB/certs/shop-api.key https://inventory.internal:9443/
echo "3 billing:";        curl -sS $R --cert $LAB/certs/billing.crt  --key
  $LAB/certs/billing.key  https://inventory.internal:9443/
echo "4 rogue-signed:";   curl -sS $R --cert $LAB/certs/rogue.crt    --key
  $LAB/certs/rogue.key    https://inventory.internal:9443/ | grep -o '<title>.*
  </title>'
```

```
1 no cert:
<title>400 No required SSL certificate was sent</title>
2 shop-api:
hello CN=shop-api verify=SUCCESS serial=45DF...79A4 days_left=6
3 billing:
authenticated as [CN=billing] but not allowed
4 rogue-signed:
<title>400 The SSL certificate error</title>
```

Client 1 never authenticated. Client 2 passed both steps. Client 3 passed **authentication** (valid certificate from the trusted CA) but failed **authorization** (`403`). Client 4 failed authentication despite claiming `shop-api` .

4. **Read the server's reasons.** Because the log level is `info` , nginx tells you exactly why:

```bash
grep -aE 'client SSL certificate verify error|no required SSL' $LAB/logs/error.log |
  sed 's/^.*\[info\] //' | cut -c1-150
```

```
... client SSL certificate verify error: (21:unable to verify the first certificate)
  while reading client request headers ...
... client sent no required SSL certificate while reading client request headers ...
```

Error 21 for the rogue certificate means "I cannot build a chain to a CA I trust". Remember these strings; they are what you will grep for in an incident.

5. **Watch the handshake: client certificates are visible in TLS 1.2 and hidden in TLS 1.3.** Capture the server port twice with different protocol versions.

```bash
for v in "--tlsv1.2 --tls-max 1.2" "--tlsv1.3"; do
  sudo timeout 5 tcpdump -i lo -nn -A -l 'tcp port 9443' > $LAB/cap.txt 2>/dev/null &
  sleep 1
  curl -sS -o /dev/null $R $v --cert $LAB/certs/shop-api.crt --key $LAB/certs/shop-
  api.key https://inventory.internal:9443/; wait
  echo "[$v] 'shop-api' in capture: $(grep -ac 'shop-api' $LAB/cap.txt), 'spiffe':
  $(grep -ac 'spiffe' $LAB/cap.txt)"
done
```

```
[--tlsv1.2 --tls-max 1.2] 'shop-api' in capture: 2, 'spiffe': 1
[--tlsv1.3] 'shop-api' in capture: 0, 'spiffe': 0
```

In TLS 1.2 the client's identity crosses the network in clear text for any observer; in TLS 1.3 it is encrypted. That is a good reason to set `ssl_protocols TLSv1.3` for internal mTLS where all clients support it. (Wireshark shows the same thing with the "Certificate Request" and "Certificate" messages decoded.)

6. **Rotate the leaf certificate.** A client simply starts presenting a new certificate; the server needs no change because it trusts the CA, not the individual certificate.

```bash
cd $LAB/certs
openssl req -newkey rsa:2048 -nodes -keyout shop-api-new.key -out shop-api-new.csr -
  subj "/CN=shop-api" 2>/dev/null
openssl x509 -req -in shop-api-new.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days
  7 -extfile shop-api.ext -out shop-api-new.crt 2>/dev/null
curl -sS $R --cert shop-api-new.crt --key shop-api-new.key
  https://inventory.internal:9443/
```

Expected: `hello CN=shop-api verify=SUCCESS serial=<a different serial> days_left=6`. In production the old and new certificates are valid at the same time, so rotation has no gap. What takes effort is making the *running process* load the new file: in this lab each `curl` reads files fresh, but a long-lived proxy such as nginx reads the certificate at start and on `nginx -s reload`.

7. **Expire a certificate.** `-days 0` makes a certificate that is already past `notAfter` a second later.

```bash
openssl x509 -req -in shop-api-new.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days
  0 -extfile shop-api.ext -out expired.crt 2>/dev/null
sleep 2
curl -sS -o /dev/null -w '%{http_code}\n' $R --cert expired.crt --key shop-api-new.key
  https://inventory.internal:9443/
grep -a 'verify error' $LAB/logs/error.log | tail -1 | sed 's/^.*\[info\] //' | cut -
  c1-110
openssl verify -CAfile ca.crt -purpose sslclient expired.crt
```

```
400
... client SSL certificate verify error: (10:certificate has expired) while reading
  client request headers
error 10 at 0 depth lookup: certificate has expired
```

This is the outage that automation exists to prevent. Note that the client sees a bare `400`; the reason is only in the server log.

8. **Roll the CA with an overlap bundle.** Create a new CA, issue a client certificate from it, and watch it fail until the server trusts both.

```bash
openssl req -x509 -newkey rsa:2048 -nodes -keyout ca2.key -out ca2.crt -days 30 -subj
  "/CN=Lab Internal CA v2" \
  -addext "basicConstraints=critical,CA:TRUE" -addext
  "keyUsage=critical,keyCertSign,cRLSign" 2>/dev/null
openssl req -newkey rsa:2048 -nodes -keyout shop-api-v2.key -out shop-api-v2.csr -subj
  "/CN=shop-api" 2>/dev/null
openssl x509 -req -in shop-api-v2.csr -CA ca2.crt -CAkey ca2.key -CAcreateserial -days
  7 -extfile shop-api.ext -out shop-api-v2.crt 2>/dev/null
curl -sS -o /dev/null -w 'before: %{http_code}\n' $R --cert shop-api-v2.crt --key
  shop-api-v2.key https://inventory.internal:9443/
cat ca.crt ca2.crt > trust-bundle.pem
sed -i "s#ssl_client_certificate .*#ssl_client_certificate $LAB/certs/trust-
  bundle.pem;#" $LAB/nginx.conf
nginx -t -c $LAB/nginx.conf -e $LAB/logs/error.log && nginx -s reload -c
  $LAB/nginx.conf -e $LAB/logs/error.log; sleep 1
curl -sS -o /dev/null -w 'new CA client:  %{http_code}\n' $R --cert shop-api-v2.crt --
  key shop-api-v2.key https://inventory.internal:9443/
curl -sS -o /dev/null -w 'old CA client:  %{http_code}\n' $R --cert shop-api.crt    --
  key shop-api.key    https://inventory.internal:9443/
```

```
before: 400
new CA client:  200
old CA client:  200
```

During the overlap both generations work, so you can migrate clients gradually. The final step (not run here) is to remove `ca.crt` from the bundle and reload after every old leaf has been replaced or has expired.

9. **Clean up.**

```bash
kill $(cat $LAB/nginx.pid); cd ~; rm -r "$LAB"; unset LAB
```

7 Real-world scenario

(Illustrative.) A platform team enables mTLS between `shop-api` and the `inventory` service using certificates issued once a year by a script and copied into Kubernetes Secrets. On the anniversary, at 02:10, every call from `shop-api` to `inventory` starts failing with `503 upstream connect error ... certificate has expired`, checkout stops, and the on-call engineer cannot find any recent deploy or code change. Investigation: dashboards show 100 percent failures between exactly these two services and nothing else. Envoy logs on the `inventory` side

state `CERTIFICATE_VERIFY_FAILED: certificate has expired`. `kubectl get secret ... -o jsonpath='{.data.tls\.crt}' | base64 -d | openssl x509 -noout -dates` confirms `notAfter` was an hour ago. Because the wiki page for renewal was written by someone who has left, the first 40 minutes go on rediscovering the script. Fix: reissue and restart, then, in the post-mortem, move to short-lived workload certificates issued by the mesh (or SPIRE) with automatic renewal at half-life, add an alert that fires when any certificate has under 14 days left, and rehearse a root-CA rollover with a trust-bundle overlap in staging. **Lesson:** long-lived certificates turn rotation into an annual emergency. Make rotation boring by making it frequent and automatic, and monitor expiry as a first-class metric.

8 Common mistakes and best practices

Don't

- Use the same CA for public web certificates and for client authentication, or trust a public CA bundle for `ssl_client_certificate`.
- Treat a valid client certificate as permission; always add an authorization step keyed on the verified identity.
- Match identities by loose substring (`*shop*`) or by CN alone when SANs are available.
- Issue long-lived (multi-year) workload certificates, or let one private key be copied across many workloads.
- Rotate the trust anchor by replacing it in place; that breaks every connection at once.
- Forget `extendedKeyUsage = clientAuth` and clock sync (NTP); both produce confusing handshake failures.

Do

- Prefer automatic, short-lived identities (SPIRE, a mesh CA, cert-manager with a private issuer) and rotate at about half of the lifetime.
- Use a dedicated private CA per environment, keep roots offline, and use intermediates for issuing.
- Use SPIFFE-style URI SANs as identities and write policy against them (mesh `AuthorizationPolicy`, Envoy RBAC, nginx `map`).
- Practise CA rollover with an overlap bundle and monitor "days until expiry" for every certificate and CA.
- Log the *reason* for verification failures server-side and alert on spikes; clients usually only see a generic error.
- Prefer TLS 1.3 internally so client identities are not visible on the wire.

9 Tasks

- [ ] **T1 — Build the mTLS server and four clients** *(≈45 min, intermediate)*
  - **Goal:** see authentication and authorization as separate steps.
  - **Steps:**
    1. Do steps 1-4 of the guide.
    2. Fill in a table: client, TLS result, HTTP status, and the log line that explains it.
  - **Done when:** your table shows `400` (no cert), `200` (shop-api), `403` (billing, valid but not allowed), `400` (rogue CA) with the matching `error.log` messages.

- [ ] **T2 — Observe the handshake** *(≈40 min, advanced)*
  - **Goal:** prove what is visible on the wire.
  - **Steps:**
    1. Do step 5, then repeat with `openssl s_client -connect 127.0.0.1:9443 -cert ... -key ... -tls1_2 -msg` and find the `CertificateRequest` message.
    2. Set `ssl_protocols TLSv1.3;` on the server, reload, and confirm a `--tls-max 1.2` client now fails.
  - **Done when:** you can show the SPIFFE ID in a TLS 1.2 capture and its absence in TLS 1.3, and the failing `--tls-max 1.2` call.

- [ ] **T3 — Break it three more ways** *(≈40 min, advanced)*
  - **Goal:** diagnose the common failure modes.
  - **Steps:**
    1. Issue a client certificate with `extendedKeyUsage = serverAuth` only and try it.
    2. Send the right certificate with the wrong key (`--key` from another client).
    3. Set `ssl_verify_client optional` and observe what `$ssl_client_verify` contains for a client with no certificate (`NONE`) and with a bad one (`FAILED:...`).
  - **Done when:** you can state, for each case, the status code, the nginx log message, and the one-line fix.

- [ ] **T4 — Rotate without downtime** *(≈45 min, advanced)*
  - **Goal:** make rotation a non-event.
  - **Steps:**
    1. Write a loop `while true; do curl ...; sleep 0.2; done` that calls the server and prints the status code.
    2. While it runs, perform the CA overlap rollover (step 8), then remove the old CA from the bundle once only new-CA clients remain.

3. Count non-200 results during each phase.

- **Done when:** the old-CA client fails only after you remove the old CA, the new-CA client succeeds throughout, and you can explain the order of operations.

☐ **T5 — Automate issuance with a script** *(≈60 min, advanced)*

- **Goal:** the "short-lived and automatic" model in miniature.
- **Steps:**
  1. Write a script that reissues `shop-api` as a 1-hour certificate every 30 minutes (or every 30 seconds with `-days 0`-style tricks for the demo) into a directory.
  2. Write a small client loop that re-reads the files for every request and a server-side check of `$ssl_client_v_remain`.
  3. Add a check that fails when a certificate has less than a threshold remaining (`openssl x509 -checkend`).
- **Done when:** the loop keeps getting `200` across several rotations, and the `-checkend` check flags a certificate that is about to expire.

☐ **T6 — Prove it: identity design for `shop-api`** *(≈60 min, advanced)*

- **Goal:** design and defend an mTLS rollout.
- **Steps:**
  1. Without the guide, rebuild the CA, server and two clients and demonstrate authentication-vs-authorization.
  2. Write a one-page plan for `shop-api`, `inventory`, `billing` and PostgreSQL: SPIFFE IDs, who issues certificates (SPIRE or mesh), lifetime, rotation trigger, how policy is expressed, how a root rollover is done, and the three alerts you would create.
- **Done when:** the demo works from memory and your plan names concrete lifetimes, an overlap procedure for the trust anchor, and an expiry alert threshold.

10 Self-check questions

1. What does mTLS add to ordinary TLS, and what does it still not do?
2. A client has a valid certificate from your internal CA but receives `403`. Which layer rejected it and why?
3. Why prefer a URI SAN such as a SPIFFE ID over the CN?
4. What are SPIFFE, SPIRE, a trust domain and an SVID?
5. How do you rotate a root CA without breaking existing connections?
6. Why are client identities visible in a TLS 1.2 packet capture but not in TLS 1.3?

ANSWERS

1. The server also demands a certificate from the client and verifies it, so both sides are authenticated cryptographically. It does not authorize actions; you still need policy on the verified identity, and it does not protect a compromised-but-valid workload.
2. The application/proxy authorization layer (here the nginx `map` plus `403`). The TLS layer accepted the certificate (authentication), but the identity is not on the allow-list.
3. SANs are the standard place for identities, can be typed (URI, DNS), are unambiguous and machine-parsable, and avoid the free-form ambiguity of CN; a SPIFFE ID names a workload independent of its network address.
4. SPIFFE is the spec for workload identity; SPIRE is its reference implementation (server plus per-node agents with attestation and the Workload API). A trust domain is the root-of-trust boundary (for example `example.com`). An SVID is the credential (X.509 certificate or JWT) proving a SPIFFE ID.
5. Add the new root alongside the old in every trust bundle, then start signing with the new root, wait until all certificates signed by the old root have been replaced or expired, then remove the old root. Never swap the root in place.
6. In TLS 1.2 the Certificate messages are sent before encryption is established; in TLS 1.3 everything after the ServerHello, including certificates, is encrypted.

11 Further reading

- SPIFFE and SPIRE documentation (spiffe.io): concepts, the SPIFFE specification, and the SPIRE deployment guides.
- Istio documentation on security and mutual TLS; Linkerd documentation on automatic mTLS and identity; Envoy documentation on TLS and the Secret Discovery Service.
- nginx documentation: `ngx_http_ssl_module` (`ssl_verify_client`, `ssl_client_certificate`, `ssl_crl` and the `$ssl_client_*` variables).
- RFC 8446 (TLS 1.3), RFC 5280 (X.509 and CRLs), and the NIST SP 800-207 *Zero Trust Architecture* publication.
- Ivan Ristic, *Bulletproof TLS and PKI*; Evan Gilman and Doug Barth, *Zero Trust Networks*.

LAB NOTES What I ran, what I saw, what it means

12.13 Rate Limiting

Topic 13 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

Rate limiting caps how much work a given caller (or the whole system) may ask of a service in a period of time: "100 requests per minute per API key", "5 login attempts per minute per IP", "10 concurrent exports per tenant". When a caller exceeds the limit, the service refuses the extra work quickly and cheaply, usually with HTTP **429 Too Many Requests** and a `Retry-After` header, instead of letting the excess overload everything for everyone.

It is one member of a family of overload controls that are easy to confuse:

- **Rate limiting / throttling:** limits *how fast* a caller may go (requests per second).
- **Quotas:** limit *how much in total* over a long period (10,000 calls per month, billing-related).
- **Concurrency limiting:** limits *how many at once* (in-flight requests), which tracks real load better than a rate when requests vary in cost.
- **Load shedding:** the server drops work when *it* is saturated, regardless of who sent it.
- **Circuit breakers** (Circuit Breakers §12.14) protect a *caller* from a failing dependency; rate limits protect a *server* from its callers.

This topic covers the four classic algorithms (token bucket, leaky bucket, fixed window, sliding window), where to enforce limits, and how to share a limit across many servers with Redis.

2 Why it matters in DevOps

- **Noisy neighbours and abuse.** One tenant's runaway script, a scraper, or a credential-stuffing botnet can consume the capacity everyone else pays for. A limit turns "site down for everyone" into "429 for one caller".
- **Protecting the expensive resource.** The app can often scale out; the database cannot. Limits at the edge keep the load the database sees bounded (Connection Pooling §12.10).

- **Retry storms are self-inflicted DDoS.** When a dependency slows down, clients retry, multiplying load exactly when capacity is shortest. Limits plus well-behaved clients (backoff with jitter, see Circuit Breakers §12.14) stop the spiral.
- **Cost control.** Pay-per-request backends (serverless, third-party APIs, LLM or SMS providers) can turn unlimited traffic into unlimited bills.
- **Fairness and contracts.** Public APIs sell tiers ("free: 60 per hour, pro: 5,000 per hour"); limits make those plans enforceable.
- **Naive implementations fail in production.** A per-process in-memory counter allows `N x limit` when you run `N` replicas; a Redis-backed counter without atomic updates races; a limiter that fails closed when Redis is down becomes an outage of its own.

3 Real-world system example

Stripe has published how it protects its API with rate limiters built on the token bucket algorithm and backed by Redis. It describes four layers: a request-rate limiter per user, a concurrent-requests limiter, a fleet-usage load shedder that reserves capacity for critical traffic, and a worker-utilisation load shedder that drops low-priority work first when servers are saturated. Its API returns `429` when you are limited, and Stripe's documentation advises clients to retry with exponential backoff.

GitHub's REST API documents primary rate limits per authenticated user or app (for example 5,000 requests per hour for a personal access token on github.com, with higher limits for some apps), exposes your state in `x-ratelimit-limit`, `x-ratelimit-remaining`, `x-ratelimit-used` and `x-ratelimit-reset` response headers, and applies additional "secondary" limits for rapid bursts and expensive operations. Clients are told to back off using `Retry-After` or the reset time.

Cloudflare has described building rate limiting that scales to millions of domains using a counter-based approximation of the sliding-window technique to avoid keeping every timestamp. **AWS API Gateway** documents a token bucket model with a steady-state rate and a burst size, applied per account, per stage and per API key through usage plans. **Envoy** supports both **local** rate limiting (a token bucket in each proxy) and **global** rate limiting by calling an external rate-limit service (Lyft's open-source `ratelimit` service uses Redis). **Kubernetes** protects its own API server with API Priority and Fairness, a form of concurrency limiting with queues per priority level. **nginx** (`limit_req`, `limit_conn`) and **HAProxy** (stick tables) provide limits at the proxy.

4 Diagram

Where limits are enforced in a typical request path, and what each layer protects:

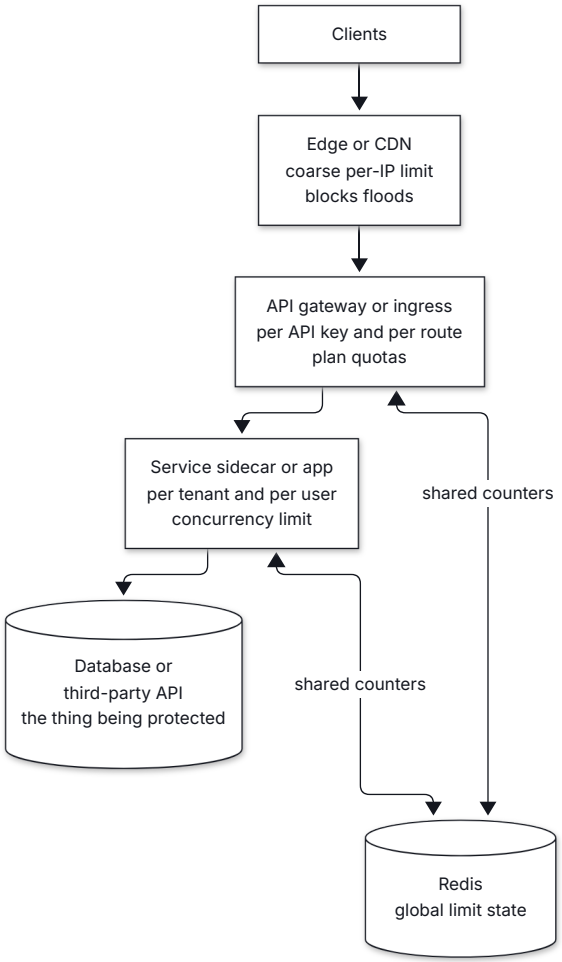


Figure 12.13-1

Token bucket: the state machine behind a request check. Tokens refill continuously up to the bucket size; each request needs one:

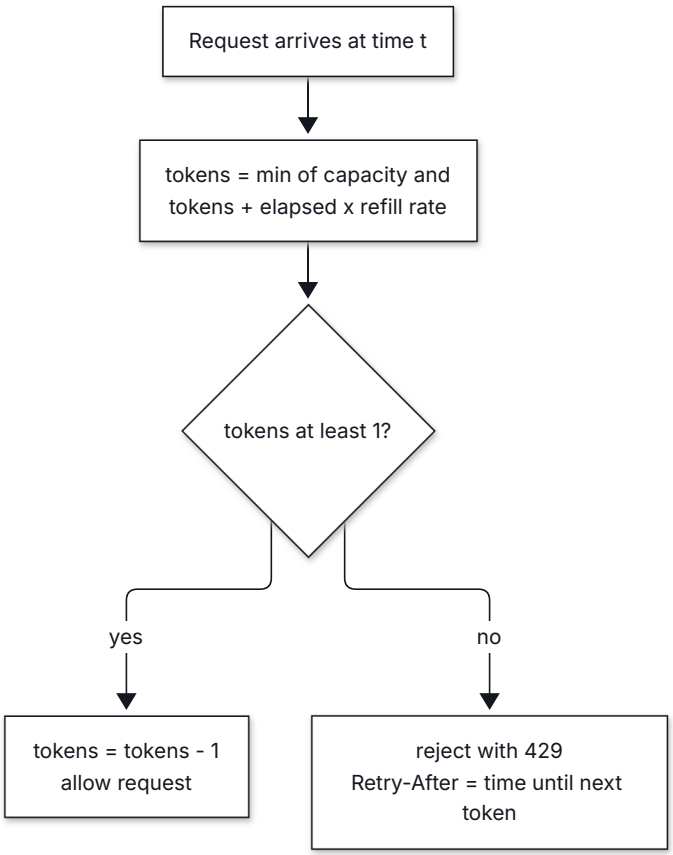


Figure 12.13-2

5 How it works

The algorithms

Algorithm	Idea	Allows bursts?	Memory per key	Strengths	Weaknesses
Token bucket	Bucket of up to `capacity` tokens refilled at `rate`; each request takes one	**Yes**, up to `capacity`	O(1): tokens + timestamp	Simple, permits controlled bursts, long-run rate is exact	Burst size and rate must be tuned together

Algorithm	Idea	Allows bursts?	Memory per key	Strengths	Weaknesses
Leaky bucket	Requests enter a queue (the bucket) drained at a fixed rate; a full queue rejects. As a *meter* it is equivalent to a token bucket with different bookkeeping	Absorbed by the queue, **output is smooth**	O(1) as a meter, O(queue) as a queue	Smooth, constant output to the backend	A queue adds latency; old requests may be served late
Fixed window	Counter per window (for example per minute), reset at window boundaries	Allows up to **2x the limit** across a boundary	O(1)	Trivial, one `INCR`	Boundary burst (the lab shows 20 of 20 allowed for "10 per second")
Sliding window log	Keep each request's timestamp; count those in the last window	No	**O(requests)**	Exact	Memory and CPU grow with the limit
Sliding window counter	Estimate = previous window count x overlap fraction + current count	Roughly bounded	O(1): two counters	Cheap and nearly accurate	An approximation (small over- or under-count)

Rule of thumb: **token bucket** for APIs that should tolerate short bursts, **leaky bucket / queue** when the backend needs a smooth flow (nginx's `limit_req` is a leaky bucket), **sliding window counter** for "N per minute" quotas at large scale, and **fixed window** only when approximate is fine.

Choosing what to count (the key)

A limit is a function `limit(key)`. Common keys: client IP (weak behind NAT or proxies and with IPv6), API key or token (best for APIs), authenticated user, tenant or account, route (`POST /login` stricter than `GET /products`), or a combination. Use cheaper limits earlier: a coarse per-IP limit at the edge, finer per-user limits later where identity is known. Behind proxies, take the client IP from a **trusted** `X-Forwarded-For`/PROXY protocol, not from the connection (TLS Termination §12.11).

What the client sees

- **Status** `429 Too Many Requests` (RFC 6585); `503` is used by some systems for load shedding rather than per-caller limits.
- `Retry-After` (RFC 9110): seconds or an HTTP date. Well-behaved clients wait at least that long.
- **Informational headers** such as `X-RateLimit-Limit`, `-Remaining`, `-Reset` (de facto; an IETF draft is standardising `RateLimit` headers). Document them; clients use them to pace themselves.

- A clear JSON error body naming the limit and how to raise it.

Local versus global (distributed) limiting

	Local (per instance)	Global (shared state)
Where state lives	In each proxy/app process	In a shared store, typically **Redis**, or a rate-limit service
Accuracy	Effective limit = `limit x instances` unless traffic is evenly spread	Accurate across the fleet
Latency / dependency	None, nothing to fail	One extra network call per request; the store is now a dependency
Best for	Protecting each instance from overload; cheap first line	Per-customer plans, billing-grade quotas

Practical designs combine them: a **local** limiter as a cheap guard and a **global** one for customer-visible limits, optionally syncing counts in batches (each node takes a slice of the budget from Redis) to avoid a round trip per request.

Doing it correctly in Redis. The check is read-modify-write, so it must be **atomic**: use a Lua script (executed atomically by Redis) or `MULTI/EXEC`; do not `GET`, compute in the app, then `SET` (two instances race and both pass). Use Redis's own clock (`TIME`) rather than each app server's clock to avoid skew. Give every key a TTL so idle buckets disappear. Typical data structures: `INCR` + `EXPIRE` (fixed window), a hash of `tokens` / `ts` (token bucket), a sorted set of timestamps (sliding window log). Decide **fail-open or fail-closed** when Redis is unreachable: public read APIs usually fail open (with a local fallback limit); security-critical limits such as login attempts may fail closed.

Tuning and pitfalls

- Set limits from data: measure normal p99 per-key traffic, add headroom, and ship in **log-only mode first** (nginx: `limit_req_dry_run on`, Envoy: a `filter_enabled` without `filter_enforced`) before enforcing.
- Rate by *cost*, not just count, when requests vary widely (an export is not a health check); or add a concurrency limit.
- Rejected requests should be cheap: reject early, before auth, parsing or database access.
- Limit retries too: a client that retries every `429` immediately makes things worse; reward `Retry-After` and add jitter.
- Keep state bounded: key cardinality (one key per IP) can itself exhaust memory; size zones and set TTLs (nginx `10m` zone holds roughly 160,000 states of 64 bytes).

6 Step-by-step guide

You will (a) simulate the algorithms and see why fixed windows leak bursts, (b) enforce a leaky-bucket limit in nginx and compare `burst`, `nodelay` and no burst, (c) write an atomic token bucket in Redis and share it between two clients, (d) rate limit in HAProxy with a stick table, and (e) read the equivalent Envoy configuration.

Prerequisites: Linux with `python3`, `nginx`, `haproxy`, `curl` and `redis-server` + `redis-cli` (`sudo apt-get install -y python3 nginx haproxy curl redis-server redis-tools`; then `sudo systemctl disable --now nginx haproxy redis-server`). Free and local. Verified with Python 3.13, nginx 1.24, HAProxy 2.x and Redis 7.0. If your shell exports a proxy, `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY`. Use Docker Compose if you prefer (one container each for nginx, HAProxy and Redis with the same files mounted); host processes keep the guide short.

Part A: the algorithms, with simulated time

1. **Write the algorithms with an injectable clock** so you can control time exactly.

```bash
export LAB=$HOME/ratelimit-lab; mkdir -p $LAB && cd $LAB
cat > limiters.py <<'EOF'
from collections import deque

class TokenBucket:                      # bursts up to capacity, average `rate` per
  second
    def __init__(self, rate, capacity):
        self.rate, self.capacity = rate, capacity
        self.tokens, self.last = capacity, 0.0
    def allow(self, now):
        self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False

class LeakyBucketMeter:                 # level drains at `rate`; a full bucket
  rejects
    def __init__(self, rate, capacity):
        self.rate, self.capacity = rate, capacity
        self.level, self.last = 0.0, 0.0
    def allow(self, now):
        self.level = max(0.0, self.level - (now - self.last) * self.rate)
        self.last = now
        if self.level + 1 <= self.capacity:
            self.level += 1
            return True
```

```
        return False

class FixedWindow:
    def __init__(self, limit, window):
        self.limit, self.window, self.count, self.start = limit, window, 0, 0
    def allow(self, now):
        w = int(now // self.window)
        if w != self.start:
            self.start, self.count = w, 0
        if self.count < self.limit:
            self.count += 1
            return True
        return False

class SlidingWindowLog:
    def __init__(self, limit, window):
        self.limit, self.window, self.log = limit, window, deque()
    def allow(self, now):
        while self.log and self.log[0] <= now - self.window:
            self.log.popleft()
        if len(self.log) < self.limit:
            self.log.append(now)
            return True
        return False

class SlidingWindowCounter:              # weights the previous window by how much of
  it still overlaps
    def __init__(self, limit, window):
        self.limit, self.window = limit, window
        self.cur_w, self.cur, self.prev = 0, 0, 0
    def allow(self, now):
        w = int(now // self.window)
        if w != self.cur_w:
            self.prev = self.cur if w == self.cur_w + 1 else 0
            self.cur, self.cur_w = 0, w
        frac = (now % self.window) / self.window
        if self.prev * (1 - frac) + self.cur < self.limit:
            self.cur += 1
            return True
        return False
EOF
```

2. **Run three traffic patterns through all of them.**

BASH

```
cat > simulate.py <<'EOF'
from limiters import *
LIMIT, WINDOW = 10, 1.0                  # "10 requests per second"
def run(name, lim, times):
    ok = sum(1 for t in times if lim.allow(t))
    print(f"{name:<26} allowed {ok:>2} of {len(times)}")
```

```
boundary = [0.95 + i * 0.001 for i in range(10)] + [1.05 + i * 0.001 for i in
  range(10)]
print("Scenario 1: 20 requests in 0.1 s across a window boundary")
run("fixed window",           FixedWindow(LIMIT, WINDOW),          boundary)
run("sliding window log",     SlidingWindowLog(LIMIT, WINDOW),     boundary)
run("sliding window counter", SlidingWindowCounter(LIMIT, WINDOW), boundary)
run("token bucket 10/s b10",  TokenBucket(10, 10),                 boundary)
run("leaky bucket 10/s q10",  LeakyBucketMeter(10, 10),            boundary)

burst = [5.0] * 30
print("\nScenario 2: quiet period, then 30 requests at once")
run("token bucket 10/s b10",  TokenBucket(10, 10),                 burst)
run("token bucket 10/s b30",  TokenBucket(10, 30),                 burst)
run("sliding window log",     SlidingWindowLog(LIMIT, WINDOW),     burst)

steady = [i / 20 for i in range(100)]
print("\nScenario 3: steady 20 requests per second for 5 s (long-run limit 10 per
  second)")
run("token bucket 10/s b10",  TokenBucket(10, 10),                 steady)
run("sliding window log",     SlidingWindowLog(LIMIT, WINDOW),     steady)
run("fixed window",           FixedWindow(LIMIT, WINDOW),          steady)
EOF
python3 simulate.py
```

```
Scenario 1: 20 requests in 0.1 s across a window boundary
fixed window               allowed 20 of 20
sliding window log         allowed 10 of 20
sliding window counter     allowed 11 of 20
token bucket 10/s b10      allowed 11 of 20
leaky bucket 10/s q10      allowed 11 of 20

Scenario 2: quiet period, then 30 requests at once
token bucket 10/s b10      allowed 10 of 30
token bucket 10/s b30      allowed 30 of 30
sliding window log         allowed 10 of 30

Scenario 3: steady 20 requests per second for 5 s (long-run limit 10 per second)
token bucket 10/s b10      allowed 59 of 100
sliding window log         allowed 50 of 100
fixed window               allowed 50 of 100
```

Read it: the **fixed window** let all 20 through in a tenth of a second (twice the intended limit) because 10 landed at the end of one window and 10 at the start of the next; the others did not (the token bucket's extra request is the token refilled in 0.1 s). A big bucket (`b30`) is a policy decision: it admits a 30-request burst after idle time. In scenario 3 the token bucket allows about 59: the initial full bucket (10) plus 5 seconds x 10 per second, which is the long-run rate plus one burst.

Part B: leaky-bucket limiting in nginx

3. **Configure `limit_req` with three variants** and a backend. Note that `limit_req` must apply to a request that is *proxied or served*; a bare `return` in the same `location` runs before the limit and would bypass it.

```bash
mkdir -p $LAB/logs $LAB/tmp/{body,proxy,fcgi,uwsgi,scgi}
cat > nginx.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/error.log warn;
pid $LAB/nginx.pid;
events { worker_connections 256; }
http {
  access_log off;
  client_body_temp_path $LAB/tmp/body; proxy_temp_path $LAB/tmp/proxy;
  fastcgi_temp_path $LAB/tmp/fcgi; uwsgi_temp_path $LAB/tmp/uwsgi; scgi_temp_path
  $LAB/tmp/scgi;

  limit_req_zone \$binary_remote_addr zone=per_ip:10m rate=5r/s;   # 5 per second per
  client IP
  limit_req_status 429;

  server {
    listen 127.0.0.1:8080;
    add_header Retry-After 1 always;
    location /strict  { limit_req zone=per_ip;                    proxy_pass
  http://127.0.0.1:8081; }
    location /burst   { limit_req zone=per_ip burst=10;           proxy_pass
  http://127.0.0.1:8081; }
    location /nodelay { limit_req zone=per_ip burst=10 nodelay;   proxy_pass
  http://127.0.0.1:8081; }
  }
  server {                                    # the protected backend
    listen 127.0.0.1:8081;
    location / { return 200 "ok\n"; }
  }
}
EOF
nginx -t -c $LAB/nginx.conf -e $LAB/logs/error.log && (nginx -c $LAB/nginx.conf -e
  $LAB/logs/error.log >/dev/null 2>&1 &)
sleep 1
```

Expected: `syntax is ok` and `test is successful`.

4. **Send 20 requests as fast as possible to each endpoint.**

```bash
burst() { for i in $(seq 1 20); do curl -s -o /dev/null -w '%{http_code} '
  http://127.0.0.1:8080$1; done; echo; }
echo strict:;  burst /strict;  sleep 3
echo burst:;   time burst /burst; sleep 4
echo nodelay:; burst /nodelay
```

```
strict:
200 429 429 429 429 429 429 429 429 429 429 429 429 429 429 429 429 429 429 429
burst:
200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200
real    0m3.8s
nodelay:
200 200 200 200 200 200 200 200 200 200 429 429 429 429 429 429 429 429 429 429
```

The three behaviours: **strict** admits one and rejects the rest of the instantaneous burst. `burst=10` (the leaky bucket's queue) *delays* excess requests so they leave at exactly 5 per second: the first ten queued requests plus pacing took about 3.8 seconds, yet nothing failed. `burst=10 nodelay` serves the first 11 immediately (1 + burst of 10) and rejects the rest; the slots then refill at 5 per second. Pick `nodelay` when latency matters more than smoothness; pick delaying when the backend needs a gentle flow.

5. **See the client-visible contract and the log.**

```bash
curl -si http://127.0.0.1:8080/strict >/dev/null; curl -si
  http://127.0.0.1:8080/strict | sed -n '1p;/Retry-After/p'
grep -a 'limiting requests' $LAB/logs/error.log | tail -1 | sed 's/^.*\
  [error\]/[error]/' | cut -c1-150
```

```
HTTP/1.1 429 Too Many Requests
Retry-After: 1
[error] ...: *309 limiting requests, excess: 10.485 by zone "per_ip", client:
  127.0.0.1, server: , request: "GET /nodelay HTTP/1.1" ...
```

`excess` is how far over the rate the bucket is, a handy number for tuning `burst`. Stop nginx for now: `kill $(cat $LAB/nginx.pid)`.

Part C: a distributed token bucket in Redis

6. **Start a throwaway Redis and write the atomic token bucket as a Lua script.**

```bash
```

```
cd $LAB
cat > tb.lua <<'EOF'
-- KEYS[1] = bucket key; ARGV[1] = refill rate (tokens/s), ARGV[2] = capacity, ARGV[3]
  = cost
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)              -- Redis clock in
  milliseconds
local rate, cap, cost = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens, ts = tonumber(b[1]), tonumber(b[2])
if tokens == nil then tokens, ts = cap, now end                 -- new key: start
  full
tokens = math.min(cap, tokens + (now - ts) * rate / 1000)       -- refill
local allowed = 0
if tokens >= cost then tokens = tokens - cost; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate * 1000) * 2) -- idle buckets
  expire
return { allowed, math.floor(tokens) }
EOF
redis-server --port 16379 --save "" --appendonly no --daemonize yes --dir $LAB --
  pidfile $LAB/redis.pid --logfile $LAB/redis.log >/dev/null
sleep 1
R="redis-cli -p 16379"
SHA=$($R SCRIPT LOAD "$(cat tb.lua)"); echo $SHA
```

Expected: a 40-character SHA1 of the script. `EVALSHA` runs it atomically, so no other client can interleave between the read and the write.

7. **Exercise it: bucket of 10, refill 5 per second.**

```
BASH
for i in $(seq 1 15); do $R EVALSHA $SHA 1 rl:user:42 5 10 1 | tr '\n' ' '; done; echo
sleep 1; echo "after 1 second:"; for i in $(seq 1 7); do $R EVALSHA $SHA 1 rl:user:42
  5 10 1 | tr '\n' ' '; done; echo
```

```
1 9 1 8 1 7 1 6 1 5 1 4 1 3 1 2 1 1 1 0 0 0 0 0 0 0 0 0 0
after 1 second:
1 4 1 3 1 2 1 1 1 0 0 0 0 0
```

Each pair is `allowed tokens-left`. The first 10 requests drain the bucket, the next 5 are refused (`0`). After one second about 5 tokens have refilled, so 5 more succeed.

8. **Prove the limit is shared and race-free: two independent clients, one bucket.** Two background loops (standing in for two app instances) hammer the same key with a bucket of

10 and a slow refill.

```bash
$R DEL rl:user:99 >/dev/null
( for i in $(seq 1 10); do $R EVALSHA $SHA 1 rl:user:99 1 10 1 | head -1; done ) >
  a.out &
( for i in $(seq 1 10); do $R EVALSHA $SHA 1 rl:user:99 1 10 1 | head -1; done ) >
  b.out &
wait; echo "allowed in total: $(cat a.out b.out | grep -c '^1')"
```

```
allowed in total: 10
```

Twenty attempts from two clients, exactly the bucket's 10 allowed (a refill of 1 per second can occasionally add one more on a slow machine). With per-process counters you would have seen 20. Sliding-window and fixed-window variants use the same pattern (`ZREMRANGEBYSCORE` / `ZCARD` / `ZADD` in Lua, or `INCR` + `EXPIRE`); try them in task T3.

Part D: HAProxy stick table and Envoy for comparison

9. **Rate limit in HAProxy by tracking each source IP's request rate.**

```bash
cat > haproxy.cfg <<'EOF'
global
  maxconn 1000
defaults
  mode http
  timeout connect 3s
  timeout client  30s
  timeout server  30s
frontend fe
  bind 127.0.0.1:8090
  stick-table type ip size 100k expire 30s store http_req_rate(1s)
  http-request track-sc0 src
  http-request deny deny_status 429 if { sc_http_req_rate(0) gt 5 }
  default_backend be
backend be
  server s1 127.0.0.1:8081
EOF
haproxy -c -f haproxy.cfg
(nginx -c $LAB/nginx.conf -e $LAB/logs/error.log >/dev/null 2>&1 &); sleep 1       #
  the 8081 backend
haproxy -f haproxy.cfg -D -p $LAB/haproxy.pid; sleep 1
for i in $(seq 1 12); do curl -s -o /dev/null -w '%{http_code} '
  http://127.0.0.1:8090/; done; echo
sleep 3; curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8090/
```

```
Configuration file is valid
200 200 200 200 200 429 429 429 429 429 429 429
200
```

The first five pass, then HAProxy denies until the measured one-second rate drops below five. Note that denied requests still count towards the rate, so a client that keeps hammering stays blocked; waiting is what frees it.

10. **Read the Envoy equivalent (reference only, not run in this lab).** Envoy's local rate limit filter is a token bucket in each proxy:

```yaml
http_filters:
  - name: envoy.filters.http.local_ratelimit
    typed_config:
      "@type":
  type.googleapis.com/envoy.extensions.filters.http.local_ratelimit.v3.LocalRateLimit
      stat_prefix: http_local_rate_limiter
      token_bucket:
        max_tokens: 10          # bucket capacity (burst)
        tokens_per_fill: 5      # refill amount
        fill_interval: 1s       # refill period → 5 per second
      filter_enabled:           # evaluate on 100% of requests
        runtime_key: local_rate_limit_enabled
        default_value: { numerator: 100, denominator: HUNDRED }
      filter_enforced:          # and enforce on 100% (set lower for a dry run)
        runtime_key: local_rate_limit_enforced
        default_value: { numerator: 100, denominator: HUNDRED }
  - name: envoy.filters.http.router
    typed_config:
      "@type": type.googleapis.com/envoy.extensions.filters.http.router.v3.Router
```

For fleet-wide per-customer limits Envoy's *global* rate limit filter calls an external gRPC rate-limit service, commonly backed by Redis as in Part C.

11. **Clean up.**

```bash
kill $(cat $LAB/haproxy.pid) $(cat $LAB/nginx.pid); redis-cli -p 16379 shutdown nosave
cd ~; rm -r "$LAB"; unset LAB
```

7 Real-world scenario

(Illustrative.) `shop-api` exposes `POST /login` and a public product search. One Saturday night, p99 latency climbs and the PostgreSQL CPU pins at 100 percent. Dashboards show that 70 percent of requests come from a few hundred IPs hitting `/login` with different email ad-

dresses (credential stuffing), and each attempt triggers a password-hash computation and a database lookup. Investigation: the only limit in place is a per-process counter inside the Node.js app: 100 requests per minute per IP. With 12 replicas the effective limit is up to 1,200 per minute per IP, and the botnet spreads across many IPs anyway. The counter is also checked *after* the database lookup. Fix: (1) at the edge, an nginx `limit_req` zone on `/login` of 5 per minute per IP with a small `burst`, plus a per-account limit of 10 attempts per hour kept in Redis with an atomic script (keyed by the submitted email, normalised); (2) the check runs before any database work; (3) the limit returns `429` with `Retry-After`; (4) global search gets a token bucket per API key; (5) a dry-run period first, then enforcement. CPU falls back within minutes. Follow-up: the security team notices the Redis limiter fails *open* during a Redis failover, so a short window of unlimited login attempts exists; they add a conservative local fallback limit and an alert on limiter errors. **Lesson:** limit early, on the right key, with shared atomic state, and decide explicitly what happens when the limiter itself fails.

8 Common mistakes and best practices

Don't

- Rely on per-instance counters for customer-facing limits when you run several replicas.
- Implement the check as `GET`, compute in the app, then `SET`; concurrent requests race through the gap.
- Use fixed windows for strict limits (the boundary doubles the burst) or client IP as the only key behind a NAT or CDN.
- Trust `X-Forwarded-For` from the open internet when picking the key.
- Do expensive work (auth, DB lookups) before the limit check.
- Reject with a vague `500`/`503` and no `Retry-After`, or let clients retry `429` immediately.
- Forget TTLs on limiter keys (memory grows without bound) or what happens when the limiter store is down.

Do

- Return `429` with `Retry-After` and clear rate-limit headers; document limits per plan.
- Combine layers: coarse per-IP at the edge, per-key/tenant at the gateway, concurrency limits near the database.
- Use token bucket for burst-tolerant APIs and a leaky bucket/queue for smoothing; use sliding-window counters for large-scale "per minute" quotas.
- Do shared limits in Redis with atomic Lua scripts and Redis's own clock; fail open or closed on purpose.

- Roll out in dry-run/log-only mode first; alert on 429 rates per key and on limiter errors.
- Limit retries on the client side too: exponential backoff with jitter and a retry budget (Circuit Breakers §12.14).

9 Tasks

- [ ] **T1 — Compare the algorithms** *(≈40 min, intermediate)*
  - **Goal:** see algorithmic differences in numbers.
  - **Steps:**
    1. Do Part A, steps 1-2.
    2. Add a fourth scenario: a request every 0.1 s for 10 seconds, with a 1-second pause in the middle.
    3. Record how many each algorithm allows and why.
  - **Done when:** you have a table of allowed counts per algorithm for each scenario and can explain the fixed-window boundary result in one sentence.
- [ ] **T2 — Tune nginx limits** *(≈40 min, intermediate)*
  - **Goal:** feel the difference between rejecting and delaying.
  - **Steps:**
    1. Do Part B.
    2. Change `rate` to `20r/m` for a new `/login` location with `burst=3 nodelay`, and send 8 requests.
    3. Use `ab -n 200 -c 10 http://127.0.0.1:8080/burst` and compare the latency distribution to `/nodelay`.
  - **Done when:** you can show the status codes for `/login`, and explain from `ab`'s percentiles why `/burst` has higher latency but zero 429s.
- [ ] **T3 — Build a sliding window in Redis** *(≈50 min, advanced)*
  - **Goal:** implement a second distributed algorithm and compare.
  - **Steps:**
    1. Write a Lua script using a sorted set: `ZREMRANGEBYSCORE key 0 (now-window)`, `ZCARD`, then `ZADD key now id` and `PEXPIRE` when under the limit.
    2. Run 12 requests with limit 10 per second: expect 10 allowed then 2 rejected, and a new success after about 1.1 s.
    3. Run the boundary pattern from Part A against it.

- **Done when:** the Redis sliding window allows exactly 10 of the 20-in-0.1-s boundary burst, unlike a fixed window (`INCR` / `EXPIRE`) which you also implement and show allowing 20.

- [ ] **T4 — Break it: race and failure modes** *(≈50 min, advanced)*
  - **Goal:** prove why atomicity and failure policy matter.
  - **Steps:**
    1. Implement a naive limiter in a shell loop using separate `GET` and `SET` commands, run two copies concurrently, and count how many get through compared with the Lua script.
    2. Stop Redis during a request loop and write down what your client sees; then decide fail-open vs fail-closed and implement a local fallback in the Python simulator.
  - **Done when:** the naive version lets through more than the limit and the Lua version never does, and you can state your fail-open or fail-closed choice for login and for search with reasons.

- [ ] **T5 — Per-key limits and a good client** *(≈60 min, advanced)*
  - **Goal:** complete the contract between limiter and caller.
  - **Steps:**
    1. Key the nginx zone on an `X-Api-Key` header instead of the IP (`limit_req_zone $http_x_api_key ...`) with two different keys.
    2. Return `Retry-After` computed from the Redis script (time until the next token).
    3. Write a client that respects `Retry-After` and adds jitter, and show it completes 100 requests without a single 429 after the first backoff.
  - **Done when:** two keys have independent limits (one blocked, the other not), and the client log shows waits that match `Retry-After`.

- [ ] **T6 — Prove it: limit plan for `shop-api`** *(≈45 min, advanced)*
  - **Goal:** design and demonstrate a layered limit.
  - **Steps:**
    1. Without the guide, demonstrate nginx `limit_req` with `nodelay` and a Redis atomic token bucket shared by two clients.
    2. Write the plan: edge limit, per-API-key limit, login limit, concurrency limit for `/export`, the key for each, algorithm, numbers, headers returned, dry-run plan, behaviour when Redis is down, and three metrics to alert on.

- **Done when:** both demos work from memory and the plan states a concrete number, key and algorithm for each limit, with a reason.

10 Self-check questions

1. Compare token bucket and leaky bucket. Which allows bursts, and which gives smooth output?
2. Why does a fixed-window limit of 10 per second sometimes allow 20 requests in a tenth of a second?
3. You run 8 replicas each with a local limit of 100 per minute per client. What is the effective limit, and how do you get an exact fleet-wide one?
4. Why must a Redis rate-limit check be atomic, and how do you make it so?
5. What should a client do when it receives `429` with `Retry-After: 30`?
6. What is the difference between rate limiting, load shedding and a circuit breaker?

ANSWERS

1. A token bucket permits bursts up to the bucket size while enforcing a long-run average. A leaky bucket drains at a constant rate, so its output is smooth, with a queue absorbing bursts (or rejecting when full).
2. The counter resets at the window boundary; 10 requests at the end of one window and 10 at the start of the next are both within their own window, giving 2x the limit in a short span.
3. Up to 800 per minute (if the client's requests spread over every replica). Keep the counter in shared state (Redis or a rate-limit service) so every replica checks the same bucket.
4. The check is read-modify-write; two concurrent requests could both read the same count and both pass. Use a Lua script or `MULTI/EXEC` so Redis runs it as one indivisible operation, and use Redis's `TIME` for the clock.
5. Stop sending, wait at least 30 seconds (plus jitter), then retry; do not retry immediately and do not retry in parallel. Reduce its own request rate going forward.
6. Rate limiting caps what each caller may send (protecting the server from its callers). Load shedding drops work when the server itself is overloaded, regardless of caller. A circuit breaker is on the caller side: it stops calling a failing dependency to let it recover and fail fast.

11 Further reading

- Stripe engineering blog, "Scaling your API with rate limiters"; GitHub REST API documentation on rate limits; Cloudflare's engineering blog on rate limiting at scale; AWS API Gateway documentation on throttling.

- nginx documentation: `ngx_http_limit_req_module` and `ngx_http_limit_conn_module`; HAProxy documentation on stick tables; Envoy documentation on local and global rate limiting.
- Redis documentation on Lua scripting (`EVAL`, `EVALSHA`) and the `TIME` command.
- RFC 6585 (429 Too Many Requests), RFC 9110 (`Retry-After`), and the IETF HTTPAPI working group's draft on RateLimit header fields.
- Betsy Beyer et al., *Site Reliability Engineering* (Google), chapters on handling overload and addressing cascading failures.

LAB NOTES What I ran, what I saw, what it means

12.14 Circuit Breakers

Topic 14 of 14 | ~5 hours | Difficulty: Advanced

1 What it is

A **circuit breaker** is a small state machine wrapped around a call to a remote dependency. While the dependency is healthy the breaker is **closed** and calls flow through. When failures pile up past a threshold, the breaker **opens**: calls are rejected *immediately*, without touching the network, so the failing dependency gets breathing room and the caller stops wasting threads, sockets and seconds waiting. After a cool-down the breaker goes **half-open** and lets a few trial calls through; if they succeed it closes again, if they fail it reopens. The name comes from the electrical device that trips to prevent a fire.

A breaker is one tool in a small toolkit for surviving partial failure. They work as a set, and each one covers a gap the others leave:

- **Timeouts** put a ceiling on how long any single call may take. Without them a hung dependency holds your threads forever.
- **Retries with exponential backoff and jitter** recover from brief, transient errors without hammering a struggling service.
- **Circuit breakers** stop calling something that is clearly down.
- **Bulkheads** isolate resources (thread pools, connection pools, semaphores) per dependency, so one slow dependency cannot consume everything. The name comes from the watertight compartments in a ship's hull.

Together they implement the idea that in a distributed system a slow dependency is worse than a dead one: **fail fast, fail small, recover gracefully.**

2 Why it matters in DevOps

- **Cascading failure is the dominant way large systems go down.** One slow service makes its callers' threads and connections pile up; they become slow; their callers pile up; within minutes a single degraded dependency has taken down the whole request path. Breakers, timeouts and bulkheads cut the chain.

- **Retries are the amplifier.** If each of three layers retries three times, one user request becomes 27 calls into the struggling service exactly when it can least cope. Retries without budgets, backoff and jitter turn a blip into an outage (a "retry storm" or "metastable failure").
- **A hung call is worse than an error.** An error returns in milliseconds and can be handled; a missing timeout holds a connection from your pool (Connection Pooling §12.10) until the pool is exhausted.
- **Graceful degradation needs a decision point.** An open breaker is the natural place to serve a fallback: cached data, a default, a "recommendations unavailable" panel, a queue for later processing.
- **It is operational, not just code.** Thresholds, timeouts and retry budgets are tuned from metrics and incidents; an open breaker is a signal you page on, and a good postmortem asks "which timeout and breaker should have caught this?".

3 Real-world system example

Netflix Hystrix popularised the pattern in the early 2010s: every dependency call was wrapped in a command with a timeout, a thread-pool or semaphore bulkhead, a circuit breaker and a fallback, with a real-time dashboard. Netflix put Hystrix into maintenance mode in 2018 and points to its successors (Resilience4j for Java); the ideas live on in Resilience4j, Polly for .NET, `gobreaker` and similar libraries, and in proxies.

Envoy (and so Istio and many gateways) separates two mechanisms that are often both called "circuit breaking": **circuit breakers** are per-cluster *concurrency limits* (maximum connections, pending requests, parallel requests, parallel retries, and an optional retry budget), while **outlier detection** *ejects* individual endpoints that return consecutive 5xx errors for a while and brings them back later, which is a passive, per-host breaker. Envoy's retry policy uses a jittered exponential backoff. **Istio** exposes these as `DestinationRule` `connectionPool` and `outlierDetection` settings (Service Mesh §12.9, Level 16: Mesh traffic management §16.10). **nginx** gives a passive form with `max_fails` / `fail_timeout` and `proxy_next_upstream`; **HAProxy** has `observe layer7` / `on-error` passive health checking.

AWS published guidance in the Amazon Builders' Library on "Timeouts, retries and backoff with jitter" and on load shedding, and AWS SDKs implement exponential backoff with jitter and client-side retry throttling by default. **Google's SRE book** describes the same family of techniques under "Handling Overload" and "Addressing Cascading Failures", including per-client retry budgets. **gRPC** supports retry policies and a retry throttling token bucket. Michael Nygard's book *Release It!* is where the names "circuit breaker" and "bulkhead" entered mainstream engineering vocabulary.

4 Diagram

The circuit breaker state machine:

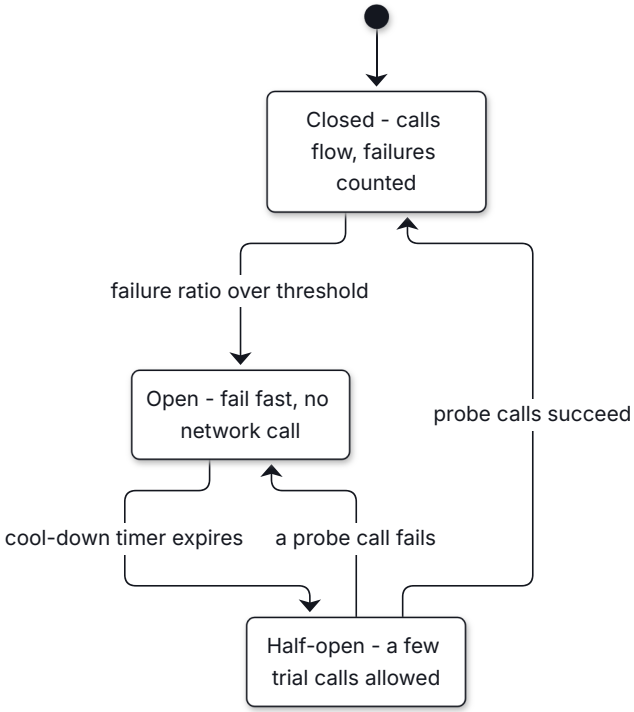


Figure 12.14-1

A caller combining timeout, retry, breaker and bulkhead around one dependency, in the or-

der they are applied:

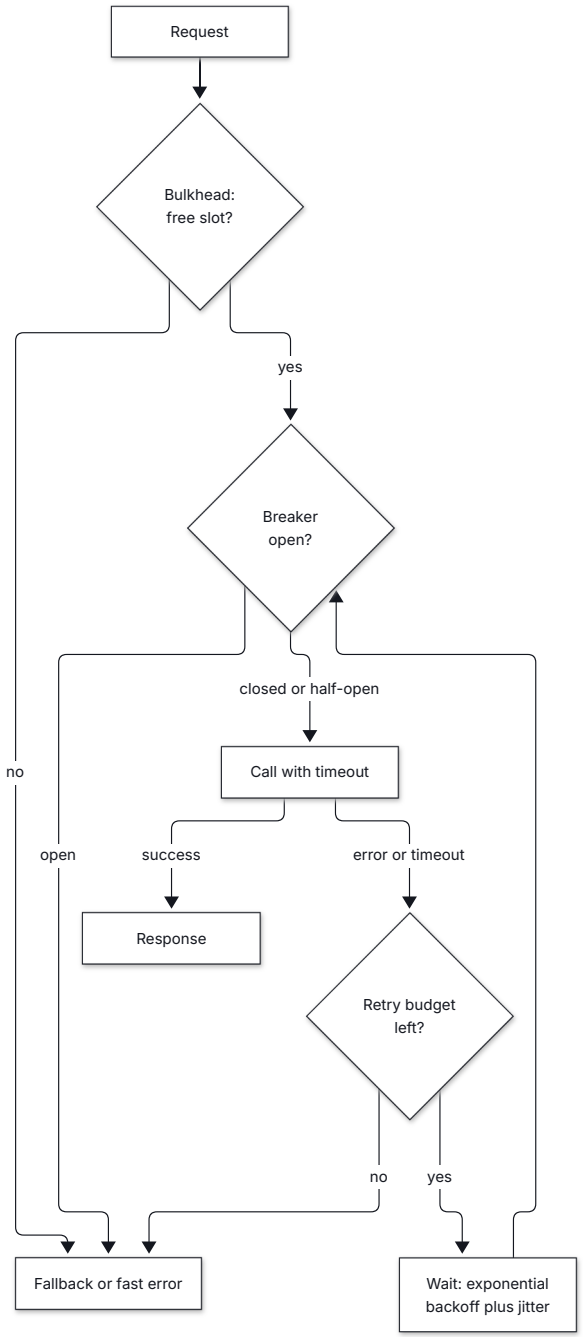


Figure 12.14-2

5 How it works

Timeouts: the foundation

Every network call needs a **connect timeout** and a **total/read timeout**, set from the dependency's observed latency (for example slightly above its p99.9), not left at library defaults (which are often infinite). Propagate a **deadline** down the call chain, so a request that has already used 900 ms of a 1 s budget does not start a 1 s call downstream. Timeouts must also shrink as you go deeper: the caller's timeout must exceed the callee's, or the caller gives up while the callee keeps working for nobody.

Retries, backoff and jitter

- **Retry only what is safe:** idempotent operations (GET, PUT, DELETE) or requests carrying an idempotency key (see Level 13: Idempotency §13.14). Retrying a non-idempotent `POST /charge` can double-charge.
- **Retry only transient errors:** connection failures, timeouts, 429/503 (honouring `Retry-After`), not 4xx client errors.
- **Exponential backoff:** wait `base x 2^attempt`, capped at a maximum.
- **Jitter:** randomise the wait so thousands of clients that failed together do not retry together. **Full jitter** picks `uniform(0, min(cap, base x 2^attempt))`. The lab shows 200 clients going from 200 simultaneous retries to a peak of about 30.
- **Budgets:** cap retries globally, for example "no more than 10 percent of requests may be retries" (Envoy `retry_budget`, gRPC retry throttling), and retry at one layer only; otherwise the multiplication across layers (3 x 3 x 3 = 27) takes the dependency down.

The breaker

State	Behaviour	Moves on
Closed	Calls pass through; outcomes recorded in a count- or time-based sliding window	Failure rate or slow-call rate over the threshold (after a minimum number of calls) opens it
Open	Calls fail immediately with a "circuit open" error (or run the fallback)	After a wait duration it becomes half-open
Half-open	A limited number of probe calls are allowed	Enough successes close it, any failure reopens it (often with a longer wait)

Design choices that matter:

- **What counts as failure:** exceptions, timeouts, 5xx (not 4xx), and **slow calls** (many libraries count calls slower than a threshold, since slowness is the real killer).

- **Minimum volume:** do not open on 2 failures out of 2 requests at 3 a.m.; require a minimum number of calls in the window.
- **Granularity:** one breaker *per dependency* (and often per endpoint or per host), not one global breaker. Envoy's outlier detection works per endpoint so one bad pod is ejected while the rest serve.
- **Probe discipline:** in half-open let only one or a few calls through so a recovering service is not stampeded.
- **Fallbacks:** cached or default data, degraded UI, enqueue for later. A fallback must itself be cheap and reliable.
- **Observability:** export state changes, rejected-call counts and failure rates; alert when a breaker has been open for several minutes (Observability §0.10).

Bulkheads

Give each dependency its own bounded resource: a separate thread pool, a semaphore with a fixed number of permits, a separate connection pool, or separate worker processes. When `inventory` degrades it exhausts its own compartment and calls to it are rejected with "bulkhead full", while `payments` keeps its capacity. Size compartments by Little's Law (concurrency = throughput x latency) plus headroom. At the infrastructure level the same idea appears as per-tenant queues, separate node pools or separate clusters (cell-based architecture).

Where to implement it

Layer	Pros	Cons
Library in the app (Resilience4j, Polly, `gobreaker`)	Understands business errors, easy fallbacks, per-method tuning	Per-language, every team must adopt it, config drift
Sidecar or proxy (Envoy, Istio, Linkerd, nginx, HAProxy)	Uniform across languages, changed by config, rich telemetry	Limited fallback logic, generic error classification
Both	Proxy handles connection-level protection; app handles semantics and fallbacks	Must avoid doubled retries (retry at one layer only)

Related Envoy settings (reference)

```yaml
clusters:
  - name: inventory
    type: STRICT_DNS
    connect_timeout: 1s
    circuit_breakers:                    # concurrency limits (bulkhead-like), not a
  failure-rate state machine
      thresholds:
        - priority: DEFAULT
```

```
          max_connections: 100
          max_pending_requests: 50
          max_requests: 200
          max_retries: 3
          retry_budget: { budget_percent: { value: 20.0 }, min_retry_concurrency: 3 }
    outlier_detection:                    # passive per-endpoint breaker (eject, then re-
  admit)
      consecutive_5xx: 5
      interval: 10s
      base_ejection_time: 30s
      max_ejection_percent: 50
# in the route:
#   timeout: 2s
#   retry_policy: { retry_on: "5xx,connect-failure,reset", num_retries: 2,
  per_try_timeout: 0.5s,
#                   retry_back_off: { base_interval: 0.1s, max_interval: 1s } }
```

6 Step-by-step guide

You will build a flaky dependency, watch a naive client suffer, wrap the client in a circuit breaker and see it trip, probe and recover, quantify jitter, demonstrate a bulkhead, and use nginx's passive failover. Everything is plain Python 3 and nginx: no Docker needed.

Prerequisites: Linux with `python3` (3.9+), `curl` and `nginx` (`sudo apt-get install -y python3 curl nginx`; then `sudo systemctl disable --now nginx`). Free and local. Verified with Python 3.13, nginx 1.24 and curl 8.5. If your shell exports a proxy, `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY`. Timings will vary slightly on your machine.

1. **Create the flaky `inventory` service.** It listens on port 9100 and misbehaves on command: `ok`, `slow` (sleeps 2 seconds) or `error` (HTTP 500).

```bash
export LAB=$HOME/breaker-lab; mkdir -p $LAB && cd $LAB
cat > backend.py <<'EOF'
"""Flaky 'inventory' service. POST /mode/ok | /mode/slow | /mode/error changes its
  behaviour."""
import json, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

MODE = {"m": "ok"}
STATS = {"requests": 0}

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def log_message(self, *a): pass
    def _send(self, code, body):
        data = body.encode()
        self.send_response(code)
```

```
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)
    def do_POST(self):
        if self.path.startswith("/mode/"):
            MODE["m"] = self.path.split("/")[2]
            return self._send(200, f"mode={MODE['m']}\n")
        self._send(404, "no\n")
    def do_GET(self):
        STATS["requests"] += 1
        if self.path == "/stats":
            return self._send(200, json.dumps({**STATS, "mode": MODE["m"]}) + "\n")
        m = MODE["m"]
        if m == "slow":
            time.sleep(2)
        if m == "error":
            return self._send(500, "boom\n")
        self._send(200, "stock=42\n")

if __name__ == "__main__":
    ThreadingHTTPServer(("127.0.0.1", 9100), H).serve_forever()
EOF
python3 backend.py > backend.log 2>&1 &
echo $! > backend.pid
sleep 1; curl -s http://127.0.0.1:9100/work
```

Expected: `stock=42`.

2. **Write the resilience toolkit: a circuit breaker, a bulkhead and a jittered backoff generator.**

```bash
cat > breaker.py <<'EOF'
import random, threading, time
from collections import deque

class CircuitOpen(Exception): pass
class BulkheadFull(Exception): pass

class CircuitBreaker:
    CLOSED, OPEN, HALF_OPEN = "closed", "open", "half-open"
    def __init__(self, window=10, min_calls=5, failure_ratio=0.5, open_seconds=3.0,
                 probes_to_close=2, clock=time.monotonic):
        self.window, self.min_calls, self.ratio = window, min_calls, failure_ratio
        self.open_seconds, self.probes_to_close, self.clock = open_seconds,
  probes_to_close, clock
        self.state, self.results = self.CLOSED, deque(maxlen=window)
        self.opened_at, self.probe_ok = 0.0, 0
        self.lock = threading.Lock()

    def _transition(self, new):
```

```
        if new ≠ self.state:
            print(f"    [breaker] {self.state} → {new}")
            self.state = new

    def before_call(self):                      # raise CircuitOpen to fail fast
        with self.lock:
            if self.state == self.OPEN:
                if self.clock() - self.opened_at ⩾ self.open_seconds:
                    self._transition(self.HALF_OPEN); self.probe_ok = 0
                else:
                    raise CircuitOpen()

    def record(self, ok):                       # report the outcome of a call that
  was allowed
        with self.lock:
            if self.state == self.HALF_OPEN:
                if not ok:
                    self.opened_at = self.clock(); self._transition(self.OPEN)
                else:
                    self.probe_ok += 1
                    if self.probe_ok ⩾ self.probes_to_close:
                        self.results.clear(); self._transition(self.CLOSED)
                return
            self.results.append(ok)
            fails = self.results.count(False)
            if len(self.results) ⩾ self.min_calls and fails / len(self.results) ⩾
  self.ratio:
                self.opened_at = self.clock(); self._transition(self.OPEN)

class Bulkhead:
    def __init__(self, max_concurrent):
        self.sem = threading.BoundedSemaphore(max_concurrent)
    def __enter__(self):
        if not self.sem.acquire(blocking=False):
            raise BulkheadFull()
    def __exit__(self, *exc):
        self.sem.release()

def backoff_delays(base=0.1, cap=5.0, attempts=5, jitter="full", rng=random):
    """Delay before retry n: exponential, capped, optionally fully jittered."""
    for n in range(attempts):
        exp = min(cap, base * 2 ** n)
        yield rng.uniform(0, exp) if jitter == "full" else exp
EOF
python3 -c "import breaker; print('toolkit loads')"
```

Expected: `toolkit loads`. (This breaker is deliberately minimal and single-threaded in its probing; production libraries add per-call permits in half-open state, slow-call tracking, metrics and thread-safe counters.)

3. **Write a client that calls the service every 200 ms with a 300 ms timeout**, with or without the breaker.

```bash
cat > demo.py <<'EOF'
import sys, time, urllib.request, urllib.error
from breaker import CircuitBreaker, CircuitOpen

URL, TIMEOUT = "http://127.0.0.1:9100/work", 0.3

def call():
    try:
        with urllib.request.urlopen(URL, timeout=TIMEOUT) as r:
            return r.status == 200
    except (urllib.error.URLError, TimeoutError, OSError):
        return False                            # 5xx raises HTTPError, a URLError
  subclass

use_breaker = sys.argv[1] == "breaker"
cb = CircuitBreaker(open_seconds=3.0)
t0 = time.monotonic()
for i in range(1, 41):
    start = time.monotonic()
    try:
        if use_breaker: cb.before_call()
        ok = call()
        if use_breaker: cb.record(ok)
        outcome = "ok" if ok else "FAIL (timeout or 5xx)"
    except CircuitOpen:
        outcome = "rejected instantly by open breaker"
    print(f"t={start - t0:5.1f}s call {i:2d}: {outcome:<36} took {(time.monotonic() -
  start) * 1000:5.0f} ms")
    time.sleep(0.2)
EOF
```

4. **Break the dependency and run the naive client.**

```bash
curl -s -X POST http://127.0.0.1:9100/mode/slow
timeout 8 python3 demo.py naive
```

```
mode=slow
t=  0.0s call  1: FAIL (timeout or 5xx)               took   319 ms
t=  0.5s call  2: FAIL (timeout or 5xx)               took   301 ms
t=  1.0s call  3: FAIL (timeout or 5xx)               took   301 ms
...
```

Every call burns the full 300 ms timeout. With a real thread pool, each of those is a thread occupied for 300 ms for nothing, and the dependency still receives every request, which is exactly what it does not need. (The timeout at least bounds the damage: remove `timeout=TIMEOUT` and each call hangs for 2 seconds.)

5. **Run with the breaker, and let the dependency recover after 4 seconds.**

```bash
curl -s -X POST http://127.0.0.1:9100/mode/slow >/dev/null
( sleep 4; curl -s -X POST http://127.0.0.1:9100/mode/ok >/dev/null ) &
python3 demo.py breaker | grep -v 'rejected instantly'
wait
```

```
t=  0.0s call  1: FAIL (timeout or 5xx)                took   319 ms
t=  0.5s call  2: FAIL (timeout or 5xx)                took   301 ms
t=  1.0s call  3: FAIL (timeout or 5xx)                took   301 ms
t=  1.5s call  4: FAIL (timeout or 5xx)                took   301 ms
    [breaker] closed → open
t=  2.0s call  5: FAIL (timeout or 5xx)                took   301 ms
    [breaker] open → half-open
t=  5.3s call 20: ok                                   took     2 ms
    [breaker] half-open → closed
t=  5.5s call 21: ok                                   took     2 ms
...
```

Run it again without the `grep` to see the whole story: five failed calls (the minimum volume, then a 100 percent failure ratio) trip the breaker; calls 6-19 are **rejected in 0 ms** for the 3-second cool-down, and during that time the dependency receives no traffic; call 20 is the first half-open probe, which succeeds because the service recovered, the second probe closes the breaker, and normal calls resume. Notice the recovery time is bounded by the cool-down, not by the client noticing: if the backend had still been slow, the probe would have failed (taking 300 ms) and the breaker would have gone back to open.

6. **Measure what jitter does to a retry storm.** 200 clients all fail at the same instant and retry four times with exponential backoff.

```bash
cat > herd.py <<'EOF'
import random
from collections import Counter
from breaker import backoff_delays

random.seed(7)
def schedule(jitter, clients=200, attempts=4):
    times = []
    for _ in range(clients):
```

```
        t = 0.0
        for d in backoff_delays(base=0.5, cap=8.0, attempts=attempts, jitter=jitter):
            t += d
            times.append(t)
    return times

for jitter in ("none", "full"):
    times = schedule(jitter)
    buckets = Counter(int(t // 0.05) for t in times)          # requests per 50 ms
  bucket
    print(f"jitter={jitter:<4} retries={len(times)}  busiest 50 ms buckets (requests):
  {[n for _, n in buckets.most_common(3)]}")
EOF
python3 herd.py
```

```
jitter=none retries=800  busiest 50 ms buckets (requests): [200, 200, 200]
jitter=full retries=800  busiest 50 ms buckets (requests): [32, 29, 28]
```

Same total work, but without jitter all 200 clients hit the recovering service in the same 50 ms, round after round, in synchronised waves; with full jitter the peak is about six times lower. That synchronised wave is the "thundering herd" that keeps a service pinned down.

7. **Demonstrate a bulkhead.** A degraded `inventory` (1 s per call) and a healthy `payments` (50 ms per call) share a pool of 4 workers, then get separate pools.

```bash
cat > bulkhead_demo.py <<'EOF'
import time
from concurrent.futures import ThreadPoolExecutor

def inventory(): time.sleep(1.0)        # degraded dependency
def payments():  time.sleep(0.05)       # healthy dependency

def run(label, inv_pool, pay_pool):
    inv = [inv_pool.submit(inventory) for _ in range(8)]      # a burst of inventory
  calls arrives first
    time.sleep(0.1)
    start = time.monotonic()
    pay_pool.submit(payments).result()                        # then one payment
    print(f"{label:<24} payment finished in {(time.monotonic() - start) * 1000:6.0f}
  ms")
    for f in inv: f.result()

shared = ThreadPoolExecutor(max_workers=4)
run("one shared pool of 4", shared, shared); shared.shutdown()
a, b = ThreadPoolExecutor(max_workers=3), ThreadPoolExecutor(max_workers=1)
run("bulkheads 3 + 1", a, b); a.shutdown(); b.shutdown()
EOF
```

```
python3 bulkhead_demo.py
```

```
one shared pool of 4     payment finished in   1950 ms
bulkheads 3 + 1          payment finished in     51 ms
```

In the shared pool the slow dependency occupies every worker and a healthy payment waits behind eight 1-second calls (about 2 seconds). With a bulkhead the payment finishes in 51 ms. This is why a slow recommendations service must never be able to take down checkout.

8. **Passive circuit breaking in nginx.** Put nginx in front of two replicas: the flaky one on 9100 and a healthy one on 9101. nginx counts failures per server (`max_fails`) and stops sending to a failed server for `fail_timeout` seconds; once that period ends it lets a request through again (a probe).

```bash
mkdir -p $LAB/logs $LAB/tmp/{body,proxy,fcgi,uwsgi,scgi}
cat > nginx.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/error.log warn;
pid $LAB/nginx.pid;
events { worker_connections 256; }
http {
  log_format up '\$status via \$upstream_addr in \$request_time s (statuses
  \$upstream_status)';
  access_log $LAB/logs/access.log up;
  client_body_temp_path $LAB/tmp/body; proxy_temp_path $LAB/tmp/proxy;
  fastcgi_temp_path $LAB/tmp/fcgi; uwsgi_temp_path $LAB/tmp/uwsgi; scgi_temp_path
  $LAB/tmp/scgi;

  upstream inventory {
    server 127.0.0.1:9100 max_fails=2 fail_timeout=5s;    # the flaky one
    server 127.0.0.1:9101;                                 # a healthy replica
  }
  server {
    listen 127.0.0.1:8080;
    location / {
      proxy_pass http://inventory;
      proxy_connect_timeout 1s;
      proxy_read_timeout 500ms;                            # a per-try timeout
      proxy_next_upstream error timeout http_500;          # what counts as a failure
  and may be retried
      proxy_next_upstream_tries 2;                         # retry budget: at most one
  retry
      proxy_next_upstream_timeout 2s;                      # overall cap across tries
    }
  }
```

```
  server { listen 127.0.0.1:9101; access_log off; location / { return 200 "healthy
  replica\n"; } }
}
EOF
nginx -t -c $LAB/nginx.conf -e $LAB/logs/error.log && (nginx -c $LAB/nginx.conf -e
  $LAB/logs/error.log >/dev/null 2>&1 &)
curl -s -X POST http://127.0.0.1:9100/mode/error; sleep 1
for i in $(seq 1 6); do curl -s -o /dev/null -w '%{http_code} '
  http://127.0.0.1:8080/; done; echo
cat $LAB/logs/access.log
```

```
200 200 200 200 200 200
200 via 127.0.0.1:9100, 127.0.0.1:9101 in 0.001 s (statuses 500, 200)
200 via 127.0.0.1:9101 in 0.000 s (statuses 200)
200 via 127.0.0.1:9100, 127.0.0.1:9101 in 0.001 s (statuses 500, 200)
200 via 127.0.0.1:9101 in 0.000 s (statuses 200)
200 via 127.0.0.1:9101 in 0.000 s (statuses 200)
200 via 127.0.0.1:9101 in 0.000 s (statuses 200)
```

The client never saw an error: requests 1 and 3 hit the bad server, got a `500`, and nginx retried on the healthy one (the `statuses 500, 200` lines). After the second failure the flaky server was marked unavailable, so requests 4-6 went straight to the healthy replica. Wait 6 seconds and send three more: one request goes to `9100` again as a probe (the half-open step), fails, and the server is parked again. Now try the slow failure:

BASH

```
curl -s -X POST http://127.0.0.1:9100/mode/slow; sleep 6
for i in 1 2 3; do curl -s -o /dev/null -w '%{http_code} %{time_total}s\n'
  http://127.0.0.1:8080/; done
```

```
200 0.000538s
200 0.501425s
200 0.000503s
```

One request paid the 500 ms per-try timeout and was retried successfully. This is a *passive* breaker (it learns from live traffic, with no separate health check) and it is crude compared with a library breaker: no failure-rate window, no half-open probe count, and failures only matter if `proxy_next_upstream` lists them.

9. **Clean up.**

BASH

```
kill $(cat $LAB/nginx.pid) $(cat $LAB/backend.pid)
cd ~; rm -r "$LAB"; unset LAB
```

7 Real-world scenario

(Illustrative.) On a busy Friday, `shop-api`'s product page calls a recommendations service that starts taking 20 seconds per request after a bad deploy of its cache layer. `shop-api` has no timeout on that call (the HTTP client default is infinite) and uses one shared thread pool of 200 for all downstream calls. Within a minute every thread is blocked on recommendations, checkout requests wait for a thread, and the load balancer's health checks begin to fail, removing instances from rotation and concentrating load on the rest. The site is down although only an optional widget is broken. Investigation: thread dumps show 200 of 200 threads in `socketRead`; recommendations latency graphs are off the chart; error rate on `shop-api` is 100 percent, almost all timeouts at the load balancer. Retries in the mobile app, three attempts each, tripled the incoming load. Fix: (1) immediate: a feature flag turns the recommendations call off; (2) permanent: a 300 ms timeout and one retry with full jitter on idempotent GETs, a breaker per downstream (open after 50 percent failures or slow calls over 20 requests, 10-second cool-down), a bulkhead of 20 concurrent calls for recommendations, and a fallback of "popular items" from a local cache; (3) the mobile client gets exponential backoff with jitter and honours `Retry-After`; (4) an alert fires when any breaker has been open for 2 minutes and a game-day drill injects 20-second latency to prove the page degrades instead of dying. **Lesson:** optional dependencies must not be able to take down critical paths. Timeouts, bulkheads and breakers turn a total outage into a missing widget.

8 Common mistakes and best practices

Don't

- Call anything over the network without a timeout, or leave library defaults (often infinite) in place.
- Retry at every layer (client, gateway, service, SDK); retries multiply (3 x 3 x 3 = 27 calls).
- Retry without jitter or without a budget, or retry non-idempotent operations without idempotency keys.
- Use one global breaker or one shared thread pool for every dependency.
- Open on tiny samples (2 of 2 requests failed) or count 4xx client errors as dependency failures.
- Let half-open admit all traffic at once; the recovering dependency collapses again.
- Ship a fallback that calls the same failing dependency, or never test the fallback.

Do

- Set connect and request timeouts per dependency, propagate deadlines, and keep caller timeouts above callee timeouts.

- Retry only transient, idempotent failures: exponential backoff, full jitter, a retry budget (for example 10 percent of traffic), honouring `Retry-After`.
- Use a breaker per dependency (and per endpoint where the library or proxy supports it), with minimum volume, slow-call detection and a limited half-open probe.
- Isolate dependencies with bulkheads (separate pools or semaphores) sized by Little's Law.
- Design graceful degradation: cached data, defaults, queues, feature flags, "partial page" rendering.
- Export breaker state, rejection counts and retry rates; alert on open breakers; rehearse failures with chaos or game-day exercises.
- Combine app-level breakers (semantics, fallbacks) with proxy-level protection (outlier detection, concurrency limits), and keep retry at one layer.

9 Tasks

- [ ] **T1 — Trip and recover the breaker** *(≈40 min, intermediate)*
  - **Goal:** watch all three states happen.
  - **Steps:**
    1. Do steps 1-5.
    2. Repeat but switch the backend to `ok` only after 12 seconds, so at least one half-open probe fails.
    3. Record the sequence of transitions with timestamps.
  - **Done when:** your log shows `closed → open → half-open → open → half-open → closed`, and you can explain why the failed probe reopened the breaker.
- [ ] **T2 — Tune it** *(≈45 min, advanced)*
  - **Goal:** understand what each parameter trades off.
  - **Steps:**
    1. Run the demo with `min_calls=2`, then `min_calls=20`, `failure_ratio=0.9`, and `open_seconds=1` and `open_seconds=10`.
    2. For each configuration count how many calls hit the dependency while it was down and how long recovery took after it came back.
  - **Done when:** you have a table of (parameters, wasted calls, time to recover) and a paragraph explaining which setting you would pick for a payment service and why.
- [ ] **T3 — Add retries with jitter and a budget** *(≈60 min, advanced)*
  - **Goal:** combine timeout, retry, breaker and fallback correctly.

- **Steps:**
  1. Write a `resilient_get()` that applies, in order: bulkhead, breaker, timeout, up to 2 retries using `backoff_delays`, then a fallback returning `stock=unknown`.
  2. Add a global retry budget (retries allowed only while retries are below 10 percent of calls in the last 10 seconds).
  3. Run it against `slow`, `error` and `ok` modes and count backend hits with `curl http://127.0.0.1:9100/stats`.
- **Done when:** in `error` mode the backend sees at most about 1.1x the requests the client attempted (budget respected), and the client prints the fallback instead of exceptions.

☐ **T4 — Break it: retry amplification** *(≈45 min, advanced)*

- **Goal:** prove that naive retries make outages worse.
- **Steps:**
  1. Start `slow` mode and run 20 concurrent clients (shell `&` or threads) that each retry 3 times with no backoff and no breaker; read `/stats` to see total requests.
  2. Repeat with breaker, jittered backoff and a budget.
  3. Compare backend request counts and the client-observed error latency.
- **Done when:** you can show the request multiplication factor for both setups (for example 4x versus close to 1x) and explain why layered retries multiply.

☐ **T5 — Production-style breaker at the proxy** *(≈60 min, advanced)*

- **Goal:** use outlier detection instead of app code.
- **Steps:**
  1. Write an Envoy bootstrap (or Istio `DestinationRule`) for a cluster of two endpoints using the `circuit_breakers` and `outlier_detection` block from section 5, with the flaky backend as one endpoint. Run it with the `envoyproxy/envoy` Docker image or `func-e`.
  2. Send traffic with `curl` while the flaky endpoint errors; inspect `/clusters` and `/stats` on the Envoy admin port for ejections (`outlier_detection.ejections_active`) and overflows.
  3. Add `retry_policy` with `per_try_timeout` and `retry_back_off`.
- **Done when:** the admin stats show the bad endpoint ejected and later re-admitted, and clients mostly receive `200`.

☐ **T6 — Prove it: resilience design for `shop-api`** *(≈60 min, advanced)*

- **Goal:** design and defend a resilience policy.

- **Steps:**
  1. Without the guide, reproduce steps 4-5 (naive client versus breaker) and the bulkhead demo, and explain the output.
  2. Write the policy for three `shop-api` dependencies (PostgreSQL, a payment provider, a recommendations service): timeout, retries (and whether safe), breaker thresholds, bulkhead size, fallback, and the alert for each.
- **Done when:** the demos run from memory, and your policy gives numeric values and a distinct fallback strategy for each dependency (including "no fallback: fail the checkout cleanly" for payments).

10 Self-check questions

1. Describe the three states of a circuit breaker and what moves it between them.
2. Why add jitter to exponential backoff? What problem does it solve that backoff alone does not?
3. Why can retrying at every layer of a call chain take a system down?
4. What is a bulkhead, and how does it differ from a circuit breaker?
5. Which calls are safe to retry automatically?
6. What is the difference between Envoy's `circuit_breakers` and `outlier_detection`?

ANSWERS

1. Closed: calls pass and failures are counted; when the failure (or slow-call) rate crosses a threshold after a minimum volume, it opens. Open: calls fail immediately for a cool-down period; then it becomes half-open. Half-open: a few probe calls are allowed; if they succeed it closes, if one fails it reopens.
2. Backoff spreads one client's retries over time, but clients that failed together still retry together in synchronised waves. Randomising the delay (full jitter) spreads the herd, lowering the peak load on the recovering service.
3. Retries multiply: three layers with three attempts each can produce 27 calls for one user request, at the moment the dependency is least able to cope. Retry at one layer, with budgets, backoff and jitter.
4. A bulkhead limits resources (threads, connections, concurrency) per dependency so one slow dependency cannot starve others; it limits the blast radius. A circuit breaker watches failures and stops calling a failing dependency. They complement each other.
5. Idempotent operations (GET, PUT, DELETE, or POSTs with an idempotency key) that failed with transient errors (connection failures, timeouts, 429 or 503), within a retry budget; not 4xx errors or non-idempotent writes without keys.

6. `circuit_breakers` set per-cluster concurrency limits (connections, pending requests, requests, retries) that reject overflow immediately; `outlier_detection` observes failures per endpoint, ejects bad hosts for a while, then re-admits them (a passive per-host breaker).

11 Further reading

- Michael Nygard, *Release It!* (stability patterns: timeouts, circuit breaker, bulkheads); Martin Fowler's article "CircuitBreaker" on martinfowler.com.
- Amazon Builders' Library: "Timeouts, retries and backoff with jitter" and the article on using load shedding to avoid overload; AWS Architecture Blog on exponential backoff and jitter.
- Google, *Site Reliability Engineering*: the chapters "Handling Overload" and "Addressing Cascading Failures".
- Resilience4j documentation (CircuitBreaker, Bulkhead, Retry); the Netflix Hystrix wiki (archived); Polly documentation for .NET.
- Envoy documentation on circuit breaking, outlier detection and retry policies; Istio documentation on `DestinationRule` traffic policy; nginx `ngx_http_upstream_module` and `ngx_http_proxy_module`.

★ Secure Edge and Resilient Services Lab

Level 12 capstone ~8 hours

1 Goal and outcome

You will build, break, observe and repair a small but realistic traffic path for `shop-api`: **an nginx edge that terminates TLS, two `shop-api` replicas, and an `inventory` service that only accepts calls over mutual TLS**, with **rate limiting** at the edge and a **circuit breaker, timeouts, retries with jitter and a bulkhead** inside `shop-api`. You will watch every hop with `curl`, `openssl`, `ss` and `tcpdump` and prove each property with a script.

This is the "Practice project" for Level 12 from the roadmap: *a lab with TLS termination, mTLS between two services, rate limiting and a circuit breaker, observed with tcpdump and curl.* By the end you can:

- explain, with packet captures as evidence, which hop is plaintext and which is encrypted, and why (TLS Termination §12.11);
- issue, present, authorise, rotate and expire a client certificate, and read the matching server log line (mTLS §12.12, TLS §12.4);
- make a service reject a flood with `429 Retry-After` instead of falling over (Rate Limiting §12.13);
- make a service degrade gracefully, with a fast fallback, when a dependency hangs, and recover by itself (Circuit Breakers §12.14, Connection Pooling §12.10);
- map the toy lab onto the real internet path of DNS, BGP, CDN, VPN and mesh (DNS §12.3, BGP §12.2, CDNs §12.7, VPNs §12.8, Service Mesh §12.9).

2 Architecture

Everything runs on `127.0.0.1`; each box is a separate process with its own port, so packet captures on the loopback interface show exactly what a network between machines would carry.

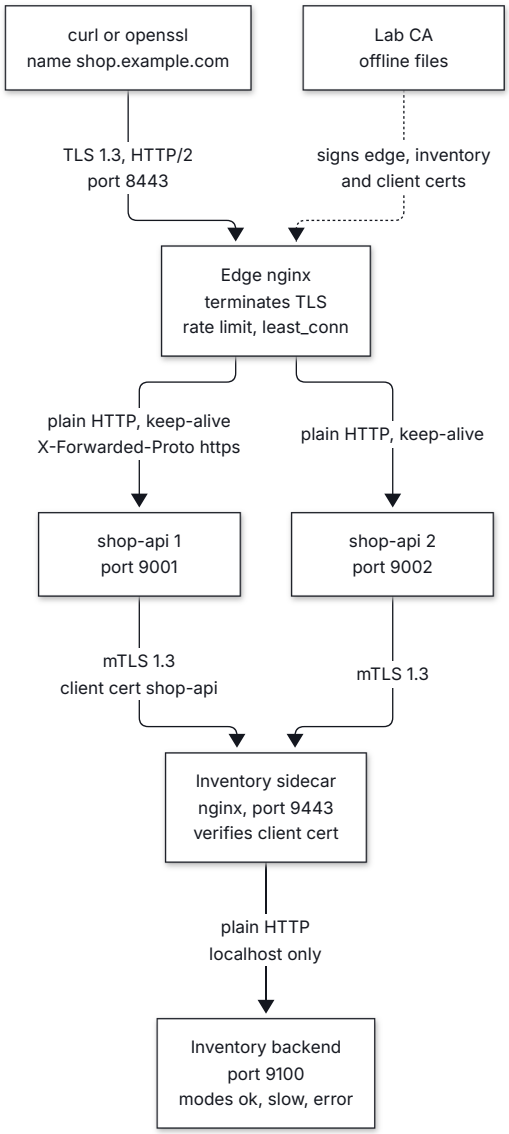


Figure 12-1

What a single `GET /api/stock` does, and where each protection acts:

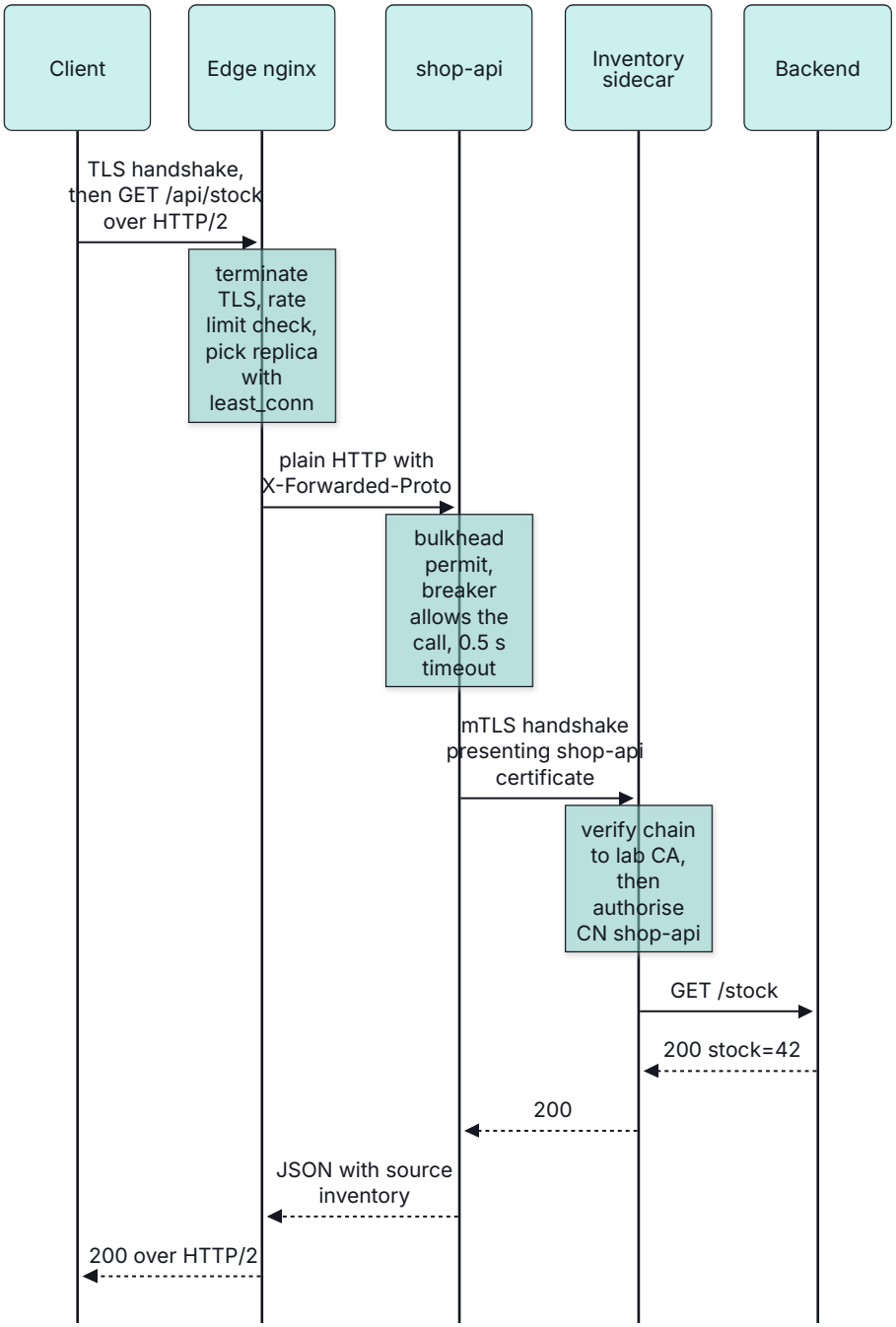


Figure 12-2

How the 14 topics of Level 12 show up in the lab:

Topic	Where you meet it
01 OSI and TCP/IP §12.1	Milestone 4: you read L3/L4 (`ss`, ports), L5/L6 (TLS version, ALPN) and L7 (HTTP) from the same capture
02 BGP §12.2	Task T6: how the edge address is announced and anycast in the real world
03 DNS §12.3	`--resolve` plays the role of DNS for `shop.example.com` and `inventory.internal`; T6 designs the real records
04 TLS §12.4	Milestone 0: your own CA, SAN certificates, chain verification, TLS 1.3
05 HTTP/2, HTTP/3, QUIC §12.5	Edge speaks HTTP/2 (ALPN `h2`); stretch goal adds HTTP/3
06 Load balancing §12.6	`least_conn` across two `shop-api` replicas, passive failover
07 CDNs §12.7	T6: where a CDN would sit in front of the edge and what it changes
08 VPNs §12.8	T6: replacing "localhost only" with a private tunnel to the inventory network
09 Service mesh §12.9	The inventory sidecar is a hand-built mesh sidecar; stretch goal replaces it with Envoy
10 Connection pooling §12.10	Edge-to-app keep-alive pool (`ss` shows reuse); bulkhead; the missing pool in `shop-api` is a task
11 TLS termination §12.11	Edge terminates, inventory sidecar re-establishes TLS on the next hop
12 mTLS §12.12	Client certificate with a SPIFFE-style identity, authorisation by CN, rotation and expiry drills
13 Rate limiting §12.13	`limit_req` with burst at the edge, `429` with `Retry-After`
14 Circuit breakers §12.14	Breaker, timeout, jittered retry and bulkhead in `shop-api`; breaker drill

3 Prerequisites and cost

- **Cost:** none. Everything is free and local; no cloud account, no containers required.
- **Machine:** Linux (a VM or WSL2 is fine) with `sudo` for `tcpdump`. About 1 GB of RAM and a few hundred MB of disk.
- **Tools:** `python3` (3.9 or newer), `nginx` (1.18+; the lab was verified on 1.24, which uses `listen ... ssl http2`; on 1.25.1+ prefer `listen ... ssl; http2 on;`, the old form still works with a warning), `openssl` 3, `curl` (with HTTP/2), `tcpdump`, `iproute2` (`ss`).

```bash
sudo apt-get install -y python3 nginx openssl curl tcpdump iproute2 bc
sudo systemctl disable --now nginx        # we run our own nginx instances as ordinary
  processes
```

- **Ports used on 127.0.0.1:** 8443 (edge), 9001 and 9002 (`shop-api`), 9443 (inventory sidecar), 9100 (inventory backend).
- **Proxies:** if your shell exports `https_proxy` / `HTTPS_PROXY`, run `unset https_proxy HTTPS_PROXY http_proxy HTTP_PROXY` so `curl` talks to localhost directly.
- **Security note:** keys are generated in the lab directory and are for this exercise only. The CA key sits next to the leaf keys purely for convenience; in real life a CA key lives offline or in a KMS/HSM.
- **Docker alternative:** if you prefer containers, run each process from `python:3.12-slim` and `nginx:stable` images with these same files mounted and the `127.0.0.1` addresses replaced by Compose service names (and `listen` addresses by `0.0.0.0`). The host-process version is used here because `tcpdump -i lo` is simpler and every step below was run end to end.

4 Build it step by step

Work in one directory. All commands assume you stay in `$LAB`.

```bash
export LAB=$HOME/l12-capstone; mkdir -p $LAB && cd $LAB
```

Milestone 0 — A private CA and four certificates (topics 03, 04)

One script creates the CA and four certificates: the **edge** server certificate for `shop.example.com`, the **inventory** server certificate for `inventory.internal`, a **client** certificate for `shop-api` that carries a SPIFFE-style URI identity, and a second valid client certificate for `billing` that we will use to prove authentication is not authorisation.

```bash
cat > make-certs.sh <<'EOF'
#!/usr/bin/env bash
# Creates a private CA and the four certificates used by the lab. Lab use only.
set -euo pipefail
mkdir -p certs && cd certs
openssl req -x509 -newkey rsa:2048 -nodes -keyout ca.key -out ca.crt -days 30 -subj
  "/CN=Lab Root CA" \
  -addext "basicConstraints=critical,CA:TRUE" -addext
  "keyUsage=critical,keyCertSign,cRLSign" 2>/dev/null
issue() {   # issue <name> <CN> <SAN> <EKU> <days>
  openssl req -newkey rsa:2048 -nodes -keyout "$1.key" -out "$1.csr" -subj "/CN=$2"
  2>/dev/null
  printf "subjectAltName=%s\nextendedKeyUsage=%s\n" "$3" "$4" > "$1.ext"
  openssl x509 -req -in "$1.csr" -CA ca.crt -CAkey ca.key -CAcreateserial -days "$5" -
  extfile "$1.ext" -out "$1.crt" 2>/dev/null
}
```

```
issue edge      shop.example.com "DNS:shop.example.com,DNS:localhost,IP:127.0.0.1"
  serverAuth 30
issue inventory inventory.internal "DNS:inventory.internal"
  serverAuth 30
issue shop-api  shop-api  "URI:spiffe://example.com/ns/prod/sa/shop-api"
  clientAuth 7
issue billing   billing   "URI:spiffe://example.com/ns/prod/sa/billing"
  clientAuth 7
rm -f *.csr *.ext *.srl
chmod 600 *.key
openssl verify -CAfile ca.crt edge.crt inventory.crt shop-api.crt billing.crt
EOF
chmod +x make-certs.sh && ./make-certs.sh
cp certs/shop-api.crt certs/client.crt; cp certs/shop-api.key certs/client.key     # the
  identity shop-api will present
```

Expected: four `OK` lines (`edge.crt: OK`, `inventory.crt: OK`, `shop-api.crt: OK`, `billing.crt: OK`). Inspect one: `openssl x509 -in certs/shop-api.crt -noout -subject -ext subjectAltName,extendedKeyUsage -dates` should show `CN = shop-api`, the URI SAN `spiffe://example.com/ns/prod/sa/shop-api`, usage `TLS Web Client Authentication` and a 7-day validity.

Milestone 1 — Inventory behind an mTLS sidecar (topics 12, 04)

The backend is a deliberately unreliable service whose mode you flip with a `POST`. The sidecar is an nginx that **requires** a client certificate from our CA, **authorises** only the identity `CN=shop-api`, and forwards to the backend over localhost.

BASH

```
cat > backend.py <<'EOF'
"""Flaky 'inventory' service. POST /mode/ok | /mode/slow | /mode/error changes its
  behaviour."""
import json, threading, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

MODE = {"m": "ok"}
STATS = {"requests": 0}

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def log_message(self, *a): pass
    def _send(self, code, body):
        data = body.encode()
        self.send_response(code)
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)
    def do_POST(self):
        if self.path.startswith("/mode/"):
```

```
            MODE["m"] = self.path.split("/")[2]
            return self._send(200, f"mode={MODE['m']}\n")
        self._send(404, "no\n")
    def do_GET(self):
        STATS["requests"] += 1
        if self.path == "/stats":
            return self._send(200, json.dumps({**STATS, "mode": MODE["m"]}) + "\n")
        m = MODE["m"]
        if m == "slow":
            time.sleep(2)
        if m == "error":
            return self._send(500, "boom\n")
        self._send(200, "stock=42\n")

if __name__ == "__main__":
    ThreadingHTTPServer(("127.0.0.1", 9100), H).serve_forever()
EOF
mkdir -p logs tmp/{body,proxy,fcgi,uwsgi,scgi}
cat > tmp.conf <<EOF
client_body_temp_path $LAB/tmp/body; proxy_temp_path $LAB/tmp/proxy; fastcgi_temp_path
  $LAB/tmp/fcgi; uwsgi_temp_path $LAB/tmp/uwsgi; scgi_temp_path $LAB/tmp/scgi;
EOF
cat > inventory.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/inv-error.log info;
pid $LAB/inv.pid;
events { worker_connections 256; }
http {
  log_format inv '\$ssl_protocol caller=[\$ssl_client_s_dn] verify=\$ssl_client_verify
  "\$request" \$status';
  access_log $LAB/logs/inv-access.log inv;
  include $LAB/tmp.conf;

  map \$ssl_client_s_dn \$caller_allowed { "CN=shop-api" 1; default 0; }

  server {
    listen 127.0.0.1:9443 ssl;
    server_name inventory.internal;
    ssl_certificate        $LAB/certs/inventory.crt;
    ssl_certificate_key    $LAB/certs/inventory.key;
    ssl_protocols          TLSv1.3;
    ssl_client_certificate $LAB/certs/ca.crt;
    ssl_verify_client      on;
    location / {
      if (\$caller_allowed = 0) { return 403 "caller not allowed\n"; }
      proxy_pass http://127.0.0.1:9100;
      proxy_connect_timeout 1s;
      proxy_read_timeout 5s;
    }
  }
}
EOF
```

```
nginx -t -c $LAB/inventory.conf -e $LAB/logs/inv-error.log
```

Expected: `syntax is ok` and `test is successful`. (The unquoted heredoc expands `$LAB`; the `\$` escapes keep nginx variables intact.)

Milestone 2 — `shop-api` with timeout, retry, bulkhead and circuit breaker (topics 14, 10, 12)

`shop-api` is a small Python service. It calls inventory over mTLS using the client certificate (re-reading it when the file changes, which makes rotation a non-event), with: a 0.5 s per-try **timeout**; one **retry** using full-jitter backoff; a **bulkhead** of 4 concurrent inventory calls; and a **circuit breaker** (opens at 50 percent failures over at least 5 calls in a window of 10, stays open 5 s, then needs 2 successful probes, one at a time). When the dependency is unavailable it returns a **fallback** (`stock: unknown`) instead of an error.

BASH

```
cat > app.py <<'EOF'
"""shop-api: calls inventory over mTLS, protected by timeout, retry with jitter, bulkhead
  and circuit breaker."""
import json, os, random, socket, ssl, sys, threading, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

HERE = os.path.dirname(os.path.abspath(__file__))
CERTS = os.path.join(HERE, "certs")
PORT = int(sys.argv[1]) if len(sys.argv) > 1 else 9001
INVENTORY = ("127.0.0.1", 9443)
INVENTORY_NAME = "inventory.internal"          # name that must appear in the server
  certificate
TIMEOUT = 0.5                                  # per-try timeout in seconds
MAX_TRIES = 2                                  # 1 call + 1 retry
BULKHEAD = threading.BoundedSemaphore(4)       # at most 4 concurrent inventory calls

def log(msg):
    print(time.strftime("%H:%M:%S"), msg, file=sys.stderr, flush=True)

# ---------- circuit breaker (closed → open → half-open → closed) ----------
class Breaker:
    def __init__(self, window=10, min_calls=5, ratio=0.5, open_seconds=5.0,
  probes_to_close=2):
        self.window, self.min_calls, self.ratio = window, min_calls, ratio
        self.open_seconds, self.probes_to_close = open_seconds, probes_to_close
        self.state, self.results, self.opened_at = "closed", [], 0.0
        self.probe_ok, self.probing, self.rejected = 0, False, 0
        self.lock = threading.Lock()
    def _to(self, new):
        if new != self.state:
            log(f"breaker {self.state} → {new}")
            self.state = new
    def allow(self):
```

```
        with self.lock:
            if self.state == "open":
                if time.monotonic() - self.opened_at < self.open_seconds:
                    self.rejected += 1
                    return False
                self._to("half-open"); self.probe_ok = 0; self.probing = False
            if self.state == "half-open":
                if self.probing:                        # one probe at a time
                    self.rejected += 1
                    return False
                self.probing = True
            return True
    def record(self, ok):
        with self.lock:
            if self.state == "half-open":
                self.probing = False
                if not ok:
                    self.opened_at = time.monotonic(); self._to("open")
                else:
                    self.probe_ok += 1
                    if self.probe_ok >= self.probes_to_close:
                        self.results = []; self._to("closed")
                return
            self.results = (self.results + [ok])[-self.window:]
            fails = self.results.count(False)
            if len(self.results) >= self.min_calls and fails / len(self.results) >=
  self.ratio:
                self.opened_at = time.monotonic(); self._to("open")

BREAKER = Breaker()

# ---------- mTLS client to inventory (reloads the certificate when the file changes) ----
  ------
_ctx = {"mtime": None, "ctx": None}
def tls_context():
    cert = os.path.join(CERTS, "client.crt")
    mtime = os.stat(cert).st_mtime
    if _ctx["mtime"] != mtime:
        ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
        ctx.minimum_version = ssl.TLSVersion.TLSv1_3
        ctx.load_verify_locations(os.path.join(CERTS, "ca.crt"))          # trust only our
  CA
        ctx.load_cert_chain(cert, os.path.join(CERTS, "client.key"))      # our identity
        _ctx.update(mtime=mtime, ctx=ctx)
        log("loaded client certificate")
    return _ctx["ctx"]

def inventory_get(path):
    raw = socket.create_connection(INVENTORY, timeout=TIMEOUT)
    with tls_context().wrap_socket(raw, server_hostname=INVENTORY_NAME) as s:
        s.settimeout(TIMEOUT)
        s.sendall(f"GET {path} HTTP/1.1\r\nHost: {INVENTORY_NAME}\r\nConnection:
  close\r\n\r\n".encode())
```

```
        data = b""
        while chunk := s.recv(4096):
            data += chunk
    head, _, body = data.partition(b"\r\n\r\n")
    status = int(head.split(b" ", 2)[1])
    if status != 200:
        raise RuntimeError(f"inventory returned {status}")
    return body.decode().strip()

def backoff(attempt, base=0.05, cap=0.5):
    return random.uniform(0, min(cap, base * 2 ** attempt))         # full jitter

def get_stock():
    if not BULKHEAD.acquire(blocking=False):
        return {"stock": "unknown", "source": "fallback", "reason": "bulkhead full"}
    try:
        for attempt in range(MAX_TRIES):
            if not BREAKER.allow():
                return {"stock": "unknown", "source": "fallback", "reason": "circuit
  open"}
            try:
                body = inventory_get("/stock")
                BREAKER.record(True)
                return {"stock": body, "source": "inventory", "attempts": attempt + 1}
            except (OSError, ssl.SSLError, RuntimeError) as e:
                BREAKER.record(False)
                log(f"inventory call failed (attempt {attempt + 1}): {type(e).__name__}:
  {e}")
                if attempt + 1 < MAX_TRIES:
                    time.sleep(backoff(attempt))
        return {"stock": "unknown", "source": "fallback", "reason": "inventory failed"}
    finally:
        BULKHEAD.release()

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def log_message(self, *a): pass
    def _json(self, code, obj):
        data = (json.dumps(obj) + "\n").encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)
    def do_GET(self):
        if self.path == "/healthz":
            return self._json(200, {"ok": True})
        if self.path == "/whoami":
            return self._json(200, {"served_by_port": PORT, "scheme_seen_by_app": "http",
                                    "x_forwarded_proto": self.headers.get("X-Forwarded-
  Proto"),
                                    "x_forwarded_for": self.headers.get("X-Forwarded-
  For")})
```

```
        if self.path == "/breaker":
            return self._json(200, {"state": BREAKER.state, "rejected": BREAKER.rejected})
        if self.path == "/api/stock":
            r = get_stock(); r["breaker"] = BREAKER.state
            return self._json(200, r)
        self._json(404, {"error": "not found"})

if __name__ == "__main__":
    ThreadingHTTPServer(("127.0.0.1", PORT), H).serve_forever()
EOF
python3 -c "import ast,sys; ast.parse(open('app.py').read()); print('app.py parses')"
```

Milestone 3 — The edge: TLS termination, HTTP/2, load balancing, rate limiting (topics 11, 05, 06, 13, 10)

The edge terminates TLS 1.2/1.3, speaks HTTP/2 to clients, spreads requests over the two replicas with `least_conn`, keeps a pool of 16 idle keep-alive connections to them, passes the original scheme in `X-Forwarded-Proto`, and limits each client IP to 5 requests per second with a burst of 5 (excess gets `429` and `Retry-After`).

BASH

```
cat > edge.conf <<EOF
worker_processes 1;
daemon off;
error_log $LAB/logs/edge-error.log warn;
pid $LAB/edge.pid;
events { worker_connections 1024; }
http {
  log_format edge '\$remote_addr \$ssl_protocol \$ssl_cipher "\$request" \$status
  upstream=\$upstream_response_time';
  access_log $LAB/logs/edge-access.log edge;
  include $LAB/tmp.conf;

  limit_req_zone \$binary_remote_addr zone=per_ip:10m rate=5r/s;
  limit_req_status 429;

  upstream shop_api {
    least_conn;
    server 127.0.0.1:9001;
    server 127.0.0.1:9002;
    keepalive 16;
  }

  server {
    listen 127.0.0.1:8443 ssl http2;
    server_name shop.example.com;
    ssl_certificate     $LAB/certs/edge.crt;
    ssl_certificate_key $LAB/certs/edge.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_session_cache   shared:SSL:10m;
```

```
    error_page 429 @limited;
    location @limited {
      add_header Retry-After 1 always;
      return 429 "rate limited, retry after 1 second\n";
    }
    location / {
      limit_req zone=per_ip burst=5 nodelay;
      proxy_pass http://shop_api;
      proxy_http_version 1.1;
      proxy_set_header Connection "";
      proxy_set_header Host              \$host;
      proxy_set_header X-Forwarded-Proto \$scheme;
      proxy_set_header X-Forwarded-For   \$proxy_add_x_forwarded_for;
      proxy_connect_timeout 1s;
      proxy_read_timeout 5s;
    }
  }
}
EOF
nginx -t -c $LAB/edge.conf -e $LAB/logs/edge-error.log
```

Now start everything and check it with the helper scripts:

BASH

```
cat > up.sh <<'EOF'
#!/usr/bin/env bash
# Starts the four processes of the lab. Run from the lab directory.
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p logs tmp/edge tmp/inv
[ -f certs/client.crt ] || { cp certs/shop-api.crt certs/client.crt; cp certs/shop-api.key
  certs/client.key; }
python3 backend.py        > logs/backend.log 2>&1 & echo $! > backend.pid
python3 app.py 9001       > logs/app1.log    2>&1 & echo $! > app1.pid
python3 app.py 9002       > logs/app2.log    2>&1 & echo $! > app2.pid
nginx -c "$PWD/inventory.conf" -e "$PWD/logs/inv-error.log"  > /dev/null 2>&1 &
nginx -c "$PWD/edge.conf"      -e "$PWD/logs/edge-error.log" > /dev/null 2>&1 &
sleep 1
echo "up: edge https://shop.example.com:8443  →  shop-api :9001  →(mTLS) inventory :9443
  →  backend :9100"
EOF
cat > down.sh <<'EOF'
#!/usr/bin/env bash
cd "$(dirname "$0")"
for p in edge.pid inv.pid app1.pid app2.pid backend.pid; do [ -f "$p" ] && kill "$(cat
  "$p")" 2>/dev/null; done
echo down
EOF
chmod +x up.sh down.sh && ./up.sh
E="--cacert $LAB/certs/ca.crt --resolve shop.example.com:8443:127.0.0.1"
curl -sS $E https://shop.example.com:8443/api/stock
```

```
curl -sS $E https://shop.example.com:8443/whoami
```

```
up: edge https://shop.example.com:8443  →  shop-api :9001  →(mTLS) inventory :9443  →
  backend :9100
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "closed"}
{"served_by_port": 9002, "scheme_seen_by_app": "http", "x_forwarded_proto": "https",
  "x_forwarded_for": "127.0.0.1"}
```

The first line proves the whole chain works. The second shows the termination effect from TLS Termination §12.11: the app saw plain `http` and learned the truth only from `X-Forwarded-Proto`. (`served_by_port` may be 9001 or 9002.) The `--resolve` flag is our stand-in for DNS: it tells `curl` that the name `shop.example.com` lives at `127.0.0.1` while still sending the right SNI and `Host`.

Milestone 4 — Observe: curl, openssl, ss and tcpdump (topics 01, 05, 10, 11, 12)

4a. Client view. What TLS and HTTP version did the client get, and where did the time go?

```bash
curl -s $E -o /dev/null -w 'http=%{http_version} tcp=%{time_connect}s tls_done=%
  {time_appconnect}s total=%{time_total}s\n' https://shop.example.com:8443/api/stock
curl -s $E --http1.1 -o /dev/null -w 'forced http=%{http_version}\n'
  https://shop.example.com:8443/api/stock
openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -CAfile certs/ca.crt
  -alpn h2 </dev/null 2>&1 | grep -E 'Verification|Protocol|ALPN|Cipher is'
```

```
http=2 tcp=0.000149s tls_done=0.003006s total=0.063492s
forced http=1.1
Verification: OK
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
ALPN protocol: h2
```

(Timings differ on your machine; `total` is larger than `tls_done` because it includes the whole mTLS call to inventory.) TLS 1.3 and the `h2` ALPN result come from the edge; compare with HTTP/2, HTTP/3 and QUIC §12.5.

4b. Connection pooling. The edge keeps connections to the replicas open and reuses them:

```bash
for i in $(seq 1 6); do curl -s $E https://shop.example.com:8443/whoami | grep -o
  '"served_by_port": [0-9]*'; sleep 0.3; done | sort | uniq -c
ss -tn state established '( dport = :9001 or dport = :9002 )' | tail -n +2 | wc -l
```

```
      3 "served_by_port": 9001
      3 "served_by_port": 9002
2
```

Six requests were spread three and three by `least_conn`, yet only two TCP connections exist from the edge to the replicas, one per replica: the keep-alive pool at work (Connection Pooling §12.10). Without `keepalive 16` and `proxy_http_version 1.1` you would see a new connection per request.

4c. Packet captures: which hop is readable? This helper captures a set of ports on the loopback interface while sending one request, then counts what an eavesdropper could read. (The edge picks either replica, so the app hop is captured on both ports.)

```bash
peek() {   # peek <label> <tcpdump filter>
  sudo timeout 5 tcpdump -i lo -nn -A -l "$2" > cap-$1.txt 2>/dev/null &
  sleep 1; curl -s $E -o /dev/null https://shop.example.com:8443/api/stock; wait
  echo "$1: request lines=$(grep -ac 'GET /' cap-$1.txt)  'stock'=$(grep -ac stock
  cap-$1.txt)  X-Forwarded-Proto=$(grep -ac X-Forwarded-Proto cap-$1.txt)  shop-api
  identity=$(grep -acE 'spiffe|shop-api' cap-$1.txt)"
}
peek client-edge 'tcp port 8443'; peek edge-app 'tcp port 9001 or tcp port 9002'; peek
  app-inventory 'tcp port 9443'
```

```
client-edge: request lines=0  'stock'=0  X-Forwarded-Proto=0  shop-api identity=0
edge-app: request lines=1  'stock'=2  X-Forwarded-Proto=1  shop-api identity=0
app-inventory: request lines=0  'stock'=0  X-Forwarded-Proto=0  shop-api identity=0
```

Read it layer by layer, like OSI and TCP/IP §12.1: on **client-edge** (port 8443) and **app-inventory** (port 9443) the HTTP is encrypted, so nothing is readable. On **edge-app** (ports 9001 and 9002) the request line, header and JSON are plain text: this is the edge-termination trade-off from TLS Termination §12.11. Note the client identity is also invisible on the inventory hop, because the sidecar enforces TLS 1.3, where certificates are encrypted (mTLS §12.12). The SNI name is still visible in the ClientHello: run `sudo tcpdump -i lo -nn -A -c 20 'tcp port 9443'` in one terminal and send a request from another, and you will find `inventory.internal` in the output.

4d. mTLS from the outside. Call the sidecar directly with different identities:

```bash
I="--cacert $LAB/certs/ca.crt --resolve inventory.internal:9443:127.0.0.1"
curl -sS $I https://inventory.internal:9443/stock | grep -o '<title>.*</title>'
curl -sS $I --cert certs/billing.crt  --key certs/billing.key
  https://inventory.internal:9443/stock
curl -sS $I --cert certs/shop-api.crt --key certs/shop-api.key
  https://inventory.internal:9443/stock
curl -sS $I --tlsv1.2 --tls-max 1.2 --cert certs/shop-api.crt --key certs/shop-api.key
  https://inventory.internal:9443/stock; echo "exit=$?"
tail -4 logs/inv-access.log
```

```
<title>400 No required SSL certificate was sent</title>
caller not allowed
stock=42
curl: (35) OpenSSL/3.0.13: error:0A00042E:SSL routines::tlsv1 alert protocol version
exit=35
TLSv1.3 caller=[-] verify=NONE "GET /stock HTTP/1.1" 400
TLSv1.3 caller=[CN=billing] verify=SUCCESS "GET /stock HTTP/1.1" 403
TLSv1.3 caller=[CN=shop-api] verify=SUCCESS "GET /stock HTTP/1.1" 200
```

No certificate: rejected at the TLS layer (400). A valid certificate for the wrong identity: authenticated, but not authorised (403). The right identity: 200. A TLS 1.2 client: refused, because the sidecar is pinned to TLS 1.3.

Milestone 5 — Break it and fix it (topics 12, 13, 14, 06)

5a. Rate limiting. Send 20 requests as fast as possible:

```bash
sleep 3
for i in $(seq 1 20); do curl -s -o /dev/null -w '%{http_code} ' $E
  https://shop.example.com:8443/healthz; done; echo
curl -si $E https://shop.example.com:8443/healthz | sed -n '1p;/retry-after/Ip'
```

```
200 200 200 200 200 200 429 429 429 429 429 429 200 429 429 429 429 429 429 429
HTTP/2 429
retry-after: 1
```

The base rate plus a burst of five are served at once (the first six); after that nearly everything gets `429` with `Retry-After`, apart from an occasional `200` when a token has refilled (5 per second). Your exact pattern will vary with how fast your machine runs `curl`. The counters are per client IP at the edge, so the upstream services never see the flood.

5b. Circuit breaker drill. Make inventory hang (each call sleeps 2 s, longer than `shop-api`'s 0.5 s timeout) and watch `shop-api` degrade:

```bash
req() { curl -s $E -w ' → %{http_code} in %{time_total}s\n'
  https://shop.example.com:8443/api/stock | paste -sd' '; }
sleep 3; req; req
curl -s -X POST http://127.0.0.1:9100/mode/slow; echo
for i in 1 2 3 4 5 6; do req; sleep 0.3; done
tail -n 6 logs/app1.log
```

```
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "closed"} → 200 in
  0.011235s
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "closed"} → 200 in
  0.008216s
mode=slow
{"stock": "unknown", "source": "fallback", "reason": "inventory failed", "breaker":
  "closed"} → 200 in 1.069847s
{"stock": "unknown", "source": "fallback", "reason": "inventory failed", "breaker":
  "closed"} → 200 in 1.053009s
{"stock": "unknown", "source": "fallback", "reason": "inventory failed", "breaker":
  "closed"} → 200 in 1.055133s
{"stock": "unknown", "source": "fallback", "reason": "circuit open", "breaker": "open"} →
  200 in 0.541513s
{"stock": "unknown", "source": "fallback", "reason": "circuit open", "breaker": "open"} →
  200 in 0.520906s
{"stock": "unknown", "source": "fallback", "reason": "circuit open", "breaker": "open"} →
  200 in 0.005615s
HH:MM:SS inventory call failed (attempt 1): TimeoutError: The read operation timed out
HH:MM:SS breaker closed → open
```

The first slow requests each pay two 0.5 s timeouts (about 1 s) and fall back. As failures accumulate in a replica's window the breaker trips mid-request (the 0.54 s lines: one timeout, then the retry is refused by the now-open breaker), and from then on that replica answers in about 5 ms with the fallback, without touching inventory. Note the status stays `200`: the page degrades instead of failing. Each replica runs its **own** breaker (they are separate processes), so how quickly the whole fleet stops calling depends on traffic reaching each one; that is the argument for per-endpoint outlier detection in a proxy or mesh. Now recover:

```bash
curl -s -X POST http://127.0.0.1:9100/mode/ok; echo
sleep 6
for i in 1 2 3 4 5 6; do req; sleep 0.3; done
grep breaker logs/app1.log | tail -n 4
```

```
mode=ok
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "half-open"} → 200
  in 0.009436s
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "half-open"} → 200
  in 0.010496s
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "closed"} → 200 in
  0.009242s
...
HH:MM:SS breaker closed → open
HH:MM:SS breaker open → half-open
HH:MM:SS breaker half-open → closed
```

After the cool-down each replica's breaker lets single probes through (half-open; the two `half-open` lines are the two replicas), two successes close it, and normal service resumes with no human action.

5c. Rotation and expiry drill. Replace the client certificate while `shop-api` is running. Write the key first and the certificate last, atomically, because `shop-api` watches the certificate file:

```bash
cd certs
openssl req -newkey rsa:2048 -nodes -keyout new.key -out new.csr -subj "/CN=shop-api"
  2>/dev/null
printf "subjectAltName=URI:spiffe://example.com/ns/prod/sa/shop-
  api\nextendedKeyUsage=clientAuth\n" > new.ext
openssl x509 -req -in new.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days 7 -extfile
  new.ext -out new.crt 2>/dev/null
echo "old: $(openssl x509 -in client.crt -noout -serial)"
cp new.key client.key.tmp && mv client.key.tmp client.key
cp new.crt client.crt.tmp && mv client.crt.tmp client.crt
echo "new: $(openssl x509 -in client.crt -noout -serial)"
cd ..; req; grep 'loaded client' logs/app1.log logs/app2.log | tail -n 2
```

```
old: serial=6FFF...1A
new: serial=DD60...CE
{"stock": "stock=42", "source": "inventory", "attempts": 1, "breaker": "closed"} → 200 in
  0.0120s
logs/app1.log:HH:MM:SS loaded client certificate
```

(The serials and which log file shows the reload differ; each replica reloads on its first call after the file changes.) No restart, no dropped request. Now simulate the outage that automation prevents, an expired certificate:

```bash
cd certs
openssl x509 -req -in new.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days 0 -extfile
  new.ext -out client.crt.tmp 2>/dev/null && mv client.crt.tmp client.crt
cd ..; sleep 2; req
grep -a 'verify error' logs/inv-error.log | tail -n 1 | sed 's/^.*\[info\] //' | cut -c1-
  100
```

```
{"stock": "unknown", "source": "fallback", "reason": "inventory failed", "breaker":
  "closed"} → 200 in 0.0474s
... client SSL certificate verify error: (10:certificate has expired) while reading client
  request headers
```

`shop-api` keeps serving through its fallback while the cause is only visible in the inventory sidecar's log, which is exactly why you alert on certificate expiry and read server-side logs. Restore a good identity before moving on:

```bash
cp certs/shop-api.crt certs/client.crt; cp certs/shop-api.key certs/client.key
```

5d. Load balancer failover. Kill one replica and keep calling the edge:

```bash
kill $(cat app2.pid); sleep 0.5
for i in $(seq 1 8); do curl -s $E https://shop.example.com:8443/whoami | grep -o
  '"served_by_port": [0-9]*'; sleep 0.3; done | sort | uniq -c
python3 app.py 9002 > logs/app2.log 2>&1 & echo $! > app2.pid
```

```
      8 "served_by_port": 9001
```

All eight requests succeed from the surviving replica: nginx marked the refused connection as a failure and retried on the other server (`grep 'Connection refused' logs/edge-error.log`). The replacement replica is picked up again after nginx's default 10-second `fail_timeout`.

Milestone 6 — Automate the proof (topics 11, 12, 13, 14)

Collect every property above into one script that reports `PASS` or `FAIL`; it is the lab's acceptance test and a starting point for a CI check.

```bash
cat > verify.sh <<'EOF'
#!/usr/bin/env bash
# Runs the lab's acceptance checks. Needs the lab running (./up.sh) and a healthy backend.
cd "$(dirname "$0")"
E="--cacert certs/ca.crt --resolve shop.example.com:8443:127.0.0.1"
I="--cacert certs/ca.crt --resolve inventory.internal:9443:127.0.0.1"
pass() { printf 'PASS  %s\n' "$1"; }
fail() { printf 'FAIL  %s\n' "$1"; }
check() { if eval "$2"; then pass "$1"; else fail "$1"; fi; }

curl -s -X POST http://127.0.0.1:9100/mode/ok >/dev/null; sleep 6          # let limiter
  and breaker settle
check "TLS terminates at the edge (HTTP/2, TLS 1.3)" \
  '[ "$(curl -s $E -o /dev/null -w "%{http_version} %{ssl_verify_result}"
  https://shop.example.com:8443/healthz)" = "2 0" ]'
check "plain HTTP to the TLS port is refused" \
  '[ "$(curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8443/)" = "400" ]'
check "app sees X-Forwarded-Proto https" \
  'curl -s $E https://shop.example.com:8443/whoami | grep -q "\"x_forwarded_proto\":
  \"https\""'
check "inventory rejects a caller with no client certificate" \
  '[ "$(curl -s $I -o /dev/null -w "%{http_code}" https://inventory.internal:9443/stock)"
  = "400" ]'
check "inventory rejects a valid but unauthorised identity (billing)" \
  '[ "$(curl -s $I --cert certs/billing.crt --key certs/billing.key -o /dev/null -w "%
  {http_code}" https://inventory.internal:9443/stock)" = "403" ]'
check "shop-api reaches inventory over mTLS" \
  'curl -s $E https://shop.example.com:8443/api/stock | grep -q "\"source\":
  \"inventory\""'
sleep 3
codes=$(for i in $(seq 1 15); do curl -s $E -o /dev/null -w "%{http_code} "
  https://shop.example.com:8443/healthz; done)
check "rate limit returns 429 after the burst" 'echo "$codes" | grep -q 429'
sleep 3
curl -s -X POST http://127.0.0.1:9100/mode/slow >/dev/null
for i in 1 2 3 4 5 6; do curl -s $E -o /dev/null https://shop.example.com:8443/api/stock;
  sleep 0.3; done
check "circuit breaker opens in both replicas when inventory is slow" \
  'curl -s http://127.0.0.1:9001/breaker | grep -q "\"state\": \"open\"" && curl -s
  http://127.0.0.1:9002/breaker | grep -q "\"state\": \"open\""'
out=$(curl -s $E -w " %{time_total}" https://shop.example.com:8443/api/stock)
check "open breaker answers with a fast fallback" 'echo "$out" | grep -q "circuit open" &&
  echo "$out" | awk "{ exit !(\$NF < 0.2) }"'
curl -s -X POST http://127.0.0.1:9100/mode/ok >/dev/null; sleep 6
for i in 1 2 3 4 5 6; do curl -s $E -o /dev/null https://shop.example.com:8443/api/stock;
  sleep 0.3; done
```

```
check "breakers close again after recovery" \
  'curl -s http://127.0.0.1:9001/breaker | grep -q "\"state\": \"closed\"" && curl -s
  http://127.0.0.1:9002/breaker | grep -q "\"state\": \"closed\""'
EOF
chmod +x verify.sh && ./verify.sh
```

```
PASS  TLS terminates at the edge (HTTP/2, TLS 1.3)
PASS  plain HTTP to the TLS port is refused
PASS  app sees X-Forwarded-Proto https
PASS  inventory rejects a caller with no client certificate
PASS  inventory rejects a valid but unauthorised identity (billing)
PASS  shop-api reaches inventory over mTLS
PASS  rate limit returns 429 after the burst
PASS  circuit breaker opens in both replicas when inventory is slow
PASS  open breaker answers with a fast fallback
PASS  breakers close again after recovery
```

The script takes about 40 seconds because it waits for the limiter and breaker to settle. If a check fails, run the corresponding command by hand from Milestones 3-5 and read the logs in `$LAB/logs/`.

5 Tasks

- [ ] **T1 — Stand up the secure path** *(≈90 min, intermediate; Milestones 0-3; topics 04, 11, 12)*
  - **Goal:** a working edge, two replicas and an mTLS-protected inventory.
  - **Steps:**
    1. Create the certificates, configs and scripts and run `./up.sh`.
    2. Fetch `/api/stock` and `/whoami` through the edge, then run `openssl x509 -text` on the edge and client certificates and note each one's issuer, SAN, key usage and validity.
  - **Done when:** `/api/stock` returns `"source": "inventory"`, `/whoami` shows `x_forwarded_proto: https`, and you can say which certificate is presented on which hop and who verifies it.
- [ ] **T2 — Prove which hop is encrypted** *(≈60 min, intermediate; Milestone 4; topics 01, 05, 10, 11)*
  - **Goal:** back the architecture diagram with captures and counters.
  - **Steps:**
    1. Run `peek` for the three hops (client-edge, edge-app, app-inventory) and capture the output.

2. Add `proxy_set_header Authorization "Bearer lab-token";` to the edge, reload (`nginx -s reload -c $LAB/edge.conf -e $LAB/logs/edge-error.log`) and find the token in the capture of the edge-app hop only.
3. Fill a table of OSI layers (L3, L4, L5/L6, L7) against what you saw (IP and port via `ss`, TLS version and ALPN via `openssl s_client`, HTTP version via `curl -w`, headers via `tcpdump`).

- **Done when:** the token appears in cleartext on the edge-app hop and nowhere on 8443 or 9443, and your layer table has a command and an observed value for each row.

- [ ] **T3 — Authenticate, authorise, rotate** *(≈75 min, advanced; Milestones 1, 5c; topics 04, 12)*
  - **Goal:** control and break identity on purpose.
  - **Steps:**
    1. Reproduce the four mTLS results in 4d (no certificate, billing, shop-api, TLS 1.2).
    2. Issue a client certificate from a second, rogue CA with the same CN and show the exact `error.log` message.
    3. Do the live rotation and the expiry drill, then change `shop-api` to reload the certificate every request without a file watcher and explain the trade-off.
  - **Done when:** you can show the 400/403/200 matrix, the rogue-CA log line (`unable to verify the first certificate` or similar), and a rotation with zero failed requests (loop `curl` while rotating).

- [ ] **T4 — Tune the limiter and the balancer** *(≈75 min, advanced; Milestones 3, 5a, 5d; topics 06, 10, 13)*
  - **Goal:** understand burst, rate and algorithm choices.
  - **Steps:**
    1. Change the limiter to `burst=10 nodelay`, then to `burst=10` (delaying), run 20 rapid requests each time and compare the status codes and total time.
    2. Add a third replica on port 9003 and compare `least_conn`, round robin (remove the directive) and `ip_hash`, using `/whoami`.
    3. Make one replica slow (add `time.sleep(1)` for `/whoami` on port 9002 only) and see which algorithm handles it best under 20 concurrent requests (`xargs -P 20`).
  - **Done when:** you have a small table for limiter variants and a second one for balancing algorithms, each with a one-sentence explanation of the difference.

- [ ] **T5 — Run the resilience drills and add a missing control** *(≈90 min, advanced; Milestones 2, 5b; topics 10, 14)*
  - **Goal:** go beyond the provided breaker.

- **Steps:**
  1. Run 5b with `error` mode instead of `slow` and compare time-to-open and request latency.
  2. Make `shop-api` retry only idempotent calls and add a global retry budget (retries at most 10 percent of calls in the last 10 seconds); count backend hits via `GET http://127.0.0.1:9100/stats`.
  3. Replace the connect-per-call in `inventory_get` with a small pool of keep-alive TLS connections and measure the latency change with `curl -w`.
- **Done when:** `/stats` shows at most about 1.1x the attempted requests in `error` mode, and a before/after latency comparison shows the effect of connection reuse.

☐ **T6 — Map the lab onto the real internet** *(≈60 min, advanced; all topics, especially 02, 03, 07, 08, 09)*

- **Goal:** connect the toy to production practice.
- **Steps:**
  1. Write a one-page design (diagram plus bullets) for serving `shop.example.com` globally: DNS records and TTLs (DNS §12.3), anycast announcements (BGP §12.2), a CDN in front with its own TLS termination and origin pull (CDNs §12.7), a private link or VPN to the inventory network (VPNs §12.8), and a mesh replacing the hand-built sidecar (Service Mesh §12.9).
  2. For each box, say where TLS terminates or re-originates, what identity authenticates the hop, and what would break first if its certificate expired.
- **Done when:** the design names every Level 12 topic at least once, labels each hop plain/TLS/mTLS, and states one failure mode and one control per hop.

☐ **T7 — Prove it: rebuild and defend** *(≈2 hours, advanced; all milestones)*

- **Goal:** demonstrate mastery without the guide.
- **Steps:**
  1. From an empty directory, rebuild the stack (own CA, edge termination, mTLS inventory, limiter, breaker) using only your notes and the man pages.
  2. Run your own acceptance checks and a 15-minute walkthrough: capture proof for each hop, break and fix a certificate, trip and recover the breaker, trigger a `429`.
  3. Explain aloud why you chose re-encryption or plaintext for the edge-to-app hop, what the breaker's thresholds protect, and what happens to each control when its dependency (certificate, Redis, DNS) fails.
- **Done when:** `verify.sh` (or your own equivalent) passes from a clean state and you can answer follow-up questions on every topic in the table in section 2.

6 Verification checklist

- [ ] `./verify.sh` prints ten `PASS` lines and no `FAIL`.
- [ ] `openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -CAfile certs/ca.crt` shows `Verification: OK`, TLS 1.3 and (with `-alpn h2`) `ALPN protocol: h2`.
- [ ] `curl http://127.0.0.1:8443/` (plain HTTP to the TLS port) returns `400`, not content.
- [ ] A capture of ports 9001/9002 shows `X-Forwarded-Proto` in clear text; captures of 8443 and 9443 contain no request lines.
- [ ] The inventory sidecar log shows `verify=SUCCESS` for `shop-api`, `403` for `billing`, `verify=NONE` or an error for the rest.
- [ ] 20 rapid requests produce `200` for the first six and `429` with `Retry-After` afterwards.
- [ ] With inventory in `slow` mode, `/api/stock` returns the fallback in under 10 ms once the breaker is open, and returns `"source": "inventory"` again within about 10 seconds of switching back to `ok`.
- [ ] Rotating `certs/client.crt` triggers `loaded client certificate` in the app log with no failed request, and an expired certificate is reported in `logs/inv-error.log` as `certificate has expired`.
- [ ] Killing a replica does not produce client errors.

7 Teardown

```bash
cd $LAB && ./down.sh
# make sure nothing is left listening
ss -ltn | grep -E ':(8443|9001|9002|9443|9100)\b' || echo "all lab ports free"
cd ~ && rm -r "$LAB"; unset LAB E I
```

Nothing outside `$LAB` was modified, so removing the directory removes the CA, keys and logs. If you disabled the system nginx service earlier, re-enable it with `sudo systemctl enable --now nginx` if you want it back.

8 Stretch goals and what to learn next

- **Swap nginx for Envoy.** Re-create the edge and the sidecar with Envoy (`DownstreamTlsContext` on the listener, `UpstreamTlsContext` with a client certificate on the cluster, `circuit_breakers` plus `outlier_detection` and a `local_ratelimit` filter, all

shown in topics 11, 13 and 14) and compare the admin stats and the failure behaviour.

- **Automatic identity.** Replace `make-certs.sh` with a SPIRE server and agent (or cert-manager with a private issuer) so client certificates live for an hour and rotate on their own (mTLS §12.12).
- **Global rate limits.** Move the limiter state into Redis with the atomic Lua token bucket from Rate Limiting §12.13, shared by both edge instances, and decide fail-open versus fail-closed.
- **HTTP/3.** Put a QUIC-capable front end (nginx 1.25+ built with QUIC, Caddy or Envoy) in front, advertise it with `Alt-Svc`, and compare handshakes and head-of-line blocking against HTTP/2 (HTTP/2, HTTP/3 and QUIC §12.5).
- **Containerise it** with Docker Compose (one service per process, an internal network for the 9100/9443 hops, the edge as the only published port), then deploy to Kubernetes with an Ingress and a mesh.
- **Observability.** Export breaker state, `429` counts and handshake rates as Prometheus metrics and alert on an open breaker, a certificate with fewer than 14 days left, and a rising 429 rate (Observability §0.10).
- **Next:** Level 13: Databases and Distributed Systems §13.1 takes the resilience ideas deeper into state: replication, connection pooling for databases, idempotency and consistency, and later Level 16: Advanced Kubernetes §16.9 automates this lab's sidecars, mTLS and breakers with Istio or Linkerd.

LEVEL 13 · PART VI

Databases & Distributed Systems

13

Goal: Reason about data stores, messaging and distributed-systems trade-offs when operating production systems.

RECOMMENDED TIME	TOPICS
4-6 weeks	15

PRACTICE / CAPSTONE

Run Postgres primary+replica, Redis and Kafka (or RabbitMQ) locally; demonstrate replication lag, an idempotent consumer and a PITR restore.

Topics

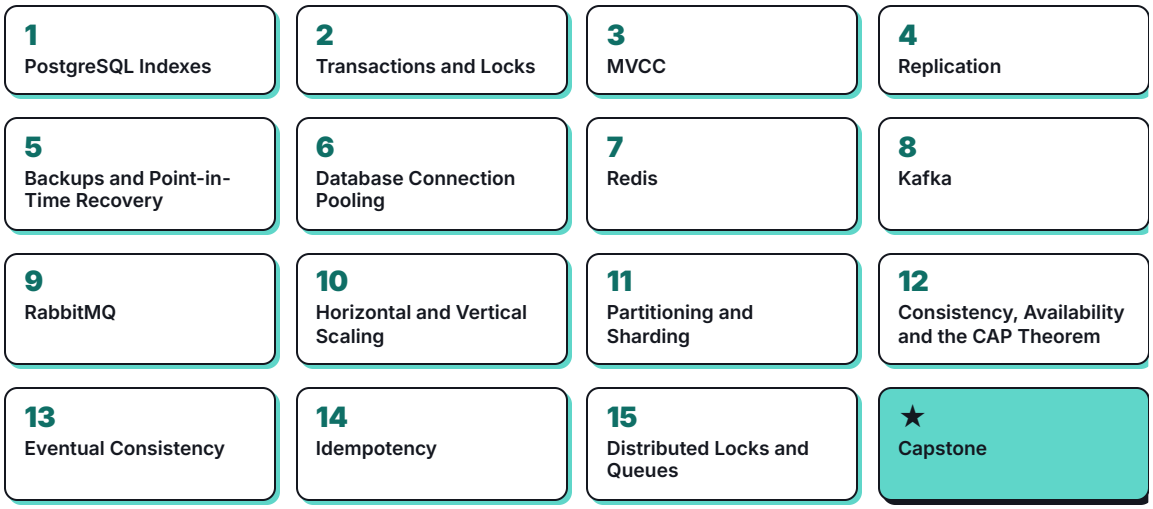
#	Topic	What you will learn
1	PostgreSQL Indexes **p.2579**	B-tree/GIN/partial, EXPLAIN ANALYZE, index cost
2	Transactions and Locks **p.2593**	ACID, isolation levels, row/table locks, deadlocks
3	MVCC **p.2607**	snapshots, xmin/xmax, vacuum, bloat, long-running transaction danger
4	Replication **p.2623**	streaming replication, sync vs async, replicas, lag, failover tools
5	Backups and Point-in-Time Recovery **p.2639**	pg_dump vs base backup + WAL, PITR, pgBackRest/WAL-G, restore drills
6	Database Connection Pooling **p.2655**	PgBouncer modes, max_connections, pool sizing math
7	Redis **p.2669**	data structures, caching patterns, persistence RDB/AOF, eviction, Sentinel/Cluster
8	Kafka **p.2685**	topics, partitions, consumer groups, offsets, retention, ordering
9	RabbitMQ **p.2701**	exchanges, queues, acks, DLQ, vs Kafka
10	Horizontal and Vertical Scaling **p.2715**	when each, stateless vs stateful, read replicas

#	Topic	What you will learn
11	Partitioning and Sharding **p.2727**	table partitioning, shard keys, hotspots, resharding
12	Consistency, Availability and the CAP Theorem **p.2739**	CAP/PACELC, linearizability, real systems classified
13	Eventual Consistency **p.2751**	convergence, read-your-writes, conflict resolution, CRDTs intro
14	Idempotency **p.2763**	idempotency keys, at-least-once delivery, dedup, retries
15	Distributed Locks and Queues **p.2777**	Redis locks, fencing tokens, leases, work queues, outbox pattern
★	Level capstone project **p.2793**	Run Postgres primary+replica, Redis and Kafka (or RabbitMQ) locally; demonstrate replication lag, an idempotent consumer and a PITR restore.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



13.1 PostgreSQL Indexes

Topic 1 of 15 | ~4 hours | Difficulty: Intermediate

1 What it is

An **index** is a separate, sorted (or otherwise organised) data structure that lets PostgreSQL find rows without reading the whole table. Without an index, a query like `WHERE user_id = 4242` forces a **sequential scan** ("seq scan"): read every page of the table and test every row. With a suitable index, PostgreSQL walks a small tree, gets a list of row locations (**TIDs**, tuple identifiers: page number + slot) and fetches only those rows from the table (the "heap").

PostgreSQL ships several index types, each suited to different questions. The default and by far the most common is the **B-tree** (balanced tree), good for equality (`=`), ranges (`<` , `BETWEEN`) and `ORDER BY` . **GIN** (Generalized Inverted Index) maps "values inside a column" (array elements, JSONB keys, full-text lexemes) to the rows containing them. **GiST**, **SP-GiST**, **BRIN** (Block Range INdex) and **Hash** cover geometric data, very large naturally-ordered tables and pure-equality lookups. A **partial index** indexes only the rows matching a `WHERE` predicate, and an **expression index** indexes the result of a function, e.g. `lower(email)` .

The tool for finding out *whether PostgreSQL uses an index and how well* is `EXPLAIN` (shows the plan) and `EXPLAIN ANALYZE` (actually runs the query and shows real timings and row counts).

2 Why it matters in DevOps

- **Most "the database is slow" pages are a missing or unused index.** A query that took 2 ms with 10 000 rows takes 20 s with 10 million rows, and CPU on the primary pins at 100% while the API times out.
- **Indexes are not free.** Every `INSERT` , `UPDATE` and `DELETE` must maintain every index on the table, indexes consume disk and RAM (they compete with table data for `shared_buffers` and the OS page cache), and they slow down replication replay and backups. Over-indexing is as real a problem as under-indexing.
- **Schema changes are operational events.** `CREATE INDEX` takes a lock that blocks writes unless you use `CREATE INDEX CONCURRENTLY` ; running the wrong form in a deploy pipeline (see Database Migrations in Pipelines §5.9) causes an outage.

- **You will be asked to read a plan.** DevOps/SRE engineers are routinely handed a slow query from `pg_stat_statements` or a slow-log and are expected to say "seq scan on a 40 GB table, add a composite index" or "the planner is right, the index is not selective".

3 Real-world system example

PostgreSQL itself, in production at scale. Many well-known products run on PostgreSQL, and their engineering blogs repeatedly describe indexing as day-to-day operational work. GitLab publishes its database guidelines publicly: they require that migrations adding indexes to large tables use concurrent index creation, and their developer docs tell engineers to include `EXPLAIN (ANALYZE, BUFFERS)` output for new queries in merge requests. Instagram's engineering team has written about using **partial indexes** and **functional indexes** in PostgreSQL to keep indexes small for hot, selective queries. Cloud-managed offerings such as Amazon RDS for PostgreSQL and Aurora PostgreSQL expose `pg_stat_statements` and Performance Insights precisely so operators can find the queries that need an index.

The pattern is the same everywhere: identify the top queries by total time (`pg_stat_statements`), read their plans, add the smallest index that turns a seq scan into an index scan, verify with `EXPLAIN ANALYZE`, and ship it with `CONCURRENTLY`.

4 Diagram

How a lookup differs with and without an index:

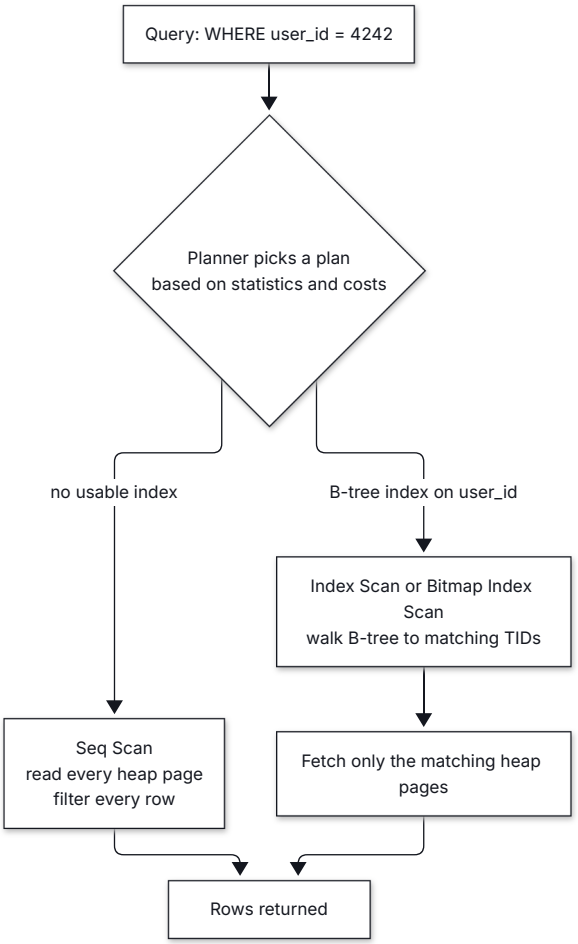


Figure 13.1-1

Inside a B-tree (simplified): each lookup touches about 3-4 pages even for tens of millions of rows.

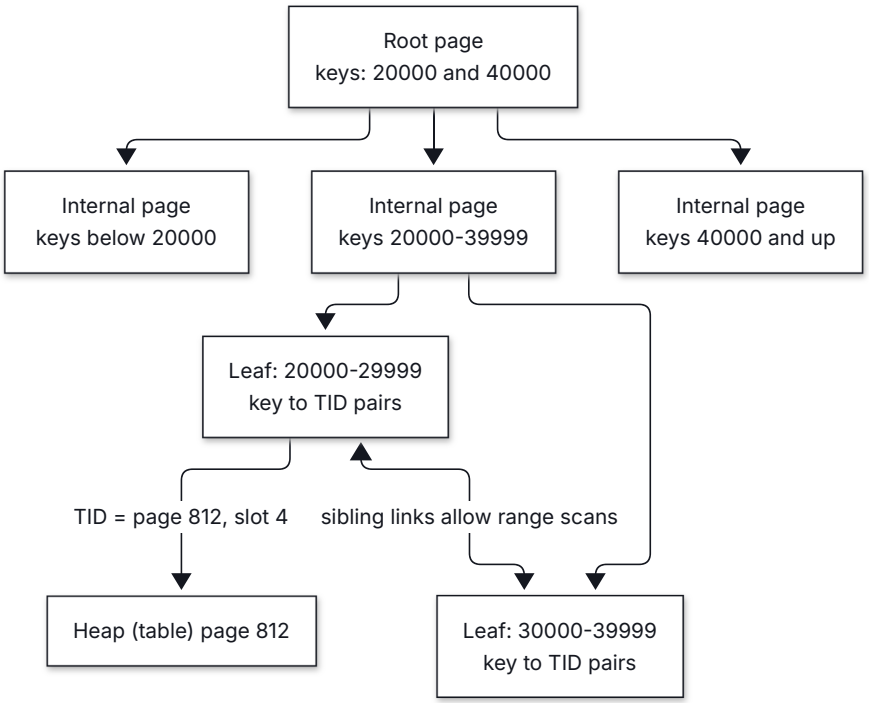


Figure 13.1-2

5 How it works

Index types at a glance

Type	Good for	Typical use	Not good for
B-tree (default)	`=`, `<`, `>`, `BETWEEN`, `ORDER BY`, `LIKE 'abc%'` (with C collation or `text_pattern_ops`)	primary keys, foreign keys, timestamps, most columns	`LIKE '%abc'`, containment inside JSON/arrays
GIN	"contains" queries: `@>`, `?`, `&&`, full-text `@@`	JSONB, arrays, `tsvector`, `pg_trgm` for `LIKE '%abc%'`	frequently-updated columns (slower writes), range scans
GiST / SP-GiST	geometry, ranges, nearest-neighbour, exclusion constraints	PostGIS, `tstzrange` overlap	plain scalar equality
BRIN	huge, naturally ordered tables (append-only logs, time series)	`created_at` on a 2 TB event table, index of a few MB	randomly ordered data

Type	Good for	Typical use	Not good for
Hash	equality only	rarely better than B-tree	everything else

Variants that matter in operations

- **Composite (multi-column) index** `(a, b)`: usable for filters on `a`, or `a` and `b`, but **not efficiently for** `b` **alone**. Column order is a design decision: equality columns first, then range/sort columns.
- **Partial index** `... WHERE status = 'pending'`: tiny, fast, and only used when the query's `WHERE` implies the predicate.
- **Expression index** `ON users (lower(email))`: required if the query says `WHERE lower(email) = ...`. Indexing `email` does not help.
- **Covering index** `ON orders (user_id) INCLUDE (total_cents)`: stores extra payload columns so PostgreSQL can answer from the index alone (an **index-only scan**) without visiting the heap. Index-only scans depend on the **visibility map**, kept current by `VACUUM` (see MVCC §13.3).
- **Unique index**: enforces a constraint and is the engine behind `PRIMARY KEY` and `UNIQUE`.

Reading `EXPLAIN (ANALYZE, BUFFERS)`

Plan node	Meaning
`Seq Scan`	reads the whole table. Fine for small tables or queries returning a large share of rows.
`Index Scan`	walks the index and fetches heap rows one by one in index order (good for `ORDER BY ... LIMIT`).
`Bitmap Index Scan` + `Bitmap Heap Scan`	builds a bitmap of matching pages first, then reads them in physical order. Used for medium selectivity and for combining indexes.
`Index Only Scan`	answers from the index; look for `Heap Fetches: 0`.
`cost=0.42..8.44`	planner's estimated startup cost..total cost in arbitrary units.
`rows=21` vs `actual ... rows=17`	estimated vs actual rows. **Large mismatches mean stale or insufficient statistics** (`ANALYZE`).
`Rows Removed by Filter: N`	work thrown away. A big N next to a seq scan is the classic "needs an index" signal.
`Buffers: shared hit=… read=…`	pages found in cache (`hit`) vs read from the OS/disk (`read`).
`Execution Time`	real time. Only `EXPLAIN ANALYZE` has it.

Warning: `EXPLAIN ANALYZE` really executes the statement. For `INSERT/UPDATE/DELETE` wrap it in a transaction and roll back: `BEGIN; EXPLAIN ANALYZE UPDATE ...; ROLLBACK;`.

Why the planner sometimes ignores your index

1. The query returns a large fraction of the table; a seq scan is genuinely cheaper.
2. The predicate wraps the column in a function or cast (`WHERE date_trunc('day', created_at) = ...`) without a matching expression index.
3. A type mismatch forces an implicit cast on the column side.
4. Statistics are stale, so the planner guesses wrong. Run `ANALYZE`.
5. The table is tiny. PostgreSQL correctly prefers a seq scan.

The cost side

- **Writes:** each index adds work to every row insert and to every `UPDATE` that touches an indexed column (or cannot use a **HOT**, heap-only tuple, update). GIN indexes are especially expensive to maintain.
- **Space:** an index is often 10-30% of the table size for a single narrow column and can exceed the table for wide composite indexes.
- **Maintenance:** indexes bloat after heavy update/delete churn and sometimes need `REINDEX CONCURRENTLY`.

6 Step-by-step guide

Prerequisites: Docker, or a local PostgreSQL 16+ with `psql`. Cost: free. Time: ~30 minutes. The sample data is about 1M rows (~100 MB).

1. **Start PostgreSQL in a throwaway container.**

```bash
docker run -d --name pg-idx -e POSTGRES_PASSWORD=<YOUR_PASSWORD> -p 5432:5432
  postgres:16
docker exec -it pg-idx psql -U postgres
```

   You should land at a `postgres=#` prompt. Then `CREATE DATABASE shop; \c shop`.

2. **Create the `shop-api` orders table and load 1 million rows.** About 97% of orders are `shipped`, 2% `pending`, 1% `cancelled`; 0.1% carry a campaign tag in JSONB.

```sql
CREATE TABLE orders (
  id          bigserial PRIMARY KEY,
  user_id     int         NOT NULL,
```

```
  status      text        NOT NULL,
  total_cents int         NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  metadata    jsonb       NOT NULL DEFAULT '{}'
);

INSERT INTO orders (user_id, status, total_cents, created_at, metadata)
SELECT (random() * 50000)::int,
       CASE WHEN r < 0.97 THEN 'shipped' WHEN r < 0.99 THEN 'pending' ELSE 'cancelled'
  END,
       (random() * 20000)::int,
       now() - random() * interval '365 days',
       CASE WHEN random() < 0.001
            THEN jsonb_build_object('channel', 'web', 'campaign', 'black-friday')
            ELSE jsonb_build_object('channel', (ARRAY['web','ios','android'])[1 +
  (random()*2)::int]) END
FROM (SELECT random() AS r FROM generate_series(1, 1000000)) s;

ANALYZE orders;
```

Expect `INSERT 0 1000000`. The `PRIMARY KEY` already created a B-tree index called `orders_pkey`.

3. **Run a slow query and read the plan.**

```sql
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE user_id = 4242;
```

Similar to (timings vary with hardware):

```
 Gather  (cost=1000.00..16808.43 rows=21 width=56) (actual time=4.268..31.186 rows=17
  loops=1)
   Workers Planned: 2
   →  Parallel Seq Scan on orders  (cost=0.00..15806.33 rows=9 width=56) (actual
  time=4.045..23.528 rows=6 loops=3)
         Filter: (user_id = 4242)
         Rows Removed by Filter: 333328
         Buffers: shared hit=10598
 Execution Time: 31.212 ms
```

About 1 million rows were read to return 17. That is the "needs an index" signature.

4. **Add a B-tree index and compare.**

```sql
CREATE INDEX idx_orders_user_id ON orders (user_id);
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE user_id = 4242;
```

Similar to:

```
Bitmap Heap Scan on orders  (cost=4.59..86.05 rows=21 width=56) (actual
 time=0.031..0.063 rows=17 loops=1)
  Recheck Cond: (user_id = 4242)
  →  Bitmap Index Scan on idx_orders_user_id  (cost=0.00..4.58 rows=21 width=0)
 (actual ... rows=17 loops=1)
        Index Cond: (user_id = 4242)
Execution Time: 0.080 ms
```

Roughly 400x faster here, and a few buffers touched instead of 10 000.

5. **Make it ordered: a composite index for "latest orders for a user".**

```sql
CREATE INDEX idx_orders_user_created ON orders (user_id, created_at DESC);
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 4242 ORDER BY created_at DESC
  LIMIT 10;
```

Expect an `Index Scan using idx_orders_user_created` with **no** `Sort` **node**: the index already delivers rows in order. You can now drop the single-column index because the composite one serves `WHERE user_id = ...` too: `DROP INDEX idx_orders_user_id;`

6. **Use a GIN index for JSONB containment.**

```sql
EXPLAIN ANALYZE SELECT id FROM orders WHERE metadata @> '{"campaign":"black-friday"}';
  -- parallel seq scan, ~100 ms
CREATE INDEX idx_orders_metadata ON orders USING gin (metadata jsonb_path_ops);
EXPLAIN ANALYZE SELECT id FROM orders WHERE metadata @> '{"campaign":"black-friday"}';
```

Expect a `Bitmap Index Scan on idx_orders_metadata` and an execution time of a couple of milliseconds. `jsonb_path_ops` makes a smaller, faster index than the default operator class but supports only `@>` (and `@?` / `@@` JSON-path operators), not `?` key-exists. GIN pays off for *selective* predicates; `@> '{"channel":"web"}'` matches a third of the table and will not benefit.

7. **Use a partial index for a hot, small subset.**

```sql
CREATE INDEX idx_orders_pending ON orders (created_at) WHERE status = 'pending';
EXPLAIN ANALYZE SELECT id FROM orders WHERE status = 'pending' ORDER BY created_at
  LIMIT 20;
SELECT pg_size_pretty(pg_relation_size('idx_orders_pending')) AS partial_size,
       pg_size_pretty(pg_relation_size('orders_pkey'))          AS pkey_size;
```

The plan is an `Index Scan using idx_orders_pending` finishing in well under a millisecond, and the index is roughly 450 kB against ~21 MB for the primary key. The same query with `status = 'shipped'` cannot use it.

8. **Get an index-only scan with a covering index.**

```sql
CREATE INDEX idx_orders_user_cov ON orders (user_id) INCLUDE (total_cents);
VACUUM (ANALYZE) orders;      -- sets visibility-map bits
EXPLAIN (ANALYZE) SELECT user_id, total_cents FROM orders WHERE user_id = 4242;
```

   Look for `Index Only Scan ... Heap Fetches: 0`.

9. **Build indexes safely on a live table.**

```sql
CREATE INDEX CONCURRENTLY idx_orders_status_created ON orders (status, created_at);
```

   `CONCURRENTLY` does not block writes but takes longer, cannot run inside a transaction block, and if it fails leaves an `INVALID` index behind. Find those with:

```sql
SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;
```

   and remove with `DROP INDEX CONCURRENTLY <name>;` before retrying.

10. **Audit index usage and size.**

```sql
SELECT s.indexrelid::regclass AS idx,
       pg_size_pretty(pg_relation_size(s.indexrelid)) AS size,
       s.idx_scan
FROM pg_stat_user_indexes s
JOIN pg_index i USING (indexrelid)
WHERE s.relname = 'orders' AND NOT i.indisunique
ORDER BY pg_relation_size(s.indexrelid) DESC;
```

    Indexes with `idx_scan = 0` over a representative period (check `pg_stat_database.stats_reset` first) are candidates for removal.

11. **Clean up.** `docker rm -f pg-idx`

7 Real-world scenario

Illustrative scenario, not a documented incident. On a Monday after a marketing email, the `shop-api` "My orders" endpoint goes from 40 ms to 6 s p95 and the RDS primary hits 95% CPU. The on-call engineer opens `pg_stat_statements`, sorts by `total_exec_time` and finds one query consuming 70% of database time: `SELECT * FROM orders WHERE user_id = $1 ORDER BY created_at DESC LIMIT 20`. `EXPLAIN (ANALYZE, BUFFERS)` shows a parallel seq scan with `Rows Removed by Filter: 41,000,000`: the table grew from 2 M to 42 M rows over the past year and the `user_id` index was never created (the ORM added only the primary key). They run `CREATE INDEX CONCURRENTLY idx_orders_user_created ON orders (user_id, created_at DESC)` during the incident; it takes 9 minutes without blocking checkout writes. The plan flips to an index scan, p95 drops to 8 ms, CPU falls to 15%. **Lesson:** read the plan before guessing; build concurrently; then add a check to the migration review process ("every foreign key and every `WHERE` column in a hot query has an index, with `EXPLAIN` output in the PR").

8 Common mistakes and best practices

Don't

- Create an index on every column "just in case": write latency, disk and replication lag all rise.
- Run plain `CREATE INDEX` on a busy production table: it blocks writes for the duration.
- Assume `EXPLAIN` (no `ANALYZE`) shows real behaviour. Estimates can be wildly wrong.
- Wrap indexed columns in functions or casts in `WHERE` clauses without an expression index.
- Run `EXPLAIN ANALYZE` on a destructive statement without a surrounding `BEGIN ... ROLLBACK`.
- Index low-cardinality columns (boolean, status with 3 values) with a full B-tree when a partial index or no index would do.

Do

- Use `EXPLAIN (ANALYZE, BUFFERS)` on realistic data volumes; a 1 000-row dev table hides every indexing problem.
- Put equality columns first and the range/sort column last in composite indexes.
- Prefer partial and covering indexes for hot queries on big tables.
- Use `CREATE INDEX CONCURRENTLY` and check for `INVALID` indexes afterwards.

- Index foreign-key columns you join or delete by (PostgreSQL does **not** index them automatically).
- Review `pg_stat_user_indexes` and `pg_stat_statements` periodically and drop unused or duplicate indexes.
- Keep statistics fresh: autovacuum runs `ANALYZE`, but run it manually after bulk loads.

9 Tasks

- [ ] **T1 — Seq scan to index scan** *(≈30 min, beginner)*
  - **Goal:** reproduce the before/after of adding a B-tree index.
  - **Steps:**
    1. Start PostgreSQL, create and load `orders` exactly as in section 6, steps 1-2.
    2. Run `EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE user_id = 4242;` and note `Execution Time` and `Buffers`.
    3. Create `idx_orders_user_id`, re-run the same query.
  - **Done when:** you have a two-line table "before vs after" showing node type, execution time and shared buffers, and the new plan uses `idx_orders_user_id`.
- [ ] **T2 — Find the planner's tipping point** *(≈30 min, intermediate)*
  - **Goal:** see that an index is not always used, and why.
  - **Steps:**
    1. With `idx_orders_user_id` in place, run `EXPLAIN SELECT * FROM orders WHERE user_id < 100;` then `< 1000`, `< 10000`, `< 40000`.
    2. Record the node type for each (Index Scan, Bitmap Heap Scan, Seq Scan) and the estimated rows.
    3. Run `SET enable_seqscan = off;` on the last query and compare `cost` and real time with `EXPLAIN ANALYZE`, then `RESET enable_seqscan;`.
  - **Done when:** you can state at roughly what fraction of the table the planner switches to a seq scan, and show that forcing the index was slower (or not) with numbers.
- [ ] **T3 — Partial and GIN indexes** *(≈45 min, intermediate)*
  - **Goal:** pick a specialised index that matches a query.
  - **Steps:**
    1. Create the GIN index from step 6 and the partial index from step 7.
    2. Compare `pg_relation_size` of the partial index, the GIN index and `orders_pkey`.
    3. Run `SELECT id FROM orders WHERE metadata @> '{"channel":"web"}'` with `EXPLAIN ANALYZE`; explain why it does or does not use the GIN index.

- **Done when:** you have the three sizes written down and a one-paragraph explanation of selectivity for the `channel` query.

- [ ] **T4 — Break it: defeat an index, then fix it with an expression index** *(≈30 min, intermediate)*
  - **Goal:** recognise a non-sargable (index-defeating) predicate and repair it.
  - **Steps:**
    1. Create a B-tree index on `created_at`. Run `EXPLAIN SELECT count(*) FROM orders WHERE date_trunc('day', created_at) = '2026-01-01';`. Observe the seq scan.
    2. Rewrite as a range: `created_at ⩾ '2026-01-01' AND created_at < '2026-01-02'`. Observe the index being used.
    3. Alternatively, try `CREATE INDEX ON orders (date_trunc('day', created_at));`. PostgreSQL rejects it with `functions in index expression must be marked IMMUTABLE`, because `date_trunc` on `timestamptz` depends on the session time zone. Create `CREATE INDEX ON orders (date_trunc('day', created_at AT TIME ZONE 'UTC'));` instead and show that `WHERE date_trunc('day', created_at AT TIME ZONE 'UTC') = '2026-01-01'` now uses it.
  - **Done when:** you have three plans (seq scan, range rewrite, expression index) and can say which fix you would choose and why.

- [ ] **T5 — Measure the write cost of indexes** *(≈45 min, advanced)*
  - **Goal:** quantify what indexes cost on writes.
  - **Steps:**
    1. Create `orders_noidx` and `orders_many` with `CREATE TABLE ... (LIKE orders INCLUDING DEFAULTS)`; add four indexes (including one GIN on `metadata`) to `orders_many`.
    2. With `\timing on`, run `INSERT INTO <table> SELECT * FROM orders LIMIT 300000;` into each.
    3. Compare sizes with `pg_total_relation_size`, and write a short script `index-audit.sql` listing unused non-unique indexes (query from step 10).
  - **Done when:** you have timings for both inserts (expect the heavily-indexed table to be several times slower, e.g. around 0.7 s vs several seconds, but it varies) and `index-audit.sql` runs without error.

- [ ] **T6 — Prove it: diagnose a slow query without the guide** *(≈45 min, advanced)*
  - **Goal:** demonstrate the full workflow on a query you did not see beforehand.
  - **Steps:**

1. Have a teammate (or yourself, a day later) write a slow query against `orders` (a join to a new `users` table with a filter and `ORDER BY ... LIMIT`).
2. Without this page, find the bottleneck from `EXPLAIN (ANALYZE, BUFFERS)`, design the minimal index, create it `CONCURRENTLY`, and verify.
3. Write a 5-line PR description: query, evidence (plan before), change, evidence (plan after), write-cost consideration.

- **Done when:** a reviewer can read your PR description and reproduce the improvement.

10 Self-check questions

1. Why is the composite index `(user_id, created_at)` useful for `WHERE user_id = 1` but not for `WHERE created_at > ...` alone?
2. A query returning 40% of a table ignores your index. Is that a bug?
3. What is the difference between `EXPLAIN` and `EXPLAIN ANALYZE`, and what is the danger of the latter?
4. What does `CREATE INDEX CONCURRENTLY` trade away, and how do you detect a failed one?
5. When would you choose a partial index over a full one?
6. Estimated rows are 10 and actual rows are 400 000. What do you suspect and do first?

ANSWERS

1. A B-tree is sorted by the first column, then the second within equal first values. Without a constraint on the leading column, PostgreSQL cannot narrow the search (it would have to scan the whole index).
2. No. Fetching 40% of rows via an index means a huge number of random heap reads; a seq scan (or bitmap scan) is cheaper. The planner is behaving correctly.
3. `EXPLAIN` shows the estimated plan only. `EXPLAIN ANALYZE` executes the statement and reports real timings and row counts, so `INSERT/UPDATE/DELETE` really change data unless you roll back.
4. It trades speed and simplicity (two table scans, waits for old transactions, cannot run in a transaction block) for not blocking writes. A failed build leaves an `INVALID` index (`pg_index.indisvalid = false`), which you `DROP INDEX CONCURRENTLY` and retry.
5. When queries target a small, stable subset (`status = 'pending'`, `deleted_at IS NULL`): the index is tiny, cheap to maintain and fast to scan.
6. Stale or inadequate statistics. Run `ANALYZE`, check for correlated columns (consider `CREATE STATISTICS`) or increase the statistics target, then re-check the plan.

11 Further reading

- PostgreSQL documentation: *Chapter "Indexes"* and *"Using EXPLAIN"* (under Performance Tips) at postgresql.org/docs.
- PostgreSQL documentation: `CREATE INDEX` reference page, including the "Building Indexes Concurrently" section.
- *The Art of PostgreSQL* by Dimitri Fontaine.
- *Use The Index, Luke!* by Markus Winand (free online) — a clear guide to how B-tree indexes interact with SQL.
- Next: Transactions and Locks §13.2 (what `CREATE INDEX` locks, and why) and Database Costs §20.8.

13.2 Transactions and Locks

Topic 2 of 15 | ~5 hours | Difficulty: Intermediate

1 What it is

A **transaction** is a group of SQL statements that the database treats as one unit of work: either all of it takes effect (`COMMIT`) or none of it does (`ROLLBACK`). Transactions are described by the **ACID** properties:

- **Atomicity** — all or nothing. A transfer that debits one account and crashes before crediting the other leaves no trace.
- **Consistency** — a transaction takes the database from one valid state to another; constraints (`CHECK`, `FOREIGN KEY`, `UNIQUE`) hold at commit.
- **Isolation** — concurrent transactions do not see each other's half-finished work; *how much* they see is set by the **isolation level**.
- **Durability** — once `COMMIT` returns, the data survives a crash (PostgreSQL guarantees this by flushing the **write-ahead log**, WAL, to disk first).

A **lock** is the mechanism a database uses to stop two transactions from clashing. PostgreSQL takes **row-level locks** (when you `UPDATE`, `DELETE` or `SELECT ... FOR UPDATE` a row) and **table-level locks** (nearly every statement takes one: even a plain `SELECT` takes `ACCESS SHARE`). When transaction B needs a lock that transaction A holds in a conflicting mode, B **waits**. If A is waiting for B at the same time, you have a **deadlock**, and PostgreSQL resolves it by aborting one of them.

2 Why it matters in DevOps

- **"The site is frozen but CPU is idle"** is the signature of lock waits: connections pile up behind one stuck transaction until the pool is exhausted (see Database Connection Pooling §13.6).
- **Migrations and deploys take locks.** `ALTER TABLE` needs an `ACCESS EXCLUSIVE` lock, and because lock requests queue, a single harmless-looking migration can block *every* query on the table behind a long-running one (demonstrated in the guide).

- **Idle transactions are poison.** An application that opens `BEGIN` and then waits on an HTTP call holds locks and, as you will see in MVCC §13.3, prevents cleanup.
- **Retries are your job.** Deadlocks and serialization failures are normal in correct systems. Application code and job runners must retry them; you set the policy (timeouts, retry budgets, alerting on rates).

3 Real-world system example

Stripe and other payment systems are the textbook case for the importance of transactional guarantees: moving money must be atomic, and retrying a failed request must not charge twice (see Idempotency §13.14). Stripe's public API documentation describes idempotency keys for exactly this reason, and engineering write-ups from payment companies commonly describe relying on database transactions plus unique constraints for the "exactly one ledger entry" guarantee.

GitLab publishes its database guidance openly; it documents that long transactions, lock waits and migrations that take heavy locks are a leading cause of incidents, and that migrations must be written to avoid blocking (using `lock_timeout` and retries, a pattern described in the GitLab developer docs). The PostgreSQL manual itself documents the table-level lock modes and which SQL commands acquire them, which is what migration tools such as `strong_migrations` (Rails) encode as rules.

Job queues on PostgreSQL. Libraries such as `River` (Go), `Graphile Worker` (Node.js) and `Oban` (Elixir) build job queues on `SELECT ... FOR UPDATE SKIP LOCKED`, a lock feature described below.

4 Diagram

A deadlock, step by step. PostgreSQL waits `deadlock_timeout` (default 1 s) before checking for a cycle, then aborts one transaction:

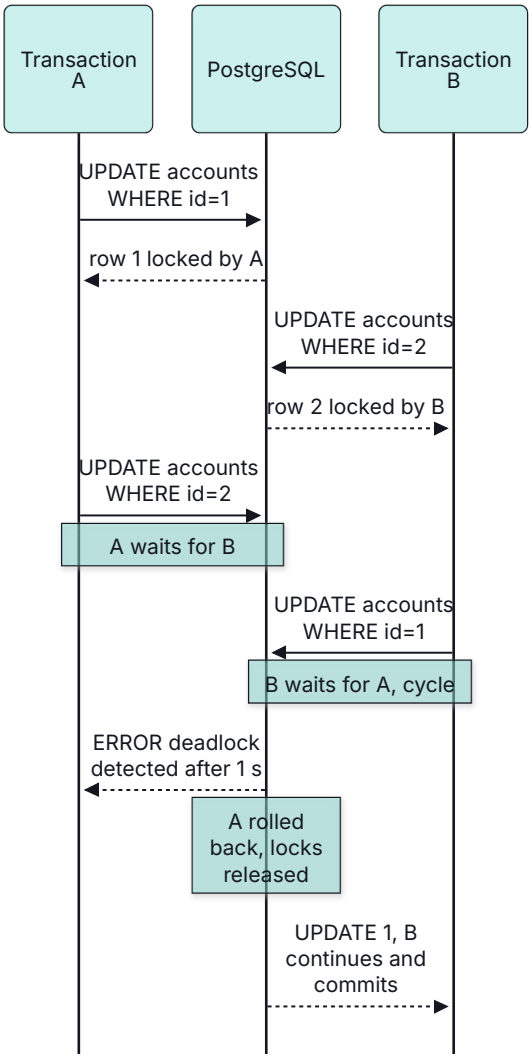


Figure 13.2-1

The lock queue that makes a migration dangerous:

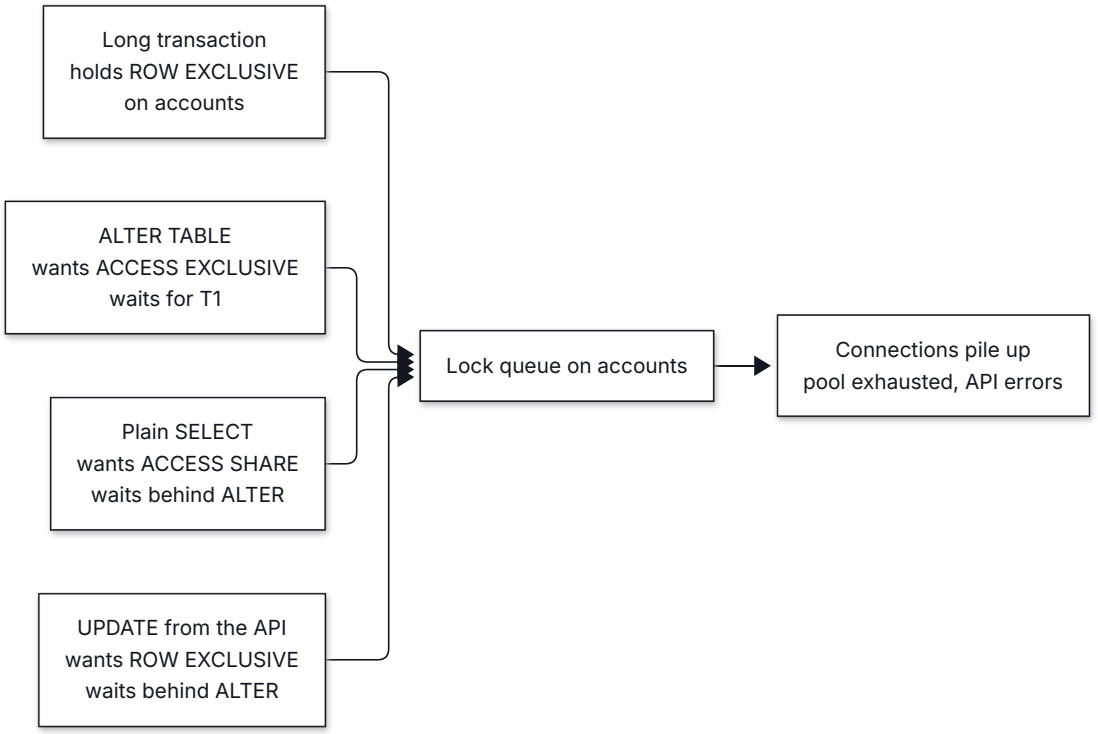


Figure 13.2-2

5 How it works

Isolation levels in PostgreSQL

Level	What a statement sees	Anomalies still possible	Typical use
READ COMMITTED (default)	a fresh snapshot per *statement*	non-repeatable reads, lost updates with read-modify-write, write skew	most OLTP code
REPEATABLE READ	one snapshot for the whole transaction	write skew (PostgreSQL's version also prevents phantom reads); concurrent updates raise `could not serialize access due to concurrent update`	reports needing consistent reads
SERIALIZABLE	as if transactions ran one at a time	none, but you get `could not serialize access` errors and must retry	correctness-critical invariants

(`READ UNCOMMITTED` exists in syntax but behaves as READ COMMITTED in PostgreSQL. Dirty reads never happen.)

Write skew is the anomaly worth remembering: two transactions each read "at least 2 engineers are on call", each remove a different engineer, both commit, and now nobody is on call. Neither wrote the same row, so row locks never conflicted. Only `SERIALIZABLE` (or an explicit lock) prevents it.

Lock types you will meet

Lock	Taken by	Blocks
`ACCESS SHARE` (table)	`SELECT`	only `ACCESS EXCLUSIVE`
`ROW EXCLUSIVE` (table)	`INSERT`, `UPDATE`, `DELETE`	`SHARE` and stronger (so it blocks plain `CREATE INDEX`)
`SHARE UPDATE EXCLUSIVE`	`VACUUM`, `CREATE INDEX CONCURRENTLY`, some `ALTER TABLE` forms	itself and stronger; does **not** block writes
`SHARE`	plain `CREATE INDEX`	all writes (`INSERT/UPDATE/DELETE`)
`ACCESS EXCLUSIVE`	most `ALTER TABLE`, `DROP TABLE`, `TRUNCATE`, `VACUUM FULL`, `REINDEX`	**everything**, including `SELECT`
Row lock `FOR UPDATE`	`UPDATE`, `DELETE`, `SELECT ... FOR UPDATE`	other writers/lockers of the same row (readers are never blocked)
Row lock `FOR SHARE` / `FOR KEY SHARE`	`SELECT ... FOR SHARE`; foreign-key checks	writers of the row
Advisory lock	`pg_advisory_lock(key)`	whoever asks for the same key; app-defined mutual exclusion

Key facts:

- **Readers never block writers and writers never block readers** for row data (this is MVCC, next topic §13.3). Only `ACCESS EXCLUSIVE` blocks reads.
- **Locks are held until the end of the transaction**, not the end of the statement. Hold them briefly.
- **Lock requests queue.** A waiting `ACCESS EXCLUSIVE` request blocks all later requests behind it, even compatible ones.
- `SELECT ... FOR UPDATE SKIP LOCKED` skips rows other sessions have locked, which is how workers pull jobs without stepping on each other. `NOWAIT` fails immediately instead of waiting.

Settings that protect you

Setting	Meaning	Common production value
`lock_timeout`	give up if a lock isn't acquired within this time	`2s` - `5s` set at the start of migrations
`statement_timeout`	cancel any statement exceeding this	per-role, e.g. `30s` for web, higher for batch
`idle_in_transaction_session_timeout`	kill sessions that sit in `BEGIN` doing nothing	`30s` - `60s` for web roles
`deadlock_timeout`	how long to wait before checking for a deadlock (default `1s`)	leave at default
`log_lock_waits`	log any lock wait longer than `deadlock_timeout`	`on`

Inspecting locks

```sql
-- who is blocked, and by whom
SELECT pid, pg_blocking_pids(pid) AS blocked_by, wait_event_type, wait_event, state,
  left(query, 60) AS query
FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

`pg_blocking_pids(pid)` returns the sessions holding or queued ahead of the lock a given backend wants. `pg_locks` shows every lock and whether it was `granted`.

6 Step-by-step guide

Prerequisites: Docker, or local PostgreSQL 16+. You need **two or three** terminals each running `psql`. Cost: free. All commands below were verified on PostgreSQL 16; error text can differ slightly between versions.

1. **Start PostgreSQL and open the first session.**

   ```bash
   docker run -d --name pg-tx -e POSTGRES_PASSWORD=<YOUR_PASSWORD> -p 5432:5432
     postgres:16
   docker exec -it pg-tx psql -U postgres        # terminal A
   ```

   Open the other terminals with the same `docker exec` command (B, C).

2. **Create sample data in A.**

```sql
CREATE TABLE accounts (id int PRIMARY KEY, owner text, balance int NOT NULL CHECK
  (balance >= 0));
INSERT INTO accounts VALUES (1, 'alice', 100), (2, 'bob', 100);
```

3. **Atomicity: a rolled-back transfer.**

```sql
BEGIN;
UPDATE accounts SET balance = balance - 50 WHERE id = 1;
UPDATE accounts SET balance = balance + 50 WHERE id = 2;
SELECT * FROM accounts ORDER BY id;     -- inside the tx: 50 / 150
ROLLBACK;
SELECT * FROM accounts ORDER BY id;     -- back to 100 / 100
```

Now try `UPDATE accounts SET balance = balance - 500 WHERE id = 1;` and note the `CHECK` constraint error: consistency enforced by the database.

4. **Row-lock blocking.** In A:

```sql
BEGIN;
UPDATE accounts SET balance = balance - 10 WHERE id = 1;     -- A now holds the row
  lock; do NOT commit
```

In B:

```sql
UPDATE accounts SET balance = balance + 1 WHERE id = 1;      -- hangs: waiting for A
```

In C, find the blocker:

```sql
SELECT pid, pg_blocking_pids(pid) AS blocked_by, wait_event_type, wait_event, state,
  left(query, 50) AS query
FROM pg_stat_activity WHERE datname = current_database() AND pid <> pg_backend_pid();
```

Similar to: B shows `wait_event_type = Lock`, `wait_event = transactionid`, `blocked_by = {<A's pid>}`. In A run `COMMIT;` and B instantly completes. Note that a plain `SELECT * FROM accounts WHERE id = 1` in C was never blocked.

5. **A deadlock.** In A: `BEGIN; UPDATE accounts SET balance = balance - 1 WHERE id = 1;`. In B: `BEGIN; UPDATE accounts SET balance = balance - 1 WHERE id = 2;`. In A: `UPDATE accounts SET balance = balance + 1 WHERE id = 2;` (waits). In B: `UPDATE accounts SET balance = balance + 1 WHERE id = 1;`. After about one second one session gets:

```
ERROR:  deadlock detected
DETAIL:  Process 7021 waits for ShareLock on transaction 777; blocked by process 7033.
Process 7033 waits for ShareLock on transaction 776; blocked by process 7021.
HINT:  See server log for query details.
```

   (PIDs and transaction ids differ.) Run `ROLLBACK;` in both. The fix is **consistent lock ordering**: always update the lower `id` first.

6. **Compare isolation levels with the on-call (write skew) example.**

```sql
CREATE TABLE oncall (engineer text PRIMARY KEY, on_call boolean NOT NULL);
INSERT INTO oncall VALUES ('alice', true), ('bob', true);
```

   In A and B, run (replacing the level as you test each one):

```sql
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM oncall WHERE on_call;                  -- both see 2
```

   Then A runs `UPDATE oncall SET on_call = false WHERE engineer = 'alice';`, B runs `UPDATE oncall SET on_call = false WHERE engineer = 'bob';`, and both `COMMIT;`. Check `SELECT count(*) FROM oncall WHERE on_call;`. With `READ COMMITTED` and `REPEATABLE READ` the result is **0**: nobody is on call. Reset with `UPDATE oncall SET on_call = true;` and repeat with `ISOLATION LEVEL SERIALIZABLE`: the second committer fails with

```
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during write.
HINT:  The transaction might succeed if retried.
```

   and the count stays 1. The application must catch SQLSTATE `40001` and retry.

7. **See the repeatable-read update conflict.** In A: `BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT balance FROM accounts WHERE id = 1;`. In B: `UPDATE accounts SET balance = balance + 20 WHERE id = 1;` (autocommits). In A: `UPDATE accounts SET balance = balance + 10 WHERE id = 1;`. Result: `ERROR: could not serialize access due to concurrent update`. In `READ COMMITTED` the same sequence just works, re-reading the new value.

8. **Reproduce the migration pile-up.** In A: `BEGIN; UPDATE accounts SET balance = balance WHERE id = 1;` (leave open). In B: `ALTER TABLE accounts ADD COLUMN note text;` (hangs). In C: `SELECT * FROM accounts;` (**also hangs**, even though nobody is writing). In another session look at `SELECT pid, pg_blocking_pids(pid), left(query,40) FROM pg_stat_activity WHERE datname = current_database();` : C is blocked by B, B by A. Then `COMMIT;` in A and watch everything drain. Now repeat with a safety net:

```sql
SET lock_timeout = '500ms';
ALTER TABLE accounts ADD COLUMN note2 text;
-- ERROR:  canceling statement due to lock timeout   (retry later instead of freezing
  the app)
```

9. **Job-queue pattern with `SKIP LOCKED`.**

```sql
CREATE TABLE jobs (id serial PRIMARY KEY, payload text, state text DEFAULT 'queued');
INSERT INTO jobs (payload) SELECT 'job-' || g FROM generate_series(1, 5) g;
-- worker 1 (terminal A)
BEGIN;
SELECT id FROM jobs WHERE state = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED;
  -- gets 1, 2
-- worker 2 (terminal B) while A is still open
SELECT id FROM jobs WHERE state = 'queued' ORDER BY id LIMIT 2 FOR UPDATE SKIP LOCKED;
  -- gets 3, 4
```

10. **Enable the production safety nets.**

```sql
ALTER ROLE postgres SET idle_in_transaction_session_timeout = '60s';
ALTER SYSTEM SET log_lock_waits = on;
SELECT pg_reload_conf();
```

    (Use a dedicated application role in real systems, not `postgres`.)

11. **Clean up.** `docker rm -f pg-tx`

7 Real-world scenario

Illustrative scenario, not a documented incident. At 10:05 a deploy of `shop-api` runs a migration `ALTER TABLE orders ADD COLUMN gift_message text`. Normally it is instant. Today, the nightly analytics export had opened a `REPEATABLE READ` transaction on `orders` at 09:58 and was still running. The `ALTER` requested `ACCESS EXCLUSIVE` and queued behind it. Within seconds every checkout `INSERT` and every `SELECT` on `orders` queued behind the `ALTER`; the pool of

100 connections filled, the load balancer health checks failed, and the status page went red. The on-call runs the blocking query using `pg_blocking_pids`, sees the chain (checkout → ALTER → export), and cancels the *ALTER* with `pg_cancel_backend`; the pile drains in about a second. They re-run the migration later with `SET lock_timeout = '3s'` inside a retry loop. **Lesson:** the dangerous party is not the long-running reader; it is the *waiting* `ACCESS EXCLUSIVE` request. Always set `lock_timeout` for DDL, schedule migrations away from long jobs, and set `statement_timeout` for exports.

8 Common mistakes and best practices

Don't

- Leave transactions open while calling external services, waiting for user input, or sleeping.
- Run DDL without `lock_timeout`.
- Assume `READ COMMITTED` prevents read-modify-write races (`SELECT balance` in the app, then `UPDATE ... SET balance = <computed>`). Use `UPDATE ... SET balance = balance - 10`, `SELECT ... FOR UPDATE`, or `SERIALIZABLE`.
- Take locks in different orders in different code paths (the recipe for deadlocks).
- Treat `deadlock detected` or `40001` as a bug to hide. They are a signal to retry; alert on a high rate.
- Use `SERIALIZABLE` globally without retry logic everywhere.

Do

- Keep transactions short: do the I/O before `BEGIN` and after `COMMIT`.
- Lock rows in a consistent order (for example, ascending primary key).
- Set `idle_in_transaction_session_timeout`, `statement_timeout`, `lock_timeout` and `log_lock_waits`.
- Use `FOR UPDATE SKIP LOCKED` for queues and `NOWAIT` for "fail fast" semantics.
- Use `CREATE INDEX CONCURRENTLY` and, for risky DDL, a retry loop with a short `lock_timeout` (see Database Migrations in Pipelines §5.9).
- Monitor `pg_stat_activity` for `idle in transaction` sessions older than a few seconds and for long `wait_event_type = Lock` waits.

9 Tasks

- [ ] **T1 — Atomicity and constraints** *(≈20 min, beginner)*
  - **Goal:** see rollback and constraint enforcement.

- **Steps:**
  1. Create `accounts` (section 6, step 2).
  2. In one transaction debit alice 50, credit bob 50, `SELECT` inside, then `ROLLBACK`; `SELECT` again.
  3. Attempt a debit that violates the `CHECK` constraint inside a transaction and observe what `COMMIT` does after the error.
- **Done when:** you have shown balances unchanged after rollback and can explain why `COMMIT` after an error acts like `ROLLBACK` (the transaction is in an aborted state).

☐ **T2 — Observe a lock wait and name the blocker** *(≈30 min, beginner)*

- **Goal:** use `pg_stat_activity` and `pg_blocking_pids` like an on-call engineer.
- **Steps:**
  1. Reproduce step 4 with three terminals.
  2. Write the query that lists blocked sessions, their blockers and the blockers' queries (join `pg_stat_activity` to itself on `pg_blocking_pids`).
  3. Also run `SELECT locktype, mode, granted, pid FROM pg_locks WHERE relation = 'accounts'::regclass;`.
- **Done when:** your query output shows one blocked session, its blocker pid, and the blocker's state `idle in transaction`.

☐ **T3 — Cause and fix a deadlock** *(≈40 min, intermediate)*

- **Goal:** produce a deadlock deliberately, find it in the server log, and remove it by lock ordering.
- **Steps:**
  1. Reproduce step 5. Enable `log_lock_waits` and read the full deadlock report in the PostgreSQL log (`docker logs pg-tx`).
  2. Write a `transfer(from_id, to_id, amount)` SQL function (or script) that always updates the lower id first.
  3. Run two concurrent opposite transfers 20 times each using `pgbench -f` custom scripts or two shell loops.
- **Done when:** the naive version produces at least one `deadlock detected`, and the ordered version produces none across all runs.

☐ **T4 — Isolation level shootout** *(≈45 min, intermediate)*

- **Goal:** demonstrate which anomalies each level permits.
- **Steps:**

1. Run the on-call experiment (step 6) under `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`; record the final count each time.
2. Run the lost-update experiment (step 7) under both `READ COMMITTED` and `REPEATABLE READ`.
3. Write a table: level × (write skew possible? serialization error raised?).

- **Done when:** your table matches the results (write skew at the first two levels, none at `SERIALIZABLE`; a `concurrent update` error at `REPEATABLE READ`).

☐ **T5 — Safe migrations: timeout and retry** *(≈45 min, advanced)*

- **Goal:** automate the safe-DDL pattern.
- **Steps:**
  1. Write a shell script `safe-ddl.sh` that runs `psql` with `SET lock_timeout='2s'; ALTER TABLE accounts ADD COLUMN flag boolean;` and retries up to 10 times with 1-5 s random sleep when it sees `lock timeout`.
  2. Hold a conflicting transaction open for 8 seconds in another terminal.
  3. Watch the script fail a few times and then succeed once the blocker commits; confirm that a concurrent `SELECT` loop never stalled more than ~2 s.
- **Done when:** the script's log shows failed attempts followed by success, and the column exists.

☐ **T6 — Prove it: explain the freeze** *(≈45 min, advanced)*

- **Goal:** teach lock queues to someone else, with evidence.
- **Steps:**
  1. Without this page, recreate the "ALTER TABLE behind a long transaction" pile-up and capture the `pg_stat_activity` blocking chain.
  2. Write a 10-line runbook: how to detect (query), how to decide what to cancel (`pg_cancel_backend` vs `pg_terminate_backend`), and how to prevent (`lock_timeout`, idle timeout).
- **Done when:** a teammate can use your runbook to resolve a staged lock pile-up in under five minutes.

10 Self-check questions

1. What does ACID's "I" mean in PostgreSQL's default isolation level, and what can still go wrong?
2. Why did the harmless `SELECT` in step 8 hang even though no one was writing?

3. Two transactions each updated a different row after reading the same count and violated a business rule. Which anomaly is it and how do you prevent it?
4. What is the difference between `pg_cancel_backend` and `pg_terminate_backend`?
5. How do you avoid deadlocks, and what must the application do when one still happens?
6. Why is `idle in transaction` dangerous?

ANSWERS

1. Each statement sees a snapshot as of its start (`READ COMMITTED`), so no dirty reads, but two reads in a transaction can differ and read-modify-write sequences can lose updates or produce write skew.
2. Lock requests queue. The `ALTER TABLE` waiting for `ACCESS EXCLUSIVE` sits in the queue and every later request, even a compatible `ACCESS SHARE`, waits behind it.
3. Write skew. Use `SERIALIZABLE` (and retry on `40001`) or take an explicit lock that makes the transactions conflict (for example `SELECT ... FOR UPDATE` on the rows, or a lock on a parent row).
4. `pg_cancel_backend` cancels the current query but keeps the connection; `pg_terminate_backend` closes the whole connection (rolling back its transaction).
5. Acquire locks in a consistent order and keep transactions short. The application must catch the error (SQLSTATE `40P01`) and retry the whole transaction.
6. It holds locks and a snapshot while doing nothing: it can block DDL and other writers, and it stops vacuum from removing dead rows (bloat).

11 Further reading

- PostgreSQL documentation: *Chapter "Concurrency Control"* (Transaction Isolation, Explicit Locking, Serialization Failure Handling) at postgresql.org/docs.
- PostgreSQL documentation: `pg_locks`, `pg_stat_activity` and `pg_blocking_pids` reference pages.
- *Designing Data-Intensive Applications* by Martin Kleppmann — chapter on transactions (isolation levels and write skew).
- Related: PostgreSQL Indexes §13.1, MVCC §13.3, Distributed Locks and Queues §13.15, and Database Migrations in Pipelines §5.9.

LAB NOTES What I ran, what I saw, what it means

13.3 MVCC

Topic 3 of 15 | ~5 hours | Difficulty: Intermediate

1 What it is

MVCC (Multi-Version Concurrency Control) is the trick that lets PostgreSQL serve readers and writers at the same time without readers blocking writers or writers blocking readers. Instead of overwriting a row in place, an `UPDATE` writes a **new version** of the row and leaves the old version where it is. Each transaction is then shown the versions that were "current" at the moment its **snapshot** was taken. Two transactions can legitimately see different values for the same logical row at the same instant.

The price is that old versions pile up. A row that is updated 1,000 times exists as 1,000 physical **tuples** (row versions) until something removes the ones nobody can see any more. That something is **VACUUM**. Almost every PostgreSQL operations problem that is not "a missing index" or "a lock" is some form of "VACUUM could not keep up": table **bloat** (disk space full of dead tuples), slow sequential scans, runaway disk usage, or, in the worst case, transaction ID **wraparound**. This topic builds the mental model you need to diagnose all of them.

2 Why it matters in DevOps

- **Disk and performance incidents:** "the table is 40 GB but holds 4 GB of live data" is bloat. It slows scans, wastes cache (RAM) and backups, and costs real money on managed databases that bill per GB (see Database Costs §20.8).
- **One forgotten transaction can hurt the whole database.** An `idle in transaction` session, an abandoned replication slot or a multi-hour report pins the **xmin horizon** so VACUUM cannot clean *any* table in that database. This is the single most common cause of mysterious bloat on production systems.
- **Wraparound is an outage, not a slowdown.** Transaction IDs are 32-bit. If anti-wraparound vacuuming falls behind far enough, PostgreSQL refuses writes to protect your data. You must monitor `age(datfrozenxid)`.
- **Replication interacts with it:** a long query on a replica or `hot_standby_feedback` can hold back cleanup on the primary (see Replication §13.4).

- **It explains the isolation levels** you met in Transactions and Locks §13.2: they differ only in *when* the snapshot is taken.

3 Real-world system example

PostgreSQL itself. MVCC with in-heap old versions and a background cleaner is the core design of PostgreSQL. The `autovacuum` daemon ships enabled by default and is the reason most databases "just work" without a DBA scheduling cleanups. The trade-off was publicly debated when **Uber's engineering blog (2016)** described moving a major workload from PostgreSQL to MySQL, citing, among other things, write amplification from PostgreSQL's row-versioning and its interaction with replication. Other engineers disputed parts of that analysis, which is a good illustration of the point: MVCC designs have real trade-offs, and which one is "better" depends on workload. MySQL's InnoDB also uses MVCC but keeps old versions in a separate **undo log** rather than in the table itself.

Transaction ID wraparound in production. Several companies have publicly written about outages or near-misses caused by wraparound protection on very large or very busy PostgreSQL databases (Sentry and the Mailchimp/Mandrill transactional-email service have both published write-ups). The common pattern is the same: autovacuum was blocked or too slow on a huge table, nobody alerted on transaction ID age, and the database finally went read-only until a long manual vacuum finished.

Managed services expose the same machinery: Amazon RDS and Aurora PostgreSQL, Google Cloud SQL and Azure Database for PostgreSQL all run autovacuum and document tuning it; Amazon RDS publishes guidance on monitoring `age(datfrozenxid)`.

4 Diagram

How one row is "updated" and who sees which version:

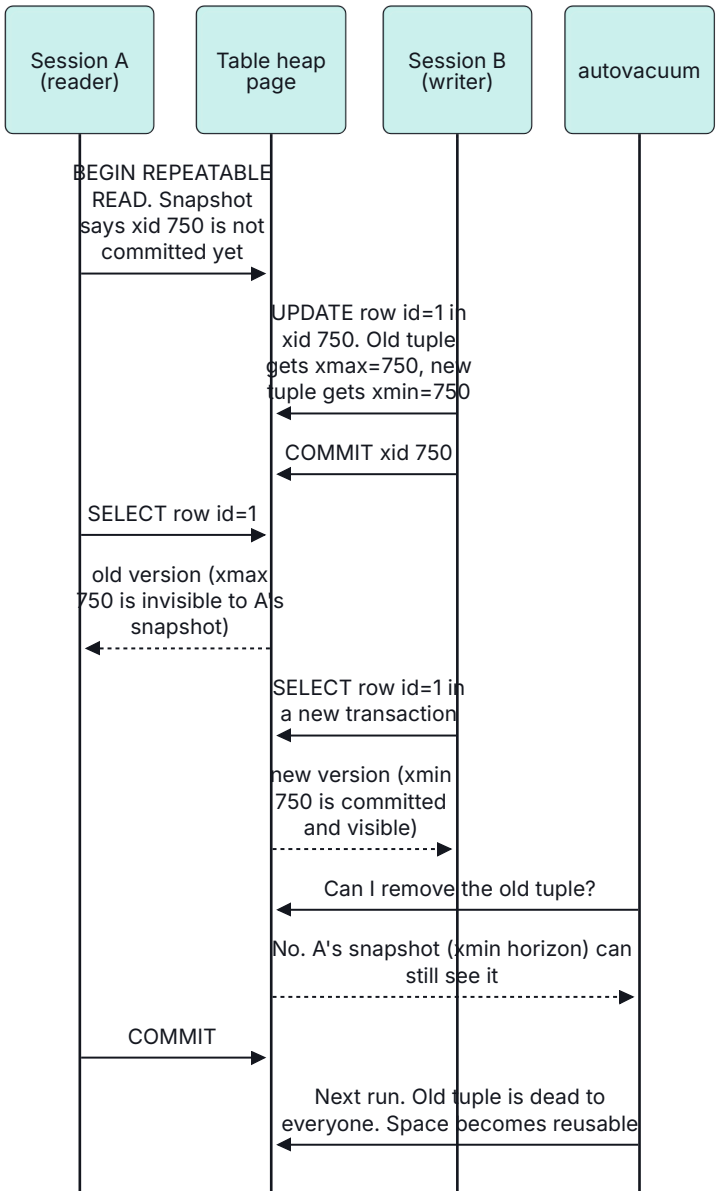


Figure 13.3-1

The life cycle of a tuple:

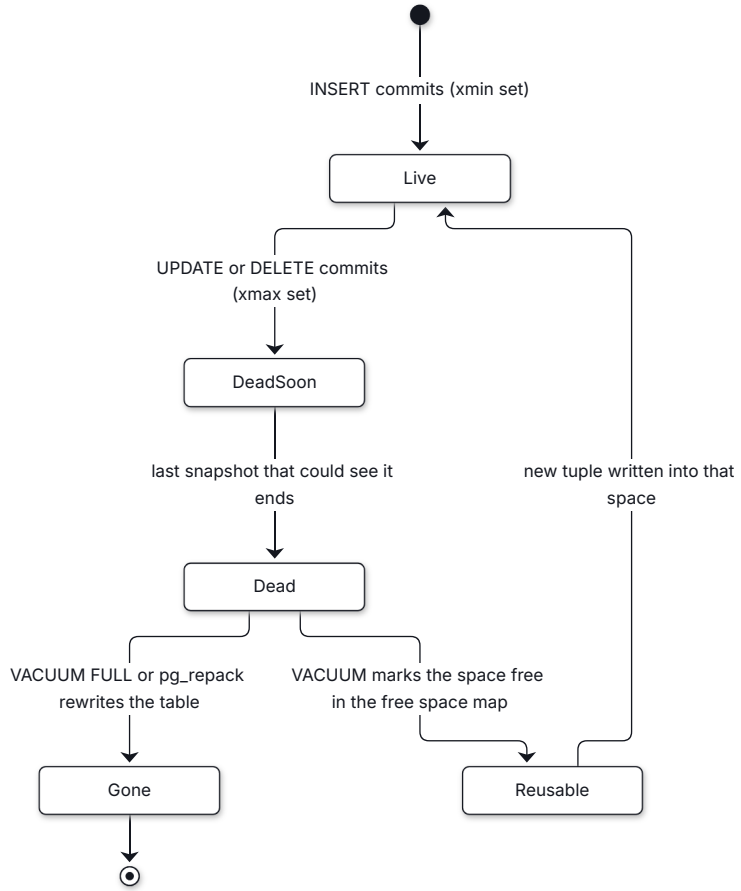


Figure 13.3-2

5 How it works

Tuple headers: xmin and xmax

Every row version carries hidden system columns you can select:

Column	Meaning
`xmin`	ID of the transaction that **created** this version (the `INSERT`, or the `UPDATE` that wrote the new version)
`xmax`	ID of the transaction that **deleted or replaced** it (0 means nobody has, or the deleter rolled back). Also used for row locks such as `SELECT ... FOR UPDATE`
`ctid`	Physical location `(page, item)`. An `UPDATE` changes it because the new version lives somewhere else

Transaction IDs (**XIDs**) are 32-bit counters that only grow. Every write transaction gets one.

Snapshots and visibility

A **snapshot** is, roughly, three facts: "everything below X is finished", "everything at or above Y has not started", and "this list of transactions in between were still running". Run `SELECT pg_current_snapshot();` to see it as `xmin:xmax:xip_list`. A tuple is visible to a snapshot when its `xmin` transaction committed *before* the snapshot and its `xmax` is either unset, aborted, or not yet visible to that snapshot. Commit status is looked up in the **commit log** (`pg_xact`), and cached as hint bits on the tuple.

When is the snapshot taken?

Isolation level	Snapshot taken	Effect
`READ COMMITTED` (default)	at the start of **each statement**	each statement sees the latest committed data
`REPEATABLE READ`	at the first statement of the transaction	whole transaction sees one frozen view
`SERIALIZABLE`	same as repeatable read, plus conflict tracking (SSI)	may abort with a serialization failure

Why UPDATE is "delete + insert"

An `UPDATE` marks the old tuple's `xmax` and inserts a new tuple. If the new version fits on the same page and no indexed column changed, PostgreSQL can do a **HOT update** (Heap-Only Tuple): no new index entries are needed, and the old version can be pruned cheaply. Leaving some free space per page (`fillfactor`, e.g. 90 or 80 for hot tables) makes HOT updates more likely.

VACUUM

`VACUUM` scans the table (using a visibility map to skip pages that are all-visible), removes index entries that point to dead tuples, and records the freed space in the free space map so future inserts and updates **reuse it**. Key facts:

- Plain `VACUUM` does **not** shrink the file (except trailing empty pages) and does not block reads or writes. It makes space *reusable*.
- `VACUUM FULL` rewrites the whole table into a new file and *does* return space to the OS, but takes an `ACCESS EXCLUSIVE` lock (blocks all reads and writes) for the duration. Online alternatives are the `pg_repack` extension or `pg_squeeze`.
- `VACUUM` also **freezes** old tuples so that XIDs can be recycled (see wraparound below), and updates the statistics `ANALYZE` uses.

- **Autovacuum** launches workers automatically when dead tuples exceed `autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor * reltuples`. Defaults: 50 + 20% of the table. On a 100M row table that is 20M dead rows before it even starts. Large hot tables need per-table settings.

The xmin horizon: why long transactions are dangerous

VACUUM may only remove a dead tuple if **no existing or future snapshot can see it**. The oldest snapshot anywhere in the cluster defines the **xmin horizon**. Things that hold it back:

Culprit	How to spot it
Long-running query or an `idle in transaction` session	`pg_stat_activity.xact_start`, `backend_xmin`
Abandoned or lagging **replication slot**	`pg_replication_slots.xmin` / `catalog_xmin`, `active = false`
`hot_standby_feedback = on` with a long query on a replica	replica's `backend_xmin` appears on the primary via the walsender
Forgotten **prepared transaction** (`PREPARE TRANSACTION`)	`pg_prepared_xacts`

While the horizon is stuck, `VACUUM` reports `N are dead but not yet removable` and every update keeps growing the table. Besides bloat, long transactions also keep locks and, with `REPEATABLE READ`, can hit serialization errors.

Wraparound

With 32-bit XIDs, "older" and "newer" are defined modulo about 2 billion. A tuple whose `xmin` is more than ~2 billion transactions in the past would suddenly look like it is in the future, so VACUUM **freezes** old tuples (marks them as visible-to-everyone). Related settings: `autovacuum_freeze_max_age` (default 200 million) forces an *anti-wraparound* autovacuum even if autovacuum is otherwise disabled. If the age approaches the hard limit, PostgreSQL stops accepting new write transactions. Monitor `age(datfrozenxid)` per database and `age(relfrozenxid)` per table.

Command and view reference

Tool	Use
`SELECT xmin, xmax, ctid, * FROM t;`	See version metadata
`pageinspect` extension, `heap_page_items()`	Look at raw tuples on a page
`pgstattuple` extension	Exact dead-tuple and free-space percentages (reads the whole table)

Tool	Use
`pg_stat_user_tables`	`n_live_tup`, `n_dead_tup`, `last_autovacuum`, `autovacuum_count`
`pg_stat_progress_vacuum`	Live progress of a running vacuum
`VACUUM (VERBOSE, ANALYZE) t;`	Manual run with a report
`idle_in_transaction_session_timeout`	Kill sessions that sit idle inside a transaction
`statement_timeout`, `transaction_timeout` (PostgreSQL 17+)	Cap runaway queries / transactions

6 Step-by-step guide

Prerequisites: PostgreSQL 16 or newer with `psql`, and contrib modules (`pageinspect`, `pgstattuple`). With Docker, the official image includes them: `docker run -d --name pg-mvcc -e POSTGRES_PASSWORD=<YOUR_PASSWORD> postgres:16` then `docker exec -it pg-mvcc psql -U postgres`. Cost: free. Time: ~40 minutes. Outputs below were captured on PostgreSQL 16.15; your transaction numbers and sizes will differ slightly.

1. **Create a database and a small table.**

```sql
CREATE DATABASE mvcc_demo;
\c mvcc_demo
CREATE EXTENSION pageinspect;
CREATE EXTENSION pgstattuple;
CREATE TABLE accounts (id int PRIMARY KEY, balance int NOT NULL);
INSERT INTO accounts VALUES (1, 100), (2, 200);
SELECT xmin, xmax, ctid, * FROM accounts;
```

```
 xmin | xmax | ctid  | id | balance
------+------+-------+----+---------
  748 |    0 | (0,1) |  1 |     100
  748 |    0 | (0,2) |  2 |     200
```

   Both rows were created by the same transaction (`xmin` is equal) and nobody deleted them (`xmax = 0`).

2. **Update a row and watch the version move.**

```sql
UPDATE accounts SET balance = 150 WHERE id = 1;
SELECT xmin, xmax, ctid, * FROM accounts;
```

```
 xmin | xmax | ctid  | id | balance
------+------+-------+----+---------
  748 |    0 | (0,2) |  2 |     200
  750 |    0 | (0,3) |  1 |     150
```

Row `id=1` now has a new `xmin` and lives at `ctid (0,3)`: it is a different physical tuple.

3. **Look at the old version that is still on the page.**

```sql
SELECT lp, t_xmin, t_xmax, t_ctid FROM heap_page_items(get_raw_page('accounts', 0));
```

```
 lp | t_xmin | t_xmax | t_ctid
----+--------+--------+--------
  1 |    748 |    750 | (0,3)
  2 |    748 |      0 | (0,2)
  3 |    750 |      0 | (0,3)
```

Item 1 is the dead version: `t_xmax = 750` and `t_ctid` points to its successor (item 3). Your `SELECT` skips it, but it is still on disk until VACUUM removes it.

4. **Prove that two sessions see different truths.** Open two `psql` windows on `mvcc_demo`.

```sql
-- Session A
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 2;      -- 200
```

```sql
-- Session B
UPDATE accounts SET balance = 999 WHERE id = 2; -- autocommits
```

```sql
-- Session A again
SELECT balance FROM accounts WHERE id = 2;      -- still 200
COMMIT;
SELECT balance FROM accounts WHERE id = 2;      -- now 999
```

Session A keeps its snapshot until it ends. Repeat with `READ COMMITTED` (the default) and the second `SELECT` returns 999.

5. **Generate bloat on a bigger table.**

```sql
CREATE TABLE events (id serial PRIMARY KEY, payload text);
INSERT INTO events (payload) SELECT repeat('x', 100) FROM generate_series(1, 100000);
SELECT pg_size_pretty(pg_relation_size('events')) AS size;
UPDATE events SET payload = repeat('y', 100);
SELECT pg_size_pretty(pg_relation_size('events')) AS size_after_update;
```

```
 size
-------
 13 MB
 size_after_update
-------------------
 27 MB
```

Same 100,000 live rows, twice the disk. The old versions of every row are still there.

6. **Now break VACUUM with a long transaction.** In a second `psql` window start a transaction and leave it open:

```sql
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM events;
-- leave this window open; do not commit
```

Back in the first window:

```sql
UPDATE events SET payload = repeat('z', 100);
VACUUM (VERBOSE) events;
```

```
tuples: 0 removed, 200000 remain, 100000 are dead but not yet removable
removable cutoff: 753, which was 1 XIDs old when operation ended
```

`not yet removable` is the signature of a pinned xmin horizon. Find the culprit:

```sql
SELECT pid, state, now() - xact_start AS xact_age, backend_xmin, age(backend_xmin) AS
  xmin_age, left(query, 40) AS query
FROM pg_stat_activity
WHERE backend_xmin IS NOT NULL
ORDER BY age(backend_xmin) DESC;
```

The idle session with the oldest `backend_xmin` is at the top. (In a real incident you would also check `pg_replication_slots` and `pg_prepared_xacts`.)

7. **Release the horizon and vacuum for real.** In the second window run `COMMIT;`, then:

```sql
VACUUM (VERBOSE) events;
```

```
tuples: 100000 removed, 100000 remain, 0 are dead but not yet removable
```

Check the size:

```sql
SELECT pg_size_pretty(pg_relation_size('events'));          -- still ~27 MB
UPDATE events SET payload = repeat('q', 100);
SELECT pg_size_pretty(pg_relation_size('events'));          -- still ~27 MB, no new
  growth
```

Plain `VACUUM` did not shrink the file, but the freed space was **reused** by the next update instead of growing the table.

8. **Measure bloat, then reclaim the space.**

```sql
UPDATE events SET payload = 'a' WHERE id ⩽ 50000;
SELECT table_len, tuple_count, dead_tuple_count, round(dead_tuple_percent) AS dead_pct
  FROM pgstattuple('events');
VACUUM FULL events;      -- takes an ACCESS EXCLUSIVE lock; fine in a lab, careful in
  production
SELECT pg_size_pretty(pg_relation_size('events'));
```

The file drops back to roughly 13 MB. In production prefer `pg_repack` (online) or fix the cause so you never need it.

9. **Add the production guard rails.** Monitoring queries you can drop into a dashboard or

alert:

```sql
-- Oldest transaction in the cluster (alert if > 15-30 minutes on an OLTP system)
SELECT max(now() - xact_start) FROM pg_stat_activity WHERE xact_start IS NOT NULL;

-- Wraparound distance per database (alert well before 1 billion; the forced-vacuum
  trigger is 200 million by default)
SELECT datname, age(datfrozenxid) FROM pg_database ORDER BY 2 DESC;

-- Tables with the most dead tuples
SELECT relname, n_live_tup, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables ORDER BY n_dead_tup DESC LIMIT 10;

-- Replication slots that might be pinning the horizon
SELECT slot_name, active, xmin, catalog_xmin, restart_lsn FROM pg_replication_slots;
```

And two safety settings (adjust to your workload; set per role if only some clients need them):

```sql
ALTER SYSTEM SET idle_in_transaction_session_timeout = '5min';
SELECT pg_reload_conf();
ALTER TABLE events SET (autovacuum_vacuum_scale_factor = 0.02,
  autovacuum_vacuum_threshold = 1000);
```

10. **Clean up.** `DROP DATABASE mvcc_demo;` (from another database) or `docker rm -f pg-mvcc`.

7 Real-world scenario

(Illustrative, not a documented incident.) **The `orders` table that ate the disk.** The `shop-api` team gets a 02:10 page: the database volume is at 91% and growing 3 GB per hour. `orders` has 6 million rows and is 55 GB. A quick `pgstattuple` sample shows 70% dead tuples. Autovacuum has been running on `orders` for hours but `VACUUM VERBOSE` says `dead but not yet removable` for millions of rows.

The on-call engineer queries `pg_stat_activity` and finds a session from a BI tool, `idle in transaction`, whose `xact_start` was 9 days ago. An analyst opened a notebook with a driver that wraps everything in a transaction and walked away. That one `backend_xmin` has pinned the cleanup horizon for the entire database. They `pg_terminate_backend()` the session, autovacuum frees the space within minutes, and growth stops. Because the file does not shrink, they schedule `pg_repack` on `orders` for the next maintenance window.

Follow-ups: `idle_in_transaction_session_timeout = '10min'` for all application and BI roles, a Prometheus alert on oldest transaction age (for example via `postgres_exporter`), a per-table autovacuum override for `orders`, and a read replica for analysts (with a documented `max_standby_streaming_delay` and `hot_standby_feedback` trade-off). **Lesson:** the bloat was a symptom; the cause was a single idle session, and the cure was a guard rail, not a bigger disk.

8 Common mistakes and best practices

Don't

- Leave sessions `idle in transaction`. Connection libraries with auto-begin and forgotten `COMMIT`s are the usual cause.
- Run `VACUUM FULL` on a busy production table without understanding that it blocks everything on that table.
- Disable autovacuum "because it uses I/O". Tune it (cost limits, scale factors) instead. Disabling it leads to bloat and eventually wraparound.
- Assume `DELETE` frees disk space. It only creates dead tuples.
- Ignore replication slots from decommissioned consumers; an inactive slot pins the horizon and retains WAL forever.
- Rely on the default 20% scale factor for tables with hundreds of millions of rows.

Do

- Set `idle_in_transaction_session_timeout` and sane `statement_timeout` values per role; keep application transactions short.
- Alert on oldest transaction age, `age(datfrozenxid)`, dead tuple ratio, inactive replication slots and disk growth rate.
- Lower `autovacuum_vacuum_scale_factor` (and set `fillfactor`) per hot table; raise `autovacuum_vacuum_cost_limit` if vacuum is throttled too hard.
- Batch large updates and deletes (for example 10,000 rows at a time) so vacuum can clean between batches.
- Use `pg_repack` or partitioning with `DROP PARTITION` (see Partitioning and Sharding §13.11) instead of mass deletes on huge tables.
- Run analytics on a replica or a warehouse, not on the primary OLTP database.

9 Tasks

☐ **T1 — See the versions** *(≈25 min, beginner)*

- **Goal:** observe `xmin`, `xmax` and `ctid` change across an update.
- **Steps:**
  1. Create `accounts` as in section 6 and run `SELECT xmin, xmax, ctid, * FROM accounts;`.
  2. `UPDATE` one row three times, selecting the system columns after each update.
  3. Use `heap_page_items(get_raw_page('accounts', 0))` to list every tuple on the page.
- **Done when:** you can point at the three dead versions in the page output and explain what each `t_xmax` and `t_ctid` means.

☐ **T2 — Two sessions, two realities** *(≈25 min, beginner)*

- **Goal:** show how the isolation level decides when the snapshot is taken.
- **Steps:**
  1. Run the Session A / Session B experiment from section 6 under `REPEATABLE READ`.
  2. Repeat under `READ COMMITTED` and record what Session A reads the second time.
  3. In Session A run `SELECT pg_current_snapshot();` before and after Session B commits (under `REPEATABLE READ`).
- **Done when:** you have a two-row table (level vs. second read value) and can explain the difference in one sentence.

☐ **T3 — Measure bloat and reclaim it** *(≈35 min, intermediate)*

- **Goal:** produce, measure and fix table bloat.
- **Steps:**
  1. Create `events` with 100,000 rows and update every row twice.
  2. Record `pg_relation_size` and `pgstattuple` results; run `VACUUM`, and record the size again.
  3. Run `VACUUM FULL` and record the size a third time.
- **Done when:** you have three size measurements and can say which command made space reusable and which returned it to the OS.

☐ **T4 — Break it: pin the horizon with a long transaction** *(≈40 min, intermediate)*

- **Goal:** reproduce `dead but not yet removable` and find the culprit like an on-call engineer.
- **Steps:**
  1. In window B open `BEGIN ISOLATION LEVEL REPEATABLE READ; SELECT count(*) FROM events;` and leave it.
  2. In window A update every row, then `VACUUM (VERBOSE) events;` and capture the `not yet removable` line.

3. Find the session with the monitoring query from step 9, then end it with `SELECT pg_terminate_backend(<pid>);`.
4. Rerun `VACUUM (VERBOSE)`.

- **Done when:** the second vacuum reports `0 are dead but not yet removable` and you can name the `pid` and `backend_xmin` that caused the first.

- [ ] **T5 — Automate the guard rails** *(≈45 min, advanced)*
  - **Goal:** make the problem alert before it becomes an outage.
  - **Steps:**
    1. Set `idle_in_transaction_session_timeout = '30s'` for a test role (`ALTER ROLE demo SET ...`), then open an idle transaction as that role and watch the server terminate it.
    2. Write one SQL query that returns a single number: seconds since the oldest `xact_start`. Wrap it in a shell script that exits non-zero above a threshold.
    3. Optional: wire the query into `postgres_exporter` custom queries or a Prometheus alert rule from Prometheus and PromQL §10.2.
  - **Done when:** the idle session is killed on its own with `FATAL: terminating connection due to idle-in-transaction timeout` and your script exits non-zero while a long transaction is open.

- [ ] **T6 — Prove it: explain a bloat incident** *(≈45 min, advanced)*
  - **Goal:** teach MVCC and vacuum from a blank page.
  - **Steps:**
    1. Without the guide, reproduce "table doubled in size and VACUUM cannot clean it" on a fresh database.
    2. Write a half-page runbook: symptoms, the three queries you run (activity, slots/prepared transactions, bloat), the fixes, and prevention.
    3. Explain aloud why `DELETE` does not free disk and why a one-line idle session can bloat tables it never touched.
  - **Done when:** a teammate can follow your runbook on a fresh lab database and arrive at the same diagnosis.

10 Self-check questions

1. What do `xmin` and `xmax` mean, and why does `UPDATE` change the row's `ctid`?
2. Why can a reader never block a writer in PostgreSQL, and what is the cost of that design?

3. What does it mean when `VACUUM VERBOSE` reports "dead but not yet removable", and what are four things that cause it?
4. Why does plain `VACUUM` not shrink the table file, and what are your options when you need the space back?
5. What is transaction ID wraparound, how does PostgreSQL protect against it, and what do you monitor?
6. The default autovacuum threshold is 20% of the table. Why is that a problem for a 500-million-row table, and what do you do?

ANSWERS

1. `xmin` is the transaction that created the tuple version; `xmax` is the one that deleted or replaced it (0 if none). An `UPDATE` writes a brand-new tuple elsewhere (or at least in a new slot), so the physical address `ctid` changes.
2. Readers look at old tuple versions according to their snapshot and never take row locks, so they do not wait for writers. The cost is dead tuples that must be cleaned by VACUUM, plus extra storage and write amplification.
3. A dead tuple still might be visible to some snapshot, so cleanup is not allowed. Causes: long-running or `idle in transaction` sessions, abandoned or lagging replication slots, `hot_standby_feedback` with long replica queries, and forgotten prepared transactions.
4. Plain `VACUUM` marks space reusable inside the file, but only trailing empty pages are given back. To return space: `VACUUM FULL` (exclusive lock, rewrites), `pg_repack` / `pg_squeeze` (online), or partition and drop old partitions.
5. XIDs are 32-bit, so very old tuples would appear to be in the future. VACUUM freezes old tuples; an anti-wraparound autovacuum is forced at `autovacuum_freeze_max_age` ; as a last resort PostgreSQL refuses new write transactions. Monitor `age(datfrozenxid)` and `age(relfrozenxid)` and the health of autovacuum.
6. Autovacuum would wait for about 100 million dead rows (20% of 500M) before starting, which is huge and slow to clean. Set a much smaller per-table `autovacuum_vacuum_scale_factor` (such as 0.01 or lower) or a fixed threshold, and consider partitioning.

11 Further reading

- PostgreSQL documentation: *Concurrency Control* (the MVCC chapter), *Routine Vacuuming* and the `VACUUM` reference page. Start from https://www.postgresql.org/docs/current/.
- PostgreSQL documentation for the `pageinspect` and `pgstattuple` modules (Appendix F, Additional Supplied Modules).

- *The Internals of PostgreSQL* by Hironobu Suzuki (free online), chapters on concurrency control and vacuum.
- *PostgreSQL 14 Internals* by Egor Rogov (free PDF from Postgres Professional), the chapters on snapshots and vacuum.
- Related: Transactions and Locks §13.2, Replication §13.4, Database Costs §20.8.

13.4 Replication

Topic 4 of 15 | ~6 hours | Difficulty: Intermediate

1 What it is

Replication means keeping copies of a database on more than one server. In PostgreSQL the standard form is **physical streaming replication**: the **primary** (the only server that accepts writes) writes every change to its **WAL** (Write-Ahead Log, the sequential journal that records each change before it is applied to data files), and one or more **standbys** (also called **replicas**) connect, receive that WAL as a stream, and **replay** it to stay byte-for-byte identical. A standby can serve read-only queries (**hot standby**) while it replays.

Copies exist for two different jobs: **availability** (if the primary dies, promote a standby and carry on) and **read scaling** (spread read-only queries across replicas). They also introduce the central trade-off of distributed data: by default replication is **asynchronous**, so a replica is always slightly *behind* ("replication lag"), and if the primary disappears at the wrong moment the most recent commits may be lost. **Synchronous** replication closes that gap at the cost of latency and availability. Everything else in this topic (monitoring lag, failover tools, split-brain) is a consequence of that choice.

2 Why it matters in DevOps

- **Single point of failure:** a database on one VM or one disk is the thing that ends your company's weekend. Replication is the first layer of High Availability §0.8.
- **Read-your-writes bugs:** a user changes their profile, the next page load reads from a lagging replica, and shows the old value. This is the most common application bug caused by replication.
- **Failover is where data gets lost.** Promoting the wrong replica, promoting while the old primary is still alive (**split-brain**), or not testing the procedure are classic postmortems. Failover is also your RTO and RPO §15.6 in practice.
- **Operational traps:** an abandoned replication slot fills the primary's disk with WAL; a long query on a replica fights WAL replay; a replica that falls too far behind has to be rebuilt.

- **Replication is not a backup.** A bad `DELETE` or `DROP TABLE` replicates in milliseconds. You still need Backups and PITR §13.5.

3 Real-world system example

Amazon RDS and Aurora. RDS for PostgreSQL offers two different replication features and the difference is exactly sync vs. async. A **Multi-AZ** deployment keeps a standby in another Availability Zone that is replicated synchronously and is used for automatic failover (you cannot read from the classic Multi-AZ instance standby). **Read replicas** are asynchronous copies you can query to scale reads and can show lag in the `ReplicaLag` CloudWatch metric. See RDS and Aurora §6.10.

GitLab documents running PostgreSQL in its Linux package with **Patroni** for leader election and automatic failover, plus PgBouncer for connection routing; **Zalando** created Patroni and runs it in its Kubernetes-based Postgres Operator. Patroni stores the cluster's "who is the leader" state in a distributed configuration store (etcd, Consul, ZooKeeper or the Kubernetes API) so that only one node can hold the leader key. **CloudNativePG**, another Kubernetes operator, manages primaries and standbys directly with the Kubernetes API instead of an external store. In every case the underlying data movement is PostgreSQL's own streaming replication; the tools add *decision making* (who is primary now?) around it.

4 Diagram

Streaming replication with one synchronous and one asynchronous standby:

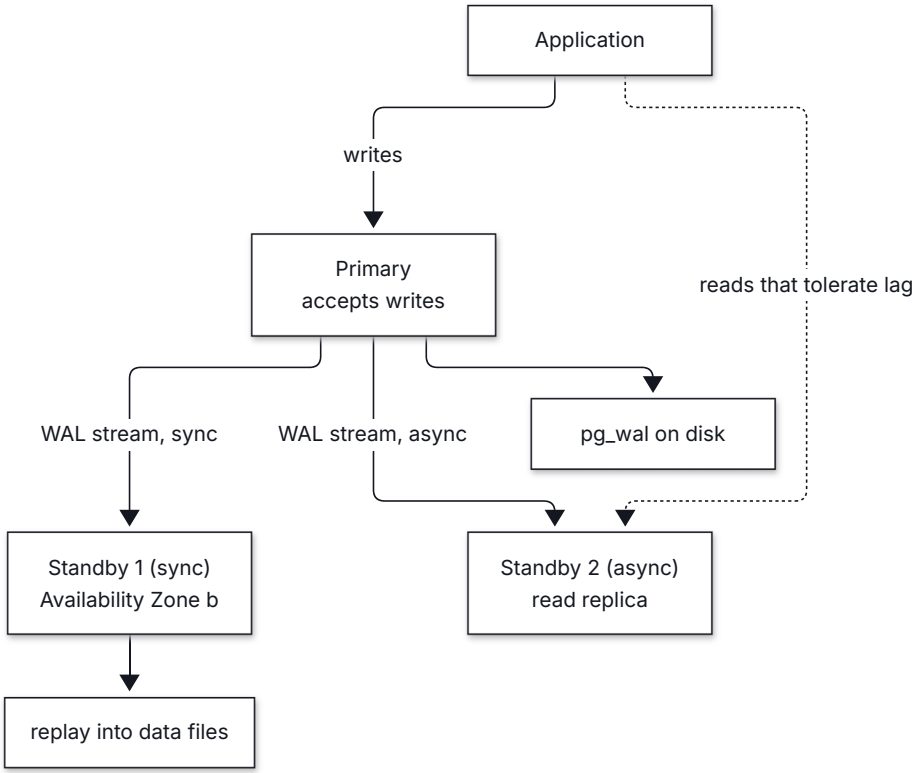


Figure 13.4-1

What happens on a commit, and where the asynchronous gap appears:

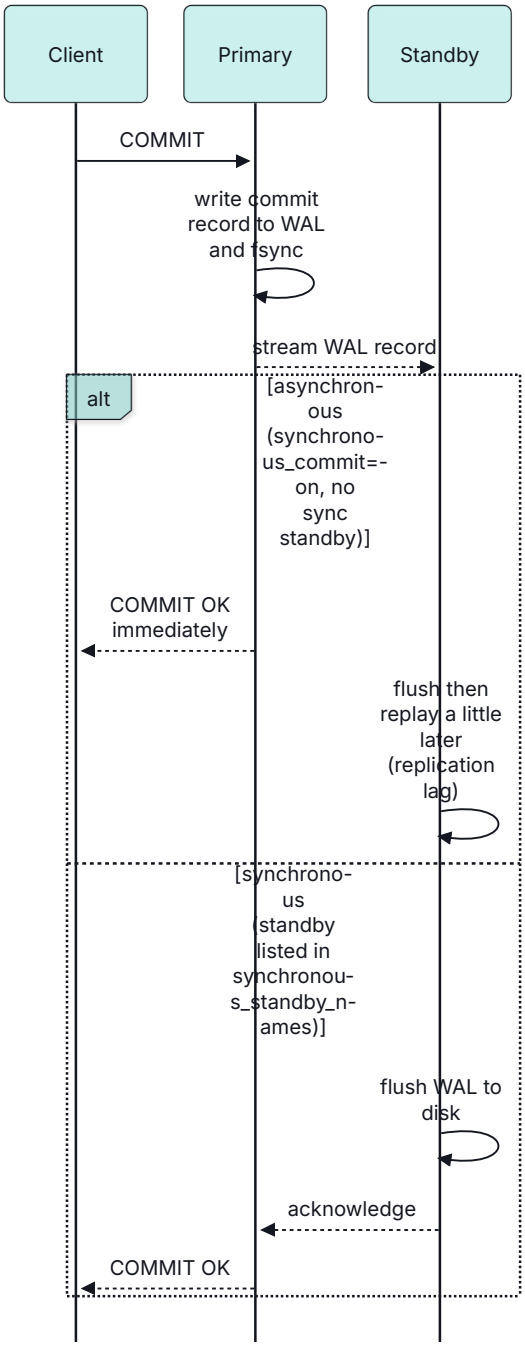


Figure 13.4-2

5 How it works

Building blocks

Piece	What it does
`wal_level = replica` (default)	WAL contains enough information to rebuild a physical copy
`max_wal_senders`	How many replication connections the primary accepts (standbys, `pg_basebackup`, `pg_receivewal`)
`pg_basebackup`	Copies the primary's data directory to seed a standby; `-R` writes `standby.signal` and `primary_conninfo` for you
`standby.signal`	An empty file in the data directory that tells PostgreSQL to start in standby (recovery) mode
`primary_conninfo`	Connection string the standby uses to reach the primary (`host=... user=replicator ...`)
Replication slot	Primary-side bookmark that remembers how much WAL a standby still needs, so WAL is not recycled too early
`hot_standby = on` (default)	Allow read-only queries on a standby
`pg_hba.conf` entry with database `replication`	Authorizes the replication role. The word `all` does *not* match replication connections

Synchronous vs. asynchronous

	Asynchronous (default)	Synchronous
Setting	`synchronous_standby_names = ''`	`synchronous_standby_names = 'FIRST 1 (replica1)'` or `'ANY 1 (a, b)'` (quorum)
Commit returns after	primary's own WAL flush	WAL is also confirmed by the standby
Data loss on primary crash (RPO)	possibly the last moments of commits	zero for acknowledged commits, as long as the sync standby survives
Commit latency	lowest	+ network round trip to the standby
If the standby goes away	nothing happens	**commits block** until it returns or you change the setting

`synchronous_commit` chooses *how far* the standby must get before the ack: `remote_write` (OS cache), `on` (flushed to disk, the default level when a sync standby is configured), `remote_apply` (replayed and therefore visible to reads on the standby; this is the setting for "read-your-writes" from replicas). `off` and `local` loosen guarantees on a per-transaction basis. Quorum commit (`ANY 2 (a, b, c)`) tolerates one slow or dead standby without stalling; a single required synchronous standby means your availability is now bounded by that one machine.

Replication lag

Lag has three places to hide, visible in `pg_stat_replication` on the primary:

- **write_lag / flush_lag / replay_lag** (time) and `sent_lsn`, `write_lsn`, `flush_lsn`, `replay_lsn` (positions). The byte gap is `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`.
- On the standby, `pg_last_wal_receive_lsn()` versus `pg_last_wal_replay_lsn()` (received but not yet applied), and `now() - pg_last_xact_replay_timestamp()` (but that value looks huge on an idle primary because there is nothing new to replay, so alert on bytes or on `replay_lag` rather than on the timestamp alone).

Typical causes: network bandwidth, a standby with slower disks than the primary, a large transaction or bulk load, and **recovery conflicts** where replay waits for a long-running query on the standby (`max_standby_streaming_delay`, default 30 seconds). The alternative, `hot_standby_feedback = on`, tells the primary not to vacuum rows that standby queries still need, which prevents cancellations but can bloat the primary (see MVCC §13.3).

Replication slots: protection and danger

A slot guarantees the primary keeps WAL until the standby has consumed it, so a replica that is offline for an hour can catch up. But an *inactive* slot (a replica you decommissioned) retains WAL forever and fills the primary's disk. Set `max_slot_wal_keep_size` as a safety limit (the slot is invalidated instead of the disk filling) and alert on `pg_replication_slots` where `active = false`.

Failover

Failover = promote a standby to be the new primary (`pg_ctl promote` or `SELECT pg_promote();`). It starts a new **timeline** (WAL file names change from `00000001...` to `00000002...`). The hard parts are not the command:

1. **Decide** that the primary is really dead (not just unreachable from your monitor).
2. **Fence** the old primary so it cannot accept writes (STONITH, "shoot the other node in the head"; revoke its network or kill the instance). Two primaries accepting writes is **split-brain**, and the data diverges.
3. **Choose the best standby** (the one with the highest replayed LSN, unless you have synchronous replication).
4. **Redirect clients** (virtual IP, DNS, HAProxy, PgBouncer, or the cloud's endpoint).
5. **Rebuild or rewind the old primary** as a standby: `pg_rewind` can resync a diverged old primary quickly, but requires `wal_log_hints = on` or data checksums enabled beforehand; otherwise use a fresh `pg_basebackup`.

Tool	Approach
Patroni	Python daemon per node; leader election via a DCS (etcd, Consul, ZooKeeper, Kubernetes); manages `pg_ctl`, config and replication; REST API for HAProxy health checks
repmgr	Cluster management and automatic failover daemon (`repmgrd`) using PostgreSQL itself for state; witness node to avoid split-brain
pg_auto_failover	A "monitor" node coordinates a primary and standby(s) with a state machine
Kubernetes operators (CloudNativePG, Zalando postgres-operator, Crunchy)	Primary/standby pods, automated failover via the Kubernetes API
Managed services (RDS/Aurora, Cloud SQL, Azure)	Provider does detection, promotion and endpoint switch; you configure Multi-AZ / HA
Manual runbook	`pg_promote()` plus a human; fine for small systems if rehearsed

Physical replication copies the *entire cluster* and requires the same major version. **Logical replication** (publications and subscriptions) replicates selected tables row by row, works across major versions and is used for zero-downtime upgrades and data integration; it is a different feature, not a substitute for HA standbys.

6 Step-by-step guide

Prerequisites: Docker. Cost: free. Time: ~60 minutes. The `psql` output shown was captured on a PostgreSQL 16.15 primary and standby set up with the same settings on a Linux VM using `pg_basebackup`; the Docker wrapper commands are standard but your container IDs, LSNs and timings will differ ("similar to"). Passwords are placeholders.

1. **Create a network and start the primary.**

```bash
docker network create pgnet
docker run -d --name pg-primary --network pgnet \
  -e POSTGRES_PASSWORD=<YOUR_PASSWORD> \
  postgres:16 \
  -c wal_level=replica -c max_wal_senders=10 -c max_replication_slots=10 -c
  wal_log_hints=on
docker exec pg-primary pg_isready -U postgres
```

   Expect `/var/run/postgresql:5432 - accepting connections` (it may take a few seconds on first start). `wal_log_hints=on` is what lets you use `pg_rewind` later.

2. **Create the replication role and allow it in `pg_hba.conf`.**

```bash
docker exec pg-primary psql -U postgres -c "CREATE ROLE replicator WITH REPLICATION
  LOGIN PASSWORD '<REPL_PASSWORD>';"
docker exec pg-primary bash -c "echo 'host replication replicator all scram-sha-256'
  >> /var/lib/postgresql/data/pg_hba.conf"
docker exec pg-primary psql -U postgres -c "SELECT pg_reload_conf();"
```

3. **Create some data to replicate.**

```bash
docker exec pg-primary psql -U postgres -c "CREATE TABLE orders (id serial PRIMARY
  KEY, item text, created timestamptz DEFAULT now());" \
  -c "INSERT INTO orders (item) VALUES ('widget');"
```

4. **Seed the standby with `pg_basebackup`** into a new volume (a throwaway container runs the copy as the `postgres` user), then start it.

```bash
docker volume create pg-replica-data
docker run --rm --network pgnet --user postgres \
  -e PGPASSWORD=<REPL_PASSWORD> \
  -v pg-replica-data:/var/lib/postgresql/data \
  postgres:16 \
  pg_basebackup -h pg-primary -U replicator -D /var/lib/postgresql/data -R -C -S
  replica1 -X stream -c fast -P -v
docker run -d --name pg-replica --network pgnet -v pg-replica-
  data:/var/lib/postgresql/data postgres:16
```

`-R` wrote `standby.signal` and `primary_conninfo`; `-C -S replica1` created a replication slot called `replica1`; `-c fast` forces an immediate checkpoint (without it the copy can wait minutes for a spread checkpoint). The tail of the output looks like:

```
pg_basebackup: base backup completed
```

5. **Confirm streaming from the primary's side.**

```bash
docker exec pg-primary psql -U postgres -x -c "SELECT application_name, client_addr,
  state, sync_state, replay_lag FROM pg_stat_replication;"
```

```
application_name | replica1
client_addr      | 172.18.0.3
state            | streaming
sync_state       | async
replay_lag       | 00:00:00.000143
```

`state = streaming` and `sync_state = async` are what you want at this stage.

6. **Read from the standby, and prove it refuses writes.**

```bash
docker exec pg-replica psql -U postgres -c "SELECT pg_is_in_recovery();" -c "SELECT *
  FROM orders;"
docker exec pg-replica psql -U postgres -c "INSERT INTO orders (item) VALUES ('x');"
```

```
 pg_is_in_recovery
-------------------
 t
ERROR:  cannot execute INSERT in a read-only transaction
```

7. **Create lag on purpose.** Pause replay on the standby, write on the primary, and measure the gap.

```bash
docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_pause();"
docker exec pg-primary psql -U postgres -c "INSERT INTO orders (item) SELECT 'bulk'
  FROM generate_series(1,5);"
docker exec pg-primary psql -U postgres -c "SELECT application_name,
  pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_gap FROM
  pg_stat_replication;"
docker exec pg-replica psql -U postgres -c "SELECT count(*) FROM orders;" \
  -c "SELECT pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn();" \
  -c "SELECT now() - pg_last_xact_replay_timestamp() AS apparent_lag;"
```

The primary shows a few hundred bytes to a few KB of replay gap; the standby still returns `count = 1` while `pg_last_wal_receive_lsn()` is ahead of `pg_last_wal_replay_lsn()`: WAL has *arrived* but is not *applied*. In a real run the `apparent_lag` interval kept growing (about 4 seconds after a few seconds' pause). Resume and watch it catch up:

```bash
docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_resume();"
docker exec pg-replica psql -U postgres -c "SELECT count(*) FROM orders;"      # 6
```

8. **Switch to synchronous replication and see the cost.**

```bash
docker exec pg-primary psql -U postgres -c "ALTER SYSTEM SET synchronous_standby_names
  = 'FIRST 1 (replica1)';" -c "SELECT pg_reload_conf();"
docker exec pg-primary psql -U postgres -c "SELECT application_name, sync_state FROM
  pg_stat_replication;"   # sync
```

Now stop the standby and try to commit (the `timeout` ends the stuck client after 6 seconds):

```bash
docker stop pg-replica
timeout 6 docker exec pg-primary psql -U postgres -c "INSERT INTO orders (item) VALUES
  ('hang');" ; echo "exit=$?"
docker exec pg-primary psql -U postgres -c "SELECT pid, wait_event, left(query,40)
  FROM pg_stat_activity WHERE wait_event = 'SyncRep';"
```

```
exit=124
 pid  | wait_event |                 left
------+------------+-----------------------------------------
 ...  | SyncRep    | INSERT INTO orders (item) VALUES ('hang'
```

The commit is **blocked waiting for the standby** (`wait_event = SyncRep`). Start the standby again (`docker start pg-replica`) and the waiting commit completes. When you are done, restore async mode:

```bash
docker exec pg-primary psql -U postgres -c "ALTER SYSTEM RESET
  synchronous_standby_names;" -c "SELECT pg_reload_conf();"
```

This is why one required synchronous standby lowers availability, and why production setups prefer a quorum such as `ANY 1 (a, b)`.

9. **Inspect and monitor the slot.**

```bash
docker exec pg-primary psql -U postgres -c "SELECT slot_name, active, wal_status,
  pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
  FROM pg_replication_slots;"
```

With the standby stopped, `active = f` and `retained_wal` grows as you write. That is the disk-fill risk.

10. **Fail over by hand.**

```bash
docker stop pg-primary                                    # the "outage"
docker exec pg-replica psql -U postgres -c "SELECT pg_promote();"
docker exec pg-replica psql -U postgres -c "SELECT pg_is_in_recovery();" \
  -c "INSERT INTO orders (item) VALUES ('after-promote');" -c "SELECT
  pg_walfile_name(pg_current_wal_lsn());"
```

```
 pg_is_in_recovery
-------------------
 f
INSERT 0 1
      pg_walfile_name
--------------------------
 00000002000000000000000B
```

The standby is now writable and has started **timeline 2** (the WAL file name begins `00000002`). Do **not** simply start `pg-primary` again, because it would accept writes as a second primary (split-brain). Rebuild it as a standby of the new primary with a fresh `pg_basebackup` (or `pg_rewind` if you enabled `wal_log_hints`).

11. **Clean up.**

```bash
docker rm -f pg-primary pg-replica
docker volume rm pg-replica-data
docker network rm pgnet
```

7 Real-world scenario

(Illustrative, not a documented incident.) **"The profile page shows the old email."** After `shop-api` adds a read replica, support tickets arrive: customers change their address and the confirmation page still shows the old one. The team's dashboard shows replica lag normally under 50 ms, so they first suspect the cache. Tracing a request shows the page `POST` goes to the primary, the redirect `GET` goes to the replica, and for batch-import minutes the replica lag spikes to 20 seconds because the import's large transaction replays slowly.

Fix, in order: route reads that follow a write in the same session to the primary for a few seconds (a "sticky" window), mark the few read-your-writes endpoints to use the primary only, and alert when `replay_lag` exceeds 5 seconds. Later the same month the primary host dies; Patroni promotes the standby in about half a minute, but a decommissioned replica's slot from an earlier experiment had retained weeks of WAL and the disk had already been at

90%. They add `max_slot_wal_keep_size`, an alert on inactive slots, and a quarterly failover game day. **Lesson:** replication adds a consistency dimension to the application (reads may be stale) and an operations dimension (slots, lag, failover rehearsal).

8 Common mistakes and best practices

Don't

- Treat replicas as backups; a destructive statement replicates within moments.
- Send read-your-writes traffic to an async replica without a strategy.
- Require exactly one synchronous standby on a two-node cluster and expect high availability; its failure freezes writes.
- Restart a failed old primary without fencing it or rebuilding it.
- Leave unused replication slots; they fill the primary's disk.
- Fail over because a monitor timed out once; flapping failovers cause more outages than they prevent.

Do

- Use `pg_basebackup -R` (or your operator) to build standbys, a replication slot (or `wal_keep_size`/WAL archive) so a standby can catch up, and a dedicated `replication` role with a strong password and TLS.
- Monitor bytes and time of replay lag, slot state and retained WAL, and `pg_stat_replication` row count (is the standby connected at all?).
- Put at least one standby in a different failure domain (Availability Zone), and consider a quorum for synchronous commit.
- Use an automated failover manager with a quorum-based store (Patroni and etcd, a cloud-managed service, or an operator) rather than a script, and **rehearse** failover on a schedule.
- Route clients through an endpoint that follows the primary (HAProxy with Patroni health checks, PgBouncer, or the cloud writer endpoint). See Database Connection Pooling §13.6.
- Keep backups and WAL archiving §13.5 in addition to replication.

9 Tasks

- [ ] **T1 — Build a primary and a streaming standby** *(≈45 min, beginner)*
  - **Goal:** have a working primary/standby pair.
  - **Steps:**
    1. Follow steps 1 to 6 in section 6.

2. Insert three rows on the primary and read them from the standby.
3. Show that writes to the standby fail.

- **Done when:** `pg_stat_replication` shows `state = streaming` and the standby returns your three rows but rejects an `INSERT` with `cannot execute INSERT in a read-only transaction`.

- [ ] **T2 — Measure lag** *(≈30 min, intermediate)*
  - **Goal:** produce and measure replication lag in both bytes and time.
  - **Steps:**
    1. Pause replay on the standby with `pg_wal_replay_pause()`.
    2. Generate load on the primary: `INSERT INTO orders (item) SELECT repeat('x', 200) FROM generate_series(1, 200000);`.
    3. Record `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)` on the primary and the receive-versus-replay LSNs on the standby.
    4. Resume replay and record how long it takes to reach zero lag.
  - **Done when:** you have a table of (bytes of lag, seconds to catch up) and can explain why `now() - pg_last_xact_replay_timestamp()` is misleading on an idle primary.

- [ ] **T3 — Break it: synchronous commit stall** *(≈30 min, intermediate)*
  - **Goal:** feel the availability cost of synchronous replication.
  - **Steps:**
    1. Set `synchronous_standby_names = 'FIRST 1 (replica1)'` and confirm `sync_state = sync`.
    2. Stop the standby and run an `INSERT`; observe `SyncRep` in `pg_stat_activity`.
    3. Start a second standby (`replica2`) and change the setting to `ANY 1 (replica1, replica2)`; stop `replica1` and show that commits proceed.
  - **Done when:** you demonstrate a commit that hangs with one required sync standby and succeeds with a quorum of two.

- [ ] **T4 — Break it: slot that eats the disk** *(≈30 min, intermediate)*
  - **Goal:** see WAL retention from an inactive slot and put a guard in place.
  - **Steps:**
    1. Stop the standby and write ~200 MB of changes on the primary.
    2. Show the growth of retained WAL with the slot query from step 9 and `du -sh` on `pg_wal`.
    3. Set `max_slot_wal_keep_size = '100MB'`, reload, write more, and watch `wal_status` become `unreserved` or `lost`.

4. Drop the unneeded slot with `pg_drop_replication_slot('replica1')` (or rebuild the standby).

- **Done when:** you have seen `wal_status` change and can state when a standby with a `lost` slot must be rebuilt.

- [ ] **T5 — Fail over and rebuild** *(≈60 min, advanced)*
  - **Goal:** run a complete failover drill including the old primary's return.
  - **Steps:**
    1. Do step 10: stop the primary and promote the standby; write a row on the new primary.
    2. Rebuild the old primary as a standby of the new one using a fresh `pg_basebackup` into a new volume (or `pg_rewind` if you set `wal_log_hints=on` and cleanly shut it down).
    3. Confirm the new standby has the row written after promotion.
    4. Write down the sequence as a numbered runbook: detect, fence, promote, re-direct, rebuild.
  - **Done when:** `pg_stat_replication` on the new primary lists the rebuilt node as `streaming` and your runbook has all five phases.

- [ ] **T6 — Prove it: compare failover options** *(≈45 min, advanced)*
  - **Goal:** explain how production systems automate what you did by hand.
  - **Steps:**
    1. Without the guide, repeat T1 and T5 on a fresh pair of containers.
    2. Read the Patroni or CloudNativePG documentation overview and write half a page: what decides the leader, how split-brain is prevented, how clients find the primary.
    3. State the RPO and RTO of your manual drill versus an automated setup.
  - **Done when:** a teammate can name, from your write-up, why async failover can lose data, what fencing is, and which component in Patroni or the operator prevents two primaries.

10 Self-check questions

1. What does the primary send to a standby, and why must the standby be the same major version?
2. Explain synchronous vs. asynchronous replication and the RPO and availability trade-off of each.

3. A user saves a change and immediately sees old data. What is the probable cause and what are two application-level fixes?
4. What is split-brain and which steps of a failover procedure prevent it?
5. Why can a replication slot fill a primary's disk, and how do you limit that risk?
6. Why is a replica not a backup?

ANSWERS

1. WAL records describing physical block changes; replay applies them to the exact on-disk format, which differs across major versions.
2. Async: commit returns after the primary's local WAL flush, so a primary crash can lose the last commits (non-zero RPO) but commits never wait for a standby. Sync: commit waits for the standby's acknowledgement, giving zero loss of acknowledged commits at the price of latency, and commits block if the required standby is unavailable (use quorum `ANY n (...)` to mitigate).
3. The read hit an async replica that had not replayed the write yet. Fixes: send reads right after a write (or critical reads) to the primary, use a sticky window or session token, or use `synchronous_commit = remote_apply` for those transactions.
4. Two nodes both accepting writes, creating diverged data. Prevent by deciding with a quorum (a DCS or witness), fencing or shutting down the old primary before promotion, and rebuilding it as a standby rather than restarting it as primary.
5. A slot makes the primary retain all WAL the consumer has not yet confirmed; an inactive consumer means unbounded retention. Set `max_slot_wal_keep_size`, alert on inactive slots and retained bytes, and drop unused slots.
6. Replication copies every change, including `DROP TABLE` and bad `DELETE`s, almost immediately. A backup plus WAL archive lets you go back to a point before the mistake.

11 Further reading

- PostgreSQL documentation: *High Availability, Load Balancing, and Replication* (log-shipping standby servers, streaming replication, synchronous replication, hot standby) and the `pg_basebackup` and `pg_rewind` reference pages. Start from https://www.postgresql.org/docs/current/.
- Patroni documentation (readthedocs) and the CloudNativePG documentation for operator-based failover.
- *PostgreSQL 14 Internals* by Egor Rogov and *The Internals of PostgreSQL* by Hironobu Suzuki (free online), chapters on replication.
- *Designing Data-Intensive Applications* by Martin Kleppmann, chapter 5 "Replication".

- Related: MVCC §13.3, Backups and PITR §13.5, Replication in disaster recovery §15.2, Failover §15.5.

13.5 Backups and Point-in-Time Recovery

Topic 5 of 15 | ~6 hours | Difficulty: Intermediate

1 What it is

A database **backup** is a copy of your data that lets you rebuild the database after something destroys it: a dropped table, a bad deploy, a corrupted disk, a deleted cloud account. PostgreSQL has two fundamentally different kinds:

- **Logical backup** (`pg_dump`, `pg_dumpall`): reads the data through SQL and writes `CREATE TABLE` / `COPY` statements (or a compressed archive of them). It is portable across PostgreSQL versions and lets you restore a single table, but it is a snapshot of *one moment* and restoring means re-loading and re-indexing every row, which gets slow as the database grows.
- **Physical backup** (a **base backup** plus archived **WAL**): copies the data directory files themselves, then continuously ships the **WAL** (Write-Ahead Log, the journal of every change) to separate storage. Restoring means copying the files back and replaying WAL. Because the WAL is a complete history, you can stop the replay at any chosen moment, which is **Point-in-Time Recovery (PITR)**: "give me the database as it was at 14:31:59, one second before the bad `DELETE`."

Two numbers frame every backup design. **RPO** (Recovery Point Objective) is how much data you can afford to lose; **RTO** (Recovery Time Objective) is how long you can afford to be down. A nightly `pg_dump` has an RPO of up to 24 hours. Base backup plus continuous WAL archiving has an RPO of seconds to minutes. See RPO §15.6 and RTO §15.7.

2 Why it matters in DevOps

- **Replication is not a backup.** A mistaken `DROP TABLE` replicates to every standby within milliseconds (see Replication §13.4). Only a backup with history lets you go back in time.
- **An untested backup is a hope, not a backup.** The classic failure is discovering during the incident that the backup job has been failing for months, that the files are empty, or that nobody knows the restore procedure. The only proof is a **restore drill**.

- **Silent failure modes:** a broken `archive_command` makes the primary keep every WAL file until the disk fills and the database stops. A backup in the same account/region/disk as the database dies with it.
- **Time pressure:** during an outage, restore *speed* is your RTO. A 2 TB `pg_dump` restore can take many hours of loading and index building; a physical restore is bounded mostly by disk and network throughput plus WAL replay. Know the number before you need it.
- **Compliance and security:** retention periods, encryption of backups at rest, and who can delete them are audit questions. A backup that the compromised production credentials can delete is not protection against ransomware.

3 Real-world system example

Amazon RDS for PostgreSQL documents exactly the model in this topic. Automated backups take a daily snapshot of the instance's storage and also capture transaction logs continuously. You choose a retention period (up to 35 days) and can restore to any point within it; RDS always restores into a **new** DB instance rather than rewinding the existing one. That is PITR as a managed feature: base backup plus continuous WAL. See RDS and Aurora §6.10.

GitLab's January 2017 database incident is one of the best-documented public backup postmortems. An engineer removed the data directory on the wrong database server while repairing replication. GitLab then found that several of their backup mechanisms had not been working as assumed (for example the scheduled `pg_dump` had been failing, and failure notification emails were not arriving), so they recovered from an older staging-related copy and lost several hours of data. They published a candid write-up and a lesson that is now standard advice: *monitor that backups succeed and regularly restore them*. (Read their postmortem for the exact details; this summary is intentionally brief.)

Tools in the ecosystem: pgBackRest (parallel, incremental, compressed, encrypted backups with built-in WAL archiving and restore, supports S3/GCS/Azure and local repositories), **WAL-G** (a Go tool descended from WAL-E for shipping base backups and WAL to object storage), and **Barman**. Kubernetes operators such as CloudNativePG and Zalando's Postgres Operator wire WAL archiving and base backups to object storage for you. They all implement the same two ideas: *base backup* and *WAL archive*.

4 Diagram

How a base backup and the WAL archive combine to restore to any moment:

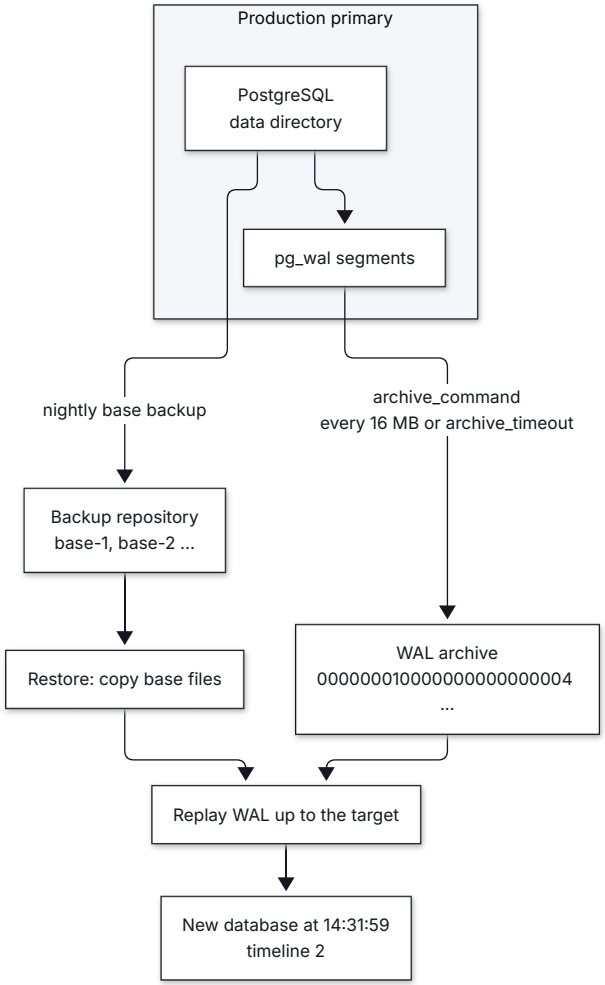


Figure 13.5-1

The timeline of an accident and what PITR does about it:

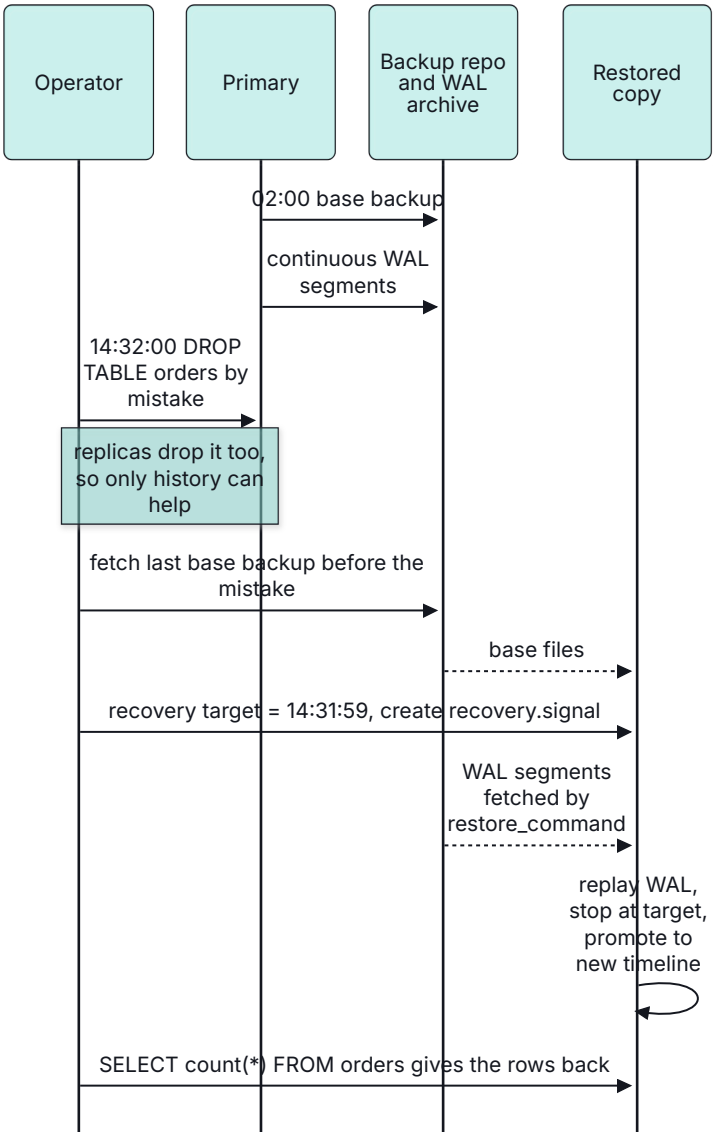


Figure 13.5-2

5 How it works

Logical vs. physical

	`pg_dump` (logical)	Base backup + WAL (physical)
What it stores	SQL / custom-format archive of objects and rows	Copy of the data files + the WAL stream
Restore to	any PostgreSQL version (usually newer), any architecture	same major version and compatible architecture
Granularity	one table, schema or database	the whole cluster (all databases)
Recovery point	the moment the dump started	any moment covered by WAL (PITR)
Restore speed	slow: re-inserts rows, rebuilds indexes	fast: file copy + WAL replay
Impact while running	long-running transaction (holds back `VACUUM` cleanup), read load	I/O load, no long transaction needed
Best for	small DBs, migrations and upgrades, extracting one table	production DR, RPO of seconds, large databases

They are complementary: physical backups for disaster recovery, `pg_dump` for portability (major-version upgrades, copying data to a developer machine).

The pieces of physical backup and PITR

Piece	Role
`wal_level = replica` (default)	WAL contains enough detail for archive recovery and replication
`archive_mode = on`	Turn on WAL archiving (changing it needs a restart)
`archive_command`	Shell command run for every completed WAL segment, `%p` = path, `%f` = file name. Must return non-zero on failure and must **never overwrite** an existing archived file
`archive_timeout`	Force a segment switch at least this often so a quiet database still archives (bounds your RPO)
`pg_basebackup`	Built-in tool that copies the running cluster consistently (uses the backup protocol; no downtime)
`backup_label` / `backup_manifest`	Metadata in a backup: where WAL replay must start, and checksums to verify it (`pg_verifybackup`)
`restore_command`	How the recovering server fetches an archived WAL file, e.g. `cp /archive/%f %p`
`recovery.signal`	Empty file that makes the server start in archive recovery (target recovery) mode

Piece	Role
`recovery_target_time` / `recovery_target_name` / `recovery_target_lsn` / `recovery_target_xid` / `recovery_target = 'immediate'`	Where to stop. A **named restore point** made with `pg_create_restore_point('name')` is the most precise way to mark "just before the migration"
`recovery_target_action`	`pause` (default: stop and let you inspect), `promote`, or `shutdown`
Timeline	After a PITR-promote the server starts a new timeline (`00000002.history`) so WAL from the "future" you rejected cannot be confused with the new history

Two details trip people up:

1. **Recovery is not instant.** After PostgreSQL reaches a consistent state it begins accepting read-only connections (hot standby) while it is *still replaying* toward the target. `pg_ctl -w start` returns at that point. Always poll `SELECT pg_is_in_recovery()` until it is `f` before you check the data or declare success.
2. **Recovery stops at the *first* point that matches.** A time target earlier than the end of the base backup is invalid (the backup itself must finish before the target). Choose a base backup taken *before* the target.

pgBackRest in one paragraph

pgBackRest is configured with a **stanza** (one PostgreSQL cluster) in `pgbackrest.conf` (`/etc/pgbackrest.conf` by default). You set `archive_command = 'pgbackrest --stanza=shop archive-push %p'`, run `stanza-create` and `check`, then schedule `backup --type=full|diff|incr`. Full = everything, differential = changes since the last full, incremental = changes since the last backup of any type. It restores with `pgbackrest restore` (adding `--type=time --target=...` for PITR, `--delta` to reuse files already on disk), handles retention (`repo1-retention-full`), verifies checksums, supports encryption (`repo1-cipher-type`) and object storage repositories, and can back up from a standby to keep load off the primary.

6 Step-by-step guide

Prerequisites: a Linux machine or VM with PostgreSQL 16 or newer client and server packages, and permission to run them as a non-root user (PostgreSQL refuses to run as root). `sudo -u postgres` on Debian/Ubuntu works. Everything below runs on one machine in a throwaway directory `/var/tmp/pitrlab`; no cloud cost. Output is from PostgreSQL 16 and is "similar to" what you will see.

1. **Create a lab cluster with WAL archiving on.**

```bash
sudo mkdir -p /var/tmp/pitrlab/archive /var/tmp/pitrlab/backups && sudo chown -R
  postgres /var/tmp/pitrlab
B=$(ls -d /usr/lib/postgresql/*/bin | tail -1)      # Debian/Ubuntu path to the
  PostgreSQL binaries
sudo -u postgres $B/initdb -D /var/tmp/pitrlab/data
sudo -u postgres tee -a /var/tmp/pitrlab/data/postgresql.conf >/dev/null <<'EOF'
port = 5455
unix_socket_directories = '/var/tmp/pitrlab'
listen_addresses = ''
archive_mode = on
archive_command = 'test ! -f /var/tmp/pitrlab/archive/%f && cp %p
  /var/tmp/pitrlab/archive/%f'
EOF
sudo -u postgres $B/pg_ctl -D /var/tmp/pitrlab/data -l /var/tmp/pitrlab/log -w start
alias lab='sudo -u postgres psql -X -h /var/tmp/pitrlab -p 5455'
```

   Expect `server started`. The `test ! -f ... && cp` pattern refuses to overwrite an existing archive file, as the PostgreSQL documentation recommends.

2. **Load some data, then take a base backup.**

```bash
lab -c "create table orders(id serial primary key, total numeric)"
lab -c "insert into orders(total) select random()*100 from generate_series(1,1000)"
sudo -u postgres $B/pg_basebackup -h /var/tmp/pitrlab -p 5455 -D
  /var/tmp/pitrlab/backups/base1 -Ft -z -X none -c fast -P
```

   `-Ft -z` writes a gzip-compressed tar, `-X none` omits WAL because we archive it separately, `-c fast` forces an immediate checkpoint. Expect `NOTICE: all required WAL segments have been archived` and a `base.tar.gz` plus `backup_manifest` in `backups/base1`.

3. **Make more changes, then mark the safe moment.**

```bash
lab -c "insert into orders(total) select 5 from generate_series(1,500)"
lab -tc "select pg_create_restore_point('before_oops')"
```

   The row count is now 1500 and the restore point `before_oops` is recorded in the WAL.

4. **Cause the disaster, and force the WAL out to the archive.**

```bash
lab -c "drop table orders"
lab -c "select pg_switch_wal()"      # close the current segment so it is archived now
lab -tc "select archived_count, failed_count, last_archived_wal from pg_stat_archiver"
```

`failed_count` should be `0`. (In production the segment fills or `archive_timeout` fires on its own.)

5. **Restore the base backup to a new directory and point it at the archive.**

```bash
R=/var/tmp/pitrlab/restored
sudo -u postgres mkdir -m 700 $R
sudo -u postgres tar -xzf /var/tmp/pitrlab/backups/base1/base.tar.gz -C $R
sudo -u postgres tee -a $R/postgresql.conf >/dev/null <<'EOF'
port = 5456
restore_command = 'cp /var/tmp/pitrlab/archive/%f %p'
recovery_target_name = 'before_oops'
recovery_target_action = 'promote'
archive_mode = off
EOF
sudo -u postgres touch $R/recovery.signal
```

We switch `archive_mode` off on the copy so it does not try to push WAL into the production archive.

6. **Start it, then wait for recovery to finish.**

```bash
sudo -u postgres $B/pg_ctl -D $R -l /var/tmp/pitrlab/restore.log -w start
until [ "$(sudo -u postgres psql -X -h /var/tmp/pitrlab -p 5456 -tAc 'select
  pg_is_in_recovery()')" = f ]; do sleep 1; done
sudo -u postgres psql -X -h /var/tmp/pitrlab -p 5456 -tAc "select count(*) from
  orders"
grep -E "recovery stopping|archive recovery complete" /var/tmp/pitrlab/restore.log
```

Expect `1500`, and log lines similar to `recovery stopping at restore point "before_oops"` and `archive recovery complete`. The dropped table is back, and `pg_control_checkpoint()` now reports `timeline_id = 2`. If you query too early you will see `1000` rows (the base backup state) and `pg_is_in_recovery() = t`: that is the "recovery is not instant" trap from section 5.

7. **Verify a plain-format backup with the built-in checker.**

```bash
sudo -u postgres $B/pg_basebackup -h /var/tmp/pitrlab -p 5455 -D
  /var/tmp/pitrlab/backups/plain1 -X stream -c fast
sudo -u postgres $B/pg_verifybackup /var/tmp/pitrlab/backups/plain1
```

   Expect `backup successfully verified`. (`pg_verifybackup` needs the plain format, not tar.)

8. **Same thing with pgBackRest.** Install it (`sudo apt-get install pgbackrest` on Debian/Ubuntu) and create a config for the lab:

```bash
sudo mkdir -p /var/tmp/pitrlab/repo /var/tmp/pitrlab/pgbr-log && sudo chown postgres
  /var/tmp/pitrlab/repo /var/tmp/pitrlab/pgbr-log
sudo -u postgres tee /var/tmp/pitrlab/pgbackrest.conf >/dev/null <<'EOF'
[global]
repo1-path=/var/tmp/pitrlab/repo
repo1-retention-full=2
log-path=/var/tmp/pitrlab/pgbr-log
start-fast=y

[shop]
pg1-path=/var/tmp/pitrlab/data
pg1-port=5455
pg1-socket-path=/var/tmp/pitrlab
EOF
C=/var/tmp/pitrlab/pgbackrest.conf
lab -c "alter system set archive_command = 'pgbackrest --config=$C --stanza=shop
  archive-push %p'"
lab -c "select pg_reload_conf()"
sudo -u postgres pgbackrest --config=$C --stanza=shop stanza-create
sudo -u postgres pgbackrest --config=$C --stanza=shop check
sudo -u postgres pgbackrest --config=$C --stanza=shop --type=full backup
sudo -u postgres pgbackrest --config=$C info
```

   `info` should show `status: ok`, the WAL archive range and the full backup with its size in the repository (smaller than the database size because it is compressed). Note the explicit `--config=` inside `archive_command`: the PostgreSQL server process does not inherit your shell's environment, so a config in a non-default location must be named in the command itself. Production setups just use `/etc/pgbackrest.conf`.

9. **PITR with pgBackRest.** Note a time, make a mistake, restore to just before it:

```bash
T=$(lab -tAc "select now()::text")
lab -c "create table payments(id serial, amt int)" -c "insert into payments(amt)
  select 1 from generate_series(1,100)"
lab -c "select pg_switch_wal()" ; sleep 3
sudo -u postgres pgbackrest --config=$C --stanza=shop --pg1-path=/var/tmp/pitrlab/pgr
  \
     --type=time "--target=$T" --target-action=promote restore
sudo grep -A5 "pgBackRest" /var/tmp/pitrlab/pgr/postgresql.auto.conf
```

   The generated `restore_command` calls `pgbackrest archive-get`, `recovery_target_time` is your timestamp, and `recovery.signal` exists. Start it with a different port (`pg_ctl -D ... -o "-p 5457" start`), wait for `pg_is_in_recovery()` to be `f`, and `payments` will not exist (it was created after the target).

10. **Clean up the lab.**

```bash
sudo -u postgres $B/pg_ctl -D /var/tmp/pitrlab/data stop -m fast
sudo -u postgres $B/pg_ctl -D /var/tmp/pitrlab/restored stop -m fast 2>/dev/null
sudo -u postgres $B/pg_ctl -D /var/tmp/pitrlab/pgr stop -m fast 2>/dev/null
sudo rm -rf /var/tmp/pitrlab
```

Logical backup in one minute (for contrast):

```bash
pg_dump -Fc -f shop.dump shop            # custom format, compressed, parallel-restorable
pg_restore --list shop.dump | head       # see what is inside without restoring
createdb shop_restored && pg_restore -d shop_restored -j 4 shop.dump
pg_dump -Fd -j 4 -f shop_dir shop        # directory format allows parallel dump too
```

7 Real-world scenario

(Illustrative, not a documented incident.) The `shop-api` team ships a data-cleanup migration that was meant to delete test customers (`WHERE email LIKE '%@test.example.com'`). A last-minute edit drops the `WHERE`. The migration runs in the deploy pipeline and commits at 14:32:07. Customer service reports empty order histories at 14:41. The read replicas already show the deletion, so failing over would change nothing.

Investigation: the on-call engineer freezes writes (puts the API in maintenance mode so nothing new is lost or entangled), reads the migration log to find the exact commit time, and picks the restore target `14:32:06`. The nightly pgBackRest full is from 02:00 and WAL has

been archiving continuously. They run `pgbackrest restore --type=time --target="2026-10-09 14:32:06+00" --target-action=pause` on a **new** VM, so the broken production database is left untouched for forensics. After replay they inspect the data, see the customers back, and extract just the affected tables with `pg_dump -t customers -t orders` from the restored copy, then load them into production and re-apply the legitimate writes made since 14:32 from the application log.

Total downtime for the affected feature: about 40 minutes, mostly the base backup copy and WAL replay (the team had measured a restore in a previous drill, so they knew the estimate in advance). Follow-ups: migrations now run inside a transaction with a row-count assertion; the pipeline creates a named restore point (`pg_create_restore_point('pre_migration_<id>')`) before every destructive migration (see Database Migrations in Pipelines §5.9); and the restore is rehearsed monthly by an automated drill. **Lesson:** the value of PITR is choosing a *moment*, but the real enabler was a rehearsed procedure and a known RTO.

8 Common mistakes and best practices

Don't

- Treat replicas, RAID or "the cloud disk is redundant" as backups. Redundancy protects against hardware loss, not against your own commands.
- Rely only on nightly `pg_dump` if your RPO is smaller than a day.
- Ignore `archive_command` failures. A failing command keeps WAL on the primary until `pg_wal` fills the disk and the database stops. Alert on `pg_stat_archiver.failed_count` growth and on `last_archived_time` being old.
- Store backups on the same disk, host, account or region as the database, or let the production credentials delete them.
- Copy the data directory with `cp` / `rsync` while PostgreSQL is running and call it a backup (it is inconsistent without the backup protocol or a coordinated snapshot).
- Delete `backup_label` "because the server won't start" (it is what tells recovery where to begin), or skip `recovery.signal`.
- Say "backups are green" without ever restoring one.

Do

- Keep a physical base backup + continuous WAL archive for production, plus occasional logical dumps for portability.

- Use a purpose-built tool (pgBackRest, WAL-G, Barman, or your cloud's managed backups) rather than hand-rolled scripts; get checksums, retention, parallelism and encryption for free.
- Follow **3-2-1**: three copies, two media types, one off-site (and consider an immutable/object-locked copy; see Off-site Backups §15.8). Encrypt backups and manage the keys separately.
- **Drill the restore** on a schedule, time it, compare against your RTO, and record the run (Testing Backup Restoration §15.9). Fail the drill if row counts or a checksum query disagree.
- Create **named restore points** before risky changes; set `archive_timeout` to match your RPO; monitor backup age and size.
- Document the runbook: who decides the target, how writes are frozen, how the new primary is promoted and how replicas are rebuilt from it (restoring creates a new timeline, so old standbys must be re-seeded or rewound).

9 Tasks

- [ ] **T1 — Dump and restore one table** *(≈30 min, beginner)*
  - **Goal:** use a logical backup and understand what it is good for.
  - **Steps:**
    1. Create a database with a few tables and ~10,000 rows (`pgbench -i -s 1` is a quick source).
    2. Run `pg_dump -Fc -f db.dump <dbname>`, then `pg_restore --list db.dump`.
    3. Create a new database and restore **only** `pgbench_accounts` with `pg_restore -t pgbench_accounts -d newdb db.dump`.
  - **Done when:** `select count(*) from pgbench_accounts` in `newdb` equals the original count, and you can state two things a dump cannot do that PITR can.
- [ ] **T2 — Restore to a named restore point** *(≈60 min, intermediate)*
  - **Goal:** perform a full PITR by hand.
  - **Steps:**
    1. Follow steps 1-6 of the guide in your own lab directory.
    2. Check `pg_stat_archiver` before and after `pg_switch_wal()`.
    3. Confirm the restored server is on timeline 2 and look at the `00000002.history` file.

- **Done when:** the dropped table exists again with the exact row count from before the drop, and you can explain why the first query right after `pg_ctl start` may show older data.

- [ ] **T3 — Restore to a timestamp, not a name** *(≈45 min, intermediate)*
  - **Goal:** practice choosing a target when nobody created a restore point.
  - **Steps:**
    1. In the lab, insert a row with `now()` in a loop every second for 30 seconds (`\watch 1` in psql works) into a `heartbeat` table.
    2. `drop table heartbeat`, then restore with `recovery_target_time` set to 10 seconds before the drop (use `heartbeat` timestamps and the log to pick it).
    3. Try `recovery_target_inclusive = off`.
  - **Done when:** the restored table's last row is the last heartbeat before your chosen time, and you can state how `inclusive` changes it.

- [ ] **T4 — Break the archive and fix it** *(≈45 min, intermediate)*
  - **Goal:** see the most dangerous silent failure and how to alert on it.
  - **Steps:**
    1. In the lab, set `archive_command = '/bin/false'` and reload.
    2. Generate WAL (`pgbench -T 60`) and watch `pg_stat_archiver.failed_count` grow and `ls pg_wal | wc -l` increase; read the PostgreSQL log warnings.
    3. Fix the command, reload, and watch the backlog drain.
    4. Write a one-line monitoring query that would have alerted you (for example `failed_count` increasing, or `now() - last_archived_time > interval '5 minutes'`).
  - **Done when:** `pg_wal` shrinks back after the fix and you have a working alert query.

- [ ] **T5 — Automate a nightly backup and a restore drill with pgBackRest** *(≈2 hours, advanced)*
  - **Goal:** make backups and proof of recoverability a scheduled job.
  - **Steps:**
    1. Complete step 8 of the guide. Add `repo1-retention-full=2` and take a `diff` and `incr` backup after changing data; compare sizes in `pgbackrest info`.
    2. Write `restore-drill.sh` that: restores the latest backup into an empty scratch directory (`--pg1-path=...`) and starts it on another port, waits for `pg_is_in_recovery() = f`, runs a validation query (row counts of key tables,

`pg_amcheck` or a `select max(created)`), records elapsed seconds, stops and deletes the copy, and exits non-zero on any mismatch.
3. Schedule full backups (weekly), incrementals (daily) and the drill (weekly) with cron or a systemd timer.

- **Done when:** the script prints something like `restore OK in 14s, orders=1500` and exits 0, and exits non-zero if you deliberately delete one row from the expected count.

☐ **T6 — Prove it: lose a production table and get it back** *(≈60 min, advanced)*

- **Goal:** demonstrate end-to-end recoverability, without the guide, with a time budget.
- **Steps:**
  1. Stand up a fresh lab cluster with WAL archiving and a base backup, load `pgbench` data.
  2. Run `pgbench -T 120` in the background to simulate traffic, then drop `pgbench_history` at a time you note.
  3. Restore to just before the drop on a different port, extract the table with `pg_dump -t pgbench_history`, and load it back into the live cluster.
  4. Write down your RPO, the measured RTO, and one change you would make to improve each.
- **Done when:** `pgbench_history` exists in the live cluster with rows from before the drop, and your write-up names the timings and the target you chose.

10 Self-check questions

1. Why is a streaming replica not a backup?
2. What two things do you need, besides the server software, to restore a PostgreSQL cluster to 14:31:59 yesterday?
3. What happens to the primary if `archive_command` keeps failing?
4. After `pg_ctl start` on a PITR restore the table is missing rows. Is the restore broken?
5. When would you choose `pg_dump` over a physical backup, and when the reverse?
6. How do you know your backups work?

ANSWERS

1. It applies every change, including mistakes, within milliseconds. It protects against hardware failure, not against logical errors such as a bad `DELETE`.
2. A base backup taken before that time and the continuous WAL archive from the end of that backup through the target moment (and a `restore_command`, `recovery.signal` and a target setting to drive the replay).

3. Completed WAL segments cannot be recycled until they are archived, so `pg_wal` grows until the disk fills and the primary stops accepting writes. Alert on archiver failures and a stale `last_archived_time`.
4. Probably not. The server accepts read-only connections once it is consistent while it is still replaying WAL. Wait for `pg_is_in_recovery()` to become false (or the log to say `archive recovery complete`) before judging the data.
5. `pg_dump` for portability (upgrading major versions, moving a single database or table, small databases). Physical backup + WAL for disaster recovery, large databases, a low RPO and PITR.
6. You restore them on a schedule into a scratch environment, validate the data, measure the time and alert when the drill fails or exceeds the RTO. Green backup jobs alone prove nothing.

11 Further reading

- PostgreSQL documentation, chapter *Backup and Restore*: "SQL Dump", "File System Level Backup" and "Continuous Archiving and Point-in-Time Recovery (PITR)"; plus the `pg_basebackup`, `pg_verifybackup` and `pg_dump` reference pages.
- pgBackRest user guide (pgbackrest.org) and WAL-G documentation on GitHub.
- GitLab's public postmortem of the 31 January 2017 database incident.
- Amazon RDS User Guide, "Backing up and restoring" and "Restoring a DB instance to a specified time".
- *PostgreSQL 14 Administration Cookbook* (Riggs and Ciolli) and *The Art of PostgreSQL* (Dimitri Fontaine) for practical backup recipes.
- Related in this repo: Replication §13.4, Backups §15.1, Restore Procedures §15.4.

LAB NOTES What I ran, what I saw, what it means

13.6 Database Connection Pooling

Topic 6 of 15 | ~5 hours | Difficulty: Intermediate

1 What it is

Every client of a PostgreSQL server holds a **connection**, and each connection is a dedicated operating-system **process** (a "backend") with its own memory, started by the server when you connect. Opening one costs a TCP handshake, usually a TLS handshake, authentication and a process fork, which is milliseconds to tens of milliseconds, and keeping one open costs several megabytes of RAM even while it does nothing. The server also has a hard ceiling, `max_connections` (default 100).

Connection pooling puts a small, reusable set of already-open database connections between your application and the database. There are two layers and you usually want both:

- **In-app pool** (HikariCP in Java, `pg.Pool` in Node.js, SQLAlchemy's pool in Python): each application process keeps N connections open and lends them to request handlers.
- **External pooler** (**PgBouncer**, Pgpool-II, Odyssey, PgCat, Amazon RDS Proxy): a separate lightweight proxy that accepts thousands of client connections and multiplexes them onto a few dozen real server connections. This is what lets 50 application instances share one database without exhausting it.

The central idea: **a database is fastest with a small number of busy connections, not a large number of idle ones.**

2 Why it matters in DevOps

- **"FATAL: sorry, too many clients already"** (or `remaining connection slots are reserved for roles with the SUPERUSER attribute`) is a top-five production Postgres error. It tends to appear exactly when traffic spikes or you scale the app out, i.e. when you can least afford it.
- **Autoscaling multiplies connections.** 20 app pods x a pool of 10 = 200 connections. Kubernetes happily scales to 40 pods during a sale; the database does not scale with them. See High Availability and Scalability §0.8.

- **Serverless and short-lived processes** (AWS Lambda, cron jobs, CI jobs) open a fresh connection per invocation, which is the worst case for a process-per-connection database.
- **More connections can mean *less* throughput.** Past a point, hundreds of active backends fight for CPU, memory and locks, and latency rises for everyone. A pool that *queues* excess work beats a database that *thrashes*.
- **Operational leverage:** a pooler lets you restart or fail over the database while clients stay connected (`PAUSE` / `RESUME`), cap each service's connections, and see who is using the database. Related: Connection Pooling §12.10 in the networking level covers the general idea; this topic is the PostgreSQL-specific case.

3 Real-world system example

Amazon RDS Proxy is AWS's managed pooler for RDS and Aurora (including PostgreSQL). Its documented purpose is exactly the one in this topic: pool and share connections to reduce database load, keep connections from Lambda functions and other bursty clients from overwhelming the database, and shorten failover time by holding client connections while the database fails over. See RDS and Aurora §6.10.

GitLab ships PgBouncer as a component of its Linux package for PostgreSQL deployments and documents running it in front of PostgreSQL (with Patroni for failover), because the Rails application would otherwise open far too many connections. **Heroku Postgres** offers a connection-pooling add-on built on PgBouncer in transaction mode, and **Supabase** documents transaction-mode pooling (via its own pooler, Supavisor) for serverless and edge clients. In every one of these, the application keeps hundreds or thousands of client connections while the database sees a few dozen.

PgBouncer is the de facto standard because it is tiny: a single-threaded C process using an event loop, a few KB per client connection, and a simple text admin console.

4 Diagram

Many clients, few server connections:

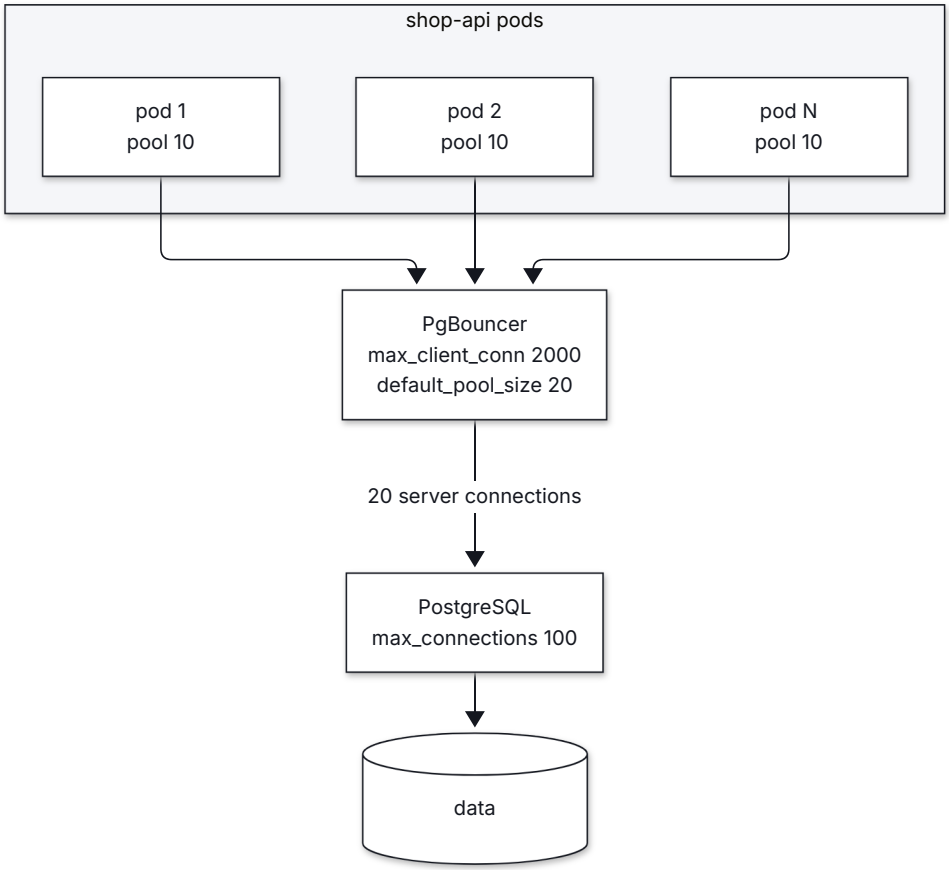


Figure 13.6-1

What transaction pooling does over time. The server connection is borrowed for one transaction and returned straight away:

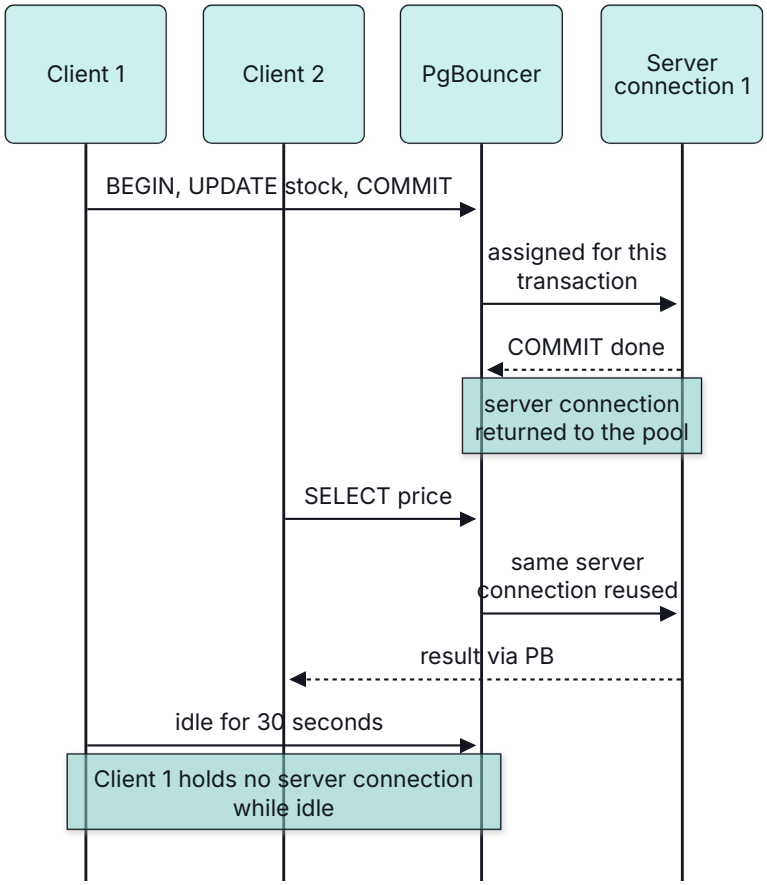


Figure 13.6-2

5 How it works

Why connections are expensive

- Each backend is a process. PostgreSQL has per-backend memory (catalog caches, plans) and can use up to `work_mem` per sort/hash node on top. Hundreds of connections multiply this.
- Every active backend takes a slot in shared structures (the process array, lock tables), and snapshot-taking scans them; very large connection counts add measurable overhead even when most are idle.

- Setup cost: TCP + TLS + authentication (SCRAM) + fork. A short-lived client that connects per query can spend more time connecting than querying.

PgBouncer pool modes

Mode	Server connection is held for	Good for	Breaks
session	the whole client connection	legacy apps that use session state; a safe default	does not reduce idle connections much; each long-lived client pins one server connection
transaction	one transaction	web/API apps and serverless: the big win	anything that relies on session state (see below)
statement	one statement	autocommit-only workloads	multi-statement transactions are not allowed

Transaction mode is the one you want for `shop-api`, and it comes with rules. Between transactions the next one may run on a *different* server connection, so session-level state can be wrong, lost, or leak to another client:

Feature	Problem in transaction mode	What to do
`SET x = ...` (session-level)	stays on the server connection and **leaks** to the next client	use `SET LOCAL` inside a transaction, or set per-role/per-database defaults in the server
SQL-level `PREPARE` / `EXECUTE`	the statement exists only on one backend: "does not exist" or "already exists" errors	use PgBouncer 1.21 or newer with `max_prepared_statements` set (it tracks protocol-level prepared statements), or disable prepared statements in the driver
Session advisory locks (`pg_advisory_lock`)	lock belongs to a server connection that gets reused	use transaction-level locks `pg_advisory_xact_lock`
`LISTEN` / `NOTIFY`	notifications need a stable session	connect that component directly to the database
`WITH HOLD` cursors, temp tables across transactions	tied to a session	keep inside one transaction

The numbers: `max_connections` and pool sizing

PostgreSQL side:

- `max_connections` (default 100) is the total number of backends. `superuser_reserved_connections` (default 3) are kept for admins, so ordinary users see a lower effective limit. Increasing it needs a restart and raises memory use.

- Replication, `pg_basebackup`, autovacuum workers and monitoring agents also count or compete for slots.

PgBouncer side (the settings you will touch):

Setting	Meaning
`pool_mode`	`session`, `transaction` or `statement`
`default_pool_size`	server connections per (database, user) pair (default 20)
`max_client_conn`	how many client connections PgBouncer accepts (default 100; raise it to thousands; limit by file descriptors)
`max_db_connections` / `max_user_connections`	hard cap on server connections per database or user across pools
`min_pool_size`, `reserve_pool_size`, `reserve_pool_timeout`	keep warm connections; allow a burst of extra connections after waiting this long
`query_wait_timeout`	how long a client may wait for a server connection before being dropped (default 120 s)
`server_idle_timeout`, `server_lifetime`	recycle server connections
`auth_type`, `auth_file`, `auth_query`	how clients authenticate (`scram-sha-256` is the modern choice; `auth_query` avoids maintaining a user file)
`server_reset_query`	cleanup run when a session-mode connection is returned (`DISCARD ALL` by default; not run in transaction mode unless configured)

Sizing math. Start from how much work the database can really do in parallel, and let everything else queue.

1. **Server-side budget.** A common starting point (from the PostgreSQL community wiki and HikariCP's pool-sizing guidance) is `connections ≈ (CPU cores x 2) + effective_spindle_count`. For an SSD-backed 8-core database that is about 16-20 *active* connections. Treat it as a starting point and load-test; more connections than this rarely add throughput for CPU-bound work.
2. **Little's law.** The number of connections busy at once is `throughput x time each request holds a connection`. `shop-api` at 1,500 requests/s where each request holds a connection for 4 ms of DB work needs about `1500 x 0.004 = 6` connections on average. Size for peaks (say 3x), so 18-20, not 200.
3. **Check the fan-out.** Without a pooler: `pods x pool_size_per_pod` must fit under `max_connections - reserved - other_clients`. `20 pods x 10 = 200 > 100`, so the database refuses connections. With PgBouncer: apps may open 20 x 10 = 200 *client* connections, but `default_pool_size = 20` keeps the database at 20.

4. **Multiple pools multiply.** PgBouncer pools are per (database, user). Three services with three roles and `default_pool_size = 20` can open 3 x 20 = 60 server connections. Add `max_db_connections` so the sum stays inside the budget.
5. **Reserve headroom.** Leave 10-20 slots for admins, replication, migrations and monitoring.
6. **Make the in-app pool small too.** An app pool of 10 per pod that *waits* for a PgBouncer connection is fine. Set app-side connection and acquire timeouts shorter than your request timeout so waiting does not pile up.

The queueing behaviour is the feature: when 200 clients ask for 20 connections, 180 wait milliseconds in PgBouncer's queue instead of making Postgres thrash. Watch `cl_waiting` and `maxwait` in `SHOW POOLS`; if they stay high you are under-provisioned or have slow queries.

6 Step-by-step guide

Prerequisites: a Linux machine (or WSL2), Docker (for the database), and `sudo apt-get install pgbouncer postgresql-client` on Debian/Ubuntu. Free, local, nothing in the cloud. The outputs below were taken from PostgreSQL 16 and PgBouncer 1.22; yours will be "similar to" these.

1. **Start a PostgreSQL with a deliberately low connection limit.**

```bash
docker run -d --name pg-pool -e POSTGRES_PASSWORD=pgadmin -p 5455:5432 postgres:16 -c
  max_connections=20
sleep 5
psql "postgresql://postgres:pgadmin@127.0.0.1:5455/postgres" -c "create role shop
  login password 'shop_pw'" -c "create database shop owner shop"
```

   Expect `CREATE ROLE` and `CREATE DATABASE`. `show max_connections;` returns `20`.

2. **Create sample data and hit the limit directly.**

```bash
export PGPASSWORD=shop_pw
pgbench -h 127.0.0.1 -p 5455 -U shop -i -s 1 shop
pgbench -h 127.0.0.1 -p 5455 -U shop -c 50 -j 2 -T 5 shop
```

(`pgbench` is in the `postgresql-contrib` or `postgresql-16` packages depending on the distro.) 50 clients cannot connect to a server that allows 20, so you should see something like:

```
pgbench: error: connection to server at "127.0.0.1", port 5455 failed: FATAL:
  remaining connection slots are reserved for roles with the SUPERUSER attribute
pgbench: error: could not create connection for client 8
```

That is the outage, reproduced.

3. **Configure PgBouncer in transaction mode.**

```bash
mkdir -p ~/pgb && cd ~/pgb
cat > pgbouncer.ini <<'EOF'
[databases]
shop = host=127.0.0.1 port=5455 dbname=shop

[pgbouncer]
listen_addr = 127.0.0.1
listen_port = 6432
auth_type = scram-sha-256
auth_file = /home/YOU/pgb/userlist.txt
admin_users = shop
pool_mode = transaction
default_pool_size = 5
max_client_conn = 200
reserve_pool_size = 2
reserve_pool_timeout = 3
query_wait_timeout = 30
logfile = /home/YOU/pgb/pgbouncer.log
pidfile = /home/YOU/pgb/pgbouncer.pid
unix_socket_dir = /home/YOU/pgb
EOF
echo '"shop" "shop_pw"' > userlist.txt && chmod 600 userlist.txt
sed -i "s#/home/YOU#$HOME#g" pgbouncer.ini
pgbouncer -d pgbouncer.ini
```

`userlist.txt` holds a plaintext password only because this is a lab. In production use `auth_query` (so PgBouncer looks users up in the database) or SCRAM verifiers, and TLS between the layers. Note `admin_users = shop` also makes `shop` an admin of the pooler, which a real deployment would not do.

4. **Run the same load through the pooler.**

```bash
pgbench -h 127.0.0.1 -p 6432 -U shop -c 50 -j 2 -T 5 shop
```

Expect `number of failed transactions: 0 (0.000%)` and a `tps` in the low thousands on a laptop. 50 clients succeeded against a 20-connection database.

5. **Look inside the pooler while it works.** Run the load in the background and open the admin console (the special database `pgbouncer`):

```bash
pgbench -h 127.0.0.1 -p 6432 -U shop -c 50 -j 2 -T 20 shop > /dev/null &
sleep 3
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "SHOW POOLS" -c "SHOW STATS" -c "SHOW
  SERVERS"
psql -h 127.0.0.1 -p 5455 -U shop shop -tAc "select count(*) from pg_stat_activity
  where usename='shop' and backend_type='client backend'"
```

In `SHOW POOLS` for `shop` you should see roughly `cl_active=5`, `cl_waiting=45` and `sv_active=5`: **50 clients are served by 5 server connections** with the rest queued. The `pg_stat_activity` count (about 5, plus this query) confirms PostgreSQL sees only those. `maxwait` shows how long the longest-waiting client has waited.

6. **See the transaction-mode trap (session state leaking).**

```bash
psql -h 127.0.0.1 -p 6432 -U shop shop -c "SET statement_timeout='7s'" -c "SELECT
  pg_backend_pid()"
psql -h 127.0.0.1 -p 6432 -U shop shop -c "SHOW statement_timeout"  -c "SELECT
  pg_backend_pid()"
```

Two different client sessions, the **same backend PID** and the second one sees `7s` set by the first. Likewise `PREPARE q AS SELECT 1` from two separate clients fails the second time with `prepared statement "q" already exists`. These are the bugs people hit when they switch a legacy app to transaction mode. The fix is to use `SET LOCAL`, drop SQL-level `PREPARE`, or (PgBouncer 1.21 and newer) set `max_prepared_statements = 100` so protocol-level prepared statements from drivers are handled; check your driver's documentation.

7. **Provoke pool exhaustion on purpose.** Set `default_pool_size = 1`, `reserve_pool_size = 0` and `query_wait_timeout = 2`, then reload:

```bash
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "RELOAD"
psql -h 127.0.0.1 -p 6432 -U shop shop -c "select pg_sleep(5)" &
sleep 0.5; psql -h 127.0.0.1 -p 6432 -U shop shop -c "select 1"
```

The second client waits about two seconds and gets `FATAL: query_wait_timeout`. That error in your application logs means the pool is too small for the queue, or a few slow queries are holding every connection. Put your settings back afterwards.

8. **Graceful maintenance with PAUSE/RESUME.**

```bash
psql -h 127.0.0.1 -p 6432 -U shop pgbouncer -c "PAUSE" -c "RESUME"
```

`PAUSE` waits for in-flight transactions to finish and holds new ones; you can restart or fail over the database, then `RESUME`. Clients see latency rather than errors.

9. **Clean up.**

```bash
kill "$(cat ~/pgb/pgbouncer.pid)"
docker rm -f pg-pool
```

7 Real-world scenario

(Illustrative, not a documented incident.) `shop-api` runs 12 pods, each with a Node.js `pg.Pool` of `max: 10`, against a PostgreSQL with `max_connections = 100`. Normal traffic is fine. During a Friday flash sale the HPA scales the API to 30 pods in four minutes. Suddenly 30 x 10 = 300 potential connections exceed 100 and the error logs fill with `sorry, too many clients already`. Checkout fails; the on-call engineer sees healthy pods but a database whose CPU is also pinned at 100% by hundreds of runnable backends.

Investigation: `select count(*), state from pg_stat_activity group by state` shows 97 connections, most `idle` (pods holding pools open) and a few `active`. The database is not busy; it is crowded. The immediate fix is to cap the HPA and restart the slowest queries; the permanent fix is a PgBouncer deployment in transaction mode between the pods and the database with `default_pool_size = 20` and `max_client_conn = 2000`, plus smaller per-pod pools. They first audit the code for `SET` statements and advisory locks, find one `pg_advisory_lock` in the order-numbering code, and change it to `pg_advisory_xact_lock`.

At the next sale 40 pods open 400 client connections; PostgreSQL sees 20 server connections, CPU is healthy, and `cl_waiting` peaks at a few dozen for milliseconds. **Lesson:** the database's concurrency budget is fixed, so fan-out must be absorbed by a queue (the pooler), and "works at 12 pods" says nothing about 40. Test the transaction-mode assumptions before the incident, not during it.

8 Common mistakes and best practices

Don't

- Raise `max_connections` to 1000 as the first response. It uses more memory and usually makes latency worse; it only postpones the problem.
- Switch to transaction mode without auditing session state (`SET`, session advisory locks, `PREPARE`, `LISTEN`, temp tables).
- Let every service create its own large pool "just in case". Totals add up across pods, pools and users.
- Point migrations, `pg_dump`, logical replication or `LISTEN` consumers at the transaction pool. Give admin and batch work a direct (or session-mode) path.
- Expose PgBouncer without TLS and authentication, or store production passwords in a plaintext `userlist.txt`.
- Forget that the pooler is now a critical component: run more than one behind a load balancer (it is cheap) and monitor it.

Do

- Size pools from the database's real capacity and Little's law, then confirm with a load test; prefer a small pool plus a queue.
- Use transaction mode for web/API/serverless workloads, session mode where you need session features, and keep a direct admin connection.
- Set timeouts on both sides (`query_wait_timeout`, app acquire timeout, `statement_timeout`, `idle_in_transaction_session_timeout`) so waits do not cascade.
- Alert on `cl_waiting`, `maxwait`, pool utilization and `FATAL: ... too many clients` in logs; export PgBouncer metrics (for example with a Prometheus exporter, see Prometheus and PromQL §10.2).
- Use `auth_query` and TLS in production, and plan for failover with `PAUSE` / `RESUME` or a managed pooler such as RDS Proxy.

9 Tasks

- [ ] **T1 — Reproduce "too many clients"** *(≈25 min, beginner)*
  - **Goal:** see the failure that connection pooling prevents.
  - **Steps:**
    1. Start Postgres in Docker with `-c max_connections=20` (step 1 of the guide).
    2. Run `pgbench -c 50 -T 5` directly against it.
    3. In a second terminal during a smaller run (`-c 15`) count connections: `select count(*), state from pg_stat_activity group by state;`.

- **Done when:** you have captured the exact `FATAL` message and can explain why 15 clients worked but 50 did not (including the reserved superuser slots).

☐ **T2 — Put PgBouncer in front and watch it multiplex** *(≈40 min, beginner)*

- **Goal:** serve many clients with few server connections.
- **Steps:**
  1. Follow guide steps 3-5 with `default_pool_size = 5`.
  2. During a 50-client run record `cl_active`, `cl_waiting` and `sv_active` from `SHOW POOLS`.
  3. Compare against the `pg_stat_activity` count from Postgres' side.
- **Done when:** you have numbers showing 50 client connections mapped to about 5 server connections and zero failed pgbench transactions.

☐ **T3 — Find the best pool size by experiment** *(≈60 min, intermediate)*

- **Goal:** replace guesswork with a measured number.
- **Steps:**
  1. With `-c 50`, run pgbench for 20 s each at `default_pool_size` of 1, 2, 4, 8, 16 (edit and `RELOAD`; remember `max_connections=20`).
  2. Record `tps` and average latency for each in a table.
  3. Compare the knee of the curve to `(CPU cores x 2)` for your machine, and write down how the answer would change for a database with slower disk or longer queries.
- **Done when:** you have a table of pool size vs tps/latency and a one-paragraph conclusion naming your chosen size.

☐ **T4 — Break transaction mode and fix it** *(≈45 min, intermediate)*

- **Goal:** experience session-state bugs safely.
- **Steps:**
  1. Reproduce the `SET` leak and the `PREPARE ... already exists` error from guide step 6.
  2. Rewrite the first as `BEGIN; SET LOCAL statement_timeout='7s'; ...; COMMIT;` and prove the next client does not see it.
  3. Replace the session advisory lock with `pg_advisory_xact_lock` inside a transaction and verify it is released at commit.
- **Done when:** a second client always sees the default `statement_timeout` after your fixed version, and you can list three features that need session mode or a direct connection.

- [ ] **T5 — Provoke and alert on pool exhaustion** *(≈45 min, intermediate)*
  - **Goal:** recognise and monitor the saturation signal.
  - **Steps:**
    1. Set `default_pool_size = 1` and `query_wait_timeout = 2`, then hold the only connection with `select pg_sleep(30)`.
    2. Make another client time out with `FATAL: query_wait_timeout`.
    3. Write a small script that runs `SHOW POOLS` every 5 s and prints a warning if `cl_waiting > 0` for more than two checks or `maxwait > 1`.
  - **Done when:** your script prints a warning while the slow query runs and stays quiet when you remove it.

- [ ] **T6 — Prove it: size the pools for a scale-out** *(≈45 min, advanced)*
  - **Goal:** produce a defensible sizing plan without the guide.
  - **Steps:**
    1. Given: 8-core database, `max_connections=200` but you want at most 30 active, 3 services (`shop-api`, `worker`, `reports`) with separate DB roles, `shop-api` scales 5 to 50 pods, peak 3,000 req/s at 5 ms DB time per request.
    2. Calculate the required concurrent connections with Little's law, choose `default_pool_size`, `max_db_connections`, `max_client_conn` and per-pod app pool size, and write the PgBouncer config.
    3. Explain in writing what happens when all 50 pods are busy and the pool is full, and which timeout fires first.
  - **Done when:** your plan shows the math (3,000 x 0.005 = 15 average, peak headroom), the server connections stay inside the budget, and a teammate can follow your reasoning.

10 Self-check questions

1. Why is a PostgreSQL connection more expensive than a connection to, say, an HTTP server?
2. Your 40 pods each hold a pool of 10 and PostgreSQL has `max_connections = 100`. What happens, and what are two fixes?
3. Which PgBouncer pool mode do you use for an API, and what is the main risk?
4. What does `cl_waiting > 0` in `SHOW POOLS` tell you?
5. Roughly how many connections should a database with 8 cores keep active, and why not 500?

6. Why can `pg_advisory_lock` be dangerous behind a transaction pooler?

ANSWERS

1. Each connection is a dedicated server process with its own memory, created by a fork plus authentication and TLS; the server also has a fixed limit of backends. HTTP servers typically multiplex many connections in a few threads.
2. 400 potential connections exceed 100, so new connections are refused with "too many clients" (and the rest of the DB is crowded). Fixes: put a pooler such as PgBouncer in transaction mode in front, shrink the per-pod pool, cap pod count, or all three. Raising `max_connections` is a last resort.
3. Transaction mode. The risk is session state (`SET` , session advisory locks, `PREPARE` , `LISTEN` , temp tables) breaking or leaking because consecutive transactions may use different server connections.
4. Clients are queued waiting for a server connection; the pool is saturated. A brief spike is the pooler doing its job; sustained waiting means pool too small or slow queries holding connections, and eventually `query_wait_timeout` errors.
5. A starting point is (cores x 2) plus disk parallelism, so roughly 16-20, then load test. More active backends than the hardware can run in parallel only adds contention and context switching and increases latency.
6. The lock belongs to a server connection, and the pooler may give that connection to another client or give you a different one later, so you may unlock on the wrong connection or leave the lock held and block others. Use `pg_advisory_xact_lock` inside a transaction.

11 Further reading

- PgBouncer documentation (pgbouncer.org): the configuration reference and the *Features* page, which lists exactly which SQL features work in each pool mode.
- PostgreSQL documentation: *Server Configuration - Connections and Authentication* (`max_connections` , `superuser_reserved_connections`) and the `pg_stat_activity` view.
- PostgreSQL wiki: *Number of Database Connections*, and the HikariCP wiki page *About Pool Sizing*.
- Amazon RDS User Guide: *Amazon RDS Proxy*.
- *PostgreSQL 14 Internals* (Egor Rogov, free PDF) for what a backend process holds.
- Related: Replication §13.4, Horizontal and Vertical Scaling §13.10.

13.7 Redis

Topic 7 of 15 | ~6 hours | Difficulty: Intermediate

1 What it is

Redis (REmote DIctionary Server) is an in-memory data-structure server. You connect over TCP, send commands such as `SET`, `HSET`, `ZADD` or `XADD`, and get answers in well under a millisecond because the data lives in RAM and commands are executed on a single main thread. Unlike a plain key-value cache, the values are rich types (strings, hashes, lists, sets, sorted sets, streams, bitmaps, HyperLogLogs and more) with server-side operations such as "increment this counter atomically" or "give me the top 10 scores".

People use it for four things: a **cache** in front of a slower database, **ephemeral state** (sessions, rate-limit counters, feature flags), **coordination** (locks, queues, leaderboards, pub/sub) and, with care, a **primary store** for small, fast-changing data. Memory is expensive and finite, so Redis comes with the questions this topic answers: what happens when it fills up (**eviction**), what happens if it restarts (**persistence**, via RDB snapshots and the AOF log) and what happens if its server dies (**replication** with **Sentinel**, or sharding with **Redis Cluster**).

> Licensing note: Redis changed its license in 2024, and the Linux Foundation project **Valkey** forked from the last BSD-licensed release. Commands, persistence and clustering behave the same for the purposes of this topic; managed services (for example Amazon ElastiCache) offer Redis OSS-compatible engines and Valkey. Check which engine and version your platform provides.

2 Why it matters in DevOps

- **Latency and database load:** a well-placed cache can remove most reads from PostgreSQL. A cache that is *cold* after a restart, or that expires all keys at once (**cache stampede**), sends that load back to the database and can take it down.
- **Memory is a hard limit.** With no `maxmemory`, Redis grows until the kernel's OOM killer ends it. With `maxmemory` and the wrong eviction policy, it either throws errors on writes or silently evicts data you needed (session keys!).

- **Persistence is a choice, not a default guarantee.** Depending on settings, a crash loses nothing, a second, or minutes of writes. Calling Redis "a database" without knowing which one you configured is a postmortem waiting to happen.
- **Single-threaded command execution:** one slow command (`KEYS *`, a huge `SMEMBERS`, a big Lua script) blocks every other client.
- **High availability is not automatic.** A lone Redis is a single point of failure. Sentinel or a managed service gives failover, and failover can still lose the last few acknowledged writes because replication is asynchronous (the same trade-off as in Replication §13.4).
- It shows up everywhere else in this roadmap: ElastiCache §6.11, rate limiting in Rate Limiting §12.13, and Distributed Locks and Queues §13.15.

3 Real-world system example

Stripe's API rate limiters. Stripe published how they protect the API with several kinds of limiters (request rate, concurrent requests, fleet-usage load shedding, worker-utilization shedding), implemented with the token-bucket algorithm backed by Redis, because a shared, atomic, low-latency counter is exactly what a fleet of stateless API servers needs.

Background-job libraries. Resque (created at GitHub) and **Sidekiq**, two widely used Ruby job libraries, keep their job queues in Redis; many Rails applications therefore treat Redis as part of their core infrastructure, and "Redis is full" becomes "jobs stop."

Managed Redis. Amazon ElastiCache runs Redis-compatible nodes with replicas across Availability Zones and automatic failover, and a cluster mode that shards data across many nodes. Kubernetes shops commonly run a Redis Sentinel or Cluster deployment with a Helm chart or operator for sessions and caches. Whatever the platform, the operational questions are the same: how much memory, what eviction policy, what persistence, how does failover work, how do clients find the new primary.

4 Diagram

The cache-aside pattern, the most common use:

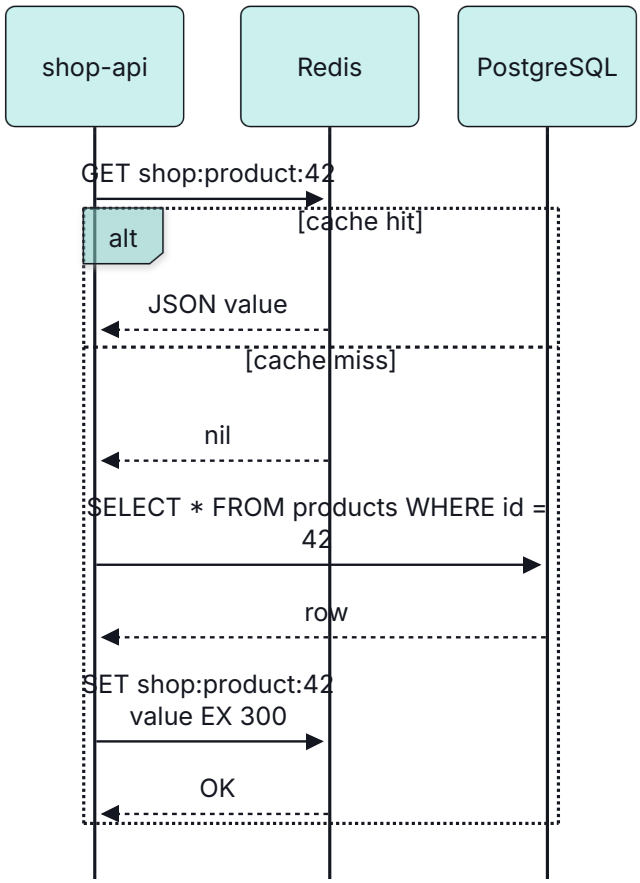


Figure 13.7-1

High availability with Sentinel (one primary, replicas, three Sentinels voting) versus sharding with Redis Cluster:

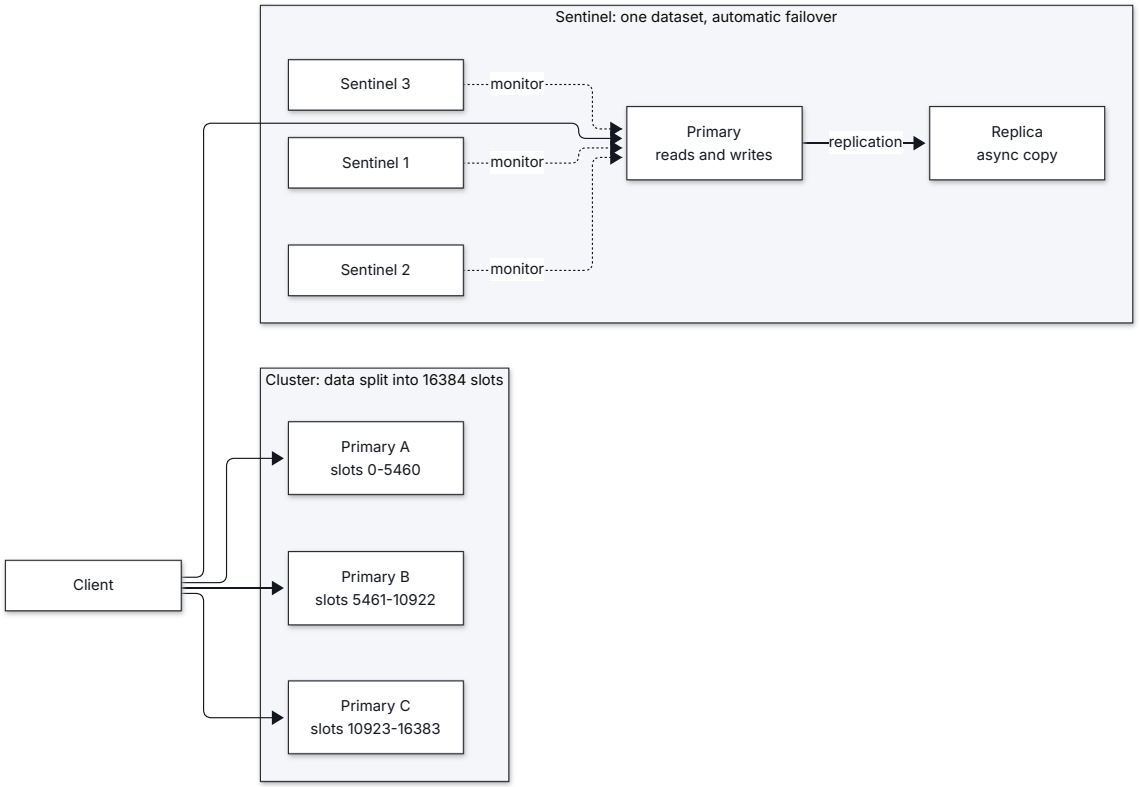


Figure 13.7-2

5 How it works

Data structures and when to use them

Type	Typical commands	Use for	Example key
String	`SET key v EX 60`, `GET`, `INCR`, `SET key v NX PX 30000`	cached JSON, counters, simple locks, feature flags	`shop:product:42`
Hash	`HSET`, `HGET`, `HGETALL`, `HINCRBY`	objects with fields: a shopping cart (`sku → qty`), session	`cart:u1`
List	`LPUSH`, `RPOP`, `BRPOP`, `LRANGE`	simple queues, recent-items feeds	`queue:email`

Type	Typical commands	Use for	Example key
Set	`SADD`, `SISMEMBER`, `SINTER`, `SCARD`	unique members, tags, "who is online"	`online`
Sorted set	`ZADD`, `ZRANGE ... REV WITHSCORES`, `ZRANGEBYSCORE`	leaderboards, rate-limit windows, delayed jobs	`leaderboard`
Stream	`XADD`, `XREADGROUP`, `XACK`	durable-ish event log with consumer groups (a small Kafka-like log; see Kafka §13.8)	`events`
Bitmap / HyperLogLog	`SETBIT` / `BITCOUNT`, `PFADD` / `PFCOUNT`	daily-active flags; approximate unique counts in about 12 KB	`visits:2026-10-09`
Pub/Sub	`PUBLISH`, `SUBSCRIBE`	fire-and-forget broadcast (messages are lost if no subscriber is connected)	`channel:orders`

Every key can carry a **TTL** (time to live): `EXPIRE key 60`, `TTL key`. Expired keys are removed lazily when touched plus by a periodic background sample, so memory may not drop at the exact second of expiry.

Two habits that matter in production: **name keys with a prefix scheme** (`service:entity:id`) and **never run** `KEYS *` on a live server. Use `SCAN` (incremental, cursor-based) instead.

Caching patterns

Pattern	How it works	Trade-off
Cache-aside (lazy loading)	App reads cache, on a miss reads the database and writes the cache	simplest; stale data until TTL or invalidation; first request after a miss is slow
Read-through	Cache layer itself loads from the DB on a miss	simpler app code; needs a caching library or proxy
Write-through	Writes go to cache and database together	cache stays fresh; slower writes; caches data that may never be read
Write-behind (write-back)	Write to cache, flush to DB asynchronously	fastest writes; **risk of data loss** if Redis dies before the flush
TTL only	Everything expires after N seconds	trivially safe; you choose how stale is acceptable

Cache problems you must plan for:

- **Stampede (thundering herd):** a hot key expires and 500 requests rebuild it at once. Mitigate with a short per-key rebuild lock (`SET lock NX PX`), **TTL jitter** (add a random few seconds), or serving stale data while one worker refreshes.
- **Penetration:** requests for keys that never exist always miss and hit the DB. Cache "not found" with a short TTL.

- **Invalidation is the hard part.** On update, delete the key (`DEL`) rather than trying to rewrite it; keep TTLs as a safety net.
- **Cold start:** an empty cache after a restart or failover can overload the database; warm it or ramp traffic.

Persistence: RDB and AOF

	RDB snapshot	AOF (Append-Only File)
What it is	Point-in-time binary dump (`dump.rdb`), written by a forked child (`BGSAVE`)	A log of every write command, replayed on start
Config	`save 3600 1 300 100 60 10000`	`appendonly yes` , `appendfsync always` , `everysec` (default), or `no`
Data you can lose on crash	everything since the last snapshot (minutes)	about 1 second with `everysec` ; almost none with `always` (much slower)
Restart speed	fast (compact file)	slower (replay), mitigated by AOF rewrite and an RDB preamble
File size	small	larger, compacted by background rewrite
Good for	backups, fast restarts, replication seeding	durability closest to a database

In Redis 7 the AOF is a set of files in an `appendonlydir/` (a base file plus incremental logs and a manifest). Using **both** is the usual recommendation when you need durability: AOF for recovery, RDB files for backups. For a pure cache you often disable persistence entirely (`save ""` , `appendonly no`) and accept a cold start. Remember that `BGSAVE` and AOF rewrite use `fork()` , and under heavy writes copy-on-write can temporarily need extra RAM, so leave headroom (do not size the host exactly to `maxmemory`). Redis is **not** a substitute for a backup of your system of record; copy RDB files off the host just like PostgreSQL backups §13.5.

Eviction: what happens at `maxmemory`

Set `maxmemory` (for example `maxmemory 2gb`) and choose `maxmemory-policy` :

Policy	Behaviour
`noeviction` (default)	reject writes with `OOM command not allowed when used memory > 'maxmemory'` , reads still work
`allkeys-lru` / `allkeys-lfu`	evict the least recently / frequently used key of any kind. **Typical for a pure cache**
`volatile-lru` / `volatile-lfu` / `volatile-ttl` / `volatile-random`	evict only keys that have a TTL. If none have, behaves like `noeviction`
`allkeys-random`	random key

The golden rule: **a cache and a data store should not share an instance.** If sessions or queues sit in the same Redis as a cache using `allkeys-lru`, memory pressure will delete them. Use separate instances (or at least separate policies) for different jobs. Watch `evicted_keys` and the hit ratio (`keyspace_hits` / (`keyspace_hits` + `keyspace_misses`)) in `INFO stats`.

High availability and scale-out

- **Replication:** `replicaof host port` makes a node an asynchronous copy. Replicas are read-only by default, so reads from them can be stale. `WAIT n timeout` blocks until n replicas acknowledge, which reduces but does not eliminate loss.
- **Sentinel:** a set of at least three small Sentinel processes that monitor the primary and replicas, agree (a **quorum**) that the primary is down (`+sdown`, then `+odown`), elect one to run a failover, promote a replica and tell clients the new address. Sentinel-aware clients ask Sentinel for the current primary (`SENTINEL get-master-addr-by-name <name>`). It gives HA but **not** sharding: your dataset must still fit on one machine.
- **Redis Cluster:** the key space is divided into **16384 hash slots** (`CRC16(key) mod 16384`) spread across primaries, each ideally with a replica. It gives both sharding and failover with no external Sentinel. Clients must be cluster-aware and follow `MOVED` redirects. Multi-key commands only work if all keys hash to the same slot; use **hash tags** (`{cart:1}:items`, `{cart:1}:total`) to force that. Otherwise you get `CROSSSLOT Keys in request don't hash to the same slot`.
- **Choosing:** one dataset that fits in RAM and needs HA means primary + replica + Sentinel (or a managed Multi-AZ service). A dataset or throughput larger than one node means Cluster. Both are asynchronous, so both can lose recent writes on failover.

6 Step-by-step guide

Prerequisites: Linux/macOS/WSL2 and `redis-server` + `redis-cli` (Debian/Ubuntu: `sudo apt-get install redis-server redis-tools`; or Docker: `docker run -d --name redis-lab -p 7001:6379 redis:7`, then `docker exec -it redis-lab redis-cli`). Redis 7 or newer. No cloud cost. The guide uses ports 7001 and up and a throwaway directory so it will not touch a system Redis on 6379. Outputs were captured with Redis 7.0.

1. **Start a Redis with AOF persistence and talk to it.**

```bash
mkdir -p ~/redislab/r1 && cd ~/redislab
redis-server --port 7001 --dir ~/redislab/r1 --appendonly yes --save "" --daemonize
  yes
redis-cli -p 7001 ping
```

Expect `PONG`.

2. **Try each data structure.**

```bash
R="redis-cli -p 7001"
$R set shop:product:42 '{"name":"mug","price":9.5}' EX 60 ; $R ttl shop:product:42
  # OK then 60
$R hset cart:u1 sku42 2 sku7 1 ; $R hgetall cart:u1
  # field/value pairs
$R incr ratelimit:u1 ; $R expire ratelimit:u1 60
  # atomic counter with a window
$R zadd leaderboard 100 alice 250 bob 175 carol ; $R zrevrange leaderboard 0 1
  withscores # bob 250, carol 175
$R lpush queue:email job1 job2 ; $R rpop queue:email
  # job1 (first in, first out)
$R sadd online u1 u2 ; $R sadd online u2 u3 ; $R scard online
  # 3 unique members
$R xadd events '*' type order_created id 7 ; $R xlen events
  # stream entry id like 1791549546599-0, then 1
$R pfadd hll a b c ; $R pfcount hll
  # 3 (approximate unique count)
$R object encoding cart:u1
  # listpack (compact small-hash encoding)
```

3. **Implement a lock and see why `NX` matters.**

```bash
$R set lock:order:7 worker-A NX PX 30000     # OK  (acquired, auto-expires in 30 s)
$R set lock:order:7 worker-B NX PX 30000     # (nil)  not acquired
```

`NX` means "only if absent" and `PX` is a millisecond expiry, in one atomic command. A safe release must only delete the key if you still own it (compare the stored value in a Lua script). See Distributed Locks and Queues §13.15 for why this is subtle.

4. **Cache-aside with stampede protection (Node.js sketch).** `npm install ioredis@5`, save as `cache.mjs`:

```javascript
import Redis from "ioredis";
const redis = new Redis(7001);
let dbHits = 0;
const loadProductFromDb = async (id) => { dbHits++; await new Promise(r =>
  setTimeout(r, 50)); return { id, name: "mug", price: 9.5 }; };

async function getProduct(id) {
  const key = `shop:product:${id}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);                    // hit
```

```
  const lockKey = `lock:${key}`;
  const got = await redis.set(lockKey, "1", "PX", 5000, "NX");  // only one caller
  rebuilds
  if (!got) { await new Promise(r => setTimeout(r, 60)); return getProduct(id); }
  try {
    const row = await loadProductFromDb(id);
    const ttl = 300 + Math.floor(Math.random() * 60);           // jitter avoids
  synchronized expiry
    await redis.set(key, JSON.stringify(row), "EX", ttl);
    return row;
  } finally { await redis.del(lockKey); }
}

await Promise.all(Array.from({ length: 50 }, () => getProduct(42)));
console.log("50 concurrent requests, db hits:", dbHits);
redis.disconnect();
```

Run `node cache.mjs`. Expect `50 concurrent requests, db hits: 1`. Remove the lock and you will see many more database hits: that is the stampede. (The simple retry loop is a sketch; production code needs a retry limit and a fallback.)

5. **Watch eviction work.** Cap memory and fill it past the cap:

```bash
$R config set maxmemory 5mb ; $R config set maxmemory-policy allkeys-lru
redis-benchmark -p 7001 -t set -n 100000 -d 100 -r 100000 -q
$R dbsize ; $R info stats | grep evicted_keys ; $R info memory | grep -E
  "used_memory_human|maxmemory_policy"
```

`dbsize` stays in the thousands (well below 100,000 distinct keys), `evicted_keys` is large and used memory stays near the cap. Now switch to refusing writes and try again:

```bash
$R config set maxmemory-policy noeviction
redis-benchmark -p 7001 -t set -n 1000 -d 100 -r 100000000 -q     # fails once memory
  is full
```

You will see `OOM command not allowed when used memory > 'maxmemory'`. Reset with `$R config set maxmemory 0` and `$R flushall`.

6. **Prove AOF durability by crashing it.**

```bash
$R set durable yes ; sleep 2 ; pkill -9 -f "redis-server.*7001"
redis-server --port 7001 --dir ~/redislab/r1 --appendonly yes --save "" --daemonize
  yes ; sleep 1
$R get durable
```

Expect `yes`: with `appendfsync everysec` the write reached disk within a second and was replayed from `r1/appendonlydir/`. Repeat after switching to RDB only (`--appendonly no --save ""`): the key is gone. Also try `$R bgsave` and look at `dump.rdb`.

7. **Replication plus Sentinel failover on one machine.**

```bash
mkdir -p r2 s1
redis-server --port 7002 --dir ~/redislab/r2 --replicaof 127.0.0.1 7001 --daemonize yes
cat > s1/sentinel.conf <<'EOF'
port 26379
dir /home/YOU/redislab/s1
sentinel monitor shopmaster 127.0.0.1 7001 1
sentinel down-after-milliseconds shopmaster 3000
sentinel failover-timeout shopmaster 10000
sentinel parallel-syncs shopmaster 1
EOF
sed -i "s#/home/YOU#$HOME#" s1/sentinel.conf
redis-server s1/sentinel.conf --sentinel --daemonize yes --logfile sentinel.log
redis-cli -p 7001 info replication | head -5                        # role:master, connected_slaves:1
redis-cli -p 26379 sentinel get-master-addr-by-name shopmaster      # 127.0.0.1 / 7001
```

The quorum of `1` is for the lab only; production uses three Sentinels on separate hosts and a quorum of 2. Now kill the primary:

```bash
redis-cli -p 7001 shutdown nosave ; sleep 8
redis-cli -p 26379 sentinel get-master-addr-by-name shopmaster      # now 127.0.0.1 / 7002
redis-cli -p 7002 info replication | head -3                        # role:master
grep -E "sdown|odown|switch-master" ~/redislab/s1/sentinel.log
```

The log shows `+sdown`, `+odown`, `+elected-leader`, `+promoted-slave` and `+switch-master shopmaster 127.0.0.1 7001 127.0.0.1 7002`. Clients that used the fixed address 7001 are now broken; Sentinel-aware clients re-ask Sentinel. That is why you configure clients with the Sentinel list, not a primary IP. (If a lab port is in use, change the numbers consistently.)

8. **Redis Cluster in three commands.**

```bash
for p in 7101 7102 7103; do mkdir -p c$p; redis-server --port $p --cluster-enabled yes
  --cluster-config-file nodes.conf --dir ~/redislab/c$p --daemonize yes; done
redis-cli --cluster create 127.0.0.1:7101 127.0.0.1:7102 127.0.0.1:7103 --cluster-
  replicas 0 --cluster-yes
redis-cli -p 7101 set user:1 a                        # MOVED 10778 127.0.0.1:7102
  (wrong node)
redis-cli -c -p 7101 set user:1 a                     # -c follows redirects, OK
redis-cli -c -p 7101 mset a 1 b 2                     # CROSSSLOT error
redis-cli -c -p 7101 mset '{cart:1}:a' 1 '{cart:1}:b' 2     # OK, same slot via a hash
  tag
```

   The slot layout is similar to 0-5460, 5461-10922, 10923-16383. A real cluster uses `--cluster-replicas 1` and six nodes.

9. **Clean up.**

```bash
for p in 7001 7002 26379 7101 7102 7103; do redis-cli -p $p shutdown nosave
  2>/dev/null; done
rm -rf ~/redislab
```

7 Real-world scenario

(Illustrative, not a documented incident.) The `shop-api` team caches product pages in Redis with `maxmemory 4gb`, the default policy and no TTL on session keys, because "Redis is fast." After a marketing push the cache fills; every write now fails with `OOM command not allowed`. Logins break (they write sessions), and the API falls back to PostgreSQL for everything, so the database CPU doubles. The on-call engineer sees `used_memory` at the cap and `evicted_keys: 0` (nothing is allowed to evict under `noeviction`).

Fix, in order: raise `maxmemory` temporarily to stop the bleeding; find the offenders with `redis-cli --bigkeys` and `MEMORY USAGE` (a `cart:*` hash with no TTL has been accumulating abandoned carts); set TTLs; **split** the instance in two: a cache instance with `allkeys-lru` and a small session instance with `noeviction` and AOF `everysec`. They add alerts on `used_memory / maxmemory > 80%`, `evicted_keys` rate and cache hit ratio, and add TTL jitter to prevent the synchronized expiry they also noticed. **Lesson:** eviction policy and workload separation are architecture decisions; a cache that cannot evict, or a session store that can, are both outages.

8 Common mistakes and best practices

Don't

- Run `KEYS *`, `FLUSHALL`, or big `SMEMBERS` / `HGETALL` / `LRANGE 0 -1` on a live production instance (one slow command stalls everyone).
- Use Redis as the only copy of important data without understanding RDB/AOF and failover loss.
- Put caches, queues and sessions in one instance with `allkeys-lru`.
- Expose Redis to the internet or run it without authentication. It was designed for trusted networks; use `requirepass`/ACLs, TLS, `bind` to private addresses and firewall port 6379 (see Security Groups and NACLs §6.6).
- Set TTLs to the same value for every key, or forget to jitter.
- Use Pub/Sub for messages you cannot afford to lose, or a single-node lock for correctness-critical mutual exclusion.

Do

- Set `maxmemory` below the host's RAM (leave room for fork copy-on-write, replica buffers and the OS) and pick the eviction policy deliberately.
- Choose persistence per instance: none for a pure cache, AOF `everysec` (plus RDB backups copied off-host) for data you care about.
- Use `SCAN`, pipelines and short commands; keep values small; use key prefixes and TTLs everywhere possible.
- Run Sentinel with three or more independent members, or a managed Multi-AZ service, and configure clients with the Sentinel/cluster discovery list.
- Monitor memory, evictions, hit ratio, connected clients, latency (`redis-cli --latency`, `SLOWLOG GET`), replication offset/lag and fork time. Export these to Prometheus (Prometheus and PromQL §10.2).
- Design the application to survive an empty Redis: the database remains the source of truth for cached data.

9 Tasks

- [ ] **T1 — Tour the data structures** *(≈40 min, beginner)*
  - **Goal:** pick the right type for a job.
  - **Steps:**
    1. Start Redis (guide step 1).

2. Model: a product cache (string with TTL), a cart (hash), a top-5 sellers board (sorted set), a "recently viewed" list (list capped with `LTRIM`), and unique visitors (HyperLogLog).
3. For each, run the commands that read it back and `TYPE key`, `TTL key`.

- **Done when:** each of the five structures exists with sensible keys and you can say why a string would have been the wrong choice for the cart.

- [ ] **T2 — Cache-aside with a measured hit ratio** *(≈60 min, intermediate)*
  - **Goal:** implement the pattern and measure its effect.
  - **Steps:**
    1. Run the `cache.mjs` sketch (or port it to your language), with a fake 50 ms DB call.
    2. Call `getProduct` for 1,000 random ids drawn from 100 popular ids.
    3. Compute hit ratio from `INFO stats` (`keyspace_hits`, `keyspace_misses`), and total elapsed time versus no cache.
  - **Done when:** you can report hit ratio and speedup (for example 90%+ hits and a large drop in DB calls) with the numbers from your run.

- [ ] **T3 — Cause and fix a cache stampede** *(≈45 min, intermediate)*
  - **Goal:** see thundering-herd load and remove it.
  - **Steps:**
    1. Remove the lock from `getProduct`, fire 200 concurrent requests for one cold key, and count DB hits.
    2. Restore the `SET NX PX` rebuild lock and compare.
    3. Add TTL jitter and show with `TTL` that keys cached at the same moment expire at different times.
  - **Done when:** DB hits fall from many to 1 for a cold key, and you can explain what happens if the lock holder crashes (the lock's `PX` expiry).

- [ ] **T4 — Choose and test an eviction policy** *(≈45 min, intermediate)*
  - **Goal:** understand what each policy keeps and drops.
  - **Steps:**
    1. Set `maxmemory 5mb`. Insert 20 "session" keys with no TTL and 50,000 "cache" keys with a 600 s TTL.
    2. Under `allkeys-lru`, check whether the session keys survive; repeat under `volatile-lru` and `noeviction`.
    3. Record `evicted_keys`, `dbsize` and the error for `noeviction`.

- **Done when:** you have a 3-row table (policy, sessions survived?, write errors?) and a written recommendation to split sessions from cache.

☐ **T5 — Crash it, then fail it over** *(≈75 min, advanced)*

- **Goal:** compare persistence options and observe Sentinel.
- **Steps:**
  1. Run guide step 6 with AOF `everysec`, then with RDB-only (`--save "" --appendonly no`), and then with `appendfsync always`; time 100,000 writes with `redis-benchmark -t set -n 100000 -q` each way.
  2. Run guide step 7. While a loop writes `INCR counter` once per 100 ms against the primary, kill the primary.
  3. Compare the last value the client saw acknowledged with the value on the promoted replica.
- **Done when:** you can state which persistence mode lost data on a crash, the throughput cost of `always`, the failover time from the Sentinel log, and whether the promoted replica was missing any acknowledged increments.

☐ **T6 — Prove it: design the Redis layer for `shop-api`** *(≈45 min, advanced)*

- **Goal:** make and defend the architecture decisions without notes.
- **Steps:**
  1. Write a one-page design covering: what goes in Redis (cache, sessions, rate limits, job queue), which instances are separate, each instance's `maxmemory-policy` and persistence, how HA works (Sentinel or managed), and how the app behaves if Redis is empty.
  2. Include the three metrics you would alert on and their thresholds.
  3. Demonstrate the one riskiest assumption on your lab instance (for example, a restart with the cache emptied while load continues) and show the database holds.
- **Done when:** a teammate can read the design, and your demo shows `shop-api` still serving correct data (slower) with an empty cache.

10 Self-check questions

1. Why is Redis fast, and what is the downside of its threading model?
2. What is the difference between RDB and AOF, and how much data can each lose?
3. You set `maxmemory 2gb` and leave the default policy. What happens when it fills?
4. When would you choose Sentinel and when Redis Cluster?
5. What is a cache stampede and two ways to prevent it?

6. Why does a failover to a replica possibly lose acknowledged writes?

ANSWERS

1. Data is in RAM, structures are simple and commands run on one main thread without locking. The downside is that one slow command blocks all clients, and a single thread can saturate one CPU core.
2. RDB is periodic snapshots (you can lose everything since the last snapshot, minutes). AOF logs every write; with `everysec` you can lose about a second, with `always` almost nothing at a throughput cost. They can be combined.
3. The default is `noeviction`, so writes are rejected with an `OOM command not allowed` error while reads continue. That may be right for a datastore but is usually wrong for a cache.
4. Sentinel gives HA for a dataset that fits on one node (single primary, replicas, automatic failover). Cluster shards across nodes for capacity and throughput and has built-in failover, at the cost of multi-key limitations and cluster-aware clients.
5. Many requests rebuild an expired hot key simultaneously and overload the database. Prevent with a per-key rebuild lock, TTL jitter, serving stale while refreshing, or pre-warming.
6. Replication is asynchronous: the primary acknowledges a write before replicas have it. If the primary dies before replicating and a replica is promoted, those writes are gone. `WAIT` narrows but does not close the gap.

11 Further reading

- Redis documentation (redis.io/docs): *Data types*, *Persistence*, *Eviction*, *High availability with Redis Sentinel*, *Scale with Redis Cluster* and the command reference. The Valkey documentation (valkey.io) covers the fork.
- Stripe Engineering blog: *Scaling your API with rate limiters*.
- Salvatore Sanfilippo (antirez), the original author of Redis, has written widely about Redis persistence and replication design on his blog.
- *Redis in Action* (Josiah Carlson) for patterns; read with the current docs in hand since some features have moved on.
- Martin Kleppmann, *Designing Data-Intensive Applications*, chapters on replication and caching trade-offs.
- Related: ElastiCache §6.11, Database Connection Pooling §13.6, Idempotency §13.14.

LAB NOTES What I ran, what I saw, what it means

13.8 Kafka

Topic 8 of 15 | ~7 hours | Difficulty: Advanced

1 What it is

Apache Kafka is a distributed, replicated, append-only **commit log** that many programs can write to and read from. A **producer** appends **records** (a key, a value, a timestamp and optional headers) to a named **topic**. Each topic is split into **partitions**, and each partition is an ordered, immutable sequence of records, each with a sequential number called its **offset**. **Consumers** read records by offset, and remember how far they have read by **committing** an offset. Records are not deleted when read; they are kept for a configurable **retention** period (or until a size limit) so that many independent consumers can read the same data, and a consumer that fell behind or was fixed after a bug can rewind and read it again.

That differs from a classic message queue such as RabbitMQ (see RabbitMQ §13.9), where a broker pushes messages to consumers and removes them once acknowledged. Kafka is a *log you read*, not a *queue you drain*. It is used to decouple services, stream events between systems, feed analytics, and to propagate database changes (change data capture).

2 Why it matters in DevOps

- **It is infrastructure you operate.** Brokers need disk, network and careful sizing; partitions and replication factors are decisions that are painful to change later; upgrades and rebalances need planning.
- **Lag is the new "queue depth".** A consumer group that falls behind means orders, emails or analytics are delayed. Consumer lag is a key SLI §14.1 for event-driven systems, and rebalances, slow consumers and poison messages are common on-call topics.
- **Ordering and duplicates surprise people.** Kafka orders records *within a partition* only, and the normal delivery guarantee is *at least once*, so consumers must be idempotent §13.14. Adding partitions silently changes which partition a key maps to.
- **Data durability depends on configuration:** `acks`, `replication.factor` and `min.insync.replicas` together decide whether an acknowledged write can be lost when a broker dies (the same trade-offs as Replication §13.4).

- **Disk fills:** retention is how you control cost and how long consumers can fall behind. A forgotten topic with infinite retention quietly eats the volume.

3 Real-world system example

LinkedIn created Kafka to handle activity-stream and operational-metrics data and open-sourced it; it became an Apache project. LinkedIn's engineering writing describes the log as the central abstraction connecting data systems, an idea laid out in Jay Kreps' essay *The Log: What every software engineer should know about real-time data's unifying abstraction*.

Netflix has written publicly about building its Keystone data pipeline on Kafka for routing large volumes of events (such as viewing and operational events) to downstream consumers, using Kafka clusters per use case and separating producers from consumers. **Uber, Shopify** and many others describe Kafka as the backbone for event streaming; and **Debezium** (change data capture) reads the PostgreSQL write-ahead log and publishes row changes into Kafka topics, linking this topic to Replication §13.4. Cloud equivalents include **Amazon MSK** (managed Kafka), **Confluent Cloud**, and, with a different API, Kinesis and Google Pub/Sub.

On Kubernetes, Kafka is commonly run with an operator such as **Strimzi**. Since Kafka 3.3, **KRaft** mode (Kafka's own Raft-based metadata quorum) is production-ready, and Kafka 4.0 removed the dependency on ZooKeeper entirely, so new clusters should use KRaft.

4 Diagram

A topic with three partitions, replicated, and a consumer group dividing the work:

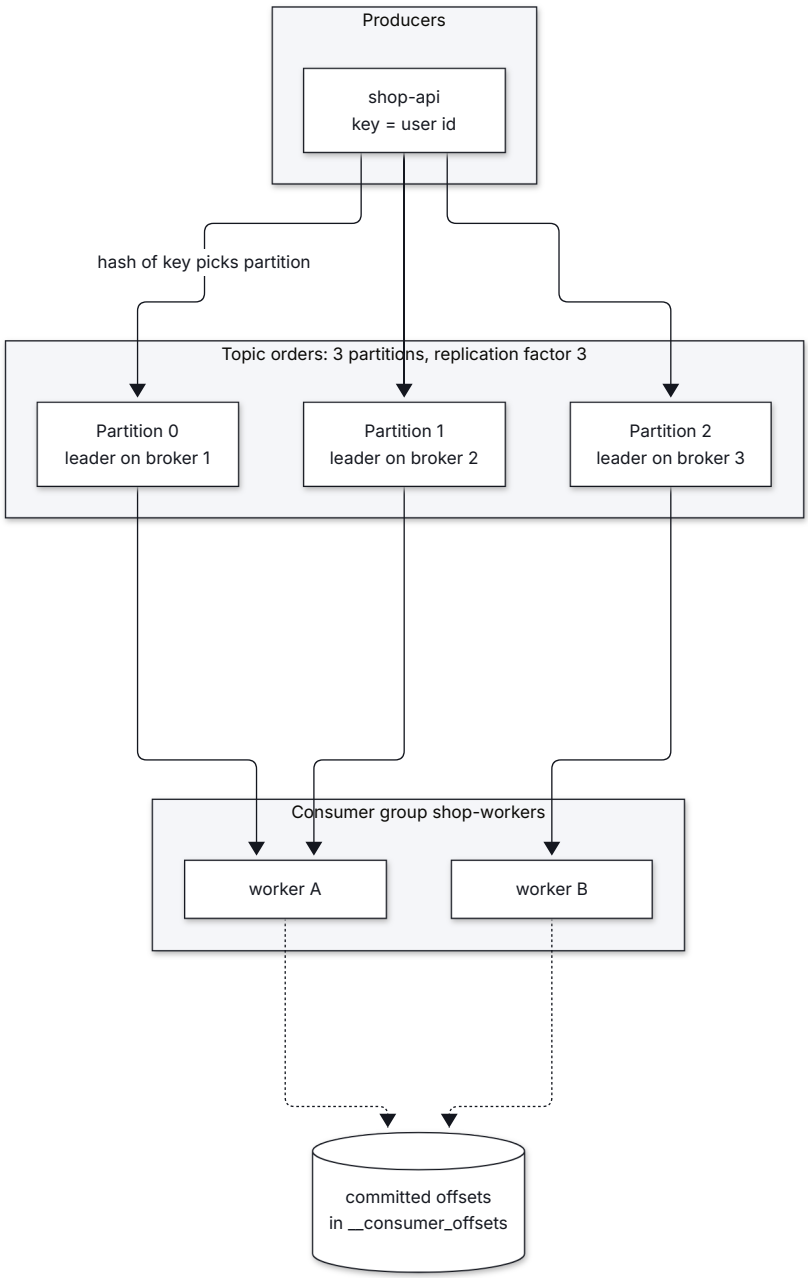


Figure 13.8-1

A durable write with `acks=all` and `min.insync.replicas=2`:

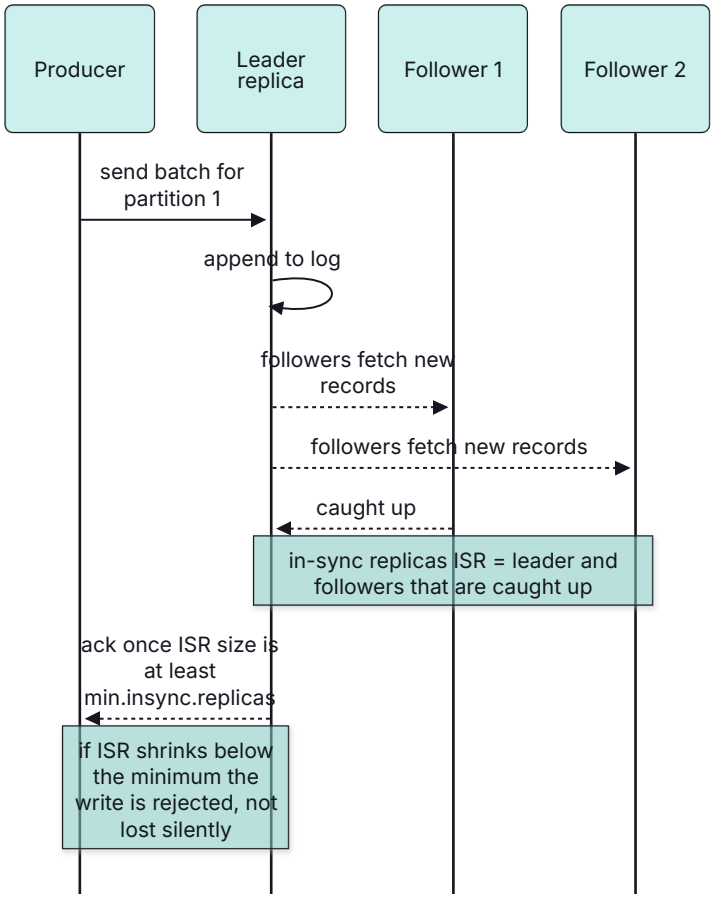


Figure 13.8-2

5 How it works

The vocabulary

Term	Meaning
Broker	A Kafka server. A cluster is typically 3+ brokers
Topic	Named stream of records (`orders`, `payments`, `inventory.events`)
Partition	One ordered log; the unit of **parallelism** and of **ordering**. A topic's partitions are spread over brokers
Offset	Position of a record in a partition (0, 1, 2 ...). Unique only within a partition

Term	Meaning
Replication factor	Copies of each partition (3 is the production norm). One is the **leader** (serves reads/writes), the others are **followers**
ISR	In-Sync Replicas: followers that are caught up with the leader. Only ISR members are eligible to become leader without data loss
Producer	Writes records. Chooses the partition by hashing the key, or round-robin/sticky when there is no key
Consumer group	A set of consumers sharing a `group.id`. Each partition is assigned to **exactly one** consumer in the group at a time
Group coordinator / rebalance	A broker tracks membership. When consumers join or leave, partitions are redistributed (a **rebalance**)
Committed offset	The group's saved position, stored in the internal topic `__consumer_offsets`. After a restart consumers resume from it
Lag	`log-end-offset - committed offset` per partition: how far behind a group is
KRaft controller	Quorum of nodes that stores cluster metadata (replaces ZooKeeper)

Partitions, consumer groups and parallelism

- Within one consumer group, **parallelism is capped by the number of partitions**. With 3 partitions, a 4th consumer sits idle. Different groups are independent and each reads the whole topic: a "billing" group and an "analytics" group both see every order.
- Choosing partition count: enough for peak consumer parallelism and throughput (a rough rule is target throughput divided by what one partition/consumer can do), with room to grow. More partitions mean more open files, more replication work and longer recovery, so avoid hundreds per topic "just in case".
- **You can add partitions but never remove them**, and adding changes the key-to-partition mapping (see ordering below).

Ordering guarantees

- Kafka guarantees order **within a partition** only. There is no ordering across partitions or across topics.
- To keep all events for one entity ordered (all events of order 123, all actions of user u1), use that entity id as the **record key**: the same key always goes to the same partition (as long as the partition count does not change).
- Retries can reorder records unless the producer is **idempotent** (`enable.idempotence=true`, the default in recent clients, which implies `acks=all` and bounded in-flight requests). It also removes duplicates caused by producer retries.

- **Adding partitions to a live keyed topic breaks per-key ordering** for a time: new records for key `u1` may land in a different partition than old ones, and a consumer may process new before old. You will reproduce this in the guide.
- A consumer processing a partition in parallel with threads, or a rebalance mid-batch, can also break your effective ordering.

Durability settings (producer + topic)

Setting	Effect
`acks=0`	Fire and forget. Fast, lossy
`acks=1`	Leader has it. Lost if the leader dies before followers copy it
`acks=all`	All in-sync replicas have it. Use with the next setting
`min.insync.replicas=2` (with RF 3)	Writes fail if fewer than 2 replicas are in sync, instead of silently accepting risky writes. RF 3 + `min.insync.replicas=2` + `acks=all` tolerates losing one broker without losing acknowledged data
`unclean.leader.election.enable=false` (default)	Never elect an out-of-sync replica (prefers unavailable to losing data)

Delivery semantics and offsets

- **At-most-once:** commit the offset *before* processing. A crash loses the message.
- **At-least-once** (the practical default): process, *then* commit. A crash between the two causes redelivery, so handlers must be idempotent.
- **Exactly-once** in Kafka means idempotent producers plus **transactions** (atomic read-process-write between Kafka topics, `isolation.level=read_committed`). Once you write to an *external* system (PostgreSQL, an email API) you are back to at-least-once plus an idempotent handler or a deduplication key. See Idempotency §13.14.
- Key consumer settings: `enable.auto.commit` (turn it off if you need control), `auto.offset.reset` (`earliest` or `latest` when the group has no committed offset), `max.poll.interval.ms` (if processing a batch takes longer, the consumer is kicked out and a rebalance happens), `session.timeout.ms`, `max.poll.records`.
- **Poison messages:** a record that always fails will block its partition forever if you retry endlessly. Common solutions are a **dead-letter topic** and bounded retries.

Retention and compaction

Policy	Config	Behaviour
Time/size retention	`retention.ms` (default 7 days), `retention.bytes`	Old **segments** (files) are deleted once they pass the limit. Deletion works on whole closed segments, a periodic check (`log.retention.check.interval.ms`, default 5 minutes) does it, so it is approximate
Log compaction	`cleanup.policy=compact`	Keeps at least the **latest record per key** (tombstones, a null value, delete a key). Used for "current state" topics, such as changelogs and `__consumer_offsets`
Both	`cleanup.policy=compact,delete`	Latest per key but also bounded by time

Also relevant: `segment.bytes` / `segment.ms` (when a new file starts), `message.max.bytes`, and the disk layout `log.dirs/<topic>-<partition>/` with `.log`, `.index` and `.timeindex` files. Consumers can only read what still exists: a group that is offline longer than retention loses data (its offsets point to deleted records and `auto.offset.reset` decides what happens).

6 Step-by-step guide

Prerequisites: Java 17 or newer (Java 11+ for clients), a Linux/macOS/WSL2 shell, about 1.5 GB RAM, and a Kafka download (3.9 or newer; current releases are listed at kafka.apache.org/downloads). Free, local. The commands are in the Kafka `bin/` directory; the outputs below came from Kafka 3.9.0 in single-node KRaft mode and yours will be "similar to" them. A single broker means replication factor 1 here: it demonstrates partitions, groups, offsets and ordering, but **not** broker-failure durability. For three brokers, use a Compose file or the Strimzi operator.

1. **Install and start a single-node KRaft broker.**

```bash
cd ~ && curl -fsSLO https://archive.apache.org/dist/kafka/3.9.0/kafka_2.13-3.9.0.tgz
tar xzf kafka_2.13-3.9.0.tgz && cd kafka_2.13-3.9.0
mkdir -p ~/kafkalab
cat > ~/kafkalab/server.properties <<'EOF'
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@localhost:19093
listeners=PLAINTEXT://localhost:19092,CONTROLLER://localhost:19093
advertised.listeners=PLAINTEXT://localhost:19092
controller.listener.names=CONTROLLER
inter.broker.listener.name=PLAINTEXT
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
log.dirs=/tmp/kafkalab/data
num.partitions=1
```

```
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
group.initial.rebalance.delay.ms=0
EOF
ID=$(bin/kafka-storage.sh random-uuid)
bin/kafka-storage.sh format -t "$ID" -c ~/kafkalab/server.properties
bin/kafka-server-start.sh -daemon ~/kafkalab/server.properties
sleep 10 && bin/kafka-topics.sh --bootstrap-server localhost:19092 --list
```

`format` prints `Formatting metadata directory ...`; the empty `--list` output means the broker is up. The ports 19092/19093 avoid clashing with a Kafka on the default 9092. (Alternative with Docker: `docker run -d --name kafka -p 9092:9092 apache/kafka:3.9.0`, then use `localhost:9092`.) Set `export BS="--bootstrap-server localhost:19092"` for the next steps.

2. **Create a topic and look at it.**

```bash
bin/kafka-topics.sh $BS --create --topic orders --partitions 3 --replication-factor 1
  --config retention.ms=604800000
bin/kafka-topics.sh $BS --describe --topic orders
```

You should see three partitions each with a `Leader: 1`, `Replicas: 1` and `Isr: 1`. On a real cluster these show different brokers.

3. **Produce keyed records and see the key-to-partition mapping.**

```bash
printf 'u1:order-1\nu2:order-2\nu1:order-3\nu3:order-4\nu2:order-5\nu1:order-6\n' | \
  bin/kafka-console-producer.sh $BS --topic orders --property parse.key=true --
  property key.separator=:
bin/kafka-console-consumer.sh $BS --topic orders --from-beginning --max-messages 6 \
  --property print.key=true --property print.partition=true --property
  print.offset=true --timeout-ms 8000
```

Expected shape (partition numbers may differ slightly by key hashing, but the pattern is fixed):

```
Partition:0  Offset:0  u2  order-2
Partition:0  Offset:1  u2  order-5
Partition:1  Offset:0  u1  order-1
Partition:1  Offset:1  u1  order-3
Partition:1  Offset:2  u3  order-4
Partition:1  Offset:3  u1  order-6
```

All `u1` records sit in one partition in the order they were sent; across partitions there is no global order. Offsets restart at 0 in every partition.

4. **Consume as a group and read the lag.**

```bash
bin/kafka-console-consumer.sh $BS --topic orders --group shop-workers --from-beginning
  --max-messages 3 --timeout-ms 8000 > /dev/null
bin/kafka-consumer-groups.sh $BS --describe --group shop-workers
```

Output similar to:

```
GROUP         TOPIC   PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
  HOST  CLIENT-ID
shop-workers  orders  0          2               2               0    -            -
  -
shop-workers  orders  1          1               4               3    -            -
  -
shop-workers  orders  2          0               0               0    -            -
  -
```

`LAG` of 3 on partition 1 is exactly "three records the group has not processed". The group has no active members now, but its committed offsets live in `__consumer_offsets`. Run the same consumer again and it continues from there.

5. **Run two consumers in one group and watch partitions get split.**

```bash
for i in 1 2; do
  bin/kafka-console-consumer.sh $BS --topic orders --group g2 --from-beginning --
  consumer-property client.id=worker-$i > /dev/null &
  sleep 6
done
bin/kafka-consumer-groups.sh $BS --describe --group g2
bin/kafka-consumer-groups.sh $BS --describe --group g2 --state
```

Each partition is assigned to exactly one `CLIENT-ID` (for example worker-1 holds partitions 0 and 1 and worker-2 holds partition 2) and the state shows `Stable` with `#MEMBERS 2`. Stop one consumer (`kill %1`) and describe again: the survivor takes everything over, within a few seconds after a clean shutdown or up to `session.timeout.ms` (45 s by default in recent clients) if the consumer is killed hard. That is a rebalance. Stop the background jobs with `kill %1 %2` when done.

6. **Reset offsets to replay history.**

```bash
bin/kafka-consumer-groups.sh $BS --group shop-workers --topic orders --reset-offsets -
  -to-earliest --dry-run
bin/kafka-consumer-groups.sh $BS --group shop-workers --topic orders --reset-offsets -
  -to-earliest --execute
```

The group must have no active members. This is how you reprocess after a bug fix (and why consumers must tolerate duplicates). `--to-datetime`, `--shift-by` and `--to-offset` are also available.

7. **Break ordering by adding partitions.**

```bash
bin/kafka-topics.sh $BS --create --topic remap --partitions 3 --replication-factor 1
printf 'u1:before\nu2:before\nu3:before\nu4:before\nu5:before\nu6:before\n' | \
  bin/kafka-console-producer.sh $BS --topic remap --property parse.key=true --property
  key.separator=:
bin/kafka-topics.sh $BS --alter --topic remap --partitions 6
sleep 3
printf 'u1:after\nu2:after\nu3:after\nu4:after\nu5:after\nu6:after\n' | \
  bin/kafka-console-producer.sh $BS --topic remap --property parse.key=true --property
  key.separator=:
bin/kafka-console-consumer.sh $BS --topic remap --from-beginning --max-messages 12 \
  --property print.key=true --property print.partition=true --timeout-ms 8000 | sort
```

Some keys keep their partition and some move. In our run `u1` and `u3` and `u6` moved from partition 1 to partition 4, `u5` from 2 to 5, while `u2` and `u4` stayed in 0. For moved keys, "before" and "after" records are now in different partitions, so a consumer can see `after` before `before`. Also try shrinking: `--alter --partitions 2` fails with `InvalidPartitionsException ... would not be an increase`.

8. **Retention, size and the files on disk.**

```bash
bin/kafka-configs.sh $BS --entity-type topics --entity-name orders --alter --add-
  config retention.ms=86400000,retention.bytes=1073741824
bin/kafka-configs.sh $BS --entity-type topics --entity-name orders --describe
ls /tmp/kafkalab/data/orders-1
bin/kafka-dump-log.sh --files /tmp/kafkalab/data/orders-1/00000000000000000000.log --
  print-data-log | head
```

You see `.log`, `.index` and `.timeindex` files, and the dump shows each record's offset, key size and payload. To see deletion happen, set `segment.bytes=1048576` (a small segment) and `retention.ms=60000` on a scratch topic, produce a few MB with `bin/kafka-producer-`

`perf-test.sh --topic scratch --num-records 100000 --record-size 200 --throughput -1 --producer-props bootstrap.servers=localhost:19092`, and wait for the retention check; old `.log` segment files disappear and the log-start offset moves forward (`bin/kafka-get-offsets.sh $BS --topic scratch --time earliest`). Exact timing depends on the check interval (default 5 minutes), so allow several minutes. (This deletion walk-through was not run in our lab; the config names are from the Kafka documentation.)

9. **A consumer that commits after processing (at-least-once).** With `pip install confluent-kafka`, save as `worker.py`:

```python
from confluent_kafka import Consumer, Producer
import json

p = Producer({"bootstrap.servers": "localhost:19092", "enable.idempotence": True,
  "acks": "all"})
for i in range(5):
    p.produce("orders", key=f"u{i % 2}", value=json.dumps({"order_id": i}))
p.flush()

c = Consumer({
    "bootstrap.servers": "localhost:19092",
    "group.id": "py-workers",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,      # commit only after the work is done
})
c.subscribe(["orders"])
while True:
    msg = c.poll(2.0)
    if msg is None:
        break
    if msg.error():
        raise RuntimeError(msg.error())
    print(f"partition={msg.partition()} offset={msg.offset()} key=
  {msg.key().decode()}")
    c.commit(message=msg)             # a crash before this line means redelivery (at-
  least-once)
c.close()
```

Run `python3 worker.py`; it prints each record and `kafka-consumer-groups.sh --describe --group py-workers` shows `LAG 0`. Kill the process in the middle (add `time.sleep(1)` in the loop) and restart: it resumes from the last committed offset, possibly repeating one record.

10. **Clean up.**

```bash
bin/kafka-server-stop.sh
rm -rf /tmp/kafkalab ~/kafkalab
```

7 Real-world scenario

(Illustrative, not a documented incident.) The `shop-api` order service publishes `order.created` events to an `orders` topic with 6 partitions, keyed by `order_id`. A `fulfilment` consumer group of six pods reads it and a `notifications` group sends emails. On a Monday morning customer support reports "I got the shipping email twice", and an hour later "orders from Saturday have not shipped."

Investigation: `kafka-consumer-groups.sh --describe --group fulfilment` shows partition 3 with a lag of 41,000 and growing, while the other five partitions are at 0, and the consumer for partition 3 keeps restarting. The logs show one malformed record (a missing `currency`) crashes the handler; the consumer restarts, re-reads the same offset, and crashes again. A poison message is blocking one partition. Meanwhile the duplicate emails: each restart replays the batch that was already partly processed before the offset was committed, and the email handler is not idempotent.

Fix: skip the bad record by moving it to a `orders.dlq` dead-letter topic and continuing, deploy a fix for the validation, and make the email handler record `order_id` in a table (unique key) before sending, so redelivery does not resend. Add alerts on lag per group/partition and on consumer restarts; and set `max.poll.interval.ms` so a slow batch does not trigger endless rebalances. **Lesson:** at-least-once delivery is a contract that your code must honour: bounded retries, a dead-letter path, idempotent side effects and lag alerts.

8 Common mistakes and best practices

Don't

- Use `acks=1` or RF 1 for data you cannot lose, or set `min.insync.replicas=1` and believe it is durable.
- Assume global ordering, or add partitions to a keyed topic without planning the ordering impact.
- Create hundreds of partitions per topic "for later", or one topic per customer.
- Retry a failing record forever without a dead-letter path, or do long processing between `poll()` calls so the consumer is evicted from the group.

- Auto-commit offsets and then do slow or failure-prone work (you may commit what you have not finished).
- Put large blobs (tens of MB) in records; store the file in object storage and send a reference.
- Leave the cluster without disk-usage, under-replicated-partition and lag alerts; run a single broker in production.

Do

- Use RF 3, `acks=all`, `min.insync.replicas=2`, idempotent producers, and spread brokers across Availability Zones (with rack awareness).
- Pick keys that give both ordering where you need it and even distribution (avoid a hot key).
- Make consumers idempotent (Idempotency §13.14) and design for replays; keep a dead-letter topic and bounded retries.
- Monitor consumer lag, under-replicated partitions, ISR shrink events, request latency and disk usage; export JMX metrics to Prometheus (Observability §10.1).
- Set retention deliberately per topic, document it, and use compaction for "latest state" topics.
- Version your event schemas (for example with a schema registry and compatible evolution) and treat topics as public APIs.
- Use KRaft, keep clients and brokers on supported versions, and rehearse broker replacement and rolling upgrades.

9 Tasks

- [ ] **T1 — Single-node KRaft broker and your first topic** *(≈45 min, beginner)*
  - **Goal:** run Kafka and inspect a topic's structure.
  - **Steps:**
    1. Follow guide steps 1-2 to start a broker and create `orders` with 3 partitions.
    2. Run `kafka-topics.sh --describe` and `kafka-get-offsets.sh --topic orders`.
    3. Look at the partition directories under `log.dirs`.
  - **Done when:** you can point at the leader, replica and ISR columns, and show three `orders-N` directories on disk, explaining what each file type is for.
- [ ] **T2 — Prove per-key ordering** *(≈45 min, intermediate)*
  - **Goal:** demonstrate that Kafka orders within a partition, not across the topic.
  - **Steps:**

1. Produce 1,000 records with keys `u1` .. `u20` and an increasing sequence number in the value.
2. Consume with `print.partition`, `print.key` and `print.offset`.
3. For each key, verify that sequence numbers are increasing and all of a key's records came from one partition (a short script is fine); then show that the global sequence is not monotonic.

- **Done when:** your check prints "all keys ordered within partition" and your output also shows at least one place where the global sequence is out of order.

☐ **T3 — Scale a consumer group and cause a rebalance** *(≈60 min, intermediate)*

- **Goal:** see partition assignment, parallelism limits and lag.
- **Steps:**
  1. Produce 200,000 records with `kafka-producer-perf-test.sh`.
  2. Start one consumer in group `g3`, then add consumers one by one (up to 4 on a 3-partition topic), recording `--describe` each time.
  3. Kill one consumer and time how long until the partitions are reassigned.
- **Done when:** you can show that the 4th consumer received no partitions, and report the reassignment time and the lag before/after.

☐ **T4 — Break ordering with partitions, then recover from a poison message** *(≈75 min, advanced)*

- **Goal:** reproduce two classic production failures.
- **Steps:**
  1. Reproduce guide step 7 and list which keys moved partition.
  2. Write a consumer that throws on records whose value is `BAD`; produce one `BAD` into the middle of a partition and show the group stuck on it with growing lag.
  3. Change the consumer to retry 3 times, then publish the record to `orders.dlq` and commit past it.
- **Done when:** lag returns to 0, the `BAD` record is readable in `orders.dlq`, and you can state how a key's ordering guarantee was affected by adding partitions.

☐ **T5 — Replay with offset reset and make the handler idempotent** *(≈60 min, advanced)*

- **Goal:** design for at-least-once delivery.
- **Steps:**
  1. Write a consumer that "sends an email" by appending `order_id` to a file, committing after processing. Run it to completion.
  2. Reset the group's offsets to earliest and run it again: observe duplicates.

3. Add a deduplication check (a set in SQLite or a PostgreSQL table with a unique key) before the side effect, and run the replay again.

- **Done when:** the second replay produces zero new "emails", and the dedup store contains exactly one row per `order_id`.

- [ ] **T6 — Prove it: choose the settings for a payments topic** *(≈45 min, advanced)*
  - **Goal:** justify durability and ordering decisions without the guide.
  - **Steps:**
    1. For a `payments` topic that must not lose or reorder a given customer's events, write the topic config (partitions, RF, `min.insync.replicas`, retention, cleanup policy), the producer config and the consumer config (commit strategy, offset reset), with a one-line reason for each.
    2. Explain what happens to the producer when one of three brokers fails, and when two fail.
    3. Apply the topic settings to your lab broker (the RF/ISR values you cannot test on one broker should be noted as untested) and demonstrate the ordering and the retention settings with `--describe`.
  - **Done when:** a teammate agrees with every setting, and you can describe the broker-failure behaviour precisely (`NotEnoughReplicas` errors when ISR drops below the minimum).

10 Self-check questions

1. What is the unit of ordering in Kafka, and how do you make sure all events for one order are processed in sequence?
2. A topic has 4 partitions and a consumer group has 6 consumers. What happens?
3. What do `acks=all`, replication factor 3 and `min.insync.replicas=2` give you together?
4. What does consumer lag measure, and what are three causes of growing lag?
5. Why can adding partitions to a topic break your application, and why can't you remove them?
6. Is Kafka "exactly once"? What do you need in the consumer?

ANSWERS

1. The partition. Use the order id as the record key so all of an order's records hash to the same partition (and keep the partition count stable).
2. Four consumers each get one partition; two sit idle. A group's parallelism is capped at the partition count.

3. An acknowledged write is on at least two replicas, so you can lose one broker without losing it. If fewer than two replicas are in sync, producers get errors instead of silently reduced durability.
4. `log-end-offset - committed offset` per partition: records waiting to be processed. Causes: slow or stuck consumers (poison message, slow downstream dependency), too few consumers or partitions for the throughput, or frequent rebalances.
5. The hash of a key mod partition count changes, so a key's new records can land in another partition and be processed out of order with old ones. Removing partitions would delete data and reorder everything, so it is not supported; you create a new topic and migrate.
6. Within Kafka (idempotent producer plus transactions with `read_committed`) it can be, but delivery to an external system is at-least-once. The consumer must be idempotent, for example by deduplicating on a unique business key.

11 Further reading

- Apache Kafka documentation (kafka.apache.org/documentation): *Design*, *Operations*, and the producer/consumer configuration reference.
- Jay Kreps, *The Log: What every software engineer should know about real-time data's unifying abstraction* (LinkedIn Engineering).
- *Kafka: The Definitive Guide* (Shapira, Palino, Sivaram, Petty), the standard book.
- *Designing Data-Intensive Applications* by Martin Kleppmann, chapter on stream processing.
- Netflix Technology Blog: articles on the Keystone data pipeline.
- Strimzi documentation (strimzi.io) for running Kafka on Kubernetes; Debezium documentation for change data capture from PostgreSQL.
- Related: RabbitMQ §13.9, Replication §13.4, Eventual Consistency §13.13, Distributed Locks and Queues §13.15.

13.9 RabbitMQ

Topic 9 of 15 | ~5 hours | Difficulty: Intermediate

1 What it is

RabbitMQ is an open-source **message broker**: a server that sits between programs that produce work ("publishers") and programs that do the work ("consumers"). A publisher hands a message to the broker and moves on; the broker holds it safely until a consumer is ready. It implements **AMQP 0-9-1** (Advanced Message Queuing Protocol) natively and supports others (MQTT, STOMP, AMQP 1.0) through plugins.

What makes RabbitMQ different from "just a queue" is **routing**. Publishers never write to a queue directly. They publish to an **exchange**, and the exchange decides — using **bindings** and a **routing key** — which **queues** receive a copy. Consumers read from queues and **acknowledge** (ack) each message when they have really finished with it. If a message cannot be processed it can be rejected and sent to a **dead-letter queue (DLQ)** instead of being lost or looping forever.

```
publisher ⟶ [exchange] --(binding + routing key)⟶ [queue] ⟶ consumer
```

2 Why it matters in DevOps

- **Decoupling and burst absorption:** the web tier answers the user in milliseconds and queues the slow work (emails, PDF generation, image resizing). A traffic spike makes the queue longer, not the site slower.
- **Reliability under failure:** with acks and durable queues, a crashed worker means the message is redelivered, not lost. The flip side is **at-least-once delivery**, so consumers must be idempotent (see Idempotency §13.14).
- **Operational hazards you get paged for:** a queue growing without bound, memory/disk alarms that block publishers, poison messages that crash-loop a consumer, unacked messages piling up behind a stuck worker, and network partitions.

- **Choosing the right tool:** teams that use a log (Kafka) where a work queue is enough, or the reverse, pay for it in complexity. Knowing the difference is a senior-level skill (see section 5 and Kafka §13.8).

3 Real-world system example

RabbitMQ is widely used as the task-distribution backbone in web back ends. Two publicly documented patterns:

- **Celery (Python task queue)** supports RabbitMQ as its recommended broker: your web app calls `task.delay(...)`, Celery publishes a message to RabbitMQ, and a pool of Celery workers consume and ack it. Django and Flask shops use exactly this for background jobs.
- **OpenStack** services use RabbitMQ as the default transport for their RPC and notification messaging between components such as Nova and Neutron (via the `oslo.messaging` library).

RabbitMQ itself is maintained by VMware/Broadcom (Tanzu) and runs on Erlang/OTP, whose lightweight processes are why a single broker node can handle very large numbers of connections and queues. Modern RabbitMQ (3.8+) offers **quorum queues**, a replicated queue type built on the Raft consensus algorithm, which is the recommended type for data you cannot lose; the old "mirrored classic queues" are deprecated in favour of them.

In our running example, `shop-api` publishes `order.created` events. A worker sends the confirmation email, another updates inventory — each with its own queue bound to the same exchange.

4 Diagram

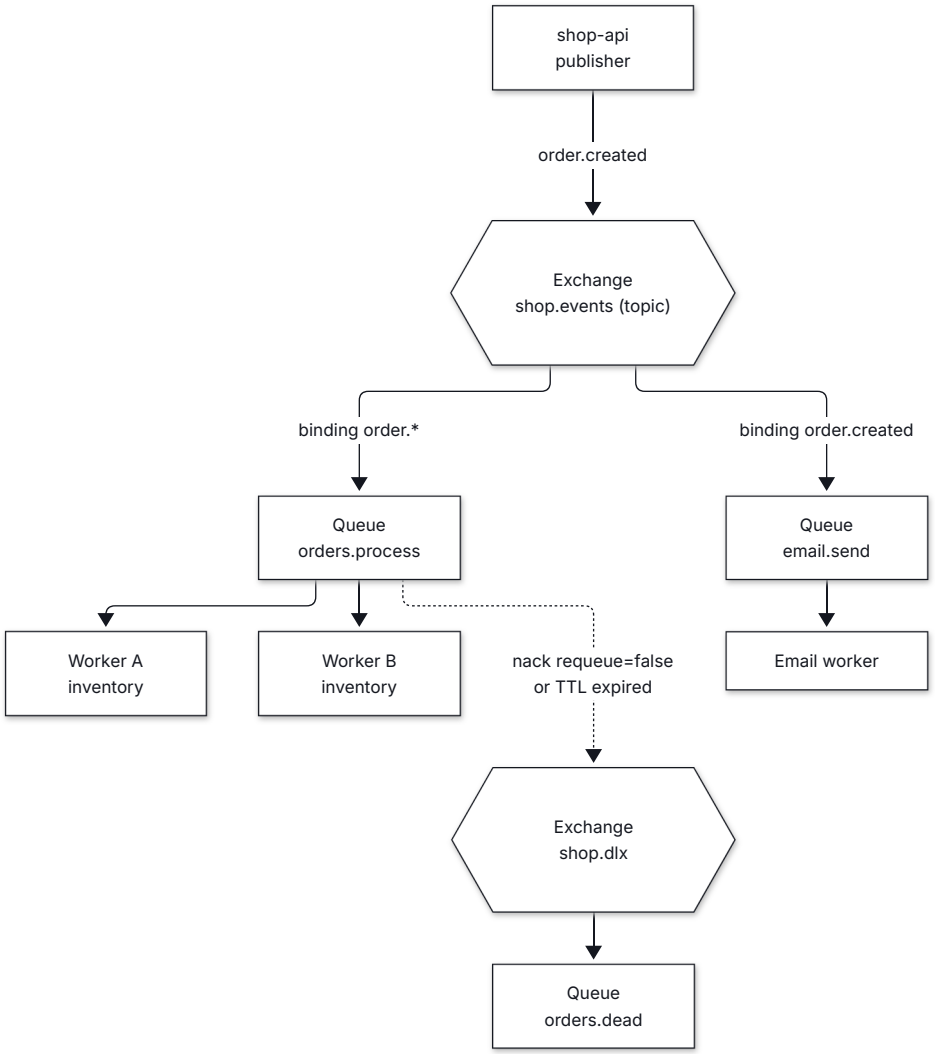


Figure 13.9-1

The life of one message, with acknowledgements and a crash:

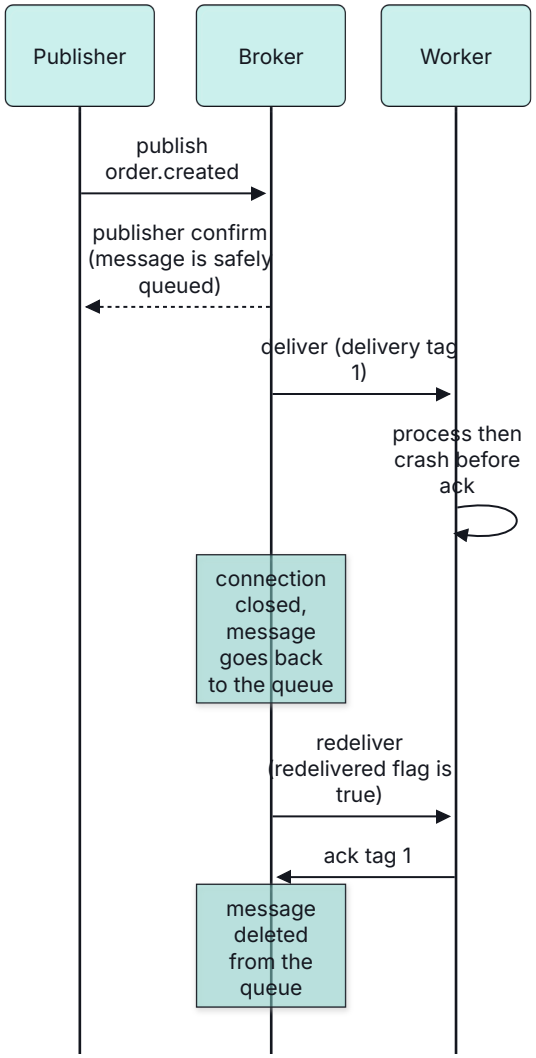


Figure 13.9-2

5 How it works

Exchange types

Exchange type	Routing rule	Typical use
`direct`	routing key must equal the binding key exactly	send a job to one specific queue
`fanout`	ignores the key, copies to every bound queue	broadcast (cache invalidation, DLX)
`topic`	key is dot-separated words, binding patterns use `*` (one word) and `#` (zero or more)	`order.*`, `*.eu.#` event routing
`headers`	matches on message headers instead of the key	rare
default (`""`)	built-in direct exchange; routing key = queue name	quick "send to queue X"

If an exchange has no matching binding the message is **dropped silently** unless the publisher sets the `mandatory` flag, in which case it is returned. This is a classic cause of "messages vanish".

Queues

- **Quorum queues** (`x-queue-type: quorum`): replicated across nodes with Raft, durable by design. Use for important data.
- **Classic queues**: single node, simplest, fastest, fine for transient or low-value work.
- **Streams**: an append-only log type (Kafka-like, non-destructive reads) added in 3.9 for replay and fan-out at scale.
- Properties: `durable` (queue survives a restart), `exclusive`, `auto-delete`, plus arguments such as `x-message-ttl`, `x-max-length`, `x-dead-letter-exchange`. Prefer setting TTL/limits/DLX through **policies** so you can change them without redeclaring queues.

Acknowledgements and reliability

Mechanism	Who	Protects against
Publisher confirms	broker to publisher	message lost between publisher and broker
Durable queue + persistent message (`delivery_mode=2`)	publisher/declarer	broker restart losing messages (non-quorum queues need both)
Consumer ack (`basic_ack`)	consumer to broker	worker crash mid-processing
nack / reject with `requeue=false`	consumer	poison messages: route to the DLX

Mechanism	Who	Protects against
Prefetch (`basic_qos`)	consumer	one slow worker hoarding unacked messages

Auto-ack ("fire and forget" delivery) means the broker deletes the message the moment it is sent. A worker crash then loses the message. Use manual acks for anything that matters.

Dead-lettering

A message is **dead-lettered** when it is rejected with `requeue=false`, its TTL expires, or the queue length limit is exceeded (and, for quorum queues, when the delivery limit is exceeded). It is republished to the queue's `x-dead-letter-exchange` with an `x-death` header recording the reason, the original queue and a count. A DLQ is just a queue you monitor and replay from — never let it grow unattended.

RabbitMQ versus Kafka

	RabbitMQ	Kafka (see Kafka §13.8)
Model	smart broker, "push" to consumers, per-message ack	dumb append-only log, consumers pull and track an offset
Message after consumption	deleted	retained for a configured time or size, replayable
Routing	rich: exchanges, topic patterns, headers	topic and partition only
Ordering	per queue, broken by multiple consumers or requeues	strict per partition
Scale-out of consumers	competing consumers on one queue, easy	limited to the partition count
Throughput	tens of thousands of msgs/s per node typical	designed for very high throughput
Retry / DLQ / delayed delivery	built in (DLX, TTL)	build it yourself with retry topics
Best for	task queues, RPC-style work, complex routing	event streaming, event sourcing, analytics pipelines, replay

Rule of thumb: **"do this job once" is a queue (RabbitMQ); "here is a fact that happened, many readers may want it, maybe later" is a log (Kafka).**

6 Step-by-step guide

Prerequisites: Docker, and Python 3.9+. No cost. The commands below were run against RabbitMQ 3.13 and pika 1.4.

1. **Start the broker with the management UI.**

```bash
docker run -d --name rmq -p 127.0.0.1:5672:5672 -p 127.0.0.1:15672:15672
  rabbitmq:3.13-management-alpine
sleep 20 && docker exec rmq rabbitmq-diagnostics -q ping
```

   Expected: `Ping succeeded`. The UI is at http://localhost:15672 (default login `guest` / `guest`, which only works from localhost — never expose it publicly; create a real user for anything else).

2. **Install a client library in a virtual environment.**

```bash
python3 -m venv venv && ./venv/bin/pip install pika
```

3. **Declare a topology and publish with confirms.** Save as `rmq_demo.py`:

```python
import pika

conn = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
ch = conn.channel()

# Dead-letter exchange + queue
ch.exchange_declare("shop.dlx", "fanout", durable=True)
ch.queue_declare("orders.dead", durable=True)
ch.queue_bind("orders.dead", "shop.dlx")

# Main topic exchange + quorum work queue wired to the DLX
ch.exchange_declare("shop.events", "topic", durable=True)
ch.queue_declare("orders.process", durable=True,
                 arguments={"x-dead-letter-exchange": "shop.dlx",
                            "x-queue-type": "quorum"})
ch.queue_bind("orders.process", "shop.events", routing_key="order.*")

ch.confirm_delivery()                       # publisher confirms
for i, key in enumerate(["order.created", "order.created", "order.cancelled",
  "user.signup"]):
    try:
        ch.basic_publish("shop.events", key, f"msg-{i}".encode(),
                         properties=pika.BasicProperties(delivery_mode=2),
                         mandatory=True)   # return unroutable messages
        print("published", key, "→ confirmed")
    except pika.exceptions.UnroutableError:
        print("published", key, "→ UNROUTABLE (returned)")

# Consume with manual acks; reject msg-1 to exercise the DLQ
while True:
```

```
    method, props, body = ch.basic_get("orders.process", auto_ack=False)
    if not method:
        break
    print("got", body.decode(), "redelivered=", method.redelivered)
    if body == b"msg-1":
        ch.basic_nack(method.delivery_tag, requeue=False)
        print("  nack(requeue=False) → dead-lettered")
    else:
        ch.basic_ack(method.delivery_tag)
        print("  ack")

method, props, body = ch.basic_get("orders.dead", auto_ack=True)
death = props.headers["x-death"][0]
print("DLQ has:", body.decode(), "reason:", death["reason"], "queue:", death["queue"])
conn.close()
```

BASH

```
./venv/bin/python rmq_demo.py
```

Expected output:

```
published order.created → confirmed
published order.created → confirmed
published order.cancelled → confirmed
published user.signup → UNROUTABLE (returned)
got msg-0 redelivered= False
  ack
got msg-1 redelivered= False
  nack(requeue=False) → dead-lettered
got msg-2 redelivered= False
  ack
DLQ has: msg-1 reason: rejected queue: orders.process
```

`user.signup` matched no binding, so `mandatory=True` made the broker return it instead of silently dropping it.

4. **Inspect the topology from the CLI.**

BASH

```
docker exec rmq rabbitmqctl list_queues name type messages consumers
docker exec rmq rabbitmqctl list_bindings | grep shop
```

You should see `orders.process` with type `quorum` and `orders.dead` with type `classic`.

5. **Prove redelivery after a crash.** Save as `rmq2.py`:

```python
import pika
c = pika.BlockingConnection(pika.ConnectionParameters("localhost")); ch = c.channel()
ch.queue_declare("demo.q", durable=True)
ch.basic_publish("", "demo.q", b"hello", pika.BasicProperties(delivery_mode=2))
m, p, b = ch.basic_get("demo.q"); print("first get redelivered=", m.redelivered, "(no
  ack, closing)")
c.close()                                   # simulates a worker crash: no ack was sent
c = pika.BlockingConnection(pika.ConnectionParameters("localhost")); ch = c.channel()
m, p, b = ch.basic_get("demo.q"); print("second get redelivered=", m.redelivered)
ch.basic_ack(m.delivery_tag)
```

```bash
./venv/bin/python rmq2.py
```

   Expected: `first get redelivered= False ...` then `second get redelivered= True`.

6. **Apply a policy instead of hard-coding arguments.**

```bash
docker exec rmq rabbitmqctl set_policy ttl-60s "^orders\." '{"message-ttl":60000}' --
  apply-to queues
docker exec rmq rabbitmqctl list_policies
```

   Every queue whose name starts with `orders.` now expires messages after 60 s; with a DLX configured they land in the DLQ.

7. **Watch the metrics that matter.**

```bash
docker exec rmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
curl -s -u guest:guest localhost:15672/api/overview | head -c 200; echo
```

   `messages_ready` growing means consumers are too slow or absent; `messages_unacknowledged` stuck high means a consumer is holding messages without finishing. The `rabbitmq_prometheus` plugin (enabled in the official 3.13 management image, serving port 15692 inside the container; add `-p 127.0.0.1:15692:15692` to scrape it) exposes both for Prometheus.

8. **Clean up.** `docker rm -f rmq`

7 Real-world scenario

(Illustrative.) At 02:10 an alert fires: "orders.process depth > 50,000 for 10 minutes". `shop-api` is healthy and customers are placing orders, but confirmation emails are hours late.

Investigation: the management UI shows `messages_ready` climbing, `messages_unacknowledged = 10` and two consumers — exactly the prefetch window times the worker count — and the consumer logs show the same message ID failing repeatedly. One malformed order (a `null` shipping country) throws an exception, the worker does `basic_nack(requeue=True)`, the broker redelivers it to the front of the queue instantly, and both workers are stuck in a tight loop on one poison message.

Fix: change the handler to `nack(requeue=False)` for non-retryable errors so the message goes to `shop.dlx`, and add a retry counter (read `x-death` count) so transient errors retry with a delay but permanent ones park in `orders.dead`. The queue drained in minutes; the DLQ held the one bad order for a human to fix and replay.

Lesson: requeue-on-failure is an infinite loop for deterministic errors. Always have a DLQ, alert on its depth, and alert on queue age, not just on queue length.

8 Common mistakes and best practices

Don't

- Use auto-ack for work you cannot afford to lose.
- `nack(requeue=True)` on every exception — it creates poison-message hot loops.
- Leave `prefetch` unlimited: one consumer can drain the whole queue into its memory while others sit idle.
- Let queues grow without a limit, TTL or alert; RabbitMQ will raise a memory/disk alarm and **block all publishers**.
- Use the default `guest` account or expose ports 5672/15672 to the internet.
- Assume exactly-once delivery. RabbitMQ gives at-least-once; duplicates will happen.
- Rely on message order when running several consumers or requeuing — order is only guaranteed for a single consumer without requeues.

Do

- Use quorum queues for important data, publisher confirms, `mandatory` publishing, durable exchanges/queues and persistent messages.
- Use manual acks, a sensible `prefetch` (start with 10-50, lower for slow jobs), and idempotent handlers.

- Configure a DLX and monitor DLQ depth and oldest-message age. Build a "replay from DLQ" tool before you need it.
- Use policies for TTL, length limits and DLX so you can change behaviour without redeploying.
- Put connections in a reconnect loop; connections drop during upgrades and partitions. Use one long-lived connection per process, with a channel per thread.
- Run an odd number (3 or 5) of nodes for quorum queues so a majority survives a node loss.

9 Tasks

- [ ] **T1 — Run the broker and the topic demo** *(≈25 min, beginner)*
  - **Goal:** publish through a topic exchange and see routing, an unroutable return and a dead-letter.
  - **Steps:**
    1. Start RabbitMQ with the Docker command from section 6 and install `pika`.
    2. Run `rmq_demo.py` unchanged.
    3. Open the management UI, find the `shop.events` exchange and its bindings.
  - **Done when:** the output matches section 6 step 3 and the UI shows `orders.process` bound with `order.*`.
- [ ] **T2 — Route with new bindings** *(≈25 min, beginner)*
  - **Goal:** add a second consumer queue and see fan-out through the exchange.
  - **Steps:**
    1. Declare queue `email.send` and bind it to `shop.events` with key `order.created`.
    2. Publish one `order.created` and one `order.cancelled`.
    3. Run `rabbitmqctl list_queues name messages` and explain the counts.
  - **Done when:** `email.send` holds 1 message and `orders.process` holds 2, and you can say why.
- [ ] **T3 — Competing consumers and prefetch** *(≈40 min, intermediate)*
  - **Goal:** observe fair dispatch with `basic_qos`.
  - **Steps:**
    1. Publish 20 messages to a queue; write a consumer using `basic_consume` that sleeps 1 s per message and acks after.
    2. Start two copies with `prefetch_count=1`, then repeat with `prefetch_count=20`.

3. Record which worker processed how many messages in each run.

- **Done when:** with prefetch 1 the load is split about evenly; with prefetch 20 the first worker grabs most of the batch.

- [ ] **T4 — Break it: poison message** *(≈40 min, intermediate)*
  - **Goal:** reproduce the requeue hot loop and fix it with a DLQ.
  - **Steps:**
    1. Write a consumer that raises on a message containing `"bad"` and calls `nack(requeue=True)`; publish one bad message and watch `redelivered=True` repeat.
    2. Change to `requeue=False` with the DLX configured; republish.
    3. Add a retry counter from the `x-death` header that dead-letters after 3 attempts.
  - **Done when:** the bad message ends in `orders.dead` with `x-death` count visible, and the queue is empty.

- [ ] **T5 — Kill the worker, keep the message** *(≈30 min, intermediate)*
  - **Goal:** prove at-least-once delivery and show why idempotency is mandatory.
  - **Steps:**
    1. Run a consumer that writes "processed " to a file, then sleeps 10 s before acking.
    2. `kill -9` it during the sleep, then start it again.
    3. Count how many times the file lists the same ID.
  - **Done when:** the same message ID appears twice in the file and you can explain which step would have to be idempotent.

- [ ] **T6 — Prove it: choose queue or log** *(≈45 min, advanced)*
  - **Goal:** justify RabbitMQ versus Kafka for three workloads without notes.
  - **Steps:**
    1. For (a) sending password-reset emails, (b) a click-stream feeding three analytics jobs plus replays, (c) an order workflow with per-customer delayed retries, pick a broker and write two sentences why.
    2. Re-do your T4 DLQ setup from memory on a fresh broker in under 15 minutes.
  - **Done when:** your written answer names ack/delete semantics, replay and routing as the deciding factors, and the fresh broker routes a rejected message to a DLQ.

10 Self-check questions

1. What is the difference between an exchange, a binding and a queue?

2. A publisher sends a message with a routing key that matches no binding. What happens, and how do you detect it?
3. Why does `nack(requeue=True)` sometimes make an outage worse?
4. Which failures do publisher confirms, durable queues and consumer acks each protect against?
5. Why are quorum queues preferred over classic queues for important data, and how many nodes do you run?
6. When would you pick Kafka over RabbitMQ?

ANSWERS

1. The exchange receives published messages and routes them; a binding is a rule (key or pattern) linking an exchange to a queue; the queue stores messages until a consumer acks them. Publishers talk to exchanges, consumers to queues.
2. By default it is dropped silently. Publishing with `mandatory=true` makes the broker return it (pika raises `UnroutableError` with confirms on); an alternate exchange is another option.
3. A deterministic failure is retried instantly and forever, occupying consumers and starving good messages. Use `requeue=false` plus a DLX and bounded retries.
4. Confirms: loss between publisher and broker. Durable queue plus persistent messages (or a quorum queue): broker restart. Consumer ack: worker crash mid-processing.
5. They replicate through Raft, so data survives a node failure and leadership fails over automatically; run 3 or 5 nodes so a majority remains.
6. When you need replay, multiple independent consumer groups reading the same history, very high throughput, strict per-key ordering or stream processing; for task queues with rich routing, per-message retry and delay, RabbitMQ is simpler.

11 Further reading

- RabbitMQ documentation (rabbitmq.com/docs): "AMQP 0-9-1 Model Explained", "Reliability Guide", "Quorum Queues", "Dead Letter Exchanges".
- *RabbitMQ in Depth* by Gavin M. Roy (Manning).
- Related topics: Kafka §13.8, Idempotency §13.14, Distributed Locks and Queues §13.15, and the circuit-breaker pattern in Circuit Breakers §12.14.

LAB NOTES What I ran, what I saw, what it means

13.10 Horizontal and Vertical Scaling

Topic 10 of 15 | ~4 hours | Difficulty: Intermediate

1 What it is

Vertical scaling ("scaling up") means giving one machine more resources: a bigger instance with more CPU cores, RAM, faster disks or network. **Horizontal scaling** ("scaling out") means adding more machines and spreading the work across them behind a load balancer, a connection router or a partitioning scheme.

Scaling up is easy and needs no code change, but it has a ceiling (the biggest machine your provider sells), a price curve that bends upward, and it is still one machine that can fail. Scaling out has almost no ceiling and gives redundancy, but only works if the work can actually be divided — which depends on whether the component is **stateless** (any instance can serve any request) or **stateful** (the instance holds data that others do not have).

2 Why it matters in DevOps

- **Capacity planning and cost:** choosing between "resize the instance" and "add three more" is a recurring on-call and budget decision.
- **Outages from hidden state:** an app that keeps sessions, uploads or counters in local memory or disk breaks the moment a second instance appears — users get logged out at random, files "disappear", counters disagree.
- **The database is the hard part:** web tiers scale out trivially; the primary database usually does not. Most real scaling work is moving load off the primary (caching, read replicas, queues) before ever sharding (see Partitioning and Sharding §13.11).
- **Autoscaling only works on stateless tiers.** Kubernetes HPAs and AWS Auto Scaling groups assume instances are interchangeable (see Auto Scaling §6.2 and High Availability and Scalability §0.8).

3 Real-world system example

- **Stack Overflow** has publicly described (in engineering blog posts and talks by its engineers) running a famously small number of powerful servers, including SQL Server with large RAM, and scaling up before out. It is the standard counter-example to "you must go distributed": a well-tuned monolith on strong hardware can serve enormous traffic.
- **Instagram** publicly documented running PostgreSQL with a large number of logical shards and read replicas as it grew; **GitHub** documented scaling MySQL with replicas and later partitioning by functional area; **Slack** and **YouTube** adopted **Vitess** to shard MySQL. The common arc: scale up → add caching and read replicas → split by function → shard.
- **Amazon RDS / Aurora** productizes both axes: change the instance class (vertical, brief downtime for a Multi-AZ failover), add up to 15 Aurora read replicas (horizontal reads), and Aurora storage grows automatically. See RDS and Aurora §6.10.
- **Kubernetes** treats stateless Deployments as horizontally scalable (HPA adds Pods) and offers StatefulSets for workloads that need stable identity and storage.

4 Diagram

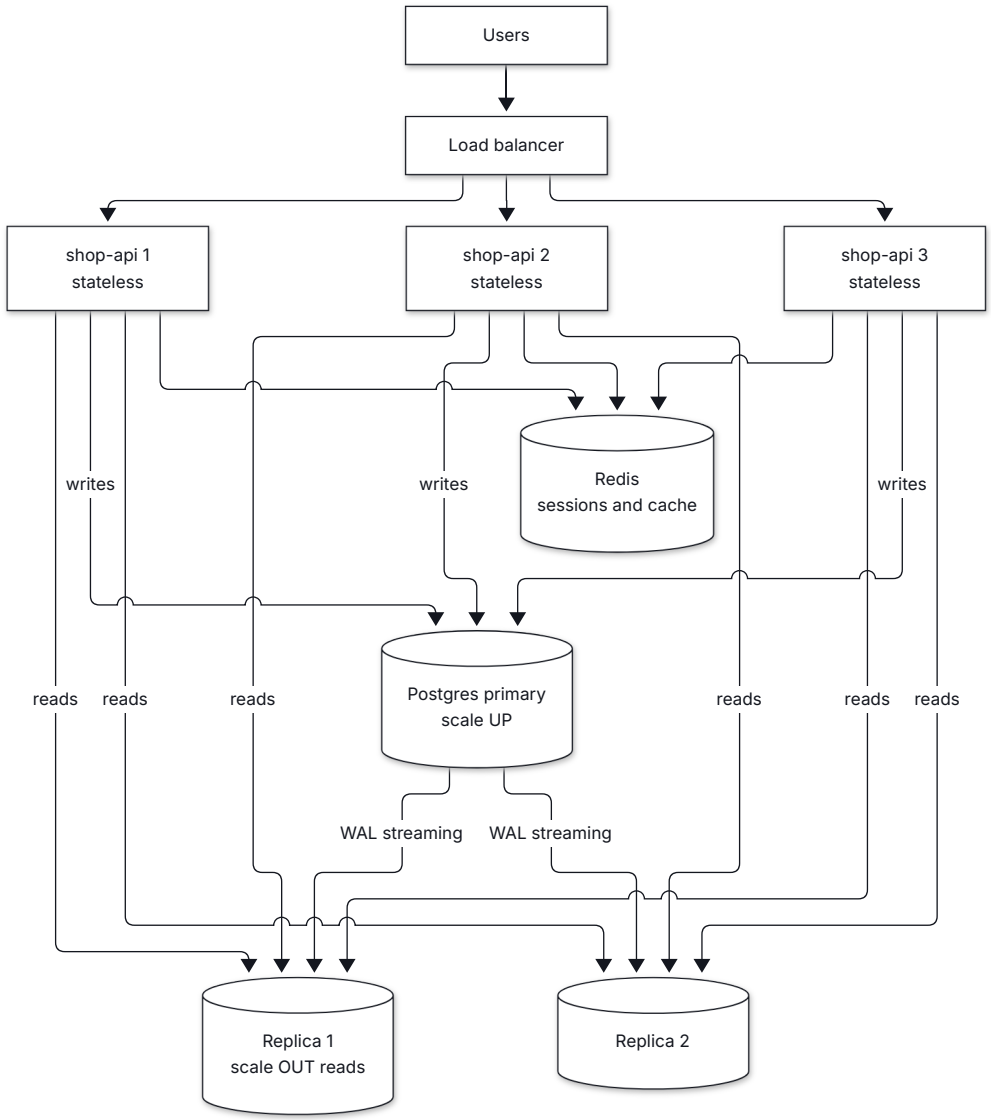


Figure 13.10-1

Where the decision comes from:

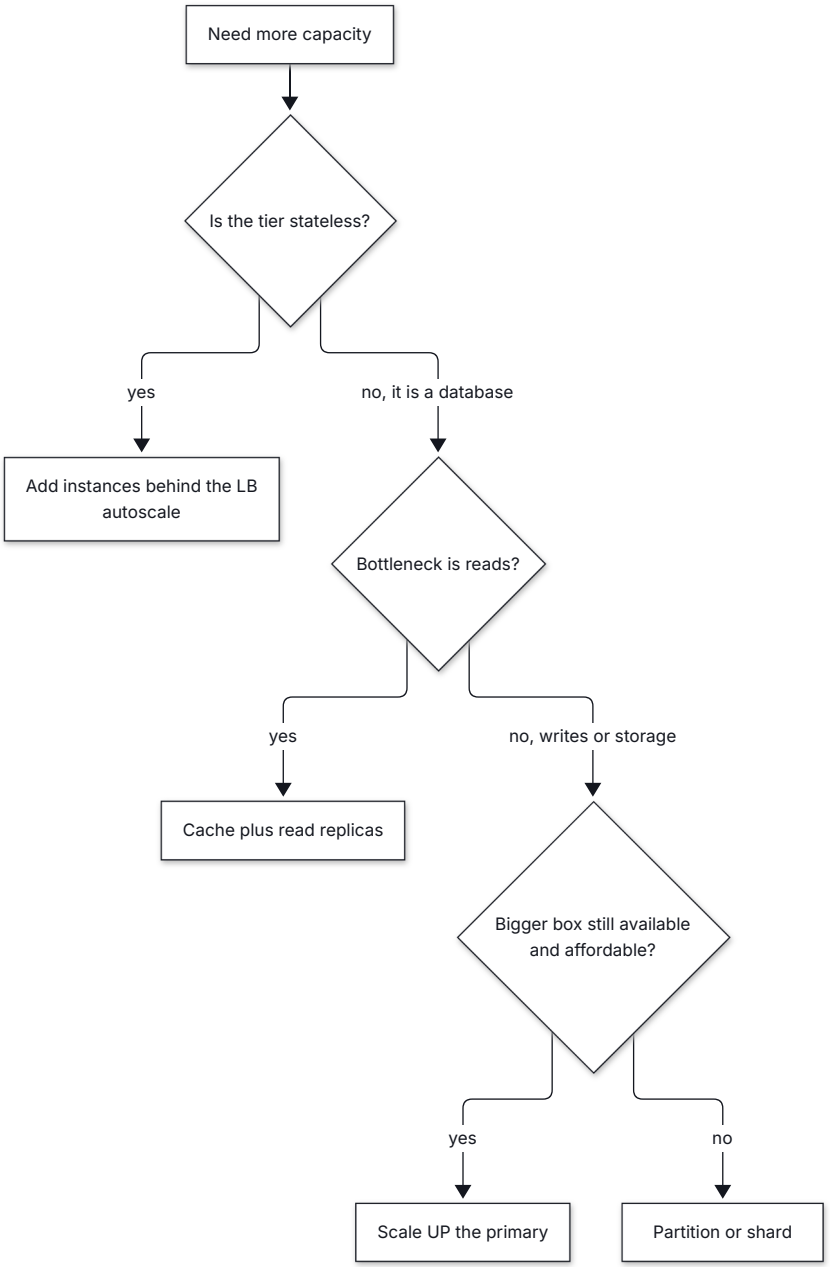


Figure 13.10-2

5 How it works

Comparison

	Vertical (up)	Horizontal (out)
How	bigger instance type	more instances
Code changes	none	often (externalize state, handle replica lag)
Ceiling	largest instance size	practically none, but coordination cost rises
Failure impact	single point of failure unless paired with a standby	one node lost = small capacity loss
Downtime to apply	usually a restart or failover	none if rolled gradually
Cost curve	superlinear at the top end	roughly linear, plus operational overhead
Best for	primary databases, quick wins, early stage	stateless web/API tiers, caches, read traffic, queues

Stateless vs stateful

A component is **stateless** when everything needed to answer a request arrives with the request or sits in a shared backing service. To make `shop-api` stateless:

State	Where it hides	Move it to
Login sessions	process memory or local files	Redis, or signed tokens (JWT)
File uploads	local disk	object storage such as S3
Counters, rate limits, locks	process variables	Redis (see Redis §13.7)
Background jobs	in-process timers or threads	a queue (RabbitMQ §13.9, Kafka §13.8)
Config and secrets	files edited on the host	environment, a config service, a secrets manager

Stateful systems (databases, Kafka brokers, Redis with data) scale out differently: by **replication** (copies for reads and failover, see Replication §13.4), by **partitioning** (each node owns a slice) or both.

Read replicas: what they do and do not fix

A read replica applies the primary's write-ahead log (WAL) and can serve `SELECT`s. This scales **read throughput**, not write throughput: every write still happens on the single primary and is replayed on every replica. Costs and caveats:

- **Replication lag:** a replica is slightly behind. A user who just saved a profile and is routed to a replica may not see it (the *read-your-writes* problem, see Eventual Consistency §13.13).
- **Routing:** the application (or a proxy such as PgBouncer plus HAProxy, or RDS Proxy / Aurora reader endpoints) must send writes to the primary and reads to replicas. Reads that must be fresh (right after a write, inside a transaction, checkout flows) stay on the primary.
- **Long queries on replicas** can conflict with WAL replay (`max_standby_streaming_delay`, `hot_standby_feedback` trade-offs).
- Each replica adds primary load for WAL shipping, and connection counts multiply (see Database Connection Pooling §13.6).

Scaling limits to keep in mind

- **Amdahl's law:** if 10 % of a request is serial (a lock, a single DB row), 10x more servers can never give more than 10x speed-up, and in practice far less.
- **Hot rows and global locks** do not scale out: a single `counter` row updated by every request caps throughput regardless of fleet size.
- **Coordination cost:** more nodes mean more connections to the database, more cache misses after deploys and more failure modes.

6 Step-by-step guide

Prerequisites: Docker and Python 3.9+; no cost. You will see the stateless/stateful problem in a minute, then fix it.

1. **Start Redis.**

```bash
docker run -d --name scale-redis -p 127.0.0.1:6379:6379 redis:7-alpine
python3 -m venv venv && ./venv/bin/pip install redis
```

2. **Write a tiny app with a switchable hit counter.** Save as `app.py`:

```python
import os, sys, json
from http.server import BaseHTTPRequestHandler, HTTPServer
import redis

PORT = int(sys.argv[1])
MODE = os.environ.get("MODE", "memory")          # memory | redis
r = redis.Redis(decode_responses=True, port=int(os.environ.get("REDIS_PORT", 6379)))
  if MODE == "redis" else None
local = {"hits": 0}
```

```
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if r:
            hits = r.incr("cart:hits")
        else:
            local["hits"] += 1
            hits = local["hits"]
        body = json.dumps({"instance": PORT, "mode": MODE, "hits": hits}).encode()
        self.send_response(200); self.send_header("Content-Type", "application/json");
  self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass

HTTPServer(("127.0.0.1", PORT), H).serve_forever()
```

3. **Run two instances with in-memory state and alternate requests (a poor man's round robin).**

```bash
MODE=memory ./venv/bin/python app.py 9001 &
MODE=memory ./venv/bin/python app.py 9002 &
sleep 1
for i in 1 2 3 4; do curl -s localhost:$((9001 + i % 2)); echo; done
kill %1 %2
```

Output similar to:

```
{"instance": 9002, "mode": "memory", "hits": 1}
{"instance": 9001, "mode": "memory", "hits": 1}
{"instance": 9002, "mode": "memory", "hits": 2}
{"instance": 9001, "mode": "memory", "hits": 2}
```

Four requests, but each instance thinks it has seen two. Add a third instance and it gets worse. This is the bug that appears the day you scale out an app that keeps state in memory.

4. **Externalize the state and repeat.**

```bash
MODE=redis ./venv/bin/python app.py 9001 &
MODE=redis ./venv/bin/python app.py 9002 &
sleep 1
for i in 1 2 3 4; do curl -s localhost:$((9001 + i % 2)); echo; done
kill %1 %2
```

Output: `hits` now reads 1, 2, 3, 4 regardless of which instance answered. The instances are interchangeable, so you can add a third or kill one without anyone noticing.

5. **Measure a vertical limit.** Use Docker CPU limits to emulate instance sizes and benchmark Postgres with `pgbench`:

```bash
nproc
for c in 1 2 4; do
  docker rm -f pgb >/dev/null 2>&1
  docker run -d --name pgb --cpus=$c -e POSTGRES_PASSWORD=pw postgres:16-alpine
  >/dev/null; sleep 8
  docker exec pgb pgbench -U postgres -i -s 5 postgres >/dev/null 2>&1
  echo "cpus=$c"; docker exec pgb pgbench -U postgres -c 8 -j 4 -T 10 postgres | grep
  tps
done
docker rm -f pgb
```

   On a 4-core sandbox the output was similar to `cpus=1` about 1,200 tps and `cpus=2` about 2,600 tps (your numbers will differ). Note where throughput stops growing as you add CPUs — past that point the limit is disk, locks or the client, not cores.

6. **Sketch read/write splitting in application code.**

```python
import psycopg2
primary = psycopg2.connect("host=db-primary dbname=shop user=app")
replica = psycopg2.connect("host=db-replica dbname=shop user=app")

def conn_for(readonly: bool, just_wrote: bool = False):
    return replica if readonly and not just_wrote else primary
```

   The `just_wrote` escape hatch (pin the session to the primary for a few seconds after a write) is the simplest read-your-writes protection. The capstone builds a real primary and replica and measures their lag.

7. **Clean up.** `docker rm -f scale-redis`

7 Real-world scenario

(Illustrative.) `shop-api` has run happily on one `m5.large`-class VM. A marketing email lands; CPU hits 100 % and p95 latency goes from 120 ms to 6 s. The team does the quick fix first: resize to a bigger instance (vertical), which works in ten minutes of downtime.

Next quarter they add two more app servers behind an ALB. Within an hour support tickets arrive: "I keep getting logged out" and "my cart empties". **Investigation:** sessions are stored in each process's memory, and the load balancer spreads requests. **Fix:** sessions move to Redis, uploads to S3. The app tier now autoscales on CPU. Then the database primary becomes the bottleneck at 85 % CPU, driven by 90 % read traffic from product pages. They add a Redis cache for hot product reads and two read replicas for reporting and search, keeping checkout on the primary.

Lesson: scale up for speed, scale out for resilience, and make state explicit first. Fix the shared bottleneck (the database) with caching and replicas before reaching for sharding.

8 Common mistakes and best practices

Don't

- Scale out an app before removing in-memory or local-disk state.
- Use sticky sessions as a permanent fix; they hide the state problem and make deploys and failures lumpy.
- Add read replicas for a write-bound database, or read from a replica right after the same user wrote.
- Assume more app instances are free: each opens database connections, and 50 instances times 20 connections can exhaust `max_connections`.
- Jump to sharding because "that is what big companies do" while a bigger instance, an index or a cache would do.

Do

- Treat app servers as cattle: immutable images, externalized state, health checks, graceful shutdown.
- Scale the primary database up first (cheap, simple), and plan the failover or maintenance window it needs.
- Offload reads with caching and replicas; route freshness-sensitive reads to the primary.
- Load-test to find the real bottleneck (CPU, I/O, locks, connections) before choosing a scaling direction.
- Put a connection pooler between a large app fleet and the database.
- Keep a documented headroom target (for example, scale at 60-70 % sustained CPU) and review it with capacity planning §14.9.

9 Tasks

- [ ] **T1 — See the state bug** *(≈20 min, beginner)*
  - **Goal:** reproduce divergent in-memory state across two instances.
  - **Steps:**
    1. Create `app.py` from section 6 and run two instances with `MODE=memory`.
    2. Send 6 alternating requests with `curl`.
  - **Done when:** the two instances report independent `hits` counters and you can explain why a load balancer breaks this.
- [ ] **T2 — Fix it with Redis** *(≈20 min, beginner)*
  - **Goal:** make the instances interchangeable.
  - **Steps:**
    1. Start Redis and run both instances with `MODE=redis`.
    2. Send 6 requests; kill instance 9001 midway and continue against 9002.
  - **Done when:** `hits` increases monotonically across instances and survives killing one.
- [ ] **T3 — Add a third instance and a tiny load balancer** *(≈40 min, intermediate)*
  - **Goal:** scale out horizontally behind real round-robin.
  - **Steps:**
    1. Write an Nginx `upstream` block with the three ports, run Nginx in Docker with `--network host` (or use `host.docker.internal` as the upstream host).
    2. Fire 30 requests at Nginx and count responses per `instance`.
  - **Done when:** roughly one third of responses come from each instance and the Redis counter reads 30.
- [ ] **T4 — Break it: connection exhaustion** *(≈40 min, intermediate)*
  - **Goal:** feel how horizontal scaling can overload a stateful tier.
  - **Steps:**
    1. Run a local Postgres with `max_connections=20`.
    2. Start 10 processes that each open a pool of 5 connections.
    3. Observe the `too many clients already` error, then fix it with PgBouncer or smaller pools (see Database Connection Pooling §13.6).
  - **Done when:** the error reproduces, and after the fix all 10 workers run with `SELECT count(*) FROM pg_stat_activity` below the limit.
- [ ] **T5 — Measure a scale-up curve** *(≈45 min, intermediate)*

- **Goal:** produce evidence of diminishing returns.
- **Steps:**
  1. Run `pgbench` against a Postgres container at `--cpus=1`, `2`, `4` (as far as your machine allows).
  2. Record transactions per second for each, and plot or tabulate.
- **Done when:** you have a three-row table and a one-sentence explanation of where and why the curve flattens.

☐ **T6 — Prove it: scaling plan** *(≈45 min, advanced)*

- **Goal:** write a scaling plan for `shop-api` without notes.
- **Steps:**
  1. List every piece of state in the app and where each should live.
  2. State, for each tier (web, cache, DB reads, DB writes), whether you would scale up or out first and what metric triggers it.
  3. Explain to a peer why read replicas will not fix a write bottleneck.
- **Done when:** your written plan covers all four tiers with a metric and a threshold each, and your peer cannot find a stateful component you missed.

10 Self-check questions

1. Give two reasons to scale a database up before scaling it out.
2. Name three kinds of hidden state that stop an app from scaling horizontally, and where each should go.
3. What does a read replica scale, and what does it not?
4. A user updates their address and the page immediately shows the old one. What probably happened, and how do you mitigate it?
5. Why can adding more app servers make the database fall over?

ANSWERS

1. It is simple (no code or data-model change) and often cheaper in engineering time; sharding adds permanent complexity (cross-shard queries, resharding) that you should defer until a single node truly cannot cope.
2. Sessions (Redis or signed tokens), uploaded files (object storage), counters/locks/rate limits (Redis), in-process jobs (a queue), local config (a config service or env).
3. It scales read throughput and provides failover candidates; it does not scale writes, which all go through the primary and are replayed on every replica.

4. The read hit a replica that had not yet replayed the write (replication lag). Pin that user's reads to the primary briefly after a write, read from primary in that flow, or wait for the replica to reach the write's LSN.
5. Each instance holds its own connection pool, so connections multiply with the fleet; use a pooler, smaller pools, and raise capacity deliberately.

11 Further reading

- *Designing Data-Intensive Applications* by Martin Kleppmann, chapters on replication and partitioning.
- *Site Reliability Engineering* (Google), chapters on load balancing and handling overload.
- The Twelve-Factor App (12factor.net), especially "Processes" and "Disposability".
- PostgreSQL documentation, "High Availability, Load Balancing, and Replication".
- Related: Replication §13.4, Partitioning and Sharding §13.11, Scalability architecture §19.1.

13.11 Partitioning and Sharding

Topic 11 of 15 | ~5 hours | Difficulty: Advanced

1 What it is

Partitioning means splitting one large logical table into smaller physical pieces ("partitions") that the database manages as a single table. In PostgreSQL this is *declarative partitioning*: you declare `PARTITION BY RANGE | LIST | HASH`, and rows are routed into child tables automatically. All partitions normally live on **one server**.

Sharding is partitioning *across servers*: the data is split by a **shard key** so each database node ("shard") owns a subset of the rows and no single node holds everything. The application, a proxy or the database itself must route each query to the right shard. Sharding is horizontal scaling for writes and storage — the thing read replicas cannot give you (see Horizontal and Vertical Scaling §13.10).

Same idea, different scope: partitioning helps one machine manage a big table (faster scans, cheap retention); sharding lets a dataset exceed what one machine can hold or write.

2 Why it matters in DevOps

- **Retention and maintenance:** dropping a month-old partition is instant, while `DELETE`-ing 200 million rows bloats the table, generates huge WAL and keeps `VACUUM` busy for hours (see MVCC §13.3).
- **Query speed:** with **partition pruning**, a query for "last week" touches one partition instead of the whole table.
- **Capacity ceiling:** when the primary is at its write limit and the biggest instance is not enough, sharding is the way out — and one of the most expensive changes you can make to a system. A wrong shard key is painful to undo.
- **Operations:** every shard needs backups, monitoring, upgrades and failover; ten shards is ten times the on-call surface. Resharding under live traffic is a classic high-risk project.

3 Real-world system example

- **Instagram** publicly described sharding PostgreSQL into many *logical* shards mapped onto fewer physical servers, embedding the shard ID inside generated 64-bit IDs so a row's location can be derived from its ID. Many logical shards on few machines makes later rebalancing a data move, not a re-hash.
- **Notion** wrote publicly about sharding its monolithic Postgres database by workspace, since most queries are scoped to one workspace.
- **Vitess** (created at YouTube, now a CNCF project) is a sharding layer for MySQL used by companies such as Slack; it handles routing (VTGate) and online resharding.
- **Redis Cluster** splits the key space into 16,384 *hash slots* assigned to nodes; **Cassandra** and **DynamoDB** hash a partition key to place data on nodes; **Kafka** partitions topics by key (see Kafka §13.8).
- **Citus** (open source Postgres extension, also offered by Microsoft as Azure Cosmos DB for PostgreSQL) distributes tables across worker nodes by a distribution column.
- **Amazon DynamoDB** is the hot-key cautionary tale: its docs devote guidance to avoiding "hot partitions" by choosing high-cardinality partition keys.

4 Diagram

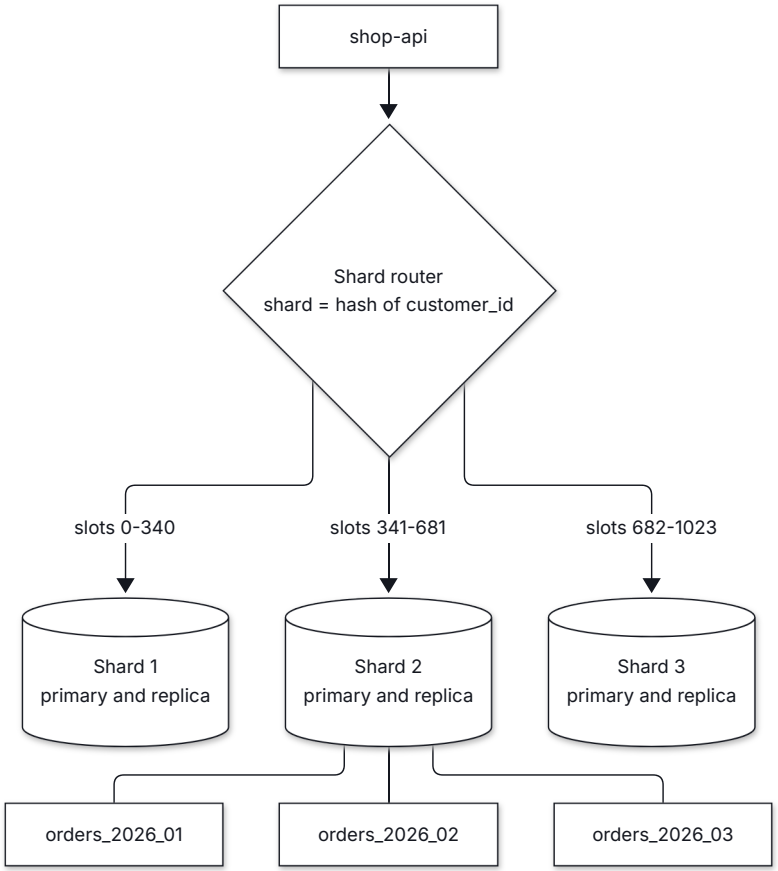


Figure 13.11-1

Each shard may itself use table partitioning (bottom of the diagram): sharding spreads across servers, partitioning organizes within one.

Why modulo hashing makes resharding painful, and slot maps fix it:

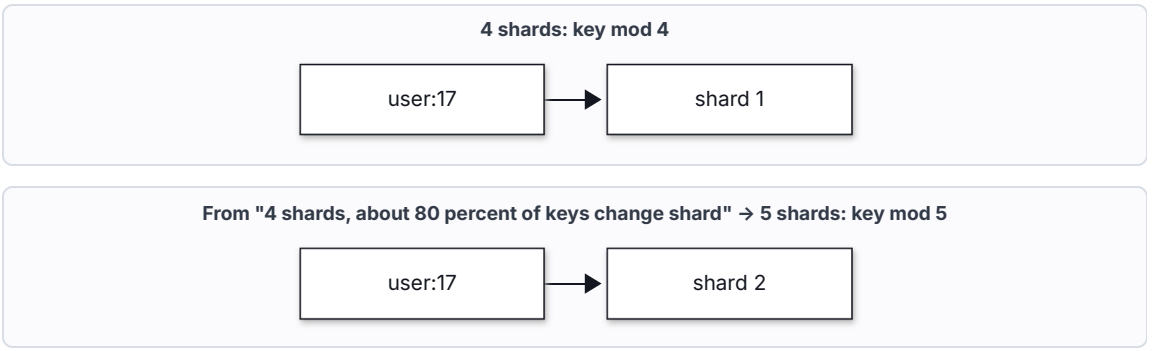


Figure 13.11-2

5 How it works

Partitioning strategies (PostgreSQL)

Strategy	Rule	Good for	Watch out
`RANGE`	value ranges (dates, ids)	time-series, logs, orders; retention by `DROP`	must create future partitions in advance (cron, `pg_partman`)
`LIST`	explicit values	region, tenant tier	uneven sizes
`HASH`	`MODULUS m REMAINDER r`	spreading load evenly when no natural range	no pruning for range queries; changing `m` rewrites data

Rules that bite:

- The **partition key must be part of every unique/primary key**: `PRIMARY KEY (id, created_at)`, not `(id)`. A global unique constraint across partitions does not exist.
- **Pruning only happens when the query filters on the partition key.** Without it, the planner scans every partition (often slower than the unpartitioned table).
- Too many partitions (thousands) slow planning and increase memory. Aim for tens to low hundreds.
- A `DEFAULT` partition catches rows outside all ranges — and blocks adding new overlapping partitions until it is checked. Monitor it, do not let it become a dumping ground.
- Detach safely: `ALTER TABLE ... DETACH PARTITION ... CONCURRENTLY` (PG 14+) avoids long locks on busy tables.

Choosing a shard key

A good shard key has **high cardinality**, is **present in most queries**, and spreads **both data and traffic** evenly.

Candidate key	Verdict for `shop-api`
`customer_id`	Good: most queries are per customer, high cardinality. Cross-customer reports need scatter-gather.
`created_at`	Bad as a shard key: all new writes hit the newest shard (a **hotspot**). Fine as a *partitioning* key within a shard.
`country`	Bad: a handful of values, wildly uneven.
`order_id` (random UUID)	Even spread, but orders of one customer scatter across all shards.
auto-increment `id` with range sharding	Monotonic: the last shard takes every insert.

Hotspots come from skew (one celebrity customer, one huge tenant), monotonic keys, and popular items. Mitigations: pick a better key, add a salt/suffix to split a hot key (`user:7#0..#9`), give big tenants their own shard (directory-based lookup), cache hot reads.

Routing models

Model	How the shard is found	Resharding
Modulo `hash(key) % N`	computed	terrible: changing N moves most keys
Fixed logical slots (Redis Cluster 16,384 slots, Instagram logical shards)	`hash(key) % slots`, slot-to-node map	move whole slots between nodes
Consistent hashing with virtual nodes (Cassandra, Dynamo-style)	position on a ring	adding a node moves about `1/N` of the keys
Directory / lookup table	table maps tenant to shard	flexible: move one tenant at a time, but the directory is a critical dependency
Range sharding	key ranges per shard (e.g. Spanner/CockroachDB style, splitting ranges automatically)	split and move ranges; watch monotonic-key hotspots

What sharding costs you

- **Cross-shard queries** (joins, aggregates, global secondary indexes) become scatter-gather or are forbidden.

- **Cross-shard transactions** need two-phase commit or sagas; most designs avoid them by keeping a tenant on one shard.
- **Global uniqueness** (emails, order numbers) needs an ID scheme (snowflake-style IDs, shard-prefixed IDs) or a central allocator.
- **Operational multiplication:** schema migrations, backups (Backups and PITR §13.5) and failover per shard.

Resharding, the safe way

Typical online resharding: (1) create new shards, (2) start copying the data for the moving slots while logging changes (logical replication or CDC), (3) catch up, (4) briefly pause writes for the moving slot, verify, flip the routing map, (5) resume and delete the old copy later. Tools like Vitess `MoveTables` / `Reshard` and Citus shard rebalancing automate this; doing it by hand is where outages happen.

6 Step-by-step guide

Prerequisites: Docker; Python 3 for the resharding experiment. No cost.

1. **Start Postgres.**

```bash
docker run -d --name pgp -e POSTGRES_PASSWORD=pw postgres:16-alpine
sleep 8 && docker exec -it pgp psql -U postgres
```

2. **Create a range-partitioned `orders` table and load data.** Paste into `psql`:

```sql
CREATE TABLE orders (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  customer_id int         NOT NULL,
  created_at  timestamptz NOT NULL,
  total_cents int         NOT NULL,
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);

CREATE TABLE orders_2026_01 PARTITION OF orders FOR VALUES FROM ('2026-01-01') TO
  ('2026-02-01');
CREATE TABLE orders_2026_02 PARTITION OF orders FOR VALUES FROM ('2026-02-01') TO
  ('2026-03-01');
CREATE TABLE orders_2026_03 PARTITION OF orders FOR VALUES FROM ('2026-03-01') TO
  ('2026-04-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;

INSERT INTO orders (customer_id, created_at, total_cents)
SELECT (random()*1000)::int,
```

```
       timestamptz '2026-01-01' + (random()*89) * interval '1 day',
       (random()*10000)::int
FROM generate_series(1, 30000);
ANALYZE orders;

SELECT tableoid::regclass AS partition, count(*) FROM orders GROUP BY 1 ORDER BY 1;
```

Expected: three partitions with roughly 10,000 rows each (exact counts vary because the data is random); `orders_default` is empty and absent from the output.

3. **Prove partition pruning.**

```
SQL
EXPLAIN (COSTS OFF) SELECT sum(total_cents) FROM orders
WHERE created_at ⩾ '2026-02-10' AND created_at < '2026-02-20';
```

Expected: the plan scans only `orders_2026_02`:

```
 Aggregate
   →  Seq Scan on orders_2026_02 orders
        Filter: ((created_at ⩾ ...) AND (created_at < ...))
```

Now filter on a non-key column:

```
SQL
EXPLAIN (COSTS OFF) SELECT sum(total_cents) FROM orders WHERE customer_id = 42;
```

Expected: an `Append` node scanning **all four** partitions. Lesson: queries must include the partition key (or you need an index per partition, which Postgres creates for you if you `CREATE INDEX` on the parent).

4. **Retire old data instantly.**

```
SQL
ALTER TABLE orders DETACH PARTITION orders_2026_01;   -- add CONCURRENTLY on a busy
  production table
SELECT count(*) FROM orders;                           -- January is gone from the
  parent
DROP TABLE orders_2026_01;                             -- or archive it to cold
  storage first
```

This took milliseconds and generated almost no WAL, unlike `DELETE FROM orders WHERE created_at < '2026-02-01'`.

5. **See a hash-partition hotspot.**

```sql
CREATE TABLE events (user_id int NOT NULL, payload text) PARTITION BY HASH (user_id);
CREATE TABLE events_p0 PARTITION OF events FOR VALUES WITH (MODULUS 4, REMAINDER 0);
CREATE TABLE events_p1 PARTITION OF events FOR VALUES WITH (MODULUS 4, REMAINDER 1);
CREATE TABLE events_p2 PARTITION OF events FOR VALUES WITH (MODULUS 4, REMAINDER 2);
CREATE TABLE events_p3 PARTITION OF events FOR VALUES WITH (MODULUS 4, REMAINDER 3);
INSERT INTO events SELECT g, 'x' FROM generate_series(1,10000) g;
SELECT tableoid::regclass, count(*) FROM events GROUP BY 1 ORDER BY 1;     -- about
  2,500 each
INSERT INTO events SELECT 7, 'celebrity' FROM generate_series(1,20000);    -- one hot
  key
SELECT tableoid::regclass, count(*) FROM events GROUP BY 1 ORDER BY 1;     -- one
  partition now has ~22,000
```

Even a perfect hash cannot spread a single hot key. On shards, that one partition would be one overloaded server.

6. **Measure how painful modulo resharding is.** Save as `reshard.py` and run `python3 reshard.py`:

```python
import hashlib, bisect

def h(s): return int(hashlib.md5(s.encode()).hexdigest(), 16)
keys = [f"user:{i}" for i in range(10000)]

# 1) naive modulo: 4 → 5 shards
a = {k: h(k) % 4 for k in keys}; b = {k: h(k) % 5 for k in keys}
print("modulo 4 → 5:", sum(a[k] ≠ b[k] for k in keys) / len(keys))

# 2) fixed logical slots: move only enough slots to fill the new node
SLOTS = 1024
slot = {k: h(k) % SLOTS for k in keys}
old = {s: s % 4 for s in range(SLOTS)}
new = dict(old)
for s in range(SLOTS // 5): new[s] = 4
print("slot map 4 → 5:", sum(old[slot[k]] ≠ new[slot[k]] for k in keys) / len(keys))

# 3) consistent hashing with virtual nodes
def ring(nodes, vn=100):
    r = sorted((h(f"{n}#{i}"), n) for n in nodes for i in range(vn))
    return [x[0] for x in r], [x[1] for x in r]
def lookup(r, k):
    pts, names = r
    return names[bisect.bisect(pts, h(k)) % len(pts)]
r4, r5 = ring(range(4)), ring(range(5))
print("consistent hash 4 → 5:", sum(lookup(r4, k) ≠ lookup(r5, k) for k in keys) /
  len(keys))
```

Output similar to:

```
modulo 4 → 5: 0.796
slot map 4 → 5: 0.2021
consistent hash 4 → 5: 0.1939
```

Modulo moves about 80 % of keys; slot maps and consistent hashing move about 20 % (the ideal `1/5`).

7. **Clean up.** `docker rm -f pgp`

7 Real-world scenario

(Illustrative.) The `events` table of `shop-api` has reached 1.8 billion rows (900 GB). Nightly `DELETE` of 90-day-old rows takes six hours, runs `autovacuum` constantly and slows checkout queries; the replica falls behind every night.

Investigation: the deletes produce enormous WAL and dead tuples; indexes are larger than RAM. Nearly every query filters by a recent time range. **Fix:** migrate to `PARTITION BY RANGE (created_at)` with daily partitions: create the partitioned table, backfill with batched copies, dual-write briefly, swap names in one transaction, and run a cron job (or `pg_partman`) that creates tomorrow's partition and detaches partitions older than 90 days. Retention becomes a metadata operation; most queries prune to one or two partitions.

Two years later, write volume outgrows the largest instance. The team chooses `customer_id` as shard key, uses 1,024 logical slots over four servers, and gives the one wholesale customer that generates 15 % of traffic a dedicated shard through a directory entry.

Lesson: partition first (cheap, local, reversible-ish); shard only when a single node cannot hold or write the data, and choose the shard key from the query patterns.

8 Common mistakes and best practices

Don't

- Shard before exhausting indexing, caching, read replicas, partitioning and a bigger instance.
- Use a monotonic key (timestamp, auto-increment) as a range shard key — all new writes land on one shard.
- Shard with `hash(key) % N` and no slot or directory layer; you will be unable to add capacity gracefully.
- Forget to create future partitions; inserts then fail or fall into `DEFAULT`.

- Run queries without the partition key and assume partitioning "made it faster".
- Rely on cross-shard transactions or foreign keys; design so related rows share a shard.

Do

- Partition by time for append-heavy data and automate create/detach/drop (cron, `pg_partman`, a Kubernetes CronJob).
- Choose the shard key from your top ten queries; check cardinality and skew with real data before committing.
- Use many logical shards over few physical nodes so rebalancing means moving slots, not re-hashing.
- Put a tenant's data on one shard and keep a directory for exceptions ("whale" customers).
- Test resharding (and restoring one shard from backup) in staging before you need it.
- Monitor per-shard size, QPS and p99 latency to spot hotspots early; see Observability §10.1.

9 Tasks

- [ ] **T1 — Partition a table by month** *(≈30 min, intermediate)*
  - **Goal:** build and populate a range-partitioned table.
  - **Steps:**
    1. Run the SQL from section 6 steps 1-2.
    2. Check row distribution per partition with `tableoid::regclass`.
  - **Done when:** each of the three partitions holds about a third of the 30,000 rows.
- [ ] **T2 — Prove pruning and its absence** *(≈25 min, intermediate)*
  - **Goal:** use `EXPLAIN` to show when pruning happens.
  - **Steps:**
    1. Run both `EXPLAIN` queries from step 3.
    2. Create `CREATE INDEX ON orders (customer_id);` and re-run the second one with `SET enable_seqscan = off;`.
  - **Done when:** you can show one query scanning one partition and another scanning four, and the per-partition index names appear in the plan.
- [ ] **T3 — Automate partition lifecycle** *(≈45 min, intermediate)*
  - **Goal:** a script that creates next month's partition and detaches the oldest.
  - **Steps:**
    1. Write a SQL function or shell script that computes next month's bounds and runs `CREATE TABLE ... PARTITION OF`.

2. Make it idempotent (`IF NOT EXISTS`) and run it twice.
3. Add the detach step for partitions older than N months.

- **Done when:** running it twice creates exactly one new partition and `\d+ orders` lists the expected set.

- [ ] **T4 — Break it: insert outside every range** *(≈20 min, intermediate)*
  - **Goal:** see what happens without a matching partition.
  - **Steps:**
    1. `DROP TABLE orders_default` (after detaching), then `INSERT` a row dated `2027-01-01` .
    2. Re-create the default partition, insert again, then try to add `orders_2027_01` and read the error.
    3. Fix by moving the rows and re-adding.
  - **Done when:** you have seen the "no partition of relation found" error, and a successful `CREATE TABLE orders_2027_01 PARTITION OF ...` after clearing the default partition.

- [ ] **T5 — Compare resharding strategies** *(≈40 min, advanced)*
  - **Goal:** quantify key movement for each routing model.
  - **Steps:**
    1. Run `reshard.py` ; then change it to go from 4 to 8 shards.
    2. Add a hot key (50 % of requests for one user) and compute the load per shard under each model.
  - **Done when:** a table shows % keys moved (4 to 5 and 4 to 8) and per-shard load with the hot key, and you can state which model you would choose and why.

- [ ] **T6 — Prove it: design a shard plan** *(≈45 min, advanced)*
  - **Goal:** write a shard design for `shop-api` without the guide.
  - **Steps:**
    1. Choose a shard key, list the five queries that become scatter-gather or impossible, and name the routing model.
    2. Describe the zero-downtime steps to go from 1 to 4 shards.
    3. Explain what you do about the biggest customer and about global unique order numbers.
  - **Done when:** a reviewer can follow your document, and it addresses hotspots, resharding steps, cross-shard queries and ID generation.

10 Self-check questions

1. What is the difference between partitioning and sharding?
2. Why must the partition key be part of the primary key in PostgreSQL?
3. Why is `created_at` a good partitioning key but a poor shard key?
4. Why does `hash(key) % N` make adding a shard so expensive?
5. What is a hotspot and name two ways to mitigate one?
6. Which problems appear the moment you shard that did not exist before?

ANSWERS

1. Partitioning splits a table into pieces managed by one database (usually one server); sharding places pieces on different servers, each owning a subset of the data, with routing in front.
2. Unique indexes are enforced per partition; Postgres can only guarantee global uniqueness if the partition key is in the key, so rows with equal keys land in the same partition.
3. Time ranges let queries prune and let you drop old data cheaply, but as a shard key all current writes hit the newest shard.
4. Changing N changes the result for most keys (about 80 % moving from 4 to 5), so most data must be copied. Fixed slots, consistent hashing or a directory limit movement to about 1/N.
5. A node or partition receiving disproportionate traffic or data. Mitigate with a better key, salting a hot key, a dedicated shard for big tenants, caching, or splitting ranges.
6. Cross-shard joins and transactions, global uniqueness, scatter-gather queries, per-shard migrations/backups/failover, and resharding.

11 Further reading

- PostgreSQL documentation: "Table Partitioning" (in the DDL chapter), and the `pg_partman` extension.
- *Designing Data-Intensive Applications* by Martin Kleppmann, chapter on partitioning.
- Vitess and Citus documentation (vitess.io, citusdata.com) for production sharding.
- Amazon DynamoDB Developer Guide: "Partition key design".
- Instagram Engineering blog post "Sharding & IDs at Instagram".
- Related: Replication §13.4, Kafka §13.8, Eventual Consistency §13.13, Scalability §19.1.

13.12 Consistency, Availability and the CAP Theorem

Topic 12 of 15 | ~5 hours | Difficulty: Advanced

1 What it is

The **CAP theorem** says that when a distributed data store is cut in two by a network failure, it must choose between two properties for the requests that arrive during the failure:

- **Consistency (C)**, in the CAP sense, means **linearizability**: every read sees the most recent completed write, and the whole system behaves as if there were a single copy of the data and each operation happened atomically at one instant.
- **Availability (A)** means every request sent to a node that is still working receives a (non-error) response, even if that node cannot reach the others.
- **Partition tolerance (P)** means the system keeps operating even when messages between nodes are lost or delayed.

Eric Brewer proposed the idea around 2000; Seth Gilbert and Nancy Lynch published the formal proof in 2002. Networks do partition (a bad switch, a misconfigured firewall, a saturated link, a long GC pause that looks like a dead node), so "pick two of three" is misleading: **P is not optional, and the real choice is C or A *during a partition*.** When there is no partition, a well-built system can offer both.

PACELC (Daniel Abadi, 2012) fixes the part CAP ignores: *if* there is a **P**artition, choose **A** or **C**; **E**lse (normal operation), choose **L**atency or **C**onsistency. Even on a healthy network, keeping replicas strictly in sync costs round trips, so a store like a synchronously replicated database trades latency for consistency every day, not only during outages.

2 Why it matters in DevOps

- **Every replicated system you operate has already made this choice** (a Postgres standby, a Redis replica, a Kafka partition, an etcd cluster). If you do not know which side it chose, you cannot predict what it does at 3 a.m. when a switch fails.

- **Failover is a CAP decision.** Promoting a replica that is behind keeps the service available but loses acknowledged writes (see Replication §13.4); refusing to promote preserves data but means downtime.
- **"Highly available" claims are partial.** A cluster that needs a quorum (majority) stops accepting writes without one. That is correct behaviour, not a bug, but your runbooks and SLOs must expect it.
- **Application bugs hide here.** Reading from a lagging replica right after a write, or writing to both sides of a split, produces bugs that no unit test finds. See Eventual Consistency §13.13 for how to live with the weaker choice.

3 Real-world system example

- **GitHub, October 2018.** GitHub's public post-incident analysis describes a roughly 43-second loss of connectivity between its US East Coast network hub and its primary data center. Automated failover promoted MySQL primaries in another region while the original primaries had accepted writes that had not replicated, so the two sides diverged. GitHub chose to preserve data integrity and reconcile, which meant hours of degraded service (the report gives the total as just over 24 hours). It is a textbook case of choosing consistency over fast recovery after a partition.
- **etcd and Kubernetes.** etcd uses the Raft consensus algorithm and requires a majority of members to commit a write. Lose the majority and the Kubernetes API server can no longer write cluster state (a **CP** behaviour), although pods already running on nodes keep running. This is why etcd clusters have 3 or 5 members, spread across failure zones.
- **Amazon Dynamo and Apache Cassandra** favour availability: the 2007 Dynamo paper describes accepting writes on any reachable replica and reconciling later with vector clocks; Cassandra exposes **tunable consistency levels** per query (`ONE`, `QUORUM`, `ALL`). The managed **Amazon DynamoDB** service offers eventually consistent reads by default and strongly consistent reads as an option.
- **Google Spanner** is described by Google as externally consistent (strictly serializable) across regions using TrueTime, a clock API with bounded uncertainty. Brewer's own 2017 article explains that Spanner is technically CP but runs on a private, carefully engineered network so that partitions are rare enough for very high availability in practice.
- **Jepsen** (Kyle Kingsbury's testing project) publishes analyses where real databases are tested under injected partitions; they are the best public source of "what a database claims versus what it does".

4 Diagram

What a partition forces a replicated register to do. Node A and node B each hold a copy of `balance`; the link between them fails:

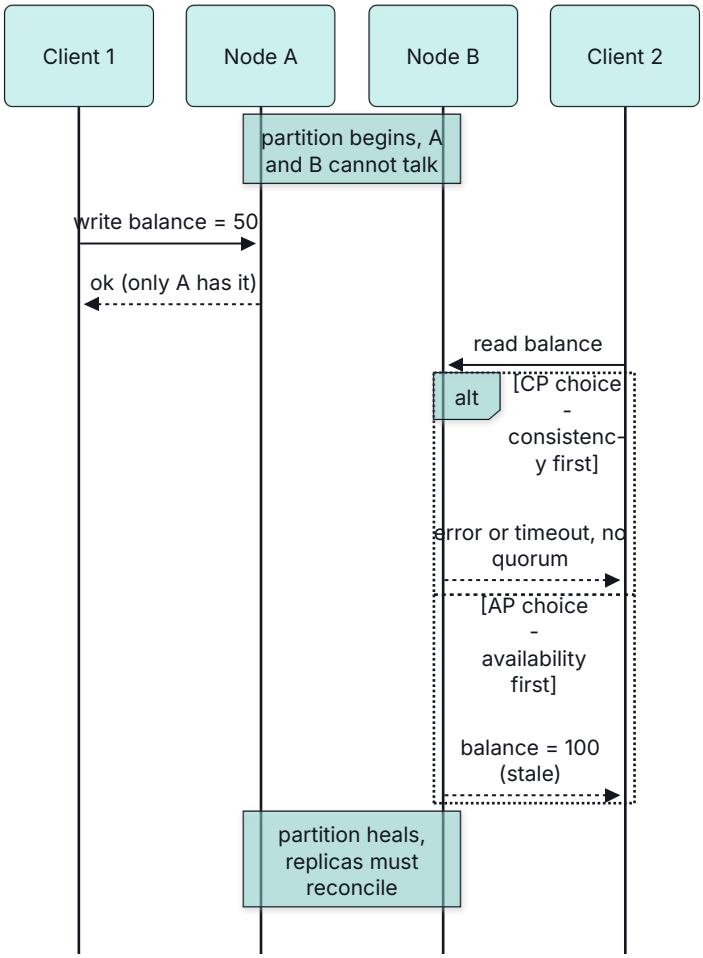


Figure 13.12-1

How the two acronyms divide the decision space:

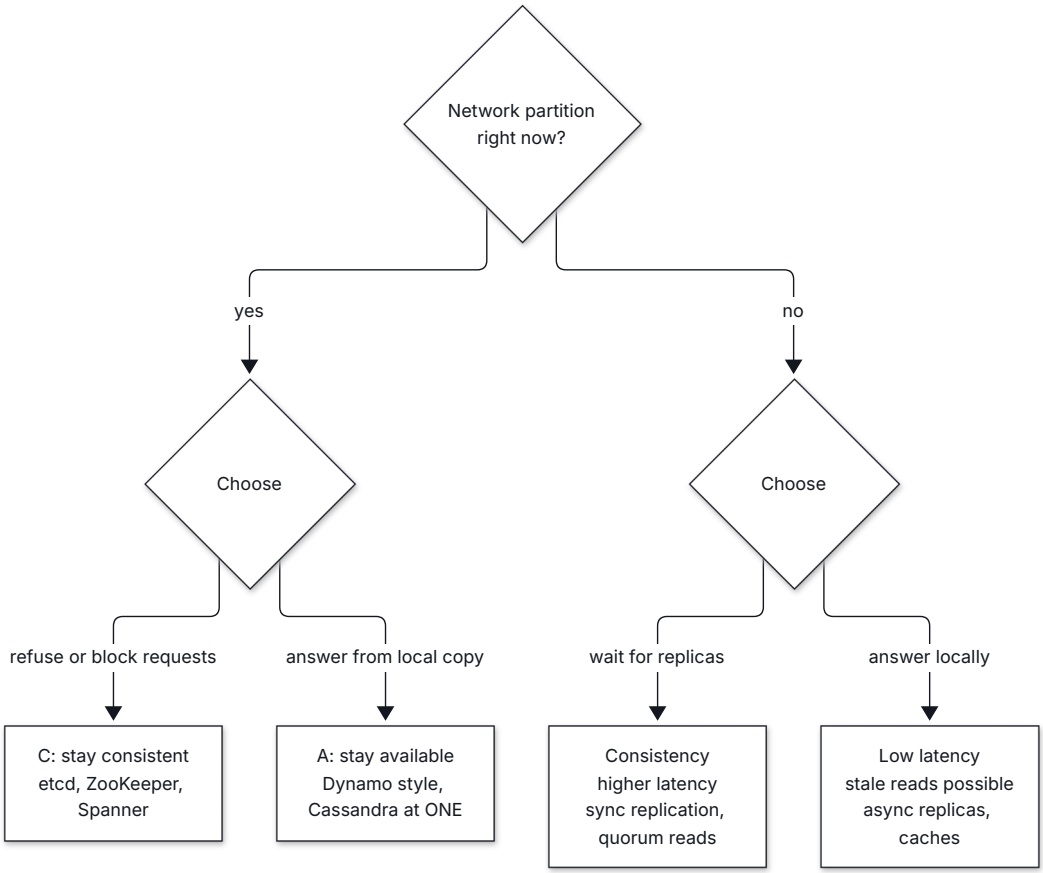


Figure 13.12-2

5 How it works

Linearizability, and what it is not

A history is **linearizable** if every operation appears to take effect atomically at some point between its start and its end, and that order matches real time: once a read returns the new value, no later read may return the old one. It is a property of a *single object* (one key, one row, one register).

It is easy to confuse with other terms:

Term	Meaning	Typical place
Linearizability	Single-object, real-time ordering ("the latest value")	etcd reads, Spanner, Postgres primary reads
Serializability	Multi-object *transactions* behave as if run one at a time (no real-time guarantee)	`SERIALIZABLE` isolation, see Transactions and Locks §13.2
Strict serializability	Both: serial order that also respects real time	Spanner
ACID "C"	Application invariants hold after a transaction (unique, foreign keys)	Any SQL database; unrelated to CAP "C"

A ladder of consistency models

Weaker models are cheaper and more available; each rung up costs latency or availability.

Model	Guarantee	Example
Linearizable	Always the latest completed write	primary reads, Raft leader reads
Sequential	All nodes see the same order, not necessarily the latest	some queue systems
Causal	If B was caused by A, everyone sees A before B	comment replies after the post
Read-your-writes / monotonic reads	You see your own writes; time never goes backward for you	sticky sessions, LSN waits
Eventual	If writes stop, all replicas converge	async replicas, DNS, caches

Where common systems sit

Classifications are tendencies, not labels: most systems are configurable and a single product can behave CP or AP depending on settings.

System (default or typical setup)	During a partition	Normally (PACELC)	Notes
etcd, Consul, ZooKeeper	**CP** (minority side refuses)	PC/EC	consensus; reads can be made linearizable
PostgreSQL, one primary + async replica	writes only on primary; replica reads stale	PA/EL for replica reads	failover may lose acknowledged writes
PostgreSQL with `synchronous_commit` and a required standby	**CP-leaning**: commits block	PC/EC	the stall you meet in Replication §13.4

System (default or typical setup)	During a partition	Normally (PACELC)	Notes
Redis primary + replicas	AP-leaning: async, can lose acked writes on failover	PA/EL	`WAIT` and `min-replicas-to-write` shift it toward C
Kafka with `acks=all`, `min.insync.replicas=2`, unclean election off	CP-leaning for produce	PC/EC	see Kafka §13.8
Cassandra / DynamoDB-style	AP (with `QUORUM` or strong reads it leans CP)	PA/EL	tunable
MongoDB	depends on write concern, read concern	varies	majority write plus majority read is the strong end
Google Spanner	**CP**	PC/EC	uses TrueTime, bounded clock uncertainty

Tuning knobs you actually touch

- **Quorums:** with N replicas, write to W and read from R. If **W + R > N**, every read overlaps at least one node holding the latest write (the basis of tunable consistency in Dynamo-style stores). Raft/etcd use majority for both.
- **Synchronous vs asynchronous replication:** a commit waits for the standby (C, latency) or does not (A, speed, possible data loss).
- **Failover policy:** automatic promotion (availability) versus human approval or fencing first (consistency); see Distributed Locks and Queues §13.15 for fencing tokens that stop a stale primary from writing.
- **Timeouts:** a "partition" is indistinguishable from a slow node; your timeout decides when a node is declared dead.

6 Step-by-step guide

Prerequisites: Linux or macOS, `redis-server` and `redis-cli` 6.2 or newer (`sudo apt-get install -y redis-server` then `sudo systemctl disable --now redis-server`, or `brew install redis`). Cost: free. Time: ~45 minutes. Everything below was run against Redis 7.0.15; ports, PIDs and timings will differ ("similar to"). We build one primary and one replica, then break the link in three ways and watch Redis pick a side each time.

1. **Start a primary and a replica.**

```bash
mkdir -p /tmp/cap/r1 /tmp/cap/r2
redis-server --port 6381 --dir /tmp/cap/r1 --save "" --daemonize yes
redis-server --port 6382 --dir /tmp/cap/r2 --save "" --daemonize yes --replicaof
  127.0.0.1 6381
sleep 2
redis-cli -p 6381 info replication | grep -E "role|slave0"
```

```
role:master
slave0:ip=127.0.0.1,port=6382,state=online,offset=14,lag=1
```

2. **Write and read from the replica.** Replication is asynchronous, so this *usually* shows the value, but nothing guarantees it.

```bash
redis-cli -p 6381 SET k v1
redis-cli -p 6382 GET k
```

   Expect `OK` then `"v1"`. A read that came back before the write arrived would be a stale (non-linearizable) read.

3. **Partition the replica.** Freeze its process so it can neither receive nor acknowledge anything (the effect of a dead link):

```bash
RP=$(redis-cli -p 6382 info server | grep process_id | tr -d '\r' | cut -d: -f2)
kill -STOP "$RP"
```

   Do not use `pgrep -f`: it can also match your own shell.

4. **Primary chooses availability: it accepts the write.** `WAIT` only counts acknowledgements for writes made on the *same connection*, so send both commands through one `redis-cli` session:

```bash
printf 'SET k v2\nWAIT 1 500\n' | redis-cli -p 6381
```

```
OK
0
```

The client got `OK` (write acknowledged) but `WAIT` reports **0 replicas** have it. If the primary died now and the replica were promoted, `v2` would be lost. That is the "A" side of the choice.

5. **Flip the primary to consistency: refuse writes without a replica.**

```bash
redis-cli -p 6381 CONFIG SET min-replicas-to-write 1
redis-cli -p 6381 CONFIG SET min-replicas-max-lag 2
sleep 3
redis-cli -p 6381 SET k v3
```

```
NOREPLICAS Not enough good replicas to write.
```

The primary now sacrifices availability to avoid unreplicated writes. Heal and watch it recover:

```bash
kill -CONT "$RP"; sleep 2
printf 'SET k v4\nWAIT 1 500\n' | redis-cli -p 6381     # OK then 1
redis-cli -p 6382 GET k                                  # "v4"
```

6. **Create a split-brain on purpose** (both sides accept writes, the AP failure mode). Promote the replica while the old primary is still alive, as an over-eager failover script would:

```bash
redis-cli -p 6381 CONFIG SET min-replicas-to-write 0
redis-cli -p 6382 replicaof no one
redis-cli -p 6381 SET cart:42 "book"
redis-cli -p 6382 SET cart:42 "book,pen"
```

Both return `OK`: two nodes each accepted a different "truth". Now reconnect the old replica and wait 5-10 seconds for it to resync:

```bash
redis-cli -p 6382 replicaof 127.0.0.1 6381
sleep 8; redis-cli -p 6382 GET cart:42
```

```
"book"
```

The `pen` write was silently discarded. The system converged but **lost an acknowledged write**, which is the cost of availability without a conflict-resolution strategy (see Eventual Consistency §13.13).

7. **Clean up.**

```bash
redis-cli -p 6382 shutdown nosave; redis-cli -p 6381 shutdown nosave; rm -rf /tmp/cap
```

7 Real-world scenario

Illustrative, not a documented incident. `shop-api` runs Postgres in two availability zones: a primary in `us-east-1a` and an async standby in `us-east-1b`, with an automatic failover tool and a 10-second health timeout. A network device between the zones starts dropping packets for 40 seconds. The tool cannot reach the primary, so it promotes the standby. But the primary was alive and kept serving the web tier in its own zone, so for about 40 seconds both databases accepted `INSERT INTO orders`. After the network heals, the old primary holds 312 orders the new primary has never seen, and order IDs overlap.

Investigation: `pg_controldata` showed two timelines; the order-ID gap and `pg_stat_replication` on the new primary made the divergence obvious. *Fix:* the team fenced the old primary (stopped it, blocked its traffic), exported its post-divergence orders, re-inserted them with new IDs through the application, and rebuilt it as a standby with `pg_rewind`. *Lesson:* automatic failover chose **availability** without a fencing step; they added STONITH-style fencing (shoot the old node first), raised the health-check timeout, required a quorum of observers to agree before promotion, and put the decision in the runbook so the next on-call knew it was a deliberate C-versus-A trade.

8 Common mistakes and best practices

Don't

- Say "we are CA". With a network in the middle, a system that is neither CP nor AP during a partition is simply undefined behaviour.
- Treat CAP "C" as the C in ACID, or linearizability as serializability.
- Read from an async replica right after a write and assume you see it.
- Auto-promote a standby without fencing the old primary.
- Pick a label from a vendor page and stop; check the actual defaults (write concern, `acks`, consistency level, `synchronous_commit`).

Do

- Decide per feature: payments and inventory want C; feeds, counters and recommendations usually tolerate stale reads.
- Run odd-sized quorums (3 or 5) across failure domains and monitor quorum health.
- Make failover a rehearsed procedure, with fencing, and test it with injected faults (a staging chaos experiment).
- Expose the choice: use read-your-writes tricks, `WAIT`, quorum reads or LSN waits where a feature needs them.
- Read the Jepsen report for any new database you adopt.

9 Tasks

- [ ] **T1 — Build the Redis pair and see the stale window** *(≈25 min, beginner)*
  - **Goal:** run one primary and one replica and read a replicated key.
  - **Steps:**
    1. Follow steps 1-2.
    2. Write 1,000 keys in a loop with `redis-cli -p 6381` and read the last one from `6382` immediately.
    3. Run `redis-cli -p 6381 info replication` and note `master_repl_offset` versus the replica's offset.
  - **Done when:** you can show both offsets and say which one is "behind" and what that means for a read.
- [ ] **T2 — Show that an acknowledged write can be unreplicated** *(≈20 min, intermediate)*
  - **Goal:** prove `OK` does not mean "durable on a second node".
  - **Steps:**
    1. Freeze the replica as in step 3, then `SET` and `WAIT 1 500` through one connection.
    2. Record the `0` result, then unfreeze and `WAIT` again.
  - **Done when:** you have the transcript showing `OK` with `WAIT` = 0 during the freeze and `1` afterward.
- [ ] **T3 — Flip the knob: refuse writes** *(≈20 min, intermediate)*
  - **Goal:** convert an AP-leaning setup to a CP-leaning one.
  - **Steps:**
    1. Set `min-replicas-to-write 1` and `min-replicas-max-lag 2`.
    2. Freeze the replica and attempt a write after 3 seconds.
    3. Restore the replica and confirm writes resume without restarting the primary.

- **Done when:** you captured `NOREPLICAS` and later a successful write, and can explain which side of CAP each state was on.

- [ ] **T4 — Break it: cause a split-brain and measure the loss** *(≈30 min, advanced)*
  - **Goal:** quantify what is thrown away.
  - **Steps:**
    1. Run step 6, but write 10 different keys on each side before reconnecting.
    2. After resync, count which keys survive on `6382` (`redis-cli -p 6382 dbsize`, `keys '*'` on a toy dataset is fine).
    3. Write a short note: which writes were lost and which node's history won.
  - **Done when:** you can list exactly the lost keys and name the mechanism (full resync from the new primary).

- [ ] **T5 — Classify your own stack** *(≈40 min, intermediate)*
  - **Goal:** make the C-or-A choice explicit for the `shop-api` components.
  - **Steps:**
    1. Fill a table for Postgres (primary and replica reads), Redis cache, Kafka topic and the load balancer's health checks: partition behaviour, normal-time behaviour (PACELC), and the setting that controls it.
    2. For each, write one sentence on what a user sees during a 60-second partition.
  - **Done when:** the table has no blank cells and each setting name is a real config key you could grep for.

- [ ] **T6 — Prove it: explain a partition to a teammate** *(≈30 min, advanced)*
  - **Goal:** teach the trade without jargon.
  - **Steps:**
    1. Without this page, draw the sequence diagram from section 4 on paper and annotate where C and A are given up.
    2. Explain why "CA" is not a real option, and what PACELC adds.
    3. Recommend CP or AP for three features (checkout, product view counter, session store) with one sentence each.
  - **Done when:** a teammate can restate your three recommendations and the reason for each.

10 Self-check questions

1. Why is "choose two of C, A and P" misleading?

2. What does *linearizable* mean, and how is it different from *serializable*?
3. What does PACELC say that CAP does not?
4. Your 3-node etcd cluster loses two nodes. What can and can't the cluster do, and what happens to running pods?
5. With N = 5 replicas, W = 3 and R = 3, do reads always see the latest write? What if W = 2 and R = 2?

ANSWERS

1. Partitions are a fact of networks, so P cannot be given up. The real decision is whether to give up C or A while a partition lasts. With no partition, you can have both.
2. Linearizable is a single-object, real-time guarantee (reads see the latest completed write). Serializable is about multi-object transactions behaving as if run one after another, with no promise about real-time order. Strict serializability is both.
3. Even with no partition there is a trade between latency and consistency (synchronous replication, quorum reads cost round trips); the "else" half of the decision matters every day.
4. A single member is a minority, so it cannot commit writes or serve linearizable reads (CP). The Kubernetes control plane cannot change state, but already-running pods on nodes keep running.
5. W + R = 6 > 5, so every read quorum overlaps the latest write quorum: yes. With W = 2 and R = 2, W + R = 4 which is not above 5, so a read can miss the latest write.

11 Further reading

- Gilbert and Lynch, *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services* (2002).
- Eric Brewer, *CAP Twelve Years Later: How the "Rules" Have Changed* (IEEE Computer, 2012) and *Spanner, TrueTime and the CAP Theorem* (2017).
- Daniel Abadi, *Consistency Tradeoffs in Modern Distributed Database System Design* (the PACELC paper).
- *Designing Data-Intensive Applications* by Martin Kleppmann: chapters on replication, consistency and consensus.
- Jepsen (jepsen.io): consistency model map and database analyses.
- GitHub Engineering blog: the October 2018 post-incident analysis.

13.13 Eventual Consistency

Topic 13 of 15 | ~5 hours | Difficulty: Advanced

1 What it is

A system is **eventually consistent** if, when updates stop, all replicas of a piece of data **converge** to the same value after some time. It promises nothing about *how long* that takes, and during the gap different readers may see different values. It is the weak end of the consistency spectrum described in Consistency, Availability and the CAP Theorem §13.12: you give up "every read sees the latest write" in exchange for lower latency, higher availability and the ability to keep working across slow or partitioned networks.

Eventual consistency is not "anything goes". Real systems add targeted guarantees on top (*session guarantees* such as **read-your-writes**), and a rule for resolving **conflicts** when two replicas accepted different writes (last-writer-wins, application merge, or **CRDTs**, Conflict-free Replicated Data Types, whose merge function is designed so replicas always converge without coordination).

2 Why it matters in DevOps

- **You already run eventually consistent components.** Async Postgres replicas, Redis replicas, CDN caches, DNS, search indexes and read models fed by Kafka all lag behind their source. Alert thresholds, cache TTLs and runbooks must assume it.
- **"I saved it but it's gone" tickets** are usually read-your-writes violations: the write went to the primary, the next read hit a lagging replica.
- **Multi-region designs depend on it.** Active-active across regions cannot wait for a cross-ocean quorum on every write, so conflicts are real and need a policy.
- **Silent data loss hides here.** Last-writer-wins with unsynchronised clocks drops writes with no error. You find out from a customer.
- **Convergence time is an SLI.** "Replica lag" and "time until the search index shows a new product" are measurable and belong on a dashboard (see Replication §13.4).

3 Real-world system example

- **Amazon Dynamo (2007 paper) and its shopping cart.** The paper describes accepting cart updates on any reachable replica and reconciling divergent versions on read by merging them; the stated trade-off was that a deleted item could occasionally reappear, which was preferred over refusing the customer's add-to-cart. Cassandra and Riak descend from these ideas, using **read repair**, **hinted handoff** and **anti-entropy** (background comparison of replicas) to converge.
- **Amazon S3.** Historically S3 was eventually consistent for overwrite and delete operations; since December 2020 AWS documents **strong read-after-write consistency** for all S3 operations. A good example of a system moving up the spectrum when engineering allowed.
- **DNS.** Record changes propagate as caches expire; the TTL is literally the upper bound on how stale a resolver may be. See DNS Resolution §2.5.
- **Collaborative editing.** Tools such as Google Docs use operational transformation; open-source libraries such as **Yjs** and **Automerge** use CRDTs so offline edits merge deterministically. Figma has written publicly that its multiplayer design is inspired by CRDT ideas but uses a central server to order changes.
- **Riak** (data types), **Redis Enterprise Active-Active** and **Azure Cosmos DB** (multi-region writes) document conflict-free or policy-based conflict resolution built on CRDT concepts or last-writer-wins.

4 Diagram

Two replicas diverge during a partition, then converge through anti-entropy and a merge function:

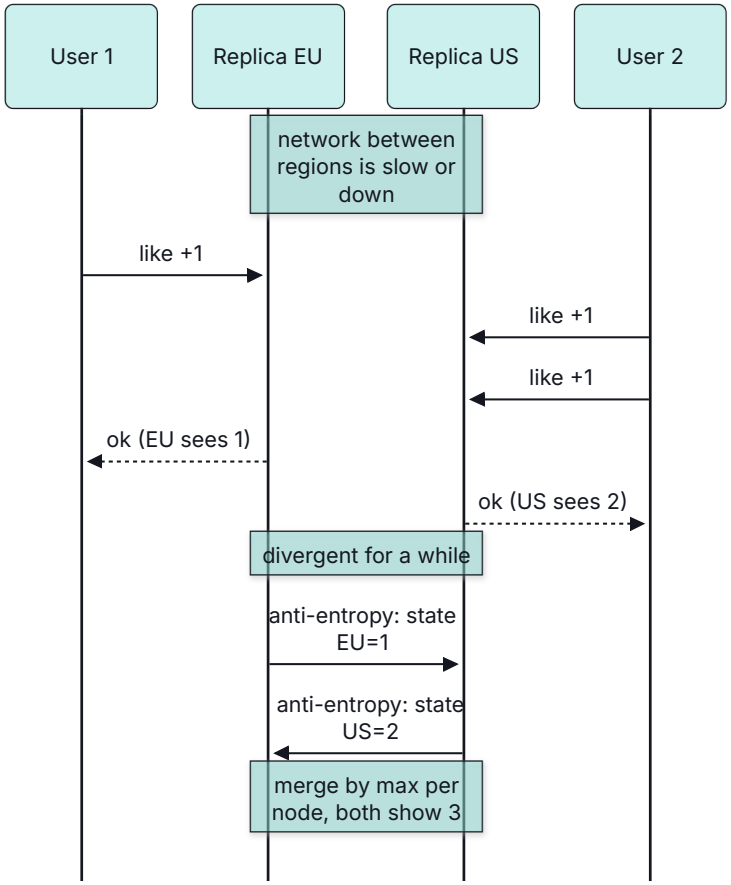


Figure 13.13-1

Where reads go decides what a user sees. A common read-your-writes routing rule:

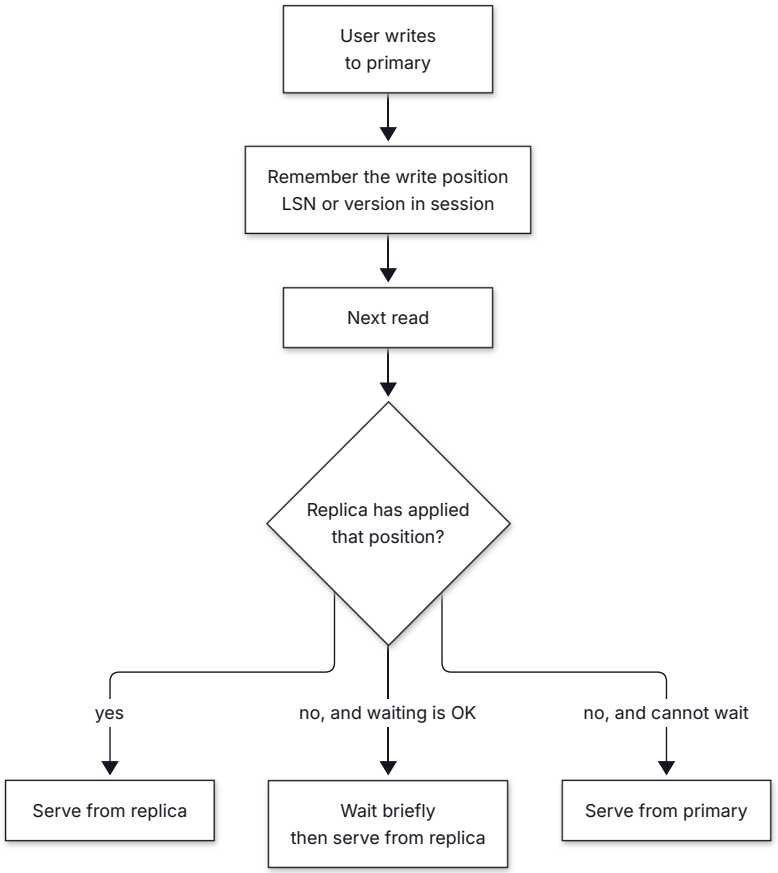


Figure 13.13-2

5 How it works

Convergence mechanisms

Mechanism	What it does	Where you see it
Asynchronous log shipping	Replica replays the primary's change stream; converges as fast as it catches up	Postgres streaming replication, Redis replicas
Read repair	A read that finds stale copies writes the fresh value back	Cassandra, Dynamo-style stores
Hinted handoff	A node holds writes for a down replica and replays them later	Cassandra, Dynamo

Mechanism	What it does	Where you see it
Anti-entropy / Merkle trees	Background process compares replica contents and syncs differences	Cassandra repair, Riak
TTL expiry	Stale copies simply age out	DNS, CDN and application caches
Event-driven projection	A consumer updates a read model after the event	Kafka consumers, search indexing (see Kafka §13.8)

Session guarantees (what applications actually need)

Guarantee	Meaning	Typical implementation
Read-your-writes	You see your own earlier writes	route to primary for a short window, or wait for replica position (LSN)
Monotonic reads	You never see data go backwards in time	pin a session to one replica
Monotonic writes	Your writes are applied in the order you issued them	per-session ordering, single partition key
Writes-follow-reads	A write based on what you read is ordered after it	causal metadata (version vectors)

Resolving conflicts

When two replicas accept concurrent writes to the same item, one of these happens:

1. **Last-writer-wins (LWW):** keep the value with the highest timestamp. Simple and always converges, but it **silently discards the other write**, and clock skew picks the "winner" arbitrarily.
2. **Version vectors:** each replica keeps a counter per node; if neither vector dominates, the writes are *concurrent* and both are kept as **siblings** for the application to merge (Dynamo, Riak).
3. **Application-level merge:** the application knows the domain rule (union of cart items, sum of counters).
4. **CRDTs:** pick a data structure whose merge is mathematically guaranteed to converge.

CRDTs in two minutes

A CRDT's merge operation is **commutative** (order does not matter), **associative** (grouping does not matter) and **idempotent** (merging the same state twice changes nothing). Those three properties mean replicas can exchange state in any order, with duplicates and delays, and still agree. This is called *strong eventual consistency*.

CRDT	Use	How it merges
G-Counter	likes, page views	one slot per node; merge = max per slot; value = sum
PN-Counter	stock, balances that move both ways	two G-Counters (adds and removes)
LWW-Register	a profile field	highest timestamp wins (loses concurrent writes)
OR-Set (observed-remove)	shopping cart, tags	adds carry unique tags; a remove deletes only the tags it saw
Sequence CRDTs	collaborative text	each character gets a unique, ordered ID (Yjs, Automerge)

Limits: CRDTs cannot enforce invariants such as "balance never below zero" or "only one winner", because that needs coordination (see Distributed Locks and Queues §13.15 and consensus-based stores).

6 Step-by-step guide

Prerequisites: Python 3.9+ for steps 1-2 (no libraries). Steps 3-4 reuse the Postgres primary and standby (`pg-primary`, `pg-replica`, database `postgres`, table `orders`) you built in Replication §13.4, or the capstone's lab; the SQL was verified on PostgreSQL 16.15 using two local instances, the `docker exec` wrapper is the standard form (container names are from topic 04). Cost: free. Time: ~60 minutes.

1. **Build and merge CRDTs.** Save as `crdt.py`:

```python
class GCounter:                                  # grow-only counter: one slot per replica
    def __init__(self, node): self.node, self.slots = node, {}
    def incr(self, n=1): self.slots[self.node] = self.slots.get(self.node, 0) + n
    def value(self): return sum(self.slots.values())
    def merge(self, other):                      # element-wise max: commutative,
  associative, idempotent
        for k, v in other.slots.items():
            self.slots[k] = max(self.slots.get(k, 0), v)

class PNCounter:                                 # increments and decrements = two G-
  Counters
```

```
    def __init__(self, node): self.p, self.n = GCounter(node), GCounter(node)
    def incr(self, k=1): self.p.incr(k)
    def decr(self, k=1): self.n.incr(k)
    def value(self): return self.p.value() - self.n.value()
    def merge(self, o): self.p.merge(o.p); self.n.merge(o.n)

# 1. Three data centres count likes while partitioned, then gossip in a messy order
  (with repeats).
a, b, c = GCounter("a"), GCounter("b"), GCounter("c")
a.incr(3); b.incr(2); c.incr(5)
for x, y in [(a, b), (b, c), (c, a), (a, b), (b, c), (a, c)]:
    x.merge(y)
print("G-Counter values:", {n.node: n.value() for n in (a, b, c)})

# 2. Why merge must be idempotent: a naive "add the other replica's total" double
  counts on retry.
naive = a.value()
for _ in range(2): naive += b.value()
print("naive sum after a duplicate message:", naive, "vs correct", a.value())

# 3. PN-Counter: stock level that goes up and down.
s1, s2 = PNCounter("w1"), PNCounter("w2")
s1.incr(10); s2.decr(3); s1.decr(1)
s1.merge(s2); s2.merge(s1)
print("PN-Counter stock:", s1.value(), s2.value())

# 4. Last-writer-wins register and clock skew. Each write is (timestamp, value).
def lww(x, y): return x if x[0] ⩾ y[0] else y
real_first  = (1000.0, "cart=[book]")            # node A, true time 1000
real_second = (1001.0 - 4.0, "cart=[book,pen]")  # node B at true time 1001, but its
  clock is 4 s slow
print("LWW keeps:", lww(real_first, real_second)[1], "(the later real write was
  lost)")
```

BASH

```
python3 crdt.py
```

```
G-Counter values: {'a': 10, 'b': 10, 'c': 10}
naive sum after a duplicate message: 30 vs correct 10
PN-Counter stock: 6 6
LWW keeps: cart=[book] (the later real write was lost)
```

All three replicas agree on 10 despite repeats and reordering; the naive "just add" merge triple counts; the last line is the silent data loss of LWW with a skewed clock.

2. **Try to break convergence.** Edit the loop in part 1 so a replica merges a *stale copy* of another (take `copy.deepcopy(b)` before `b.incr(1)` and merge the copy afterwards). The value never goes down, because `max` ignores older state. Then change `merge` to overwrite

(`self.slots = dict(other.slots)`) and observe it lose increments: this is why the merge function is the whole design.

3. **Reproduce a read-your-writes violation.** Pause replay on the standby so the lag is visible, write on the primary, read from the replica:

```bash
docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_pause();"
docker exec pg-primary psql -U postgres -At -c "INSERT INTO orders (item) VALUES
  ('ryw-demo');"
docker exec pg-replica psql -U postgres -At -c "SELECT count(*) FROM orders WHERE
  item='ryw-demo';"
```

```
INSERT 0 1
0
```

The user's order is "missing": a stale read from a lagging replica.

4. **Fix it with an LSN wait.** Capture the primary's write position *after* the commit (a separate command, otherwise you record a position before the commit record), then wait for the replica to pass it before reading:

```bash
LSN=$(docker exec pg-primary psql -U postgres -At -c "SELECT pg_current_wal_lsn();")
echo "need replica to reach $LSN"
( sleep 3; docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_resume();"
  >/dev/null ) &   # simulate lag ending
until [ "$(docker exec pg-replica psql -U postgres -At -c "SELECT
  pg_wal_lsn_diff(pg_last_wal_replay_lsn(), '$LSN') ≥ 0;")" = "t" ]; do sleep 0.2;
  done
docker exec pg-replica psql -U postgres -At -c "SELECT count(*) FROM orders WHERE
  item='ryw-demo';"
```

```
need replica to reach 0/3000148
1
```

(The LSN value will differ.) The read now blocks for the length of the lag and then returns the user's own write. In an application, the session stores the LSN (or a version number) and the router applies the decision flow from the second diagram, with a timeout that falls back to the primary.

5. **Watch eventual consistency in the wild.** DNS TTLs are the convergence clock:

   ```bash
   dig +noall +answer example.com A
   sleep 5
   dig +noall +answer example.com A
   ```

   The TTL (second column) counts down on a caching resolver; a record change on the authoritative server becomes visible to that resolver only after it reaches 0 (output varies by resolver).
6. **Clean up** the lag pause if you left it on: `docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_resume();"`.

7 Real-world scenario

Illustrative, not a documented incident. After `shop-api` adds a Postgres read replica, support receives "my order disappeared" tickets that nobody can reproduce. *Symptom:* right after checkout the "Order confirmed" page runs `SELECT ... WHERE id = ?` against the replica and shows a 404 about one time in two hundred. *Investigation:* replica lag graphs show spikes of 1-2 seconds during the nightly import job; the failing requests line up exactly with those spikes. *Fix:* the confirmation endpoint reads the primary for the first 10 seconds after an order is created (a short "primary-only window" stored in a signed cookie), and the order-history page, which can tolerate staleness, stays on the replica. A later iteration stored the commit LSN in the session and waited up to 200 ms for the replica. *Lesson:* the bug was not the replica; it was an unstated consistency requirement. The team added "replica lag p99" as an SLI and wrote the read-your-writes rule into the API design doc.

8 Common mistakes and best practices

Don't

- Assume a read after a write sees the write when the read can be served from a replica or cache.
- Use last-writer-wins on data you cannot afford to lose, or rely on wall-clock timestamps from unsynchronised machines.
- Treat eventual consistency as "no guarantees": you can specify bounds (lag p99) and session guarantees.
- Build counters or sets by read-modify-write across replicas; that is how updates get lost.

- Use a CRDT for an invariant that needs coordination (unique username, non-negative balance).

Do

- Measure and alert on convergence time (replica lag, index freshness, cache age).
- Give each feature an explicit consistency requirement: strong for money and inventory reservation, eventual for feeds and analytics.
- Implement read-your-writes with a primary window, an LSN/version wait or sticky sessions.
- Choose merge semantics deliberately: sibling merge, domain merge or a CRDT.
- Make consumers idempotent so redelivery cannot change the converged result (see Idempotency §13.14).

9 Tasks

- [ ] **T1 — Run the CRDT demo** *(≈20 min, beginner)*
  - **Goal:** watch replicas converge despite reordered and duplicated merges.
  - **Steps:**
    1. Save and run `crdt.py` from step 1.
    2. Shuffle the merge order in the loop and re-run three times.
  - **Done when:** every run prints the same value (10) for all three counters.
- [ ] **T2 — Break convergence with a bad merge** *(≈25 min, intermediate)*
  - **Goal:** prove idempotency and commutativity are what make a CRDT safe.
  - **Steps:**
    1. Replace `merge` with `self.slots[k] = self.slots.get(k, 0) + v`.
    2. Run the same gossip loop and record the values.
    3. Restore `max` and explain the difference in one sentence.
  - **Done when:** you have both outputs and can name which property the broken merge violates.
- [ ] **T3 — Add an OR-Set** *(≈45 min, advanced)*
  - **Goal:** implement a mergeable set where add and remove can happen concurrently.
  - **Steps:**
    1. Implement `add(x)` creating a unique tag, `remove(x)` deleting only observed tags, and `merge` as the union of added tags minus the union of removed tags.

2. Simulate replica A removing `pen` while replica B concurrently adds `pen`, then merge both ways.

- **Done when:** both replicas end with `pen` present (the concurrent add wins) and identical contents.

- [ ] **T4 — Reproduce and fix read-your-writes** *(≈30 min, intermediate)*
  - **Goal:** turn a stale read into a correct one.
  - **Steps:**
    1. Do steps 3 and 4 of the guide on your own replica pair.
    2. Time how long the LSN wait blocks with `time`.
  - **Done when:** the naive read returns `0` and the LSN-gated read returns `1`, with a measured wait.

- [ ] **T5 — Automate a lag SLI** *(≈40 min, advanced)*
  - **Goal:** make convergence time observable.
  - **Steps:**
    1. Write a script that inserts a heartbeat row on the primary every second and measures how long until it appears on the replica.
    2. Print `p50` and `max` over one minute, once normally and once with replay paused for 5 seconds.
  - **Done when:** the paused run shows a max of at least 5 seconds and the normal run is well under 1 second.

- [ ] **T6 — Prove it: design a conflict policy** *(≈40 min, advanced)*
  - **Goal:** choose a resolution strategy and defend it.
  - **Steps:**
    1. Without the guide, list four `shop-api` fields (profile name, cart, view count, inventory level) and choose LWW, merge or CRDT for each.
    2. For each, write the worst thing that happens under your choice if two regions write concurrently.
  - **Done when:** a teammate agrees with each choice or you can explain the disagreement.

10 Self-check questions

1. What exactly does "eventually consistent" guarantee, and what does it not?
2. Why does last-writer-wins lose data, and what makes it worse in practice?

3. Name the three properties a CRDT merge must have, and what each buys you.
4. A user posts a comment and does not see it after a refresh. List two root causes and two fixes.
5. Why can a CRDT not enforce "balance never below zero"?

ANSWERS

1. If updates stop, replicas converge to the same value. It does not bound the time or promise what a read sees in the meantime.
2. Of two concurrent writes it keeps one and discards the other with no error. Clock skew makes the "latest" arbitrary, so it can discard the genuinely later write.
3. Commutative (order of merges does not matter), associative (grouping does not matter), idempotent (duplicate merges are harmless). Together they allow unordered, duplicated delivery with guaranteed convergence.
4. Causes: a read from a lagging replica; a cache or search index not yet updated. Fixes: read from the primary for a window or wait for the write's LSN; invalidate or write-through the cache; make the UI optimistic.
5. An invariant that spans replicas needs coordination to prevent two replicas from each spending the last of the balance concurrently. CRDT merges are coordination-free by design.

11 Further reading

- DeCandia et al., *Dynamo: Amazon's Highly Available Key-value Store* (SOSP 2007).
- Shapiro et al., *Conflict-free Replicated Data Types* (2011) and the CRDT.tech resource page.
- Werner Vogels, *Eventually Consistent* (ACM Queue / All Things Distributed).
- *Designing Data-Intensive Applications* by Martin Kleppmann, chapters on replication lag and leaderless replication.
- Docs for Cassandra (repair, hinted handoff), Yjs and Automerge.

13.14 Idempotency

Topic 14 of 15 | ~5 hours | Difficulty: Advanced

1 What it is

An operation is **idempotent** if doing it once or doing it many times leaves the system in the same state. `SET balance = 50` is idempotent; `ADD 10 TO balance` is not. In HTTP, `GET`, `PUT` and `DELETE` are defined as idempotent while `POST` is not, which is why "create a payment" is the classic hard case.

Idempotency matters because networks and queues **retry**. A client that times out cannot know whether the server did the work and the reply got lost, or never received the request at all. The only safe client behaviour is to try again, so the server must make "again" harmless. The common building blocks are an **idempotency key** (a unique ID the client attaches to one logical operation, reused on every retry), **deduplication** (remembering IDs already handled) and **at-least-once delivery** (a messaging guarantee that every message arrives one or more times, never zero). Combined: at-least-once delivery plus an idempotent consumer behaves like exactly-once *processing*.

2 Why it matters in DevOps

- **Double charges and duplicate emails** are the visible symptom; the cause is a retry, a re-delivered message or a replayed webhook that was not deduplicated.
- **Retries are everywhere you configure them:** load balancers, service meshes, SDKs, queue consumers, Kubernetes Jobs and CI runners all retry on your behalf. Turning on a retry policy without idempotent handlers converts a transient blip into data corruption.
- **Recovery depends on it.** Replaying a Kafka topic, restoring from backup and re-running a failed pipeline are only safe if the handlers can be re-run.
- **Infrastructure tools are built on it.** Terraform plans, Ansible tasks and `kubectl apply` are designed to be idempotent; scripts that are not (`mkdir` without `-p`, `INSERT` without a guard) break re-runs.
- See Rollback §5.7 and Database Migrations §5.9: re-runnable migrations are idempotent migrations.

3 Real-world system example

- **Stripe** documents an `Idempotency-Key` request header for `POST` requests: the first request's status code and body are stored under the key and returned for any retry with the same key (including errors), and a retry whose parameters differ from the original is rejected. Stripe's docs say keys may be removed from the system after they are at least 24 hours old. An IETF draft, *The Idempotency-Key HTTP Header Field*, is working to standardise the pattern.
- **Amazon SQS** standard queues deliver at least once, and AWS recommends idempotent consumers. **FIFO queues** add a `MessageDeduplicationId` and drop duplicates sent within a five-minute deduplication interval.
- **Apache Kafka** producers with `enable.idempotence=true` use producer IDs and sequence numbers so broker-side retries do not write duplicates to a partition. Kafka transactions give exactly-once semantics for read-process-write *inside Kafka*; a consumer that also writes to an external database still needs its own idempotent write (see Kafka §13.8).
- **AWS Lambda** event sources (SQS, S3, EventBridge) can invoke a function more than once for the same event; AWS guidance is to write handlers to be idempotent. **Powertools for AWS Lambda** ships an idempotency utility built on this pattern.
- **Kubernetes** is idempotent by design at the API layer: applying the same manifest twice converges to the same state, and controllers re-run reconcile loops continuously.
- **GitHub, Shopify and many SaaS products** deliver webhooks at least once and include a delivery ID so receivers can deduplicate.

4 Diagram

A client retry that would double-charge without a key, and does not with one:

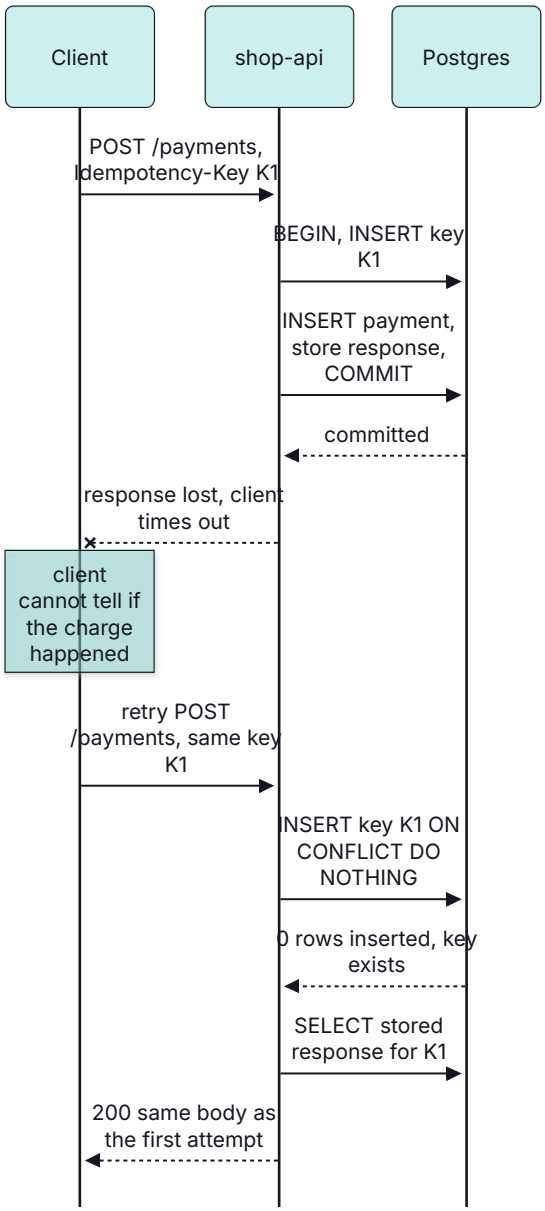


Figure 13.14-1

The consumer side: a message may arrive twice, so the dedup record and the business effect commit together:

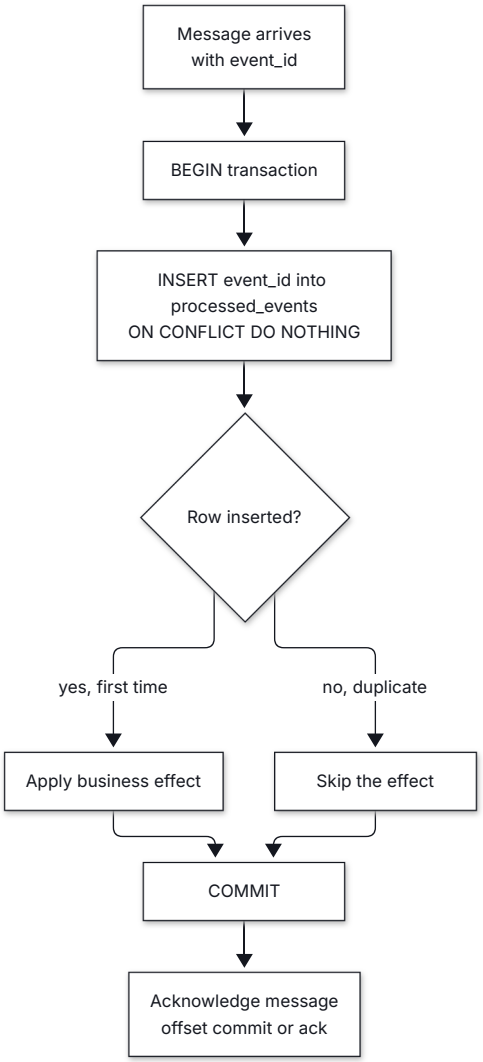


Figure 13.14-2

5 How it works

Delivery guarantees and what they cost

Guarantee	Meaning	How it fails	Needs idempotent consumer?
At-most-once	Never redelivered; may be lost	message lost if the consumer crashes after receive, before processing	no, but you lose data
At-least-once	Never lost; may be duplicated	duplicate if the ack is lost or the consumer crashes after processing	**yes**
Exactly-once (delivery)	Delivered exactly one time	not achievable end-to-end in general, because the ack itself can be lost	n/a
Exactly-once (processing)	at-least-once delivery + idempotent effect	achievable and standard	built from the two above

Most queues (SQS standard, RabbitMQ with manual acks, Kafka with commit-after-process) are at-least-once. The crash window is always the same: *effect done, acknowledgement not yet recorded*. After the restart the message comes again (see RabbitMQ §13.9 for redelivery and Kafka §13.8 for offset commits).

Four ways to make an effect idempotent

Technique	Example	Notes
Naturally idempotent write	`UPDATE orders SET status='shipped' WHERE id=7` or `PUT` of a full resource	safe to repeat; beware `status = status + 1`
Unique constraint / upsert	`INSERT ... ON CONFLICT DO NOTHING`, `ON CONFLICT DO UPDATE`	the database enforces the dedup atomically
Dedup table (inbox)	`processed_events(event_id PRIMARY KEY)` in the same transaction as the effect	the general solution for consumers
Conditional / versioned write	`UPDATE ... WHERE version = 5`, compare-and-set	rejects stale or repeated updates

Designing an idempotency key

- **Client-generated** (a UUID created once per logical operation, reused for every retry), not generated per attempt.
- **Scoped** to a user or tenant and an operation, so a key cannot collide across customers.
- **Bound to the request:** store a hash of the parameters; same key with different parameters is a client bug, return `422` or `409`.

- **Stored with the outcome** (status and body), so retries receive the *same* answer, including a deterministic failure.
- **Expire eventually** (hours to days) so the table does not grow forever; a key reused after expiry is treated as new.
- **Atomic with the effect** when the effect lives in the same database. When the effect is an external call (a card charge), store an `in_progress` row first, pass the provider *its own* idempotency key, and handle crashes by retrying with that same key.

Retries done right

Retry only operations that are idempotent (or keyed), with **exponential backoff and jitter** so thousands of clients do not retry in lockstep, a cap on attempts and a total deadline. Without jitter, a short outage causes a retry storm that extends the outage; see Circuit Breakers §12.14.

6 Step-by-step guide

Prerequisites: Docker and Python 3.9+ with `pip install psycopg2-binary`. Cost: free. Time: ~60 minutes. All code and SQL below was run against PostgreSQL 16.15 started natively on a Linux VM (Docker is the portable way for you to start the same server); output with numbers such as payment IDs will differ ("similar to").

1. **Start a database.**

```bash
docker run -d --name pg-idem -e POSTGRES_PASSWORD=<YOUR_PASSWORD> -p
  127.0.0.1:5432:5432 postgres:16
export PRIMARY_DSN="postgresql://postgres:<YOUR_PASSWORD>@127.0.0.1:5432/postgres"
until docker exec pg-idem pg_isready -U postgres; do sleep 1; done
```

2. **See the dedup primitive in plain SQL.** The second insert is rejected silently:

```bash
docker exec pg-idem psql -U postgres -c "CREATE TABLE processed_events (event_id uuid
  PRIMARY KEY, processed_at timestamptz DEFAULT now());"
docker exec pg-idem psql -U postgres -c "INSERT INTO processed_events (event_id)
  VALUES ('11111111-1111-1111-1111-111111111111') ON CONFLICT DO NOTHING;"
docker exec pg-idem psql -U postgres -c "INSERT INTO processed_events (event_id)
  VALUES ('11111111-1111-1111-1111-111111111111') ON CONFLICT DO NOTHING;"
```

```
INSERT 0 1
INSERT 0 0
```

`INSERT 0 0` (zero rows inserted) is the signal "I have seen this event before". Applications check the row count.

3. **Write the idempotent charge function.** Save as `idem.py`. The key row, the payment and the stored response commit in one transaction:

```python
import hashlib, json, os, threading, psycopg2

DSN = os.environ["PRIMARY_DSN"]

SCHEMA = """
CREATE TABLE IF NOT EXISTS idempotency_keys (
  key          text PRIMARY KEY,
  request_hash text NOT NULL,
  response     jsonb,
  created_at   timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE IF NOT EXISTS payments (
  id bigserial PRIMARY KEY, customer_id int NOT NULL, amount_cents int NOT NULL
);
"""

def charge(key, customer_id, amount_cents):
    body = json.dumps([customer_id, amount_cents])
    h = hashlib.sha256(body.encode()).hexdigest()
    with psycopg2.connect(DSN) as conn, conn.cursor() as cur:
        cur.execute("INSERT INTO idempotency_keys (key, request_hash) VALUES (%s, %s)
  "
                    "ON CONFLICT (key) DO NOTHING", (key, h))
        if cur.rowcount == 0:                       # key seen before (waits if the
  first request is still committing)
            cur.execute("SELECT request_hash, response FROM idempotency_keys WHERE key
  = %s", (key,))
            old_hash, response = cur.fetchone()
            if old_hash ≠ h:
                return 422, {"error": "idempotency key reused with different
  parameters"}
            return 200, response | {"replayed": True}
        cur.execute("INSERT INTO payments (customer_id, amount_cents) VALUES (%s, %s)
  RETURNING id",
                    (customer_id, amount_cents))
        response = {"payment_id": cur.fetchone()[0], "amount_cents": amount_cents}
        cur.execute("UPDATE idempotency_keys SET response = %s WHERE key = %s",
  (json.dumps(response), key))
        return 200, response                        # effect + key + response commit
  together

if __name__ == "__main__":
    with psycopg2.connect(DSN) as c, c.cursor() as cur:
        cur.execute(SCHEMA)
    key = "6f1c2a9e-order-1042"
```

```
    print("first   :", charge(key, 7, 4999))
    print("retry   :", charge(key, 7, 4999))        # client never saw the first
  response
    print("mismatch:", charge(key, 7, 9999))
    out = []
    ts = [threading.Thread(target=lambda: out.append(charge("race-key", 8, 1500))) for
  _ in range(8)]
    [t.start() for t in ts]; [t.join() for t in ts]
    print("race    :", sum(1 for s, r in out if "replayed" not in r), "executed,",
          sum(1 for s, r in out if "replayed" in r), "replayed")
    with psycopg2.connect(DSN) as c, c.cursor() as cur:
        cur.execute("SELECT customer_id, count(*) FROM payments GROUP BY 1 ORDER BY
  1")
        print("payments rows per customer:", cur.fetchall())
```

BASH

```
python3 idem.py
```

```
first   : (200, {'payment_id': 1, 'amount_cents': 4999})
retry   : (200, {'payment_id': 1, 'amount_cents': 4999, 'replayed': True})
mismatch: (422, {'error': 'idempotency key reused with different parameters'})
race    : 1 executed, 7 replayed
payments rows per customer: [(7, 1), (8, 1)]
```

Four things to notice: the retry returns the *same* `payment_id`; reuse with different parameters is rejected; eight concurrent requests with one key create one payment (the unique index makes the others wait for the first transaction, then see the conflict); and there is exactly one payment row per customer.

4. **Prove the retry problem and the fix together.** Save as `retry.py` next to `idem.py`. It loses 60% of replies after the server has committed, and retries with backoff and jitter:

PYTHON

```
import random, time, uuid
import psycopg2
from idem import charge, DSN

def count(customer):
    with psycopg2.connect(DSN) as c, c.cursor() as cur:
        cur.execute("SELECT count(*) FROM payments WHERE customer_id = %s",
  (customer,))
        return cur.fetchone()[0]

def lose_reply(result):
    if random.random() < 0.6:                       # the server committed, but the
  client never hears back
        raise TimeoutError("response lost on the way back")
```

```
    return result

def naive_charge(customer, cents):                    # no key: every attempt is a
  brand-new payment
    with psycopg2.connect(DSN) as c, c.cursor() as cur:
        cur.execute("INSERT INTO payments (customer_id, amount_cents) VALUES (%s,
  %s)", (customer, cents))
    return lose_reply((200, {"ok": True}))

def keyed_charge(key, customer, cents):
    return lose_reply(charge(key, customer, cents))

def with_retries(fn, *args, attempts=8, base=0.05, cap=1.0):
    for n in range(1, attempts + 1):
        try:
            return n, fn(*args)
        except TimeoutError:
            time.sleep(random.uniform(0, min(cap, base * 2 ** n)))   # exponential
  backoff with full jitter
    raise RuntimeError("gave up")

random.seed(4)
before = count(42)
n, _ = with_retries(naive_charge, 42, 2500)
print(f"naive : client needed {n} attempts, rows created = {count(42) - before}")

random.seed(4)
before, key = count(43), str(uuid.uuid4())           # ONE key for the whole logical
  operation
n, res = with_retries(keyed_charge, key, 43, 2500)
print(f"keyed : client needed {n} attempts, rows created = {count(43) - before}, final
  reply = {res[1]}")
```

BASH

```
python3 retry.py
```

```
naive : client needed 4 attempts, rows created = 4
keyed : client needed 4 attempts, rows created = 1, final reply = {'payment_id': 9,
  'amount_cents': 2500, 'replayed': True}
```

The same four attempts created four charges without a key and one with it. (The seed makes the attempt count repeatable; remove `random.seed` to see it vary. The `payment_id` depends on earlier runs.)

5. **Inspect the stored keys.**

```bash
docker exec pg-idem psql -U postgres -c "SELECT key, left(request_hash, 8) AS hash,
  response FROM idempotency_keys ORDER BY created_at;"
```

   Each logical operation has one row holding the outcome that retries replay. In production add a scheduled `DELETE ... WHERE created_at < now() - interval '24 hours'`.

6. **Break it: put the dedup outside the transaction.** In `idem.py`, commit the key row first (a separate `with psycopg2.connect` block), then insert the payment in a second transaction, and `raise SystemExit` between them on the first call. The retry then finds the key with no stored response and never creates the payment: the key says "done" while nothing was done (in this code `None | {...}` raises a `TypeError`; a more forgiving implementation would happily return success). This is why the key and the effect must be one atomic unit, or the `in_progress` state must be handled explicitly.

7. **Clean up.**

```bash
docker rm -f pg-idem
```

7 Real-world scenario

Illustrative, not a documented incident. `shop-api` sends "order shipped" emails from a queue consumer. During a deploy the consumer pods are killed while processing a batch; the broker redelivers every unacknowledged message to the new pods. *Symptom:* about 1,800 customers receive the same shipping email twice, and support gets a spike of "is this a scam?" replies. *Investigation:* logs show each message processed twice with the same `order_id` and the same `event_id`, minutes apart; the handler sends the email, then acknowledges, and the pod died in between. *Fix:* a `processed_events` table keyed on `event_id`; the handler inserts the key, and only if the insert succeeds does it enqueue the email to the provider, passing the provider's own idempotency key (the `event_id`) as well. *Lesson:* the redelivery was correct broker behaviour; the missing piece was an idempotent handler. The team also added a test that delivers every message twice in CI, and a graceful-shutdown hook so deploys finish in-flight work first.

8 Common mistakes and best practices

Don't

- Generate a new idempotency key for each retry attempt (it defeats the point).

- Record "I handled it" in a different transaction, or a different datastore, than the effect without an `in_progress` protocol.
- Dedupe in memory or in a cache with a short TTL when the broker can redeliver days later.
- Retry non-idempotent operations blindly, or retry with no backoff, jitter or attempt limit.
- Assume the broker's "exactly-once" label covers your database write or your third-party API call.
- Return a *different* response for a replayed request (a replay should look like the original).

Do

- Make every consumer and webhook receiver idempotent from day one, and test by delivering each message twice.
- Use natural keys: unique constraints on `(provider, external_id)` are free deduplication.
- Pass your key through to downstream providers that support their own idempotency keys.
- Expire dedup records on a schedule that exceeds your longest retry and redelivery window.
- Log the idempotency key on every request so a duplicate can be traced end to end (see Correlation IDs §10.7).
- Use the transactional outbox on the producing side so an event exists if and only if the state change committed (see Distributed Locks and Queues §13.15).

9 Tasks

- [ ] **T1 — Dedup with SQL only** *(≈15 min, beginner)*
  - **Goal:** see the `INSERT 0 0` signal.
  - **Steps:**
    1. Do step 1 and step 2.
    2. Insert three different event IDs and one repeated, then `SELECT count(*) FROM processed_events;`.
  - **Done when:** the table holds 3 rows after 4 inserts and you can explain why the fourth reported `INSERT 0 0`.
- [ ] **T2 — Run the keyed charge** *(≈25 min, beginner)*
  - **Goal:** run the idempotent API function and read its output.
  - **Steps:**
    1. Create `idem.py` and run it.

2. Run it a second time without clearing tables.

- **Done when:** the second run replays both keys (`replayed: True` for `6f1c2a9e-order-1042`) and the payments table has not grown for customers 7 and 8.

☐ **T3 — Show the damage without a key** *(≈25 min, intermediate)*

- **Goal:** reproduce duplicate charges from retries.
- **Steps:**
  1. Create `retry.py` and run it.
  2. Change the seed and the loss probability (`0.6`) and rerun five times.
- **Done when:** you have five runs where the naive row count equals the attempts and the keyed count is always 1.

☐ **T4 — Break it: split the transaction** *(≈30 min, advanced)*

- **Goal:** demonstrate why key and effect must be atomic.
- **Steps:**
  1. Follow step 6 to commit the key before the payment and crash in between.
  2. Retry and record what the client receives and what the database contains.
  3. Fix it by restoring the single transaction (or add an `in_progress` status and a recovery path).
- **Done when:** you captured the inconsistent state (key present, no payment) and the fixed run where retry completes the payment.

☐ **T5 — Idempotent consumer for a webhook** *(≈45 min, advanced)*

- **Goal:** automate dedup for incoming events.
- **Steps:**
  1. Write a tiny HTTP receiver (Flask or `http.server`) for `POST /webhooks` that reads an `event_id` and applies an effect using the `processed_events` pattern.
  2. Send the same payload ten times with `curl` in parallel (`xargs -P`).
- **Done when:** the effect (a counter row) increased by exactly 1 and the logs show 1 `applied` and 9 `duplicate`.

☐ **T6 — Prove it: design the key lifecycle** *(≈40 min, advanced)*

- **Goal:** explain end-to-end exactly-once processing.
- **Steps:**
  1. Without the guide, draw the sequence of a `POST /payments` that calls an external card API: where the key row is written, which key goes to the provider, and what happens if the server crashes after the provider call but before the commit.

2. State the TTL for stored keys and justify it.

- **Done when:** your design survives each crash point you can name, and a teammate cannot find a window that charges twice.

10 Self-check questions

1. Which HTTP methods are idempotent, and why is `POST` not?
2. A queue is at-least-once. Where exactly can a duplicate come from?
3. Why must the dedup record and the business effect commit in the same transaction?
4. What should a server do when it receives a known idempotency key with different parameters?
5. Why add jitter to retry backoff?

ANSWERS

1. `GET`, `HEAD`, `PUT`, `DELETE` (and `OPTIONS`) are idempotent by definition; `POST` creates a new resource or triggers a side effect each time unless the server adds a key.
2. When the consumer processes a message but crashes, times out or loses the connection before the acknowledgement (ack or offset commit) is recorded, the broker redelivers it. Producer retries after a lost send-acknowledgement can also duplicate a message.
3. If they are separate, a crash between them leaves "marked done but not done" (lost effect) or "done but not marked" (duplicate on retry). One transaction makes them succeed or fail together.
4. Reject it (typically `422` or `409`): it is a client bug, and returning the old response would hide that the client changed the request.
5. Without jitter all clients retry at the same instants and re-create the overload (a retry storm); random delays spread the load.

11 Further reading

- Stripe API documentation: *Idempotent requests*.
- IETF draft *The Idempotency-Key HTTP Header Field* (httpapi working group).
- Amazon Builders' Library: *Making retries safe with idempotent APIs* and *Timeouts, retries, and backoff with jitter*.
- *Designing Data-Intensive Applications* by Martin Kleppmann: the chapters on end-to-end argument, exactly-once and fault tolerance.
- Chris Richardson, *Microservices Patterns*: Idempotent Consumer and Transactional Outbox.

- Apache Kafka documentation: producer idempotence and transactions. AWS SQS documentation: FIFO deduplication.

13.15 Distributed Locks and Queues

Topic 15 of 15 | ~6 hours | Difficulty: Advanced

1 What it is

A **distributed lock** lets many processes on many machines agree that only one of them is doing a particular thing right now: running the nightly report, being the leader, rebuilding a cache entry, updating a record. Unlike a lock inside one program, the holder can crash, freeze or be cut off from the network while still believing it owns the lock. For that reason practical distributed locks are **leases**: a lock with an expiry time that the holder must renew, so a dead holder cannot block everyone forever.

A **work queue** is the other tool for the same goal from the opposite angle: instead of fighting over who does a job, producers put jobs in a queue and competing workers each take one. The queue hands each job to one worker at a time and gives it back if that worker disappears. Add the **outbox pattern** (write the event to a table in the same database transaction as the state change, and publish it afterwards) and you can publish messages reliably without a distributed transaction between your database and your broker.

2 Why it matters in DevOps

- **Duplicate cron jobs.** Run a scheduled task on three replicas and, without a lock or leader election, you send three invoices. Every Kubernetes CronJob/Deployment decision runs into this.
- **Stale lock holders corrupt data.** A pause (garbage collection, a VM live migration, an overloaded node) outlives the lease; a second worker takes over; then the first wakes up and writes. A naive Redis lock cannot stop that; a **fencing token** can.
- **Queues smooth load and isolate failure,** but bring the delivery semantics from Idempotency §13.14: jobs may run twice, and a job stuck in "processing" must be re-queued.
- **The dual-write bug** (update the database, then publish an event, and crash in between) silently loses events. The outbox pattern is the standard fix.
- Operating all of this means monitoring queue depth, oldest-job age, lock contention and lease expiries; see Observability §0.10.

3 Real-world system example

- **Kubernetes leader election.** Controllers such as `kube-scheduler` and `kube-controller-manager` run several replicas but only one is active; the active one holds a `Lease` object (API group `coordination.k8s.io`) and renews it periodically. If renewal stops, another replica acquires the lease. The Lease is stored in etcd, which gives it consensus-backed correctness (see Cluster Architecture §8.1).
- **Google Chubby** (OSDI 2006 paper) is the lock service behind much of Google's infrastructure; it introduced "sequencers", which are the same idea as fencing tokens. **Apache ZooKeeper**, **etcd** and **HashiCorp Consul** offer lock and leader-election recipes built on consensus and sessions or leases.
- **Redis as a lock and queue store.** Sidekiq (Ruby) and many other job libraries use Redis lists; the Redis documentation describes `SET key value NX PX` for locks and the Redlock algorithm for multi-node setups. Martin Kleppmann's "How to do distributed locking" and Redis author Salvatore Sanfilippo's reply are the standard public debate about whether Redlock is safe for correctness-critical locks (Kleppmann's argument: you need fencing tokens regardless).
- **PostgreSQL as a queue.** `SELECT ... FOR UPDATE SKIP LOCKED` (PostgreSQL 9.5 and later) lets many workers pull different rows without blocking each other. Job libraries such as Oban, GoodJob and River are built on it, and advisory locks (`pg_advisory_lock`) give cheap cross-process mutual exclusion inside a database you already run.
- **Amazon SQS** uses a **visibility timeout**: a received message is hidden for a period; if the worker does not delete it in time it reappears. That is a lease on a message.
- **Debezium** (open source, CNCF-hosted ecosystem) reads the database's change log and publishes it to Kafka, and documents an **outbox event router** for the pattern described here (see Kafka §13.8).

4 Diagram

The lock-expiry hazard and how a fencing token closes it:

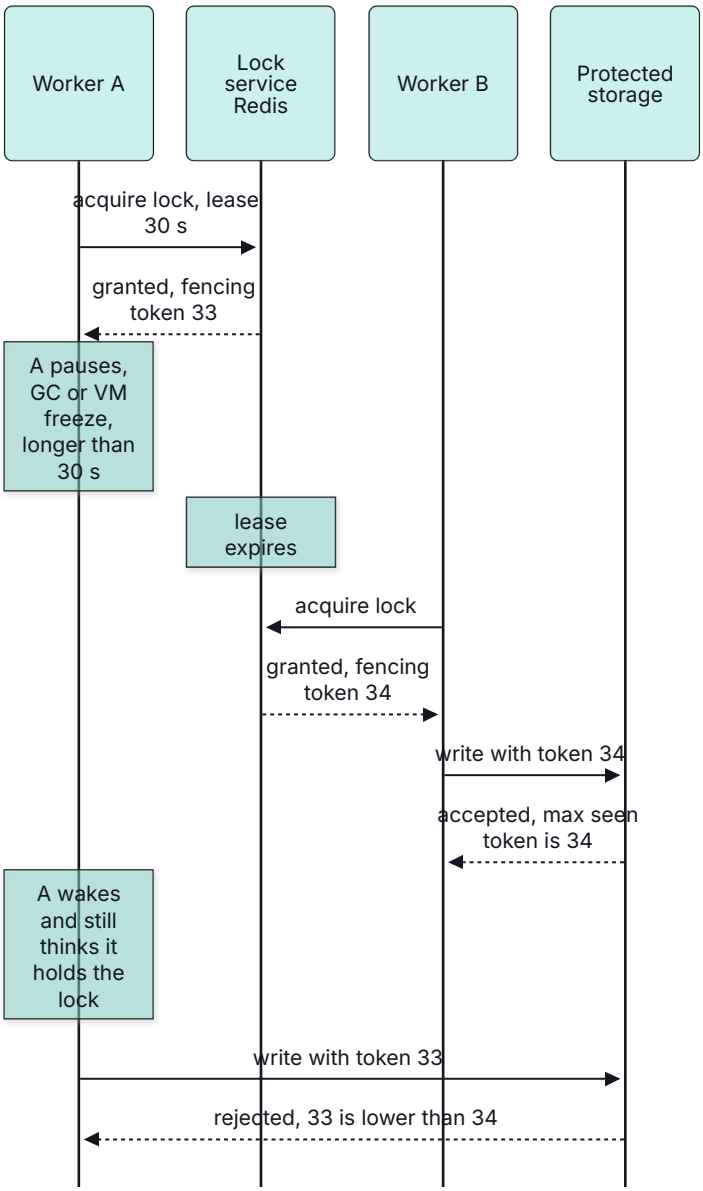


Figure 13.15-1

The outbox pattern, which avoids writing to the database and the broker as two separate steps:

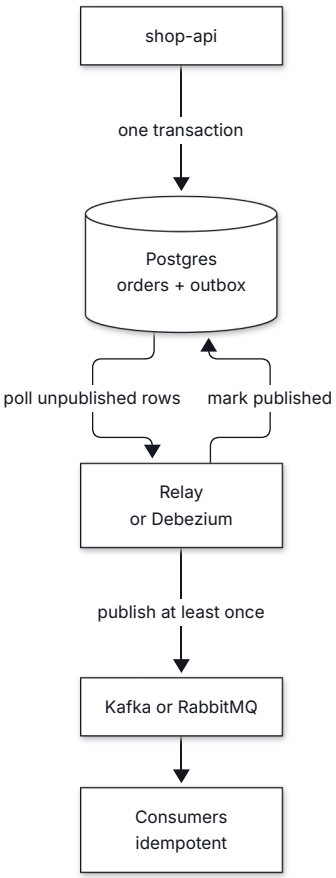


Figure 13.15-2

5 How it works

Locks: correctness versus efficiency

Ask what happens if two workers hold the lock at once:

- **Efficiency lock:** the work is wasted but harmless (two workers rebuild the same cache). A single Redis node with a TTL is fine.
- **Correctness lock:** data is damaged (double payment, corrupted file). A lease alone is *not enough*, because leases can expire while the holder is paused. Use a fencing token checked by the protected resource, or better, avoid the lock and use a conditional write

(`UPDATE ... WHERE version = n`) or a consensus-backed system.

The Redis lock recipe

Piece	Why
`SET lock:name <random-token> NX PX 30000`	`NX` = only if absent (mutual exclusion), `PX` = expires after 30 s (a crashed holder cannot block forever), token = identifies the holder
Release with a **Lua script**: delete only if the value equals my token	prevents deleting a lock that expired and was acquired by someone else
Renew (extend `PX`) while working, only if the token still matches	long jobs keep the lease; a "watchdog" thread does this
`INCR fence:name` at acquisition	a monotonically increasing **fencing token** to send with every write
Never `SETNX` then `EXPIRE` as two commands	a crash in between leaves a lock that never expires

Caveats worth knowing: with a single Redis node, a failover can lose the lock (asynchronous replication, see CAP §13.12 and Redis §13.7); Redlock tries to fix that using several independent nodes but remains debated for correctness locks; clock jumps and pauses still break lease assumptions.

Lock options compared

Option	Strength	Weakness
Redis `SET NX PX`	fast, simple, already deployed	not consensus-backed, pauses and failover can break it
PostgreSQL advisory lock (`pg_try_advisory_lock`)	released automatically when the session ends, no extra system	tied to a connection (watch connection poolers, see Connection Pooling §13.6)
etcd / ZooKeeper / Consul	consensus-backed leases and sessions	more infrastructure, still needs fencing for external side effects
Kubernetes `Lease`	built-in leader election for pods	coarse (seconds), for controller-style leadership
Database row with version check	no separate lock service	not a lock: it detects conflicts after the fact

Work queues: patterns and their lease mechanism

Queue	Claim	Lease / recovery	Notes
Redis list (reliable queue)	`LMOVE jobs processing RIGHT LEFT`	a sweeper moves old items from `processing` back	plain `RPOP` loses the job if the worker dies

Queue	Claim	Lease / recovery	Notes
Redis Streams + consumer group	`XREADGROUP`	`XPENDING` lists unacked, `XAUTOCLAIM` reassigns idle ones, `XACK` completes	built-in acknowledgement tracking
Postgres table	`UPDATE ... WHERE id = (SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1)`	`claimed_at` plus a sweeper	transactional with your data; modest throughput
SQS	`ReceiveMessage`	visibility timeout	delete after success
RabbitMQ	consume with manual ack, `prefetch`	unacked messages requeue when the channel closes	see RabbitMQ §13.9
Kafka	consumer group assigns partitions	rebalance when a consumer misses heartbeats	ordering per partition, see Kafka §13.8

Everything above is **at-least-once**: pair it with idempotent handlers.

The outbox pattern

The *dual-write problem*: `COMMIT` the order, then `publish(event)`. Crash between them and the event is lost; publish first, then commit fails, and a phantom event exists. Instead, in **one** database transaction insert the order *and* a row into an `outbox` table. A separate relay reads unpublished rows, publishes them and marks them done (or a change-data-capture tool like Debezium streams the table). A crash after publish but before marking re-publishes, so the event carries a stable `event_id` for consumers to deduplicate.

6 Step-by-step guide

Prerequisites: Docker; Python 3.9+ with `pip install redis psycopg2-binary` for the fencing and queue scripts. Cost: free. Time: ~75 minutes. Commands and scripts were run against Redis 7.0.15 and PostgreSQL 16.15 started natively; Docker versions are the standard equivalent, and ids, timings and tokens will differ ("similar to").

1. **Start Redis and Postgres.**

```bash
docker run -d --name redis -p 127.0.0.1:6379:6379 redis:7
docker run -d --name pg -e POSTGRES_PASSWORD=<YOUR_PASSWORD> -p 127.0.0.1:5432:5432
  postgres:16
docker exec redis redis-cli ping            # PONG
```

2. **Take a lock, contend for it, release it safely.**

```bash
docker exec redis redis-cli SET lock:report-42 tokenA NX PX 30000     # OK  (A holds it)
docker exec redis redis-cli SET lock:report-42 tokenB NX PX 30000     # (nil)  B is refused
docker exec redis redis-cli PTTL lock:report-42                       # about 29990 ms left
docker exec redis redis-cli EVAL "if redis.call('get',KEYS[1])==ARGV[1] then return redis.call('del',KEYS[1]) else return 0 end" 1 lock:report-42 tokenB
docker exec redis redis-cli EVAL "if redis.call('get',KEYS[1])==ARGV[1] then return redis.call('del',KEYS[1]) else return 0 end" 1 lock:report-42 tokenA
```

The first `EVAL` (B's token) returns `0`: B cannot release A's lock. The second returns `1`. The compare-and-delete must be one atomic script, otherwise there is a race between `GET` and `DEL`.

3. **Watch a lease expire.**

```bash
docker exec redis redis-cli SET lock:x t NX PX 100
sleep 0.2; docker exec redis redis-cli GET lock:x      # (nil): expired, someone else could now take it
```

4. **Reproduce the stale-holder problem and fix it with a fencing token.** Save as `fence.py`:

```python
import time, uuid, redis

r = redis.Redis(port=6379, decode_responses=True)
RELEASE = "if redis.call('get',KEYS[1])==ARGV[1] then return redis.call('del',KEYS[1]) else return 0 end"

class Storage:                                  # stands in for the resource the lock protects
    def __init__(self, check_fence): self.check, self.max_fence, self.data = check_fence, 0, None
    def write(self, who, fence, data):
        if self.check and fence < self.max_fence:
            print(f"  storage: REJECT {who} (fence {fence} < {self.max_fence})"); return
        self.max_fence, self.data = max(self.max_fence, fence), data
        print(f"  storage: accept {who} (fence {fence})")

def acquire(ttl_ms=500):
    token = uuid.uuid4().hex                    # unique per holder, so only the holder can release
    if r.set("lock:report", token, nx=True, px=ttl_ms):
```

```
        return token, r.incr("fence:report")     # strictly increasing fencing token
    return None, None

def race(check_fence):
    r.delete("lock:report", "fence:report")
    store = Storage(check_fence)
    ta, fa = acquire();  print("  A got the lock, fence", fa)
    time.sleep(0.7)                              # A stalls (GC pause, VM freeze) past
  its 500 ms lease
    tb, fb = acquire();  print("  B got the lock, fence", fb)
    store.write("B", fb, "written by B")
    store.write("A", fa, "written by A")         # A wakes up and still believes it
  holds the lock
    print("  final data:", store.data)
    print("  A release →", r.eval(RELEASE, 1, "lock:report", ta), "| B release →",
  r.eval(RELEASE, 1, "lock:report", tb))

print("Without a fencing check:"); race(False)
print("With a fencing check:");    race(True)
```

BASH

```
python3 fence.py
```

```
Without a fencing check:
  A got the lock, fence 1
  B got the lock, fence 2
  storage: accept B (fence 2)
  storage: accept A (fence 1)
  final data: written by A
  A release → 0 | B release → 1
With a fencing check:
  A got the lock, fence 1
  B got the lock, fence 2
  storage: accept B (fence 2)
  storage: REJECT A (fence 1 < 2)
  final data: written by B
  A release → 0 | B release → 1
```

In the first run the lock held for A, expired, and A still overwrote B: the lock gave no safety. In the second run the *resource* refuses the old token. Note that the safe release (`A release → 0`) is not enough on its own.

5. **Build a reliable Redis list queue (and see why `RPOP` is not reliable).**

```bash
docker exec redis redis-cli LPUSH jobs job1 job2 job3
docker exec redis redis-cli LMOVE jobs processing RIGHT LEFT      # "job1": claimed,
  now in the processing list
docker exec redis redis-cli LRANGE processing 0 -1                # job1
docker exec redis redis-cli LREM processing 1 job1                # 1: worker
  finished, ack by removal
```

   If the worker crashed instead, `job1` would still sit in `processing`; a sweeper moves it back with `LMOVE processing jobs RIGHT LEFT` after a timeout. With plain `RPOP`, the job would just be gone.

6. **Use Redis Streams for built-in tracking of unacknowledged work.**

```bash
docker exec redis redis-cli XGROUP CREATE jobs:stream workers '$' MKSTREAM
docker exec redis redis-cli XADD jobs:stream '*' task resize:1
docker exec redis redis-cli XREADGROUP GROUP workers w1 COUNT 1 STREAMS jobs:stream
  '>'     # w1 receives it, never acks (crash)
sleep 1.2
docker exec redis redis-cli XAUTOCLAIM jobs:stream workers w2 1000 0-0 COUNT 10
  # w2 takes over anything idle over 1000 ms
```

   `XAUTOCLAIM` returns the stalled entry (`task resize:1`) now owned by `w2`; after processing, `XACK jobs:stream workers <id>` clears it from the pending list (check with `XPENDING jobs:stream workers`).

7. **Build a Postgres work queue with `SKIP LOCKED`.** Save as `q.py`:

```python
import os, threading, time, psycopg2
DSN = os.environ["PRIMARY_DSN"]        # e.g. postgresql://postgres:
  <YOUR_PASSWORD>@127.0.0.1:5432/postgres
with psycopg2.connect(DSN) as c, c.cursor() as cur:
    cur.execute("DROP TABLE IF EXISTS jobs; CREATE TABLE jobs (id bigserial PRIMARY
  KEY, payload text, state text NOT NULL DEFAULT 'queued', claimed_by text, claimed_at
  timestamptz)")
    cur.execute("INSERT INTO jobs (payload) SELECT 'email-' || g FROM
  generate_series(1, 20) g")
CLAIM = """UPDATE jobs SET state='running', claimed_by=%s, claimed_at=now()
 WHERE id = (SELECT id FROM jobs WHERE state='queued' ORDER BY id FOR UPDATE SKIP
  LOCKED LIMIT 1)
 RETURNING id"""
done = {}
def worker(name):
    conn = psycopg2.connect(DSN); got = []
    while True:
```

```
        with conn, conn.cursor() as cur:
            cur.execute(CLAIM, (name,)); row = cur.fetchone()
        if not row: break
        got.append(row[0]); time.sleep(0.01)
    done[name] = got
ts = [threading.Thread(target=worker, args=(f"w{i}",)) for i in range(4)]
[t.start() for t in ts]; [t.join() for t in ts]
allids = sorted(i for g in done.values() for i in g)
print({k: len(v) for k, v in sorted(done.items())}, "total", len(allids), "unique",
  len(set(allids)))
```

BASH

```
export PRIMARY_DSN="postgresql://postgres:<YOUR_PASSWORD>@127.0.0.1:5432/postgres"
python3 q.py
```

```
{'w0': 5, 'w1': 5, 'w2': 5, 'w3': 5} total 20 unique 20
```

Four workers drained 20 jobs with no job claimed twice and no worker blocked by another (the split between workers varies run to run). The `state='running'` rows with an old `claimed_at` are the ones a sweeper would reset to `queued`.

8. **Try a Postgres advisory lock** (a lock tied to a session). In two `psql` sessions, run `SELECT pg_try_advisory_lock(42);` in the first (`t`), then in the second (`f`); close the first session and run it again in the second (`t`). Because the lock disappears when its connection ends, it self-heals after a crash, but a connection pooler in transaction mode would break the assumption (use `pg_advisory_xact_lock` inside a transaction there).
9. **Implement the outbox.** One transaction writes the order and the event; a rolled-back transaction leaves neither.

BASH

```
docker exec -i pg psql -U postgres <<'SQL'
CREATE TABLE orders (id bigserial PRIMARY KEY, customer_id int NOT NULL, total_cents
  int NOT NULL);
CREATE TABLE outbox (id bigserial PRIMARY KEY, event_id uuid NOT NULL DEFAULT
  gen_random_uuid(),
                     topic text NOT NULL, payload jsonb NOT NULL,
                     created_at timestamptz NOT NULL DEFAULT now(), published_at
  timestamptz);
BEGIN;
INSERT INTO orders (customer_id, total_cents) VALUES (7, 4999);
INSERT INTO outbox (topic, payload)
  VALUES ('orders.created', jsonb_build_object('order_id', currval('orders_id_seq'),
  'customer_id', 7));
COMMIT;
BEGIN;
INSERT INTO orders (customer_id, total_cents) VALUES (8, 100);
```

```
INSERT INTO outbox (topic, payload) VALUES ('orders.created', '{"order_id": 2}');
ROLLBACK;
SELECT count(*) AS orders FROM orders;
SELECT id, topic, payload, published_at IS NULL AS pending FROM outbox;
SQL
```

```
 orders
--------
      1
 id |     topic      |              payload               | pending
----+----------------+------------------------------------+---------
  1 | orders.created | {"order_id": 1, "customer_id": 7} | t
```

Now the relay's claim step, which several relays can run safely at once:

BASH

```
docker exec pg psql -U postgres -c "BEGIN; SELECT id, payload FROM outbox WHERE
  published_at IS NULL ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED; UPDATE outbox SET
  published_at = now() WHERE id IN (1); COMMIT;"
```

In a real relay, you would publish the selected rows to the broker *before* the `UPDATE`. A crash between the two means the next run republishes: that is why the event has an `event_id` and consumers dedupe. The capstone wires this end to end.

10. **Clean up.**

BASH

```
docker rm -f redis pg
```

7 Real-world scenario

Illustrative, not a documented incident. `shop-api` has a nightly job that generates invoices; it runs inside the API process on each of three replicas, guarded by `SET lock:invoices NX PX 60000`. *Symptom:* a customer is invoiced twice, once at 02:00:01 and again at 02:01:30, with identical totals but different invoice numbers. *Investigation:* logs show replica 1 acquired the lock at 02:00:00. A stop-the-world garbage collection pause of about 70 seconds (a heap-size misconfiguration) let the 60-second lease expire; replica 2 acquired the lock and began generating invoices; replica 1 woke up and carried on from where it stopped, believing it still held the lock. *Fix:* short term, an `invoices` table with `UNIQUE (customer_id, billing_period)` so the second insert fails harmlessly (idempotency beats locking); medium term, the job moved to a single Kubernetes CronJob with `concurrencyPolicy: Forbid`, and

the lock now returns a fencing token that the invoice writer checks. *Lesson:* a lease only bounds how long a *dead* holder blocks others; it cannot stop a *paused* one. Correctness has to be enforced where the data lives.

8 Common mistakes and best practices

Don't

- Use `SETNX` followed by `EXPIRE`, a lock with no expiry, or a fixed token such as `1`.
- Release a lock with plain `DEL`: you may delete someone else's lock after your lease expired.
- Rely on a lock for correctness when the protected resource cannot check a fencing token or version.
- Use `RPOP` / `BRPOP` for jobs you cannot lose (a crash loses the job).
- Publish to the broker and write to the database as two independent steps.
- Hold a lock across slow external calls without renewing, or assume wall clocks agree across machines.

Do

- Prefer designs that need no lock: unique constraints, conditional updates, idempotent handlers, partitioning work by key.
- Use a random token per acquisition, Lua compare-and-delete to release, a TTL longer than the worst normal run, and a renewal watchdog for long work.
- Send a fencing token with each write to the resource and reject lower ones.
- Use consensus-backed leases (etcd, ZooKeeper, Kubernetes Lease) for leader election of important components.
- Make every worker idempotent, set visibility timeouts or ack deadlines above the normal job time, and sweep stuck jobs.
- Use the outbox plus a stable `event_id`, and monitor queue depth, oldest job age and outbox lag.

9 Tasks

- [ ] **T1 — Take and release a Redis lock correctly** *(≈20 min, beginner)*
  - **Goal:** use `SET NX PX` and a token-checked release.
  - **Steps:**
    1. Run step 2 with two tokens.

2. Observe `PTTL` count down, then let a lock expire (step 3).

- **Done when:** B is refused, B's release returns 0, A's release returns 1, and an expired key returns `(nil)`.

- [ ] **T2 — Reproduce and fix the stale holder** *(≈30 min, intermediate)*
  - **Goal:** see a lock fail and a fencing token save the data.
  - **Steps:**
    1. Run `fence.py` as in step 4.
    2. Change the stall to `0.3` s (shorter than the lease). B is now refused, so add a guard in `race()`: `if tb is None: print(" B refused"); return`.
    3. Explain why the short-stall run is safe even without the check (A's lease is still valid) while the long-stall run needs it.
  - **Done when:** you can show the output for the long stall with and without the check, and the short-stall run, and your explanation matches.

- [ ] **T3 — Build and crash a reliable queue** *(≈40 min, intermediate)*
  - **Goal:** prove no job is lost when a worker dies.
  - **Steps:**
    1. Push 10 jobs; write a worker that uses `LMOVE ... processing`, sleeps 1 s, then `LREM` s.
    2. Kill the worker mid-job (Ctrl+C), then write a sweeper that moves jobs back after 5 s.
    3. Start a second worker and run to completion.
  - **Done when:** all 10 jobs are completed exactly once from your log, and `LLEN processing` is 0.

- [ ] **T4 — Break it: duplicate delivery** *(≈30 min, advanced)*
  - **Goal:** show at-least-once in action and neutralise it.
  - **Steps:**
    1. In the Streams demo, have a worker read but not ack, let `w2` `XAUTOCLAIM` it, and process the entry in both workers.
    2. Add a `processed_events` insert (see Idempotency §13.14) so the second processing is skipped.
  - **Done when:** the effect (a counter) increments once even though two workers handled the message.

- [ ] **T5 — Outbox with a crashing relay** *(≈50 min, advanced)*

- **Goal:** demonstrate no lost events and tolerated duplicates.
- **Steps:**
  1. Create the outbox (step 9), insert 5 orders and write a relay that prints events and then marks them published.
  2. Raise an exception after printing but before the `UPDATE` once; run the relay again.
  3. Count how often each `event_id` was printed.
- **Done when:** all 5 events were delivered, at least one was delivered twice, and a dedup set of `event_id`s reduces it to exactly 5.

☐ **T6 — Prove it: choose the mechanism** *(≈40 min, advanced)*

- **Goal:** justify lock versus queue versus constraint.
- **Steps:**
  1. Without the guide, choose and justify a mechanism for: nightly invoice generation, cache stampede on a hot key, leader of a stream processor, sending one welcome email per signup.
  2. For each, state what happens when a worker freezes for twice the lease.
- **Done when:** each choice names the failure case and the control (fencing, idempotency key, unique constraint, consensus lease) that handles it.

10 Self-check questions

1. Why must the lock value be a random token, and why must release be a script?
2. A worker holds a Redis lock with a 30 s TTL and pauses for 45 s. What goes wrong, and what stops the damage?
3. When is a plain Redis `SET NX PX` lock good enough, and when is it not?
4. What is the dual-write problem, and how does the outbox solve it?
5. A job was claimed by a worker that crashed. How does each of SQS, RabbitMQ and a Postgres queue get the job back?

ANSWERS

1. The token identifies the holder so you never release (or renew) a lock that expired and now belongs to someone else. Compare-and-delete must be atomic, so it runs as one Lua script; separate `GET` and `DEL` leave a race.
2. The lease expires at 30 s, another worker takes the lock, and the first resumes and writes with no idea it lost the lock. A fencing token checked by the resource (or a conditional write) rejects the stale write.

3. Good enough for efficiency locks where duplicate work is harmless. Not enough for correctness locks (money, files, leadership) unless the resource enforces fencing, because failover, pauses and clock issues break the guarantee.
4. Updating the database and publishing to a broker are two operations that cannot be made atomic; a crash between them loses or invents an event. The outbox writes the event in the same transaction as the state change and a relay publishes it afterwards, at least once.
5. SQS: the visibility timeout expires and the message reappears. RabbitMQ: the unacked message is requeued when the consumer's channel or connection closes. Postgres: a sweeper resets rows whose `claimed_at` is too old (or the transaction rolls back if the claim and work share one).

11 Further reading

- Redis documentation: distributed locks with Redis (Redlock) and the Streams and `SET` command pages.
- Martin Kleppmann, *How to do distributed locking* (blog) and *Designing Data-Intensive Applications*, chapter on the trouble with distributed systems.
- Mike Burrows, *The Chubby Lock Service for Loosely-Coupled Distributed Systems* (OSDI 2006).
- Kubernetes documentation: leases and leader election.
- PostgreSQL documentation: explicit locking (advisory locks) and `SELECT ... FOR UPDATE SKIP LOCKED`.
- Chris Richardson, *Microservices Patterns* and microservices.io: the Transactional Outbox pattern. Debezium documentation: Outbox Event Router.

LAB NOTES What I ran, what I saw, what it means

LEVEL 13: DATABASES & DISTRIBUTED SYSTEMS

★ Postgres, Redis and Kafka under failure

Level 13 capstone ~12 hours

1 Goal and outcome

Build a small but real data platform for the `shop-api` sample app on your laptop, then **break it on purpose** and prove you understand what happens:

- **PostgreSQL primary + streaming replica**, with WAL archiving and a base backup.
- **Redis** for caching and a fenced distributed lock.
- **Kafka** (KRaft mode, one broker) fed through a **transactional outbox**, with an **idempotent consumer**.
- Three demonstrations the roadmap asks for: **replication lag** (and a read-your-writes fix), an **idempotent consumer** that survives duplicate delivery, and a **point-in-time recovery (PITR) restore** that undoes a destructive `DELETE`.

When you finish you will have exercised all 15 topics of Level 13, a repeatable lab, and evidence (command output) for each claim.

Topics exercised	Where
01 Indexes §13.1, 03 MVCC §13.3, 11 Partitioning §13.11	Milestone 1
04 Replication §13.4, 13 Eventual consistency §13.13, 10 Scaling §13.10	Milestone 2
07 Redis §13.7, 15 Locks and queues §13.15	Milestone 3
08 Kafka §13.8, 14 Idempotency §13.14, 02 Transactions §13.2, 09 RabbitMQ §13.9	Milestone 4
05 Backups and PITR §13.5	Milestone 5
12 CAP §13.12, 06 Connection pooling §13.6	Milestone 6

2 Architecture

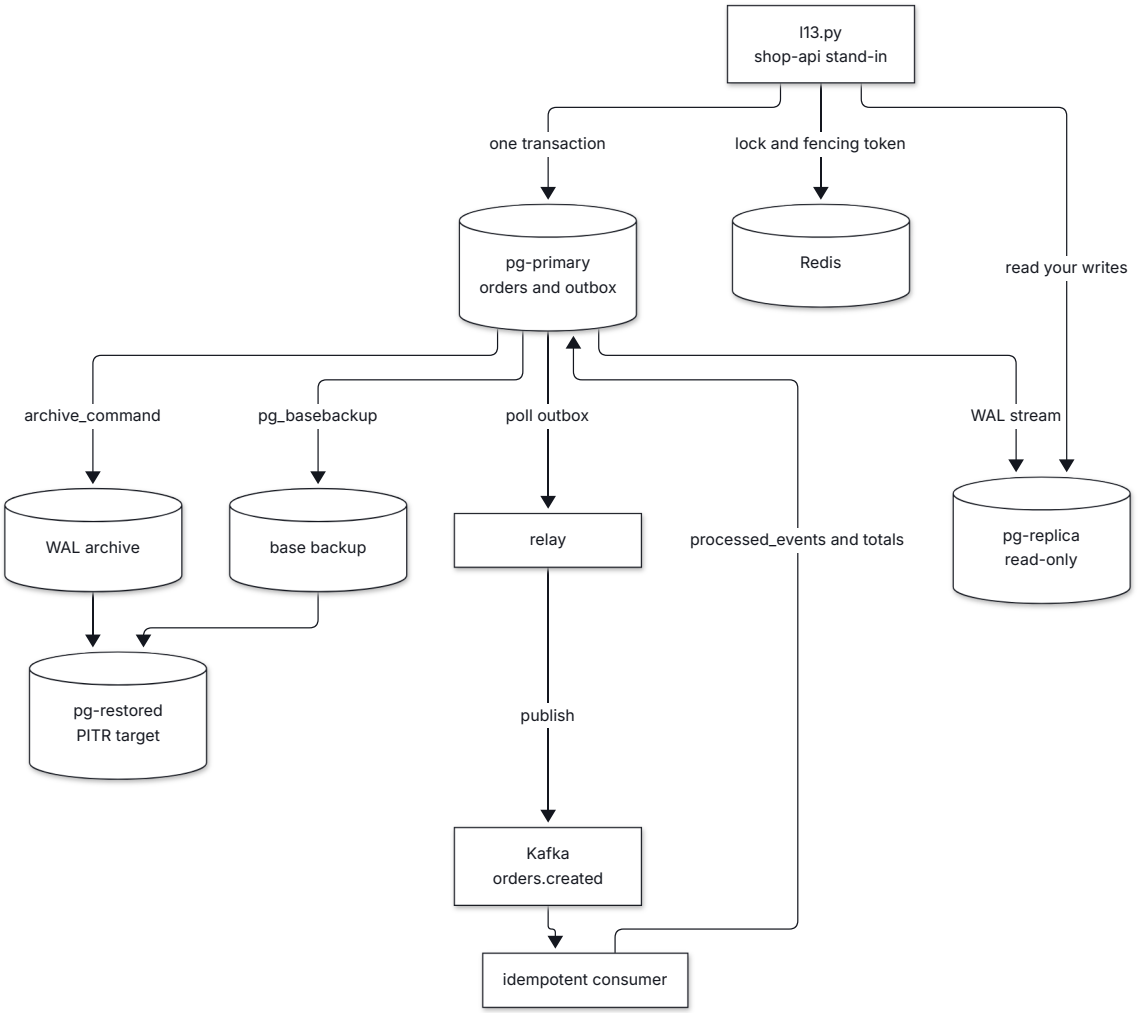


Figure 13-1

The life of one order, and the PITR timeline that rescues you from a bad delete:

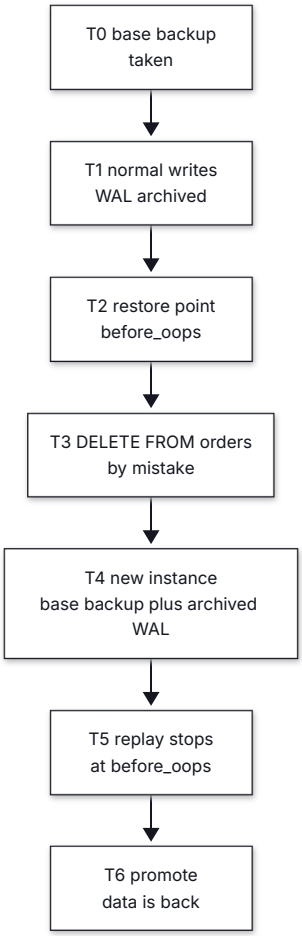


Figure 13-2

3 Prerequisites and cost

- **Cost:** free. Everything runs in containers on your machine; no cloud account is needed.
- **Machine:** 8 GB RAM free recommended, about 5 GB disk, Docker with Compose v2, Python 3.9+, `psql` client (optional but handy).
- **Python packages:** `pip install psycopg2-binary redis confluent-kafka`.
- **Prior reading:** the topic files linked above; you can do the capstone with the guides open.

- **Verification note:** the SQL, WAL archiving and PITR settings, lag measurement, the Python script, the Redis commands and the Kafka commands in this capstone were run natively on PostgreSQL 16.15, Redis 7.0.15, Apache Kafka 3.9.0 (KRaft) and Python 3.13 with psycopg2, redis-py and confluent-kafka 2.x. The Docker and Compose wrappers are the standard forms, but they were not run in the authoring sandbox (no Docker daemon), so container names, IP addresses, LSNs, timings and IDs in the sample output will differ ("similar to"). All passwords are placeholders.

4 Build it step by step

Milestone 0 — Start the lab

Create a working directory (`mkdir l13-lab && cd l13-lab`) and save this as `compose.yaml` :

YAML

```yaml
name: l13
services:
  init-volumes:                       # volumes are created root-owned; the postgres user
  has uid 999 in the official image
    image: busybox:1.36
    command: ["chown", "-R", "999:999", "/archive", "/backup"]
    volumes: ["pg-archive:/archive", "pg-backup:/backup"]
  pg-primary:
    image: postgres:16
    container_name: pg-primary
    depends_on:
      init-volumes: { condition: service_completed_successfully }
    environment:
      POSTGRES_PASSWORD: ${PG_PASSWORD:?export PG_PASSWORD first}
      POSTGRES_DB: shop
    command:
      - postgres
      - -c
      - wal_level=replica
      - -c
      - max_wal_senders=10
      - -c
      - max_replication_slots=10
      - -c
      - wal_log_hints=on
      - -c
      - archive_mode=on
      - -c
      - archive_command=test ! -f /archive/%f && cp %p /archive/%f
    ports: ["127.0.0.1:5432:5432"]
    volumes: ["pg-primary-data:/var/lib/postgresql/data", "pg-archive:/archive", "pg-
  backup:/backup"]
  redis:
```

```
    image: redis:7
    container_name: redis
    ports: ["127.0.0.1:6379:6379"]
  kafka:
    image: apache/kafka:3.9.0        # single-node KRaft broker, advertises
  localhost:9092
    container_name: kafka
    ports: ["127.0.0.1:9092:9092"]
networks:
  default:
    name: l13net
volumes:
  pg-primary-data:
  pg-archive:
  pg-backup:
```

BASH

```
export PG_PASSWORD=<YOUR_PASSWORD>
docker compose up -d
docker exec pg-primary pg_isready -U postgres -d shop
docker exec redis redis-cli ping
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
  # empty list, no error
```

Expect `accepting connections`, `PONG`, and no error from Kafka (the broker may need ~15 seconds on first start).

Now the schema. Save as `schema.sql` and load it:

SQL

```
CREATE TABLE orders (
  id          bigserial PRIMARY KEY,
  customer_id int NOT NULL,
  item        text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE TABLE outbox (                       -- Milestone 4: events written in the same
  transaction as the order
  id           bigserial PRIMARY KEY,
  event_id     uuid NOT NULL DEFAULT gen_random_uuid(),
  topic        text NOT NULL,
  payload      jsonb NOT NULL,
  created_at   timestamptz NOT NULL DEFAULT now(),
  published_at timestamptz
);
CREATE TABLE processed_events (             -- Milestone 4: the consumer's dedup table
  (inbox)
  event_id     uuid PRIMARY KEY,
  processed_at timestamptz NOT NULL DEFAULT now()
);
```

```
CREATE TABLE order_totals (
  customer_id int PRIMARY KEY,
  orders_seen int NOT NULL DEFAULT 0
);
```

```bash
docker exec -i pg-primary psql -U postgres -d shop < schema.sql
```

Milestone 1 — Indexes, MVCC and partitioning (topics 01, 03, 11)

```bash
psqlp() { docker exec -i pg-primary psql -U postgres -d shop "$@"; }     # helper used in
  the next milestones
psqlp -c "INSERT INTO orders (customer_id, item) SELECT (random()*10000)::int, 'item-' ||
  g FROM generate_series(1, 300000) g" -c "ANALYZE orders"
psqlp -c "EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF) SELECT * FROM orders WHERE customer_id
  = 4242"
```

Expect a `Seq Scan` (possibly parallel) that removes about 300,000 rows by filter and runs in roughly 10-50 ms. Create the index and ask again:

```bash
psqlp -c "CREATE INDEX orders_customer_id_idx ON orders (customer_id)"
psqlp -c "EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF) SELECT * FROM orders WHERE customer_id
  = 4242"
```

```
 Bitmap Heap Scan on orders (actual rows=35 loops=1)
   Recheck Cond: (customer_id = 4242)
   Heap Blocks: exact=34
   →  Bitmap Index Scan on orders_customer_id_idx (actual rows=35 loops=1)
         Index Cond: (customer_id = 4242)
 Execution Time: 0.256 ms
```

Now see MVCC (an `UPDATE` writes a *new row version*, it does not overwrite):

```bash
psqlp -c "SELECT ctid, xmin, xmax FROM orders WHERE id = 1" -c "UPDATE orders SET item =
  'changed' WHERE id = 1" -c "SELECT ctid, xmin, xmax FROM orders WHERE id = 1"
```

The row's `ctid` (physical location) and `xmin` (creating transaction) change, e.g. `(0,1)` and `818` become `(2205,121)` and `821`. The old version is a dead tuple until `VACUUM`.

Finally, partition a time-series table so old data can be dropped instantly:

```bash
psqlp <<'SQL'
CREATE TABLE page_views (id bigserial, viewed_at timestamptz NOT NULL, path text NOT NULL,
  PRIMARY KEY (id, viewed_at)) PARTITION BY RANGE (viewed_at);
CREATE TABLE page_views_2026_09 PARTITION OF page_views FOR VALUES FROM ('2026-09-01') TO
  ('2026-10-01');
CREATE TABLE page_views_2026_10 PARTITION OF page_views FOR VALUES FROM ('2026-10-01') TO
  ('2026-11-01');
INSERT INTO page_views (viewed_at, path) SELECT '2026-09-01'::timestamptz + (g || '
  minutes')::interval, '/p/' || (g % 50) FROM generate_series(1, 80000) g;
ANALYZE page_views;
EXPLAIN (COSTS OFF) SELECT count(*) FROM page_views WHERE viewed_at ⩾ '2026-10-05' AND
  viewed_at < '2026-10-06';
SQL
```

The plan scans only `page_views_2026_10` (partition pruning). Retention is then `ALTER TABLE page_views DETACH PARTITION page_views_2026_09; DROP TABLE page_views_2026_09;`, instant, with no `DELETE` and no bloat.

Milestone 2 — Replica, lag and read-your-writes (topics 04, 13, 10)

Create the replication role, allow it in `pg_hba.conf`, and seed a standby with `pg_basebackup` (same method as Replication §13.4):

```bash
docker exec pg-primary psql -U postgres -c "CREATE ROLE replicator WITH REPLICATION LOGIN
  PASSWORD '<REPL_PASSWORD>';"
docker exec pg-primary bash -c "echo 'host replication replicator all scram-sha-256' >>
  /var/lib/postgresql/data/pg_hba.conf"
docker exec pg-primary psql -U postgres -c "SELECT pg_reload_conf();"
docker volume create pg-replica-data
docker run --rm --network l13net --user postgres -e PGPASSWORD=<REPL_PASSWORD> \
  -v pg-replica-data:/var/lib/postgresql/data postgres:16 \
  pg_basebackup -h pg-primary -U replicator -D /var/lib/postgresql/data -R -C -S replica1
  -X stream -c fast -P
docker run -d --name pg-replica --network l13net -p 127.0.0.1:5433:5432 -v pg-replica-
  data:/var/lib/postgresql/data postgres:16
sleep 5
docker exec pg-primary psql -U postgres -x -c "SELECT application_name, state, sync_state,
  replay_lag FROM pg_stat_replication;"
```

```
application_name | walreceiver
state            | streaming
sync_state       | async
replay_lag       | 00:00:00.000249
```

(`application_name` is `walreceiver` unless you set one in `primary_conninfo`; the slot is `replica1`.) The replica is read-only and in recovery:

```bash
docker exec pg-replica psql -U postgres -d shop -c "SELECT pg_is_in_recovery();" -c
  "SELECT count(*) FROM orders;"
```

Now save the lab script as `l13.py` (it is used by Milestones 2-4). Set the connection strings first (replace the placeholder password):

```bash
export PRIMARY_DSN="postgresql://postgres:<YOUR_PASSWORD>@127.0.0.1:5432/shop"
export REPLICA_DSN="postgresql://postgres:<YOUR_PASSWORD>@127.0.0.1:5433/shop"
```

```python
#!/usr/bin/env python3
"""L13 capstone lab: outbox relay, idempotent consumer, lag probe, fenced Redis lock."""
import json, os, sys, time, uuid
import psycopg2, redis
from confluent_kafka import Consumer, Producer

PRIMARY = os.getenv("PRIMARY_DSN", "postgresql://postgres:
  <YOUR_PASSWORD>@127.0.0.1:5432/shop")
REPLICA = os.getenv("REPLICA_DSN", "postgresql://postgres:
  <YOUR_PASSWORD>@127.0.0.1:5433/shop")
KAFKA = os.getenv("KAFKA", "127.0.0.1:9092")
TOPIC = "orders.created"
R = redis.Redis.from_url(os.getenv("REDIS_URL", "redis://127.0.0.1:6379/0"),
  decode_responses=True)


def order(customer_id, item):
    """Outbox pattern: the order row and the event row commit atomically."""
    with psycopg2.connect(PRIMARY) as c, c.cursor() as cur:
        cur.execute("INSERT INTO orders (customer_id, item) VALUES (%s, %s) RETURNING id",
  (customer_id, item))
        oid = cur.fetchone()[0]
        cur.execute("INSERT INTO outbox (topic, payload) VALUES (%s, %s)",
                    (TOPIC, json.dumps({"order_id": oid, "customer_id": customer_id,
  "item": item})))
    with psycopg2.connect(PRIMARY) as c, c.cursor() as cur:   # AFTER the commit: LSN at
  or beyond our commit record
        cur.execute("SELECT pg_current_wal_lsn()")
        return oid, cur.fetchone()[0]


def relay():
```

```
    """Publish unpublished outbox rows. A crash after publish but before UPDATE re-sends
  them: at-least-once."""
    p = Producer({"bootstrap.servers": KAFKA, "enable.idempotence": True})
    with psycopg2.connect(PRIMARY) as c, c.cursor() as cur:
        cur.execute("SELECT id, event_id, payload FROM outbox WHERE published_at IS NULL "
                    "ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED")
        rows = cur.fetchall()
        for _id, event_id, payload in rows:
            p.produce(TOPIC, key=str(payload["customer_id"]), value=json.dumps(payload),
                      headers=[("event_id", str(event_id).encode())])
        p.flush()
        cur.execute("UPDATE outbox SET published_at = now() WHERE id = ANY(%s)", ([r[0]
  for r in rows],))
    print(f"relayed {len(rows)} events")


def handle(conn, event_id, payload):
    """Idempotent handler: dedup row and business effect share ONE transaction."""
    with conn.cursor() as cur:
        cur.execute("INSERT INTO processed_events (event_id) VALUES (%s) ON CONFLICT DO
  NOTHING", (event_id,))
        if cur.rowcount == 0:
            conn.commit()
            return "duplicate-skipped"
        cur.execute("INSERT INTO order_totals (customer_id, orders_seen) VALUES (%s, 1) "
                    "ON CONFLICT (customer_id) DO UPDATE SET orders_seen =
  order_totals.orders_seen + 1",
                    (payload["customer_id"],))
    conn.commit()
    return "applied"


def consume(max_idle=10):
    c = Consumer({"bootstrap.servers": KAFKA, "group.id": "totals", "auto.offset.reset":
  "earliest",
                  "enable.auto.commit": False})
    c.subscribe([TOPIC])
    conn, idle, seen = psycopg2.connect(PRIMARY), 0, {"applied": 0, "duplicate-skipped":
  0}
    while idle < max_idle:
        m = c.poll(1.0)
        if m is None or m.error():
            idle += 1
            continue
        idle = 0
        event_id = dict(m.headers())["event_id"].decode()
        seen[handle(conn, event_id, json.loads(m.value()))] += 1
        c.commit(message=m)          # offset commit AFTER the database commit
    c.close()                        # leave the group cleanly so the next run does not
  wait for a session timeout
    print(seen)
```

```
def lag_probe(customer_id=999):
    oid, lsn = order(customer_id, "probe")
    with psycopg2.connect(REPLICA) as r, r.cursor() as cur:
        cur.execute("SELECT count(*) FROM orders WHERE id = %s", (oid,))
        print("naive replica read sees row:", cur.fetchone()[0] == 1)
        t0 = time.time()
        while True:                  # read-your-writes: wait until the replica replayed
  past our LSN
            cur.execute("SELECT pg_wal_lsn_diff(pg_last_wal_replay_lsn(), %s) ≥ 0",
  (lsn,))
            if cur.fetchone()[0]:
                break
            time.sleep(0.05)
        cur.execute("SELECT count(*) FROM orders WHERE id = %s", (oid,))
        print(f"after waiting {time.time()-t0:.2f}s replica sees row:", cur.fetchone()[0]
  == 1)


RELEASE = "if redis.call('get',KEYS[1])==ARGV[1] then return redis.call('del',KEYS[1])
  else return 0 end"


def lock_demo():
    with psycopg2.connect(PRIMARY) as c, c.cursor() as cur:
        cur.execute("CREATE TABLE IF NOT EXISTS job_state (id int PRIMARY KEY, fence
  bigint NOT NULL, data text)")
        cur.execute("INSERT INTO job_state VALUES (1, 0, NULL) ON CONFLICT DO NOTHING")

    def acquire(name):
        token = str(uuid.uuid4())
        if R.set("lock:job", token, nx=True, px=1000):       # 1 s lease
            return token, R.incr("fence:job")                # fencing token: strictly
  increasing
        return None, None

    def write(name, fence):
        with psycopg2.connect(PRIMARY) as c, c.cursor() as cur:
            cur.execute("UPDATE job_state SET fence=%s, data=%s WHERE id=1 AND fence <
  %s", (fence, name, fence))
            print(f"{name}: write with fence {fence} →", "ACCEPTED" if cur.rowcount else
  "REJECTED (stale)")

    ta, fa = acquire("A"); print("A holds lock, fence", fa)
    time.sleep(1.2)                                          # A pauses (GC, VM stall) and
  its lease expires
    tb, fb = acquire("B"); print("B holds lock, fence", fb)
    write("B", fb)
    write("A", fa)                                           # A wakes up and still
  believes it holds the lock
    print("A release:", R.eval(RELEASE, 1, "lock:job", ta), "(0 = lock was not ours any
  more)")
    print("B release:", R.eval(RELEASE, 1, "lock:job", tb))
```

```
if __name__ == "__main__":
    cmd, args = sys.argv[1], sys.argv[2:]
    {"order": lambda: print(order(int(args[0]), args[1])), "relay": relay, "consume":
  consume,
     "lag": lag_probe, "lock": lock_demo}[cmd]()
```

Demonstrate replication lag. Freeze replay on the replica, write on the primary, and compare:

```
BASH
docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_pause();"
docker exec pg-primary psql -U postgres -d shop -c "INSERT INTO orders (customer_id, item)
  SELECT 7, 'widget' FROM generate_series(1,1000);"
docker exec pg-primary psql -U postgres -d shop -c "SELECT application_name,
  pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_gap FROM
  pg_stat_replication;"
docker exec pg-replica psql -U postgres -d shop -c "SELECT pg_last_wal_receive_lsn() >
  pg_last_wal_replay_lsn() AS received_not_applied;" \
  -c "SELECT now() - pg_last_xact_replay_timestamp() AS apparent_lag;"
```

The primary reports a replay gap of some KB (it was 161 kB in the authoring run), and the standby says `received_not_applied = t`: the WAL has arrived but is not applied, so reads are stale. Then run the lag probe while a background command ends the pause after three seconds:

```
BASH
( sleep 3; docker exec pg-replica psql -U postgres -c "SELECT pg_wal_replay_resume();"
  >/dev/null ) &
python3 l13.py lag
```

```
naive replica read sees row: False
after waiting 2.84s replica sees row: True
```

The first line is the stale read (a user would see "order not found"). The second is **read-your-writes**: the script recorded the primary's WAL position (LSN) *after* its commit and waited until the replica replayed past it. With replay running normally, both lines say `True` and the wait is near 0.00 s.

Milestone 3 — Redis: cache and a fenced lock (topics 07, 15)

Cache-aside with a TTL, the cheapest form of horizontal read scaling:

```bash
docker exec redis redis-cli SET order:1 '{"id":1,"item":"widget"}' EX 30
docker exec redis redis-cli GET order:1
docker exec redis redis-cli TTL order:1            # counts down from 30
```

Now the lock demo: worker A takes a one-second lease, stalls for 1.2 seconds, worker B takes the lock, and both write to a database row that accepts only increasing fencing tokens:

```bash
python3 l13.py lock
```

```
A holds lock, fence 1
B holds lock, fence 2
B: write with fence 2 → ACCEPTED
A: write with fence 1 → REJECTED (stale)
A release: 0 (0 = lock was not ours any more)
B release: 1
```

(On a repeat run the fence numbers keep increasing.) The lock did not protect the data; the **fencing token checked by the database** did. See the Distributed Locks and Queues §13.15 discussion of leases.

Milestone 4 — Kafka, outbox and an idempotent consumer (topics 08, 14, 15, 02)

```bash
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --
  create --topic orders.created --partitions 3
for i in 1 2 3 4 5; do python3 l13.py order $((i % 2 + 1)) widget$i; done     # each
  prints (order_id, lsn)
python3 l13.py relay
python3 l13.py consume
docker exec pg-primary psql -U postgres -d shop -c "SELECT * FROM order_totals ORDER BY 1"
  -c "SELECT count(*) AS processed FROM processed_events"
```

Expect `relayed 5 events`, then `{'applied': 5, 'duplicate-skipped': 0}`, then totals that add up to the number of orders (for example `orders_seen` of 2 and 3) and `processed = 5`. (A harmless librdkafka warning about the idempotence PID being retried can appear on the first publish.)

Each `order` call wrote the order and the outbox row in **one transaction**, so there is never an event without an order or an order without an event. The relay publishes with the `event_id` as a header; the consumer commits its dedup row and business effect together, and commits the Kafka offset only afterwards.

Now simulate the crash window: the relay "forgot" it published (as if it died before its `UPDATE`), so it publishes the same five events again:

```bash
docker exec pg-primary psql -U postgres -d shop -c "UPDATE outbox SET published_at = NULL"
python3 l13.py relay
python3 l13.py consume
docker exec pg-primary psql -U postgres -d shop -c "SELECT * FROM order_totals ORDER BY 1"
  -c "SELECT count(*) AS processed FROM processed_events"
```

```
relayed 5 events
{'applied': 0, 'duplicate-skipped': 5}
```

The totals and `processed = 5` are **unchanged**: five duplicate deliveries, zero duplicate effects. If the consumer prints nothing for about a minute, a previous consumer in the same group exited without leaving it and the group is waiting out its session timeout; run it again after a minute (the script now closes its consumer cleanly to avoid this).

Milestone 5 — Backups and point-in-time recovery (topic 05)

The primary already archives every finished WAL segment to the `pg-archive` volume (`archive_mode=on`). Take a base backup, make some changes, set a named restore point, then destroy the table:

```bash
docker exec -u postgres pg-primary pg_basebackup -U postgres -D /backup/base1 -X stream -c
  fast
docker exec pg-primary psql -U postgres -d shop -c "INSERT INTO orders (customer_id, item)
  SELECT 100+g, 'late-order' FROM generate_series(1,50) g;"
docker exec pg-primary psql -U postgres -d shop -Atc "SELECT count(*) FROM orders;"
  # note this number (about 301056 if you followed every step)
docker exec pg-primary psql -U postgres -d shop -c "SELECT
  pg_create_restore_point('before_oops');"
docker exec pg-primary psql -U postgres -d shop -c "DELETE FROM orders;"
  # the mistake
docker exec pg-primary psql -U postgres -d shop -c "SELECT pg_switch_wal();"
  # force the last segment into the archive
sleep 3
docker exec pg-primary psql -U postgres -d shop -Atc "SELECT count(*) FROM orders;"
  # 0
```

(`DELETE FROM orders` also removes the rows from the replica within moments, so the replica is *not* a backup.) Now restore into a **new** volume with a recovery target, leaving the damaged primary untouched:

```bash
docker volume create pg-restored
cat > recovery-snippet.conf <<'CFG'
restore_command = 'cp /archive/%f %p'
recovery_target_name = 'before_oops'
recovery_target_action = 'promote'
CFG
docker run --rm --user root -v pg-backup:/backup:ro -v pg-restored:/restore -v
  "$PWD/recovery-snippet.conf:/snippet:ro" postgres:16 \
  bash -c 'cp -a /backup/base1/. /restore/ && cat /snippet >> /restore/postgresql.conf &&
  touch /restore/recovery.signal && chown -R postgres:postgres /restore && chmod 700
  /restore'
docker run -d --name pg-restored --network l13net -v pg-restored:/var/lib/postgresql/data
  -v pg-archive:/archive:ro postgres:16
sleep 8
docker exec pg-restored psql -U postgres -d shop -c "SELECT count(*) AS orders_restored
  FROM orders;" -c "SELECT pg_is_in_recovery();"
docker logs pg-restored 2>&1 | grep -E "restored log file|recovery stopping|promote"
```

```
 orders_restored
-----------------
          301056
 pg_is_in_recovery
-------------------
 f
... LOG:  restored log file "000000010000000000000004" from archive
... LOG:  recovery stopping at restore point "before_oops", time ...
```

The restored instance replayed archived WAL on top of the base backup, **stopped exactly at the named restore point**, and was promoted. Every row from before the `DELETE` is back, including the 50 `late-order` rows written *after* the backup was taken. That is the difference between a restore and a point-in-time restore. (In a real incident you would then copy the lost rows back to the primary, or switch traffic to the restored instance.)

Milestone 6 — Break it: CAP and pooling (topics 12, 06)

Choose consistency over availability. Make the primary wait for a standby on every commit, then take the standby away:

```bash
docker exec pg-primary psql -U postgres -c "ALTER SYSTEM SET synchronous_standby_names =
  'ANY 1 (*)';" -c "SELECT pg_reload_conf();"
docker exec pg-primary psql -U postgres -c "SELECT application_name, sync_state FROM
  pg_stat_replication;"      # sync_state = quorum
docker stop pg-replica
timeout 5 docker exec pg-primary psql -U postgres -d shop -c "INSERT INTO orders
  (customer_id, item) VALUES (1,'hang');"; echo "exit=$?"
docker exec pg-primary psql -U postgres -c "SELECT wait_event, left(query,30) FROM
  pg_stat_activity WHERE wait_event = 'SyncRep';"
```

The insert does not return (`exit=124` from `timeout`), and `wait_event = SyncRep` shows the commit waiting for the standby: this is the **CP choice**, refusing to acknowledge a write that exists on only one machine. Restore availability:

```bash
docker start pg-replica
docker exec pg-primary psql -U postgres -c "ALTER SYSTEM RESET synchronous_standby_names;"
  -c "SELECT pg_reload_conf();"
```

Connection pooling. PgBouncer (`sudo apt-get install pgbouncer`, or `brew install pgbouncer`) multiplexes many client connections onto a few server connections. Save as `pgbouncer.ini` and adjust the password:

```ini
[databases]
shop = host=127.0.0.1 port=5432 dbname=shop user=postgres password=<YOUR_PASSWORD>

[pgbouncer]
listen_addr = 127.0.0.1
listen_port = 6432
auth_type = trust
auth_file = userlist.txt
pool_mode = transaction
default_pool_size = 5
max_client_conn = 200
admin_users = postgres
```

```bash
echo '"postgres" ""' > userlist.txt            # lab only: trust auth on a loopback
  listener
pgbouncer -d pgbouncer.ini
for i in $(seq 1 60); do psql -h 127.0.0.1 -p 6432 -U postgres -d shop -qAtc "SELECT
  pg_sleep(1)" & done; sleep 1.5
psql -h 127.0.0.1 -p 6432 -U postgres -d pgbouncer -c "SHOW POOLS"
wait
```

Sixty clients ran at once, but `SHOW POOLS` shows `sv_active = 5` and most clients in `cl_waiting`: only five real PostgreSQL backends were used (the pooled run in the authoring sandbox showed 5 active servers and about 50 waiting clients). `auth_type = trust` and the empty userlist password are for a loopback-only lab; use `scram-sha-256` and a real userlist in any shared environment.

5 Tasks

- [ ] **T1 — Index and read the plan** *(≈30 min, beginner; topics 01, 03)*
  - **Goal:** prove an index changes the plan and the time, and see an MVCC row version move.
  - **Steps:**
    1. Run Milestone 1 `EXPLAIN (ANALYZE)` before and after `CREATE INDEX`.
    2. Run the `ctid` / `xmin` query around an `UPDATE`.
    3. Run `VACUUM (VERBOSE) orders` and find the removed dead row versions in the output.
  - **Done when:** you have both plans with their execution times, and can say why `ctid` changed.
- [ ] **T2 — Provoke a lock wait and a deadlock** *(≈40 min, intermediate; topic 02)*
  - **Goal:** observe row locks in `pg_stat_activity` / `pg_locks`, then break it.
  - **Steps:**
    1. In session 1: `BEGIN; UPDATE orders SET item='a' WHERE id=2;` (leave open). In session 2: `UPDATE orders SET item='b' WHERE id=2;` and watch it hang; from a third session query `pg_stat_activity` for `wait_event_type = 'Lock'`.
    2. Create a deadlock by updating rows 2 and 3 in opposite order in the two sessions.
    3. Rewrite the outbox claim with `FOR UPDATE SKIP LOCKED` (already in `relay()`) and show two relays running in parallel never block each other.

- **Done when:** you captured the waiting query, the PostgreSQL `deadlock detected` error, and two relay runs that claim disjoint rows.

- [ ] **T3 — Partition and retire data** *(≈30 min, intermediate; topic 11)*
  - **Goal:** use pruning and drop a month instantly.
  - **Steps:**
    1. Do the partition step in Milestone 1 and confirm pruning with `EXPLAIN`.
    2. Detach and drop the oldest partition and note the elapsed time versus `DELETE FROM page_views WHERE viewed_at < '2026-10-01'` on a copy.
    3. Explain which Kafka mechanism plays the same role as a shard key for `orders.created` (hint: the message key and partition count).
  - **Done when:** you have the pruned plan, both timings, and a one-sentence answer for Kafka.

- [ ] **T4 — Measure and fix replication lag** *(≈45 min, intermediate; topics 04, 13, 10)*
  - **Goal:** quantify lag and make a feature read-your-writes safe.
  - **Steps:**
    1. Run the pause, write and `lag` sequence from Milestone 2.
    2. Change the pause length to 1 s and 6 s and record the "after waiting" time each time.
    3. Add a 0.5 s timeout to the wait loop in `lag_probe` that falls back to reading the primary, and show it taking the fallback on the 6 s run.
    4. Write down which `shop-api` reads may go to the replica (product listing) and which may not (order confirmation).
  - **Done when:** you have three timings that track your pause lengths and a working fallback.

- [ ] **T5 — Duplicate delivery, zero duplicate effects** *(≈45 min, advanced; topics 08, 14, 15)*
  - **Goal:** prove the consumer is idempotent under redelivery and under a crash.
  - **Steps:**
    1. Run the Milestone 4 flow, then reset `published_at` and re-relay three times.
    2. Add `os._exit(1)` in `consume()` after `handle(...)` but before `c.commit(...)`; run it, then remove the line and run again.
    3. Compare `order_totals` with the true event counts: `SELECT (payload->>'customer_id')::int AS customer_id, count(*) FROM outbox GROUP BY 1 ORDER BY 1`.

- **Done when:** the totals equal the true order counts after all duplicates and the crash, and you can show the `duplicate-skipped` count.

- [ ] **T6 — Redis lock under pressure, then a cache failure** *(≈40 min, advanced; topics 07, 12, 15)*
  - **Goal:** connect locks, fencing and the cache's consistency limits.
  - **Steps:**
    1. Run `python3 l13.py lock` twice and explain why the fencing numbers grow.
    2. Change the lease to 5 s (`px=5000`) and remove the stall; confirm B is now refused (add a guard for `None`) and no write is rejected.
    3. Update an order in Postgres but not in Redis and read the stale cache entry; fix it with delete-on-write.
  - **Done when:** you have output for the stalled and non-stalled runs and a stale-then-fresh cache read.

- [ ] **T7 — Run the PITR drill and time it** *(≈60 min, advanced; topic 05)*
  - **Goal:** recover to a named point and to a timestamp.
  - **Steps:**
    1. Repeat Milestone 5 and time it from "oops" to a verified row count.
    2. Redo the restore into a fresh volume with `recovery_target_time = '<timestamp just before the DELETE>'` instead of the named point (record the time with `SELECT now()` before deleting).
    3. Write down your measured recovery time and the data you could lose (the RPO) if the archive had stopped five minutes earlier.
  - **Done when:** both recovery targets bring the count back and you have a measured restore time in minutes.

- [ ] **T8 — Take away the standby, then the pooler** *(≈45 min, advanced; topics 12, 06, 04)*
  - **Goal:** classify the stack and compare failure modes.
  - **Steps:**
    1. Do the synchronous commit experiment in Milestone 6 and capture `SyncRep` .
    2. Switch to `ANY 1 (*)` with a second replica (add another `pg_basebackup` standby) and show that stopping one standby no longer blocks commits.
    3. Run the PgBouncer experiment and compare backend counts with and without the pooler (`SELECT count(*) FROM pg_stat_activity`).
  - **Done when:** you have the blocked-commit evidence, the unblocked run with two standbys, and the backend counts.

- [ ] **T9 — Swap Kafka for RabbitMQ** *(≈60 min, advanced; topic 09)*
  - **Goal:** keep the outbox and the idempotent consumer, change the broker, and see the different semantics.
  - **Steps:**
    1. Start `rabbitmq:3.13-management-alpine` as in RabbitMQ §13.9; declare a durable queue `orders.created`.
    2. Change `relay()` to publish persistent messages with the `event_id` as the message ID (publisher confirms on), and `consume()` to ack only after the database commit.
    3. Kill the consumer mid-message and watch the message be redelivered; confirm `order_totals` stays correct.
  - **Done when:** the redelivered message is a duplicate at the broker but not in `order_totals`, and you can state one difference between acks and Kafka offsets.

- [ ] **T10 — Prove it: break and explain the platform** *(≈90 min, advanced; all topics)*
  - **Goal:** demonstrate mastery without the guide.
  - **Steps:**
    1. Run the teardown, rebuild the whole lab from your saved files, and explain in writing what each of the three demonstrations (lag, idempotent consumer, PITR) proves.
    2. Kill Kafka (`docker stop kafka`) while creating five orders, bring it back, and show that no order is lost and no event is duplicated in `order_totals`.
    3. Write a half-page runbook: "primary is gone, replica is behind by 30 seconds": what you check, what you promote, what data you might have lost, and why.
  - **Done when:** a teammate can follow your runbook and your outputs show all three demonstrations working from a rebuilt lab.

6 Verification checklist

- [ ] `pg_stat_replication` shows `state = streaming` and the replica answers `pg_is_in_recovery() = t`.
- [ ] A replica read after a write returned stale data while replay was paused, and the LSN-gated read returned the row.
- [ ] The `orders` query switched from a sequential scan to an index scan after `CREATE INDEX`, and a partition-pruned plan touches one partition.
- [ ] Writing an order created exactly one `orders` row and one `outbox` row, and a rolled-back attempt created neither.

- [ ] After re-publishing the same five events, `order_totals` and `processed_events` did not change (`duplicate-skipped: 5`).
- [ ] `python3 l13.py lock` printed `REJECTED (stale)` for the stale writer.
- [ ] The restored instance returned the pre-`DELETE` row count, stopped at the `before_oops` restore point, and is no longer in recovery.
- [ ] A commit hung with `wait_event = SyncRep` while the only standby was down, and recovered when it came back.
- [ ] PgBouncer served 60 clients with about 5 server connections.
- [ ] You recorded a measured replication lag, restore time and queue/consumer lag (`kafka-consumer-groups.sh --describe --group totals`).

7 Teardown

```bash
pkill pgbouncer 2>/dev/null; rm -f userlist.txt pgbouncer.ini recovery-snippet.conf
docker rm -f pg-replica pg-restored 2>/dev/null
docker compose down -v                      # stops primary, Redis, Kafka and removes
  their volumes
docker volume rm pg-replica-data pg-restored 2>/dev/null
docker network rm l13net 2>/dev/null
unset PG_PASSWORD PRIMARY_DSN REPLICA_DSN
```

Check with `docker ps -a` and `docker volume ls` that nothing named `pg-*`, `redis` or `kafka` remains.

8 Stretch goals and what to learn next

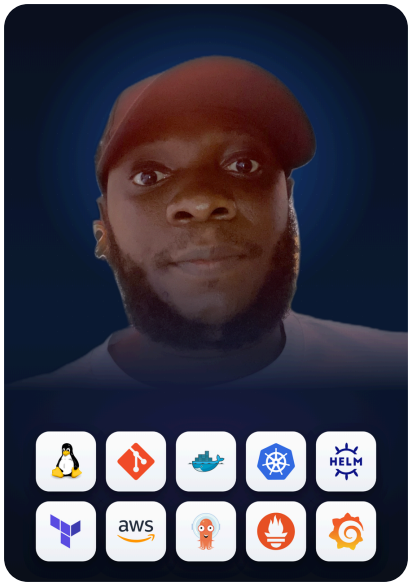
- Replace the manual pause with a failure drill: kill `pg-primary`, promote `pg-replica` (`SELECT pg_promote();`), fence the old primary and rebuild it with `pg_rewind` (see Replication §13.4).
- Add Patroni or Postgres operators for automatic failover, and observe the CAP trade-off in their health checks.
- Replace the polling relay with change data capture (Debezium's outbox event router).
- Add a Kafka consumer lag exporter and a replication-lag alert, then wire both into the dashboards from Level 10 §10.3.
- Model a CRDT counter for page views across two regions and compare it with `order_totals` (see Eventual Consistency §13.13).

- Shard `orders` by `customer_id` with Citus or application-level routing and measure the cost of cross-shard queries (Partitioning and Sharding §13.11).

Next: Level 14 — Site Reliability Engineering §14.1 turns lag, queue depth and restore time into SLIs, SLOs and error budgets, and Level 15 — Disaster Recovery §15.1 formalises the backup, failover and PITR drills you just ran into RPO/RTO targets and rehearsed procedures.

LAB NOTES What I ran, what I saw, what it means

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.

Do not aim to know 50 tools.

Aim to take a software application and independently make it production-ready.

Secure · Observable · Scalable · Highly available · Automatically deployable · Recoverable · Cost-efficient

This is the full curriculum behind the open-source DevOps Roadmap, set down in print across six volumes. Twenty-one levels climb from Linux, networking and Git through Docker, CI/CD, AWS, Terraform, Kubernetes and Helm, on to observability, security, SRE, disaster recovery, GitOps, platform engineering, architecture and cost.

Every topic explains what a thing is and why it matters in production, shows it in a real-world system, draws the mechanism, walks you through it step by step, then hands you tasks you can verify and questions a senior engineer would expect you to answer. Nine projects chain it all into one evolving production system built around a single sample application.

Volume 4: Under the Hood
Part VI: Advanced Networking, Databases and Distributed Systems



Ubong Eduok
Author and instructor of the DevOps Roadmap: Beginner to Senior.
github.com/mascot200/devops-roadmap

DEVOPS · CLOUD · PLATFORM ENGINEERING · BEGINNER TO SENIOR